

Lecture Notes in Control and Information Sciences -
Proceedings

Julian Berberich
Andrea Iannelli
Frank Allgöwer *Editors*

Systems Theory in Data and Optimization

Proceedings of SysDO 2024



Lecture Notes in Control and Information Sciences - Proceedings

Series Editors

Frank Allgöwer, Universität Stuttgart, Institute for Systems Theory and Automatic Control, Stuttgart, Germany

Manfred Morari, Department of Electrical and Systems Engineering, University of Pennsylvania, Zürich, USA

This distinguished conference proceedings series publishes the latest research developments in all areas of control and information sciences – quickly, informally and at a high level. Typically based on material presented at conferences, workshops and similar scientific meetings, volumes published in this series will constitute comprehensive state-of-the-art references on control and information science and technology studies.

Please send your proposals to Oliver Jackson, Editor, Springer. e-mail: oliver.jackson@springer.com

Julian Berberich · Andrea Iannelli · Frank Allgöwer
Editors

Systems Theory in Data and Optimization

Proceedings of SysDO 2024

Editors

Julian Berberich 
Institute for Systems Theory and Automatic
Control
University of Stuttgart
Stuttgart, Germany

Andrea Iannelli 
Institute for Systems Theory and Automatic
Control
University of Stuttgart
Stuttgart, Germany

Frank Allgöwer 
Institute for Systems Theory and Automatic
Control
University of Stuttgart
Stuttgart, Germany

ISSN 2522-5383

ISSN 2522-5391 (electronic)

Lecture Notes in Control and Information Sciences - Proceedings

ISBN 978-3-031-83190-4

ISBN 978-3-031-83191-1 (eBook)

<https://doi.org/10.1007/978-3-031-83191-1>

“MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See <https://www.mathworks.com/trademarks> for a list of additional trademarks.”

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2025, corrected publication 2025

Chapter 19 is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>). For further details see license information in the chapter.

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Organization

The Symposium on Systems Theory in Data and Optimization 2024 was organized by the Institute for Systems Theory and Automatic Control, University of Stuttgart, Germany.

General Chair

Andrea Iannelli, University of Stuttgart, Germany

General Co-Chair

Frank Allgöwer, University of Stuttgart, Germany

Program Chair

Julian Berberich, University of Stuttgart, Germany

International Program Committee

Mahathi Anand, University of Stuttgart, Germany

Nicola Bastianello, KTH Royal Institute of Technology, Sweden

Ruggero Carli, University of Padua, Italy

Angela Fontan, KTH Royal Institute of Technology, Sweden

Johannes Köhler, ETH Zürich, Switzerland

Michael Mühlebach, Max Planck Institute for Intelligent Systems, Tübingen, Germany

Anastasios Tsiamis, ETH Zürich, Switzerland

Henk van Waarde, University of Groningen, Netherlands

Sponsors

German Research Foundation (DFG), Grant AI 106/1-1—549655951

International Federation of Automatic Control (IFAC)

Stuttgart Center for Simulation Science (SimTech)

Preface

This volume contains the proceedings of the Symposium on Systems Theory in Data and Optimization (SysDO) 2024. The symposium was held from September 30 to October 2 in Stuttgart, Germany, and it was organized by the Institute for Systems Theory and Automatic Control at the University of Stuttgart, Germany, on the occasion of its 25th anniversary.

SysDO brought together researchers working in control, data, optimization, and in particular at the intersection of these topics. The symposium was an international event with participants from overall more than 15 countries. Specific topics addressed at the symposium included but were not limited to: learning-based control, theory of machine learning, optimization algorithms, game theory, data-driven control, optimization-based control, and cutting-edge applications in various domains including energy systems. The symposium featured oral and poster presentations of contributed papers and extended abstracts as well as plenary talks given by leading international researchers in the field. The single-track format and extensive poster sessions allowed for stimulating discussions and exchange of new ideas. From overall 80 submissions to the symposium (41 contributed papers, 39 extended abstracts), 21 were accepted as contributed papers published as chapters in this volume and 43 were distributed to the symposium participants in an extended abstract format.

This volume contains 21 chapters addressing diverse areas connected to control, data, and optimization. The increasing prevalence of cyber-physical systems sparks the need for new methods to integrate measured data and different forms of feedback, especially optimization-based, inside the decision-making mechanism. There are distinctive challenges that arise in this scenario, such as the existence of different time scales, the need to guarantee sufficient richness of the collected data, or the effect of suboptimal decisions under uncertainty. The chapters of this volume address various theoretical and practical research questions connected to these challenges. Aligned with the topical focus of the symposium, they are grouped into four main chapters: data-driven and learning-based control, theory and applications of machine learning, model predictive control, and optimization. We are especially excited to see that the chapters not only address all three pillar areas of the symposium (control,

data, optimization) but also cover the full range of their interface areas. We hope that these contributions will have a lasting effect on the research landscape and, thereby, help to address the variety of challenges arising in this emerging research field.

In closing, we are thankful to our sponsors: the German Research Foundation (DFG), the International Federation of Automatic Control (IFAC), and the Stuttgart Center for Simulation Science (SimTech).

Moreover, we would like to thank the chapter authors for their valuable contributions, the conference participants for the inspiring and successful symposium, the International Program Committee and the reviewers for their assistance in the reviewing process, and all members of the Institute for Systems Theory and Automatic Control for their immense efforts in putting together this exciting symposium.

Stuttgart, Germany
September 2024

Julian Berberich
Andrea Iannelli
Frank Allgöwer

Contents

Part I Data-Driven and Learning-Based Control

- 1 PACSBO: Probably Approximately Correct Safe Bayesian Optimization** 3
Abdullah Tokmak, Thomas B. Schön, and Dominik Baumann
- 2 Value of Communication: Data-Driven Topology Optimization for Distributed Linear Cyber-Physical Systems** 19
Michael Nestor and Fei Teng
- 3 Variance-Informed Model Reference Gaussian Process Regression: Utilizing Variance Information for Control in Nonlinear Systems** 37
Hyuntae Kim
- 4 Data-Driven Dynamic Model and Model Reference Control of Inverter-Based Resources** 53
Ganesh Marasini, Kenneth McDonald, Zhihua Qu, and Thomas E. McDermott
- 5 Adaptive Tracking MPC for Nonlinear Systems via Online Linear System Identification** 69
Tatiana Strelnikova, Johannes Köhler, and Julian Berberich

Part II Machine Learning: Theory and Applications

- 6 Investigation of the Influence of Training Data and Methods on the Control Performance of MPC Utilizing Gaussian Processes** 87
Florian Diepers, Elmar Ahle, and Dirk Söffker
- 7 Wiener Chaos in Kernel Regression: Toward Untangling Aleatoric and Epistemic Uncertainty** 105
Timm Faulwasser and Oleksii Molodchyk

8	A Universal Reproducing Kernel Hilbert Space for Learning Nonlinear System Operators	121
	Mircea Lazar	
9	On Robust Reinforcement Learning with Lipschitz-Bounded Policy Networks	137
	Nicholas H. Barbara, Ruigang Wang, and Ian R. Manchester	
10	Solving Partial Differential Equations with Equivariant Extreme Learning Machines	153
	Hans Harder, Jean Rabault, Ricardo Vinuesa, Mikael Mortensen, and Sebastian Peitz	
11	Adaptive Robust L2 Loss Function Using Fractional Calculus	169
	Mert Can Kurucu, Müjde Güzelkaya, Ibrahim Eksin, and Tufan Kumbasar	
12	Sparse Reconstruction of Forces, Torques, and Velocity Signals for a Swimmer in a Wake	181
	Ivan Gushkov, Simon Hoff, Amer Orucevic, Kristin Y. Pettersen , and Jan Tommy Gravdahl	
13	Control Theoretic Approach to Fine-Tuning and Transfer Learning	197
	Erkan Bayram, Shenyu Liu, Mohamed-Ali Belabbas, and Tamer Başar	

Part III Model Predictive Control

14	Accelerating Multi-objective Model Predictive Control Using High-Order Sensitivity Information	215
	Markus Herrmann-Wicklmayr and Kathrin Flaßkamp	
15	On Discount Functions for Economic Model Predictive Control Without Terminal Conditions	231
	Lukas Schwenkel, Daniel Briem, Matthias A. Müller, and Frank Allgöwer	
16	Multi-Parametric Programming with Constraint Relaxation for the Optimal Operation of Micro-grids Integrating Renewables	247
	Federico Bianchi, Alessandro Falsone, and Maria Prandini	
17	Multi-Objective Learning Model Predictive Control	261
	Siddharth H. Nair, Charlott Vallon, and Francesco Borrelli	
18	Terminal Set of Nonlinear Model Predictive Control with Koopman Operators	277
	Yajing Zhang, Yangyang Feng, Shuyou Yu, and Hong Chen	

Part IV Optimization

19 Optimal Dynamic Pricing in Energy Markets: A Stackelberg Game Approach	295
Panagiotis D. Grontas, Marta Fochesato, Carlo Cenedese, Giuseppe Belgioioso, Florian Dörfler, and John Lygeros	
20 Distributed Newton Optimization with ADMM-Based Consensus	313
Guido Carnevale, Nicola Bastianello, Giuseppe Notarstefano, and Ruggero Carli	
21 Inexactness in Bilevel Nonlinear Optimization: A Gradient-Free Newton's Method Approach	331
Torbjørn Cunis and Ilya Kolmanovsky	
Correction to: Investigation of the Influence of Training Data and Methods on the Control Performance of MPC Utilizing Gaussian Processes	C1
Florian Diepers, Elmar Ahle, and Dirk Söffker	
Author Index	349

Part I
Data-Driven and Learning-Based Control

Chapter 1

PACSBO: Probably Approximately Correct Safe Bayesian Optimization



Abdullah Tokmak , Thomas B. Schön , and Dominik Baumann 

Abstract Safe Bayesian optimization (BO) algorithms promise to find optimal control policies without knowing the system dynamics while at the same time guaranteeing safety with high probability. In exchange for those guarantees, popular algorithms require a smoothness assumption: a known upper bound on a norm in a reproducing kernel Hilbert space (RKHS). The RKHS is a potentially infinite-dimensional space, and it is unclear how to—in practice—obtain an upper bound on the norm of an unknown function in its corresponding RKHS. In response, we propose an algorithm that estimates an upper bound on the RKHS norm of an unknown function from data and investigate its theoretical properties. Moreover, akin to Lipschitz-based methods, we treat the RKHS norm as a local rather than a global object and, thus, allow for more optimistic exploration without compromising safety. Integrating the RKHS norm estimation and the local interpretation of the RKHS norm into a safe BO algorithm yields PACSBO, an algorithm for probably approximately correct safe Bayesian optimization. We demonstrate the effectiveness of PACSBO and its benefits over popular safe BO algorithms in numerical and hardware experiments.

Keywords Safe Bayesian optimization · Reproducing kernel Hilbert spaces · Probably approximately correct learning · Robotics

A. Tokmak (✉) · D. Baumann
Cyber-Physical Systems Group, Aalto University, Espoo, Finland
e-mail: abdullah.tokmak@aalto.fi

D. Baumann
e-mail: dominik.baumann@aalto.fi

T. B. Schön · D. Baumann
Department of Information Technology, Uppsala University, Uppsala, Sweden
e-mail: thomas.schon@it.uu.se

1.1 Introduction

In recent years, reinforcement learning (RL) has achieved remarkable success in controlling high-dimensional systems without requiring a dynamics model [1, 2]. However, most of the impressive results have been obtained in simulation environments. Applying RL algorithms to real-world robotic systems is challenging because (i) when interacting with real-world environments, it is crucial to guarantee safety, and (ii) each sample corresponds to a potentially time-consuming experiment, and hence sample efficiency is essential. Popular RL algorithms fail to provide both safety and sample efficiency.

Combining Gaussian process (GP) regression [3] with Bayesian optimization (BO) [4] provides a sample efficient alternative to RL and has been successfully applied on various hardware platforms [5–7]. Based on GP regression and BO, several algorithms have been proposed that can, in addition, provide probabilistic safety guarantees [8, 9]. These safe learning algorithms aim at optimizing an unknown reward function while satisfying (unknown) constraints with high probability. In exchange for the safety guarantees, they require smoothness assumptions for reward and constraint functions. Particularly, they assume that reward and constraint functions have a known upper bound on the norm in a reproducing kernel Hilbert space (RKHS). The RKHS norm is a norm in a potentially infinite-dimensional space, and assuming exact knowledge of that norm (or a tight upper bound on it) in unknown environments is highly unrealistic. Notably, a loose upper bound on the RKHS norm leads to conservative exploration, whereas an underestimation might lead to constraint violations. We address this shortcoming by estimating the RKHS norm from data and incorporating the estimate into a safe BO algorithm.

Popular BO algorithms treat the RKHS norm as a global object [8–10]. Since functions may exhibit vastly different smoothness properties on different parts of their parameter space, Lipschitz-based methods regularly consider a local Lipschitz constant to reduce conservatism [11]. Inspired by Lipschitz-type approaches, we introduce a local interpretation of the RKHS norm.

Related Work. Learning control policies while providing safety guarantees has attracted growing interest over the past years; see Brunke et al. [12] for a review. In the area of GP regression and BO, SAFEOPT [8, 9], which is based on GP-UCB-type algorithms [10, 13], is arguably the most well-known and popular safe learning algorithm. Various algorithms based on SAFEOPT have been proposed, targeting, e.g., better scalability [14, 15] or global optimization [16]. However, all aforementioned works require a known, tight upper bound on the RKHS norm, and it is unclear how to compute a reliable RKHS norm over-estimation of an unknown function. The recent publication [17] acknowledges this downside of popular BO algorithms and proposes an alternative approach requiring instead the Lipschitz constant and an upper bound on the noise amplitude, both of which are usually unknown. Especially estimating the Lipschitz constant of unknown functions is a delicate problem in itself [18].

An alternative to assuming a known RKHS norm upper bound is estimating it from data, which has not been extensively investigated. References [19, 20] discuss ways to under-estimate the unknown RKHS norm. Nevertheless, relying on an underestimation of the RKHS norm implies overly optimistic exploration, which can lead to constraint violations. Also, Tokmak et al. [21] introduce a simple heuristic RKHS norm extrapolation to estimate an upper bound on the RKHS norm, however, without providing any guarantees. We draw inspiration from the ideas in Hashimoto et al. [20], Scharnhorst et al. [19], Tokmak et al. [21] and propose an algorithm that estimates an upper bound on the RKHS norm from data, eliminating the need for correctly guessing the RKHS norm. Moreover, we investigate the theoretical properties of that algorithm.

Contribution. In this paper, we propose PACSBO, a novel algorithm for probably approximately correct safe BO. We make the following contributions:

1. We propose an algorithm that estimates the RKHS norm of an unknown function with probabilistic guarantees. Hence, we eliminate the need to guess the RKHS norms of the reward and constraint functions correctly.
2. We treat the RKHS norm as a local object. As a result, we can explore less conservatively.
3. We extend SAFEBO with the RKHS norm over-estimation and the local interpretation of the RKHS norm, yielding PACSBO.
4. We show the benefits of PACSBO compared to SAFEBO in numerical experiments. Furthermore, we demonstrate the successful deployment of PACSBO on a real Furuta pendulum.

1.2 Problem Setting and Preliminaries

We propose a safe BO algorithm that does not require a priori knowledge of the RKHS norm. In the following, we make the problem setting precise and introduce preliminaries, in particular, SAFEBO.

Problem Setting. We formulate the learning problem as optimizing an unknown reward function $f: \mathcal{A} \rightarrow \mathbb{R}$ that maps from the parameter space $\mathcal{A} \subseteq \mathbb{R}^n$ with parameters $a \in \mathcal{A}$ to a scalar reward value, while guaranteeing safety. We express safety through $s \in \mathbb{N}$ unknown constraint functions $g_i: \mathcal{A} \rightarrow \mathbb{R}, i \in \mathcal{I}_g = \{1, \dots, s\}$. Akin to Berkenkamp et al. [9], we introduce the surrogate function $h: \mathcal{A} \times \mathbb{N} \rightarrow \mathbb{R}$ with $h(a, i) = f(a)$ if $i = 0$, and $h(a, i) = g_i(a)$ if $i \in \mathcal{I}_g$, and define $\mathcal{I} := \{0\} \cup \mathcal{I}_g$. Hence, we can write the optimization problem as

$$\max_{a \in \mathcal{A}} h(a, 0) \quad \text{subject to} \quad h(a, i) \geq 0, \quad \forall i \in \mathcal{I}_g. \quad (1.1)$$

In practice, the reward function $h(\cdot, 0)$ may correspond to the control performance, the parameters a to the parameters of the control policy, and the constraints $h(\cdot, i)$, $i \in \mathcal{I}_g$, to the distance to obstacles.

SafeOpt. As h is unknown, solving (1.1) is infeasible without further assumptions. Thus, we make three assumptions analogous to SAFEOPt. (i), we assume that we can conduct experiments with parameters a' and, in return, receive measurements $\hat{h}(a', i) = h(a', i) + \epsilon$, where ϵ is σ -sub-Gaussian measurement noise. We define the set of parameters that we have already tried as $A := [a_1, \dots, a_N]^\top$, and the corresponding measurements as $\hat{h}_{A,i} := [\hat{h}(a_1, i), \dots, \hat{h}(a_N, i)]^\top$ for all $i \in \mathcal{I}$. (ii), we assume that a set S_0 with $\emptyset \neq S_0 \subseteq \mathcal{A}$ of initial safe parameters is given, and (iii) that $h(\cdot, i)$ is a member of the RKHS of kernel k_i , i.e., $h(\cdot, i) \in H_{k_i}$ for all $i \in \mathcal{I}$. For ease of exposition, we set $k_i = k_j$ for all $(i, j) \in \mathcal{I} \times \mathcal{I}$ and, hence, drop the index on the kernel k . The three assumptions allow us to provide safety guarantees when estimating h with GPs if, in addition, an upper bound on the RKHS norm is available [9, Theorem 4.1].

The GP posterior mean and covariance are

$$\begin{aligned}\mu_A(a, i) &= k_A(a)^\top (K_A + \sigma^2 I_N)^{-1} \hat{h}_{A,i} \\ \sigma_A^2(a) &= k(a, a) - k_A(a)^\top (K_A + \sigma^2 I_N)^{-1} k_A(a),\end{aligned}\tag{1.2}$$

respectively [3]. We denote the kernel evaluated at $a \in \mathcal{A}$ by $k(a, a)$, the covariance vector by $k_A(a) = [k(a, a_1), \dots, k(a, a_N)]^\top \in \mathbb{R}^N$, the covariance matrix with entry $k(a_i, a_j)$ at row i and column j for all $i, j \in \{1, \dots, N\}$ by $K_A \in \mathbb{R}^{N \times N}$, and the identity matrix by $I_N \in \mathbb{R}^{N \times N}$. We can now obtain confidence intervals around the posterior mean that contain the ground truth h with high probability. Specifically, when combining [22, Theorem 3.11] with [22, Remark 3.13], we have, for any confidence parameter $\delta \in (0, 1)$ and any fixed $i \in \mathcal{I}$,

$$|h(a, i) - \mu_A(a, i)| \leq \left(\|h(\cdot, i)\|_k + \sqrt{\sigma \log \det(I_N + K_A/\sigma)} - 2\sigma \log \delta \right) \sigma_A(a),\tag{1.3}$$

with probability $1 - \delta$. In (1.3), $\|h(\cdot, i)\|_k$ is the ground truth RKHS norm, which we explain in the subsequent paragraph. Using the confidence intervals (1.3), we can probabilistically quantify whether a policy parameter $a \in \mathcal{A}$ is safe. Particularly, we restrict function evaluations to a safe set $S_A \subseteq \mathcal{A}$ that only contains parameters a that are safe with high probability, i.e., parameters that correspond to constraint values greater than zero. We start with $S_A = S_0$ and sequentially augment the GP model by conducting experiments, leading to a monotonically growing safe set S_A . However, not all $a \in S_A$ are evaluated. In particular, we compute a set of potential maximizers $M_A \subseteq S_A$ and a set of potential expanders $G_A \subseteq S_A$ and choose to sample the most uncertain $a \in M_A \cup G_A$ at each iteration, thus, balancing exploration and exploitation while guaranteeing safety. The acquisition function and the definitions of the sets S_A , M_A , and G_A are equivalent to SAFEOPt. Therefore, we refer to the

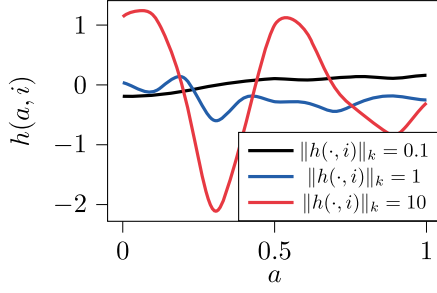


Fig. 1.1 Different RKHS norms

computation of these sets as the SAFEOPT-subroutine and point to [8] for a detailed introduction.

RKHS Norm. The confidence intervals (1.3) and the resulting safety and exploration guarantees of SAFEOPT [9, Theorem 4.1] depend on the unknown RKHS norm $\|h(\cdot, i)\|_k$. The RKHS norm of the function $h(\cdot, i) \in H_k$ is given by $\|h(\cdot, i)\|_k^2 = \sum_{s=1}^{\infty} \sum_{t=1}^{\infty} \alpha_s \alpha_t k(x_s, x_t)$ with α the coefficients and x the center points, which can both be infinite dimensional [23, Section 4]. Figure 1.1 illustrates the “smoothness” of functions given their RKHS norm. As mentioned in Sect. 1.1, we require an upper bound $B_i \geq \|h(\cdot, i)\|_k$; however, it is unclear how to get a *tight* upper bound on the RKHS norm of unknown functions. Clearly, if we assume a higher RKHS norm bound, we have larger confidence intervals (1.3), which may cause overly conservative exploration, while assuming a lower RKHS norm bound may result in constraint violations. In this work, we drop the assumption of a priori knowing a tight upper bound B_i . Instead, we propose an algorithm that estimates the RKHS norm from data while assuming $h(\cdot, i) \in H_k$, which is significantly less restrictive. We investigate the theoretical properties of this algorithm and improve exploration by treating the RKHS norm as a *local* object to, e.g., exploit locally smaller slopes.

1.3 Proposed Algorithm

In this section, we develop PACSBO, a novel algorithm for probably approximately correct safe BO. This includes the estimation of an upper bound for the RKHS norm of an unknown function, the treatment of the RKHS norm as a local object, and a theoretical analysis.

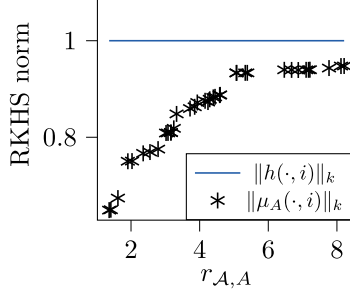


Fig. 1.2 RKHS norm over $r_{\mathcal{A},A}$

1.3.1 RKHS Norm Estimation

As introduced in Sect. 1.2, given samples $A \subseteq \mathcal{A}$, we estimate $h(\cdot, i)$ using GPs with mean $\mu_A(\cdot, i)$ and covariance $\sigma_A(\cdot)^2$. For an increasing set A sampled unbiasedly over $\mathcal{A}$, the mean $\mu_A(\cdot, i)$ serves as a close representation of the ground truth $h(\cdot, i)$ on $\mathcal{A}$. Thus, the RKHS norm $\|\mu_A(\cdot, i)\|_k$ is a reasonable estimate of $\|h(\cdot, i)\|_k$ given a sufficiently high sampling density. To quantify the sampling density, we introduce the reciprocal covariance integral $r_{\mathcal{A},A} := (\int_{a \in \mathcal{A}} \sigma_A(a)^2 da)^{-1}$. Intuitively, as we collect more samples over $\mathcal{A}$, the covariance shrinks and, hence, its reciprocal integral increases. Thus, for an increasing $r_{\mathcal{A},A}$, we expect $\|\mu_A(\cdot, i)\|_k$ to approach $\|h(\cdot, i)\|_k$. This expectation is empirically confirmed by Fig. 1.2. As $r_{\mathcal{A},A}$ increases, we see that $\|\mu_A(\cdot, i)\|_k$ approaches $\|h(\cdot, i)\|_k$ from below, as observed in Hashimoto et al. [19], Scharnhorst et al. [20]. Thus, directly using $\|\mu_A(\cdot, i)\|_k$ as an estimate of the RKHS norm might lead to safety constraint violations as we would be too optimistic.

To get a more reliable estimate of $\|h(\cdot, i)\|_k$, we construct a predictor η , which is described in Algorithm 1.1. In particular, we train a multi-layer perceptron (MLP) that extrapolates the RKHS norm $\|h(\cdot, i)\|_k$ based on the RKHS norm of the mean $\|\mu_A(\cdot, i)\|_k$ and the reciprocal covariance integral $r_{\mathcal{A},A}$. Clearly, we require data to train the MLP. Therefore, we let SAFEOPT optimize random, artificial RKHS functions $\rho_j(\cdot, i)$, $j = 1, \dots, m$, with known RKHS norms $\|\rho_j(\cdot, i)\|_k$. In each iteration of SAFEOPT, we compute $\|\mu_A(\cdot, i)\|_k$ and $r_{\mathcal{A},A}$ and store them as sequences X_j . Those sequences serve as the inputs for training the MLP, whereas the true RKHS norms of the random functions, $\|\rho_j(\cdot, i)\|_k$, are the labels.¹ As the sequences X_j of collected $\|\mu_A(\cdot, i)\|_k$ and $r_{\mathcal{A},A}$ naturally have different lengths, and MLPs work with fixed input sizes, we use zero-padding at the beginning of the inputs. For evaluating the MLP, we compute the sequence X containing $\|\mu_A(\cdot, i)\|_k$ and $r_{\mathcal{A},A}$ up until the current parameter set A and receive $B_i = \eta(X)$. After training, the returned value B_i should be a reasonable estimate of $\|h(\cdot, i)\|_k$. However, since we require a *guaranteed over-estimation* of $\|h(\cdot, i)\|_k$, we use the prediction of the MLP merely as an initial guess.

¹ One can, e.g., also use $2 \cdot \|\rho_j(\cdot, i)\|_k$ as the label to introduce some conservatism.

To determine a guaranteed upper bound on the RKHS norm, we create q independent and identically distributed (i.i.d.) random RKHS functions given samples A , $\rho_{A,j}(\cdot, i) \in H_k$, $j \in \{1, \dots, q\}$, with known RKHS norms $\|\rho_{A,j}(\cdot, i)\|_k$, similar to the random RKHS functions used for training the MLP. These random RKHS functions are designed to interpolate the given samples subject to σ -sub-Gaussian noise, and their remaining coefficients are sampled from a bounded interval. Using the RKHS norms of the random RKHS functions and the value provided by the MLP, we compute the RKHS norm over-estimation, for which we present the theoretical guarantees in Sect. 1.3.4.

Algorithm 1.1 Training of the MLP

Require: RKHS functions $\rho_j(\cdot, i)$, parameter space $\mathcal{A}$, kernel k , MLP architecture

- 1: Init: $(\bar{X}, \bar{Y}) = \emptyset$
 - 2: **for** each $\rho_j(\cdot, i)$ **do**
 - 3: Randomly initialize set of safe samples S_0 and initialize $X_j = \emptyset$
 - 4: **for** each SAFEOPT step **do**
 - 5: Compute $\|\mu_A(\cdot, i)\|_k$ and $r_{\mathcal{A},A}$ given samples A and domain $\mathcal{A}$
 - 6: $X_j \leftarrow X_j \cup \{\|\mu_A(\cdot, i)\|_k, r_{\mathcal{A},A}\}$
 - 7: $\bar{X} \leftarrow \bar{X} \cup X_j$, $\bar{Y} \leftarrow \bar{Y} \cup \{\|\rho_j(\cdot, i)\|_k\}$ ▷ Collect training data for all $\rho_j(\cdot, i)$
 - 8: Train MLP with given architecture, inputs $\bar{X}$, and labels $\bar{Y}$
-

1.3.2 Local Interpretation of the RKHS Norm

We have seen that the RKHS norm of the mean function $\|\mu_A(\cdot, i)\|_k$ is a reasonable estimate of the unknown RKHS function $\|h(\cdot, i)\|_k$ for a high sampling density $r_{\mathcal{A},A}$. However, to guarantee safety, we restrict ourselves to a subspace $S_A \subseteq \mathcal{A}$ and consciously leave potentially large parts of the parameter space $\mathcal{A} \setminus S_A$ unexplored. Thus, we can expect a small sampling density $r_{\mathcal{A},A}$ with respect to the complete parameter space $\mathcal{A}$ even after many iterations. At the same time, since we only explore a subspace, considering the RKHS norm on its complement is superfluous. Furthermore, it is especially in unsafe regions $\mathcal{A} \setminus S_A$ where we might experience less smooth behavior, i.e., large RKHS norms, e.g., if the dynamics in those regions are unstable. Finally, we can exploit local smoothness for more aggressive exploration due to tighter confidence intervals using local RKHS norms. These arguments motivate a local interpretation of the RKHS norm on sub-domains instead of one RKHS norm for the complete parameter space $\mathcal{A}$.

Akin to SAFEOPT and other safe BO algorithms, we start exploration around the set of initial safe samples S_0 . We start by sampling each $a \in S_0$, which initially yields $A \equiv S_0$. Hence, a reasonable interpretation of locality is a neighborhood

around the samples A . Therefore, we introduce $\tilde{\mathcal{A}}$ —the convex hull of samples A —which we expect to be a sub-domain with relatively high sampling density. Corresponding to the convex hull $\tilde{\mathcal{A}}$, we define the GPs and the RKHS norm upper bounds $\tilde{\mathcal{GP}}_i$ and $\tilde{B}_i$, respectively. To allow for exploration outside of the convex hull $\tilde{\mathcal{A}}$ and to progressively enlarge the safe set while still ensuring a high sampling density, we define a second sub-domain $\hat{\mathcal{A}}$ with $\hat{B}_i$ and $\hat{\mathcal{GP}}_i$ such that $\tilde{\mathcal{A}} \subseteq \hat{\mathcal{A}} \subseteq \mathcal{A}$. Note that we additionally preserve the global RKHS norm estimates B_i and GPs $\mathcal{GP}_i$, which yields three total partitions.

Remark 1.1 Using three partitions, namely, $\tilde{\mathcal{A}}$, $\hat{\mathcal{A}}$, and $\mathcal{A}$, is a very simple heuristic, and extending this to more local objects or even an adaptive partitioning of the parameter space is a promising area for future research. Moreover, the definition of the intermediate domain $\hat{\mathcal{A}}$ is an additional hyperparameter whose choice influences the performance of the proposed algorithm. Empirically, we noticed that defining $\hat{\mathcal{A}}$ to be close to the convex hull $\tilde{\mathcal{A}}$ is a promising choice. We want to highlight that locality does not impact safety and mainly improves exploration.

1.3.3 PACSBO

We now combine the RKHS norm over-estimation and the local interpretation of the RKHS norm into PACSBO, summarized in Algorithm 1.2. We start by computing the sub-domains $\tilde{\mathcal{A}}$ and $\hat{\mathcal{A}}$ (ℓ . 3) and defining the GPs corresponding to each of the three parameter spaces (ℓ . 5). Next, we query the MLP (ℓ . 6–8) to receive an initial guess on the RKHS norm upper bound B_i . Then, we compute the empirical mean of the known RKHS norms of the random RKHS functions to which we add a safety term w_i ; the precise form of w_i becomes clear in Theorem 1.1. In this way, we obtain a representative value using the RKHS norms of the random RKHS functions and take the maximum between that value and the return of the MLP as the guaranteed RKHS norm over-estimation (ℓ . 10). Next, we execute the SAFEOPt-subroutine and compute the sets of potential maximizers and potential expanders (ℓ . 11). Finally, we choose the policy parameter to be tried next as the most uncertain policy parameter of the three candidates returned by each SAFEOPt-subroutine (ℓ . 12) before conducting a new experiment (ℓ . 13), and updating the sample sets (ℓ . 14). We continue with the described procedure for $\bar{N}$ iterations and return our estimate of the optimal safe policy parameter (ℓ . 15).

The code of PACSBO is available at <https://github.com/tokmak1/PACSBO>.

1.3.4 Theoretical Analysis

In this section, we show that B_i computed in Algorithm 1.2 (ℓ . 10) is indeed an over-estimation of the unknown RKHS norm $\|h(\cdot, i)\|_k$. Specifically, we prove that

Algorithm 1.2 PACSBO

Require: $k, \mathcal{A}, S_0, \eta, \delta, \gamma, q, \bar{N}, \mathcal{I}$

- 1: Init: $A \leftarrow S_0, \hat{h}_{A,i} \leftarrow \hat{h}(S_0, i) \forall i \in \mathcal{I}$
- 2: **for** $1 : \bar{N}$ **do**
- 3: Determine $\tilde{\mathcal{A}}, \hat{\mathcal{A}}$ given $\mathcal{A}$ and A ▷ Compute sub-domains
- 4: **for** $\mathbb{A} \in \{\tilde{\mathcal{A}}, \hat{\mathcal{A}}, \mathcal{A}\}$ **do**
- 5: $\mathcal{GP}_i \leftarrow$ Define GPs on $\mathbb{A}$ with samples A
- 6: Compute $\|\mu_A(\cdot, i)\|_k$ and $r_{\mathbb{A},A}$ given $\mathbb{A}, A, \hat{h}_{A,i}, k$, and $\mathcal{GP}_i$
- 7: $X \leftarrow X \cup \{\|\mu_A(\cdot, i)\|_k, r_{\mathbb{A},A}\}$
- 8: $B_i \leftarrow \eta(X)$ ▷ MLP prediction
- 9: Create q random RKHS functions $\rho_{A,j}(\cdot, i)$ and $\|\rho_{A,j}(\cdot, i)\|_k$ ▷ Section 1.4
- 10: $B_i \leftarrow \max\{B_i, \frac{1}{q} \sum_{j=1}^q \|\rho_{A,j}(\cdot, i)\|_k + w_i\}$ ▷ Theorem 1.1
- 11: $M_A \cup G_A \leftarrow \text{SAFEOPT-subroutine with } (B_i, \mathcal{GP}_i)$
- 12: $a \leftarrow$ Most uncertain parameter $a \in (\tilde{M}_A \cup \tilde{G}_A) \cup (\hat{M}_A \cup \hat{G}_A) \cup (M_A \cup G_A)$
- 13: $\hat{h}(a, i) \leftarrow$ Conduct experiment with policy parameter a
- 14: Append a to $A, \hat{h}(a, i)$ to $\hat{h}_{A,i}$ ▷ Update sample sets
- 15: **return** Best safely evaluable policy parameter

the RKHS norm over-estimation using random RKHS functions is probably approximately correct (PAC) [24]. Although we present this section considering the RKHS norm on the parameter space $\mathcal{A}$, adjustments to the RKHS norm of the function restricted to the sub-domains $\tilde{\mathcal{A}}$ or $\hat{\mathcal{A}}$ work analogously.

First, we require an assumption that asymptotically connects the RKHS norms of the random RKHS functions and the ground truth RKHS norm.

Assumption 1.1 Consider any $i \in \mathcal{I}$ and any $A \subseteq \mathcal{A}$. Given samples A , we have $\|h(\cdot, i)\|_k \leq \lim_{s \rightarrow \infty} \frac{1}{s} \sum_{j=1}^s \|\rho_{A,j}(\cdot, i)\|_k$.

Assumption 1.1 states that the expected value of the RKHS norm of the random functions overestimates the RKHS norm of $h(\cdot, i)$. We discuss Assumption 1.1 in Sect. 1.4 as it also depends on the random RKHS function generation. Finally, we present our main theoretical contribution.

Theorem 1.1 Consider any $i \in \mathcal{I}$ and let Assumption 1.1 hold. If

$$B_i \geq \frac{1}{q} \sum_{j=1}^q \|\rho_{A,j}\|_k + w_i, w_i := \sqrt{\frac{\ln(\frac{2}{\gamma})(\max_{\alpha,x} \|\rho_A(\cdot, i)\|_k - \min_{\alpha,x} \|\rho_A(\cdot, i)\|_k)^2}{2q}}, \quad (1.4)$$

then $B_i \geq \|h(\cdot, i)\|_k$ with a probability of at least $1 - \gamma$, where $\gamma \in (0, 1)$.

Proof (Idea) We use Hoeffding’s inequality [24, Lemma 4.5] to upper bound the deviation between the empirical mean computed by q random RKHS functions and the true mean computed with infinitely many random RKHS functions. Then, we leverage Assumption 1.1 to connect the true mean and the ground truth RKHS norm $\|h(\cdot, i)\|_k$. The details can be found in Sect. 1.7.

1.4 Implementation

In this section, we discuss the practical implementation of PACSBO, particularly the random RKHS function generation, and comment on Assumption 1.1.

Random RKHS Function Generation. We need random RKHS functions to obtain training data for the MLP in Algorithm 1.1 and for the RKHS norm over-estimation in Algorithm 1.2. Constructing random RKHS functions considering the entire RKHS H_k would require us to compute infinite sums, which is impossible. Hence, we compute functions $\rho_{A,j} = \sum_{s=1}^{\hat{N}} \alpha_s k(x_s, \cdot)$ with finite $\hat{N} \gg N$, essentially restricting $\rho_{A,j}(\cdot, i)$ to a pre-RKHS $H_{0,k} \subseteq H_k$ [25, Sect. 2.3], similar to the pre-RKHS approach presented in Fiedler et al. [26, Appendix C.1] with random α and x . To ensure the interpolating property of the random RKHS functions for the PAC validation, the coefficients $\alpha_1, \dots, \alpha_N$ are determined by the samples A and $\hat{h}_{A,i}$. The remaining coefficients are sampled uniformly within the interval $[-\bar{\alpha}, \bar{\alpha}]$. Moreover, we enforce that the first N center points of $\rho_{A,j}(\cdot, i)$ in Algorithm 1.2 are equal to A and sample the remaining center points uniformly from $\mathcal{A}$.

Assumption 1.1, Tightness, and Randomness. To account for Assumption 1.1, i.e., to ensure that the RKHS norm of the random RKHS functions over-estimate the unknown RKHS norm in expectation, we choose $\bar{\alpha}$ and $\hat{N}$ in a way that $\|\rho_{A,j}(\cdot, i)\|_k$ can get large for small sample sets A , which is illustrated in Fig. 1.3. Figure 1.3 also shows that $\|\rho_{A,j}(\cdot, i)\|_k$ approaches $\|h(\cdot, i)\|_k$ for an increasing sample set A . Hence, we expect a tighter over-estimate $B_i \geq \|h(\cdot, i)\|_k$ for more samples A , i.e., a smaller right-hand side of (1.4) in Theorem 1.1. In particular, for the three toy examples in Fig. 1.3 with $\|h(\cdot, i)\|_k = 1$ and $q = 5 \cdot 10^3$, the right-hand side of (1.4) yields 5.71 (left, 5 samples), 2.96 (center, 20 samples), and 1.71 (right, 50 samples), respectively, which supports that we can obtain tighter RKHS norm over-estimations with denser sampling.

Remark 1.2 Our design choices reduce the randomness of the random RKHS functions. Furthermore, as mentioned in Fiedler et al. [26, Appendix C.1], generating RKHS functions by following the pre-RKHS approach is prone to introducing a slight bias. Nevertheless, our numerical investigations display adequate randomness and capture a wide range of RKHS functions.

Remark 1.3 It is possible to design RKHS functions $h(\cdot, i)$ for which Assumption 1.1 is violated and, hence, Theorem 1.1 becomes inapplicable. This can especially happen with a very large ground truth RKHS norm combined with a small number of samples and a too small $\bar{\alpha}$. The former may cause a relatively small mean RKHS norm $\|\mu_A(\cdot, i)\|_k$, which may result in a too small prediction of the MLP. We reduce this risk by following a localized approach and increasing sampling density. Furthermore, we choose a large enough bound on the coefficients $\bar{\alpha}$ to capture large ground truth RKHS norms. Finally, the toy example in Fig. 1.3 and the forthcoming experiments in Sect. 1.5 support the sensibility and applicability of our approach.

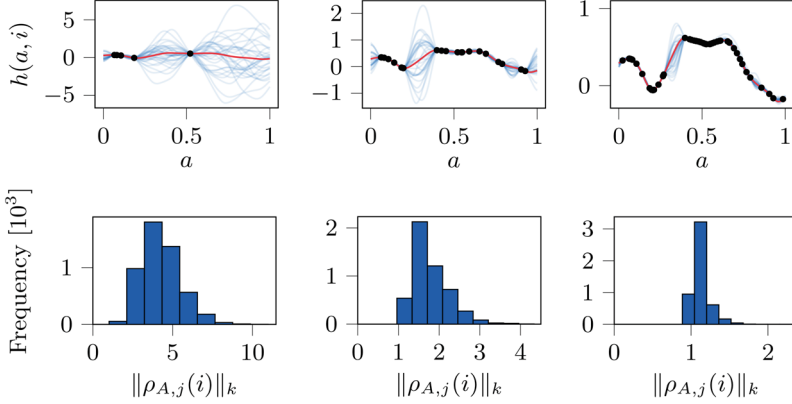


Fig. 1.3 Random RKHS functions and their RKHS norms. The upper sub-figures show the ground truth $h(\cdot, i)$ (red line) and the random RKHS functions $\rho_{A,j}(\cdot, i)$ (blue lines) given samples A (black dots), while the lower sub-figures illustrate the frequency of the RKHS norms $\|\rho_{A,j}(\cdot, i)\|_k$. For an increasing sample set, $\|\rho_{A,j}(\cdot, i)\|_k$ approaches $\|h(\cdot, i)\|_k = 1$, whereas $\|\rho_{A,j}(\cdot, i)\|_k$ tends to conservatively over-estimate $\|h(\cdot, i)\|_k$ for fewer samples. We used the Matérn-32 kernel is miswritten with length scale $\ell = 0.1$, $\hat{N} = 100$, and $\bar{\alpha} = 1$. We sampled the parameters A randomly from a uniform distribution. The upper sub-figures show a random subset of 30 out of the $q = 5 \cdot 10^3$ random RKHS functions

Remark 1.4 Notably, even if Assumption 1.1 is violated and the right-hand side of (1.4) of Theorem 1.1 is too small, we still leverage collected samples and a sophisticated predictor to estimate the RKHS norm (Algorithm 1.2, $\ell = 10$). This is the reason why we take the MLP prediction into account even though it does not appear in Theorem 1.1. Hence, an accurate initial prediction of the MLP can still yield a reasonable estimate of the RKHS norm; however, the obtained PAC bounds would become mundane. In contrast, SAFEOPt guesses the RKHS norm bound a priori and maintains that guess. Particularly, numerous SAFEOPt-applications, e.g., [9], use a constant multiple of the posterior covariance as confidence intervals when conducting experiments.

1.5 Results

In this section, we present numerical experiments and a hardware experiment using PACSBO. All experiments used the Matérn-32 kernel with $\ell = 0.1$, $q = 10^3$, and confidence parameters $\delta = \gamma = 0.1$. Moreover, we choose the sub-domain $\hat{\mathcal{A}}$ to be the 10 % uniform enlargement of the convex hull $\tilde{\mathcal{A}}$, and set $\mathcal{A} = [0, 1]^n$.

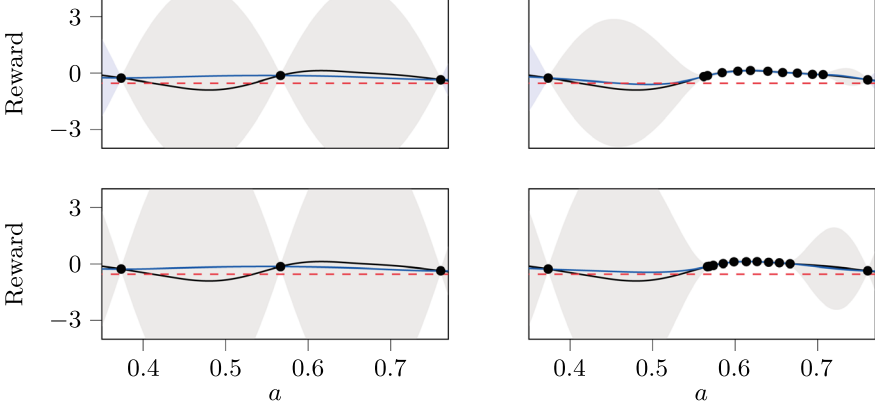


Fig. 1.4 A conservative smoothness assumption allows PACSBO (upper figures) to explore faster than SAFEOPT (lower figures)

1.5.1 Numerical Experiments

Setting. We compare PACSBO and SAFEOPT by maximizing an unknown RKHS function $h(\cdot, 0)$ with $\|h(\cdot, 0)\|_k = 2$. We have $\mathcal{I}_g = \{1\}$ and set $h(\cdot, 1) = h(\cdot, 0) - f_g$, where $f_g \in \mathbb{R}$ is the pre-defined safety threshold. PACSBO and SAFEOPT both start with the same three policy parameters S_0 .

Results. Figure 1.4 compares PACSBO with $\hat{N} = 100$ and $\bar{\alpha} = 1$ and SAFEOPT with $B_i = 10$. Note that this choice of $\hat{N}$ and $\bar{\alpha}$ heuristically yields random RKHS functions with RKHS norms of up to 10 (see Fig. 1.3). Depicted are the GP mean (blue line), the reward $h(\cdot, 0)$ (black line), the samples A (black dots), and the safety threshold f_g (dashed red line). The confidence intervals of PACSBO for $\tilde{\mathcal{A}}$, $\hat{\mathcal{A}}$, and $\mathcal{A}$ are depicted in gray, blue, and green, respectively, whereas the confidence intervals of SAFEOPT are shown in gray. PACSBO shows superior exploration compared to SAFEOPT, mainly because: (i) PACSBO learns the RKHS norm from data and is, thus, less conservative; (ii) PACSBO makes use of a local interpretation of the RKHS norm.

Figure 1.5 compares PACSBO with $\hat{N} = 100$ and $\bar{\alpha} = 0.04$ and SAFEOPT with $B = 0.4$ for the same function $h(\cdot, 0)$ as in the previous experiment. Hence, both algorithms make too optimistic smoothness assumptions. SAFEOPT cannot correct that assumption and samples unsafely. In contrast, PACSBO learns the RKHS norm and stays safe, as discussed in Remark 1.4.

1.5.2 Hardware Experiment

Next, we evaluate PACSBO on a real Furuta pendulum [27].

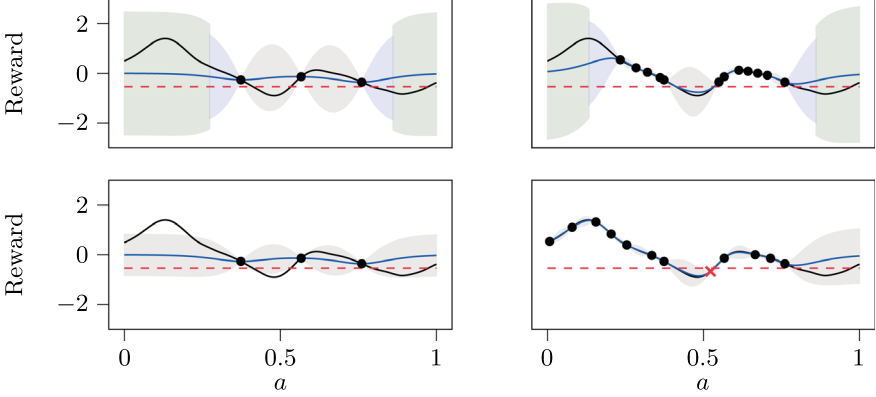


Fig. 1.5 An optimistic smoothness assumption yields unsafe experiments (red cross) with SAFEOPT (bottom), whereas PACSBO (top) stays safe

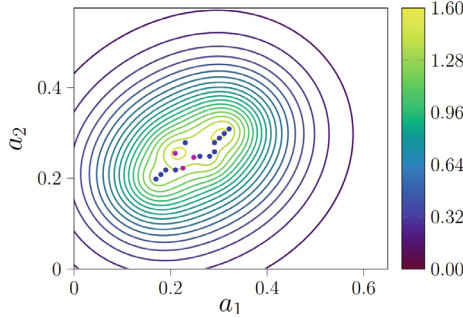


Fig. 1.6 PACSBO for Furuta pendulum

Setting. We consider the same experimental setup as Baumann et al. [28]. In particular, we consider state feedback control to stabilize the pendulum in an upright position and tune the first two entries of the static gain matrix. We model $h(\cdot, i)$ with $\mathcal{I}_g = \{1\}$ as independent GPs. Particularly, the reward $h(\cdot, 0)$ encourages stable behavior, while the constraint $h(\cdot, 1)$ considers the angle between the pendulum and the upright position; see Baumann et al. [28] for more details. For PACSBO, we use $\hat{N} = 100$ and $\bar{\alpha} = 1$.

Results. We show the explored region of $h(\cdot, 1)$ after 15 samples in Fig. 1.6, where the purple dots illustrate the initial sample set S_0 . We see that PACSBO explores the parameter space without prior knowledge of an RKHS norm upper bound. Importantly, the algorithm did not incur any failure during exploration. This demonstrates that PACSBO can safely learn control policies for hardware systems.

1.6 Conclusions

In this paper, we presented PACSBO, a novel algorithm for probably approximately correct safe BO. Unlike prior works on safe BO, especially SAFEOP and extensions of it, we drop the assumption of knowing tight upper bounds on the RKHS norms of underlying reward and constraint functions. Instead, we estimate the upper bound from data and theoretically investigate this RKHS norm estimation. Besides, we treat the RKHS norm as a local object in the area we are interested in. This local treatment reduces conservatism compared to assuming one global upper bound on the entire parameter space. Numerical experiments highlight the benefits of PACSBO compared to SAFEOP, while a hardware experiment demonstrates the applicability of PACSBO to real-world systems.

This paper is a first step towards removing the RKHS norm upper bound as a standing assumption in safe BO and replacing it with a data-driven estimate. As such, it leaves room for further work and improvements, starting from a more sophisticated design for the predictor, over different ways of approaching the locality of the RKHS norm, to proving safety and investigating scalability of PACSBO. Those extensions and investigations are left for future work.

1.7 Proof of Theorem 1.1

We seek to bound the maximum deviation between the empirical average of the norm of our random RKHS functions and their expected value. To this end, we leverage Hoeffding's inequality [24, Lemma 4.5]. Thus, we first show that the norms of the random RKHS functions are i.i.d. random variables conditioned on the samples. As we describe in Sect. 1.4, the randomness of the functions comes from the randomness of the coefficients α and the center points x . Specifically, given $A \subseteq \mathcal{A}$, $x_{1:N} := [x_1, \dots, x_N]^\top$ and $\alpha_{1:N} := [\alpha_1, \dots, \alpha_N]^\top$ are determined by the interpolating property and we independently sample $x_{N+1:\hat{N}}$ and $\alpha_{N+1:\hat{N}}$ from a uniform distribution. If the mapping represented by the RKHS norm from the i.i.d. coefficients and center points is measurable, then the RKHS norms are i.i.d. random variables [29, Theorem 1.3.5].

Choose any $A \subseteq \mathcal{A}$ and consider $F: (\mathbb{R}^{\hat{N}-N}, \mathcal{R}^{\hat{N}-N}) \rightarrow (\mathbb{R}_{\geq 0}, \mathcal{R}_{\geq 0})$, $F(O) = \sqrt{\sum_{s=1}^{\hat{N}} \sum_{t=1}^{\hat{N}} \alpha_s \alpha_t k(x_s, x_t)}$, a function that maps from vectors of random center points and coefficients to the RKHS norm with corresponding Borel σ -algebras. We next prove that F is a measurable function [30, Chapter 4.1]. Suppose that F is not measurable, i.e., $\exists E \in \mathcal{R}_{\geq 0} : \{O \in \mathbb{R}^{\hat{N}-N} | F(O) \in E\} \notin \mathcal{R}^{\hat{N}-N}$. In words, if F is not measurable, then there exists an event $E \in \mathcal{R}_{\geq 0}$ that contains $F(O)$ of outcomes $O \in \mathbb{R}^{\hat{N}-N}$ such that the pre-image of E under F , i.e., the outcomes O that generated $F(O)$, is not in the σ -algebra $\mathcal{R}^{\hat{N}-N}$, which is a contradiction

since the σ -algebra is the Borel σ -algebra. Hence, F is measurable. By exploiting [29, Theorem 1.3.5], we deduce that $F(O)$ is a random variable and, therefore, $\|\rho_{A,j}\|_k$ are i.i.d. random variables. Naturally, their expected value is given by $\lim_{s \rightarrow \infty} \sum_{j=1}^s \frac{1}{s} \|\rho_{A,j}(\cdot, i)\|_k$. Hence, we can state:

Proposition 1.1 *Let $\|\rho_{A,1}\|_k, \dots, \|\rho_{A,q}\|_k$ be a sequence of i.i.d. random variables and choose any $\delta \in (0, 1)$. Then, with probability at least $1 - \delta$, it holds that $\left| \frac{1}{q} \sum_{j=1}^q \|\rho_{A,j}(\cdot, i)\|_k - \lim_{s \rightarrow \infty} \frac{1}{s} \sum_{j=1}^s \|\rho_{A,j}(\cdot, i)\|_k \right| \leq w_i$.*

Proof The proposition follows from Hoeffding’s inequality [24, Lemma 4.5].

From Proposition 1.1, we can deduce that

$$\lim_{s \rightarrow \infty} \frac{1}{s} \sum_{j=1}^s \|\rho_{A,j}(\cdot, i)\|_k \leq \frac{1}{q} \sum_{j=1}^q \|\rho_{A,j}(\cdot, i)\|_k + w_i \quad (1.5)$$

holds with a probability of at least $1 - \delta$. Therefore, with probability $1 - \delta$,

$$B_i \stackrel{\text{Alg.2}}{\geq} \frac{1}{q} \sum_{j=1}^q \|\rho_{A,j}(\cdot, i)\|_k + w_i \stackrel{(5)}{\geq} \lim_{s \rightarrow \infty} \frac{1}{s} \sum_{i=1}^s \|\rho_{A,i}\|_k \stackrel{\text{Asm.1}}{\geq} \|h(\cdot, i)\|_k.$$

Acknowledgements We thank Sara Pérez-Vieites and Mingwei Deng for helpful comments. This research was partially supported by *Kjell och Märta Beijer Foundation*.

Disclosure of Interests The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Duan, Y., Chen, X., Houthooft, R., Schulman, J., Abbeel, P.: Benchmarking deep reinforcement learning for continuous control. In: International Conference on Machine Learning (2016)
2. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning (2019). arXiv preprint [arXiv:1509.02971v6](https://arxiv.org/abs/1509.02971v6)
3. Rasmussen, C.E., Williams, C.K.: Gaussian Processes for Machine Learning. MIT Press (2006)
4. Frazier, P.I.: A tutorial on Bayesian optimization (2018). arXiv preprint [arXiv:1807.02811](https://arxiv.org/abs/1807.02811)
5. Antonova, R., Rai, A., Atkeson, C.G.: Deep kernels for optimizing locomotion controllers (2017). arXiv preprint [arXiv:1707.09062](https://arxiv.org/abs/1707.09062)
6. Calandra, R., Seyfarth, A., Peters, J., Deisenroth, M.P.: Bayesian optimization for learning gaits under uncertainty. In: Annals of Mathematics and Artificial Intelligence (2016)
7. Marco, A., Hennig, P., Bohg, J., Schaal, S., Trimpe, S.: Automatic LQR tuning based on Gaussian process global optimization. In: IEEE International Conference on Robotics and Automation (2016)
8. Sui, Y., Gotovos, A., Burdick, J., Krause, A.: Safe exploration for optimization with Gaussian processes. In: International Conference on Machine Learning (2015)

9. Berkenkamp, F., Krause, A., Schoellig, A.P.: Bayesian optimization with safety constraints: safe and automatic parameter tuning in robotics. In: *Machine Learning* (2023)
10. Srinivas, N., Krause, A., Kakade, S.M., Seeger, M.: Information-theoretic regret bounds for Gaussian process optimization in the bandit setting. In: *IEEE Transactions on Information Theory*, vol. 5 (2012)
11. Calliess, J.-P., Roberts, S.J., Rasmussen, C.E., Maciejowski, J.: Lazily adapted constant Kinky Inference for nonparametric regression and model-reference adaptive control. *Automatica* **122** (2020)
12. Brunke, L., Greeff, M., Hall, A.W., Yuan, Z., Zhou, S., Panerati, J., Schoellig, A.P.: Safe learning in robotics: From learning-based control to safe reinforcement learning. In: *Annual Review of Control, Robotics, and Autonomous Systems* (2022)
13. Chowdhury, S.R., Gopalan, A.: On kernelized multi-armed bandits. In: *International Conference on Machine Learning* (2017)
14. Sui, Y., Zhuang, V., Burdick, J., Yue, Y.: Stagewise safe Bayesian optimization with Gaussian processes. In: *International Conference on Machine Learning* (2018)
15. Duivenvoorden, R.R., Berkenkamp, F., Carion, N., Krause, A., Schoellig, A.P.: Constrained Bayesian optimization with particle swarms for safe adaptive controller tuning. In: *IFAC World Congress* (2017)
16. Sukhija, B., Turchetta, M., Lindner, D., Krause, A., Trimpe, S., Baumann, D.: GoSafeOpt: scalable safe exploration for global optimization of dynamical systems. *Artif. Intell.* (2023)
17. Fiedler, C., Menn, J., Kreisköther, L., Trimpe, S.: On safety in safe Bayesian optimization (2024). arXiv preprint [arXiv:2403.12948](https://arxiv.org/abs/2403.12948)
18. Wood, G., Zhang, B.: Estimation of the Lipschitz constant of a function. *J. Glob. Optim.* (1996)
19. Hashimoto, K., Saoud, A., Kishida, M., Ushio, T., Dimarogonas, D.V.: Learning-based symbolic abstractions for nonlinear control systems. *Automatica* (2022). Extended version on [arxiv:1612.05327v3](https://arxiv.org/abs/1612.05327v3)
20. Scharnhorst, P., Maddalena, E.T., Jiang, Y., Jones, C.N.: Robust uncertainty bounds in reproducing kernel Hilbert spaces: a convex optimization approach. *IEEE Trans. Autom. Control* (2023)
21. Tokmak, A., Fiedler, C., Zeilinger, M.N., Trimpe, S., Köhler, J.: Automatic nonlinear MPC approximation with closed-loop guarantees. *IEEE Trans. Autom. Control* (2025)
22. Abbasi-Yadkori, Y.: Online learning for linearly parametrized control problems. Ph.D. thesis. University of Alberta (2013)
23. Steinwart, I., Christmann, A.: *Support Vector Machines*. Springer (2008)
24. Shalev-Shwartz, S., Ben-David, S.: *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press (2014)
25. Kanagawa, M., Hennig, P., Sejdinovic, D., Sriperumbudur, B.K.: Gaussian processes and kernel methods: a review on connections and equivalences (2018). arXiv preprint [arXiv:1807.02582](https://arxiv.org/abs/1807.02582)
26. Fiedler, C., Scherer, C.W., Trimpe, S.: Practical and rigorous uncertainty bounds for Gaussian process regression. In: *AAAI Conference on Artificial Intelligence* (2021). Extended version on [arXiv:2105.02796v2](https://arxiv.org/abs/2105.02796v2)
27. Furuta, K., Yamakita, M., Kobayashi, S.: Swing-up control of inverted pendulum using pseudo-state feedback. *Proc. Inst. Mech. Eng., Part I: J. Syst. Control Eng.* (1992)
28. Baumann, D., Marco, A., Turchetta, M., Trimpe, S.: GoSafe: Globally optimal safe robot learning. In: *IEEE International Conference on Robotics and Automation* (2021)
29. Durrett, R.: *Probability: Theory and Examples*. Cambridge University Press (2019)
30. DiBenedetto, E.: *Real Analysis*. Springer (2002)

Chapter 2

Value of Communication: Data-Driven Topology Optimization for Distributed Linear Cyber-Physical Systems



Michael Nestor and Fei Teng

Abstract Communication topology is a crucial part of a distributed control implementation for cyber-physical systems, yet is typically treated as a constraint within control design problems rather than a design variable. We propose a data-driven method for designing an optimal topology for the purpose of distributed control when a system model is unavailable or unaffordable, via a mixed-integer second-order conic program. The approach demonstrates improved control performance over random topologies in simulations and efficiently drops links which have a small effect on predictor accuracy, which we show correlates well with closed-loop control cost.

Keywords Communication topology · Data-driven control · Cyber-physical systems

2.1 Introduction

2.1.1 Context and Motivation

The communication topology and its impact on control performance is an emerging topic within the field of distributed control. Distinguished from centralized control, where all local agents must communicate with the central control agent, and decentralized control, where there is no communication at all, distributed control of interconnected dynamical systems has the flexibility to design which agents each agent should send information to or receive information from, even during run time given the increasing reconfigurability of communication solutions, such as software-defined networks. A higher degree of connectivity in the communication graph will improve control performance as each agent has a better picture of the global system, but will lead to increased communication overheads. This paper focuses on how to

M. Nestor (✉) · F. Teng

Department of Electrical and Electronic Engineering, Imperial College London, London, UK

e-mail: m.nestor22@imperial.ac.uk

balance these competing objectives of minimizing communication while achieving satisfactory control performance.

Distributed control that achieves cooperation through communication has been proposed as offering better performance than decentralized schemes while providing improved flexibility, privacy, and computational efficiency compared to centralized control [10]. Consider the example of power distribution grids; significant growth of distributed energy resources such as electric vehicles will require coordination to provide grid services and avoid violating grid constraints. A distributed control structure mirrors the distributed nature of these resources, with the potential for enhanced flexibility and cooperation between active grid entities such as virtual power plant operators or prosumer communities [18].

There must be a defined scheme for information exchange between agents in order to implement distributed control. All-to-all communication may be optimal from a control perspective, but is likely to lead to prohibitive communication and computation overheads in large-scale, complex systems [20]. Instead defining agent *neighborhoods* reduces the communication burden, where agents communicate with the agents in their neighborhood; however, determining these neighborhoods is not a trivial task. Poor choices may lead to inefficient communication at best and unsatisfactory control performance at worst. The neighborhoods can be determined from a system dynamics model, if a suitably accurate model is available. However, identifying such a model can be expensive and time-consuming. Recently, direct data-driven control methods that avoid model identification have gained traction [4, 9, 12]. However, it is unclear, under the data-driven setting, how to design adequate and yet efficient communication topology.

2.1.2 Literature Review

Typical distributed control problems take the communication topology to be a constraint rather than a design variable; the System Level Synthesis framework [3] treats controller sparsity structures as a constraint within the distributed control design problem. Distributed model predictive control (MPC) algorithms such as Conte [7] and data-driven distributed MPC schemes [1, 21] assume a known a priori set of neighbors for each agent. State feedback control in Celi et al. [5] guarantees convergence for an arbitrary communication graph but no information is provided concerning the influence of the topology on the convergence rate.

Previous works addressing communication topology design for control include sparsity-promoting optimal controllers [14] and co-design of topology and distributed controller to balance communication costs against performance [16]. Graph-based methods include community detection [10] and an optimization framework to find stabilizing control structures [24]. We refer to Chanfreut et al. [6] for a detailed survey. These works assume knowledge of the global system model, and provide a

basis for this data-driven approach. Recent work [8] uses data-driven methods for network design; this addresses the underlying physical network structure rather than the communication topology of a cyber-layer controller.

2.1.3 *Main Contributions and Paper Structure*

We consider a linear time-invariant (LTI) networked system consisting of subsystems coupled through their outputs. The problem we address is to design a communication topology suitable for a distributed control implementation without prior knowledge of a system model. We accomplish this through a centralized optimization problem carried out by a system coordinator, where we collect persistently exciting data from the global system that can be used to directly synthesize a communication topology and a multi-step output predictor using a least-square prediction error method. We frame our work in the multi-agent setting and take a modular approach such that we only tackle the topology design via the coordinator to set up the control design problem for the individual control agents to analyze; we assume that the topology is (re-)designed on a much slower timescale than control decisions are taken on. We co-optimize predictor and topology within a mixed-integer optimization problem to find a topology that balances communication costs and prediction accuracy and is optimal relative to the specified communication link cost weights. We validate the topology optimization through simulations, showing that prediction cost increases as links are dropped due to greater communication costs, and investigate the connection between the topology design and closed-loop control cost via distributed MPC simulations.

In summary, this paper makes the following original contributions:

- To derive a mixed-integer second-order conic optimization problem (MISOCP) that directly uses data to compute an optimal communication topology and linear predictor and to assess hyperparameter tuning choices.
- To find an upper bound on the prediction error induced by a given topology and the associated error in an open-loop control cost.
- To analyze the trade-off between the prediction error and the number of communication links (and the associated communication costs), and the nature of the connection between this trade-off and the realized control cost.

The mathematical formulation of the distributed dynamical system is given in Sect. 2.3. In Sect. 2.4, we present the mixed-integer conic optimization problem and derive bounds on the prediction error dependent on the chosen topology before simulation results and conclusions are given in Sects. 2.5 and 2.6.

2.2 Preliminaries

We write a T -sample long signal composed of the sequence $\{x(k)\}_{k=1}^T$ in vector form as $x_{[1:T]} = [x^\top(1) \dots x^\top(T)]^\top$. Its Hankel matrix form is defined by

$$H_N(x) = \begin{bmatrix} x(1) & x(2) & \cdots & x(T - N + 1) \\ x(2) & x(3) & \cdots & x(T - N + 2) \\ \vdots & \vdots & \ddots & \vdots \\ x(N) & x(N + 1) & \cdots & x(T) \end{bmatrix}.$$

Definition 2.1 (*Persistency of Excitation*) A signal $x_{[1:T]}$ where $x(k) \in \mathbb{R}^n$ is persistently exciting of order L if $\text{rank}(H_L(x)) = nL$.

We denote the matrix (block-) diagonal with n (block-) diagonal elements A_i by $\text{diag}\{A_i\}_{i=1}^n$, and use the notation that a matrix A

$$A = \begin{bmatrix} A_{11} & \cdots & A_{1N_2} \\ \vdots & \ddots & \vdots \\ A_{N_11} & \cdots & A_{N_1N_2} \end{bmatrix}$$

may be represented by $A = \{A_{ij}\}_{i=1:N_1, j=1:N_2}$, and if $N_1 = N_2$ we can write $A = \{A_{ij}\}_{i,j=1:N_1}$. $\mathbb{I}^{a \times b}$ is the identity matrix of dimension $a \times b$, $\mathbf{0}^{a \times b}$ is the zero matrix of dimension $a \times b$, and $\mathbb{1}^{a \times b}$ is a matrix of ones of dimension $a \times b$. $\mathbb{Z}_{>0}$ is the set of positive integers.

2.3 System Formulation

We consider a set of subsystems $\mathcal{V} = \{1, \dots, M\}$, where M is the total number of subsystems, within a multi-agent cyber-physical system (CPS) framework. Under this approach, we abstract two directed graphs: one for the physical layer, $\mathcal{G}_P = (\mathcal{V}, \mathcal{E}_P)$, and a second for the cyber-layer, $\mathcal{G}_C = (\mathcal{V}, \mathcal{E}_C)$, with $\{\mathcal{E}_P, \mathcal{E}_C\} \subseteq \mathcal{V} \times \mathcal{V}$. The physical layer graph represents the physical subsystems and the interconnections between them, while the cyber-layer graph represents the control agents (one for each subsystem) and the cyber-layer communication between the agents. Therefore, both graphs share the same vertex set but have differing edge sets. The edge set $\mathcal{E}_P$ is defined by the physical interconnections between subsystems, i.e., $(j, i) \in \mathcal{E}_P$ if the physical state of subsystem $i \in \mathcal{V}$ is directly influenced by subsystem $j \in \mathcal{V}$. Meanwhile, $\mathcal{E}_C$ is defined by the communication links between agents; $(j, i) \in \mathcal{E}_C$ if agent $j \in \mathcal{V}$ sends information to agent $i \in \mathcal{V}$. We write $\mathcal{N}_i^C = \{j : (j, i) \in \mathcal{E}_C\}$, with $\mathcal{N}_i^C$ denoting the cyber-neighborhood of agent i (the set of agents sending information

to agent i). The Boolean variable $\delta_{ij} \in \{0, 1\}$ represents whether a communication link exists from agent j to agent i :

$$\delta_{ij} = \begin{cases} 1 & \text{if } (j, i) \in \mathcal{E}_C \\ 0 & \text{otherwise.} \end{cases} \quad (2.1)$$

There is a cost associated with increasing the total number of communication links due to bandwidth limitations, and phenomena such as rapid increases in packet loss when the communication channel utilization threshold is breached [25]. We make a simplifying assumption that each link has a fixed cost.

Assumption 2.1 There is a scalar link cost $c_{ij} \in \mathbb{R}_{\geq 0}$ associated with every possible cyber-layer edge $(j, i) \in \mathcal{V} \times \mathcal{V}$.

A typical control problem is to determine the control inputs to minimize some cost as a function of the inputs and outputs of all the agents over a (possibly infinite) time horizon, subject to the system dynamics and without leaving a feasible region that inputs and/or outputs must stay within. In the multi-agent setting, each agent calculates its control input with a control function $\kappa_i(\cdot)$:

$$u_i(k) = \kappa_i(\xi_i(k), \theta_i, \xi_{\mathcal{N}_i^C}(k), \theta_{\mathcal{N}_i^C}), \quad (2.2)$$

where designing $\kappa_i(\cdot)$ is the control design problem, θ_i is a vector of local problem parameters such as the weights of the cost of $u_i(k)$ and $y_i(k)$, and $\xi_i(k)$ is an extended state vector of recent local inputs and output measurements (i.e., $\xi_i(k) = [u_i^\top(k-1) \dots u_i^\top(k-N_P) \ y_i^\top(k-1) \dots y_i^\top(k-N_P)]^\top$ for some $N_P \in \mathbb{Z}_{>0}$). Alongside local information θ_i and $\xi_i(k)$, agent i also has information $\theta_{\mathcal{N}_i^C} = \{\theta_j : j \in \mathcal{N}_i^C\}$ and $\xi_{\mathcal{N}_i^C} = \{\xi_j(k) : j \in \mathcal{N}_i^C\}$ from its set of neighboring agents $\mathcal{N}_i^C$ that it can use to improve its decision-making and reduce the control cost. We frame our approach with the following problem definition.

Problem 2.1 Design the cyber-layer edge set $\mathcal{E}_C$ with knowledge of the set of subsystems and agents $\mathcal{V}$, but without knowing the physical layer edge set $\mathcal{E}_P$.

If two subsystems are physically interconnected, then enabling information sharing between the controlling agents of the subsystems will reduce (or at worst, not increase) their closed-loop control cost. Stronger interconnections are likely to lead to a greater reduction in cost. Information sharing is enabled by an edge connecting these agents in the cyber-layer communication graph.

A distributed LTI system where subsystems are coupled through outputs and outputs are subject to noise may be described by

$$x_i(k+1) = A_i x_i(k) + B_i u_i(k) + \sum_{j=1}^M E_{ij} y_j(k) \quad (2.3a)$$

$$y_i(k) = C_i x_i(k) + D_i u_i(k) + v_i(k), \quad (2.3b)$$

where for subsystem $i \in \mathcal{V}$ and time step k , $x_i(k) \in \mathbb{R}^{n_i}$ is the state, $u_i(k) \in \mathbb{R}^{m_i}$ is the input, $y_i(k) \in \mathbb{R}^{p_i}$ is the output, and $v_i(k) \in \mathbb{R}^{p_i}$ is zero-mean white noise. The interconnection matrix $E_{ij} = \mathbf{0}^{n_i \times p_j} \forall i = j$. The global system dynamics can be written as

$$x^{(g)}(k+1) = Ax^{(g)}(k) + Bu^{(g)}(k) \quad (2.4a)$$

$$y^{(g)}(k) = Cx^{(g)}(k) + Du^{(g)}(k) + v^{(g)}(k), \quad (2.4b)$$

where the global state, input, output and noise vectors are given by $x^{(g)}(k) = [x_1^\top(k) \dots x_M^\top(k)]^\top$, $u^{(g)}(k) = [u_1^\top(k) \dots u_M^\top(k)]^\top$, $y^{(g)}(k) = [y_1^\top(k) \dots y_M^\top(k)]^\top$ and $v^{(g)}(k) = [v_1^\top(k) \dots v_M^\top(k)]^\top$, respectively. The global state-space matrices in (2.4) are given by

$$A = \begin{bmatrix} A_1 & E_{12}C_2 & \dots & E_{1M}C_M \\ E_{21}C_1 & A_2 & \dots & E_{2M}C_M \\ \vdots & \vdots & \ddots & \vdots \\ E_{M1}C_1 & E_{M2}C_2 & \dots & A_M \end{bmatrix}, B = \begin{bmatrix} B_1 & E_{12}D_2 & \dots & E_{1M}D_M \\ E_{21}D_1 & B_2 & \dots & E_{2M}D_M \\ \vdots & \vdots & \ddots & \vdots \\ E_{M1}D_1 & E_{M2}D_2 & \dots & B_M \end{bmatrix} \quad (2.5a)$$

$$C = \text{diag}\{C_i\}_{i=1}^M, D = \text{diag}\{D_i\}_{i=1}^M. \quad (2.5b)$$

Assumption 2.2 The subsystems in the set $\mathcal{V}$ all follow dynamics (2.3), and the global physical system made up of the graph $\mathcal{G}_P$ obeys dynamics (2.4). The pair (A, C) is observable, and the pair (A, B) is controllable. Furthermore, while we consider systems obeying the dynamics in (2.4), we assume that the matrices A_i , B_i , C_i , and D_i in (2.3) are unknown $\forall i \in \mathcal{V}$, and E_{ij} is unknown $\forall j \in \mathcal{V}$, $j \neq i$.

Due to Assumption 2.2, we use a central coordinator to generate an input signal $u_{[1:T]}^{(g),d}$ that obeys Assumption 2.3 to excite the dynamics; resulting output measurements allow an understanding of system behavior. The superscript d denotes collected data used in topology design.

Assumption 2.3 ([9, Lemmas 4.1 and 4.2]) For $\{T_{\text{ini}}, N\} \in \mathbb{Z}_{>0}$, the signal $u_{[1:T]}^{(g),d} \in \mathbb{R}^{m^{(g)}T}$ is persistently exciting of order $T_{\text{ini}} + N + n^{(g)}$; $m^{(g)} = \sum_{i=1}^M m_i$ and $n^{(g)} = \sum_{i=1}^M n_i$. T is chosen such that $T \geq (m^{(g)} + 1)(T_{\text{ini}} + N + n^{(g)}) - 1 = T_{\text{min}}$, and $T_{\text{ini}} \geq \ell$, where ℓ is the system lag (that is, the smallest integer such that the observability matrix $[C^\top (CA)^\top \dots (CA^{\ell-1})^\top]^\top$ has rank $n^{(g)}$).

The corresponding collected output signal is given by $y_{[1:T]}^{(g),d} \in \mathbb{R}^{p^{(g)}T}$, with $p^{(g)} = \sum_{i=1}^M p_i$. The collected input and output data may be written as Hankel matrices $U^{(g),d} = H_{T_{\text{ini}}+N}(u_{[1:T]}^{(g),d})$, $Y^{(g),d} = H_{T_{\text{ini}}+N}(y_{[1:T]}^{(g),d})$. We permute and partition these matrices to end up with four subsystem-ordered data matrices:

$$U^P = \begin{bmatrix} U_1^P \\ \vdots \\ U_M^P \end{bmatrix}, \quad Y^P = \begin{bmatrix} Y_1^P \\ \vdots \\ Y_M^P \end{bmatrix}, \quad U^F = \begin{bmatrix} U_1^F \\ \vdots \\ U_M^F \end{bmatrix}, \quad Y^F = \begin{bmatrix} Y_1^F \\ \vdots \\ Y_M^F \end{bmatrix}, \quad (2.6)$$

where $\forall i \in \mathcal{V} : U_i^P = H_{T_{\text{ini}}}(u_i^d[1:T-N])$, $Y_i^P = H_{T_{\text{ini}}}(y_i^d[1:T-N])$, $U_i^F = H_N(u_i^d[N+1:T])$, $Y_i^F = H_N(y_i^d[N+1:T])$, and the Hankel matrices are dimensioned by $U^P \in \mathbb{R}^{T_{\text{ini}}m^{(g)} \times (T-(T_{\text{ini}}+N)+1)}$, $Y^P \in \mathbb{R}^{T_{\text{ini}}p^{(g)} \times (T-(T_{\text{ini}}+N)+1)}$, $U^F \in \mathbb{R}^{Nm^{(g)} \times (T-(T_{\text{ini}}+N)+1)}$ and $Y^F \in \mathbb{R}^{Np^{(g)} \times (T-(T_{\text{ini}}+N)+1)}$. Note we have separated the first T_{ini} from the remaining N block rows of the Hankel matrices. If there is zero noise, the data matrix $[U^{(g),d^\top} Y^{(g),d^\top}]^\top$ will span the subspace of trajectories of length $T_{\text{ini}} + N$ if it has rank $Nm^{(g)} + n^{(g)}$ [23]. This result forms the cornerstone of the behavioral control methods [4, 9] and is aligned with the Subspace Predictive Control (SPC) approach [15]. The SPC methodology is based on subspace identification and makes use of a linear output predictor:

$$\hat{y}^{(g)} = K \begin{bmatrix} u_{\text{ini}}^{(g)} \\ y_{\text{ini}}^{(g)} \\ u_f^{(g)} \end{bmatrix}, \quad (2.7)$$

where $u_{\text{ini}}^{(g)} \in \mathbb{R}^{m^{(g)}T_{\text{ini}}}$ and $y_{\text{ini}}^{(g)} \in \mathbb{R}^{p^{(g)}T_{\text{ini}}}$ are the initialization trajectories (most recent T_{ini} inputs and outputs), $u_f^{(g)} \in \mathbb{R}^{m^{(g)}N}$ is the future N -step input sequence, and $y_f^{(g)} \in \mathbb{R}^{p^{(g)}N}$ is the N -step output prediction trajectory. $K \in \mathbb{R}^{p^{(g)}N \times (m^{(g)}+p^{(g)})T_{\text{ini}}+m^{(g)}N}$ is the predictor matrix. In Sect. 2.4, we will examine a standard least-norm optimization problem to determine K before including the influence of the communication topology within the prediction. First, we make a final assumption building on Assumption 2.2 to ensure the existence of a stabilizing control law for the designed topology.

Assumption 2.4 The parameters of the global system dynamics defined by (2.4) are such that the global system does not have any unstable decentralized fixed modes (DFMs) [11], hence there exists a decentralized control law (i.e., when $\delta_{ij} = 0 \forall \{i, j\} \in \mathcal{V}$) that results in a stable closed-loop system response, i.e., each subsystem can stabilize itself despite the coupling with neighbors.

Note that each agent i has access to the local information u_i and y_i required for a decentralized control implementation where $u_i(k) = \kappa_i(\theta_i, \xi_i(k))$ without communication, hence Assumption 2.4 holds if there are no unstable DFMs.

2.4 Communication Topology and Predictor Optimization

Traditional control approaches consist of a two-stage procedure of first estimating a parametric model of the system to be controlled before designing a controller to stabilize/steer the model according to the desired behavior, often with a third

intermediate step to design a state estimator with the model. The SPC approach [15] instead identifies a predictor of future system behavior using least-squares regression, which can then be used in an explicit control law to determine the optimal input sequence.

2.4.1 Least-Squares Predictor Identification

A similar prediction error method (PEM) that is combined with MPC [19] finds an optimal linear predictor that is used in the MPC optimization problem; this is formulated as a bilevel optimization. The predictor is found by considering each column in the Hankel matrices (2.6) as a trajectory; we can use (2.7) to make a prediction $\hat{Y}^F$ of Y^F ; $\hat{Y}^F = K \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix}$. The predictor K optimizing the least-squares fitting criterion is found as the minimizer of [13]:

$$\min_{\hat{K}} \left\| Y^F - \hat{K} \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2 \quad (2.8a)$$

$$\text{s.t. } \hat{K} = [\hat{K}_p^u \ \hat{K}_p^y \ \hat{K}_f] \quad (2.8b)$$

$$\text{rank}([\hat{K}_p^u \ \hat{K}_p^y]) = n^{(g)} \quad (2.8c)$$

$$\hat{K}_f \text{ Lower-Block Triangular,} \quad (2.8d)$$

where the rank constraint ensures LTI dynamics of order $n^{(g)}$ and the lower block triangular constraint ensures causality in predictions. Supposing Assumptions 2.2 and 2.3 hold, then the Hankel data matrices (2.6) in (2.8) span the trajectory subspace [9] and K calculated with (2.8) is able to accurately predict an output trajectory according to (2.7). When data is noise free, an exact closed-form solution exists and is given by Y^F multiplied by the pseudo-inverse of the concatenated remaining Hankel matrices (i.e., $K = Y^F [U^{P^\top} \ Y^{P^\top} \ U^{F^\top}]^\top$) as the solution to the unconstrained least-squares problem.

2.4.2 Including the Communication Costs in Predictor Optimization

We now consider the desire to minimize communication costs defined by Eq.(2.1) and supposing Assumption 2.1 holds, i.e., $\min_{\{\delta_{ij}\}_{i,j=1:M}} \sum_{i,j=1}^M c_{ij} \delta_{ij}$ with the set of Boolean δ_{ij} variables defining $\mathcal{E}_C$. Consider that if $\delta_{ij} = 0$, there is no link

transferring information from agent j to agent i , therefore agent i 's predictions cannot be calculated using the measurements or decisions of agent j . By writing K as

$$K = \begin{bmatrix} K_{p,11}^u & \cdots & K_{p,1M}^u & K_{p,11}^y & \cdots & K_{p,1M}^y & K_{f,11} & \cdots & K_{f,1M} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ K_{p,M1}^u & \cdots & K_{p,MM}^u & K_{p,M1}^y & \cdots & K_{p,MM}^y & K_{f,M1} & \cdots & K_{f,MM} \end{bmatrix}, \quad (2.9)$$

the communication topology influence on predictor structure is encoded by $\delta_{ij} = 0 \implies [K_{p,ij}^u \ K_{p,ij}^y \ K_{f,ij}] = 0$. We seek to choose the edge set $\mathcal{E}_C$ that minimizes communication costs and least-square prediction error by optimizing over $\delta_{ij} \ \forall \{i, j\} \in \mathcal{V}$ and $\hat{K}$ within a multi-criteria mixed-integer program:

$$\min_{\Delta, \hat{K}} \sum_{i,j=1}^M c_{ij} \delta_{ij} + \left\| Y^F - \hat{K} \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2 \quad (2.10a)$$

$$\text{s.t. } \delta_{ij} \in \{0, 1\} \quad \forall \{i, j\} \in \mathcal{V} \quad (2.10b)$$

$$\Delta = \{\delta_{ij}\}_{i,j=1:M} \quad (2.10c)$$

$$\hat{K} = [\hat{K}_p^u \ \hat{K}_p^y \ \hat{K}_f] \quad (2.10d)$$

$$\text{rank}([\hat{K}_p^u \ \hat{K}_p^y]) = n^{(g)} \quad (2.10e)$$

$$\hat{K}_f \text{ Lower-Block Triangular} \quad (2.10f)$$

$$\hat{K}_p^u = \{\hat{K}_{p,ij}^u\}_{i,j=1:M}, \ \hat{K}_p^y = \{\hat{K}_{p,ij}^y\}_{i,j=1:M}, \ \hat{K}_f = \{\hat{K}_{f,ij}\}_{i,j=1:M} \quad (2.10g)$$

$$\delta_{ij} = 0 \implies [\hat{K}_{p,ij}^u \ \hat{K}_{p,ij}^y \ \hat{K}_{f,ij}] = 0 \quad \forall \{i, j\} \in \mathcal{V}, \ i \neq j. \quad (2.10h)$$

In order to solve (2.10), we first perform a convex relaxation and drop the rank (2.10e) and causality (2.10f) constraints. Our approach is to increase the data trajectory length T and average over multiple datasets to reduce the effect of noise and implicitly the need for the rank constraint. As is noted in Dörfler et al. [13], ensuring a causal predictor structure is beneficial but not a strict requirement for predictive control; non-causal predictors can still achieve acceptable performance. Next, we perform a convex big-M relaxation [16] on the implication constraint (2.10h); that is, we replace it with:

$$\left. \begin{aligned} -\overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j T_{\text{ini}}} &\leq \hat{K}_{p,ij}^u \leq \overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j T_{\text{ini}}} \\ -\overline{M} \delta_{ij} \mathbb{1}_{p_i N \times p_j T_{\text{ini}}} &\leq \hat{K}_{p,ij}^y \leq \overline{M} \delta_{ij} \mathbb{1}_{p_i N \times p_j T_{\text{ini}}} \\ -\overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j N} &\leq \hat{K}_{f,ij} \leq \overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j N} \end{aligned} \right\} \forall \{i, j\} \in \mathcal{V}, \ i \neq j. \quad (2.11)$$

This relaxation ensures (2.10h) holds both when $\delta_{ij} = 0$, and when $\delta_{ij} = 1$ assuming that all elements of $\hat{K}$ are bounded by $-\overline{M} \leq \hat{K}_{ij} \leq \overline{M}$. The choice of $\overline{M}$ should be as small as possible to improve solver efficiency but large enough that it does not

constrain the predictor sub-matrices from giving accurate predictions. We arrive at a mixed-integer quadratic program:

$$\min_{\Delta, \hat{K}} \sum_{i,j=1}^M c_{ij} \delta_{ij} + \left\| Y^F - \hat{K} \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2 \quad (2.12a)$$

$$\text{s.t. } \delta_{ij} \in \{0, 1\} \quad \forall \{i, j\} \in \mathcal{V} \quad (2.12b)$$

$$\Delta = \{\delta_{ij}\}_{i,j=1:M} \quad (2.12c)$$

$$\hat{K} = [\hat{K}_p^u \quad \hat{K}_p^y \quad \hat{K}_f] \quad (2.12d)$$

$$\hat{K}_p^u = \{\hat{K}_{p,ij}^u\}_{i,j=1:M}, \quad \hat{K}_p^y = \{\hat{K}_{p,ij}^y\}_{i,j=1:M}, \quad \hat{K}_f = \{\hat{K}_{f,ij}\}_{i,j=1:M} \quad (2.12e)$$

$$\left. \begin{aligned} -\overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j T_{\text{ini}}} &\leq \hat{K}_{p,ij}^u \leq \overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j T_{\text{ini}}} \\ -\overline{M} \delta_{ij} \mathbb{1}_{p_i N \times p_j T_{\text{ini}}} &\leq \hat{K}_{p,ij}^y \leq \overline{M} \delta_{ij} \mathbb{1}_{p_i N \times p_j T_{\text{ini}}} \\ -\overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j N} &\leq \hat{K}_{f,ij} \leq \overline{M} \delta_{ij} \mathbb{1}_{p_i N \times m_j N} \end{aligned} \right\} \quad \forall \{i, j\} \in \mathcal{V}, \quad i \neq j, \quad (2.12f)$$

which may be represented as an MISOCP. While this can take significant computation to solve for very large systems, we note that the communication topology is designed offline therefore computational efficiency is of reduced importance compared to an online algorithm.

In the case of noisy data, the solution to problem (2.12) may give very low prediction error due to overfitting to the noise as we have dropped the rank constraint (2.10e). A common approach to get around this issue is to collect $N_{\text{coll}} \in \mathbb{Z}_{>0}$ datasets using exactly the same input sequence, and then average the Hankel matrices obtained from the N_{coll} experiments [19]. Since the noise is assumed to be zero mean, the effect is to remove the noise from the data. The online measurements used to make predictions (e.g., in Eq. (2.7)) will also be subject to noise; estimators such as the extended Kalman filter for output-feedback behavioral control [2] may be used to deal with this issue. Application of such approaches is outside the scope of this work as we focus on optimality of the communication topology and associated predictor and do not address online control. Supposing Assumption 2.4 holds, then a stabilizing control law exists for the optimized K .

2.4.3 Bounding the Prediction Error

By considering the case where the communication graph is fully connected, we can deduce a bound on the prediction error when links are dropped. Intuitively, we expect that a graph with more links will lead to a lower prediction error; Theorem 2.1 shows that the topology optimization scheme (2.12) is consistent with this and that the prediction cost of (2.12) under the optimized topology is bounded by the cost of the dropped links. We will use this to find an upper bound on the open-loop prediction error induced by dropping links. Adding a constraint that $\delta_{ij} = 1 \quad \forall \{i, j\} \in \mathcal{V}, \quad i \neq j$

leads to (2.12) simplifying to an unconstrained optimization, assuming that $\bar{M}$ is sufficiently large:

$$\min_{\hat{K}} J'(K') = \min_{\hat{K}} \sum_{i,j=1, i \neq j}^M c_{ij} + \left\| Y^F - \hat{K} \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2. \quad (2.13)$$

We denote a minimizer of (2.13) by K' , and the value the cost function of (2.13) evaluated at the minimizer by $J'(K')$. The topology-predictor pair minimizing (2.12) is given by (Δ^*, K^*) , and the cost function evaluation at this minimizer is denoted as $J(\Delta^*, K^*)$, with $\Delta^* = \{\delta_{ij}^*\}_{i,j=1:M}$.

Theorem 2.1 *Considering the t -th column of Y^F as an N -step output trajectory y_f^t of (2.4) and the t -th column of $\hat{Y}^F$ to be a prediction $\hat{y}_f^t$ of y_f^t where $1 \leq t \leq T - (T_{ini} + N) + 1$, the sum of the Euclidean norm of the trajectory prediction errors $\sum_{t=1}^{T-(T_{ini}+N)+1} \|y_f^t - \hat{y}_f^t\|_2^2$ induced by a given optimized topology Δ^* and predictor K^* is lower bounded by the prediction error with a fully con-*

nected communication graph equal to $\|Y^F(I - \Pi)\|_F^2$, where $\Pi = \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix}^\dagger \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix}$, and upper bounded by the sum of this lower bound with the sum of the communication costs of the links not included in the optimized topology, equal to $\|Y^F(I - \Pi)\|_F^2 + \sum_{i,j=1, i \neq j}^M c_{ij}(1 - \delta_{ij}^)$.*

Proof We write $J(\Delta^*, K^*) = \sum_{i,j=1}^M c_{ij}\delta_{ij}^* + \left\| Y^F - K^* \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2$. Since $\Delta_1 = \{\delta_{ij}\}_{i,j=1:M}$ where $\delta_{ij} = 1 \forall \{i, j\} \in \mathcal{V}$, $i \neq j$ is a feasible topology for (2.12), which has a feasible optimal predictor K' when $\bar{M}$ is sufficiently large, (Δ_1, K') is a feasible solution of (2.12). Therefore $J(\Delta^*, K^*) \leq J(\Delta_1, K') = J'(K')$, so $J(\Delta^*, K^*) - J'(K') \leq 0$:

$$\sum_{i,j=1}^M c_{ij}\delta_{ij}^* - \sum_{i,j=1, i \neq j}^M c_{ij} + \left\| Y^F - K^* \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2 - \left\| Y^F - K' \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2 \leq 0. \quad (2.14)$$

Optimization (2.13) is unconstrained and the communication costs do not affect the location of the minimum, hence K' is a global minimizer of $J'(\hat{K})$ and the

least-square prediction error given by $K' = Y^F \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix}^\dagger$. This naturally leads to

$$\left\| Y^F \left(I - \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix}^\dagger \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right) \right\|_F^2 = \|Y^F(I - \Pi)\|_F^2 \leq \left\| Y^F - K^* \begin{bmatrix} U^P \\ Y^P \\ U^F \end{bmatrix} \right\|_F^2. \quad (2.15)$$

Considering the Frobenius norm as the sum of the 2-norm of the matrix columns (with each column a trajectory and a prediction, respectively, in the left and right matrix evaluations inside the Frobenius norm), from (2.14) we find

$$\sum_{t=1}^{T-(T_{\text{ini}}+N)+1} \|y_f^t - \hat{y}_f^t\|_2^2 \leq \sum_{i,j=1, i \neq j}^M c_{ij}(1 - \delta_{ij}^*) + \|Y^F(I - \Pi)\|_F^2. \quad (2.16)$$

Note we can always set $\delta_{ii} = 0 \ \forall i \in \mathcal{V}$ without influencing prediction accuracy. Applying the same consideration to (2.15) gives

$$\|Y^F(I - \Pi)\|_F^2 \leq \sum_{t=1}^{T-(T_{\text{ini}}+N)+1} \|y_f^t - \hat{y}_f^t\|_2^2, \quad (2.17)$$

providing the upper and lower bounds on the total prediction error over all the samples in the collected dataset as desired. Note that the lower bound evaluates to zero when data is noise free as exact predictions may be found. $\square$

While bounds on the overall prediction error induced by a given topology are informative, it is useful to have bounds on the error for an individual trajectory prediction as would be used in solving a control problem.

Lemma 2.1 *Consider a control output penalty term for a regulation problem of the form $\|y_f\|_Q^2$ where $Q \succeq 0$ is a weighting matrix. The error in the open-loop prediction cost, given by $\|y_f - \hat{y}_f\|_Q^2$, is bounded by $\lambda_{Q,\max}(\sum_{i,j=1, i \neq j}^M c_{ij}(1 - \delta_{ij}^*) + \|Y^F(I - \Pi)\|_F^2)$, where $\lambda_{Q,\max}$ is the largest eigenvalue of Q .*

Proof A worst-case bound on any individual trajectory y_f is found from (2.16) as

$$\|y_f - \hat{y}_f\|_2^2 \leq \sum_{i,j=1, i \neq j}^M c_{ij}(1 - \delta_{ij}^*) + \|Y^F(I - \Pi)\|_F^2. \quad (2.18)$$

We can write $Q = V_Q \Lambda_Q V_Q^\top$ since Q is symmetric, where V_Q is a matrix of eigenvectors of Q and Λ_Q is a matrix of its eigenvalues. Let $y_f - \hat{y}_f = V_Q \tilde{y}$, then $\|y_f - \hat{y}_f\|_Q^2 = \tilde{y}^\top V_Q^\top V_Q \Lambda_Q V_Q^\top V_Q \tilde{y} = \tilde{y}^\top \Lambda_Q \tilde{y}$ since $V_Q^\top = V_Q^{-1}$. Note

that $\|y_f - \hat{y}_f\|_2^2 = \tilde{y}^\top V_Q^\top V_Q \tilde{y} = \tilde{y}^\top \tilde{y}$ and $\tilde{y}^\top \Lambda_Q \tilde{y} \leq \lambda_{Q,\max} \tilde{y}^\top \tilde{y}$, therefore $\|y_f - \hat{y}_f\|_Q^2 \leq \lambda_{Q,\max} \|y_f - \hat{y}_f\|_2^2$. Combining this fact with (2.18) gives the result:

$$\|y_f - \hat{y}_f\|_Q^2 \leq \lambda_{Q,\max} \left(\sum_{i,j=1, i \neq j}^M c_{ij}(1 - \delta_{ij}^*) + \|Y^F(I - \Pi)\|_F^2 \right). \quad (2.19)$$

□

This upper bound gives the worst-case output prediction weighted according to the control weight matrix Q ; however, it is likely to be rather conservative. We next present simulation results using (2.12) where we optimize communication topology and predictor and observe that the open-loop prediction cost error, bounded by Lemma 2.1, does indeed correlate with closed-loop control cost.

2.5 Simulations

We use the linearized and discretized swing dynamics as given in Alonso et al. [1] for topology optimization and control simulations. Each subsystem has dynamics (2.3), with $x_i(k) = [\theta_i(k) \ \omega_i(k)]^\top$, $B_i = \left[0 \ \frac{1}{m_i}\right]^\top$, $C_i = [1 \ 0]$, $D_i = \mathbf{0}^{p_i \times m_i}$ and the matrices A_i and E_{ij} are given by

$$A_i = \begin{bmatrix} 1 & \Delta t \\ -\frac{k_i}{m_i} \Delta t & 1 - \frac{d_i}{m_i} \Delta t \end{bmatrix} \quad E_{ij} = \begin{bmatrix} 0 \\ \frac{k_{ij}}{m_i} \Delta t \end{bmatrix}, \quad (2.20)$$

where θ_i and ω_i represent phase angle and frequency deviations, and m_i , d_i , and k_{ij} represent inertia, damping, and coupling for each subsystem, with the time step given by $\Delta t = 0.2$ and $k_i = \sum_{j=1/i}^M k_{ij}$. We choose $M = 4$, $m_1 = 1.4$, $m_2 = 0.8$, $m_3 = 0.6$, $m_4 = 0.9$, $d_1 = 0.6$, $d_2 = d_4 = 0.65$, $d_3 = 0.75$, $k_{12} = k_{21} = 1.25$, $k_{23} = k_{32} = 1.2$, $k_{34} = k_{43} = 0.075$, $k_{13} = k_{14} = k_{24} = k_{31} = k_{41} = k_{42} = 0$. A persistently exciting input signal $u_{[1:T]}^{(g),d}$ was created using a pseudo-randomly generated sequence of zero-mean normally distributed numbers with variance $\sigma_d^2 = 1$ and injected into the test system within MATLAB® simulations. The collected output data $y_{[1:T]}^{(g),d}$ and input $u_{[1:T]}^{(g),d}$ form the data Hankel matrices (2.6) in order to solve (2.12). We choose $T_{\text{ini}} = 3$, $N = 5$, and $\overline{M} = 5$. Following tuning described below, T and N_{coll} were chosen as 200 and 50, respectively.

The optimization (2.12) is solved in MATLAB® using GUROBI [17] with the YALMIP toolbox [22], and we use an identical link cost $c_{ij} = c \ \forall \{i, j\} \in \mathcal{V}$. We test prediction accuracy and assess if overfitting to noise is occurring by injecting a random signal (each element sampled from a zero-mean normal distribution with variance σ_d^2) into each subsystem and calculating the mean-squared error (MSE) of

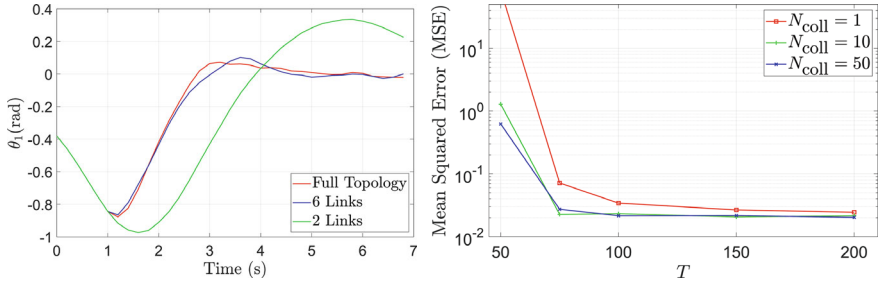
Table 2.1 Optimization results with an SNR of 10^3 , showing that links are dropped as the communication cost grows, resulting in increasing prediction cost and MSE

c	# Links	Topology	Pred. cost	MSE
0.001	12	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$	0.31	0.100
1	6	$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$	0.68	0.119
20	4	$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$	16.73	0.457
1000	2	$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$	1034.71	1.247

predictions compared to true (denoised) outputs. Results for an SNR of 10^3 averaged over 50 trials are presented in Table 2.1, giving the adjacency matrix for $\mathcal{G}_C$, optimization prediction cost and prediction MSE for different communication costs. Links are first dropped between subsystems that are uncoupled before links between coupled subsystems are dropped. Results show that half the links can be dropped before significant prediction errors occur; note that there are 6 couplings (non-zero k_{ij}) out of a possible maximum of 12.

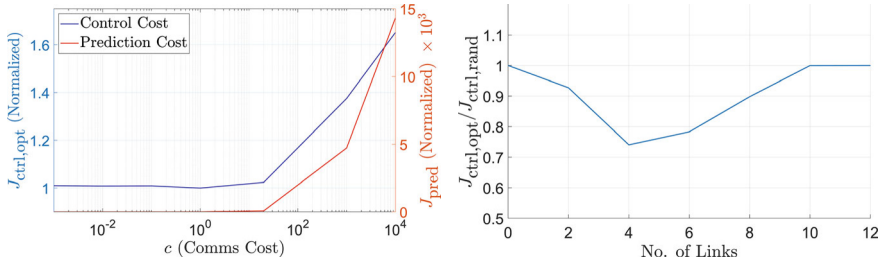
Our approach is to reduce the effects of noise by averaging over N_{coll} datasets and using a $T > T_{\text{min}}$. We investigate varying these hyperparameters to provide sufficient performance without unduly large computational burdens. Results averaged over 500 trials based on an all-to-all topology are shown in Fig. 2.1b. MSE converges as T and N_{coll} increase; we choose $T = 200$ and $N_{\text{coll}} = 50$ to benefit both topology optimization and data-driven control algorithms.

Finally, we investigate the influence of topology choice on control cost and its correlation with prediction cost using test system (2.20). Control is implemented using a non-cooperative data-driven MPC scheme based on Kohler et al. [21] with a minor extension using slack variables, similar to Coulson et al. [9], to guarantee feasibility and stability with measurement noise. Output trajectories are shown in Fig. 2.1a for 3 optimized topologies. There is a clear correlation between prediction cost of the topology optimization and closed-loop control cost, shown in Fig. 2.2a, in accordance with our approach to use prediction cost as a metric for topology optimality. Note the significant difference in growth rate of control and prediction



(a) Evolution of θ_1 for all-to-all communication and optimized topologies with 2 and 6 links
 (b) MSE decreases with increasing T and N_{coll} (results for an SNR of 10^3)

Fig. 2.1 Simulation results for **a** no noise and **b** tuning T and N_{coll}



(a) Control cost and prediction cost from topology optimization are closely correlated
 (b) Ratio of control cost for optimized topologies and a random topology with the same number of links

Fig. 2.2 Control simulation results showing **a** control cost $J_{\text{ctrl,opt}}$ and prediction cost J_{pred} for optimized topologies and **b** the ratio of $J_{\text{ctrl,opt}}$ to random topology control cost $J_{\text{ctrl,rand}}$

costs. The control cost for an optimized topology consistently outperforms a random topology, shown in Fig. 2.2b; topologies with both 0 and 12 links cannot be random so these costs are equal. There is a maximum reduction of around 25 % in control cost for the optimized topology compared to the average random topology. This demonstrates the capability of the scheme to find an effective communication topology for distributed control.

2.6 Conclusions and Outlook

We have proposed a fully data-driven method for optimizing the communication topology of distributed control schemes via an MISOCP for LTI cyber-physical systems that do not have any unstable DFMs, assuming knowledge of a persistently exciting input sequence of sufficient order. The prediction cost under the optimized

topology is upper bounded by the sum of the link costs not included in the topology with the prediction cost under all-to-all communication. Simulations demonstrated that the optimized topology achieves low prediction error up to a threshold where prediction accuracy decreases as further links are dropped. Control simulations showed that the prediction cost of the topology optimization is correlated with realized control cost, and demonstrated reductions in control cost using the optimized topology compared to a random topology. Future work aims to improve scalability via distributed optimization, further analyze the connection between prediction cost and closed-loop control cost, and investigate data-driven methods to guarantee stability when unstable DFMs exist.

References

1. Alonso, C.A., Yang, F., Matni, N.: Data-driven distributed and localized model predictive control. *IEEE Open J. Control Syst.* **1**, 29–40 (2022)
2. Alpag0, D., Dörfler, F., Lygeros, J.: An extended Kalman filter for data-enabled predictive control. *IEEE Control Syst. Lett.* **4**(4), 994–999 (2020)
3. Anderson, J., Doyle, J.C., Low, S.H., Matni, N.: System level synthesis. *Ann. Rev. Control* **47**, 364–393 (2019)
4. Berberich, J., Köhler, J., Müller, M.A., Allgöwer, F.: Data-driven model predictive control with stability and robustness guarantees. *IEEE Trans. Autom. Control* **66**, 1702–1717 (2019)
5. Celi, F., Baggio, G., Pasqualetti, F.: Distributed data-driven control of network systems. *IEEE Open J. Control Syst.* 1–15 (2023)
6. Chanfreut, P., Maestre, J.M., Camacho, E.F.: A survey on clustering methods for distributed and networked control systems. *Ann. Rev. Control* **52**, 75–90 (2021)
7. Conte, C., Jones, C.N., Morari, M., Zeilinger, M.N.: Distributed synthesis and stability of cooperative distributed model predictive control for linear systems. *Automatica* **69**, 117–125 (2016)
8. Coraggio, M., di Bernardo, M.: Data-driven design of complex network structures to promote synchronization. *arXiv preprint, submitted to IEEE Control Systems Letters and 2023 American Control Conference (ACC)* (2023)
9. Coulson, J., Lygeros, J., Dörfler, F.: Data-enabled predictive control: in the shallows of the DeePC. In: 2019 18th European Control Conference (ECC), pp. 307–312 (2019)
10. Daoutidis, P., Tang, W., Jogwar, S.S.: Decomposing complex plants for distributed control: perspectives from network theory. *Comput. Chem. Eng.* **114**, 43–51 (2018), fOCAPO/CPC 2017
11. Davison, E., Özgüner, U.: Characterizations of decentralized fixed modes for interconnected systems. *Automatica* **19**(2), 169–182 (1983)
12. De Persis, C., Tesi, P.: Formulas for data-driven control: stabilization, optimality, and robustness. *IEEE Trans. Autom. Control* **65**(3), 909–924 (2020)
13. Dörfler, F., Coulson, J., Markovsky, I.: Bridging direct and indirect data-driven control formulations via regularizations and relaxations. *IEEE Trans. Autom. Control* **68**(2), 883–897 (2023)
14. Dörfler, F., Jovanović, M.R., Chertkov, M., Bullo, F.: Sparsity-promoting optimal wide-area control of power networks. *IEEE Trans. Power Syst.* **29**(5), 2281–2291 (2014)
15. Favoreel, W., Moor, B.D., Gevers, M.: SPC: subspace predictive control. *IFAC Proc.* **32**(2), 4004–4009 (1999), 14th IFAC World Congress, Beijing, China, 5–9 July 1999
16. Groß, D., Stursberg, O.: Optimized distributed control and network topology design for interconnected systems. In: 2011 50th IEEE Conference on Decision and Control and European Control Conference, pp. 8112–8117 (2011)

17. Gurobi Optimization, LLC: Gurobi optimizer reference manual (2023). <https://www.gurobi.com>
18. Han, X., Heussen, K., Gehrke, O., Bindner, H.W., Kroposki, B.: Taxonomy for evaluation of distributed control strategies for distributed energy resources. *IEEE Trans. Smart Grid* **9**(5), 5185–5195 (2018)
19. Huang, L., Coulson, J., Lygeros, J., Dörfler, F.: Data-enabled predictive control for grid-connected power converters. In: 2019 IEEE 58th Conference on Decision and Control (CDC), pp. 8130–8135 (2019)
20. Jovanović, M.R., Dhingra, N.K.: Controller architectures: tradeoffs between performance and structure. *Eur. J. Control* **30**, 76–91 (2016), 15th European Control Conference, ECC16
21. Kohler, M., Berberich, J., Müller, M.A., Allgower, F.: Data-driven distributed MPC of dynamically coupled linear systems. *IFAC-PapersOnLine* **55**(30), 365–370, 25th International, p. 2022. Symposium on Mathematical Theory of Networks and Systems, MTNS (2022)
22. Löfberg, J.: Yalmip : A toolbox for modeling and optimization in Matlab. In: Proceedings of the CACSD Conference, Taipei, Taiwan (2004)
23. Markovsky, I., Dörfler, F.: Behavioral systems theory in data-driven analysis, signal processing, and control. *Ann. Rev. Control* **52**, 42–64 (2021)
24. Mosalli, H., Babazadeh, M.: Stabilizing control structures: an optimization framework. *IEEE Trans. Autom. Control* **67**(7), 3738–3745 (2022)
25. Wu, D., Nie, X., Asmare, E., Arkhipov, D.I., Qin, Z., Li, R., McCann, J.A., Li, K.: Towards distributed SDN: mobility management and flow scheduling in software defined urban IoT. *IEEE Trans. Parallel Distrib. Syst.* **31**(6), 1400–1418 (2020)

Chapter 3

Variance-Informed Model Reference Gaussian Process Regression: Utilizing Variance Information for Control in Nonlinear Systems



Hyuntae Kim 

Abstract Recent advances in data-driven control systems have highlighted the adaptability of Gaussian Process Regression (GPR) in managing complex, nonlinear dynamics. Unlike traditional approaches that sequentially conduct system identification and model-based control design, this paper introduces the Variance-Informed Model Reference Gaussian Process Regression (MR-GPR). This method integrates GPR directly into a model reference framework using input/state data to infer inverse dynamics, enhancing control strategies through the utilization of predictive variance. Applied to discrete-time nonlinear systems, the variance-informed MR-GPR approach effectively adapts to changing conditions by leveraging predictive variance and real-time feedback, thereby enhancing control system robustness without the need for explicit system identification.

Keywords Data-driven control · Nonlinear system · Gaussian process

3.1 Introduction

In modern control theory, managing the unpredictability of complex, nonlinear systems poses a significant challenge. Gaussian Process Regression (GPR) has emerged as a crucial tool in this context, prized for its ability to model dynamic systems without the constraints of traditional mathematical formulations [1–5]. This probabilistic technique excels in managing systems with complex and evolving dynamics, frequently encountered in sectors that demand high adaptability and precision [6–9]. The dual capability of GPR to predict system behavior and quantify the associated uncertainty is particularly valuable in environments where operational conditions vary widely [10–13]. Consequently, GPR empowers controllers to adjust strategies dynamically based on real-time data, significantly enhancing system responsiveness

H. Kim (✉)

ASRI, Seoul National University, Seoul, South Korea

e-mail: htkim@csl.kr

and stability [14, 15]. This shift toward more data-driven control methods marks a substantial evolution from traditional approaches, promoting increased adaptability and robustness in critical applications.

Traditionally, GPR has been a cornerstone within the machine learning field, renowned for its precision in making predictions without relying on parametric models [16, 17]. Its ability to manage large, complex datasets and quantify uncertainty has made it invaluable for constructing robust predictive models [18, 19]. Extending these capabilities into control systems, GPR is also used to develop adaptive controllers that dynamically adjust to the changing parameters of the systems they govern, addressing the challenge of unknown and hard-to-estimate system parameters [20]. This adaptability is critical in applications where traditional, static models fail to capture the dynamics of complex environments [21, 22]. By leveraging continuous data, GPR enhances the stability and effectiveness of control systems, making them more responsive and adaptable to unpredictable conditions [23].

The advent of Model Reference Gaussian Process Regression (MR-GPR) controllers signifies a crucial advancement in the realm of nonlinear control systems [24, 25]. These controllers effectively harness the robust capabilities of GPR to navigate complex dynamics without relying on conventional mathematical models. While MR-GPR offers significant potential, it faces inherent challenges in managing the uncertainty associated with predictive modeling. This uncertainty can impede the effectiveness of control actions, particularly in environments characterized by high variability and unpredictability.

Addressing these challenges, this paper introduces the variance-informed MR-GPR method. This approach incorporates a direct evaluation of prediction variance into the control strategy, providing a nuanced response to uncertainties faced during real-time system operation. By integrating variance assessments, Variance-Informed MR-GPR not only adapts to the immediate state of the system but also fine-tunes control actions based on the confidence levels associated with its predictions. This enhancement significantly bolsters the robustness and dependability of the control system, particularly in applications where conditions are constantly evolving.

Section 3.2 outlines the problem formulation and assumptions. Section 3.3 details the controller design and stability analysis. Section 3.4 presents an illustrative example, and Sect. 3.5 concludes with findings and implications for various applications.

Notation: For column vectors a and b , $[a; b]$ denotes $[a^T, b^T]^T$. For discrete-time vector sequences $y(t)$ and $z(t)$, we define a vector

$$z_{[k, k+T]} := [z(k); z(k+1); \dots; z(k+T)],$$

and a set

$$\begin{aligned} & \{(y(t), z(t))\}_{t=k}^{k+T} \\ & := \{(y(k), z(k)), \dots, (y(k+T), z(k+T))\}. \end{aligned}$$

The notation $\mathcal{B}_r(0_n)$ denotes an open ball in n -dimensional space, centered at 0_n with radius r .

3.2 Problem Formulation

Consider the following discrete-time nonlinear system described by the equation:

$$x(t + 1) = f(x(t), u(t)), \quad (3.1)$$

where t denotes the discrete-time index over the integers $\mathbb{Z}$. The state variable $x(t)$ resides in $\mathbb{R}^n$, and the control input $u(t)$ is constrained within a compact subset $\mathcal{U} \subset \mathbb{R}^m$. The dynamics of the system are characterized by the continuously differentiable mapping $f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$. Notably, the explicit representation of $f(\cdot, \cdot)$ remains unknown, and only the state variables and control inputs are observable. For simplicity, we define the permissible set of states as $\mathcal{X} := \mathcal{B}_r(0_n)$, where $x \in \mathcal{B}_r(0_n) \subset \mathbb{R}^n$.

For this system, we impose the following assumption:

Assumption 3.1 For any given state $x(t)$ within $\mathcal{X}$ and at any non-negative discrete time t , there exists a control input $u(t)$ within the set $\mathcal{U} \subset \mathbb{R}^m$, ensuring that the state trajectory adheres to

$$y(t) = f(x(t), u(t)) \in \mathcal{X},$$

where $\|y(t)\| < \|x(t)\|$. □

This assumption enables the formulation of the control mapping

$$u(t) = c([x(t); x(t + 1)]), \quad (3.2)$$

applicable for all $t \geq 0$ for given $x(t), x(t + 1) \in \mathcal{X}$. When the control input $u(t) = c([x(t); z(t)])$ with the desired future state $z(t) \in \mathcal{X}$ is applied to the system dynamics (3.1), the next state $x(t + 1)$ is determined as follows:

$$\begin{aligned} x(t + 1) &= f(x(t), u(t)) \\ &= f(x(t), c([x(t); z(t)])) \\ &= z(t). \end{aligned}$$

In previous MR-GPR studies [24, 25], the control strategy was based on the assumption that the desired state $x(t + 1)$ could be represented as $x(t + 1) = Rx(t)$, where R is a known Schur matrix. This matrix R was crucial in ensuring that the system's state remains within the desired set $\mathcal{X}$, thereby guaranteeing system stability. The control input was then determined using the mapping:

$$u(t) = c([x(t); Rx(t)]).$$

However, the practical application of this state feedback control strategy involves challenges due to the variability of R and the specificity of the control impacts. Consider the following example.

Example 3.1 Consider the system dynamics as defined by (3.1), where the system is subject to the discrete-time nonlinear control process:

$$\begin{aligned}x_1(t+1) &= x_2(t), \\x_2(t+1) &= f([x_1; x_2]) + g([x_1; x_2])u(t),\end{aligned}$$

where x_1 , x_2 , and u are all scalars. While this configuration adheres to the structure specified in (3.1), the process of arbitrarily selecting a transformation matrix R proves to be impractical due to the singular influence of the control input $u(t)$ on $x_2(t)$ only. This restricts the applicable form of R to those that are Schur with specific structure, typically

$$R = \begin{bmatrix} 0 & 1 \\ R_{21} & R_{22} \end{bmatrix},$$

where all eigenvalues must be less than one to ensure stability. This exemplifies the challenges associated with the control input's specificity and its localized impact on the system, emphasizing that not any R is suitable for this system configuration. $\square$

This example illustrates the necessity for a precise and dynamically adaptable control strategy that can accommodate specific system characteristics and variabilities. In the conventional approach, where R was previously known and assumed time invariant, the direct application becomes untenable under conditions where R is unknown or its functional form is subject to change over time. Hence, an adaptive method is required to effectively manage these dynamics.

To address these complexities, the methodology proposed in this paper leverages data-driven techniques not only to approximate the control mapping but also to identify an optimal state $y(t)$ for the control actions. This approach is encapsulated in the modified control equation:

$$u(t) = c([x(t); y(t)]), \quad (3.3)$$

where $y(t)$ is defined such that $\|y(t)\| < \|x(t)\|$, ensuring a reduction in the state norm and thereby aiming to achieve a controlled trajectory that gravitates toward system stability.

By employing GPR, particularly capitalizing on its ability to handle variance information, the methodology aims to refine the control mappings continuously. This process allows for the dynamic adaptation of the controller in response to real-time feedback and changing conditions, thereby facilitating the design of a robust control system even when an appropriate transformation matrix R or an optimal state $y(t)$ is unknown.

3.3 Main Result

This section introduces a control methodology that harnesses GPR based on state and input measurements from the system described in (3.1).

The primary goal of utilizing GPR is to comprehensively understand the functionality of the control mapping c , as specified in (3.2). The method interprets the concatenated states $[x(t); x(t+1)]$ as inputs to the control mapping c , with $u(t)$ as the resultant output, in accordance with the relationship described in (3.3). The control mapping c operates over the domain $\mathcal{C} := \mathcal{X} \times \mathcal{X}$, where $\mathcal{X}$ represents the set of permissible states. For GPR, we split the function $c(\cdot)$ as

$$c(\cdot) := \begin{bmatrix} c_1(\cdot) \\ c_2(\cdot) \\ \vdots \\ c_m(\cdot) \end{bmatrix},$$

where $c_i : \mathbb{R}^{2n} \rightarrow \mathbb{R}$ for $i = 1, \dots, m$.

To implement this strategy, we commence by gathering pairs of input and state data¹ as per the system configuration in (3.1):

$$\{(u_d(t), x_d(t))\}_{t=1}^N, \quad (3.4)$$

where $N > 1$ represents the number of these pairs collected. Subsequently, this data is reorganized to formulate the training inputs:

$$\xi_d(t) := [x_d(t); x_d(t+1)] \in \mathbb{R}^{2n},$$

and the corresponding outputs:

$$u_d(t) \in \mathbb{R}^m,$$

thus creating a comprehensive training dataset:

$$\mathcal{D} := \{(\xi_d(t), u_d(t))\}_{t=1}^{N-1}. \quad (3.5)$$

In the framework of control systems, GPR serves a crucial role by characterizing each control mapping c_i through its mean and kernel functions, defined as $l_i : \mathcal{C} \rightarrow \mathbb{R}$ and $k_i : \mathcal{C} \times \mathcal{C} \rightarrow \mathbb{R}$, respectively, for $i = 1, \dots, m$. We employ a zero-mean Gaussian process with a squared exponential (SE) kernel incorporating automatic relevance determination (ARD) for its broad applicability and flexibility in handling various types of data. The ARD mechanism allows the model to assign different

¹ The subscript **d** denotes data samples obtained through experimental observations of the system.

length scales to each input dimension, effectively learning the relevance of each input variable:

$$k_i(\xi, \xi') = \sigma_i^2 \exp \left(-\frac{1}{2} (\xi - \xi')^T H_i^{-1} (\xi - \xi') \right), \quad (3.6)$$

where σ_i is a signal variance and $H_i = \text{diag}(h_{i,1}^2, \dots, h_{i,2n}^2)$ is a diagonal matrix of length scales $h_{i,j}$ for each input dimension $j = 1, \dots, 2n$, representing the hyperparameters for each i within $i = 1, \dots, m$. These hyperparameters are optimized using the marginal likelihood method.

To utilize GPR for control mapping, we impose the following assumption on the control mappings:

Assumption 3.2 For each i , the control mapping c_i is assumed to belong to a reproducing kernel Hilbert space² (RKHS) $\mathcal{H}_{k_i}$, induced by a corresponding kernel function k_i . $\square$

Utilizing the dataset $\mathcal{D}$ defined in (3.5), GPR allows for the computation of mean and variance functions for any test input ξ within the control domain $\mathcal{C}$:

$$\begin{aligned} \mu_{\mathcal{D}}^i(\xi) &= \mathbf{k}_i^T(\xi) \mathbf{K}_i^{-1} \Upsilon^i, \\ \sigma_{\mathcal{D}}^i(\xi) &= k_i(\xi, \xi) - \mathbf{k}_i^T(\xi) \mathbf{K}_i^{-1} \mathbf{k}_i(\xi), \end{aligned}$$

respectively, where the components are detailed as follows:

$$\begin{aligned} \Upsilon^i &= [u_d^i(1); \dots; u_d^i(N-1)], \\ \mathbf{k}_i(\xi) &= [k_i(\xi_d(1), \xi); \dots; k_i(\xi_d(N-1), \xi)], \\ \mathbf{K}_i &= \begin{bmatrix} k_i(\xi_d(1), \xi_d(1)) & \dots & k_i(\xi_d(1), \xi_d(N-1)) \\ \vdots & \ddots & \vdots \\ k_i(\xi_d(N-1), \xi_d(1)) & \dots & k_i(\xi_d(N-1), \xi_d(N-1)) \end{bmatrix} \end{aligned}$$

for $i = 1, \dots, m$.

This GPR framework supports the design of a MR-GPR controller, integrating the computed mean functions $\mu_{\mathcal{D}}^i$ to create a comprehensive control mapping:

$$u(t) = \mu_{\mathcal{D}}([x(t); y(t)]) := \begin{bmatrix} \mu_{\mathcal{D}}^1([x(t); y(t)]) \\ \mu_{\mathcal{D}}^2([x(t); y(t)]) \\ \vdots \\ \mu_{\mathcal{D}}^m([x(t); y(t)]) \end{bmatrix}. \quad (3.7)$$

² The space $\mathcal{H}_{k_i}$ is a Hilbert space of functions mapping $\mathbb{R}^{2n}$ to $\mathbb{R}$, characterized by the inner product $\langle \cdot, \cdot \rangle_{k_i}$. For every $x \in \mathbb{R}^{2n}$, the function $k_i(\cdot, x)$ resides in $\mathcal{H}_{k_i}$ and satisfies the reproducing property $f(x) = \langle f, k_i(\cdot, x) \rangle_{k_i}$ for all $f \in \mathcal{H}_{k_i}$. The norm induced by this inner product is denoted by $\| \cdot \|_{k_i}$. This Hilbert space $\mathcal{H}_{k_i}$ is uniquely determined by the choice of the kernel function k_i [27].

Furthermore, the posterior variance functions $\sigma_{\mathcal{D}}^i$ are consolidated into a unified variance profile, $\sigma_{\mathcal{D}}$, enhancing the reliability and robustness of the control outputs:

$$\sigma_{\mathcal{D}}([x(t); y(t)]) := \begin{bmatrix} \sigma_{\mathcal{D}}^1([x(t); y(t)]) \\ \sigma_{\mathcal{D}}^2([x(t); y(t)]) \\ \vdots \\ \sigma_{\mathcal{D}}^m([x(t); y(t)]) \end{bmatrix}. \quad (3.8)$$

It is crucial to emphasize that the posterior mean function, represented as $\mu_{\mathcal{D}}$, functions as an approximation or estimate of the control mapping c , and is derived exclusively from the observed input/state data as described in (3.1). These estimates play a pivotal role in the formulation of data-driven control strategies. Concurrently, the reliability of these estimates is quantitatively assessed through the corresponding variance function $\sigma_{\mathcal{D}}$, which serves as a measure of confidence in the estimations provided by the Gaussian process model. This relationship can be understood through the following well-known lemma.

Lemma 3.1 ([28, Corollary 3.11]) *There exists a positive β , such that*

$$\|\mu_{\mathcal{D}}(\xi) - c(\xi)\| \leq \beta \sqrt{\|\sigma_{\mathcal{D}}(\xi)\|}, \quad \forall \xi \in \mathcal{C},$$

where $\mu_{\mathcal{D}}$ is the predictive mean (3.7) and $\sigma_{\mathcal{D}}$ is the predictive variance (3.8) of the GPR. $\square$

To utilize Lemma 3.1, we assume knowledge of an upper bound for β . Additionally, we impose a further assumption on the system's property to facilitate the main theorem.

Assumption 3.3 Given the continuously differentiable property of $f(\cdot, \cdot)$, we define a Lipschitz constant L for the input term. We assume that the value of $\bar{\lambda}$ is known such that $L\beta < \bar{\lambda}$. $\square$

The validity of this assumption is further explored in the remark following the main theorem.

Based on this foundation, we present the main theorem of this paper as follows:

Theorem 3.1 *Consider the discrete-time nonlinear system governed by (3.1), alongside the MR-GPR controller (3.7) as*

$$\mu_{\mathcal{D}}([x(t); y(t)]).$$

Given a non-negative scalar $\lambda > \bar{\lambda}$, a contraction scalar α where $1 > \alpha > 0$, an initial condition $x(0) \in \mathcal{X}$, and the dataset $\mathcal{D}$, suppose there always exists a subsequent possible state $y(t) \in \mathbb{R}^n$ that minimizes the objective function:

$$J(y(t)) := \|y(t)\| + \lambda \sqrt{\|\sigma_{\mathcal{D}}([x(t); y(t)])\|},$$

subject to the condition that

$$J(y(t)) \leq \alpha \|x(t)\|,$$

where $\sigma_{\mathcal{D}}([x; y])$ represents the variance as defined in (3.8) for every time step t . Then, the state $\|x(t)\|$ converges to zero as $t \rightarrow \infty$. $\square$

Proof For brevity, x_t is used instead of $x(t)$ in this proof. We aim to demonstrate through mathematical induction that $\|x_t\| \leq \alpha^t \|x_0\|$ holds for all time t . Starting with the optimal solution y_0 for the optimization problem at x_0 , the application of the MR-GPR controller yields

$$\begin{aligned} \|x_1\| &= \|f(x_0, \mu_{\mathcal{D}}([x_0; y_0]))\| \\ &\leq \|f(x_0, c([x_0; y_0]))\| \\ &\quad + \|f(x_0, \mu_{\mathcal{D}}([x_0; y_0])) - f(x_0, c([x_0; y_0]))\| \\ &\leq \|y_0\| + L\beta\sqrt{\|\sigma_{\mathcal{D}}([x_0; y_0])\|} \\ &\leq \|y_0\| + \lambda\sqrt{\|\sigma_{\mathcal{D}}([x_0; y_0])\|} \\ &\leq \alpha \|x_0\|. \end{aligned}$$

If it holds that $\|x_{k-1}\| \leq \alpha^{k-1} \|x_0\|$, then for x_k :

$$\begin{aligned} \|x_k\| &= \|f(x_{k-1}, \mu_{\mathcal{D}}([x_{k-1}; y_{k-1}]))\| \\ &\leq \|f(x_{k-1}, c([x_{k-1}; y_{k-1}]))\| \\ &\quad + \|f(x_{k-1}, \mu_{\mathcal{D}}([x_{k-1}; y_{k-1}])) - f(x_{k-1}, c([x_{k-1}; y_{k-1}]))\| \\ &\leq \|y_{k-1}\| + L\beta\sqrt{\|\sigma_{\mathcal{D}}([x_{k-1}; y_{k-1}])\|} \\ &\leq \|y_{k-1}\| + \lambda\sqrt{\|\sigma_{\mathcal{D}}([x_{k-1}; y_{k-1}])\|} \\ &\leq \alpha \|x_{k-1}\| \\ &\leq \alpha^k \|x_0\|. \end{aligned}$$

Thus, $\|x_t\| \leq \alpha^t \|x_0\|$ holds for all t , and as t approaches infinity, $\|x_t\|$ converges to zero. $\blacksquare$

Remark 3.1 The assumption regarding the knowledge of the maximum permissible value of $L\beta$ is derived from our analytical understanding of the error boundaries inherent in MR-GPR controller implementations. Initially, the value of λ is set significantly high. When directing toward a target point y , the introduced error is approximately $L\beta\sqrt{\|\sigma_{\mathcal{D}}(\cdot)\|}$, where $\sigma_{\mathcal{D}}(\cdot)$ denotes the computable variance of the Gaussian process. This measurable error metric is essential for determining whether the initially selected λ is adequate or needs adjustment. Specifically, if the empirically calculated

error exceeds the expected threshold under the initial setting of λ , it suggests the necessity to adjust λ upward. Consistently observing errors that surpass expected values prompts a re-assessment and potential increase of λ . This iterative adjustment process facilitates an approximate determination of the maximum permissible λ . $\square$

However, while this empirical approach provides practical guidance for adjusting λ , it does not address the theoretical challenge of knowing the precise values of the constants L and β , which are critical for establishing a rigorous upper bound for λ .

Remark 3.2 Assumption 3.3 presupposes knowledge of an upper bound $\bar{\lambda}$ such that $\bar{\lambda} > L\beta$, with L being the Lipschitz constant of the system dynamics and β related to the RKHS norm. The Lipschitz constant L quantifies the maximum rate at which the output of the system dynamics $f(x, u)$ can change in response to variations in the input u . Specifically, L provides a bound on the sensitivity of the system output to changes in the input, indicating the worst-case scenario for output variation per unit input change. The parameter β measures the discrepancy between the RKHS function μ and the actual control law c . Estimating these constants is a nontrivial task, as discussed in works such as Wood and Zhang [29], Tokmak et al. [30], Fiedler et al. [31]. $\square$

Remark 3.3 The strategic management of the dataset $\mathcal{D}$ in our MR-GPR control system is pivotal for reducing variance, thereby enhancing the precision and reliability of control outputs. Although this paper does not delve into the specifics of this strategy, it is crucial to focus on areas of high nonlinearity or sensitivity within the system's operational space. By targeting regions where variance reduction can significantly improve model accuracy and robustness, we can adapt our control strategy more effectively to the system's dynamics. This approach ensures that our interventions are both precise and tailored to current conditions. The theoretical insights from Lederer et al. [26] and Kanagawa et al. [28, Corollary 3.11] underscore the importance of variance reduction in improving control precision and system stability, suggesting that careful dataset management is essential for data-driven control enhancements. For more detailed strategies on dataset management, we refer the reader to these studies. $\square$

3.4 Illustrative Example

The FitzHugh–Nagumo oscillator, originally developed to describe neuronal dynamics, serves as an illustrative example for demonstrating the capabilities of MR-GPR controller. The system's dynamics, capturing the essential features of excitability and oscillation in neurons, are described by FitzHugh [32], Nagumo et al. [33]:

$$\begin{aligned}\dot{x}_1(t) &= x_1(t) - \frac{x_1^3(t)}{3} - x_2(t) + I, \\ \dot{x}_2(t) &= \frac{x_1(t) - \gamma_1 + \gamma_2 x_2(t)}{\gamma_3},\end{aligned}$$

where x_1 represents the membrane potential, x_2 the recovery variable, and I is treated as the control input $u(t)$ in our study. Parameters $\gamma_1 = 0.7$, $\gamma_2 = 0.8$, and $\gamma_3 = 3.0$ are selected to emulate typical neuronal behaviors.

For the purpose of control and simulation within a digital framework, these equations are discretized using the Euler method, with a time step Δt chosen based on the dynamic requirements and numerical stability considerations. A typical value for Δt in neuronal models is 0.01 s, allowing for an accurate capture of the rapid changes in membrane potential and recovery dynamics. We have also employed a Δt of 0.01 s in our simulations.

The discrete-time version of the FitzHugh–Nagumo model with the control input is thus formulated as

$$\begin{aligned}x_1(t+1) &= x_1(t) + \Delta t \left(x_1(t) - \frac{x_1^3(t)}{3} - x_2(t) + u(t) \right), \\ x_2(t+1) &= x_2(t) + \Delta t \left(\frac{x_1(t) - \gamma_1 + \gamma_2 x_2(t)}{\gamma_3} \right),\end{aligned}$$

where $u(t)$ is the externally applied current, dynamically adjusted at each time step. This setup enables MR-GPR to modulate $u(t)$ based on real-time feedback, aiming to control the oscillatory behavior effectively toward stabilizing the system at the origin. This system satisfies all the conditions outlined in Assumption 3.1.

To understand the control mechanism, we can write the control mapping c for this system as

$$\begin{aligned}c([x(t); x(t+1)]) \\ = \frac{x_1(t+1) - \{x_1(t) + \Delta t(x_1(t) - \frac{x_1^3(t)}{3} - x_2(t))\}}{\Delta t},\end{aligned}\tag{3.9}$$

where

$$x(t) := \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}.$$

In the context of the FitzHugh–Nagumo oscillator, we have employed a data collection strategy that diverges from the traditional single trajectory method, aimed at enriching the dataset for a more comprehensive analysis. This method is designed to enrich the training dataset $\mathcal{D}$ by harnessing a broader range of system dynamics, and is defined as

$$\mathcal{D} = \{(\xi_d^i(0), u_d^i(0))\}_{i=1}^N,$$

where each dataset entry $\xi_d^i(0)$ consists of concatenated states:

$$\xi_d^i(0) = [x_d^i(0); x_d^i(1)],$$

derived from the i th experiment initiated from randomly distributed initial conditions: $x_{1,d}^i(0) \in [-0.5, 0.5]$ and $x_{2,d}^i(0) \in [-0.5, 0.5]$, with

$$x_d^i(0) = [x_{1,d}^i(0); x_{2,d}^i(0)].$$

The control inputs are similarly randomized, with $u_d^i(0) \in [-1, 1]$ for $i = 1, \dots, N$. This method not only adheres to but also extends the theoretical principles detailed in Kim et al. [24, Remark 1]. By utilizing the sampled data $\mathcal{D}$, we conducted an optimization of the hyperparameters for the SE kernel, as delineated in (3.6).

Utilizing this dataset, we construct the variance-informed MR-GPR controller as described in Theorem 3.1, where the control function $u(t)$ is determined as follows:

$$u(t) = \mu_{\mathcal{D}}([x(t); y(t)]),$$

and the parameters are set with $\alpha = 0.999$ and $\lambda = 0.1$. The optimization of the control problem was conducted using the MATLAB[®] *fmincon* function, effectively deploying the model's predictions to govern the oscillator's dynamics and ensure its stability within the targeted operational regime.

In the illustrative example of the FitzHugh–Nagumo oscillator, the variance $\sigma_{\mathcal{D}}$ was calculated across the range $x_1(1) \in [-3, 3]$ and $x_2(1) \in [-3, 3]$ for datasets $\mathcal{D}$ with $N = 200$ and $N = 400$, starting from the initial condition $x(0) = [0.3; 0.3]$. The resulting graphs, shown in Figs. 3.1 and 3.2, illustrate that with $N = 200$, the variance remains high across most of the region except at the center where it is

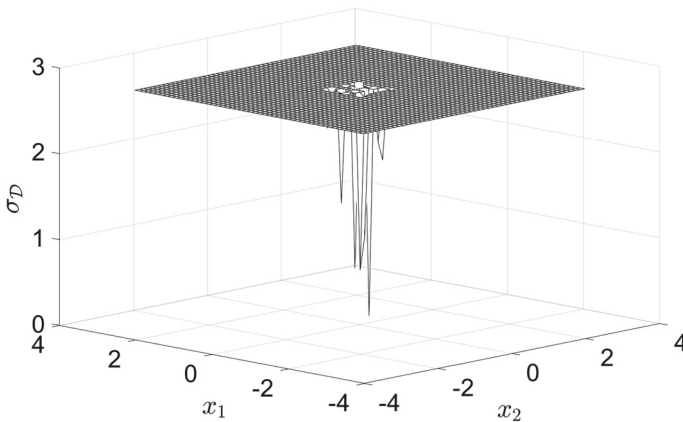


Fig. 3.1 Graph of the values of $\sigma_{\mathcal{D}}$ for each point when the next step moves to $x_1 \in [-3, 3]$ and $x_2 \in [-3, 3]$, starting from the initial condition $(0.3, 0.3)$ with $N = 200$

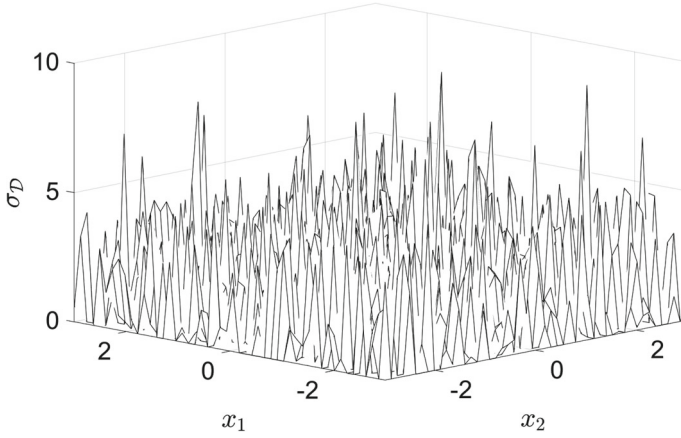


Fig. 3.2 Graph of the values of σ_D for each point when the next step moves to $x_1 \in [-3, 3]$ and $x_2 \in [-3, 3]$, starting from the initial condition $(0.3, 0.3)$ with $N = 400$

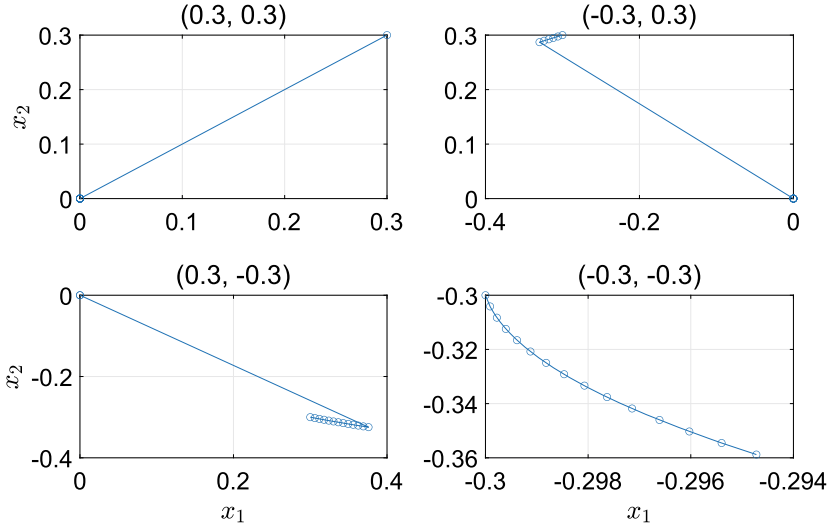


Fig. 3.3 Trajectories of the state variables for each initial condition with $N = 200$

low. However, with $N = 400$, the variance approaches zero throughout most of the region, demonstrating that increased data availability leads to lower variance in the computed control inputs.

Furthermore, the performance of the closed-loop system with the MR-GPR controller was compared under different data volumes, as depicted in Figs. 3.3 and 3.4. This comparison utilized four distinct initial conditions:

$$(x_1(0), x_2(0)) \in \{(0.3, 0.3), (-0.3, 0.3), (0.3, -0.3), (-0.3, -0.3)\}.$$

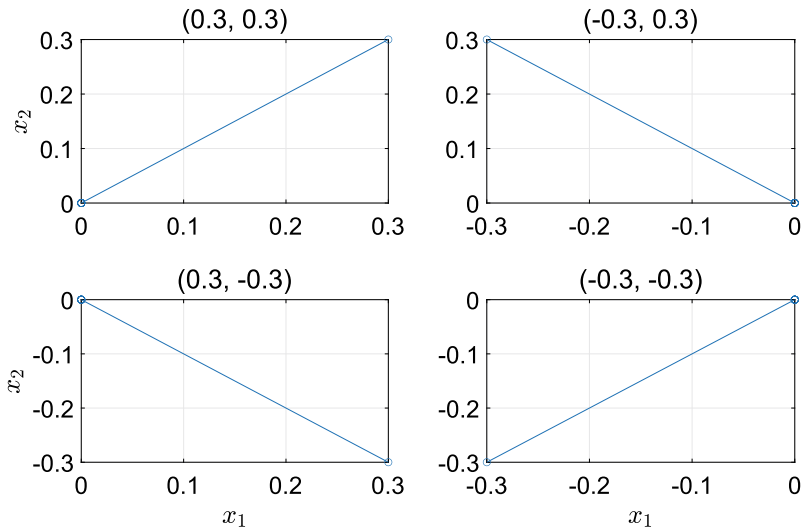


Fig. 3.4 Trajectories of the state variables for each initial condition with $N = 400$

When trained with data from $N = 200$ experiments, Fig. 3.3 shows successful control toward the origin for the conditions $\{(0.3, 0.3), (-0.3, 0.3), (0.3, -0.3)\}$, but a failure in the case of the initial condition $(-0.3, -0.3)$. In contrast, Fig. 3.4, representing $N = 400$ experiments, demonstrates successful control for all tested conditions. These results underscore that a larger dataset is crucial for ensuring stable and robust control performance.

3.5 Conclusion

In this paper, we introduced the variance-informed MR-GPR controller, a data-driven state feedback controller designed for discrete-time nonlinear systems. Unlike traditional approaches, the controller's design leverages GPR that directly utilizes input/state data, not for system identification but specifically for controller construction. This approach enables dynamic adaptation of the controller based on real-time data, significantly enhancing the robustness and effectiveness of the control system.

The potential of the variance-informed MR-GPR controller was demonstrated through targeted simulations on the FitzHugh–Nagumo oscillator model, showing that control performance improves with increased volume and diversity of training data. While our simulations were focused on a single system example, they underscored the critical role of dataset size in improving control accuracy and reducing variance. These results suggest that the variance-informed MR-GPR framework is particularly well suited for applications where operational conditions are variable, and data can be incrementally collected and updated.

Future work will explore the scalability of this approach across a broader range of systems and datasets, including higher dimensional models and real-world applications. Additionally, further research into optimizing data collection strategies could enhance the controller's adaptability and performance under varying operational conditions.

References

1. Kocijan, J.: *Modelling and Control of Dynamic Systems Using Gaussian Process Models*. Springer International Publishing, Cham (2016)
2. Murray-Smith, R., Sbarbaro, D., Rasmussen, C.E., Girard, A.: Adaptive, cautious, predictive control with Gaussian process priors. *IFAC Proc.* **36**(16), 1155–1160 (2003)
3. Jackson, J., Laurenti, L., Frew, E.: Safety verification of unknown dynamical systems via Gaussian process regression. *IEEE Trans. Autom. Control* (2020)
4. Tang, S., Fujimoto, K., Maruta, I.: Learning dynamic systems using Gaussian process regression with analytic ordinary differential equations as prior information. *IEICE Trans. Inform. Syst.* (2021)
5. Xiao, W., Lederer, A., Hirche, S.: Learning stable nonparametric dynamical systems with Gaussian process regression. *IFAC-PapersOnLine* (2020)
6. Kocijan, J., Murray-Smith, R., Rasmussen, C.E., Girard, A.: Gaussian process model based predictive control. In: *Proceedings of American Control Conference*, pp. 2214–2219 (2004)
7. Cao, G., Lai, E., Alam, F.: Gaussian process model predictive control of an unmanned quadrotor. *J. Intell. Robot. Syst.* **88**(1), 147–162 (2017)
8. Azman, K., Kocijan, J.: Dynamical systems identification using Gaussian process models with incorporated local models. *Eng. Appl. Artif. Intell.* (2011)
9. Beckers, T., Hirche, S.: Prediction with approximated Gaussian process dynamical models. *IEEE Trans. Autom. Control* (2021)
10. Chowdhary, G.V., Johnson, E.N.: Theory and flight-test validation of a concurrent-learning adaptive controller. *J. Guid. Control Dyn.* **36**(2), 623–639 (2013)
11. Schreiter, E., Deisenroth, M., Weiss, C.: Safe exploration for active learning with Gaussian processes. In: *European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning* (2015)
12. Chowdhary, G., Kingravi, H.A., How, J.P.: Bayesian nonparametric adaptive control using Gaussian processes. *IEEE Trans. Neural Netw. Learn. Syst.* **25**(1), 114–127 (2014)
13. Joshi, G., Chowdhary, G.: Adaptive control using Gaussian-process with model reference generative network. In: *IEEE Conference on Decision and Control (CDC)* (2018)
14. Solak, E., Murray-Smith, R., Leithead, W.E., Leith, D.J., Rasmussen, C.E.: Derivative observations in Gaussian process models of dynamic systems. In: *NIPS* (2003)
15. Cen, R., Jiang, T., Tang, P.: Modified Gaussian process regression based adaptive control for quadrotors. *Aerosp. Sci. Technol.* **107** (2021)
16. Riihimäki, J., Vehtari, A.: Gaussian processes with monotonicity information. In: *AISTATS* (2010)
17. Damianou, A., Titman, M.K., Lawrence, N.D.: Deep Gaussian processes and variational propagation of uncertainty. In: *AISTATS* (2013)
18. Hensman, J., Fusi, N., Lawrence, N.D.: Gaussian processes for big data. *Uncertainty in Artificial Intelligence* (2013)
19. Titsias, M.K.: Variational learning of inducing variables in sparse Gaussian processes. In: *AISTATS* (2009)
20. Kuss, M., Rasmussen, C.E.: Assessing approximate inference for binary Gaussian process classification. *J. Mach. Learn. Res.* (2005)

21. Garnett, R., Osborne, M.A., Roberts, S.J.: Bayesian optimization for sensor set selection. In: ACM/IEEE International Conference on Information Processing in Sensor Networks (2010)
22. Gramacy, R.B.: Surrogates: Gaussian Process Modeling, Design, and Optimization for the Applied Sciences. CRC Press (2020)
23. Deisenroth, M., Fox, D., Rasmussen, C.E.: Gaussian processes for data-efficient learning in robotics and control. *IEEE Trans. Pattern Anal. Mach. Intell.* **37**(2) (2015)
24. Kim, H., Chang, H., Shim, H.: Model reference Gaussian process regression: data-driven output feedback controller. In: 2023 American Control Conference (ACC) (2023)
25. Kim, H., Chang, H.: Model reference Gaussian process regression: data-driven state feedback controller. *IEEE Access* **11** (2023)
26. Lederer, A., Umlauf, J., Hirche, S.: Posterior variance analysis of Gaussian processes with application to average learning curves (2019). [arXiv:1906.01404](https://arxiv.org/abs/1906.01404)
27. Aronszajn, N.: Theory of reproducing kernels. *Trans. Am. Math. Soc.* **68**(3), 337–404 (1950)
28. Kanagawa, M., Hennig, P., Sejdinovic, D., Sriperumbudur, B.K.: Gaussian processes and kernel methods: a review on connections and equivalences (2018). [arXiv:1807.02582](https://arxiv.org/abs/1807.02582)
29. Wood, G.R., Zhang, B.P.: Estimation of the Lipschitz constant of a function. *J. Glob. Optim.* **8**, 91–103 (1996)
30. Tokmak, A., Fiedler, C., Trimpe, S., Zeilinger, M.N., Köhler, J.: Approximate MPC with kernel-based methods. In: Proceedings of the Conference on Decision and Control (CDC) (2022)
31. Fiedler, C., Menn, J., Kreisköther, L., Trimpe, S.: On safety in safe Bayesian optimization (2024). *arXiv preprint* [arXiv:2403.12948](https://arxiv.org/abs/2403.12948)
32. FitzHugh, R.: Impulses and physiological states in theoretical models of nerve membrane. *Biophys. J.* (1961)
33. Nagumo, J., Arimoto, S., Yoshizawa, S.: An active pulse transmission line simulating nerve axon. In: Proceedings of the IRE (1962)

Chapter 4

Data-Driven Dynamic Model and Model Reference Control of Inverter-Based Resources



Ganesh Marasini[✉], Kenneth McDonald[✉], Zhihua Qu[✉],
and Thomas E. McDermott[✉]

Abstract Accurate and computationally tractable dynamic models of inverter-based resources (IBRs) are necessary for control design and performance evaluation of these devices in the present grid. This paper discusses the data-driven approach to obtain a nonlinear data-driven dynamic model of an IBR. Specifically, a Hammerstein–Wiener (HW) model of a three-phase inverter is obtained using the open-source *dynoNet* application to obtain the parameters of the model. To demonstrate the application of the model, a model reference control (MRC) strategy is proposed for the inverter to achieve the grid-forming objective of reference voltage tracking. The high fidelity simulation results from the test in MATLAB[®]/Simulink[®] environment demonstrate the good performance of the HW model and its efficacy in tracking the reference voltage through MRC strategy.

Keywords Data-driven dynamic model · Inverter-based resources · Model reference control

4.1 Introduction

The usage of power electronic devices such as DC/DC converters and DC/AC inverters for the integration of renewable energy resources in the smart grid is ever increasing due to their flexibility in implementing the associated control algorithms [1]. For successful and efficient integration of these devices, it is essential to analyze their impacts on the performance and stability of the grid, since the operating principle of

This work was supported by the U.S. Department of Energy under Award DE-EE0009028.

G. Marasini · K. McDonald · Z. Qu (✉)
Department of Electrical and Computer Engineering, University of Central Florida,
Orlando, FL, USA
e-mail: qu@ucf.edu; Zhihua.Qu@ucf.edu

T. E. McDermott
Meltran, Charlottesville, VA, USA
e-mail: t.mcdermott@ieee.org

these devices is different from that of the conventional synchronous generators. It is also important to ensure that the controllers designed for these devices contribute to the overall system stability by correctly meeting operational objectives.

To properly design robust control for, and conduct comprehensive system analysis involving power electronic devices, it is crucial to develop an accurate and computationally tractable model. An accurate model should be able to capture the dynamics related to the on/off operation modes of PWM switches, as well as the effects of passive elements such as inductors, capacitors, and resistors. Additionally, the model should account for the influence of various control elements, including dead bands and saturation techniques.

The most accurate modeling approach is to explicitly consider the behavior of the device during each on/off operating modes of the PWM switches [2], where the dynamics of the passive elements are typically represented through state equations. Such a detailed model is able to capture all the switching dynamics accurately. However, it is computationally demanding to use in simulation studies and mathematically complex to use in the controller design. Another approach is the averaged switch modeling technique [3]. In this approach, the waveforms as the result of the switching actions of the converter are averaged over an interval that is short compared to the natural time constant of the system. Such an averaging, if done without significantly altering the system's response, removes the switching harmonics, while preserving the low-frequency components of the waveforms. Eliminating the small but mathematically complicated switching harmonics leads to a relatively simple and tractable converter model. Such a model is useful when only the fundamental component response of the inverter is necessary. In a strong power system network with low penetration of IBRs, the dynamic response of the inverter and the controller design for the inverter are done using the average model. However, the increasing penetration of IBRs in the power system network has resulted in the need to study the broader dynamic response of the inverters and design the controllers accordingly.

Of the more recent approaches, the data-driven approach stands out. The data-driven approach exploits the simulated or experimental input–output data to facilitate the development of a model, either linear or nonlinear, that accurately reflect the dynamics of the system. A transfer function is mostly used to obtain a linear model while the Hammerstein–Wiener (HW) modeling approach can be used to obtain the nonlinear model [4, 5]. Once the structure of the desired model has been determined, various system identification and machine learning methods can be utilized to identify and tune the model parameters. In Guruwacharya et al. [6, 7], transfer function models are obtained for commercial-off-the-shelf inverters. A power hardware-in-the-loop (PHIL) test system consisting of a commercial inverter connected to a grid simulator was used to collect the required data and the model parameters were obtained using the MATLAB[®] system identification toolbox. However, the linear models cannot capture the nonlinear dynamics of the system and as a workaround the nonlinear dynamics can be piece-wise linearized, and a linear model can be obtained for each linearized region. In Patcharaprakiti et al. [8], HW modeling for a single-phase grid-connected photovoltaic inverter has been discussed. A three-phase inverter system has many complexities such as reference frame transformations, coupling between

axes, etc. These complexities are not present in the single-phase inverter system. Since the use of three-phase inverter is ubiquitous in today's grid, it is relevant to explore the nonlinear modeling capabilities for such inverters. There are hardly any works on the literature that have focused on this research gap.

Reference [9] discusses the HW model alternatives, architecture, training, validation, and stability for three-phase IBR. It describes other use cases for continuous time domain, integration with electromagnetic transient (EMT) simulators, and model portability. The HW model is trained with the AC output filter included, which improves accuracy and guarantees that training data are accessible at the inverter's AC terminals. The trained HW dynamic model, like any other inverter dynamic model, can be used for various applications such as accurate and faster simulation studies, controller design, etc. This paper builds on the work in McDermott et al. [9] by removing AC output filter effects from the HW model, which is necessary for the use case of real-time control. We then demonstrate use of the compensated model to improve control performance in tracking the reference voltage.

The rest of the paper is organized as follows: Sect. 4.2 introduces the test system and presents the average modeling and HW modeling approach for the test system. Section 4.3 presents the details on HW modeling of the test system, and the MRC for the test system to achieve reference voltage tracking is presented in Sect. 4.4. Section 4.5 presents and discusses the results, and Sect. 4.6 concludes the paper.

4.2 Test System and IBR Models

Figure 4.1 shows the circuit diagram of the test system considered in this work. The test system consists of photovoltaic (PV) array, DC/DC boost converter with its controller (not shown in Fig. 4.1), DC-link capacitor, and a three-level neutral point clamped (NPC) inverter with LC filter at its output terminals which are connected to local loads. Figure 4.2 shows the simplified single line diagram of the test system; the DC stage is simplified as a controllable voltage source. For a given weather condition, the DC-link voltage is regulated by the controller (not shown in these figures) for the boost converter. R_f , L_f , and C_f are the resistance, inductance, and capacitance of the LC filter. The measurement variables i_s , v_o , i_o , and V_{DC} are the inductor current,

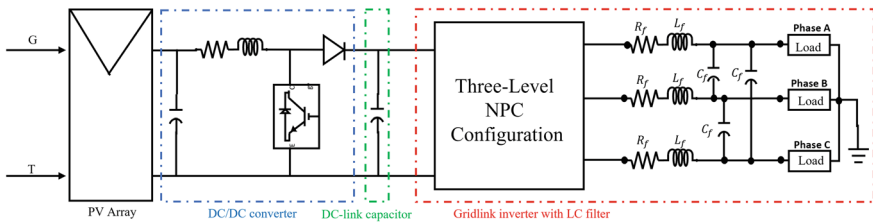
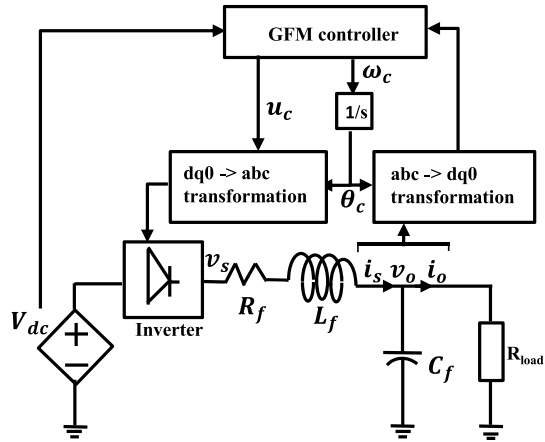


Fig. 4.1 Circuit diagram of the test system

Fig. 4.2 Simplified single line diagram of the test system



output voltage, output current, and DC voltage, respectively. The control variables u_c and ω_c are the modulation index for the PWM operation and control frequency, respectively. The grid-forming (GFM) controller, in this paper, is responsible to generate the control signal u_c so that the inverter tracks the reference voltage. The variable v_s is the fundamental component of the voltage at the PWM terminal of the inverter. It is to be noted that such a complex system (involving the dynamics of PV array, DC/DC converter and its controller, three-level NPC inverter and its PWM, and reference frame transformation) was intentionally chosen to demonstrate the ability of the trained dynamic model to accurately represent a complex system.

Ideally, the detailed model of Fig. 4.1, which represents the inverter dynamics through switching stages of PWM switches and differential equations of LCL filter, is the closest representation of the inverter hardware [2]. However, such a model is not computationally tractable for real-time control and is not in the suitable form to be used in the controller design process. As an alternative, other models are used in the literature for control design and analysis purpose. These models are described below.

4.2.1 Average Model

The simplified dynamic model of the inverter, in synchronous (or $dq0$) reference frame, is given as

$$v_s^{\text{avg}} = \frac{v_{dc}}{2} u_c, \quad \dot{\delta} = \omega - \omega_0, \quad \omega = \omega_c, \quad (4.1)$$

where angle δ represents the deviation of operating/control frequency of the inverter (ω/ω_c) from the nominal frequency (ω_0). When the inverter is connected to power

system network, δ determines the power flow from the inverter into the network. The LCL filter has the following vector-form state-space model in the synchronous reference frame:

$$\frac{dv_o}{dt} = A_{21}i_o + A_{22}v_o + A_{23}i_s, \quad (4.2)$$

$$\frac{di_s}{dt} = A_{32}v_o + A_{33}i_s + B_3u, \quad (4.3)$$

where $i_s = [i_{sd} \ i_{sq}]^\top$, $v_o = [v_{od} \ v_{oq}]^\top$, $i_o = [i_{od} \ i_{oq}]^\top$, $u_c = [u_{cd} \ u_{cq}]^\top$,

$$\begin{aligned} A_{21} &= -\frac{1}{C_f} I_{2 \times 2}, & A_{22} &= \begin{bmatrix} 0 & \omega_c \\ -\omega_c & 0 \end{bmatrix}, \\ A_{32} &= -\frac{1}{L_f} I_{2 \times 2}, & A_{33} &= \begin{bmatrix} -\frac{R_f}{L_f} & \omega_c \\ -\omega_c & -\frac{R_f}{L_f} \end{bmatrix}, \\ B_3 &= -\frac{v_{dc}}{2} A_{32}, & A_{23} &= -A_{21}. \end{aligned}$$

It can be noted from (4.1) that the average model assumes that the PWM terminal voltage has only the fundamental component (v_s), and this fundamental component depends on the modulation index u_c (which is a control variable) and DC voltage. This is because the LC filter that comes after the PWM stage filters out most of the higher order harmonics, and the voltage signal at the output terminal has mostly the fundamental component. However, the PWM terminal voltage is a stair-case signal with multiple harmonics. Even the magnitude of the fundamental component at the PWM terminal would be greater than at the output terminal considering the losses in the inverter-side inductor of the filter. Thus, the average model does not account for the harmonics (the EMT component) of v_s and v_o , and hence is not an accurate model of the test system.

4.2.2 Data-Driven Model

An analytical expression that accurately represents v_s , and is computationally tractable and simple enough for control design, is difficult to obtain. For this reason, this work uses a data-driven approach to obtain such a model. Specifically, a block-oriented open-source neural network architecture, *dynoNet* [10], is chosen to identify the model.

By enabling long-term (or even infinite) time dependencies with a smaller number of parameters, *dynoNet* provides a clear advantage for system identification over traditional 1D convolutional neural networks. Furthermore, by leveraging linear recurrence equations in its LTI blocks, *dynoNet* can process time series data with less computational burden, along with providing the opportunity for parallelization

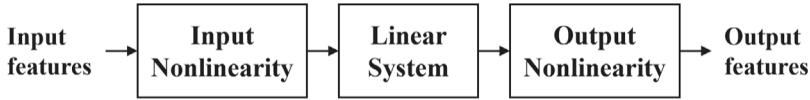


Fig. 4.3 Block diagram of Hammerstein–Weiner model

as compared to traditional recurrent neural networks. Among the various available *dynoNet* architectures, Hammerstein–Weiner model structure is commonly used for system identification purpose [10]. The Hammerstein–Weiner (HW) model structure, as seen in Fig. 4.3, is comprised of an input nonlinearity, followed by a linear dynamic system, and then followed by an output nonlinearity. The input and output nonlinearities are neural network based and can be further characterized by a combination of linear transformation layers and activation functions, such as tanh. The linear system is a stable, rational transfer function. By optimizing the weights and biases of the linear transformation layers embedded in the nonlinearity blocks, along with the transfer function coefficients, one can capture the nonlinear dynamics of the system while maintaining the generic block structure.

Once the HW model structure is adopted, a data-driven modeling approach can be applied to the considered test system. The choices of different components of the HW model shown in Fig. 4.3 are discussed in the next section for the system under study.

4.3 HW Model of the Test System

The test system shown in Fig. 4.1 is implemented in MATLAB®/Simulink® environment. The loads considered at the output of the inverter are linear in nature. The controller for the DC/DC converter has the objective of regulating the DC-link voltage to 600 V if there is sufficient solar irradiance. The details of this controller are not discussed as it is not the focus of this paper. Similarly, during the training data collection, the GFM controller is disabled, and the pre-defined input training features are used to collect the response of the test system. From the control design perspective, such an operation of the test system is known as open-loop operation as the control signal u_c does not depend on the feedback signals.

The data from open-loop control responses of the test system were used to train the *dynoNet* HW model. The input features for the model are (a) irradiance (G), (b) temperature (T), (c) control frequency (f_c), (d) direct axis (d) component of modulation index (u_{cd}), (e) quadrature axis (q) component of modulation index (u_{cq}), (f) d-component of output terminal voltage (v_{od}), (g) q-component of output terminal voltage (v_{oq}), and (h) polynomial feature ($G\sqrt{rms}$). Similarly, the output features are (a) DC-link voltage (V_{dc}), (b) DC current (I_{dc}), (c) d-component of the output current (i_{od}), and (d) q-component of the output current (i_{oq}). The parameters T and G were selected to account for the dependence of PV system on weather conditions, f_c was

chosen to capture the effect of control frequency on the inverter (due to matrices A_{22} and A_{33} in the state-space model). The modulation indices u_{cd} and u_{cq} were chosen as input features to capture the dependence of inverter's output on d - and q -components of modulation index. Similarly, v_{od} and v_{oq} were selected for the model to be able to infer any change in operating conditions at its output terminal (for example, they provide information about the load R_{load} change at the inverter terminal). The polynomial feature GV_{rms} is used to represent the expected output from the inverter, and it also helps in fitting accuracy and model stability during the model training [9]. V_{dc} , I_{dc} , i_{od} , and i_{oq} are usually the signals of interest while analyzing the considered test systems, therefore these signals were chosen as output signals. However, making such choices is an art and could differ depending upon the intended application of the trained model. After the dataset was obtained from the MATLAB[®]/Simulink[®] simulations, the training of the HW model was done in *dynoNet* platform using Python programming language.

The specifics of the HW model are illustrated by Fig. 4.4. In this model, u represents the input feature vector, while matrix W and vector b correspond to the weight and bias parameters of the linear transformation layers within the nonlinearity blocks. The variables y_{i3} , y_{ls} , y_{o3} denote the outputs from the input nonlinearity block, the linear system block, and the output nonlinearity block, respectively. The linear system block is characterized by the transfer function $G(q)$, where $A(q)$ and $B(q)$ are polynomials in the time delay operator q^{-1} (where $q^{-1}u(t) = u(t - 1)$) [10]. We selected a sixth-order numerator and denominator for the system, as higher orders did not yield significant improvements in accuracy. The dimensions of the hidden layers in the input and output nonlinearities were chosen based on observations through trial and error. We started with 20 hidden layers for both and adjusted accordingly, ensuring sufficient capacity to learn intricate patterns while avoiding overfitting. The $\tanh()$ activation function was preferred over ReLU and sigmoid because it outputs values in the range of -1 to 1 , allowing it to capture negative inputs effectively. This choice allowed our model to converge slightly faster compared to other choices. The learning rate was chosen to be 0.001, which is a typical starting point for most deep learning applications. A lower learning rate would significantly increase convergence time whereas a higher rate would risk overshooting the optimal solution. The number of training iterations was carefully chosen considering the order of the transfer function, the complexity of the nonlinearities, and the selected learning rate, ensuring

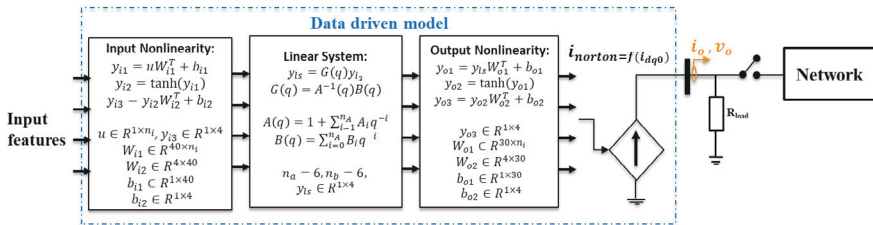
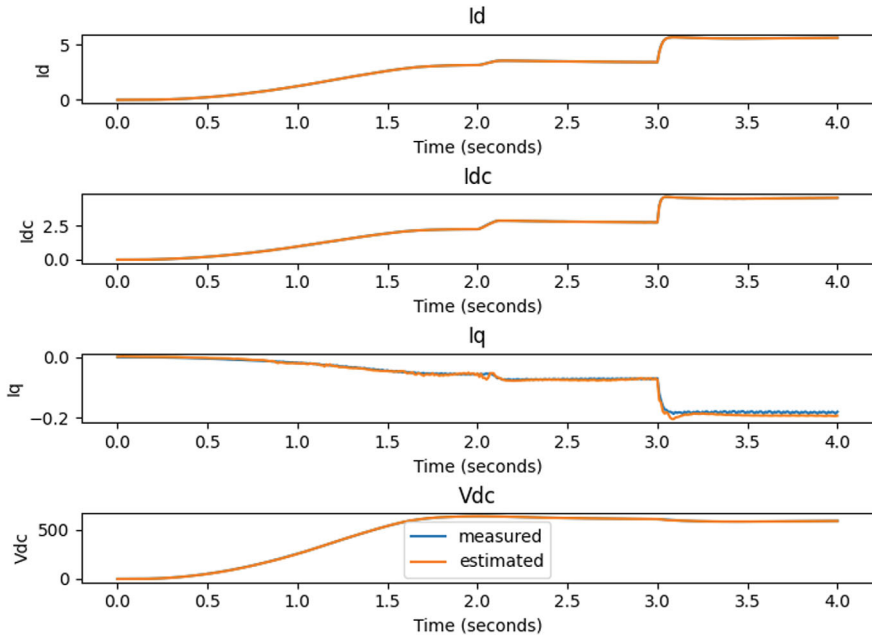


Fig. 4.4 Network interface for the trained data-driven model

Table 4.1 *dynoNet* modeling parameters

Training parameter	Value
$G(q)$ Numerator coefficients n_b	6
$G(q)$ Denominator coefficients n_a	6
Input hidden nodes	40
Output hidden nodes	30
Activation function	tanh
Training iterations	1000
Learning rate	0.001

**Fig. 4.5** Model validation case

efficient convergence while preventing overfitting and underfitting. The details of the training are provided in Table 4.1. The root mean square error (RMSE) values for the model training are [0.0019 0.0021 0.0017 0.0016] for I_d , I_{dc} , I_q , V_{dc} . The performance of the trained model for a validation data is provided in Fig. 4.5.

The trained data-driven model can be interfaced with the power system network using a controllable current source as shown in Fig. 4.4. The outputs i_d and i_q from the HW model (assuming zero component i_0 is 0) can be used to obtain the current phasor signal i_{norton} which drives the network/local load to produce i_o and v_o . Once v_o is established, v_s (if needed) can be reproduced by passing v_o through the inverse

transfer function of the LC filter. In the next section, we demonstrate how this model can be used to design a model-based controller for the IBR to track its reference voltage.

4.4 Model Reference Controls of IBR

The standard approach for reference voltage tracking controller for a three-phase inverter is to design a proportional integral-based cascaded control loop with an inner reference current tracking loop and the outer reference current generation loop [11]. Despite the popularity of this controller due to its simplicity, it has several limitations such as (i) poor performance in unbalanced network conditions and (ii) requirement of anti-windup schemes to saturate and reset the action of the integrators in the outer loop during network fault conditions [12]. Alternatively, recursive feedback controller has been proposed in the literature [13], but they are model-based controllers and assume that the average model of the inverter accurately represents the dynamics of the inverter. However, this is not true in reality as the average model leaves out the harmonic components present in the real hardware circuit. As an improvement, model reference control (MRC) approach can be used which is the subject of discussion in this section.

MRC is a standard model-based feedback control framework [14]. As illustrated in Fig. 4.6(a), MRC calculates the tracking error between the reference model output and the plant output and uses it as the feedback control signal to design a dynamic feedback control under which the closed-loop input–output behavior of the plant matches that of the reference model. One can readily apply MRC to design IBR controls, as shown in Fig. 4.6(b). For instance, by choosing the reference model as

$$G_m(s) = \frac{1}{(s + \gamma_{vo})(s + \gamma_{is})}, \quad \gamma_{is}, \gamma_{vo} > 0,$$

the following GFM control has been developed in Nag et al. [13] to track the reference voltage (v_o^*):

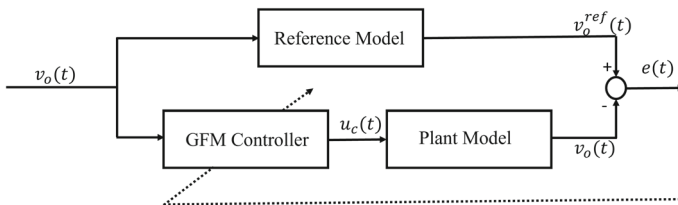


Fig. 4.6 MRC for IBR GFM control

$$u^* = -B_3^{-1} \left[A_{32}v_o + A_{33}i_s + \gamma_{is}(i_s - i_s^*) - \frac{di_s^*}{dt} \right], \quad (4.4)$$

$$i_s^* = -A_{23}^{-1} \left[A_{21}i_o + A_{22}v_o + \gamma_{vo}(v_o - v_o^*) - \frac{dv_o^*}{dt} \right], \quad (4.5)$$

where $\gamma_{vo}, \gamma_{is} > 0$ are control gains and i_s^* is the reference current.

The MRC framework is generally applicable to linear and nonlinear systems, but its drawback is that the plant model must be analytically known. In this paper, we aim to extend MRC by including a data-driven model and their corresponding control design, as described in the following two subsections.

4.4.1 MRC for IBRs with a Data-Driven EMT Model

PWM-controlled inverters do inherently have electromagnetic transients (EMTs) that are not captured by their average dynamic model. In the context of this paper, EMT is defined as those components present in the dynamic response of the detailed model (or accurately trained HW model) but absent in the response of the average model. Using the proposed HW model, one can model inverter dynamics including EMT to greater degree of accuracy. This means that a data-driven model of EMT is obtained. Specifically, the inverter dynamics (Fig. 4.7(a)) is captured by its HW model (Fig. 4.7(b)) and since the IBR output can be decomposed into standard non-switching (i.e., ideal) RMS response described by the average model and an additive EMT response as shown in Fig. 4.7(c), the data-driven EMT model is found to be that depicted in Fig. 4.7(d).

Upon having the data-driven EMT model, the MRC framework can be applied to compensate for EMT, as illustrated in Fig. 4.8. The control structure in Fig. 4.8 consists of two concatenated MRC loops: the outer loop is the same as that in Fig. 4.6(b),

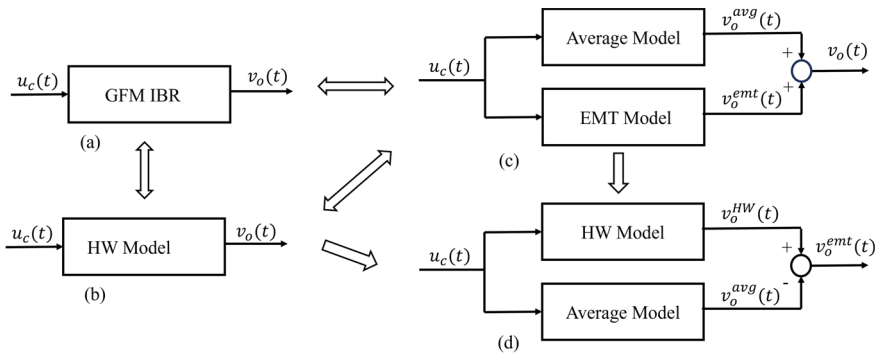


Fig. 4.7 Inverter EMT modeling: **a** inverter; **b** HW model; **c** EMT decomposition; **d** EMT model

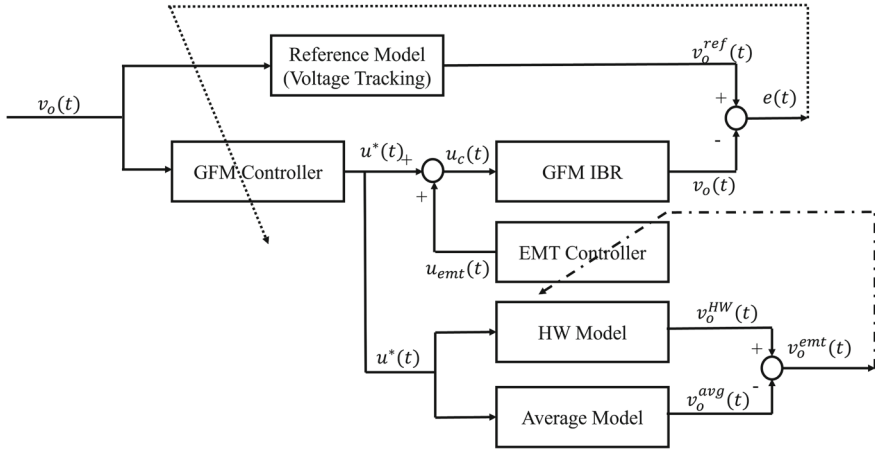


Fig. 4.8 Model reference control for compensating for EMT

while the inner loop is the EMT control loop. There is no reference model explicitly given for the inner loop because it is zero (in the sense that removal of EMT is the control objective).

In the next subsection, the EMT compensation control is explicitly developed and synthesized.

4.4.2 MRC Design to Compensate for EMT

As shown in Fig. 4.7, we know that, around the nominal operating voltage v_o^* and under any control input v_s ,

$$v_o = v_o^{\text{avg}} + v_o^{\text{emt}}. \quad (4.6)$$

Our objective is to design the following MRC:

$$u_c = u^* + u_{\text{emt}}, \quad (4.7)$$

where u^* is the nominal GFM control under which v_o^{avg} converges to v_o^* . The incremental control u_{emt} is to be designed to remove v_o^{emt} from v_o (to the extent possible).

It follows from (4.1), (4.2), and (4.3) that

$$v_o^{\text{avg}} = G_v(s)v_s^{\text{avg}} + G_f(s)i_o = G_v(s)v_s^{\text{avg}} + \frac{1}{Z(s)}G_f(s)v_o^{\text{avg}},$$

or simply

$$v_o^{\text{avg}} = G(s)v_s^{\text{avg}} = \frac{V_{dc}}{2}G(s)u_c, \quad G(s) = \frac{G_v(s)Z(s)}{Z(s) - G_f(s)}, \quad (4.8)$$

where $Z(s)$ is the equivalent impedance of the grid, and the transfer functions $G_v(s)$ and $G_f(s)$ are defined as

$$G_v(s) = H(sI - F)^{-1}G, \quad G_f(s) = (sI - F)^{-1}E. \quad (4.9)$$

$$F = \begin{bmatrix} A_{22} & A_{23} \\ A_{32} & A_{33} \end{bmatrix}, \quad G = \frac{1}{L_f} \begin{bmatrix} 0_{2 \times 2} \\ I_{2 \times 2} \end{bmatrix},$$

$$H = \begin{bmatrix} I_{2 \times 2} & 0_{2 \times 2} \end{bmatrix}, \quad E = -\frac{1}{C_f}H^T.$$

Since the HW model has been trained to represent the actual inverter system, we know from Fig. 4.8 that, under input $u^*(t)$, the HW model predicts well the output voltage of the inverter as

$$\begin{aligned} v_o^{HW} &\simeq v_o \\ &= v_o^{\text{avg}}|_{u=u^*} + v_o^{\text{emt}}|_{u=u^*} \\ &= v_o^* + v_o^{\text{emt}}|_{u=u^*}, \end{aligned}$$

or, simply,

$$v_o^{\text{emt}}|_{u=u^*} \simeq v_o^{HW} - v_o^*. \quad (4.10)$$

It follows from (4.6) and (4.8) that, under MRC (4.7),

$$\begin{aligned} v_o|_{u=u^*+u_{\text{emt}}} &= v_o^{\text{avg}}|_{u=u^*+u_{\text{emt}}} + v_o^{\text{emt}}|_{u=u^*+u_{\text{emt}}} \\ &= \frac{V_{dc}}{2}G(s)(u^* + u_{\text{emt}}) + v_o^{\text{emt}}|_{u=u^*+u_{\text{emt}}} \\ &= v_o^* + \frac{V_{dc}}{2}G(s)u_{\text{emt}} + v_o^{\text{emt}}|_{u=u^*+u_{\text{emt}}} \\ &\simeq v_o^* + \frac{V_{dc}}{2}G(s)u_{\text{emt}} + v_o^{\text{emt}}|_{u=u^*} \\ &\simeq \frac{V_{dc}}{2}G(s)u_{\text{emt}} + v_o^{HW}, \end{aligned}$$

in which (4.10) is applied and the following approximation is made:

$$v_o^{\text{emt}}|_{u=u^*+u_{\text{emt}}} \simeq v_o^{\text{emt}}|_{u=u^*},$$

because v_{emt} is much smaller compared to v_o^* and, due to u_{emt} being much smaller than u^* , the EMT incremental change is neglectable. Given the control objective of making $v_o|_{u=u^*+u_{\text{emt}}}$ converge to output component v_o^{avg} (that is, zeroing out v_o^{emt}), we can choose the following feedback control:

$$u_{\text{emt}} = -k_1 \hat{G}^{-1}(s) \frac{2}{V_{dc}} [v_o^{HW} - k_2 v_o^{\text{avg}}] \quad (4.11)$$

$$= -k_1 [v_s^{HW} - k_2 v_s^{\text{avg}}], \quad (4.12)$$

where k_1 and k_2 are positive control gains, and $\hat{G}^{-1}(s)$ is any causal inverse of transfer function $G(s)$. Expressions (4.11) and (4.12) are mathematically equivalent but, for implementation, control (4.12) is simpler as it does not require information about the grid while v_s^{HW} and v_s^{avg} can be either measured or estimated.

4.5 Results and Discussions

The test system in Fig. 4.1 is implemented in MATLAB®/Simulink®, with a three-level neutral point clamped (NPC) inverter. In this implementation, dead bands are used in the PWM scheme to separate the operation of complementary switches in the same leg. These nonlinear effects are absent in the average model implementation, but they together with PWM switchings contribute to EMT. In addition, the proposed HW model trained in *dynoNet* and the average model are also implemented using MATLAB®/Simulink®.

The following MRC with specific gain choices is implemented:

$$u_{\text{emt}} = -\frac{u_d^*}{v_{s,d}^{HW}} [v_s^{HW} - k_2 v_s^{\text{avg}}], \quad (4.13)$$

where $k_2 = 1$ is the nominal value but can be fine-tuned further. Feedback v_s^{avg} is obtained from (4.1), and v_s^{HW} is estimated by feeding v_o^{HW} through the inverse of LC filter's transfer function. The voltage tracking results and the EMT compensation results are presented in Figs. 4.9 and 4.10, respectively. The entire test duration consists of the following three segments:

- $t \in [0, 1.1)$ second: The solar irradiance ramps up from 0 to 1000 W/M². The GFM control is disabled, and the test system is excited with an open-loop control signal $u_c = [1; 0.001; 0]$. This step is carried out in order to let the trained HW model initialize properly as it has been trained in the similar manner. Figure 4.9 shows a spike in the response of this HW model implementation in MATLAB® at $t = 0$ s, after which it responds accurately. Better initialization procedures are available [9], but were not necessary for this application.
- $t \in [1.1, 4)$: The open-loop control signal is changed to $u_c = [1; 0.001; 0]$. This is done to check how the HW model would behave once the irradiation reaches

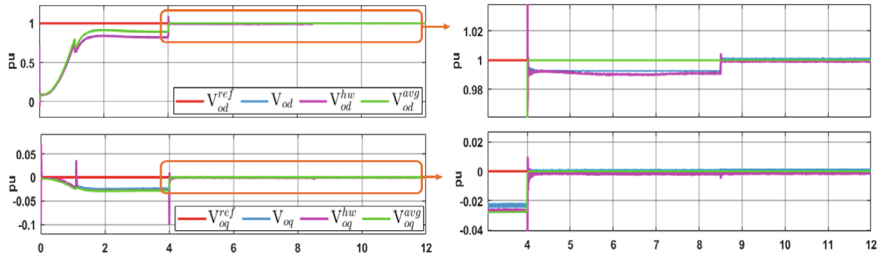


Fig. 4.9 Voltage tracking using model reference control approach

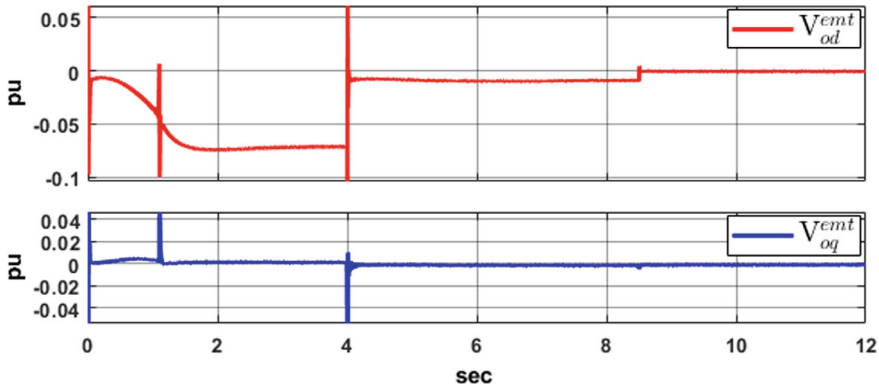


Fig. 4.10 EMT compensation using MRC approach

the steady state. It can be observed from Fig. 4.9 that the HW model is responding more accurately compared to average model.

- $t \in [4, 8.5]$: The nominal GFM control is activated. It can be observed from Fig. 4.9 that the inverter is able to track the reference voltage with a high degree of accuracy even though the average model assumed for the inverter control design does not hold true. This shows that the nominal GFM control designed is already a robust controller. In addition, the perfect voltage tracking would be observed (i.e., the green curves) should the assumption of average model would hold true, and this result also reveals that the performance of the GFM controller could be improved further by accounting for modeling inaccuracies. Upon applying the proposed MRC, it can be observed from Fig. 4.10 that the EMT components of the output voltage are substantially reduced.
- $t \in [8.5, 12]$: The MRC is tuned to one with $k_2 = 1.08$ in (4.13). It can be observed from Fig. 4.9 that the inverter is able to accurately track the reference voltage. The slight deviation of k_2 from one is because the MRC itself has an effect on EMT, which is not captured by the HW model since u^* is used with the HW model as shown in Fig. 4.8. And, this tuning further reduces the EMT components, as shown in Fig. 4.10.

4.6 Conclusion

In this paper, a nonlinear data-driven model is proposed for grid operation of three-phase inverter-based resources (IBRs). The proposed model is a Hammerstein–Wiener (HW) model whose parameters can be easily trained using open-source deep learning platform, *dynoNet*. The proposed model can accurately capture the EMT dynamics present in the detailed switching model of the inverter, and it is computationally comparable to any continuous-time nonlinear dynamic model and hence adequate for real-time control implementation. As an application of the HW model, an MRC is developed for three-phase IBRs to perform reference voltage tracking and also compensate for their own EMT. Our high-fidelity results show that the proposed MRC is very effective in both grid-forming functionality and EMT compensation. Future works include training HW models for different types of IBRs under various loads and grid conditions, and exploring the use of the trained models in studying EMT effects in power networks with a large number of IBRs.

References

1. Solar Energy Technologies Office: Solar integration: inverters and grid services basics. <https://www.energy.gov/eere/solar/solar-integration-inverters-and-grid-services-basics>
2. Gole, A.M., Keri, A., Nwankpa, C., Gunther, E.W., Dommel, H.W., Hassan, I., Marti, J.R., Martinez, J.A., Fehrle, K.G., Tang, L., McGranaghan, M.F., Nayak, O.B., Ribeiro, P.F., Iravani, R., Lasseter, R.: Guidelines for modeling power electronics in electric power engineering applications. *IEEE Trans. Power Deliv.* **12**(1), 505 (1997)
3. Vorperian, V., Tymerski, R., Lee, F.C.Y.: Equivalent circuit models for resonant and PWM switches. *IEEE Trans. Power Electron.* **4**(2), 205–212 (1989)
4. Ljung, L.: *System Identification: Theory for the User*. Prentice Hall (1999)
5. Mathworks, Inc.: System identification toolbox (2024). https://www.mathworks.com/help/ident/index.html?s_tid=CRUX_lftnav. Accessed Aug. 2024
6. Guruwacharya, N., Bhujel, N., Tamrakar, U., Rauniyar, M., Subedi, S., Berg, S.E., Hansen, T.M., Tonkoski, R.: Data-driven power electronic converter modeling for low inertia power system dynamic studies. *IEEE Power Energy Soc. Gen. Meet.* **2020**, 1–5 (2020)
7. Guruwacharya, N., Bhandari, H., Subedi, S., Vasquez-Plaza, J.D., Stoel, M.L., Tamrakar, U., Wilches-Bernal, F., Andrade, F., Hansen, T.M., Tonkoski, R.: Data-driven modeling of commercial photovoltaic inverter dynamics using power hardware-in-the-loop. In: *International Symposium on Power Electronics, Electrical Drives, Automation and Motion (SPEEDAM)*, vol. 2022, pp. 924–929 (2022)
8. Patcharaprakiti, N., Kirtikara, K., Monyakul, V., Chenvidhya, D., Thongpron, J., Sangswang, A., Muenpinij, B.: Modeling of single phase inverter of photovoltaic system using Hammerstein-wiener nonlinear system identification. *Curr. Appl. Phys.* **10**(3, Suppl), S532–S536 (2010)
9. McDermott, T.E., Huang, Q., Liu, Y., Glover, D., Ramesh, M., McDonald, K., Marasini, G., Qu, Z.: Deep learning-enhanced block diagram modeling of solar power systems (2024), accepted to present in 2024 IEEE eGrid Workshop
10. Forgione, M., Piga, D.: *dynoNet*: a neural network architecture for learning dynamical systems. *Int. J. Adapt. Control Signal Process.* **35**(4), 612–626 (2021)

11. Sangwongwanich, A., Abdelhakim, A., Yang, Y., Zhou, K.: Control of single-phase and three-phase DC/AC converters. In: *Control of Power Electronic Converters and Systems*, pp. 153–173. Academic Press (2018)
12. Fan, B., Liu, T., Zhao, F., Wu, H., Wang, X.: A review of current-limiting control of grid-forming inverters under symmetrical disturbances. *IEEE Open J. Power Electron.* **3**, 955–969 (2022)
13. Nag, S., Qu, Z., Xu, Y.: A unified grid-forming and grid-following primary control design with optimized enforcement of grid operational constraints. *IEEE Access* **11**, 57415–57427 (2023)
14. Butler, H.: *Model Reference Adaptive Control: From Theory to Practice*. Prentice Hall (1992)

Chapter 5

Adaptive Tracking MPC for Nonlinear Systems via Online Linear System Identification



Tatiana Strelnikova, Johannes Köhler, and Julian Berberich

Abstract This paper presents an adaptive tracking model predictive control (MPC) scheme to control unknown nonlinear systems based on an adaptively estimated linear model. The model is determined based on linear system identification using a moving window of past measurements, and it serves as a local approximation of the underlying nonlinear dynamics. We prove that the presented scheme ensures practical exponential stability of the (unknown) optimal reachable equilibrium for a given output setpoint. Finally, we apply the proposed scheme in simulation and compare it to an alternative direct data-driven MPC scheme based on the Fundamental Lemma.

Keywords Predictive control · System identification · Nonlinear systems

5.1 Introduction

Designing controllers for unknown systems based on data is a subject of increasing interest in the recent literature. One popular approach is to employ the Fundamental Lemma [25] for parametrizing trajectories using directly measured data. Model predictive control (MPC) is a modern control technique which can handle nonlinear systems, performance criteria, and constraints on system variables [22]. One of the main applications of the Fundamental Lemma is the design of direct data-driven MPC schemes for linear systems [3, 8], see Berberich and Allgöwer [2], Faulwasser et al. [11], Markovsky and Dörfler [19], Markovsky et al. [20], Verheijen et al. [24]

T. Strelnikova
University of Stuttgart, Stuttgart, Germany

J. Köhler
Institute for Dynamic Systems and Control, ETH Zürich, Zürich, Switzerland
e-mail: jkoehle@ethz.ch

J. Berberich (✉)
Institute for Systems Theory and Automatic Control, University of Stuttgart, Stuttgart, Germany
e-mail: julian.berberich@ist.uni-stuttgart.de

for recent survey papers. These MPC schemes even admit desirable closed-loop guarantees in the presence of noise [3].

Alternatively, data can be used in an indirect data-driven approach, determining a model via system identification [17] which can then be employed for model-based MPC. For example, in the field of adaptive MPC, set membership techniques have been employed to robustly control uncertain systems while additionally reducing uncertainty based on online measurements [1, 12, 18, 23]. Despite the surge of research on direct data-driven MPC, the connection between direct and indirect data-driven MPC approaches is still subject to ongoing research [7, 9, 10, 14].

In this paper, we present an indirect data-driven MPC scheme for controlling unknown nonlinear systems based on measured data. The proposed approach uses a moving window of online measurements in order to estimate an approximate local model of the nonlinear system via linear system identification. We prove that the resulting indirect data-driven MPC approach practically exponentially stabilizes the closed-loop system. That is, the closed-loop trajectory exponentially converges to a region around the optimal reachable steady state whose size depends on the initially generated data. The presented approach is inspired by and closely follows the recent works [4, 5]. The main difference is that Berberich et al. [4] use the true linearized dynamics for prediction (i.e., it is a model-based MPC scheme) and Berberich et al. [5] use only input–output data via the Fundamental Lemma [25] for prediction (i.e., it is a direct data-driven MPC scheme). We show that using an indirect data-driven control approach leads to comparable theoretical guarantees as in Berberich et al. [5]. Further, we compare the performance of both approaches in a numerical case study, highlighting individual advantages and drawbacks.

The remainder of the paper is structured as follows. In Sect. 5.2, we present the problem setup as well as the considered system identification approach. Section 5.3 introduces the proposed indirect data-driven MPC scheme, whose theoretical guarantees are proven in Sect. 5.4. Further, in Sect. 5.5, we apply the proposed approach to an example system from the nonlinear MPC literature and compare the outcome to the approaches from Berberich et al. [4, 5]. Finally, Sect. 5.6 concludes the paper with a discussion of advantages and drawbacks of the proposed indirect data-driven MPC scheme in comparison to the direct data-driven approach from Berberich et al. [5].

Notation: For a vector x and a matrix $P = P^\top$, we write $\|x\|_2$ for the Euclidean norm of x and $\|x\|_P := \sqrt{x^\top P x}$ for the weighted norm of x . Moreover, we denote the maximum eigenvalue and minimum singular value of P by $\lambda_{\max}(P)$ and $\sigma_{\min}(P)$, respectively. We write $\mathcal{K}_\infty$ for the set of all continuous functions $\alpha : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ which are strictly increasing, unbounded, and satisfy $\alpha(0) = 0$.

5.2 Preliminaries

In this section, we introduce the problem setup (Sect. 5.2.1) as well as the employed system identification approach (Sect. 5.2.2).

5.2.1 Problem Setup

We consider a discrete-time, control-affine nonlinear system of the form:

$$\begin{aligned} x_{t+1} &= f(x_t, u_t) = f_0(x_t) + Bu_t, \\ y_t &= h(x_t, u_t) = h_0(x_t) + Du_t, \end{aligned} \quad (5.1)$$

with state $x_t \in \mathbb{R}^n$, control input $u_t \in \mathbb{R}^m$, output $y_t \in \mathbb{R}^p$, and time $t \in \mathbb{N}$. The vector fields $f_0 : \mathbb{R}^n \rightarrow \mathbb{R}^n$, $h_0 : \mathbb{R}^n \rightarrow \mathbb{R}^p$ are twice continuously differentiable and $B \in \mathbb{R}^{m \times n}$ and $D \in \mathbb{R}^{p \times m}$ are real matrices. The system is subject to input constraints $u_t \in \mathbb{U}$, $t \in \mathbb{N}$, with a compact set $\mathbb{U} \subseteq \mathbb{R}^m$. The goal is to steer the nonlinear system (5.1) toward a steady state that tracks a constant output reference $y^r \in \mathbb{R}^p$, i.e., $\lim_{t \rightarrow \infty} \|y_t - y^r\|_2 = 0$. The key challenge is that the system dynamics (5.1) are unknown, i.e., f_0 , h_0 , B , and D are not available, and instead we only have access to state and output measurements of the system (5.1).

5.2.2 System Identification

The proposed approach relies on the online identification of an affine model that locally approximates the nonlinear dynamics (5.1). In particular, for a state $\tilde{x} \in \mathbb{R}^n$ and time $t \in \mathbb{N}$, we define the affine dynamics

$$\begin{aligned} x_{t+1} &= f_{\tilde{x}}(x_t, u_t) := A_{\tilde{x}}x_t + Bu_t + e_{\tilde{x}}, \\ y_t &= h_{\tilde{x}}(x_t, u_t) := C_{\tilde{x}}x_t + Du_t + r_{\tilde{x}}, \end{aligned} \quad (5.2)$$

where

$$\begin{aligned} A_{\tilde{x}} &= \left. \frac{\partial f_0}{\partial x} \right|_{\tilde{x}}, \quad e_{\tilde{x}} := f_0(\tilde{x}) - A_{\tilde{x}}\tilde{x}, \\ C_{\tilde{x}} &= \left. \frac{\partial h_0}{\partial x} \right|_{\tilde{x}}, \quad r_{\tilde{x}} := h_0(\tilde{x}) - C_{\tilde{x}}\tilde{x}, \end{aligned}$$

which corresponds to linearizing the nonlinear system (5.1) at $(\tilde{x}, 0)$. We use input–state–output measurements to identify these affine dynamics (5.2) online and, thereby, provide a local approximation of the nonlinear dynamics (5.1). More

precisely, at time t , we estimate the dynamics linearized at x_t using the most recent N measurements of the nonlinear system (5.1), i.e., using the dataset $\mathcal{D}_t := \{x_k\}_{k=t-N}^t, \{u_k, y_k\}_{k=t-N}^{t-1}$.

The main idea is that, if the closed-loop trajectory does not change too rapidly, then $\mathcal{D}_t$ can be approximately explained from the linearized dynamics (5.2). Thus, they can be used to identify the linearized dynamics via least-squares estimation: Given the dataset $\mathcal{D}_t$ at time t , we determine an estimate $(\hat{A}_t, \hat{B}_t, \hat{e}_t)$ of the linearized dynamics (A_{x_t}, B, e_{x_t}) as the minimizer of

$$\min_{\tilde{A}, \tilde{B}, \tilde{e}} \sum_{k=t-N}^{t-1} \|x_{k+1} - \tilde{A}x_k - \tilde{B}u_k - \tilde{e}\|_2^2. \quad (5.3)$$

Likewise, an estimate $(\hat{C}_t, \hat{D}_t, \hat{r}_t)$ of (C_{x_t}, D, r_{x_t}) is determined by solving

$$\min_{\tilde{C}, \tilde{D}, \tilde{r}} \sum_{k=t-N}^{t-1} \|y_k - \tilde{C}x_k - \tilde{D}u_k - \tilde{r}\|_2^2. \quad (5.4)$$

Let us denote

$$\begin{aligned} X_t &:= [x_{t-N}, x_{t-N+1}, \dots, x_{t-1}], & U_t &:= [u_{t-N}, u_{t-N+1}, \dots, u_{t-1}], \\ X_t^+ &:= [x_{t-N+1}, x_{t-N+2}, \dots, x_t], & Y_t &:= [y_{t-N}, y_{t-N+1}, \dots, y_t], \end{aligned}$$

and $Z_t := [X_t^\top U_t^\top \mathbb{I}^\top]^\top$ with $\mathbb{I} = [1 \ 1 \ \dots \ 1]^\top$. If Z_t has full row rank (compare Assumption 5.4), then the minimizers of (5.3) and (5.4) are given by

$$[\hat{A}_t \ \hat{B}_t \ \hat{e}_t] = X_t^+ Z_t^\top (Z_t Z_t^\top)^{-1}, \quad [\hat{C}_t \ \hat{D}_t \ \hat{r}_t] = Y_t Z_t^\top (Z_t Z_t^\top)^{-1}. \quad (5.5)$$

Let us now define the identified dynamics for the given data $\mathcal{D}_t$ as

$$\begin{aligned} x_{k+1} &= \hat{f}_t(x_k, u_k) := \hat{A}_t x_k + \hat{B}_t u_k + \hat{e}_t, \\ y_k &= \hat{h}_t(x_k, u_k) := \hat{C}_t x_k + \hat{D}_t u_k + \hat{r}_t, \quad k \in \mathbb{N}. \end{aligned} \quad (5.6)$$

5.3 Adaptive Tracking MPC for Nonlinear Systems via Linear System Identification

In this section, we present the proposed linear tracking MPC scheme that steers the nonlinear system (5.1) toward a desired target setpoint y^r . This scheme relies on online system identification using least-squares estimation and, as shown in Sect. 5.4, admits desirable closed-loop guarantees.

At time $t \geq N$, the MPC is based on the following optimization problem:

$$\begin{aligned} & \underset{\hat{x}(t), \hat{u}(t), \hat{x}^s(t), \hat{u}^s(t), \hat{y}^s(t)}{\text{minimize}} && J_{\text{MPC}}(\hat{x}(t), \hat{u}(t), \hat{x}^s(t), \hat{u}^s(t), \hat{y}^s(t)), \\ & && \hat{x}^s(t), \hat{u}^s(t), \hat{y}^s(t) \end{aligned} \quad (5.7a)$$

$$\text{subject to } \hat{x}_{k+1}(t) = \hat{A}_t \hat{x}_k(t) + \hat{B}_t \hat{u}_k(t) + \hat{e}_t, \quad (5.7b)$$

$$\hat{x}_0(t) = x_t, \hat{x}_L(t) = \hat{x}^s(t), \quad (5.7c)$$

$$\hat{x}^s(t) = \hat{A}_t \hat{x}^s(t) + \hat{B}_t \hat{u}^s(t) + \hat{e}_t, \quad (5.7d)$$

$$\hat{y}^s(t) = \hat{C}_t \hat{x}^s(t) + \hat{D}_t \hat{u}^s(t) + \hat{r}_t, \quad (5.7e)$$

$$\hat{u}_k(t) \in \mathbb{U}, k = 0, 1, \dots, L-1, \hat{u}^s(t) \in \mathbb{U}^s, \quad (5.7f)$$

where the MPC cost function is defined as

$$\begin{aligned} & J_{\text{MPC}}(\hat{x}(t), \hat{u}(t), \hat{x}^s(t), \hat{u}^s(t), \hat{y}^s(t)) \\ & := \sum_{k=0}^{L-1} \|\hat{x}_k(t) - \hat{x}^s(t)\|_Q^2 + \|\hat{u}_k(t) - \hat{u}^s(t)\|_R^2 + \|\hat{y}^s(t) - y^r\|_S^2, \end{aligned} \quad (5.8)$$

with user-specified, positive definite weight matrices Q, R, S . Here, $\hat{x}(t)$ and $\hat{u}(t)$ denote the state and input trajectories which are predicted at time t over the horizon L . The prediction model (5.7b) relies on the identified dynamics (5.6) which serve as an approximation of the local linearization (5.2) at x_t . Further, Eq. (5.7c) contains initial conditions as well as a terminal equality constraint which ensures that, at the end of the prediction horizon, the predicted state is equal to *some* (artificial) steady-state $\hat{x}^s(t)$, compare [15]. To be precise, $\hat{x}^s(t)$, $\hat{u}^s(t)$, and $\hat{y}^s(t)$ are online optimization variables which, according to (5.7d) and (5.7e), are steady states for the identified dynamics. The cost (5.8) ensures that the predicted trajectory is kept close to the artificial steady state (first and second terms in (5.8)), whose distance to the setpoint y^r is penalized (third term in (5.8)). Online optimization over artificial steady states can substantially increase the region of attraction and allows MPC schemes to cope with online setpoint changes [15]. Finally, Eq. (5.7f) contains the desired input constraints as well as a constraint on the artificial equilibrium input for some convex and compact $\mathbb{U}^s \subset \text{int}(\mathbb{U})$, which is required for a technical argument in the theoretical analysis below.

If $\mathbb{U}, \mathbb{U}^s$ are polytopic, then problem (5.7) is a convex quadratic program which can be solved efficiently. We denote the minimizer of (5.7) by $\hat{x}(t)^*, \hat{u}^*(t), \hat{x}^{s*}(t), \hat{u}^{s*}(t), \hat{y}^{s*}(t)$ and the corresponding optimal cost by $J_{\text{MPC}}^*(x_t, \mathcal{D}_t)$.

Algorithm 5.1 summarizes the proposed MPC scheme. The algorithm consists of two main steps, indicating its *indirect* data-driven control nature: At time t , the last N input–state–output measurements are used to estimate an affine prediction model (5.2) which serves as a local approximation of the nonlinear dynamics (5.1). Next, the MPC problem (5.7) is solved and the first n steps of the optimal input trajectory are applied to the nonlinear system. We consider such a multi-step imple-

mentation to ensure recursive feasibility despite the model approximation error and the terminal equality constraints, see Berberich et al. [6].

Algorithm 5.1 Adaptive tracking MPC scheme

Offline: Choose $Q, R, S, \mathbb{U}, L, y^r$, and generate initial data trajectory $\mathcal{D}_N$ of length N .

1. At time t and given data $\mathcal{D}_t$, compute $\hat{A}_t, \hat{B}_t, \hat{e}_t, \hat{C}_t, \hat{D}_t, \hat{r}_t$ as in (5.5).
 2. Solve problem (5.7) and apply $u_{t+k} = \hat{u}_k^*(t), k = 0, 1, \dots, n-1$, over the next n time steps.
 3. Set $t = t + n$ and go back to 1.
-

We note that Algorithm 5.1 is analogous to the MPC approach in Berberich et al. [4] with the main difference that, instead of the true linearized dynamics, a data-driven estimate thereof is used for prediction. This makes the proposed approach an indirect data-driven control method. Alternatively, Berberich et al. [5] employ an analogous MPC scheme using the Fundamental Lemma [25] for prediction, which leads to a direct data-driven control method. In Sect. 5.6, we discuss differences of the proposed approach to Berberich et al. [5], in particular, advantages and drawbacks.

5.4 Theoretical Analysis

In this section, we show that Algorithm 5.1 ensures that the closed-loop system practically exponentially stabilizes the (unknown) steady state that tracks the desired reference y^r . The main idea is analogous to Berberich et al. [4, 5]: If the cost matrix S is sufficiently small and the current state x_t is close to the steady-state manifold, then minimization of the cost (5.8) implies that the optimal artificial steady-state $\hat{x}^{**}(t)$ is close to x_t . In particular, the entire predicted trajectory $\hat{x}(t)$ remains in a small region around x_t , where the dynamics linearized at x_t (5.2) are a good approximation of the nonlinear dynamics (5.1). Similarly, in closed-loop operation, the past data trajectory $\mathcal{D}_t$ varies within a small region around x_t and, therefore, the least-squares estimate used for prediction is a close approximation of the linearized dynamics. In summary, a small choice of S ensures that the closed loop moves toward y^r *slowly enough* such that the identified linear model is a sufficiently close approximation of the nonlinear dynamics.

In the remainder of this section, we will make this argument more precise by proving practical exponential stability under the proposed MPC scheme. In Sect. 5.4.1, we first introduce technical assumptions and definitions concerning the system dynamics as well as the available data that are required for the theoretical results. We note that these assumptions are analogous to the assumptions made on model-based and data-driven linearization-based MPC for nonlinear systems in Berberich et al. [4, 5]. Next, in Sect. 5.4.2, we present our main stability result.

5.4.1 Technical Assumptions and Definitions

The steady-state manifold of (5.1) is given by

$$\mathcal{Z}^s := \{(x^s, u^s) \in \mathbb{R}^n \times \mathbb{U}^s \mid x^s = f(x^s, u^s)\}. \quad (5.9)$$

We define the projection on the output as

$$\mathcal{Z}_y^s := \{y^s \in \mathbb{R}^p \mid \exists (x^s, u^s) \in \mathcal{Z}^s : y^s = h(x^s, u^s)\}. \quad (5.10)$$

The best possible behavior we can achieve in closed loop is to steer the system to the *optimal reachable equilibrium* which is defined as the minimizer y^{sr} of

$$J^* := \min_{y^s \in \mathcal{Z}_y^s} \|y^s - y^r\|_S^2. \quad (5.11)$$

We denote the corresponding steady state, which we assume is unique (cf. Assumption 5.1), by x^{sr} . Since the proposed approach relies on least-squares estimation of the linearized dynamics, we also define the corresponding elements for the identified dynamics. To be precise, for data $\mathcal{D}_t$, we define the steady-state manifold for the identified system as

$$\begin{aligned} \hat{\mathcal{Z}}^s(\mathcal{D}_t) &:= \{(x^s, u^s) \in \mathbb{R}^n \times \mathbb{U}^s \mid x^s = \hat{A}_t x^s + \hat{B}_t u^s + \hat{e}_t\}, \\ \hat{\mathcal{Z}}_y^s(\mathcal{D}_t) &:= \{y^s \in \mathbb{R}^p \mid \exists (x^s, u^s) \in \hat{\mathcal{Z}}^s(\mathcal{D}_t) : y^s = \hat{C}_t x^s + \hat{D}_t u^s + \hat{r}_t\}. \end{aligned}$$

Moreover, we also define the optimal reachable equilibrium w.r.t. the identified dynamics as the minimizer $\hat{y}^{\text{sr}}(\mathcal{D}_t)$ of

$$\hat{J}^*(\mathcal{D}_t) := \min_{y^s \in \hat{\mathcal{Z}}_y^s(\mathcal{D}_t)} \|y^s - y^r\|_S^2. \quad (5.12)$$

To ensure the existence and uniqueness of the optimal reachable equilibrium, we make following assumption.

Assumption 5.1 There exists $\sigma_s > 0$, such that for any $t \geq N$, it holds that $\sigma_{\min} \left(\begin{bmatrix} \hat{A}_t - I & \hat{B}_t \\ \hat{C}_t & \hat{D}_t \end{bmatrix} \right) \geq \sigma_s$ and the matrix $\begin{bmatrix} \hat{A}_t - I & \hat{B}_t \\ \hat{C}_t & \hat{D}_t \end{bmatrix}$ has full column rank.

Assumption 5.1 is a common assumption in tracking MPC [16, Remark 1], [22, Lemma 1.8]. It means that, for any output steady state for the identified dynamics, there exists a unique state–input pair which is also a corresponding steady state. More precisely, there exist a unique affine map $g_{\mathcal{D}_t} : \hat{\mathcal{Z}}_y^s(\mathcal{D}_t) \rightarrow \hat{\mathcal{Z}}^s(\mathcal{D}_t)$ such that $\hat{y}^s \mapsto (\hat{x}^s, \hat{u}^s)$, where $\hat{y}^s = \hat{h}_t(\hat{x}^s, \hat{u}^s)$. Further, $g_{\mathcal{D}_t}$ is uniformly Lipschitz continuous for all $t \geq N$. We write $(\hat{x}^{\text{sr}}(\mathcal{D}_t), \hat{u}^{\text{sr}}(\mathcal{D}_t)) := g_{\mathcal{D}_t}(\hat{y}^{\text{sr}}(\mathcal{D}_t))$ for the input and state

corresponding to the optimal reachable output $\hat{y}^{\text{sr}}$. Moreover, analogous to Berberich et al. [4, 5], we make the following technical assumptions.

- Assumption 5.2** (a) The pair $(\hat{A}_t, \hat{B}_t)$ is uniformly controllable for all $t \geq N$, i.e., the minimum singular value of $[\hat{B}_t \dots \hat{A}_t^{n-1} \hat{B}_t]$ is uniformly bounded.
 (b) There exists $\sigma_l > 0$, such that $\sigma_{\min}(I - \hat{A}_t) \geq \sigma_l$ for all $t \geq N$.
 (c) There exists a compact set $\mathcal{B}$ such that $\hat{\mathcal{Z}}^s(\mathcal{D}_t) \subseteq \mathcal{B}$ for all t .

Assumption 5.2(a) is required to construct the candidate solution in the proof of our main result. Moreover, Assumption 5.2(b) implies non-singularity of the dynamics in the sense that, for any steady-state input, there exists a unique corresponding state component. Finally, Assumption 5.2(c) ensures compactness of the steady-state manifold. We refer to Berberich et al. [4, Sect. II.B] for a detailed discussion of the above assumptions and possibilities to satisfy or relax them.

Assumption 5.3 There exist constants $c_l, c_u > 0$ such that, for any $t \in \mathbb{N}$, it holds that

$$c_l \|x_t - \hat{x}^{\text{sr}}(\mathcal{D}_t)\|_2^2 \leq \|x_t - x^{\text{sr}}\|_2^2 \leq c_u \|x_t - \hat{x}^{\text{sr}}(\mathcal{D}_t)\|_2^2. \quad (5.13)$$

Assumption 5.3 relates the optimal reachable steady states of the nonlinear and the identified dynamics. In Berberich et al. [4], it is shown that the assumption holds when replacing $\hat{x}^{\text{sr}}(\mathcal{D}_t)$ by the optimal reachable steady state for the linearized dynamics (5.2), assuming that y^r is reachable and $m = p$. The latter distinction between linearized and identified dynamics does not pose problems for our purposes since the following theoretical analysis ensures in closed loop that the model identified at time t is close to the true linearization at x_t (modulo a small approximation error that can be handled using a robustness argument). Similarly, in closed-loop operation, Assumptions 5.1 and 5.2 are implied by the corresponding assumptions for the true linearized dynamics from Berberich et al. [4].

Finally, we assume persistence of excitation (PE) of the closed-loop trajectories generated via Algorithm 5.1.

Assumption 5.4 There exists $\sigma_Z > 0$ such that $\sigma_{\min}(Z_t) \geq \sigma_Z$ and Z_t has full row rank for all $t \in \mathbb{N}$.

While all previous assumptions are satisfied under realistic conditions [4], Assumption 5.4 is not guaranteed a priori under the proposed control approach. Although a complete theoretical solution to ensuring closed-loop PE remains an open problem, there are various effective heuristics that yield good practical results, e.g., adding a PE disturbance signal to the input, stopping the data updates once the system is close to the setpoint, or adding a cost term which encourages PE trajectories [18].

5.4.2 Practical Exponential Stability

The following result shows that, under the above assumptions, the proposed MPC scheme practically exponentially stabilizes the optimal reachable steady state x^{sr} for the nonlinear system (5.1) in closed loop. The proof relies on the Lyapunov function candidate

$$V(x_t, \mathcal{D}_t) := J_{\text{MPC}}^*(x_t, \mathcal{D}_t) - \hat{J}^*(\mathcal{D}_t). \quad (5.14)$$

Theorem 5.1 *Suppose $t \geq N \geq n$ and Assumptions 5.1–5.4 hold. Then, there exists $\bar{\varepsilon} > 0$ such that, for any $\varepsilon \in [0, \bar{\varepsilon}]$, there exist $V_{\max}, s_u, C > 0$ and $0 < c_V < 1$, $\beta \in \mathcal{K}_{\infty}$, such that, if*

$$\lambda_{\max}(S) < s_u, V(x_N, \mathcal{D}_N) \leq V_{\max} \quad (5.15)$$

$$\|x_N - x_k\|_2^2 \leq \varepsilon \text{ for all } k = 0, \dots, N-1, \quad (5.16)$$

then, for any $t = N + ni$, $i \geq 0$, the problem (5.7) is feasible and the closed loop under Algorithm 5.1 satisfies

$$\|x_t - x^{\text{sr}}\|_2^2 \leq c_V^i C \|x_0 - x^{\text{sr}}\|_2^2 + \beta(\varepsilon). \quad (5.17)$$

Proof The proof follows similar arguments as Berberich et al. [4, Theorem 1] and [5, Theorem 2], and we only provide a sketch in the following. For simplicity of exposition, we assume that all state values considered in this proof lie within a compact set X . This can be ensured using an invariance argument based on the employed Lyapunov function, see the proofs of Berberich et al. [4, Theorem 1], [5, Theorem 2] for details.

(1) System identification bound

The main difference to Berberich et al. [4, Theorem 1] is that the proposed MPC scheme relies on the identified dynamics (5.6) rather than the linearized dynamics (5.2). Therefore, we first derive a bound on the difference between the two. To this end, we write the nonlinear dynamics applied to the past data points $k = t - N, \dots, t - 1$ as

$$f(x_k, u_k) = f_{x_t}(x_k, u_k) + \Delta_{t,k}, \quad (5.18)$$

$f_{x_t}(x_k, u_k)$ as in (5.2) and with some $\Delta_{t,k}$ which can be bounded using a Taylor series argument as

$$\|\Delta_{t,k}\|_2 \leq c_{\Delta,1} \|x_t - x_k\|_2^2, \quad (5.19)$$

for some $c_{\Delta,1} > 0$, see Berberich et al. [5, Lemma 1]. In view of (5.18), we can interpret the process of estimating f_{x_t} from data of the nonlinear system f as a *perturbed* identification problem with nominal affine dynamics f_{x_t} and disturbance

$\Delta_{t,k}$. This allows us to use standard error bounds for least-squares estimation [26]: Defining $\Delta_{[t]} := [\Delta_{t,t-N} \ \Delta_{t,t-N+1} \ \dots \ \Delta_{t,t-1}]$, we obtain

$$\begin{aligned} & [\hat{A}_t \ \hat{B}_t \ \hat{e}_t] - [A_{x_t} \ B \ e_{x_t}] \\ &= X_t^+ Z_t^\top (Z_t Z_t^\top)^{-1} - (X_t^+ - \Delta_{[t]})(Z_t^\top (Z_t Z_t^\top)^{-1} - \Delta_{[t]} Z_t^\top (Z_t Z_t^\top)^{-1}). \end{aligned} \quad (5.20)$$

Thus, using (5.19), the PE bound in Assumption 5.4, as well as compactness of X and $\mathbb{U}$, we obtain

$$\|\hat{f}_t(x, u) - f_{x_t}(x, u)\|_2 \leq c_{\Delta,2} \|\Delta_{[t]}\|_2 \leq c_{\Delta,3} \sum_{k=t-N}^{t-1} \|x_t - x_k\|_2^2, \quad (5.21)$$

for any $x \in X$, $u \in \mathbb{U}$ and with some $c_{\Delta,2}, c_{\Delta,3} > 0$. Analogously, it can be shown that

$$\|\hat{h}_t(x, u) - h_{x_t}(x, u)\|_2 \leq c_{\Delta,4} \sum_{k=t-N}^{t-1} \|x_t - x_k\|_2^2, \quad (5.22)$$

for any $x \in X$, $u \in \mathbb{U}$ and with some $c_{\Delta,4} > 0$.

(2) Lyapunov function bounds

Suppose now that the bound (5.16) holds and let $t = N$. This implies that the error between the linearized dynamics (5.2) and the identified dynamics (5.6) is bounded as

$$\begin{aligned} \|\hat{f}_t(x, u) - f_{x_t}(x, u)\|_2 &\leq c_f \varepsilon, \\ \|\hat{h}_t(x, u) - h_{x_t}(x, u)\|_2 &\leq c_h \varepsilon, \end{aligned} \quad (5.23)$$

for some $c_f, c_h > 0$, compare (5.21) and (5.22). Using an inherent robustness argument to cope with the identification error (5.23), compare [6], a straightforward adaptation of the proof of Berberich et al. [4, Theorem 1] yields the following: If ε is sufficiently small, $V(x_t, \mathcal{D}_t) \leq V_{\max}$, $\hat{J}^*(\mathcal{D}_t) \leq J_{\max}$, $\hat{J}^*(\mathcal{D}_{t+n}^\top) \leq J_{\max}$ for some $J_{\max} > 0$, then the problem (5.7) is feasible at time $t + n$ and there exist $0 < c_V < 1$, $c_\varepsilon > 0$, such that

$$V(x_{t+n}, \mathcal{D}_{t+n}) \leq c_V V(x_t, \mathcal{D}_t) + c_\varepsilon \varepsilon \sqrt{V_{\max} + J_{\max}}. \quad (5.24)$$

Using similar ideas as in the proof of Berberich et al. [5, Theorem 2], it can be shown that a variation of the bound (5.16) holds recursively. To be precise, if

$$\sum_{k=j}^{j+n} \|x_t - x_k\|_2^2 \leq (V_{\max} + J_{\max})^\alpha, \quad (5.25)$$

for $j = t - N, \dots, t - n$, then

$$\sum_{k=t}^{t+n} \|x_t - x_k\|_2^2 \leq (V_{\max} + J_{\max})^\alpha, \quad (5.26)$$

with some $\alpha \in (0, 1)$. As a result, the bound (5.16) holds recursively as well in the sense that

$$\|x_{t+n} - x_k\|_2^2 \leq c_\varepsilon (V_{\max} + J_{\max})^\alpha, \quad (5.27)$$

for some $c_\varepsilon > 0$ and for $k = t + n - N, \dots, t + n - 1$. Choose now $\varepsilon = (V_{\max} + J_{\max})^\alpha$ and note that $J_{\max}$ can be made arbitrarily small when $\lambda_{\max}(S)$ is sufficiently small. Then, Eq. (5.24) implies

$$V(x_{t+n}, \mathcal{D}_{t+n}) \leq c_V V(x_t, \mathcal{D}_t) + c_\varepsilon (V_{\max} + J_{\max})^{\alpha+1/2}. \quad (5.28)$$

Moreover, choose $J_{\max} = V_{\max}$, and $\alpha > 1/2$. Then, if $V_{\max}$ is sufficiently small, we infer $V(x_{t+n}, \mathcal{D}_{t+n}) \leq V_{\max}$ such that the above bounds hold for $t = N + n$ and, therefore, they hold recursively for all $t = N + ni, i \in \mathbb{N}$.

Finally, in analogy to Berberich et al. [4, Lemma 1], suitable lower and upper bounds on the Lyapunov function $V(x_t, \mathcal{D}_t)$ can be obtained. It now follows from standard Lyapunov arguments together with Assumption 5.3 that x^{sr} is practically exponentially stable. $\square$

Theorem 5.1 shows that the optimal reachable steady state x^{sr} of the nonlinear system (5.1) is practically exponentially stable w.r.t. ε in closed loop. This means that the closed-loop trajectory exponentially converges to a region around x^{sr} whose size increases with the bound ε on the distance between the initial data points, compare (5.16). Analogous to Berberich et al. [5], the main proof idea is that, if the latter distance is small initially, then it remains small in closed loop such that the estimated linear model serves as a good local approximation of the nonlinear dynamics. Beyond the technical assumptions from Sect. 5.4.1, the result requires (5.15), meaning that the matrix S is sufficiently small and that the Lyapunov function candidate $V(x_N, \mathcal{D}_N)$ is sufficiently small, which can be fulfilled if the initial state x_0 is sufficiently close to the steady-state manifold.

Theorem 5.1 should be viewed as a qualitative theoretical result, providing insights into the interplay of different system, design, and data parameters as well as the associated closed-loop guarantees. Rigorously verifying all the associated conditions, e.g., computing bounds $s_u, V_{\max}$ as required in (5.15), is challenging in general and poses an interesting direction for future research.

5.5 Numerical Example

In this chapter, we present numerical simulations for the proposed MPC algorithm and compare it to the ones presented in Berberich et al. [4, 5]. The code for the following results is available on <https://github.com/tastr/IdentificationBasedMPC>.

The proposed MPC scheme is applied to the nonlinear continuous stirred tank reactor from Mayne et al. [21], which was also considered in Berberich et al. [4, 5] and is given by

$$f(x, u) := \begin{pmatrix} x_1 + \frac{T_s}{\theta}(1 - x_1) - T_s \bar{k} e^{-\frac{M}{x_2}} \\ x_2 + \frac{T_s}{\theta}(x_f - x_2) + T_s \bar{k} x_1 e^{-\frac{M}{x_2}} - T_s \alpha u (x_2 - x_c) \end{pmatrix}, \quad (5.29)$$

$$h(x, u) := x_2. \quad (5.30)$$

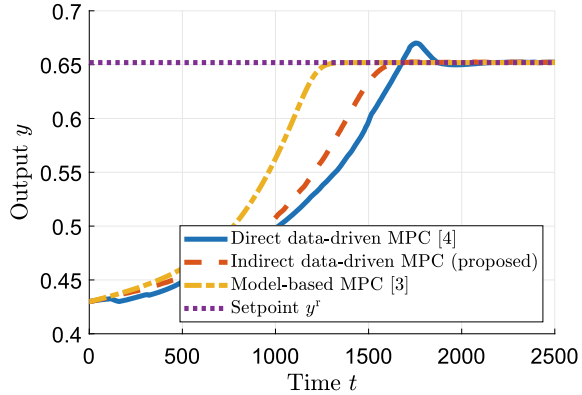
The states x_1 and $x_2 = y$ represent temperature and concentration. The input u is the coolant flow rate. The system results from a Euler discretization of the continuous time dynamics with time step $T_s = 0.2$. The values of the other parameters are $\theta = 20$, $\bar{k} = 300$, $M = 5$, $x_f = 0.3947$, $x_c = 0.3816$, $\alpha = 0.117$. We choose the input constraint sets $\mathbb{U} = [0.1, 2]$, $\mathbb{U}^s = [0.11, 1.99]$, the weighting matrices $Q = I$ and $R = 0.05$, and the prediction horizon $L = 41$. As the above system is not input affine, we define a new input $\Delta u_k := u_{k+1} - u_k$ and treat u as an additional state. The control goal is to steer the output to the setpoint $y^r = 0.6519$ when starting from the initial condition $x_0 = [0.4 \ 0.6 \ 0.1]^\top$. For better practical reliability, we add a regularization to the identification approach introduced in Sect. 5.2.2, i.e., instead of (5.5) we compute the estimated quantities as

$$[\hat{A}_t \ \hat{B}_t \ \hat{e}_t] = X_t^+ Z_t^\top (Z_t Z_t^\top + \lambda I)^{-1}, \quad [\hat{C}_t \ \hat{D}_t \ \hat{r}_t] = Y_t Z_t^\top (Z_t Z_t^\top + \lambda I)^{-1}, \quad (5.31)$$

with regularization parameter $\lambda = 10^{-12}$. Extending our theoretical results to such a regularized least-squares estimate is an interesting direction for future research. As discussed in Berberich et al. [4], Assumptions 5.1–5.3 are fulfilled for the above system. In order to achieve closed-loop PE (Assumption 5.4), we stop the identification update once the state difference between two time steps is smaller than $5 \cdot 10^{-6}$.

Figure 5.1 shows the simulation results for the proposed indirect data-driven MPC scheme as well as the ones from Berberich et al. [4] (model based) and Berberich et al. [5] (direct data driven). For the model-based and the indirect data-driven approach, the weight matrix S is set to $S = 100$, whereas for the direct data-driven approach it is $S = 10$. The reason is that the latter includes a slack variable which encourages faster convergence, hence necessitating a smaller choice of S to avoid too fast changes in the state evolution. Further, we note that, for the direct and indirect data-driven approaches, the initial dataset was generated based on the model-based approach in order to simplify the comparison.

Fig. 5.1 Closed-loop simulations under the MPC schemes from Berberich et al. [4] (model-based MPC) and Berberich et al. [5] (direct data-driven MPC) as well as the approach proposed in the present paper (indirect data-driven MPC)



While the closed loop converges to the desired value for all three algorithms, we observe that the smoothest trajectory is generated by the model-based approach, followed by the identification based. The direct data-driven approach shows slightly slower convergence (which is partly due to the smaller value for S) as well as overshooting behavior. Moreover, the data length N needed for successful tracking in the direct data-driven MPC scheme from Berberich et al. [5] is substantially larger than in the present indirect approach. To be precise, while we use $N = 25$ data points for the proposed approach, the one from Berberich et al. [5] requires a theoretical lower bound of $N = 90$ data points and, for the shown simulations, uses $N = 120$ data points.

Further, we compare the sensitivity to hyperparameters for the indirect and direct data-driven MPC schemes by comparing the tracking error

$$\sum_{t=0}^{2500} \|y_t - y^r\|_2, \quad (5.32)$$

for different parameters. Figures 5.2 and 5.3 show the dependency of the error (5.32) on the regularization parameters and the data length N for both approaches. The white cells denote simulations with numerical issues (in particular, infeasibility of the optimization). It can be seen that the indirect approach has significantly improved robustness against parameter variations compared to the direct approach. In particular, while the direct data-driven MPC scheme from Berberich et al. [5] requires a data length (roughly) below $N = 150$ to avoid a too high linearization error, the proposed indirect approach can cope with substantially higher values up to $N = 300$. Similarly, the range of the regularization parameters leading to satisfactory performance is significantly higher in the indirect approach compared to the direct one.

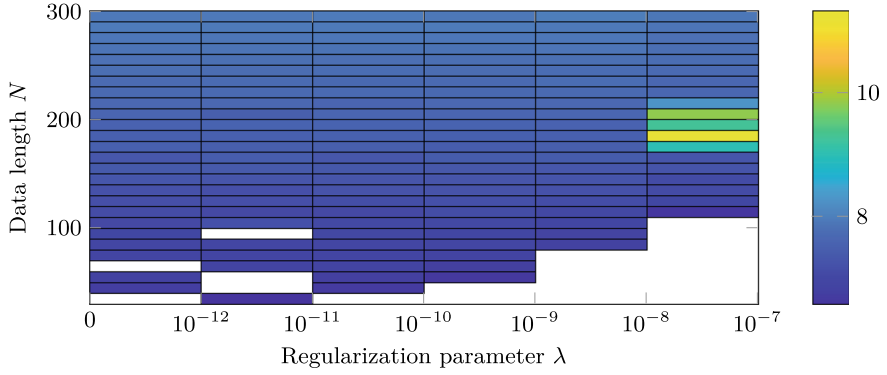


Fig. 5.2 Error (5.32) for indirect data-driven MPC (proposed approach)

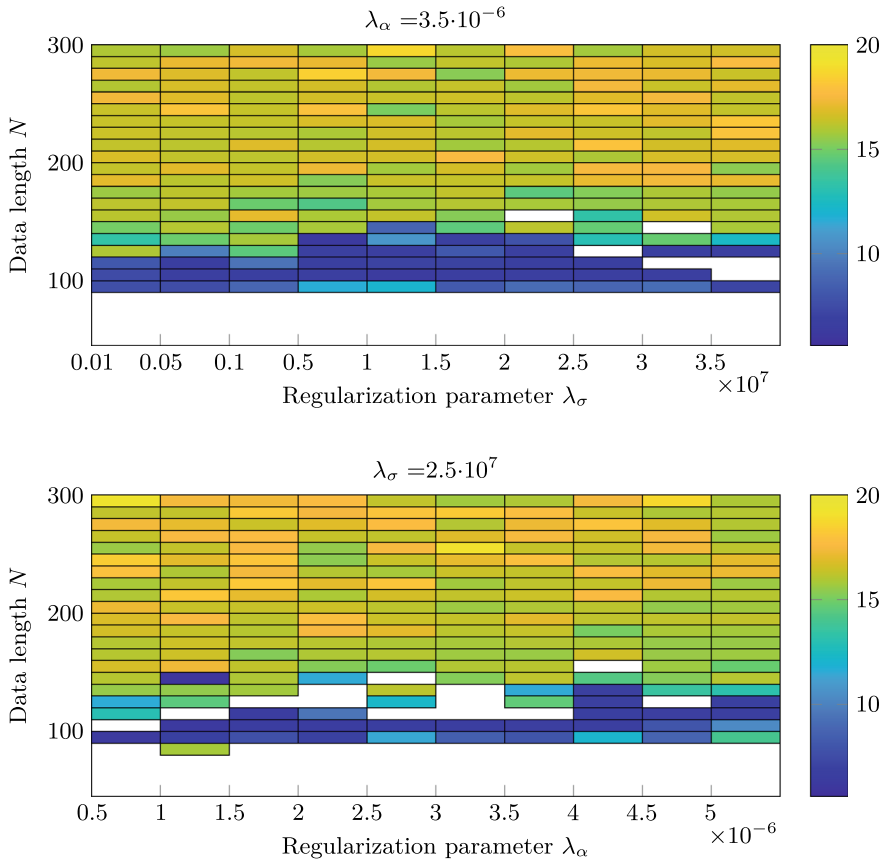


Fig. 5.3 Error (5.32) for direct data-driven MPC (approach from Berberich et al. [5])

5.6 Conclusion

In this paper, we have introduced an indirect data-driven MPC scheme for nonlinear systems combining online linear system identification and tracking MPC, and admitting desirable closed-loop guarantees. The presented indirect approach is closely related to the existing direct data-driven MPC scheme from Berberich et al. [5] which uses the Fundamental Lemma for prediction. Our analysis reveals that, despite the fundamental difference between the two employed prediction models (identified model versus Fundamental Lemma), both schemes admit comparable theoretical guarantees. Yet, several notable differences should be highlighted.

In comparison to Berberich et al. [5], the main benefit of the indirect data-driven MPC scheme is that it is simpler to tune and admits a (slightly) easier theoretical analysis. Further, contrary to its direct counterpart, the computational complexity of the indirect approach does not scale with the data length. While the complexity of the employed identification algorithm scales with the data length, a recursive identification algorithm could be used to overcome this issue. On the other hand, the main benefit of the direct approach from Berberich et al. [5] is that it allows to control unknown system based only on input–output data, whereas the presented indirect scheme requires state measurements. Finally, the indirect approach employs a one-step model whereas the direct approach uses a multi-step predictor, which causes further inherent differences [13]. A more formal theoretical investigation of the connections between direct and indirect data-driven MPC is an interesting issue for future research.

Acknowledgements Johannes Köhler was supported by the Swiss National Science Foundation under NCCR Automation (grant agreement 51NF40 180545).

References

1. Adetola, V., Guay, M.: Robust adaptive MPC for constrained uncertain nonlinear systems. *Int. J. Adapt. Control Signal Process.* **25**(2), 155–167 (2011)
2. Berberich, J., Allgöwer, F.: An overview of systems-theoretic guarantees in data-driven model predictive control (2024). [arXiv:2406.04130](https://arxiv.org/abs/2406.04130)
3. Berberich, J., Köhler, J., Müller, M.A., Allgöwer, F.: Data-driven model predictive control with stability and robustness guarantees. *IEEE Trans. Autom. Control* **66**(4), 1702–1717 (2021)
4. Berberich, J., Köhler, J., Müller, M.A., Allgöwer, F.: Linear tracking MPC for nonlinear systems part I: the model-based case. *IEEE Trans. Autom. Control* **67**(9), 4390–4405 (2022)
5. Berberich, J., Köhler, J., Müller, M.A., Allgöwer, F.: Linear tracking MPC for nonlinear systems part II: the data-driven case. *IEEE Trans. Autom. Control* **67**(9), 4406–4421 (2022)
6. Berberich, J., Köhler, J., Müller, M.A., Allgöwer, F.: Stability in data-driven MPC: an inherent robustness perspective. In: *Proceedings of 61st IEEE Conference on Decision and Control (CDC)*, pp. 1105–1110 (2022)
7. Breschi, V., Chiuso, A., Formentin, S.: Data-driven predictive control in a stochastic setting: a unified framework. *Automatica* **152**, 110961 (2023)

8. Coulson, J., Lygeros, J., Dörfler, F.: Data-enabled predictive control: in the shallows of the DeePC. In: *Proceedings of European Control Conference (ECC)*, pp. 307–312 (2019)
9. Dörfler, F.: Data-driven control: Part two of two: hot take: why not go with models? *IEEE Control Syst. Mag.* **43**(6), 27–31 (2023)
10. Dörfler, F., Coulson, J., Markovsky, I.: Bridging direct & indirect data-driven control formulations via regularizations and relaxations. *IEEE Trans. Autom. Control* **68**(2), 883–897 (2023)
11. Faulwasser, T., Ou, R., Pan, G., Schmitz, P., Worthmann, K.: Behavioral theory for stochastic systems? A data-driven journey from Willems to Wiener and back again. *Ann. Rev. Control* **55**, 92–117 (2023)
12. Guay, M., Adetola, V., DeHaan, D.: *Robust and Adaptive Model Predictive Control of Nonlinear Systems*. Control, Robotics Sensors., Institution of Engineering and Technology (2015)
13. Köhler, J., Wabersich, K.P., Berberich, J., Zeilinger, M.N.: State space models vs. multi-step predictors in predictive control: Are state space models complicating safe data-driven designs? In: *Proceedings of 61st IEEE Conference Decision and Control (CDC)*, pp. 491–498 (2022)
14. Krishnan, V., Pasqualetti, F.: On direct vs indirect data-driven predictive control. In: *Proceedings of 60th IEEE Conference on Decision and Control (CDC)*, pp. 736–741 (2021)
15. Limón, D., Alvarado, I., Alamo, T., Camacho, E.F.: MPC for tracking piecewise constant references for constrained linear systems. *Automatica* **44**(9), 2382–2387 (2008)
16. Limon, D., Ferramosca, A., Alvarado, I., Alamo, T.: Nonlinear MPC for tracking piece-wise constant reference signals. *IEEE Trans. Autom. Control* **63**(11), 3735–3750 (2018)
17. Ljung, L.: *System Identification: Theory for the User*. Prentice-Hall, Englewood Cliffs, NJ (1987)
18. Lu, X., Cannon, M., Koksál-Rivet, D.: Robust adaptive model predictive control: performance and parameter estimation. *Int. J. Robust Nonlinear Control* **31**(18), 8703–8724 (2021)
19. Markovsky, I., Dörfler, F.: Behavioral systems theory in data-driven analysis, signal processing, and control. *Ann. Rev. Control* **52**, 42–64 (2021)
20. Markovsky, I., Huang, L., Dörfler, F.: Data-driven control based on the behavioral approach: from theory to applications in power systems. In: *IEEE Control Systems* (2023)
21. Mayne, D.Q., Kerrigan, E.C., van Wyk, E., Falugi, P.: Tube-based robust nonlinear model predictive control. *Int. J. Robust Nonlinear Control* **21**, 1341–1353 (2011)
22. Rawlings, J.B., Mayne, D.Q., Diehl, M.M.: *Model Predictive Control: Theory, Computation, and Design*, 3rd edn. Nob Hill Pub (2020)
23. Tanaskovic, M., Fagiano, L., Smith, R., Morari, M.: Adaptive receding horizon control for constrained MIMO systems. *Automatica* **50**(12), 3019–3029 (2014)
24. Verheijen, P., Breschi, V., Lazar, M.: *Handbook of linear data-driven predictive control: theory, implementation and design*. *Ann. Rev. Control* **56**, 100914 (2023)
25. Willems, J.C., Rapisarda, P., Markovsky, I., De Moor, B.: A note on persistency of excitation. *Syst. Contr. Lett.* **54**, 325–329 (2005)
26. Ziemann, I., Tsiamis, A., Lee, B., Jedra, Y., Matni, N., Pappas, G.J.: A tutorial on the non-asymptotic theory of system identification. In: *Proceedings of 62nd IEEE Conference on Decision and Control (CDC)*, pp. 962–980 (2023)

Part II
Machine Learning: Theory
and Applications

Chapter 6

Investigation of the Influence of Training Data and Methods on the Control Performance of MPC Utilizing Gaussian Processes



Florian Diepers, Elmar Ahle, and Dirk Söffker

Abstract In learning-based control, model predictive control (MPC) is frequently employed due to the broad usability of various data-based models for predicting system dynamics. The uncertainty inherent in these models, influenced by factors such as training data or method, is typically treated as unknown. In this work, Gaussian Processes (GPs) are chosen as data-based models, because GPs have the advantage of quantifying the uncertainty of their predictions. In MPC, the quality of the prediction model has a significant impact on the resulting control performance. In the following, the correlation between prediction quality of the model and the control performance is investigated. Systematic exploration of various parameters affecting training is conducted. The following parameters are varied: different training datasets, different methods of rescaling the training data, different methods for constructing the reduced training subset, varying the number of training points in the subset, and employing two different methods for GP training (i.e., hyperparameter optimization). Using these methods, a GP is trained and utilized in an MPC to control a dynamic system. A simulated crane pendulum is used as a test system. The prediction and control performance is evaluated using criteria for error and input energy.

The original version of the chapter has been revised. A correction to this chapter can be found at https://doi.org/10.1007/978-3-031-83191-1_22

F. Diepers (✉) · E. Ahle

Faculty of Electrical Engineering and Computer Science, University of Applied Sciences Niederrhein, Krefeld, Germany

e-mail: florian.diepers@hs-niederrhein.de

E. Ahle

e-mail: elmar.ahle@hs-niederrhein.de

F. Diepers · D. Söffker

Chair of Dynamics and Control, University of Duisburg-Essen, Duisburg, Germany

e-mail: soeffker@uni-due.de

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025, corrected publication 2025

J. Berberich et al. (eds.), *Systems Theory in Data and Optimization*, Lecture Notes in Control and Information Sciences - Proceedings,

https://doi.org/10.1007/978-3-031-83191-1_6

Keywords Gaussian Process (GP) · Gaussian Process State-Space Models (GP-SSM) · Model Predictive Control (MPC) · Systematic investigation training · Crane pendulum

6.1 Introduction

The design and application of many control methods require a high-quality system model. The modeling is becoming increasingly challenging due to the rising complexity of systems. At the same time, more measured data are available for many systems, enabling the generation of data-based models.

Model predictive control (MPC) is a popular approach in model-based control. It offers advantages like direct consideration of constraints and easy integration of different models in the optimization loop. However, the quality of MPC relies on the quality of the prediction model. In cases where systems are either unknown or challenging to model, MPC can leverage data-based models to enhance performance.

In this work, Gaussian Processes (GPs) are used as the only prediction model within an MPC, resulting in the term Gaussian Process MPC (GP-MPC). A GP can be used to model a wide range of nonlinear functions. Compared to other machine learning models, GPs can be successfully trained with relatively little training data (e.g., Kocijan [1], Eleftheriadis et al. [2]). Additionally, GPs provide a measure of prediction uncertainty through the variance of the posterior distribution. The combination of MPC with GPs has been used in various applications, e.g., Eleftheriadis et al. [2], Liu et al. [3], Di Natale et al. [4]. To apply GPs to multi-dimensional dynamic systems, they can be embedded in external structures. In Diepers et al. [5], GP state-space models (GP-SSMs) and GP-based nonlinear autoregressive exogenous (GP-NARX) structures used inside MPC are compared. In this work only a GP-SSM is used, using the simulation results from Diepers et al. [5].

Although GPs have some distinct advantages, one important drawback is their computational burden which is influenced by the size of the training dataset. The computational load scales cubically for the training phase (caused by repeated inversion of the kernel matrix during hyperparameter optimization), quadratically for evaluation of the posterior variance, and linearly for evaluation of the posterior mean [1, 6, 7]. As a result, relatively small training sets are preferred to speed up GP predictions, additionally various sparse GP approximations can be applied [1]. Fast predictions support applications in real-time MPC and fast online learning. However, the model quality and ultimately the closed-loop control performance should not suffer. Therefore, the effect of the GP training process, including the training dataset, on the control performance of a simulated crane pendulum is investigated in this paper.

The workflow for generating GPs is divided into the five parts: generation of datasets, data scaling, selection method and size of a subset for training, and the training method. Initially, two distinct datasets are used, both derived from a simulated crane pendulum. Three different methods to scale the data and two methods to

select a subset of this data are employed. This subset is investigated across six varying sizes. After data preprocessing, the GPs are trained with two different methods for hyperparameter optimization. For all 144 possible combinations of the different options, the trained GPs are utilized in MPC and the model quality as well as the control performance is evaluated and compared.

In the following, Sect. 6.2 provides a brief introduction to GPs, while Sect. 6.3 covers MPC utilizing GPs. Then, the different methods utilizing the training data are presented. After that, the test scenario is introduced in Sect. 6.5, and the results of prediction model and controlled system are presented in Sect. 6.6. Finally, a short summary and an outlook are given.

6.2 Gaussian Process Dynamic Models

In this work, the dynamic system under control is modeled solely with GPs. A GP is a non-parametric model which is well suited for approximating nonlinear functions with relatively few training data [2, 8]. By representing predictions as probability distributions, a GP offers means to quantify prediction uncertainty via the variance of the posterior distribution [2].

By mapping inputs to corresponding outputs, a GP models static nonlinearities. To model multi-dimensional dynamic systems, multiple GPs can be embedded in additional external structures. In [5], two such structures, namely, GP-NARX and GP-SSM, are investigated to control an inverted pendulum. Compared to NARX models, state-space models (SSMs) offer greater flexibility in their internal structure [1]. This can result in more accurate and smoother predictions, which are simulatively validated for an inverted pendulum on a cart in Diepers et al. [5]. Because the dynamics of the crane pendulum is similar to the inverted pendulum, GP-SSM is used in this work. This section provides a short overview of GPs and introduces the dynamic model GP-SSM.

6.2.1 Gaussian Process

An unknown nonlinear function $f(\mathbf{z}) : \mathbb{R}^{n_z} \rightarrow \mathbb{R}$ with the input $\mathbf{z} \in \mathbb{R}^{n_z}$ is considered. Noisy observations $y \in \mathbb{R}$ subject to Gaussian distributed measurement noise $\nu \sim \mathcal{N}(0, \sigma_n^2) \in \mathbb{R}$ can be generated from this function as

$$y = f(\mathbf{z}) + \nu. \quad (6.1)$$

The latent function f is modeled as a Gaussian process. A GP is a stochastic process, which is defined as a collection of random variables where any finite subset of variables is jointly Gaussian distributed [7]. A GP is denoted as

$$f(\mathbf{z}) \sim \mathcal{GP}(m(\mathbf{z}), k(\mathbf{z}, \mathbf{z}')), \quad (6.2)$$

and completely specified by a mean function $m : \mathbb{R}^{n_z} \rightarrow \mathbb{R}$ and a covariance (or kernel) function $k : \mathbb{R}^{n_z} \times \mathbb{R}^{n_z} \rightarrow \mathbb{R}$. Typically, the mean function is set to zero in the absence of prior information about f . In this work, the widely used squared exponential kernel with automatic relevance determination (SE-ARD) is employed as the covariance function, defined by

$$k(\mathbf{z}, \mathbf{z}') = \sigma_f^2 \exp\left(-\frac{1}{2}(\mathbf{z} - \mathbf{z}')^\top \mathbf{L}(\mathbf{z} - \mathbf{z}')\right), \quad (6.3)$$

where $\sigma_f^2 \in \mathbb{R}$ represents the output variance and the lengthscales are captured by the diagonal matrix $\mathbf{L} = \text{diag}([l_1, \dots, l_{n_z}])^{-2} \in \mathbb{R}^{n_z \times n_z}$ [7]. These hyperparameters are summarized in the vector $\boldsymbol{\theta} = [\sigma_f^2, l_1, \dots, l_{n_z}]^\top \in \mathbb{R}^{1+n_z}$.

The GP is trained using a dataset $\mathcal{S} = \{(\mathbf{z}_i, y_i) | i = 1, \dots, n_s\} = (\mathbf{Z}, \mathbf{y})$, where $\mathbf{z}_i$ represents the input and y_i denotes the observed output. These inputs and outputs can be concatenated to form the input training matrix $\mathbf{Z} \in \mathbb{R}^{n_z \times n_s}$ and the output training vector $\mathbf{y} \in \mathbb{R}^{n_s}$. The hyperparameters are selected by optimizing a criterion over the training dataset. Typically, the negative log marginal likelihood (MLL) is minimized, which is derived from Bayesian inference [7]:

$$\log p(\mathbf{y}|\mathbf{Z}, \boldsymbol{\theta}) = -\frac{1}{2} \mathbf{y}^\top \mathbf{K}_y^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}_y| - \frac{n_s}{2} \log(2\pi). \quad (6.4)$$

The matrix $\mathbf{K}_y \in \mathbb{R}^{n_s \times n_s}$ is defined as $\mathbf{K}_y = \mathbf{K} + \sigma_n^2 \mathbf{I}$ with the covariance matrix $\mathbf{K}$ computed element-wise using the kernel function $K_{ij} = k(\mathbf{z}_i, \mathbf{z}_j)$. The measurement noise σ_n^2 is added to the main diagonal.

Alternatively, another criterion for hyperparameter optimization involves minimizing the log predictive density error (LPD) [1] as

$$\text{LPD} = \frac{1}{N} \sum_{i=1}^N \log p(y_i|\mathbf{Z}, \mathbf{y}, \boldsymbol{\theta}) = \frac{\log(2\pi)}{2} + \frac{1}{2N} \sum_{i=1}^N \left(\log(\sigma_i^2) + \frac{(y_i - \mu_i)^2}{\sigma_i^2} \right), \quad (6.5)$$

where μ_i and σ_i denote the posterior mean and variance of the i th training point. The LPD compares the known outputs y_i of the dataset $\mathcal{S}$ to the predicted Gaussian distribution of the GP.

To predict an output based on a new input $\mathbf{z}'$ and the given dataset $\mathcal{S}$, the posterior mean and variance of the Gaussian process are evaluated as Rasmussen and Williams [7]

$$\mu(\mathbf{z}') = \mathbf{k}(\mathbf{Z}, \mathbf{z}')^\top \mathbf{K}_y^{-1} \mathbf{y}, \quad (6.6)$$

$$\sigma^2(\mathbf{z}') = k(\mathbf{z}', \mathbf{z}') - \mathbf{k}(\mathbf{Z}, \mathbf{z}')^\top \mathbf{K}_y^{-1} \mathbf{k}(\mathbf{Z}, \mathbf{z}'), \quad (6.7)$$

with $\mathbf{k}(\mathbf{Z}, \mathbf{z}') = [k(\mathbf{z}_1, \mathbf{z}'), \dots, k(\mathbf{z}_{n_s}, \mathbf{z}')]^\top \in \mathbb{R}^{n_s}$.

6.2.2 Gaussian Process State-Space Models

In general, dynamic systems can have multiple inputs and multiple outputs (MIMO). Those systems can be described by nonlinear discrete-time state-space models

$$\mathbf{x}_{t+1} = \mathbf{f}(\mathbf{x}_t, \mathbf{u}_t) + \boldsymbol{\nu}_t, \quad (6.8)$$

with the discrete time step t . The system input is denoted by $\mathbf{u}_t \in \mathbb{R}^{n_u}$, all states $\mathbf{x}_t \in \mathbb{R}^{n_x}$ are assumed to be measurable and map to the output $\mathbf{y}_t \in \mathbb{R}^{n_y}$ with a known relation $\mathbf{y}_t = \mathbf{C} \mathbf{x}_t$, where $\mathbf{C} \in \mathbb{R}^{n_y \times n_x}$. The model (6.8) incorporates discrete-time Gaussian distributed white noise $\boldsymbol{\nu}_t \in \mathbb{R}^{n_x}$.

An ordinary GP can have multiple inputs but only predicts one output. Therefore, each state transition is modeled by an independent GP. Consequently, n_x independent GPs are required to model the n_x dimensional state space from model (6.8) as GP-SSM. The GP-SSM, based on Beckers and Hirche [9], is given by

$$\mathbf{x}_{t+1} = \begin{cases} f_1(\mathbf{z}_t) \sim \mathcal{GP}(m_1(\mathbf{z}_t), k_1(\mathbf{z}_t, \mathbf{z}'_t)) \\ \vdots \\ f_{n_x}(\mathbf{z}_t) \sim \mathcal{GP}(m_{n_x}(\mathbf{z}_t), k_{n_x}(\mathbf{z}_t, \mathbf{z}'_t)). \end{cases} \quad (6.9)$$

Here, the input of the GP $\mathbf{z}_t \in \mathbb{R}^{n_x + n_u}$ is composed of the states and the system input of the current time step $\mathbf{z}_t = [\mathbf{x}_t^\top, \mathbf{u}_t^\top]^\top$. The inputs and outputs of the training dataset for the GP-SSM consist of consecutive pairs of state values $\mathcal{S} = \{(\mathbf{z}_t, \mathbf{y}_t) | t = 1, \dots, n_s\}$ with $\mathbf{y}_t = \mathbf{x}_{t+1}$. GP-based state-space models are applied in various studies, such as stated in Beckers and Hirche [9], Bradford et al. [10, 11].

6.3 Model Predictive Control with Gaussian Processes

Model predictive control is an advanced control technique applied across various domains (e.g., Mayne et al. [12], Holkar and Waghmare [13]). It is particularly suitable for multivariate systems with state and input constraints. The idea of MPC is to use a constrained optimization problem to derive an optimal control law for a specified objective function J . Typically, the control law is implicit, optimizing a sequence of system input values $\mathbf{U} \in \mathbb{R}^{n_u \times n_{N_c}}$ directly. Employing an explicit prediction model $\hat{\mathbf{f}}$, MPC predicts the future system behavior, which is evaluated using J . The optimal sequence of system inputs is calculated by minimizing the objective function. The first element of this sequence is applied to the system, and the

optimization process is repeated for subsequent time steps, thus closing the control loop.

If the prediction model is no longer deterministic but stochastic, like model (6.8), the MPC formulation must accommodate for unknown noise and disturbances. Stochastic MPC (SMPC) is used to cope with this stochastic behavior. The optimal control problem (OCP) is extended to a stochastic OCP that integrates stochastic states into the objective function and constraints [14]. Typically, the objective function in SMPC is defined as the expected value of the original objective function, denoted as $J_{\text{SMPC}} = \mathbb{E}[J]$, e.g., in Farina et al. [15]. In this work, J_{SMPC} is formulated over the finite prediction horizon N_p and control horizon N_c , incorporating penalty terms. These terms penalize deviations between the predicted output $\hat{\mathbf{y}}_t \in \mathbb{R}^{n_y}$ and the reference $\mathbf{y}_t^{\text{ref}} \in \mathbb{R}^{n_y}$, as well as high system inputs $\hat{\mathbf{u}}_t \in \mathbb{R}^{n_u}$ and changes in system inputs $\Delta\hat{\mathbf{u}}_t = \hat{\mathbf{u}}_t - \hat{\mathbf{u}}_{t-1} \in \mathbb{R}^{n_u}$. While the system states and outputs are treated as random variables, the system inputs remain deterministic. By defining $\text{Var}(\cdot)$ to denote the variance of a random variable, the objective function can be formulated as

$$\begin{aligned} J_{\text{SMPC}} &= \mathbb{E} \left[\sum_{i=1}^{N_p} \|\mathbf{y}_{t+i|t}^{\text{ref}} - \hat{\mathbf{y}}_{t+i|t}\|_{\mathbf{Q}}^2 + \sum_{i=0}^{N_c-1} \|\hat{\mathbf{u}}_{t+i|t}\|_{\mathbf{R}_u}^2 + \sum_{i=0}^{N_c-1} \|\Delta\hat{\mathbf{u}}_{t+i|t}\|_{\mathbf{R}_u}^2 \right], \\ &= \sum_{i=1}^{N_p} \left(\|\mathbf{y}_{t+i|t}^{\text{ref}} - \mathbb{E}[\hat{\mathbf{y}}_{t+i|t}]\|_{\mathbf{Q}}^2 + \text{tr}(\mathbf{Q} \text{Var}(\hat{\mathbf{y}}_{t+i|t})) \right) \\ &\quad + \sum_{i=0}^{N_c-1} \|\hat{\mathbf{u}}_{t+i|t}\|_{\mathbf{R}_u}^2 + \sum_{i=0}^{N_c-1} \|\Delta\hat{\mathbf{u}}_{t+i|t}\|_{\mathbf{R}_u}^2. \end{aligned} \quad (6.10)$$

Here, the subscript $t + i|t$ denotes the value of the variable at the future discrete time step $t + i$ for time indices $i \in \{0, \dots, N_p\}$ within the prediction horizon based on the knowledge of the system at discrete time step t . The control horizon is bounded by $1 \leq N_c \leq N_p$, resulting in reduced computational complexity for $N_c < N_p$. For prediction time steps $i \geq N_c$, the inputs remain constant at the value $\hat{\mathbf{u}}_{t+N_c|t}$. With the matrices $\mathbf{Q}$, $\mathbf{R}_u$, and $\mathbf{R}_u$ the outputs and inputs can be weighted. The notation $\|\mathbf{x}\|_{\mathbf{Q}}^2$ represents the quadratic term $\mathbf{x}^\top \mathbf{Q} \mathbf{x}$.

A mathematical formulation of the stochastic OCP for SMPC is given by

$$\min_{\hat{\mathbf{u}}} J_{\text{SMPC}}, \quad (6.11)$$

subject to

$$\hat{\mathbf{x}}_{t+i+1|t} = \hat{\mathbf{f}}(\hat{\mathbf{x}}_{t+i|t}, \hat{\mathbf{u}}_{t+i|t}) \quad i \in \{0, \dots, N_p\}, \quad (6.12)$$

$$\hat{\mathbf{y}}_{t+i+1|t} = \hat{\mathbf{g}}(\hat{\mathbf{x}}_{t+i|t}, \hat{\mathbf{u}}_{t+i|t}) \quad i \in \{0, \dots, N_p\}, \quad (6.13)$$

$$\hat{\mathbf{u}}_{t+i|t} \in \mathbb{U} \quad i \in \{0, \dots, N_c - 1\}, \quad (6.14)$$

$$\Pr[h(\hat{y}_{t+i|t}) \leq 0] \geq 1 - \beta \quad i \in \{1, \dots, N_p\}, \quad (6.15)$$

where (6.14) denotes the input constraints with the valid set $\mathbb{U}$. Due to the stochastic nature, chance constraints are imposed on the outputs in (6.15). The probability $\Pr(\cdot)$ of satisfying the constraint function h should be greater or equal to $1 - \beta$ with the tuning parameter $\beta \in \mathbb{R}$. The OCP restarts at each time step with the measured state $\mathbf{x}_{t|t}$. A general overview of SMPC can be found in Heirung et al. [14], Farina et al. [15], Mesbah [16]. In recent years, an increasing number of publications have combined SMPC and GPs (e.g., Bradford et al. [10, 17], Morabito et al. [18]). The predictive distribution of GPs can, for example, be used as a random state variable in SMPC.

6.4 Utilization of Training Data

The main contribution of this paper is to investigate how training data and processes impact control performance. Corresponding to Fig. 6.1, five key aspects of generating GPs are distinguished. Initially, a training dataset must be generated or measured. Here, two distinct datasets are used, both derived from system simulations. The first dataset is generated by applying random system inputs to the open-loop system, inducing changes of the states. The second dataset is generated by employing a nonlinear MPC controller with the correct model in a closed-loop simulation.

The subsequent sections elaborate the options of the other key aspects of Fig. 6.1: scaling methods, selection methods, and training methods. The missing aspect, varying the size of the subset used for hyperparameter optimization, is straightforward. The subset size is varied and data points from the original datasets are added according to the selection method. All potential combinations are tested, and the results are presented in Sect. 6.6.

6.4.1 Scaling Methods

Data scaling is applied to ensure consistent value ranges within the training dataset (inputs as well as outputs). In GPs, the covariance functions involve calculating the difference between input features $(\mathbf{z} - \mathbf{z}')$ and weighing them with the lengthscale

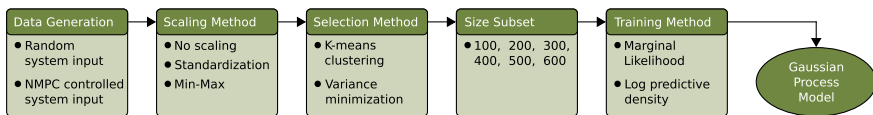


Fig. 6.1 Workflow for building GPs with the tested variations

(cf. Eq. 6.3). The summation of different input features in most kernels can lead to biased influence for different features, especially without employing ARD techniques. Similarly, the outputs of GPs are predicted Gaussian distributions and standardization of the known outputs can yield improvements. Therefore, data scaling can achieve model improvements, particularly for heterogeneous systems.

In Ahsan et al. [19], it is shown that the choice of scaling methods impacts prediction results, even without one method clearly outranking the others. In this work, two scaling methods are compared to each other and to the case of no scaling. The two most commonly used data scaling methods are applied:

- Standardization: Scale dataset to zero mean and a standard deviation of one

$$z^{\text{scaled}} = \frac{z - \mu_z}{\sigma_z} \text{ and}$$

- Min-max scaling: Scale all values to fit in a defined interval (typically [0, 1])

$$z^{\text{scaled}} = \frac{z - \min z}{\max z - \min z}.$$

6.4.2 Selection Methods

To reduce the computational burden associated with GPs, various sparse GP techniques are available. These methods aim to generate GP models with reduced training points and can be classified into *global* and *local* approximations [20]. In this work, a global approximation method utilizing a subset of the available training data within the GP is employed. This subset of data is referred to as the active set [20]. To build this subset of training data, two different methods are used.

The first method employs k-means clustering to assign each data point of the training dataset to one of k clusters. The number of clusters correspond to the desired active set size. In the active set, each cluster is represented by one data point from the original training dataset that is closest to the centroid of the cluster. The initial clusters are based on random values.

The second method is from the field of active learning, distinguishing between *static* and *dynamic* problems [21]. Although a dynamic system is considered, static active learning is applied on the previously sampled training datasets as available in industry. This limits the possible samples to the existing dataset but allows selection of any arbitrary point. A greedy algorithm is used to add data points to the active set until the desired size is reached. The criterion used is the variance prediction of the GP. The goal is to minimize the uncertainty of the GP predictions. Starting with an empty active set, a random data point from the entire dataset is added and the GP is trained with this active set. After that, the GP predicts the variance for each remaining point in the original training dataset. The point with maximum variance σ^2 is then added to the active set. This is repeated until the active set reaches the desired size. For MIMO systems, this method can result in individual active sets for each GP (for example, a different active set for every state). In contrast, k-means clustering generates a single active set for a training dataset.

6.4.3 Training Methods

In SMPC, GPs are applied to generate the required predictions for the entire horizon N_p . This can be achieved by either multiple one-step predictions or by multi-step predictions. However, as the objective function of the SMPC requires predictions for every time step in N_p to penalize undesired behavior, multiple one-step predictions are used.

The first training method involves minimizing the negative log marginal likelihood in (6.4), which is the default approach for training GPs. With this method a balance between data fit and model complexity can be achieved [1], making it well suited for training GPs for one-step predictions.

The second method for training GPs for one-step predictions is the LPD from (6.5). This method is an alternative often used to validate GP models [1]. Unlike other loss functions such as squared error, LPD considers not only the posterior mean but also the posterior variance [7]. With this training method, the GP makes predictions in the form of Gaussian distributions based on the data within the GP. Then, the probability that the known output belongs to the predicted distribution is calculated. Notably, the data within the GP and the data to compare the predictions do not have to be identical. Rather, the GP data can be a subset of the training dataset, offering flexibility in model training.

6.5 Test Scenario

The previously described methods are employed to train a GP-SSM which is utilized inside a SMPC to control a simulated crane pendulum taken from Diepers et al. [5]. In this section, a model of the system, derived from a physical setup including an external drive controller, is introduced. Additionally, an evaluation criterion for assessing the control performance as well as the simulation configuration for all test scenarios is presented.

6.5.1 Crane Pendulum

The investigation of the training methods is conducted using a crane pendulum as shown in Fig. 6.2. The system is simulated with the continuous time t_c . The velocity of the cart $v(t_c) \in \mathbb{R}$ is controlled by an external drive and can be modeled as $T_1 \dot{v}(t_c) + v(t_c) = K_v U(t_c)$. The control voltage of the external drive $U(t_c) \in \mathbb{R}$ serves as the system input, while the outputs are the cart position $s(t_c) \in \mathbb{R}$ and the angle $\varphi(t_c) \in \mathbb{R}$ of the pendulum. Replacing the cart velocity $v(t_c)$ with the derivative of the cart position $\dot{s}(t_c)$, the two differential equations, already presented in Diepers et al. [5], are

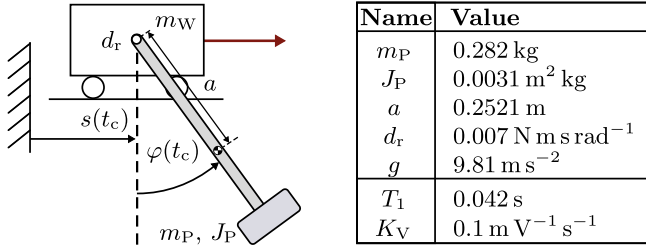


Fig. 6.2 Crane pendulum

$$T_1 \ddot{s}(t_c) = K_V U(t_c) - \dot{s}(t_c), \quad (6.16)$$

$$(J_P + m_P a^2) \ddot{\varphi}(t_c) = m_P a [-\ddot{s}(t_c) \cos(\varphi(t_c)) - g \sin(\varphi(t_c))] - d_r \dot{\varphi}(t_c). \quad (6.17)$$

The system's parameters are given in Fig. 6.2. The system is represented using the state vector $\mathbf{x}(t_c) = [s(t_c) \ \dot{s}(t_c) \ \varphi(t_c) \ \dot{\varphi}(t_c)]^\top$, the state equation

$$\dot{\mathbf{x}}(t_c) = \begin{bmatrix} x_2(t_c) \\ \frac{1}{T_1} (K_V U(t_c) - x_2(t_c)) \\ x_4(t_c) \\ \frac{1}{J_P + m_P a^2} (m_P a P - d_r x_4(t_c)) \end{bmatrix}, \quad (6.18)$$

with $P = -\frac{1}{T_1} (K_V U(t_c) - x_2(t_c)) \cos(x_3(t_c)) - g \sin(x_3(t_c))$, and the output equation $y(t_c) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \mathbf{x}(t_c)$.

6.5.2 Evaluation Criterion

To assess control performance across various training methods or parameterizations, a two-dimensional evaluation criterion is used. The criterion considers a measurement for input effort and control error. The input measurement is composed of the absolute system input u_t , which is scalar in this test scenario, and the absolute change of input $\Delta u_t = u_t - u_{t-1}$. The sum of the absolute errors of each output determines the absolute control error $e_t = |s_t^{\text{ref}} - s_t| + |\varphi_t^{\text{ref}} - \varphi_t|$. The criterion is defined over the entire simulation time $t \in [0, T_{\text{end}}]$ as

$$P_{K^*} = \frac{1}{T_{\text{end}}} \left[\sum_{t=0}^{T_{\text{end}}} (|u_t| + |\Delta u_t|), \sum_{t=0}^{T_{\text{end}}} |e_t| \right]_{K^*}. \quad (6.19)$$

A trajectory consisting of multiple points P_{K^*} (one for each simulation) is generated by varying the simulation parameters K^* (e.g., scaling method or training method).

In this work, the active set size for GPs is increased along the trajectory. Multiple trajectories are generated for each combination of initial dataset, data scaling, data selection, and training method.

6.5.3 Simulation Configuration

For training, two different datasets $\mathcal{S}_1$ and $\mathcal{S}_2$ are generated from a simulated crane pendulum (6.18). Dataset $\mathcal{S}_1$ employs random system inputs which are uniformly distributed across the input space of ± 10 V. The initial state is the origin. The second dataset uses system inputs generated by a nonlinear MPC (NMPC) controller utilizing the correct physical model. The initial state and MPC weights are given in the following paragraphs. During this control, the reference position is applied as square wave signal with varying amplitudes around zero from 0.1 to 0.2 m. For both datasets, the data points are sampled with $T_s = 20$ ms for a simulation time of 12.5 s, yielding datasets with 625 data points $\mathbf{z}_t = [\mathbf{x}_t^\top, \mathbf{u}_t^\top]^\top$. The system input, and outputs of the full datasets are visualized in Fig. 6.3.

The system is modeled with a GP-SSM with one GP for each of the system states, which predicts the next time step based on the current state. All GPs are modeled using the SE-ARD kernel and a zero-mean function. No additional noise is added to the simulation, only the modeling error of the GP-SSM is present.

The configuration of the SMPC remains consistent for generating the training data and testing of the trained GP-SSM. The prediction and control horizon are chosen to $N_p = 50$ and $N_u = 10$. The weights of the objective function are $\mathbf{Q} = \text{diag}([16, 16]) \in \mathbb{R}^{2 \times 2}$, $R_u = 10^{-6}$, and $R_{\Delta u} = 0.1$. Constraints are only imposed on the system input which is limited between ± 10 V.

Results are generated by controlling the crane pendulum with SMPC using a GP-SSM in a simulation. Each simulation of $T_{\text{end}} = 12$ s follows the same procedure. A reference trajectory for the position s^{ref} is given. It starts at 0.3 m, steps up to 0.5 m after four seconds and back to 0 m after eight seconds. The angle is to be stabilized at the equilibrium $\varphi^{\text{ref}} = 0$ rad. All simulations start at an initial state of $\mathbf{x}_0 = [-0.1, 0, 0.0175, 0]^\top$, with all states assumed to be measurable. All simulations are performed using MATLAB®.

6.6 Results and Discussion

In this section, the results using different combinations of data preprocessing and training methods are presented. Initially, the full training datasets, each containing 625 data points, are either left in the original range of values or individually scaled for every input dimension of $\mathbf{z}_t$. The scaling is done using either standardization or

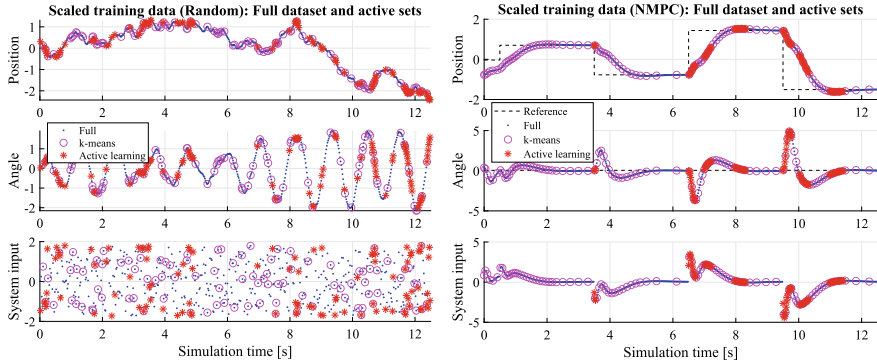


Fig. 6.3 Full training datasets $\mathcal{S}_1$, $\mathcal{S}_2$ with random system input (left) and system input generated by an NMPC (right)

min-max scaling. After that, a subset of the training dataset is selected by either k-means clustering or active learning. This active set is then used to train the individual GPs.

In Fig. 6.3, the full training datasets $\mathcal{S}_1$ and $\mathcal{S}_2$ are visualized, showing the standardized input and outputs of the crane pendulum under both random and controlled system inputs. Additionally, an exemplary active set based on the scaled datasets for each selection method is presented. The shown active sets have a size of 100 data points. Generally, the results for data selection with k-means clustering and active learning are significantly different. Active sets generated by k-means clustering exhibit a more uniform distribution over time compared to those generated via active learning. This is valid for both sets.

For the training dataset using the controlled system input, the data density of the active set using k-means clustering is lower near the equilibrium points. This can be attributed to more redundant information in these areas. Conversely, the active set based on active learning concentrates more on dynamic areas, particularly in the second half of the training data. Using random system inputs, the distribution of the active sets appears random as well. However, using active learning the data points are more concentrated in certain regions.

In the following, the simulated test case from Sect. 6.5 with a reference change of the position from 0.3 m to 0.5 m and back to 0 m is considered. Initially, a baseline is established using the physical model within an NMPC to control this test case. The control generates a system input trajectory and resulting states for the entire simulation. The prediction quality of each GP-SSM is then investigated based on the system input from the baseline.

Starting from an initial state $\mathbf{x}_0$, the progression of the future states is simulated using the different GP-SSMs based on the last state and the system input trajectory provided by the benchmark simulation. The mean absolute error (MAE) of the GP-SSM predictions is calculated for the entire simulation time of 12 seconds. The prediction error is calculated between the baseline outputs $\mathbf{y}^{\text{nom}}$ of the nominal model (6.18)

and the GP-SSM predictions $\mathbf{y}^{\text{pred}}$ to $e_t^{\text{pred}} = |s_t^{\text{nom}} - s_t^{\text{pred}}| + |\varphi_t^{\text{nom}} - \varphi_t^{\text{pred}}|$. In Fig. 6.4, the MAE is plotted against the size of the active set. The magnitude of most errors is comparable and employing the LPD training method results in higher prediction errors distributed over a larger region for both training datasets. Three outliers using the LPD training method are outside the plot section shown to improve visibility of the plot. Generally, errors tend to be slightly larger for GPs trained with smaller active sets compared to larger ones. However, the effect is not substantial, and increasing the training data does not yield improvements for more than 300 data points. Additionally, outliers with unusual high errors are observed, e.g., the elevated MAE for the NMPC dataset where the GP-SSM is trained with MLL, k-means clustering, and min-max scaling of an active set with 400 data points.

Using the references from the test scenario in Sect. 6.5, the crane pendulum is controlled utilizing the different GP-SSMs. The results are separated for the two datasets and shown in Fig. 6.5. The baseline control performance is visualized as a green diamond. In each case, the top plot gives an overview of all test cases, while the marked region of good control performance is detailed in the bottom plots. For each dataset, both lower plots show the same detailed region separated by the two training methods. The trajectories are grouped by the training method, active set selection, and data scaling. Each trajectory starts with an active set size of 100 data points (shown as a circle) which is increased to 600.

The results for both datasets show that the highest errors and input energies are caused by GP-SSMs trained with unscaled data. In comparison, more and larger outliers are generated based on the random dataset than the NMPC dataset. Using the random dataset all but some small outliers are based on LPD training. In general, the training with LPD generates trajectories distributed in a wider range of errors

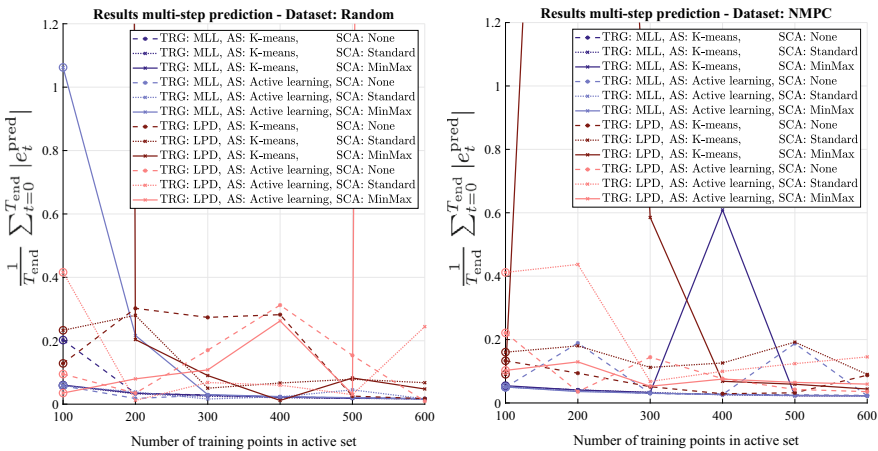


Fig. 6.4 MAE between baseline output trajectories and multi-step predictions grouped by different methods of training (TRG), active set selection (AS), and data scaling (SCA) in the training process (left: random dataset; right: NMPC dataset)

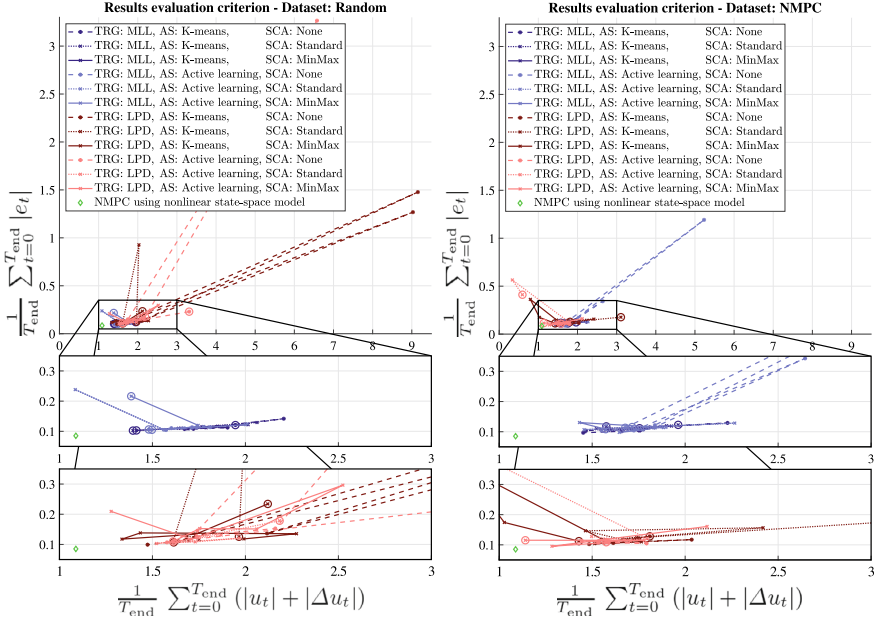


Fig. 6.5 Control performance for different methods of training (TRG), active set selection (AS), and data scaling (SCA) in the training process (left: random dataset; right: NMPC dataset)

for both datasets. This corresponds to the more fluctuating errors in the multi-step prediction shown in Fig. 6.4.

Most results show good control performance with comparable errors and input energy for both datasets. The MAE mostly ranges between 0.1 and 0.16, while the input energy spans a wider range from 1.3 to 2.3. However, a significant gap remains compared to the baseline (NMPC using the physical model). The trajectories in the highlighted region with the best control performance show comparable input energies but different MAEs. Using MLL training method, nearly all trajectories are on one line with similar control error but varying input energy. In contrast, trajectories trained with LPD have higher values and more fluctuations of the control error.

Comparing the control performance based on different scaling methods, combinations using no scaling for the training lead to the largest outliers. Scaling the training input and output has a positive effect on the control performance. Although the difference between standardization and min-max scaling is small, it appears that min-max scaling leads to the smallest errors. This result may be due to the fact that all states have the same value range (between 0 and 1). Next, differences depending on the active set method are investigated. Using k-means clustering yields the best results for both datasets, except for LPD training using the NMPC dataset. However, results using k-means clustering are slightly more distributed, especially the system inputs. Generally, errors for combinations using the LPD training method are more widely distributed and slightly larger. The best results are achieved using the MLL

method, which leads to more compact results. The input energy of both training methods is comparable.

Finally, the results from Figs. 6.4 and 6.5 are compared. In Fig. 6.5, distinct outliers with very high errors and inputs can be seen. For instance, the control performance for LPD training with k-means clustering, no scaling, and random training dataset with 200 and 400 data points has such errors. Similarly, using MLL training with active learning, no scaling, and the NMPC dataset for 200 and 500 data points of the active set show similar outliers. For those combinations, the multi-step predictions in Fig. 6.4 have an increased error compared to the baseline of mostly identical predictions using the MLL method. However, there are more combinations with higher multi-step prediction errors than outliers in the control performance. Therefore, outliers in the control performance cannot be determined by looking at the multi-step prediction error alone, although it can be an indication. Additionally, the magnitude of the error for prediction and control performance do not correlate. For example, the high multi-step prediction errors for LPD training, k-means clustering, no scaling, and active set sizes of 200, 300, and 400 data points do not consistently translate to high errors in control performance. The control performance only has high errors for active set sizes of 200 and 400. The results for 300 data points are quite good and close to other successful combinations with lower prediction errors. One possible explanation is that the model with 300 data points provides better prediction quality in the MPC's working range compared to the other cases.

6.7 Summary and Outlook

This work explores the training process of GPs and its impact on the control performance when utilized within SMPC. For the considered test case, three data scaling methods, two active set building methods, and two hyperparameter training approaches for GPs are compared for two different datasets. Additionally, the size of the active set is varied. Training GPs with MLL yields better results for this test case than LPD does for model prediction and control performance. Data scaling also enhances control performance because larger outliers in control error and input are prevented, especially for the random dataset. Comparable results are achieved using both active set generation methods. Varying active set size demonstrates a decrease in prediction error with an increased set size when MLL is used, though LPD shows error fluctuations. However, no clear trend emerges regarding the influence of active set size on the control performance. While significant control errors often correspond to larger prediction errors, not every model with poor prediction quality results in poor control performance.

In the future, the training process is investigated further. This includes extending investigations to further and more complex systems, additional datasets, and more training options for improved statistical validation. Furthermore, the training meth-

ods can be extended to concurrently train the GP-SSM rather than individual GPs. Additionally, criteria to assess control performance based on model quality can be investigated.

References

1. Kocijan, J.: *Modelling and Control of Dynamic Systems Using Gaussian Process Models*. Springer International Publishing, Cham (2016)
2. Eleftheriadis, S., Nicholson, T., Deisenroth, M., Hensman, J.: Identification of Gaussian process state space models. In: *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc. (2017)
3. Liu, Y., Shi, S., Zhang, Z., Di, Z., Babayomi, O.: Data-driven model predictive control for wave energy converters using gaussian process. *Symmetry* **14**(7), 1284 (2022)
4. Di Natale, L., Lian, Y., Maddalena, E.T., Shi, J., Jones, C.N.: Lessons learned from data-driven building control experiments: contrasting Gaussian process-based MPC, Bilevel DeePC, and deep reinforcement learning. In: *Proceedings of 61st IEEE Conference on Decision and Control (CDC)*, Cancun, Mexico, pp. 1111–1117 (2022)
5. Diepers, F., Polke, D., Ahle, E., Söfker, D.: Comparison of different Gaussian process models and applications in model predictive control. In: *Proceedings of 23rd International Conference on Control, Automation and Systems (ICCAS)*, Yeosu, Republic of Korea, pp. 54–59 (2023)
6. Chalupka, K., Williams, C.K.I., Murray, I.: A framework for evaluating approximation methods for Gaussian process regression. *J. Mach. Learn. Res.* **14**(1), 333–350 (2013)
7. Rasmussen, C.E., Williams, C.K.I.: *Gaussian Process for Machine Learning*. The MIT Press, London, England (2006)
8. Liu, M., Chowdhary, G., Da Castro Silva, B., Liu, S.-Y., How, J.P.: Gaussian processes for learning and control: a tutorial with examples. *IEEE Control Syst.* **38**(5), 53–86 (2018)
9. Beckers, T., Hirche, S.: Prediction with approximated Gaussian process dynamical models. *IEEE Trans. Autom. Control* **67**(12), 6460–6473 (2022)
10. Bradford, E., Imsland, L., Zhang, D., Del Rio Chanona, E.A.: Stochastic data-driven model predictive control using Gaussian processes. *Comput. Chem. Eng.* **139**(106844) (2020)
11. Bradford, E., Imsland, L., Del Rio-Chanona, E.A.: Nonlinear model predictive control with explicit back-offs for Gaussian process state space models. In: *Proceedings of 58th IEEE Conference on Decision and Control (CDC)*, Nice, France, pp. 4747–4754 (2019)
12. Mayne, D.Q., Rawlings, J.B., Rao, C.V., Scokaert, P.: Constrained model predictive control: stability and optimality. *Automatica* **36**(6), 789–814 (2000)
13. Holkar, K.S., Waghmare, L.M.: An overview of model predictive control. *Int. J. Control Autom.* **3**(4) (2010)
14. Heirung, T.A.N., Paulson, J.A., O’Leary, J., Mesbah, A.: Stochastic model predictive control—how does it work? *Comput. Chem. Eng.* **114**, 158–170 (2018)
15. Farina, M., Giullioni, L., Scattolini, R.: Stochastic linear model predictive control with chance constraints—a review. *J. Process Control* **44**, 53–67 (2016)
16. Mesbah, A.: Stochastic model predictive control: an overview and perspectives for future research. *IEEE Control Syst.* **36**(6), 30–44 (2016)
17. Bradford, E., Imsland, L.: Stochastic nonlinear model predictive control using Gaussian processes. In: *Proceedings of European Control Conference (ECC)*, Limassol, pp. 1027–1034 (2018)
18. Morabito, B., Pohlodek, J., Kranert, L., Espinel-Ríos, S., Findeisen, R.: Efficient and simple gaussian process supported stochastic model predictive control for bioreactors using HILO-MPC. *IFAC-PapersOnLine* **55**(7), 922–927 (2022)
19. Ahsan, M., Mahmud, M., Saha, P., Gupta, K., Siddique, Z.: Effect of data scaling methods on machine learning algorithms and model performance. *Technologies* **9**(3), 52 (2021)

20. Liu, H., Ong, Y.-S., Shen, X., Cai, J.: When Gaussian process meets big data: a review of scalable GPs. *IEEE Trans. Neural Netw. Learn. Syst.* **31**(11), 4405–4423 (2020)
21. Buisson-Fenet, M., Solowjow, F., Trimpe, S.: Actively learning Gaussian process dynamics. In: *Proceedings of 2nd Conference Learning for Dynamics and Control*, vol. 120, pp. 5–15. PMLR (2020)

Chapter 7

Wiener Chaos in Kernel Regression: Toward Untangling Aleatoric and Epistemic Uncertainty



Timm Faulwasser and Oleksii Molodchyk

Abstract Gaussian Processes (GPs) are a versatile method that enables different approaches toward learning for dynamics and control. Gaussianity assumptions appear in two dimensions in GPs: The positive semi-definite kernel of the underlying reproducing kernel Hilbert space is used to construct the covariance of a Gaussian distribution over functions, while measurement noise (i.e., data corruption) is usually modeled as i.i.d. additive Gaussians. In this note, we generalize the setting and consider kernel ridge regression with additive i.i.d. non-Gaussian measurement noise. To apply the usual kernel trick, we rely on the representation of the uncertainty via polynomial chaos expansions, which are series expansions for random variables of finite variance introduced by Norbert Wiener. We derive and discuss the analytic $\mathcal{L}^2$ solution to the arising *Wiener kernel regression*. Considering a polynomial dynamic system as a numerical example, we show that our approach allows us to distinguish the uncertainty that stems from the noise in the data samples from the total uncertainty encoded in the GP posterior distribution.

Keywords Kernel regression · Polynomial chaos expansion · Aleatoric uncertainty · Epistemic uncertainty · Non-Gaussian distribution

7.1 Introduction

Gaussian Processes (GPs) are a widely considered supervised learning method used for classification and regression. At the core of GPs are Gaussian distributions over functions with continuous domains. In the context of learning for dynamics and

T. Faulwasser · O. Molodchyk (✉)
Institute of Control Systems, Hamburg University of Technology, Hamburg, Germany
e-mail: oleksii.molodchyk@tuhh.de

T. Faulwasser
e-mail: tim.faulwasser@ieee.org

Institute of Energy Systems, Energy Efficiency and Energy Economics, TU Dortmund University, Dortmund, Germany

control, early successful application of GPs to model dynamic systems can be traced back at least to Murray-Smith et al. [23]. Early application in predictive control is considered by Murray-Smith et al. [18], while Deisenroth [8] discusses the use of GPs in reinforcement learning; see also Rasmussen and Williams [26] for a tutorial reference on GPs. Due to space constraints, we do not provide a detailed overview of existing results but rather refer to the recent papers [5, 19, 27] and references therein for further details.

A particularly appealing feature of GPs is the posterior covariance estimate of the predictions. This estimate is of interest in Bayesian optimization [12] as it allows to approach the exploration–exploitation trade-off. Moreover, the posterior covariance estimate is of relevance whenever GPs are trained to predict exogenous disturbances, e.g., energy consumption or renewable generation [6, 9, 12, 20].

On a formal level, the covariance estimate of GPs is closely related to the underlying kernel and the Gaussianity assumption made on the measurement noise [15]. Indeed there exists a close relation between Reproducing Kernel Hilbert Spaces (RKHS) and GPs, see, e.g., the treatments [15, 26] and the classic reference [14]. We refer to Aronszajn [1], Berlinet and Thomas-Agnan [3] for an introduction to RKHS.

In the present paper, we take a fresh look at kernel regression problems with noise-corrupted data samples, whereby the measurement noise does not need to be Gaussian. Specifically, we combine Polynomial Chaos Expansions (PCE), which originated in the most-cited journal paper of Wiener [33], with kernel regression problems. Recently, PCE has seen use in data-driven control of linear systems [10, 24]. While the link between PCE and the RKHS underlying GPs settings has been investigated by Yan et al. [35], a numerical comparison of both approaches as function approximators is conducted by Gratiet et al. [11]. However, the previous two papers as well as the work in Torre et al. [30] do not explore the potential of considering PCE-based measurement noise models in regression problems with arbitrary positive semi-definite kernel functions for dynamics and control.

The contributions and the outline of the present paper are as follows: After a concise problem statement in Sect. 7.2, we consider an intrusive uncertainty quantification approach for kernel regression problems in Sect. 7.3. Put differently, we use the PCE framework to quantify the effect of non-Gaussian measurement noise contained in the data samples on the prediction obtained via kernel ridge regression. Moreover, we discuss the relation of this uncertainty estimate to the classic GP posterior covariance estimate which measures epistemic uncertainty [13]. In Sect. 7.4, we draw upon a numerical example to illustrate how our approach allows to untangle the uncertainty caused by the noisy data samples from the uncertainty related to insufficient exploration of the hypothesis space. The paper ends with conclusions and outlook in Sect. 7.5.

7.2 Problem Statement and Preliminaries

Uncertainty can be described by random variables which are elements of a probability space $(\Omega, \mathcal{F}, \mathbb{P})$. This triple comprises the set of outcomes Ω , the sigma-algebra $\mathcal{F}$ over Ω , and the probability measure $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$. We consider a random variable $V \in (\Omega, \mathcal{F}, \mathbb{P}; \mathcal{V})$, which takes realizations in $\mathcal{V}$, as a measurable function $V : \Omega \rightarrow \mathcal{V}$. Hence, the realization of V associated with the outcome $\omega \in \Omega$ is denoted as $V(\omega) \in \mathcal{V}$. Moreover, the probability space $\mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathcal{V})$ contains random variables with finite expectation and finite variance such that $\mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathcal{V}) \subset (\Omega, \mathcal{F}, \mathbb{P}; \mathcal{V})$ and $V \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathcal{V})$ implies that $V : \Omega \rightarrow \mathcal{V}$ has finite $\mathcal{L}^2$ norm. For an in-depth discussion on probability spaces, see Sullivan [29].

The weight-space view of GPs. Following the classic exposition of Rasmussen and Williams [26] we briefly recall the weight-space view on GPs. Consider the data

$$\mathcal{D} = \{(x_i, y_i) \mid i \in \{1, \dots, D\}\},$$

obtained from sampling the unknown function $f : x \in \mathbb{R}^{n_x} \mapsto y \in \mathbb{R}^{n_y}$. For the sake of simplified exposition and to limit the notation overhead, we henceforth consider the setting with $\dim y = n_y \doteq 1$. Moreover, we use the shorthands $\mathbf{x} \doteq \{x_1, x_2, \dots, x_D\}$, $\mathbf{y} \doteq [y_1 \ y_2 \ \dots \ y_D]^\top$, and $\mathbb{I}_D \doteq \{1, \dots, D\}$ for any $D \in \mathbb{N}$. The data $\mathcal{D}$ is obtained via

$$y_i \doteq f(x_i) + M_i(\omega), \quad i \in \mathbb{I}_D, \quad M_i \sim \mathcal{N}(0, \sigma_M^2), \quad (7.1)$$

i.e., from measurements of the output of the unknown function f corrupted by realizations $M_i(\omega) \in \mathbb{R}$ of independent and identically distributed (i.i.d.) white Gaussian noise of known distribution $\mathcal{N}(0, \sigma_M^2)$. Suppose that the function f is of the form $f(x) = w^\top \phi(x)$ with known feature map $\phi : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_\phi}$ and unknown weights $w \in \mathbb{R}^{n_\phi}$. Initially, GPs treat the unknown vector w as a random variable W with i.i.d. entries $W_j \sim \mathcal{N}(0, \sigma_W^2)$, $\forall j \in \mathbb{I}_{n_\phi}$. After receiving the data $\mathcal{D}$, this prior distribution of W is updated. As a result, for each $x \in \mathbb{R}^{n_x}$, the learned GP predicts the value $f(x)$ as a (Gaussian) random variable $Y(x) \doteq W^\top \phi(x) \sim \mathcal{N}(\mu(x), \sigma_{\text{GP}}^2(x))$ with

$$\mu(x) = \mathbf{k}(x)^\top (\mathbf{K} + \sigma_M^2 \mathbf{I})^{-1} \mathbf{y} \quad (7.2a)$$

$$\sigma_{\text{GP}}^2(x) = k(x, x) - \mathbf{k}(x)^\top (\mathbf{K} + \sigma_M^2 \mathbf{I})^{-1} \mathbf{k}(x), \quad (7.2b)$$

where $k(x, x') \doteq \sigma_W^2 \cdot \phi(x)^\top \phi(x')$ is the evaluation of the kernel function for any pair $(x, x') \in \mathbb{R}^{n_x} \times \mathbb{R}^{n_x}$, $\mathbf{k}(x) \doteq [k(x_1, x) \ k(x_2, x) \ \dots \ k(x_D, x)]^\top$ is the column vector obtained by evaluating the kernel on the entire data $\mathbf{x}$, and $\mathbf{K}$ is the square Gram matrix with $\mathbf{K}_{ij} \doteq k(x_i, x_j)$ evaluated for all pairs of samples in $\mathbf{x}$. Here, $\mathbf{I}$ denotes the identity matrix of appropriate dimensions. Notice, since the noise is i.i.d., the random variable $\tilde{Y}(x) = Y(x) + M$ follows $\mathcal{N}(\mu(x), \tilde{\sigma}_{\text{GP}}^2(x))$, with

$$\tilde{\sigma}_{\text{GP}}^2(x) = \sigma_{\text{GP}}^2(x) + \sigma_M^2. \quad (7.3)$$

The close relation of the mean predictor (7.2a) to kernel ridge regression and to the representer theorem [17, 28] is well known [15, 26]. Indeed, the mean (7.2a) of the GP posterior is always within the RKHS specified by the kernel $k(\cdot, \cdot)$ since it is a linear combination of kernel evaluations centered at datapoints $\mathbf{x}$, cf. [3, Theorem 3]. Moreover, Eq. (7.2a) coincides with the solution to the kernel ridge regression when the ridge parameter is set to the noise variance σ_M^2 .

Problem statement. We consider the non-Gaussian extension to (7.1) which reads

$$y_i \doteq f(x_i) + M_i(\omega), \quad i \in \mathbb{I}_D, \quad M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R}), \quad (7.4)$$

i.e., the additive noise is modeled as an i.i.d. random variable $M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ of known—not necessarily Gaussian—distribution characterized by the probability measure $\mathbb{P}$. Given the dataset $\mathcal{D}$ obtained from (7.4) and supposing knowledge of the distribution of the additive measurement noise $M \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$, we are interested in deriving an extension to the classic GP predictor (7.2) which allows for non-Gaussian noise on the measurements. To this end, we exploit the Hilbert space structure of $\mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ equipped with a suitable inner product $\langle \cdot, \cdot \rangle_{\mathcal{L}^2}$.

Fundamentals of polynomial chaos expansions. PCE is an established method for uncertainty quantification and it dates back to Norbert Wiener [33]. It is based on the fact that any Hilbert space can be spanned by a polynomial basis, see Sullivan [29] for a general introduction. PCE is frequently considered in systems and control [16, 21, 25]; we refer to Faulwasser et al. [10], Pan [24] for applications in data-driven control. Consider an *orthonormal* polynomial basis $\{\varphi^j(\xi)\}_{j=0}^\infty$ with some argument $\xi \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$. The basis leads to the orthogonality relation

$$\langle \varphi^i(\xi), \varphi^j(\xi) \rangle_{\mathcal{L}^2} \doteq \int_{\omega \in \Omega} \varphi^i(\xi(\omega)) \varphi^j(\xi(\omega)) \, d\mathbb{P}(\omega) = \delta^{ij},$$

where δ^{ij} is the Kronecker delta. As $\{\varphi^j(\xi)\}_{j=0}^\infty$ spans the probability space $\mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$, any $V \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ can be expressed via the series expansion:

$$V = \sum_{j=0}^{\infty} \mathbf{v}^j \varphi^j(\xi) \quad \text{with} \quad \mathbf{v}^j = \langle V, \varphi^j(\xi) \rangle_{\mathcal{L}^2},$$

where $\mathbf{v}^j \in \mathbb{R}$ is called the j th PCE coefficient. It is customary when working with PCE to consider the first polynomial to be $\varphi^0 = 1$ and that all other polynomials $\varphi^j(\xi)$, $j \neq 0$ have zero mean. Indeed, depending on the considered distribution of V , one adapts the algebraic structure of the basis and its argument ξ [16, 34].

From a computational point of view, infinite PCE series are impractical, and hence only finitely many terms are usually considered. This, however, may induce

truncation errors [22]. It is well known that random variables following some widely used distributions admit exact finite-dimensional PCEs (i.e., with finitely many series terms) in suitable polynomial bases, see Xiu and Karniadakis [34].

Definition 7.1 (*Exact PCE representation* [22]) The PCE of a random variable $V \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ is said to be exact with dimension $L \in \mathbb{N}$ if $V - \sum_{j=0}^{L-1} \mathbf{v}^j \varphi^j(\xi) = 0$.

Given an exact PCE of finite dimension L , the expected value, the variance, and the covariance of entries in a vector-valued random variable $V \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R}^{n_v})$ with $n_v \in \mathbb{N}$ can be efficiently calculated from the PCE coefficients via

$$\mathbb{E}[V] = \mathbf{v}^0, \quad \mathbb{V}[V] = \sum_{j=1}^{L-1} (\mathbf{v}^j)^2, \quad \text{CoV}[V] = \sum_{j=1}^{L-1} \mathbf{v}^j \mathbf{v}^{j\top}, \quad (7.5)$$

where $\mathbf{v}^j \in \mathbb{R}^{n_v}$ and $(\mathbf{v}^j)^2 \doteq \mathbf{v}^j \circ \mathbf{v}^j$ is the Hadamard product [29]. Note that any $V \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{V})$ admits a two-term exact PCE representation $V = \mathbf{v}^0 \varphi^0(\xi) + \mathbf{v}^1 \varphi^1(\xi)$ with $\xi = (V - \mathbb{E}[V])/\sqrt{\mathbb{V}[V]}$, $\varphi^0(\xi) = 1$, and $\varphi^1(\xi) = \xi$, whereas $\mathbf{v}^0 = \mathbb{E}[V]$ and $\mathbf{v}^1 = \sqrt{\mathbb{V}[V]}$.

7.3 Main Results

Next, we consider the PCE approach to tackle the regression problem derived from the non-Gaussian setting (7.4). To this end, we suppose that the i.i.d. additive measurement noise M_i entails an exact PCE of dimension L , i.e.: $M_i = \sum_{j=0}^{L-1} \mathbf{m}^j \varphi^j(\xi_i) \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$. Note that for all $i \in \mathbb{I}_D$, the PCE coefficients $\mathbf{m}^j$ are identical due to the i.i.d.-ness of the random variables M_i .

Regression with $\mathcal{L}^2$ noise description. Similar to usual kernel regression problems, we consider the linear ansatz via feature map $\phi : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_\phi}$, i.e., we presume the structure $f(x) = w^\top \phi(x)$. With the additive noise in $\mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$, we construct the following least-square (ℓ^2) loss regression problem:

$$\min_{w \in \mathbb{R}^{n_\phi}} \sum_{i=1}^D (y_i - M_i(\omega) - w^\top \phi(x_i))^2 + \rho^2 \cdot \|w\|^2.$$

The problem above includes the quadratic regularization term $\rho^2 \cdot \|w\|^2$ with the Euclidean norm $\|\cdot\|$ weighted by some constant ridge parameter $\rho^2 > 0$. Note that for each data sample, the ℓ^2 loss includes the unknown noise realizations $M_i(\omega) \in \mathbb{R}$. One possible workaround would be to neglect the realizations of the measurement noise and to work with the corrupted measurements y_i . Alternatively, we consider the random variable description of $M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$. Rewriting the objective in the expected value sense of statistical learning yields

$$\min_{W \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R}^{n_\phi})} \sum_{i=1}^D \mathbb{E} \left[(y_i - M_i - W^\top \phi(x_i))^2 \right] + \rho^2 \cdot \mathbb{E} [\|W\|^2]. \quad (7.6)$$

Recall that the data $\mathcal{D}$ consists of real-valued tuples (x_i, y_i) while in the regression problem (7.6), the measurement noise $M_i, i \in \mathbb{I}_D$ is considered as i.i.d. $\mathcal{L}^2$ random variables. Hence, the weights W are also lifted to this space. It is easy to see that if one does lift the decision variables W to $\mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R}^{n_\phi})$, then in the first-order optimality condition, the realization of W matches the measurement uncertainty M_i to reduce the expected loss. We refer to Bienstock et al. [4] for a similar discussion of equality constraints in stochastic optimization problems for energy systems.

Wiener kernel regression. To avoid the technicalities of solving the regression in the infinite-dimensional $\mathcal{L}^2$ space directly, we use the PCE representation of the noise $M_i, i \in \mathbb{I}_D$ and of the weights W . In particular, the weights $W = [W_1 \dots W_{n_\phi}]^\top$ are component-wise presented in the joint scalar basis $\{\psi^j\}_{j=0}^{L_W-1}$ with

$$\psi^0 \doteq \varphi^0 = 1 \text{ and } \psi^j \doteq \varphi^{[(j-1) \bmod (L-1)]+1}(\xi_{[(j-1) \bmod (L-1)]+1}), \quad \forall j \in \mathbb{I}_{L_W-1}.$$

The basis is of dimension $L_W \doteq D \cdot (L-1) + 1$; it includes the bases for all $M_i, i \in \mathbb{I}_D$. Put differently, we consider one basis polynomial ψ^0 for the mean and $D \cdot (L-1)$ basis polynomials for the non-mean parts of $W \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R}^{n_\phi})$ corresponding to each data sample i . We note that the i.i.d. property of $M_i, i \in \mathbb{I}_D$ is reflected in the fact that the bases for M_i and M_k differ in their arguments ξ_i and ξ_k . This way, two random variables of the same distribution—and hence living in the same $\mathcal{L}^2$ space—can be distinguished in their PCE representations, while their PCE coefficients are equal. Using the joint basis $\{\psi^j\}_{j=0}^{L_W-1}$ and keeping the ridge parameter $\rho^2 > 0$ from (7.6), we arrive at the PCE-reformulated problem:

$$\begin{aligned} \min_{\substack{\mathbf{w}^j \in \mathbb{R}^{n_\phi}, \\ j \in \{0, \dots, L_W-1\}}} \sum_{i=1}^D \mathbb{E} \left[\left(y_i - \sum_{j=0}^{L-1} \mathbf{m}^j \varphi^j(\xi_i) - \sum_{j=0}^{L_W-1} \phi(x_i)^\top \mathbf{w}^j \psi^j \right)^2 \right] \\ + \rho^2 \cdot \mathbb{E} \left[\left\| \sum_{j=0}^{L_W-1} \mathbf{w}^j \psi^j \right\|^2 \right], \end{aligned} \quad (7.7)$$

where $W \doteq \sum_{j=0}^{L_W-1} \mathbf{w}^j \psi^j$ is the PCE representation of $W \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R}^{n_\phi})$ with vectorized PCE coefficients $\mathbf{w}^j \in \mathbb{R}^{n_\phi}$ and scalar basis functions $\psi^j \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ which we substitute into (7.6). We refer to the reformulated problem (7.7) as *Wiener kernel regression*. Its solution is given in the following lemma.

Lemma 7.1 (Correspondence of optimization problems) *Suppose that the i.i.d. additive measurement noise $M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R}), i \in \mathbb{I}_D$, admits an exact PCE of dimension $L = 2$, i.e., $M_i = \mathbf{m}^0 + \mathbf{m}^1 \varphi^1(\xi_i)$. Then, the following statements hold:*

- (i) The optimal solution to (7.6) is unique in the $\mathcal{L}^2$ sense and given by $\mathbf{W}^* = \sum_{j=0}^{L_W-1} \mathbf{w}^{j*} \varphi^j$ with $L_W - 1 = D$, whereby the PCE coefficients $\mathbf{w}^{j*}$ are optimal in (7.7).
- (ii) The unique optimal solution to (7.7) with $\rho^2 > 0$ from (7.6) is given by

$$\mathbf{w}^{j*} = \Phi(\mathbf{K} + \rho^2 \mathbf{I})^{-1} \mathbf{y}^j, \quad j \in \{0, \dots, D\}, \quad (7.8a)$$

where Φ is a $n_\phi \times D$ matrix, whose i th column is defined by $\phi(x_i)$, $i \in \mathbb{I}_D$, $\mathbf{K}$ is the $D \times D$ kernel matrix from (7.2), and

$$\mathbf{y}^0 = \mathbf{y} - \mathbf{m}^0 \mathbf{1}_D \quad \text{and} \quad \mathbf{y}^j = -e_j \mathbf{m}^1 \quad \text{for } j \geq 1, \quad (7.8b)$$

with $\mathbf{1}_D \doteq [1 \dots 1]^\top \in \mathbb{R}^D$ and e_j being the j th Euclidean basis vector of $\mathbb{R}^D$.

Proof Due to space limitations we only sketch the main steps of the proof. The assumption of an exact PCE of dimension $L = 2$ for all M_i gives that $L_W = D + 1$, i.e., the PCE for W reads $\mathbf{w}^0 + \sum_{j=1}^D \mathbf{w}^j \varphi^1(\xi_j)$ and its dimension corresponds to the number of available data samples plus one extra dimension for the mean $\mathbf{w}^0$. The first part of the objective in (7.7) with $L = 2$ can be rewritten as

$$\sum_{i=1}^D \mathbb{E} \left[\left([y_i - \mathbf{m}^0 - \phi(x_i)^\top \mathbf{w}^0] + \sum_{j=1}^D [-\delta^{ij} \mathbf{m}^1 - \phi(x_i)^\top \mathbf{w}^j] \varphi^1(\xi_j) \right)^2 \right].$$

Similar to the objective reformulations considered for stochastic data-driven optimal control by Faulwasser et al. [10], we exploit that the measurement noise is i.i.d. and that all PCE basis functions with $j > 0$ have zero mean. Next we set $\mathbf{y}_i^0 \doteq y_i - \mathbf{m}^0$ and $\mathbf{y}_i^j \doteq -\delta^{ij} \mathbf{m}^1$ according to (7.8b). Utilizing that $\mathbb{E}[V^2] = (\mathbb{E}[V])^2 + \mathbb{V}[V]$ for any $V \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ and using (7.5), we rewrite the objective in (7.7) as

$$\sum_{i=1}^D [\mathbf{y}_i^0 - \phi(x_i)^\top \mathbf{w}^0]^2 + \rho^2 \cdot \|\mathbf{w}^0\|^2 + \sum_{j=1}^D \left[\sum_{i=1}^D [\mathbf{y}_i^j - \phi(x_i)^\top \mathbf{w}^j]^2 + \rho^2 \cdot \|\mathbf{w}^j\|^2 \right],$$

where we also exploit that (7.7) is an unconstrained minimization of a strictly convex quadratic function over a real-valued vector space of finite dimension, and hence there exists a unique minimizer. Observe that without loss of generality, we swapped the summations in the equation above. Hence, the above problem can be solved for each PCE dimension j individually. Indeed, for each PCE dimension j it corresponds to a usual kernel regression with regularized ℓ^2 loss. Hence, we obtain (7.8a) and the computation of $\mathbf{y}^j$ follows from the PCE dimension $L = 2$ and the i.i.d.-ness of the random variables M_i , $i \in \mathbb{I}_D$. This proves Assertion ii).

Assertion (i) is shown similar to Pan et al. [24, Proposition 1] via contradiction: Recall the assumption of exact PCEs for M_i , $i \in \mathbb{I}_D$ and the corresponding joint PCE basis. Now consider an extension of the joint basis for M_i and W . Suppose that the optimal solution admits non-zero coefficients $\mathbf{w}^{j*}$ beyond the first $L_W = D + 1$ basis

functions. These non-zero coefficients strictly increase the considered ℓ^2 -objective as the corresponding coefficients for $M_i : m^j, j > D$ are zero. $\square$

Under the assumptions of the above lemma, we can now give the Wiener kernel regression for the non-Gaussian setting of (7.4). The $\mathcal{L}^2$ -optimal prediction is given by the scalar random variable $\hat{Y}(x) \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ as

$$\hat{Y}(x) = \mathbf{k}(x)^\top (\mathbf{K} + \rho^2 \mathbf{I})^{-1} \left[\mathbf{y}^0 + \sum_{j=1}^D \mathbf{y}^j \varphi^1(\xi_j) \right]. \quad (7.9)$$

Observe that $\hat{Y}(x)$ is given as a linear combination of finitely many scalar basis functions $\varphi^1(\xi_j)$ of the underlying probability space $\mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$. Moreover, the term $\mathbf{k}(x)^\top (\mathbf{K} + \rho^2 \mathbf{I})^{-1}$ is identical for all PCE dimensions. Notice that the above result easily generalizes to orthogonal (but not normal) PCE basis functions and to the case of $L \in \mathbb{N}$ and other convex loss functions. Notice that the PCE basis dimensions of $W \doteq \sum_{j=0}^{L_W-1} \mathbf{w}^j \psi^j$ in (7.7) and of the estimate $\hat{Y}(x)$ in (7.9) are proportional to the product of L and D . Put differently, for $L > 2$ additional terms appear in the sum in (7.9).

Quantification of aleatoric and epistemic uncertainty. The distinction of epistemic and aleatoric uncertainties is an established concept in ML [13, 31]. Specifically, Hüllermeier and Waegeman [13] identify the GP posterior variance $\sigma_{\text{GP}}^2(x)$ in (7.2b) as epistemic uncertainty and the variance σ_M^2 of the additive measurement noise—included in (7.3)—as the one related to aleatoric uncertainty. While the former variance refers to the uncertainty due to a lack of model knowledge/data, the latter refers to the uncertainty which is inherently due to randomness (measurement noise M) and cannot be eliminated even with infinite samples at every point in $\mathbb{R}^{n_x}$. Hence, it remains to discuss the differences and commonalities of the classic GP prediction (7.2) and our Wiener kernel approach (7.9). The relation between moments and PCE coefficients in (7.5) leads to the following result.

Lemma 7.2 (Moments of Wiener kernel predictors) *Suppose that, for all $i \in \mathbb{I}_D$, the i.i.d. additive measurement noise $M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ has an exact PCE of dimension $L = 2$. Then, we have*

$$(i) \quad \mathbb{E}[\hat{Y}(x)] = \mathbf{k}(x)^\top (\mathbf{K} + \rho^2 \mathbf{I})^{-1} \mathbf{y}^0, \quad \mathbf{y}^0 = \mathbf{y} - \mathbf{m}^0 \mathbf{1}_D. \quad (7.10a)$$

(ii) *Specifically, for $M_i \sim \mathcal{N}(0, \sigma_M^2)$ the choice $\rho^2 = \sigma_M^2$ implies that the GP mean value prediction from (7.2a) is equivalent to $\mathbb{E}[\hat{Y}(x)]$.*

(iii) *Moreover, for any i.i.d. $M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$, $i \in \mathbb{I}_D$ it holds that*

$$\mathbb{V}[\hat{Y}(x)] = \sigma_M^2 \mathbf{k}(x)^\top (\mathbf{K} + \rho^2 \mathbf{I})^{-2} \mathbf{k}(x), \quad \sigma_M^2 = (\mathbf{m}^1)^2. \quad (7.10b)$$

Proof Due to space constraints, we sketch only the main steps. (i) Follows directly from (7.5). Notice that in the (normalized) Hermite polynomial series, a zero-mean Gaussian $\mathcal{N}(0, \sigma_M^2)$ can be represented via PCE with $L = 2$ while the coefficients of M_i are $\mathbf{m}^0 = 0$ and $\mathbf{m}^1 = \sigma_M$, which shows (ii). To show (iii), we apply the variance formula from (7.5) to (7.9). This yields

$$\mathbb{V}[\widehat{Y}(x)] = \mathbf{k}(x)^\top (\mathbf{K} + \rho^2 \mathbf{I})^{-1} \text{CoV} \left[\sum_{j=1}^D \mathbf{y}^j \varphi^1(\xi_j) \right] (\mathbf{K} + \rho^2 \mathbf{I})^{-1} \mathbf{k}(x),$$

which—after evaluating the above covariance term—simplifies to (7.10b). $\square$

The careful reader has surely recognized the difference of the classic GP variance $\sigma_{\text{GP}}^2(x)$ from (7.2b) and the variance of the optimal PCE solution to the Wiener kernel regression (7.10b). Notice that both expressions do not depend on the actually acquired measurements $\mathbf{y}$ but only on the data samples $\mathbf{x}$.

It is well known that the classic GP variance (7.2b) corresponds to the Gaussian distribution of the next prediction conditioned on the data $\mathcal{D}$ via the covariance encoded in the kernel $k(\cdot, \cdot)$. Recall that in Hüllermeier and Waegeman [13], Eq. (7.2b) is viewed as a proxy for epistemic uncertainty. Indeed, a formal analysis shows that in the infinite data limit and for Lipschitz continuous kernels, $\sigma_{\text{GP}}^2(x) \rightarrow 0$ holds [15, 19]. This means that (7.2b) vanishes with sufficiently large (and rich) datasets $\mathcal{D}$. In the case of white additive measurement noise, however, the additive structure of (7.3) implies that the variance $\tilde{\sigma}_{\text{GP}}^2(x)$ is fundamentally bounded from below by the variance of the measurement noise σ_M^2 ; hence, the association of σ_M^2 with aleatoric uncertainty.

An interesting observation is that despite being described as epistemic uncertainty, $\sigma_{\text{GP}}^2(x)$ in (7.2b) still depends on the noise-related σ_M^2 , which comes from the fact that the GP posterior (7.2) is inferred from the noisy data (7.1). Since the noise is assumed to be i.i.d., this, in turn, raises a question of the decomposition of (7.2b) into purely model-dependent and purely noise-related variance terms. In the case of the i.i.d. Gaussian noise (7.1), we argue that one avenue toward factoring out the noise-related term of (7.2b) is paved by the variance estimate

$$\sigma_{\text{WK}}^2(x) \doteq \sigma_M^2 \mathbf{k}(x)^\top (\mathbf{K} + \sigma_M^2 \mathbf{I})^{-2} \mathbf{k}(x), \quad (7.11)$$

derived via (7.10b) by setting $\rho^2 \doteq \sigma_M^2$ such that $\mathbb{E}[Y(x)] = \mathbb{E}[\widehat{Y}(x)]$, see Lemma 7.2.

Indeed, (7.10b) can be understood as the result of propagating the (not necessarily Gaussian) measurement noise $M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ through the linear mean value predictor (7.2a). A rigorous convergence analysis of (7.10b) in view of the available data is open at this point. As a first step, the following result shows that (7.10b) converges to zero at any point $\bar{x} \in \mathbb{R}^{n_x}$ as the number $N \in \mathbb{N}$ of samples (7.4) taken at $\bar{x}$ goes to infinity.

Lemma 7.3 (Asymptotics of Wiener kernel variance) *Suppose that after collecting $\mathcal{D}$ via (7.4), one has N repeated samples at some $\bar{x} \in \mathbb{R}^{n_x}$. Denote the resulting dataset as $\mathcal{D}' = \mathcal{D} \cup \bar{\mathcal{D}}$, where $\bar{\mathcal{D}} \doteq \{(\bar{x}, f(\bar{x}) + M_{D+i}(\omega)) \in \mathbb{R}^{n_x} \times \mathbb{R} \mid \forall i \in \mathbb{I}_N\}$ are the N repeated measurements of f at $\bar{x}$. Let $\mathbb{V}_N(x) \doteq \sigma_{\text{WK}}^2(x)$ be the variance (7.10b) based on the data in $\mathcal{D}'$ and with $\rho^2 > 0$. Then,*

$$\lim_{N \rightarrow \infty} \mathbb{V}_N(\bar{x}) \leq \lim_{N \rightarrow \infty} \frac{\sigma_M^2}{N} = 0.$$

Proof For $\bar{x} \in \mathbb{R}^{n_x}$, we have $\mathbb{V}_N(\bar{x}) = \sigma_M^2 \mathbf{k}_N(\bar{x})^\top (\mathbf{K}_N + \rho^2 \mathbf{I})^{-2} \mathbf{k}_N(\bar{x})$, where $\mathbf{k}_N$ and $\mathbf{K}_N$ contain the evaluations of the positive semi-definite kernel $k(\cdot, \cdot)$ based on $\mathcal{D}'$. For the Gram matrix $\mathbf{K}_N$, we have

$$\mathbf{K}_N = \begin{bmatrix} \mathbf{K} & \bar{\mathbf{K}} \\ \bar{\mathbf{K}}^\top & \mathbf{K}_{\bar{\mathcal{D}}} \end{bmatrix},$$

with $\bar{\mathbf{K}} \doteq [\mathbf{k}(\bar{x}) \dots \mathbf{k}(\bar{x})] \in \mathbb{R}^{D \times N}$ and $\mathbf{K}_{\bar{\mathcal{D}}} \doteq k(\bar{x}, \bar{x}) \cdot \mathbf{1}_{N \times N}$, whereas $\mathbf{k}_N(\bar{x}) \doteq [\mathbf{k}(\bar{x})^\top k(\bar{x}, \bar{x}) \dots k(\bar{x}, \bar{x})]^\top \in \mathbb{R}^{D+N}$. Clearly, for any $N \in \mathbb{N}$, the linear system $\mathbf{K}_N b = \mathbf{k}_N(\bar{x})$ is under-determined since $b \doteq [\mathbf{0}_D^\top \alpha^\top]^\top$ with arbitrary $\alpha \in \mathbb{R}^N$, $\alpha^\top \mathbf{1}_N = 1$, is a solution. In particular, if we set $\alpha \doteq (1/N) \cdot \mathbf{1}_N$, then $\|b\|^2 = 1/N$ holds. This choice allows us to derive the following inequality:

$$\begin{aligned} \sigma_M^2 \mathbf{k}_N(\bar{x})^\top (\mathbf{K}_N + \rho^2 \mathbf{I})^{-2} \mathbf{k}_N(\bar{x}) &= \sigma_M^2 \left\| (\mathbf{K}_N + \rho^2 \mathbf{I})^{-1} \mathbf{K}_N b \right\|^2 \leq \\ \sigma_M^2 \left\| (\mathbf{K}_N + \rho^2 \mathbf{I})^{-1} \mathbf{K}_N \right\|^2 \|b\|^2 &= \frac{\sigma_M^2 (\lambda_{\max}[\mathbf{K}_N])^2 \|b\|^2}{(\lambda_{\max}[\mathbf{K}_N] + \rho^2)^2} \leq \sigma_M^2 \|b\|^2 = \frac{\sigma_M^2}{N}. \end{aligned}$$

The matrix norm $\|\cdot\|$ in the equation above is induced by the usual Euclidean norm. For $A \in \mathbb{R}^{n \times n}$ we have $\|A\| = \sqrt{\lambda_{\max}[A^\top A]}$, where $\lambda_{\max}[\cdot]$ denotes the largest eigenvalue. Recall that $\|Ab\| \leq \|A\| \|b\|$ holds for any $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$. Thus, with the upper bound from the above equation, we obtain $0 \leq \mathbb{V}_N(\bar{x}) \leq \sigma_M^2/N$ which proves the assertion. $\square$

Lemma 7.3 shows that assuming the knowledge of the distribution of the i.i.d. noise $M_i \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$, the value $f(\bar{x})$ can be exactly determined if one takes infinitely many samples at $\bar{x}$, i.e., $\lim_{N \rightarrow \infty} \{y_i = f(\bar{x}) + M_i(\omega), i \in \mathbb{I}_N\}$. Indeed, in this case, from the central limit theorem, we have that

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N y_i = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N (f(\bar{x}) + M_i(\omega)) = \mathbb{E}[f(\bar{x}) + M] = f(\bar{x}) + \mathbf{m}^0.$$

Thus, one can find $f(\bar{x})$ via $f(\bar{x}) = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N y_i - \mathbf{m}^0$, where $\mathbf{m}^0$ is known. Moreover, whenever one can estimate an upper bound of σ_M , Lemma 7.3 provides

an avenue to estimate the number of samples to be taken at a point $\bar{x}$ in order to meet a target variance $\sigma_{\text{WK}}^2(x)$ at $\bar{x}$ for the learned model.

Interpretation. The Wiener kernel variance (7.11) expresses the uncertainty in the predictions computed at a previously visited point $\bar{x}$ associated exclusively with the noise in the labels/targets $\mathbf{y}$ in the available data $\mathcal{D}$ (Lemma 7.3). In other words, it quantifies the effect of the aleatoric uncertainty in the measurements $\mathbf{y}$ taken at a point x on the epistemic uncertainty of the learned model evaluated at the same point x . Moreover, our numerical experiments in Sect. 7.4 suggest that $\sigma_{\text{GP}}^2(x)$ in (7.2b) and $\sigma_{\text{WK}}^2(x)$ in (7.11) satisfy $\sigma_{\text{WK}}^2(x) \leq \sigma_{\text{GP}}^2(x)$ for all $x \in \mathbb{R}^{n_x}$. However, the formal analysis of this relation remains an open question.

7.4 Numerical Example

We consider the following scalar discrete-time GP state-space model from Beckers and Hirche [2]

$$x_{k+1} = 0.01x_k^3 - 0.2x_k^2 + 0.2x_k + M_k(\omega), \quad (7.12)$$

where $x_k \in \mathbb{R}$ denotes the state at time $k \in \mathbb{N}$ and the additive noise $M_k \in \mathcal{L}^2(\Omega, \mathcal{F}, \mathbb{P}; \mathbb{R})$ is i.i.d. First we assume that $M_k, k \in \mathbb{I}_D$ follow the standard normal distribution $\mathcal{N}(0, 1^2)$, i.e., $\mathbf{m}^0 = 0$ and $\mathbf{m}^1 = 1$ in the (normalized) Hermite basis. We extend to a non-Gaussian case later. Given exact measurements of x_k , we are interested in predicting the next system state x_{k+1} while the right-hand side function in (7.12) is not known. We use a squared exponential kernel $k(x, x') \doteq \sigma_f^2 \exp(-\|x - x'\|^2 / (2l^2))$ with $\sigma_f = 4.21$ and $l = 3.59$ obtained in Beckers and Hirche [2] through maximization of the marginal likelihood.

Gaussian measurement noise. We train the above-mentioned GP using data $\mathcal{D}$ obtained at N_x different linearly spaced x -locations within $[-5, 5]$. The results are depicted in Fig. 7.1, where data points are marked by *black crosses*. The first row of plots (A)–(C) shows results in which the data was obtained at $N_x \in \{2, 3, 5\}$ different x -locations. This experiment is repeated by increasing the number of samples $N_{\text{sam}} \in \{1, 5, 25\}$ per x -location in plots (A)–(C), (D)–(F), and (G)–(I).

In each of the plots, the *blue solid line* indicates the true function from (7.12) without measurement noise M . The *green-shaded tubes* are calculated via $\mu(x) \pm \sigma_{\text{WK}}(x)$ where $\mu(x)$ corresponds to the predicted mean (*red solid line*) (7.2a)/(7.10a), and $\sigma_{\text{WK}}(x)$ is the standard deviation derived from the variance (7.11) of the Wiener kernel regression. The GP counterpart of this uncertainty is shown using *dark-gray tubes* $\mu(x) \pm \sigma_{\text{GP}}(x)$ where $\sigma_{\text{GP}}(x)$ is from (7.2b). Finally, *light-gray tubes* illustrate $\mu(x) \pm \tilde{\sigma}_{\text{GP}}(x)$, where $\tilde{\sigma}_{\text{GP}}^2(x)$ is the GP posterior variance with added measurement noise, cf. Eq. (7.3).

We note that both of the considered GP variance tubes are larger than the green tube obtained via the Wiener kernel regression. While the usual GP posterior variance $\sigma_{\text{GP}}^2(x)$ quantifies the uncertainty associated both with the incomplete exploration of

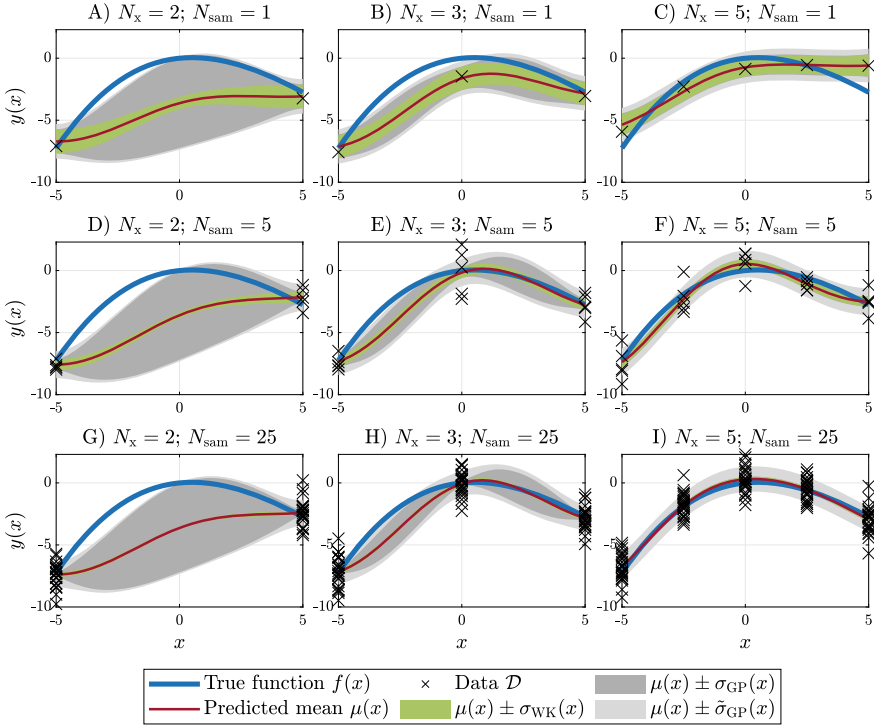


Fig. 7.1 Comparison of the variance estimates from GP and Wiener kernel regression

the hypothesis space (i.e., the set of all possible GP posterior sample functions) and with the measurement noise contained in the collected data $\mathcal{D}$, the Wiener kernel estimate $\sigma_{\text{WK}}^2(x)$ quantifies exclusively the influence of the measurement noise. Specifically, in the considered numerical example, plots (A), (D), and (G) of Fig. 7.1 show that as the number of samples (N_{sam}) increases, the noise-associated uncertainty—i.e., the green tube—shrinks (cf. Lemma 7.3), while the dark-gray tube of the GP posterior remains.

When five linearly spaced locations are considered, the spacing between the neighboring x -locations in $\mathcal{D}$ reduces to 2.5, which can be covered by the lengthscale l of k . This results in a much better exploration of the hypothesis space. As shown in plot C), the dark-gray area coincides with the noise-related green-colored uncertainty tube. As we eliminate this noise-related uncertainty by increasing the number of samples per x -location, we notice that our kernel regression result improves dramatically, cf. plots (F) and (I) in Fig. 7.1. However, the light-gray tube associated with the variance $\tilde{\sigma}_{\text{GP}}^2(x)$ remains quite large due to its additive structure (7.3), which includes the aleatoric uncertainty σ_M^2 . We observe similar behavior using other hyperparameter values and kernels such as polynomial and exponential (results not shown here).

Non-Gaussian noise. We extend (7.12) to a non-Gaussian setting, i.e., we consider the i.i.d. noise M_k , $k \in \mathbb{I}_D$ to follow a γ -distribution $\text{Gamma}(\alpha, \beta; x) = (1/\Gamma(\alpha)) \beta^{-\alpha} x^{\alpha-1} \exp(-x/\beta)$ in which Γ denotes the Gamma function. Unlike Gaussians, this distribution allows for asymmetry, i.e., non-zero skewness. We set the parameters to $\alpha \doteq 0.25$ and $\beta \doteq 2$. Thus, the mean of this distribution is $\mu_M = \alpha\beta = 0.5$, while the variance reads $\sigma_M^2 = \alpha\beta^2 = 1$. To comply with Lemma 7.1, for each M_i , $i \in \mathcal{I}$ with $\xi_i \sim \text{Gamma}(0.25, 2)$ we choose the (orthonormal) basis of dimension $L = 2$ with $\varphi^0(\xi_i) \doteq 1$ and $\varphi^1(\xi_i) \doteq (\xi_i - \mu_M)/\sigma_M$. The corresponding PCE coefficients are $\mathbf{m}^0 \doteq \mu_M = 0.5$ and $\mathbf{m}^1 \doteq \sigma_M = 1$.

Next we use the same kernel as before to compute $\hat{Y}(x)$ via Wiener kernel regression (7.9). The linearly spaced training data $\mathcal{D}$ is chosen within $x \in [-5, 5]$ with $N_x = 5$ and $N_{\text{sam}} = 1$. We observe that the distribution of $\hat{Y}(x)$ is modeled via a linear combination of $D = 5$ i.i.d. γ -distributed random variables. In general, it is difficult to characterize the underlying probability density function analytically. To obtain a numerical approximation, we sample the individual arguments ξ_i of the series expansion in (7.9). This way we generate a collection of function realizations $\mathcal{C}(\hat{Y}) = \{\hat{Y}(x; \omega) \mid \omega \in \Omega_0\}$ for some (finite) subset of outcomes $\Omega_0 \subset \Omega$. The left plot of Fig. 7.2 depicts these functions as *pink solid lines* with the true function $f(x)$ (*blue solid line*) and the predicted mean $\mu(x)$ (*red solid line*) of the kernel regression. Using $\mathcal{C}(\hat{Y})$, one can create a histogram and a corresponding probability density function (PDF) fit at the selected x -locations. In the left plot of Fig. 7.2, these fitted PDFs are shown as *red solid lines* at the x -locations of the data $\mathcal{D}$ using $|\mathcal{C}(\hat{Y})| = 5000$ samples. On the right-hand side of Fig. 7.2, we project onto the plane corresponding to $x = 0$. Here, we plot the histogram of $\mathcal{C}(\hat{Y}(x = 0))$ and its fitted PDF as a *red solid line*. For the sake of comparison, we also plot the Gaussians $\mathcal{N}(\mu(x = 0), \sigma_{\text{GP}}^2(x = 0))$ and $\mathcal{N}(\mu(x = 0), \sigma_{\text{WK}}^2(x = 0))$, which would define the uncertainty tubes stemming from the GP and the Wiener kernel regression, respectively. Similar to the plots in Fig. 7.1, we also present the distribution

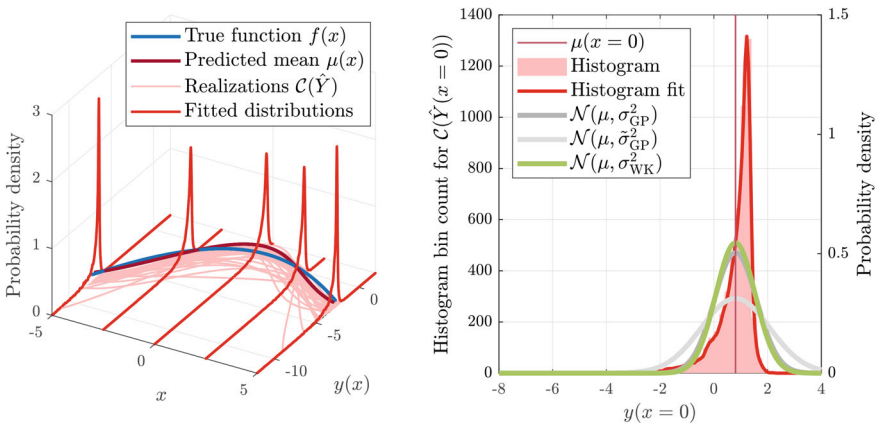


Fig. 7.2 Example (7.12) with γ -distributed measurement noise

$\mathcal{N}(\mu(x=0), \tilde{\sigma}_{\text{GP}}^2(x=0))$. As seen from the right-hand side of Fig. 7.2, the fitted distribution of $\hat{Y}(x)$ is asymmetric. That is, the skewed measurement noise can be captured and quantified via (7.10b).

7.5 Conclusions and Outlook

The present paper discussed kernel ridge regression with training data corrupted by i.i.d. additive non-Gaussian noise of finite variance. Relying on the framework of Polynomial Chaos Expansions (PCE), we introduced a novel framework for Wiener kernel regression which handles the non-Gaussian measurement noise effectively. When the ridge parameter value is set to the variance of the i.i.d. noise, the mean predictions of standard GPs and the proposed Wiener kernel regression coincide. The variance predictions differ, however, in their meaning. While the posterior GP variance is a measure of total epistemic uncertainty caused both by the insufficient exploration of the hypothesis space and due to the noisy data, the Wiener kernel variance only measures the effect of the (aleatoric) noise in the data samples on the prediction. We illustrate this uncertainty decomposition via numerical examples where we compare both of the variance estimates.

Our preliminary results point toward further research. As for the presented Wiener kernel regression, there is a need for investigations regarding the properties of the derived variance (7.10b) concerning its analytic relation to (7.2b). The interpretation of (7.10b) in the context of computational uncertainty [32] remains to be done. While this work considered the weight-space view on GPs, the extension to the function space view is still an open problem. Tailored methods for hyperparameter tuning under non-Gaussian noise and the consideration of noise corruption on the sample locations $\mathbf{x}$ are another question for future research. The link between our results and results available for additive sub-Gaussian noise [7] has to be analyzed. Finally, the proposed approach might provide new avenues for handling the exploration–exploitation trade-off in Bayesian optimization under non-Gaussian noise.

Acknowledgements The authors thank Ruchuan Ou for helpful feedback on the implementation.

References

1. Aronszajn, N.: Theory of reproducing Kernels. *Trans. Am. Math. Soc.* **68**(3), 337–404 (1950)
2. Beckers, T., Hirche, S.: Equilibrium distributions and stability analysis of Gaussian process state space models. In: *IEEE 55th Conference on Decision and Control (CDC)* (2016)
3. Berlinet, A., Thomas-Agnan, C.: *Reproducing Kernel Hilbert Spaces in Probability and Statistics*. Springer, New York, NY (2004)
4. Bienstock, D., Chertkov, M., Harnett, S.: Chance-constrained optimal power flow: risk-aware network control under uncertainty. *SIAM Rev.* **56**(3), 461–495 (2014)

5. Capone, A., Hirche, S., Pleiss, G.: Sharp calibrated Gaussian processes. *Adv. Neural Inform. Process. Syst.* **36** (2024)
6. Chen, N., Qian, Z., Nabney, I., Meng, X.: Wind power forecasts using Gaussian processes and numerical weather prediction. *IEEE Trans. Power Syst.* **29**(2), 656–665 (2013)
7. Chen, Y., Wang, Y., Zhu, K.: On generalized sub-Gaussian canonical processes and their applications (2024). Preprint [arXiv:2401.15314](https://arxiv.org/abs/2401.15314)
8. Deisenroth, M.: *Efficient Reinforcement Learning Using Gaussian Processes*, vol. 9. KIT Scientific Publishing (2010)
9. Drgoña, J., Arroyo, J., Figueroa, I., Blum, D., Arendt, K., Kim, D., Ollé, E., Oravec, J., Wetter, M., Vrabie, D., et al.: All you need to know about model predictive control for buildings. *Ann. Rev. Control* **50**, 190–232 (2020)
10. Faulwasser, T., Ou, R., Pan, G., Schmitz, P., Worthmann, K.: Behavioral theory for stochastic systems? A data-driven journey from Willems to Wiener and back again. *Ann. Rev. Control* **55**, 92–117 (2023)
11. Gratiet, L., Marelli, S., Sudret, B.: *Metamodel-based sensitivity analysis: polynomial chaos expansions and Gaussian processes* (2016). Preprint [arXiv:1606.04273](https://arxiv.org/abs/1606.04273)
12. Heo, Y., Zavala, V.: Gaussian process modeling for measurement and verification of building energy savings. *Energy Build.* **53**, 7–18 (2012)
13. Hüllermeier, E., Waegeman, W.: Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Mach. Learn.* **110**(3), 457–506 (2021)
14. Kailath, T.: RKHS approach to detection and estimation problems-I: deterministic signals in Gaussian noise. *IEEE Trans. Inform. Theory* **17**(5), 530–549 (1971)
15. Kanagawa, M., Hennig, P., Sejdinovic, D., Sriperumbudur, B.: Gaussian processes and kernel methods: a review on connections and equivalences (2018). Preprint [arXiv:1807.02582](https://arxiv.org/abs/1807.02582)
16. Kim, K., Shen, D., Nagy, Z., Braatz, R.: Wiener’s polynomial chaos for the analysis and control of nonlinear dynamical systems with probabilistic uncertainties. *IEEE Control Syst.* **33**(5), 58–67 (2013)
17. Kimeldorf, G., Wahba, G.: A correspondence between Bayesian estimation on stochastic processes and smoothing by splines. *Ann. Math. Stat.* **41**(2), 495–502 (1970)
18. Kocijan, J., Murray-Smith, R., Rasmussen, C., Likar, B.: Predictive control with Gaussian process models. In: *The IEEE Region 8 EUROCON 2003. Computer as a Tool*, vol. 1, pp. 352–356 (2003)
19. Lederer, A., Umlauf, J., Hirche, S.: Posterior variance analysis of Gaussian processes with application to average learning curves (2019). Preprint [arXiv:1906.01404](https://arxiv.org/abs/1906.01404)
20. Van der Meer, D., Shepero, M., Svensson, A., Widén, J., Munkhammar, J.: Probabilistic forecasting of electricity consumption, photovoltaic power generation and net demand of an individual building using Gaussian processes. *Appl. Energy* **213**, 195–207 (2018)
21. Mesbah, A., Streif, S., Findeisen, R., Braatz, R.: Stochastic nonlinear model predictive control with probabilistic constraints. In: *2014 American Control Conference*, pp. 2413–2419. IEEE (2014)
22. Mühlplfordt, T., Findeisen, R., Hagenmeyer, V., Faulwasser, T.: Comments on truncation errors for polynomial chaos expansions. *IEEE Control Syst. Lett.* **2**(1), 169–174 (2018)
23. Murray-Smith, R., Johansen, T., Shorten, R.: On transient dynamics, off-equilibrium behaviour and identification in blended multiple model structures. In: *1999 European Control Conference (ECC)*, pp. 3569–3574 (1999)
24. Pan, G., Ou, R., Faulwasser, T.: On a stochastic fundamental lemma and its use for data-driven optimal control. *IEEE Trans. Automat. Contr.* **68**, 5922–5937 (2023)
25. Paulson, J., Mesbah, A., Streif, S., Findeisen, R., Braatz, R.: Fast stochastic model predictive control of high-dimensional systems. In: *53rd IEEE Conference on Decision and Control*, pp. 2802–2809. IEEE (2014)
26. Rasmussen, C., Williams, C.: *Gaussian Processes for Machine Learning*. Adaptive Computation and Machine Learning, The MIT Press (2006)
27. Rose, A., Pfefferkorn, M., Nguyen, H.H., Findeisen, R.: Learning a Gaussian process approximation of a model predictive controller with guarantees. In: *62nd IEEE Conference on Decision and Control (CDC)*, pp. 4094–4099 (2023)

28. Schölkopf, B., Herbrich, R., Smola, A.: A generalized representer theorem. In: International Conference on Computational Learning Theory, pp. 416–426. Springer (2001)
29. Sullivan, T.: Introduction to Uncertainty Quantification, vol. 63. Springer (2015)
30. Torre, E., Marelli, S., Embrechts, P., Sudret, B.: Data-driven polynomial chaos expansion for machine learning regression. *J. Comput. Phys.* **388**, 601–623 (2019)
31. Umlauft, J., Lederer, A., Beckers, T., Hirche, S.: Real-time uncertainty decomposition for online learning control (2020). Preprint [arXiv:2010.02613v2](https://arxiv.org/abs/2010.02613v2)
32. Wenger, J., Pleiss, G., Pförtner, M., Hennig, P., Cunningham, J.P.: Posterior and computational uncertainty in gaussian processes (2023). Preprint [arXiv:2205.15449](https://arxiv.org/abs/2205.15449)
33. Wiener, N.: The homogeneous chaos. *Am. J. Math.* **60**(4) 897–936 (1938)
34. Xiu, D., Karniadakis, G.: The Wiener-Askey polynomial chaos for stochastic differential equations. *SIAM J. Sci. Comput.* **24**(2), 619–644 (2002)
35. Yan, L., Duan, X., Liu, B., Xu, J.: Gaussian processes and polynomial chaos expansion for regression problem: linkage via the RKHS and comparison via the KL divergence. *Entropy* **20**(3), 191 (2018)

Chapter 8

A Universal Reproducing Kernel Hilbert Space for Learning Nonlinear System Operators



Mircea Lazar 

Abstract In this work, we consider the problem of learning nonlinear operators that correspond to discrete-time nonlinear dynamical systems with inputs. Given an initial state and a finite input trajectory, such operators yield a finite output trajectory compatible with the system dynamics. Inspired by the universal approximation theorem of operators tailored to radial basis functions neural networks, we construct a class of kernel functions as the product of kernel functions in the space of input trajectories and initial states, respectively. We prove that for positive definite kernel functions, the resulting *product reproducing kernel Hilbert space* is dense and even complete in the space of nonlinear systems operators, under suitable assumptions. This provides a universal kernel-function-based framework for learning nonlinear systems operators, which is intuitive and easy to apply to general nonlinear systems.

Keywords Nonlinear dynamical systems · Nonlinear operators · Kernel functions · Data-driven learning and control

8.1 Introduction

The field of data-driven modeling and control has recently received an increased interest not only due to advances in artificial intelligence and computing hardware, but also due to new applications of the fundamental lemma [23] in data-driven simulation, prediction, and control [2, 6, 13, 14]. The current effort is focused on data-driven modeling and control of nonlinear systems and one of the popular approaches in this field is based on kernel functions and reproducing kernel Hilbert spaces, see the comprehensive surveys [15, 18]. In general, we encounter methods that use kernel functions to learn *functions* or *mappings* and methods that use kernel functions to learn *operators*.

M. Lazar (✉)

Eindhoven University of Technology, Eindhoven, The Netherlands

e-mail: m.lazar@tue.nl

Within the first category mentioned above, kernel functions have been used to learn/model the system dynamics with the purpose of designing data-driven (predictive) controllers, in [8, 9, 11]. In particular, [9] uses kernels to parameterize multi-step input–output predictors for data-enabled predictive control (DeePC) [6] and minimizes the multi-step prediction error when computing the interpolation coefficients. The kernel functions are defined in a lifted space that includes both initial conditions (past inputs and outputs) and an input sequence of finite length in time (given by the prediction horizon). Other relevant works include [19], which provides optimized bounds for kernel-based approximations, and the recent work [17], which explores links between the fundamental lemma [23] and kernel regression for linear systems and specific classes of nonlinear systems, i.e., Hammerstein and flat nonlinear systems. In particular, [17] formulates the kernel regression problem for vector-valued mappings and extends standard bounds on the approximation error [7, 8] to singular Gramians.

Alternatively, a discrete-time dynamical system can be regarded as an operator defined on a Hilbert space [22] that maps sequences of inputs to sequences of outputs (or input functions of time to output functions of time, in the continuous-time case). In [22], the operator learning problem is formulated as a kernel regression problem that includes knowledge of physical properties of the operator, via integral quadratic constraints. Also, [22] provides a comprehensive summary of results on reproducing kernel Hilbert spaces for vector-valued operators. More recently, [20] continues in this framework by considering kernel-based learning of monotone operators related to passive dynamical systems.

Two fundamental results on universal approximation of nonlinear operators originate much earlier in the fundamental works of Tianping Chen and Hong Chen, i.e., [5], which considers deep neural networks, and [4], which considers radial basis function neural networks. These works have been recently brought into attention by the development of the deep operator network (DeepONet) [12] for learning nonlinear operators, with application to continuous-time dynamical systems and partial differential equations. [12] also provides a bound on the number of samples of the input signals required to achieve a desired approximation error. More recently, [1] developed a kernel-based framework for learning operators defined on vector-valued reproducing kernel Hilbert spaces, associated with partial differential equations. Therein, conditions for asymptotic convergence to zero of the approximation error were established.

In this paper, we consider the problem of learning a nonlinear system operator using the framework of reproducing kernel Hilbert spaces (RKHS) [3, 7, 21] and we consider the following questions: which assumptions regarding the set of data points, the class and construction of kernel functions, and the topology of the corresponding Hilbert space are necessary to achieve a RKHS that is dense and even complete in the space of operators of interest. Specifically, we focus on operators arising in discrete-time nonlinear systems described by difference equations with initial states, inputs, and outputs restricted to compact sets. We explicitly parameterize output trajectories as functions of input trajectories *and* initial states (or initial conditions in a more general sense). As such, we define the domain of the operator as a *product* Hilbert

space and the corresponding kernel functions are defined as the product of kernel functions in the input trajectories space and the state space, respectively. Based on the universal approximation theorem for operators using radial basis functions neural networks [4], we prove that for positive definite kernels [16] the resulting *product* RKHS is dense in the space of nonlinear systems operators and its unique minimizer is a universal approximator.

The developed product RKHS framework for learning nonlinear system operators brings important contributions with respect to existing frameworks, such as (i) regarding [4, 12], it provides a universal operator learning framework that does not require training a large neural network, i.e., the interpolation coefficients are simply obtained by inverting the product RKHS Gram matrix; (ii) regarding [9], the developed product RKHS framework scales much better with the number of data points in terms of building and inverting the Gram matrix.

8.2 Preliminary Definitions and Results

Let $\mathbb{R}$ and $\mathbb{N}$ denote the sets of real and natural numbers, respectively. For any finite number $q \in \mathbb{N}_{\geq 1}$ of vectors (or functions) $\{v_1, \dots, v_q\}$, we will make use of the operator $\text{col}(v_1, \dots, v_q) := [v_1^\top, \dots, v_q^\top]^\top$. For two matrices $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{m \times m}$, $A \otimes B \in \mathbb{R}^{nm \times nm}$ denotes their Kronecker product. For two vectors $a \in \mathbb{R}^n$, $b \in \mathbb{R}^m$, $a \otimes b \in \mathbb{R}^{nm}$ denotes their Kronecker vector product, i.e., for $a = [a_1 \ a_2]^\top$, $b = [b_1 \ b_2]^\top$, $a \otimes b = [a_1 b_1 \ a_1 b_2 \ a_2 b_1 \ a_2 b_2]^\top$.

As the data generating system, we consider an unknown discrete-time nonlinear system with input $u : \mathbb{N} \rightarrow \mathbb{R}^m$, output $y : \mathbb{N} \rightarrow \mathbb{R}^p$, and state $x : \mathbb{N} \rightarrow \mathbb{R}^n$. We assume that all measurable signals are noise free and for ease of notation we use the state x to denote either a minimal, true state, or a non-minimal one, i.e., represented by past inputs and outputs. For some finite $N \in \mathbb{N}_{\geq 1}$, we denote the corresponding Hilbert spaces $\mathcal{U} := \ell_2(\{0, 1, \dots, N\}, \mathbb{R}^m)$, $\mathcal{Y} := \ell_2(\{0, 1, \dots, N\}, \mathbb{R}^p)$ and $\mathcal{X} := \ell_2(\{0\}, \mathbb{R}^n)$, where the notation was inspired by [20]. Note that these are spaces of square summable sequences by definition.

The considered operator learning problem is formulated as follows: Given a set of input trajectories $\{\mathbf{u}_1, \dots, \mathbf{u}_{T_u}\}$ with $\mathbf{u}_i \in \mathcal{U}$, a set of initial states $\{x_1, \dots, x_{T_x}\}$ with $x_j \in \mathcal{X}$, and a corresponding set of output trajectories

$$\{\mathbf{y}_1^1, \dots, \mathbf{y}_{T_u}^1, \dots, \mathbf{y}_1^{T_x}, \dots, \mathbf{y}_{T_u}^{T_x}\}, \quad \mathbf{y}_i^j \in \mathcal{Y},$$

find an accurate approximation of the operator $G(\mathbf{u})(x) = \mathbf{y} \in \mathbb{R}^{p(N+1)}$, with respect to some suitable norm. The operator G maps sequences $\mathbf{u} \in \mathcal{U}$ into sequences $\mathbf{y} \in \mathcal{Y}$ for every $x \in \mathcal{X}$ and it is assumed that the sequences $\mathbf{y}$ are continuous functions of x . Above $T_u \in \mathbb{N}_{\geq 1}$ is the number of input trajectories and $T_x \in \mathbb{N}_{\geq 1}$ is the number of initial states. Let $[G(\mathbf{u})(x)]_q$ denote the q -th element of $G(\mathbf{u})(x)$, with $q = 1, \dots, p(N+1)$.

8.2.1 Universal Approximation of Operators Based on Radial Basis Functions Neural Networks

Next we recall the fundamental operator approximation result for radial basis function (RBF) neural networks [4, Theorem 5] adapted to the specific setting of this paper, i.e., (i) discrete-time dynamical systems, which means the input and output signals are already sampled in time; (ii) sampling in the space of initial conditions; and (iii) vector-valued operators. This result will be instrumental in the construction of a *dense* RKHS in the next section.

First we need to recall the notation in [4]. Let $g \in C(\mathbb{R}) \cap S'(\mathbb{R})$ denote a suitable radial basis function, where $C(\mathbb{R})$ denotes the Banach space of all continuous functions defined on $\mathbb{R}$ with norm $\|f\|_{C(\mathbb{R})} = \max_{s \in \mathbb{R}} |f(s)|$. Furthermore, $S'(\mathbb{R})$ denotes the set of all linear functional defined on $S(\mathbb{R})$, i.e., the set of infinitely differentiable functions which are rapidly decreasing at infinity. This definition covers most typical RBFs, but even polynomials are excluded [4].

Theorem 8.1 [4, Theorem 5] *Suppose that $g \in C(\mathbb{R}) \cap S'(\mathbb{R})$ is not an even polynomial, $\mathbb{U} \subseteq \mathcal{U}$, $\mathbb{X} \subseteq \mathcal{X}$ are two compact sets in $\mathcal{U}$ and $\mathcal{X}$, respectively, and G is a nonlinear continuous operator, which maps $\mathbb{U}$ into $C(\mathbb{X})$. Then for any $\epsilon > 0$ there are positive integers T_u, T_x , constants $c_{iq}^j, \mu_i, \lambda_j \in \mathbb{R}, i = 1, \dots, T_u, j = 1, \dots, T_x, q = 1, \dots, p(N + 1)$, points $\mathbf{u}_1, \dots, \mathbf{u}_{T_u} \in \mathbb{U}, x_1, \dots, x_{T_x} \in \mathbb{X}$, such that*

$$\left| [G(\mathbf{u})(x)]_q - \sum_{i=1}^{T_u} \sum_{j=1}^{T_x} c_{iq}^j g(\mu_i \|\mathbf{u} - \mathbf{u}_i\|) g(\lambda_j \|x - x_j\|) \right| < \epsilon \quad (8.1)$$

for all $\mathbf{u} \in \mathbb{U}$, all $x \in \mathbb{X}$ and all $q = 1, \dots, p(N + 1)$.

Note that [4, Theorem 5] considers input and output trajectories continuous in time and sampled in time, so therein x represents continuous time and x_j are time samples (although sampling in state is also mentioned as a possibility). Since in the discrete-time case $\mathbf{u}$ and $\mathbf{y}$ are sampled trajectories (sequences) by definition, it is no longer necessary to sample in time; instead we consider sampling in the state space to cover all output trajectories generated by a set of initial conditions. In [4, Theorem 5] $\mathcal{U}$ must be a Banach space, which is true in our case since $\mathcal{U}$ is a Hilbert space.

It is also important to point out that $g \in C(\mathbb{R}) \cap S'(\mathbb{R})$ rather denotes a class of functions in [4, Theorem 5] instead of a fixed function. That is, the universal approximation theorem holds with different functions for the input sequences space (i.e., g_u) and the state space (i.e., g_x), respectively, as long as these functions belong to $C(\mathbb{R}) \cap S'(\mathbb{R})$ and they are not an even polynomial. In fact, [4, Remark 4] states that radial basis functions could be mixed with affine basis functions in the universal approximation theorem.

Another relevant aspect is that [4, Theorem 5] considers real-valued operators. However, since the approximation theorem holds for every separate element of the

operator, we can construct a common set of points in $\mathbb{U}$ and $\mathbb{X}$ by aggregating all the points that exist for all elements and then we obtain (8.1) by setting some of the c_{iq}^j equal to zero, for each q -th element. Also, [4, Theorem 5] uses a different set of points $\mathbf{u}_i$ for each point x_j , i.e., $\mathbf{u}_{ij}$. By using a similar aggregate construction, we can group all the $\mathbf{u}_{ij}$ points for all j in a single set of points $\mathbf{u}_i$ and then set some of the coefficients c_{iq}^j equal to zero for each j to obtain (8.1).

Note that the radial basis functions g used in [4, Theorem 5] can be alternatively defined as kernel functions. For example, let $s := x - x_j$, $\lambda := \frac{1}{\sigma^2}$ and consider a Gaussian radial basis function:

$$g(\lambda \|s\|) = e^{-\lambda \|s\|^2} = e^{-\frac{1}{\sigma^2} \|s\|^2} = e^{-\frac{1}{\sigma^2} \|x - x_j\|^2} = k(x, x_j), \quad (8.2)$$

i.e., we have obtained the Gaussian kernel function. This analogy will be instrumental in the next section for constructing a product RKHS. To this end, next we introduce some basic RKHS definitions.

8.2.2 Reproducing Kernel Hilbert Spaces

Compared to the operator learning approaches using RKHS in [20, 22], for the setting of this paper, i.e., discrete-time dynamical systems and Hilbert spaces of square summable sequences, we adopt a simplified approach. More specifically, we regard operators $G : \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{X}$ denotes a generic Hilbert space of functions in this subsection, and $\mathcal{Y}$ is the Hilbert space of output trajectories, as an aggregation of a finite number of functionals from $\mathcal{X}$ to the set of real numbers. Moreover, if the elements of $\mathcal{X}$ are square summable sequences of finite length that can take arbitrary values in a suitable vector space, the functionals can be regarded as functions. As such we can make use of the standard RKHS theory for real-valued functions/functionals [7] and we can adopt a similar approach as in [17], i.e., we can assume a common RKHS for all functions/functionals. To this end, we recall the following standard RKHS definitions, see, e.g., [7, 10].

Definition 8.1 A function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is called a *kernel function* if it satisfies the following properties: (i) it is symmetric, i.e., $k(x_1, x_2) = k(x_2, x_1)$ for all $(x_1, x_2) \in \mathcal{X} \times \mathcal{X}$ and (ii) it is positive semi-definite, i.e., for every positive integer T and distinct set of points $\{x_1, \dots, x_T\} \in \mathcal{X}$ the matrix

$$K := \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) & \dots & k(x_1, x_T) \\ k(x_2, x_1) & k(x_2, x_2) & \dots & k(x_2, x_T) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_T, x_1) & k(x_T, x_2) & \dots & k(x_T, x_T) \end{pmatrix} \in \mathbb{R}^{T \times T} \quad (8.3)$$

is positive semi-definite. The matrix K is called the *Gram matrix*. Moreover, a kernel function k is called a *universal kernel* (or a *positive definite kernel*) if its corresponding Gram matrix is positive definite.

Two important classes of universal kernels [16], which are also radial basis functions that comply with the conditions of [4, Theorem 5], are Gaussian kernels:

$$k(x_1, x_2) := e^{-\frac{1}{\sigma^2} \|x_1 - x_2\|^2} \quad (8.4)$$

and Hardy reverse multiquadratic kernels:

$$k(x_1, x_2) := \left(1 + \frac{1}{\sigma^2} \|x_1 - x_2\|^2\right)^{-\frac{1}{2}}, \quad (8.5)$$

where σ is a positive real number.

Definition 8.2 Given a kernel function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, a Hilbert space $\mathcal{H}(k, \mathcal{X})$ is a *reproducing kernel Hilbert space* for k if (i) for every $x \in \mathcal{X}$, the function $k(x, \cdot) \in \mathcal{H}(k, \mathcal{X})$ and (ii) the reproducing property holds, i.e., $f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}(k, \mathcal{X})}$ for every $f \in \mathcal{H}(k, \mathcal{X})$ and $x \in \mathcal{X}$.

In data-based learning (or fitting) problems [7], typically a data-dependent finite-dimensional Hilbert space is considered, i.e.:

$$\mathcal{H}(k, \mathcal{X}) = \text{span}\{k(\cdot, x_1), k(\cdot, x_2), \dots, k(\cdot, x_T)\},$$

where $\{x_1, \dots, x_T\}$ are distinct data points and $T \geq 1$ is sufficiently large. In what follows we will make use of the following notation for the vector kernel corresponding to the standard basis, i.e., $\text{span}\{k(\cdot, x_1), k(\cdot, x_2), \dots, k(\cdot, x_T)\}$:

$$\mathbf{k}(x) := \text{col}(k(x_1, x), k(x_2, x), \dots, k(x_T, x)) \in \mathbb{R}^T, \quad \forall x \in \mathcal{X}. \quad (8.6)$$

Given a set of observations (measurements) $\{y_1, \dots, y_T\}$ with each $y_i \in \mathbb{R}^{n_y}$ corresponding to the set of data points $\{x_1, \dots, x_T\}$ with each $x_j \in \mathcal{X}$, the operator learning problem can be formulated in the RKHS $\mathcal{H}(k, \mathcal{X})$ as [3, 17]

$$\min_{\Theta \in \mathbb{R}^{T \times n_y}} \|Y^\top - K\Theta\|_F^2,$$

where $\|M\|_F$ denotes the Frobenius norm of a matrix M , K is the corresponding Gram matrix, and $Y = (y_1 \ y_2 \ \dots \ y_T) \in \mathbb{R}^{n_y \times T}$. If k is a positive definite kernel and hence K is invertible, there exists a unique solution corresponding to the unique interpolated surface passing through all the points y_i , which yields the optimal approximation:

$$G_{\mathcal{H}(k, \mathcal{X})}^*(x) := (\Theta^*)^\top \mathbf{k}(x) = YK^{-1}\mathbf{k}(x), \quad \forall x \in \mathcal{X}. \quad (8.7)$$

Through the prism of Willems' fundamental lemma [23], we can write down the following system of equations for any $x \in \mathcal{X}$:

$$K\mathbf{g} = \mathbf{k}(x) \quad (8.8a)$$

$$Y\mathbf{g} = G_{\mathcal{H}(k, \mathcal{X})}(x), \quad (8.8b)$$

where $\mathbf{g} = \mathbf{g}(x) \in \mathbb{R}^T$ is a vector of optimization variables (not to be confused with the RBF function g from the previous subsection). That is, for every $x \in \mathcal{X}$, there should exist a $\mathbf{g}$ that satisfies (8.8). We observe that the first block of equations in (8.8a) correspond to property (i) of the RKHS, i.e., for any $x \in \mathcal{X}$, it must hold that $\mathbf{k}(x) \in \mathcal{H}(k, \mathcal{X}) := \text{span}\{\mathbf{k}(x_1), \mathbf{k}(x_2), \dots, \mathbf{k}(x_T)\}$. Furthermore, for a positive definite kernel, when K is invertible, the second block of equations in (8.8b) ensures that

$$G_{\mathcal{H}(k, \mathcal{X})}^*(x) = Y\mathbf{g}^* = YK^{-1}\mathbf{k}(x) = \langle G, \mathbf{k}(x) \rangle_{\mathcal{H}(k, \mathcal{X})},$$

which is the reproducing property.

8.2.3 Problem Formulation

Several existing works have exploited physical properties of the operator [20, 22] or of the underlying system dynamics/functions [9, 17, 19] to define kernel functions and corresponding RKHS. Differently, in this paper, we do not make specific assumptions about the class of the underlying nonlinear dynamics and we ask the following fundamental questions:

- Q1 Does there exist a class of kernel functions k such that the resulting RKHS $\mathcal{H}(k, \mathcal{X})$ is dense in the considered space of nonlinear system operators?
- Q2 Does there exist a class of kernel functions k such that the resulting RKHS $\mathcal{H}(k, \mathcal{X})$ is complete with respect to the considered space of nonlinear system operators?

In the next section, we provide a possible answer to the above questions and we show that construction of a universal RKHS for learning nonlinear systems operators is linked to how initial states or initial conditions are incorporated in the learning problem.

8.3 Product Reproducing Kernel Hilbert Spaces

In what follows we adopt the following definitions of a dense and complete subspace in a Hilbert space from [24, Chap. 3].

Definition 8.3 Let $\mathcal{V} = \text{span}\{v_i : i = 1, 2, \dots\}$ be a subspace in a Hilbert space $\mathcal{X}$. $\mathcal{V}$ is *dense* in $\mathcal{X}$ if its closure agrees with the whole space $\mathcal{X}$, i.e., if every point in $\mathcal{X}$ is a closure point of $\mathcal{V}$.

The above dense property can be regarded as a universal approximation property similar to (8.1), i.e., for every point $x \in \mathcal{X}$ and any $\epsilon > 0$, there exist points $\{v_1, \dots, v_n\}$, $v_i \in \mathcal{V}$ and coefficients c_i such that $\|x - \sum_{i=1}^n c_i v_i\| \leq \epsilon$.

Definition 8.4 A family $\{e_n\}_{n=1}^\infty$ of non-zero elements in a Hilbert space $\mathcal{X}$ is said to be an *orthogonal system* if $\langle e_i, e_j \rangle = 0$ for all $i \neq j$. If, further, $\|e_i\| = 1$ for all i , then $\{e_n\}_{n=1}^\infty$ is called an *orthonormal system*.

Definition 8.5 An orthonormal system $\{e_n\}_{n=1}^\infty$ in a Hilbert space $\mathcal{X}$ is said to be *complete* if it has the property that the only element orthogonal to all e_n is 0.

Recall next the considered operator learning problem, i.e., consider a set of input trajectories $\{\mathbf{u}_i : i = 1, 2, \dots\}$ with $\mathbf{u}_i \in \mathcal{U}$, a set of initial states $\{x_j : j = 1, 2, \dots\}$ with $x_j \in \mathcal{X}$ and a corresponding set of output trajectories:

$$\{\mathbf{y}_i^j : i = 1, 2, \dots, j = 1, 2, \dots\}, \quad \mathbf{y}_i^j \in \mathcal{Y},$$

where $\mathcal{U}, \mathcal{X}, \mathcal{Y}$ are suitable Hilbert spaces. We would like to use these trajectories to learn the underlying operator $G(\mathbf{u})(x)$ that maps sequences in $\mathcal{U}$ into sequences in $\mathcal{Y}$ for every point in $\mathcal{X}$. To this end we will construct a *product* RKHS as follows.

Let $k_u : \mathcal{U} \times \mathcal{U} \rightarrow \mathbb{R}$ and $k_x : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be kernel functions, respectively. Further, for any set of points $\{\mathbf{u}_i : i = 1, 2, \dots\}$ with $\mathbf{u}_i \in \mathcal{U}$ and set of initial states $\{x_j : j = 1, 2, \dots\}$ with $x_j \in \mathcal{X}$ let K_u, K_x denote the corresponding Gram matrices and let $\mathbf{k}_u, \mathbf{k}_x$ denote the corresponding vector kernels. Then we can construct a product kernel function (e.g., see [21, Lemma 4.6]):

$$k_\otimes : (\mathcal{U} \times \mathcal{X}) \times (\mathcal{U} \times \mathcal{X}) \rightarrow \mathbb{R}, \quad k_\otimes((\mathbf{u}_1, x_1), (\mathbf{u}_2, x_2)) := k_u(\mathbf{u}_1, \mathbf{u}_2)k_x(x_1, x_2) \quad (8.9)$$

with corresponding Gram matrix and vector kernel, i.e.:

$$K_\otimes = K_u \otimes K_x \quad \text{and} \quad \mathbf{k}_\otimes(\mathbf{u}, x) = \mathbf{k}_u(\mathbf{u}) \otimes \mathbf{k}_x(x), \quad (8.10)$$

and unique *product* RKHS $\mathcal{H}(k_\otimes, \mathcal{U} \times \mathcal{X})$.

We consider given a set of observations (measurements) $\{y_1^1, \dots, y_{T_u}^{T_u}\}$ with $y_i^j \in \mathbb{R}^{n_y}$ (e.g., for an output trajectory from time 0 to N , $n_y = p(N+1)$, where p is the number of outputs) corresponding to the set of data points $\{u_1, \dots, u_{T_u}\}$ with each $u_i \in \mathbb{R}^{n_u}$ (e.g., for an input trajectory from time 0 to N , $n_u = m(N+1)$, where m is the number of inputs) and $\{x_1, \dots, x_{T_x}\}$ with each $x_j \in \mathbb{R}^{n_x}$. Recall here that x need not be the true state, i.e., it can be formed from past input and output observations and the number of states n_x depends on this choice. Then, letting $T := T_u T_x$ the operator learning problem can be formulated in the *product* RKHS $\mathcal{H}(k_\otimes, \mathcal{U} \times \mathcal{X})$ as

$$\min_{\Theta \in \mathbb{R}^{T \times ny}} \|Y^\top - K_\otimes \Theta\|_F^2,$$

where

$$Y = \begin{pmatrix} y_1^1 & y_2^1 & \dots & y_{T_u}^1 & \dots & y_1^{T_x} & y_2^{T_x} & \dots & y_{T_u}^{T_x} \end{pmatrix} \in \mathbb{R}^{ny \times T}.$$

The corresponding fundamental system of equations for any $(\mathbf{u}, x) \in \mathcal{U} \times \mathcal{X}$ is

$$K_\otimes \mathbf{g} = \mathbf{k}_\otimes(\mathbf{u}, x) \quad (8.11a)$$

$$Y \mathbf{g} = G_{\mathcal{H}(k_\otimes, \mathcal{U} \times \mathcal{X})}(\mathbf{u}, x), \quad (8.11b)$$

where $\mathbf{g} \in \mathbb{R}^T$. If $K_\otimes \in \mathbb{R}^{T \times T}$ is full rank and thus invertible, the corresponding unique minimizer is obtained as

$$\begin{aligned} G_{\mathcal{H}(k_\otimes, \mathcal{U} \times \mathcal{X})}^*(\mathbf{u}, x) &= (\Theta^*)^\top \mathbf{k}_\otimes(\mathbf{u}, x) = Y \mathbf{g}^* \\ &= Y K_\otimes^{-1} \mathbf{k}_\otimes(\mathbf{u}, x) = \langle G, \mathbf{k}_\otimes(\mathbf{u}, x) \rangle_{\mathcal{H}(k_\otimes, \mathcal{U} \times \mathcal{X})}. \end{aligned} \quad (8.12)$$

We observe that the constructed product RKHS has a special topology, i.e., it corresponds to the product of two Hilbert spaces, i.e., the space of square summable input sequences of finite length and the space of initial states. This allows sampling these two spaces independently to generate the dataset.

Next, we state the main result, which establishes conditions for universality of the constructed product RKHS.

Theorem 8.2 *Suppose that $k_u, k_x \in C(\mathbb{R}) \cap S'(\mathbb{R})$ are not even polynomials and that they are positive definite kernel functions, e.g., Gaussian or reverse multi-quadratic radial functions. Let $\mathbb{U} \times \mathbb{X} \subseteq \mathcal{U} \times \mathcal{X}$ be a compact set in $\mathcal{U} \times \mathcal{X}$ and let G be a nonlinear continuous operator, which maps $\mathbb{U}$ into $\mathbb{Y} \subset C(\mathbb{X})$ for every $x \in \mathbb{X}$ and belongs to $\mathcal{H}(k_\otimes, \mathbb{U} \times \mathbb{X})$. Then the following results hold:*

- (i) *For any $\epsilon > 0$ there are positive integers T_u, T_x , and points $\{\mathbf{u}_1, \dots, \mathbf{u}_{T_u}\}, \mathbf{u}_i \in \mathbb{U}, \{x_1, \dots, x_{T_x}\}, x_j \in \mathbb{X}$, such that*

$$\|G(\mathbf{u})(x) - G_{\mathcal{H}(k_\otimes, \mathbb{U} \times \mathbb{X})}^*(\mathbf{u}, x)\|_2 < \epsilon, \quad (8.13)$$

for all $\mathbf{u} \in \mathbb{U}$ and all $x \in \mathbb{X}$, i.e., $\text{span}\{\{k_\otimes(\cdot, (u_i, x_j))\}_{i=1, \dots, T_u, j=1, \dots, T_x}\}$ is dense in the space of nonlinear systems operators that map $\mathbb{U}$ into $\mathbb{Y} \subset C(\mathbb{X})$.

- (ii) *There exists a possibly infinite set of points $\{(\mathbf{u}, x)_i : i = 1, 2, \dots\}, (\mathbf{u}, x)_i \in \mathbb{U} \times \mathbb{X}$, such that the $\text{span}\{\{k_\otimes(\cdot, (\mathbf{u}, x)_i)\}_{i=1, 2, \dots}\}$ induces a complete orthonormal system in the space of nonlinear systems operators that map $\mathbb{U}$ into $\mathbb{Y} \subset C(\mathbb{X})$.*

Proof (i) Recall the relation (8.2) between kernel functions $k(z_1, z_2)$ and radial basis functions $g(\lambda \|z_1 - z_2\|)$. The parameter $\lambda > 0$ is typically also present in kernel functions, but not explicitly mentioned and it is chosen independently of the

data points. Since the universal approximation theorem [4, Theorem 5] and condition (8.1) are formulated in terms of data-dependent λ_j, μ_i , we will first show that when the RBFs g correspond to universal radial functions/kernel, inequality (8.1) holds with data-independent parameters λ (and μ). To this end, define an RBF function $g \in (\mathcal{Z} \times \mathcal{Z}) \times \mathbb{R}_+ \rightarrow \mathbb{R}$ for some suitable set $\mathcal{Z}$, corresponding to a universal positive definite radial kernel function k as $g(\lambda\|z_1 - z_2\|) := k(z_1, z_2)$, which explicitly reveals the dependence on the parameter λ . For such functions, e.g., as defined in (8.4), (8.5), it holds that $0 < g(\lambda\|z_1 - z_2\|) < g(\lambda\|z_1 - z_1\|) = g(0)$ for any $(z_1, z_2) \in \mathcal{Z} \times \mathcal{Z}$ and $\lambda \in \mathbb{R}_+$, e.g., for Gaussian and reverse multiquadratics $g(0) = 1$. Hence, for any $\lambda(z_2), \lambda \in \mathbb{R}_+$, there exists a $\bar{\lambda}(z_2) \in \mathbb{R}_+$ such that $g(\lambda(z_2)\|z_1 - z_2\|) = \bar{\lambda}(z_2)g(\lambda\|z_1 - z_2\|)$, i.e., $\bar{\lambda}(z_2) := \frac{g(\lambda(z_2)\|z_1 - z_2\|)}{g(\lambda\|z_1 - z_2\|)}$.

Next, define the RBF approximator corresponding to radial basis functions g_u, g_x obtained from the positive definite radial kernel functions k_u, k_x , i.e., which are defined using parameters $\mu, \lambda \in \mathbb{R}_+$ (e.g., these parameters are typically chosen as the average distances between neighboring points over the whole dataset or they could be identified from data), respectively, i.e.:

$$[G_{RBF}(\mathbf{u}, x)]_q := \sum_{i=1}^{T_u} \sum_{j=1}^{T_x} \bar{c}_{iq}^j g_u(\mu\|\mathbf{u} - \mathbf{u}_i\|) g_x(\lambda\|x - x_j\|), \quad (8.14)$$

where $\bar{c}_{iq}^j := c_{iq}^j \bar{\mu}(\mathbf{u}_i) \bar{\lambda}(x_j)$ for all (i, j) and $q = 1, \dots, n_y$. Then by Theorem 8.1 (recall that we can apply this theorem with different functions g_u, g_x as long as they are in the right class of radial functions; choosing the same kernel function and the same g is also possible), for any $\epsilon > 0$ there exist positive integers T_u, T_x , corresponding set of points and coefficients $\bar{c}_{iq}^j$ such that

$$\|G(\mathbf{u})(x) - G_{RBF}(\mathbf{u}, x)\|_2^2 < \epsilon, \quad \forall \mathbf{u} \in \mathbb{U}, x \in \mathbb{X}. \quad (8.15)$$

Indeed, we can apply Theorem 8.1 to attain (8.1) with ϵ replaced by $\sqrt{\frac{\epsilon}{n_y}}$ to attain (8.15). Then we can expand (8.14) and write out the RBF approximator as a function of the corresponding kernels k_u, k_x , i.e.:

$$\begin{aligned} [G_{RBF}(\mathbf{u}, x)]_q &= \sum_{i=1}^{T_u} \sum_{j=1}^{T_x} \bar{c}_{iq}^j g_u(\mu\|\mathbf{u} - \mathbf{u}_i\|) g_x(\lambda\|x - x_j\|) \\ &= \sum_{i=1}^{T_u} \sum_{j=1}^{T_x} \bar{c}_{iq}^j k_u(\mathbf{u}, \mathbf{u}_i) k_x(x, x_j) \\ &= \theta_q^\top \mathbf{k}_\otimes(\mathbf{u}, x), \end{aligned} \quad (8.16)$$

where $\theta_q = (\bar{c}_{1q}^1 \dots \bar{c}_{T_u q}^{T_x})^\top \in \mathbb{R}^T$ for all $q = 1, \dots, n_y$. Hence we can build a matrix of coefficients $\Theta \in \mathbb{R}^{T \times n_y}$ with its q -th column equal to θ_q such that $G_{RBF}(\mathbf{u}, x) = \Theta^\top \mathbf{k}_\otimes(\mathbf{u}, x)$.

Next, we observe that since k_u and k_x are positive definite kernels, their corresponding Gramians K_u, K_x have full rank and are invertible for any set of distinct points in $\mathbb{U}$ and $\mathbb{X}$, respectively. Then, by the property of the Kronecker product, we have that $\text{rank}(K_\otimes) = \text{rank}(K_u) \text{rank}(K_x)$, i.e., the product Gram matrix $K_\otimes$ also has full rank and it is invertible. This could also be established by proving that the product kernel $k = k_u k_x$ is positive definite when k_u and k_x are positive definite. Hence, the fundamental system of Eq. (8.8a) has a unique solution $\mathbf{g}^*$ for each $(\mathbf{u}, x)$, which yields

$$G_{RBF}(\mathbf{u}, x) = \Theta^\top \mathbf{k}_\otimes(\mathbf{u}, x) = \Theta^\top K_\otimes \mathbf{g}^*(\mathbf{u}, x), \quad \forall (\mathbf{u}, x) \in \mathbb{U} \times \mathbb{X}.$$

Hence, $G_{RBF}(\mathbf{u}, x)$ is an interpolant in the product RKHS $\mathcal{H}(k_\otimes, \mathbb{U} \times \mathbb{X})$. Since we assume that $G(\mathbf{u})(x)$ belongs to $\mathcal{H}(k_\otimes, \mathbb{U} \times \mathbb{X})$, it follows that [7] $G_{\mathcal{H}(k_\otimes, \mathbb{U} \times \mathbb{X})}^*(\mathbf{u}, x)$ provides the best approximation of $G(\mathbf{u})(x)$ from $\mathcal{H}(k_\otimes, \mathbb{U} \times \mathbb{X})$ with respect to the 2-norm, which yields

$$\|G(\mathbf{u})(x) - G_{\mathcal{H}(k_\otimes, \mathbb{U} \times \mathbb{X})}^*(\mathbf{u}, x)\|_2 \leq \|G(\mathbf{u})(x) - G_{RBF}(\mathbf{u}, x)\|_2 < \epsilon,$$

for all $(\mathbf{u}, x) \in \mathbb{U} \times \mathbb{X}$, which completes the proof of statement (i).

To prove statement (ii), note that by the positive definiteness of the product kernel function $k_\otimes$, for any set of distinct points $\{(\mathbf{u}, x)_i : i = 1, 2, \dots\}$, $(\mathbf{u}, x)_i \in \mathbb{U} \times \mathbb{X}$, the Gram matrix $K_\otimes$ is full rank, and thus the sequence of kernel vectors $\{k_\otimes(\cdot, (u, x)_1), \dots, k_\otimes(\cdot, (u, x)_n), \dots\}$ is linearly independent, i.e., any finite subset is linearly independent. Then, one can choose an orthonormal family $\{e_1, \dots, e_n, \dots\}$ such that for every n , $\text{span}\{e_1, \dots, e_n\} = \text{span}\{k_\otimes(\cdot, (u, x)_1), \dots, k_\otimes(\cdot, (u, x)_n)\}$ via the Gram–Schmidt orthohonalization procedure, see, e.g., [24, Chap. 3, Problem 1]. Since in statement (i) we have shown that $\text{span}\{k_\otimes(\cdot, (u, x)_1), \dots, k_\otimes(\cdot, (u, x)_n)\}$ is dense in the space of nonlinear systems operators that map $\mathbb{U}$ into $\mathbb{Y} \subset C(\mathbb{X})$, it follows that $\text{span}\{e_n : n = 1, 2, \dots\}$ is dense in this space as well. Then, by [24, Proposition 3.2.19] we have that the orthonormal system $\{e_n\}_{n=1}^\infty$ is complete. $\square$

The above results show that the developed product RKHS offer a universal framework for learning operators arising in discrete-time nonlinear dynamical systems, which is intuitive and computationally efficient, i.e., it scales well with the number of data points in terms of building the product Gram matrix $K_\otimes$. See the illustrative example for numerical details.

A similar product RKHS can be developed for continuous dynamical system operators by additionally sampling in time, i.e., by considering a product Hilbert space $\mathcal{U} \times \mathcal{X} \times \mathcal{T}$ with associated product kernel functions and RKHS. Alternatively, finitely parameterized input sequences could be pursued as done in [20] in combination with a product RKHS in $\mathcal{U} \times \mathcal{X}$.

The developed results yield a new insight in persistency of excitation for nonlinear systems, i.e., that a persistently exciting input sequence should be paired with a linearly independent/distinct set of points in the space of initial conditions. This is not necessary for linear systems, but seems to play a key role for nonlinear systems as the product Gram matrix is full rank if and only if the Gram matrix in the input space and the Gram matrix in the space of initial conditions are full rank. Last but not least, the developed results can be extended to Gram matrices that are not full rank by using singular value decomposition or orthogonal projection to remove the data points that result in very small or zero singular values for the product Gram matrix. This is interesting because it would allow using any type of kernel functions, not just positive definite ones.

8.4 Illustrative Example

Consider a discrete-time state-space model of the van der Pol oscillator

$$\begin{aligned}x_1(k+1) &= x_1(k) + T_s x_2(k) \\x_2(k+1) &= x_2(k) + T_s (\mu(1 - x_1^2(k))x_2(k) - x_1(k) + u(k)) \\y(k) &= x(k),\end{aligned}$$

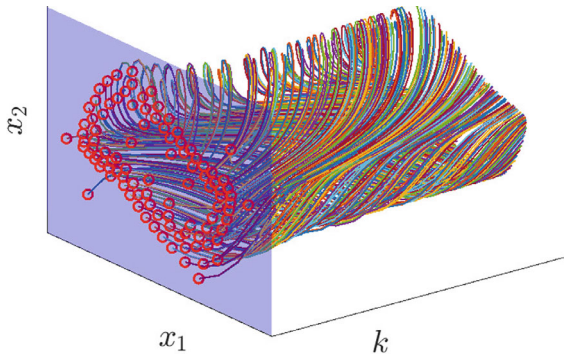
with $\mu = 1$ and sampling period $T_s = 0.1$. The output is equal to the full state and we would like to learn an operator that predicts the output trajectories of the system $N = 10$ steps ahead. We will implement the developed product RKHS framework and we will compare the results with the standard RKHS framework. The two approaches differ in terms of data generation and how they scale with the number of data points. As such, to ensure a reasonably fair comparison, we fine-tune each approach to get the best possible results, while using the same type of kernel function and the same method for generating a persistently exciting input signal. The results are obtained using the same laptop computer (Lenovo ThinkPad X1, Intel vPro i7 processor) and MATLAB® 2023a.

To generate a persistently exciting input we use the MATLAB® function *idinput*, a sum-of-sinusoidal type of input signal with Range as $[-5, 5]$, SineData as $[25, 40, 1]$, Band as $[0, 1]$, NumPeriod as 1, and Nu as 1. As the kernel function we use the Hardy reverse multiquadratic kernel (8.5) with $\sigma = \sqrt{4}$ for the standard kernel approach and $\sigma_x = \sqrt{0.4}$ and $\sigma_u = \sqrt{2}$ for the product kernel approach.

For the product kernel approach, we select $T_x = 150$ initial conditions (states) and we construct $T_u = 290$ input sequences via the Hankel matrix approach from a generated persistently exciting input signal of sufficient length. The resulting output trajectories used for operator learning are plotted in Fig. 8.1.

Building the Gram matrices K_u and K_x requires $T_u^2 + T_x^2 = 106600$ kernel function evaluations, which takes less than 5 min. Then the inverse product Gram matrix $K_{\otimes}^{-1}$ is obtained by inverting K_u and K_x and computing their Kronecker product

Fig. 8.1 Output trajectories used for learning the operator in the product RKHS



which takes less than 2 s. By merging the initial states and the input sequences, the product RKHS approach generates 43500 data points in the lifted (x, u) space and a product Gram matrix $K_{\otimes}$ of dimension 43500^2 , i.e., roughly 1.8 billions. Using so many data points in the standard RKHS approach would require 1.8 billion kernel function evaluations just to build the Gram matrix, which is intractable/very time-consuming.

The prediction capability of the operator learned using the product RKHS approach is shown in Fig. 8.2.

To test the standard RKHS approach, we generated a persistently exciting input signal of length 10000, which results in 9990 data points in the lifted space (x, u)

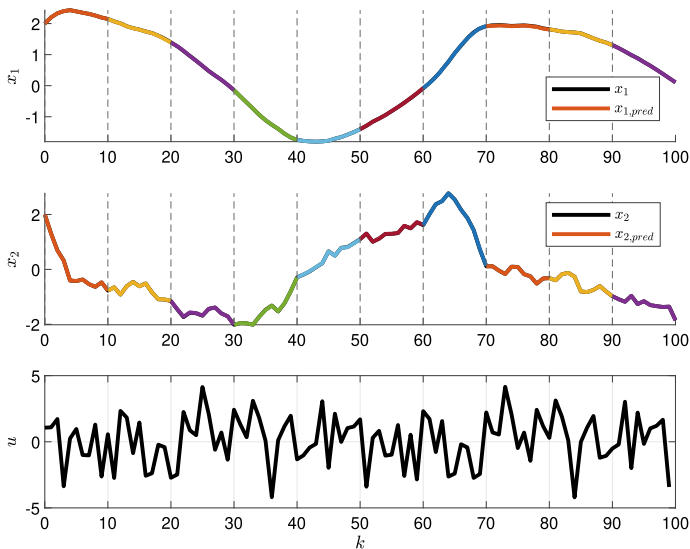


Fig. 8.2 Validation of the product kernel operator predictions for several initial conditions not part of the training data and a random input signal

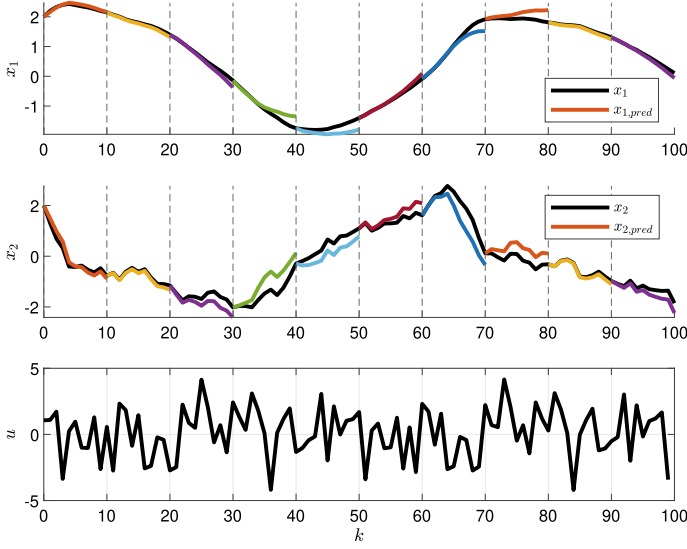


Fig. 8.3 Validation of the standard kernel mapping predictions for the same initial conditions and random input signal used to validate the product kernel operator

via the Hankel matrix approach. This yields a Gram matrix K of dimension 9990^2 or roughly 0.1 billions. Building the Gram matrix took more than an hour and inverting it took 20 s. Attempting to build the Gram matrix for a signal of length 20000 did not return a result after several hours. The prediction capability of the mapping learned using the standard RKHS approach is shown in Fig. 8.3.

To better compare the accuracy of the implemented learning methods, in Fig. 8.4, we provide a Root Mean Square (RMS) prediction error analysis per each time step, for the first state x_1 . The obtained results demonstrate the efficiency and accuracy of the developed product RKHS framework for learning nonlinear system operators.

8.5 Conclusions

Based on the universal approximation theorem of operators tailored to radial basis functions neural networks, we constructed a universal class of kernel functions as the product of kernel functions in the space of input sequences and initial states, respectively. We proved that for positive definite kernel functions, the resulting *product reproducing kernel Hilbert space* is dense and even complete in the space of nonlinear system operators that map input sequences and initial states into output trajectories. This provides a universal kernel-function-based framework for learning nonlinear system operators that is intuitive and easy to apply.

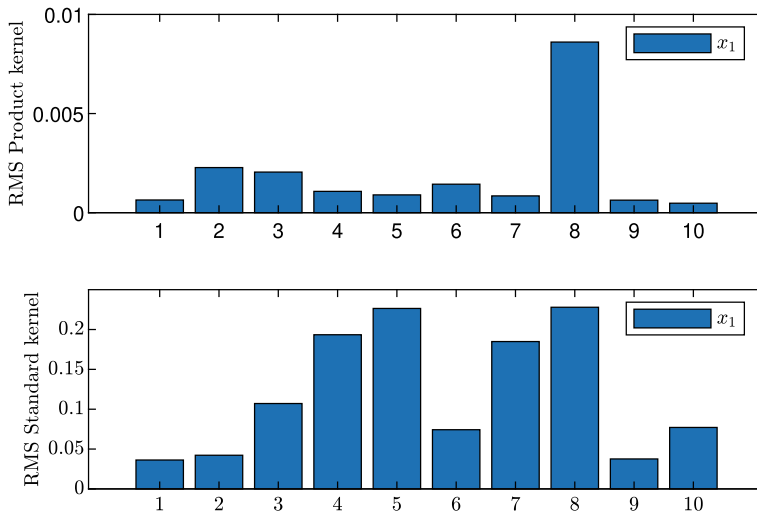


Fig. 8.4 RMS for the prediction of x_1 for each time step $i = 1, \dots, 10$

In future work, we will present a relation between product RKHS operators and the Koopman operator with important implications for the nonlinear fundamental lemma. Also, we will exploit product RKHS operators to design data-enabled predictive controllers for nonlinear systems.

Acknowledgements The author would like to express his gratitude to the PhD researcher Thomas de Jong for building the simulation scripts and the figures for the illustrative example. The author is also grateful to the anonymous reviewers for their helpful comments.

References

1. Battle, P., Darcy, M., Hosseini, B., Owhadi, H.: Kernel methods are competitive for operator learning. *J. Comput. Phys.* **496**, 112549 (2024)
2. Berberich, J., Köhler, J., Müller, M.A., Allgöwer, F.: Data-driven model predictive control with stability and robustness guarantees. *IEEE Trans. Autom. Control* **66**(4), 1702–1717 (2021)
3. Bishop, C.M.: *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer (2006)
4. Chen, T., Chen, H.: Approximation capability to functions of several variables, nonlinear functionals, and operators by radial basis function neural networks. *IEEE Trans. Neural Netw.* **6**(4), 904–910 (1995)
5. Chen, T., Chen, H.: Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Trans. Neural Netw.* **6**(4), 911–917 (1995)
6. Coulson, J., Lygeros, J., Dörfler, F.: Data-enabled predictive control: in the shallows of the DeePC. In: *18th European Control Conference*, pp. 307–312. Napoli, Italy (2019)

7. Fasshauer, G.E.: Positive definite kernels: past, present and future. *Dolomites Res. Notes Approx.* **4** (2011)
8. Hu, Z., De Persis, C., Tesi, P.: Learning controllers from data via kernel-based interpolation. In: 2023 62nd IEEE Conference on Decision and Control (CDC), pp. 8509–8514 (2023)
9. Huang, L., Lygeros, J., Dörfler, F.: Robust and kernelized data-enabled predictive control for nonlinear systems. *IEEE Trans. Control Syst. Technol.* **32**(2), 611–624 (2024)
10. Kalnishkan, Y.: An introduction to kernel methods. Technical Report (Royal Holloway University of London) (2009)
11. Lian, Y., Jones, C.N.: Nonlinear data-enabled prediction and control. In: Proceedings of the 3rd Conference on Learning for Dynamics and Control, Proceedings of Machine Learning Research, vol. 144, pp. 523–534. PMLR (2021)
12. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E.: Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **3**, 218–229 (2021)
13. Markovsky, I.: Data-driven simulation of generalized bilinear systems via linear time-invariant embedding. *IEEE Trans. Autom. Control* **68**(2), 1101–1106 (2023)
14. Markovsky, I., Huang, L., Dörfler, F.: Data-driven control based on the behavioral approach: from theory to applications in power systems. *IEEE Control Syst. Mag.* **43**(5), 28–68 (2023)
15. Martin, T., Schön, T.B., Allgöwer, F.: Guarantees for data-driven control of nonlinear systems using semidefinite programming: a survey. *Annu. Rev. Control* **56**, 100911 (2023)
16. Micchelli, C.A., Xu, Y., Zhang, H.: Universal kernels. *J. Mach. Learn. Res.* **7**(95), 2651–2667 (2006)
17. Molodchik, O., Faulwasser, T.: Exploring the links between the fundamental lemma and kernel regression. *IEEE Control Syst. Lett.* **8**, 2045–2050 (2024)
18. Pillonetto, G., Dinuzzo, F., Chen, T., De Nicolao, G., Ljung, L.: Kernel methods in system identification, machine learning and function estimation: a survey. *Automatica* **50**(3), 657–682 (2014)
19. Scharnhorst, P., Maddalena, E.T., Jiang, Y., Jones, C.N.: Robust uncertainty bounds in reproducing kernel Hilbert spaces: a convex optimization approach. *IEEE Trans. Autom. Control* **68**(5), 2848–2861 (2023)
20. Shali, B.M., van Waarde, H.J.: Towards a representer theorem for identification of passive systems. [arXiv:2404.08297](https://arxiv.org/abs/2404.08297) (2024)
21. Steinwart, I., Christmann, A. (eds.): Support Vector Machines. Information Science and Statistics. Springer (2008)
22. van Waarde, H.J., Sepulchre, R.: Kernel-based models for system analysis. *IEEE Trans. Autom. Control* **68**(9), 5317–5332 (2023)
23. Willems, J.C., Rapisarda, P., Markovsky, I., De Moor, B.L.: A note on persistency of excitation. *Syst. Control Lett.* **54**(4), 325–329 (2005)
24. Yamamoto, Y.: From Vector Spaces to Function Spaces. SIAM, Philadelphia (2012)

Chapter 9

On Robust Reinforcement Learning with Lipschitz-Bounded Policy Networks



Nicholas H. Barbara, Ruigang Wang, and Ian R. Manchester

Abstract This paper presents a study of robust policy networks in deep reinforcement learning. We investigate the benefits of policy parameterizations that naturally satisfy constraints on their Lipschitz bound, analyzing their empirical performance and robustness on two representative problems: pendulum swing-up and Atari Pong. We illustrate that policy networks with smaller Lipschitz bounds are more robust to disturbances, random noise, and targeted adversarial attacks than unconstrained policies composed of vanilla multi-layer perceptrons or convolutional neural networks. However, the structure of the Lipschitz layer is important. We find that the widely used method of spectral normalization is too conservative and severely impacts clean performance, whereas more expressive Lipschitz layers such as the recently proposed Sandwich layer can achieve improved robustness without sacrificing clean performance.

Keywords Reinforcement learning · Robustness · Lipschitz networks

9.1 Introduction

Deep reinforcement learning (deep RL) has been the driving force behind many recent successes in learning-based control, including discrete game-like problems [10], robotic manipulation [5], and locomotion [17]. However, the applicability of deep RL to performance- and safety-critical systems is currently limited by questions of its robustness [4]. It is well known that neural networks can be highly sensitive to small input perturbations [20], making policy networks learned via deep RL potentially

N. H. Barbara (✉) · R. Wang · I. R. Manchester
Australian Centre for Robotics, School of Aerospace, Mechanical and Mechatronic Engineering,
The University of Sydney, Sydney, NSW, Australia
e-mail: nicholas.barbara@sydney.edu.au

R. Wang
e-mail: ruigang.wang@sydney.edu.au

I. R. Manchester
e-mail: ian.manchester@sydney.edu.au

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025
J. Berberich et al. (eds.), *Systems Theory in Data and Optimization*, Lecture Notes
in Control and Information Sciences - Proceedings,
https://doi.org/10.1007/978-3-031-83191-1_9

unrobust to disturbances, noise, and targeted adversarial attacks. Despite sharing similar sensitivity issues to neural classifiers, for which many robust neural networks have recently been developed [15, 22, 24], the use of robust policy architectures in deep RL has not been widely studied.

Most common approaches to improving policy robustness in deep RL are based on adversarial training [13], where adversarial attacks on a policy’s inputs are optimized during training to encourage the network to perform well under perturbations. While this works well in applications where the structure of the perturbations is always similar to those seen during training, adversarial training only certifies a lower bound on a policy’s robustness. It is therefore possible to find new, out-of-sample attacks that cause the policy to fail [18]. An alternative strategy is to learn control policies with a certified *upper* bound on their sensitivity to perturbations using methods like randomized smoothing [6, 25] and loss-function regularization [11, 12]. These methods bound the sensitivity of a learned policy during the training process. It is interesting to ask whether, via careful choice of a policy’s architecture and parameterization, we can directly bound the sensitivity of a policy independently of how it is trained.

One promising approach is to use neural network policies that are certifiably robust to perturbations *by construction*. This can be achieved by constraining the Lipschitz bound of the network. A neural network $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is said to have an ℓ_2 Lipschitz bound of γ if

$$\|f(x_1) - f(x_2)\|_2 \leq \gamma \|x_1 - x_2\|_2, \quad \forall x_1, x_2 \in \mathbb{R}^n. \quad (9.1)$$

The true ℓ_2 Lipschitz constant, denoted $\text{Lip}(f)$, is the smallest γ satisfying (9.1). Lipschitz-bounded networks are “smoother” and less sensitive to input perturbations than unconstrained networks since small variations in their inputs only induce small variations in their outputs. Despite the wealth of recent work on constructing Lipschitz-bounded deep networks [14, 22, 24], to the best of our knowledge, the only method tested in deep RL is spectral normalization [2, 21] which is known to give conservative bounds on the true Lipschitz constant.

This paper therefore investigates the following questions:

1. Can Lipschitz-bounded deep networks improve the empirically observed robustness of control policies in deep RL?
2. If so, does the policy network architecture matter? That is, do more sophisticated policy parameterizations with less-conservative Lipschitz bounds give finer control over the performance-robustness trade-off?

We provide an initial empirical study of Lipschitz-bounded policy networks in deep RL on two representative problems: pendulum swing-up, a classical benchmark problem in control and RL and Atari Pong, a simple proxy for vision-based autonomous decision-making tasks. To the best of our knowledge, our comparison of Lipschitz-bounded policy architectures in deep RL is the first of its kind.

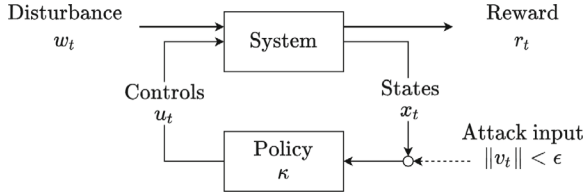


Fig. 9.1 Reinforcement learning and adversarial attacks

9.2 Background and Prior Work

In this section, we first introduce the concepts of deep reinforcement learning and adversarial attacks. We then provide an overview of recent developments in robust neural networks with certifiable Lipschitz bounds.

9.2.1 Deep Reinforcement Learning

We introduce RL from a control-theoretic perspective (Fig. 9.1). Consider a discrete-time nonlinear dynamical system

$$x_{t+1} = f(x_t, u_t, w_t) \quad (9.2)$$

with state vector $x_t \in \mathbb{R}^n$, controlled inputs $u_t \in \mathbb{R}^m$, and disturbances $w_t \in \mathbb{R}^p$. The task is to learn feedback control policies of the form $u_t = \kappa(x_t; \theta)$ parameterized by $\theta \in \mathbb{R}^q$ which (locally and approximately) solve

$$\max_{\theta} R := \mathbb{E}_{x_0, w} \left[\sum_{t=0}^{\infty} \rho^t r_t(x_t, u_t) \right] \text{ s.t. } x_{t+1} = f(x_t, u_t, w_t), u_t = \kappa(x_t; \theta), \quad (9.3)$$

where r_t is the instantaneous reward, $\rho \in (0, 1)$ is a “discount factor,” R is the total expected reward, and the expectation $\mathbb{E}[\cdot]$ is taken over some known distributions of disturbances w_t and initial conditions x_0 . In deep RL, the controller κ is a deep neural network (DNN) parameterized by θ . In this paper, we take Proximal Policy Optimization (PPO) [19] as our training method due to its simplicity, speed, and performance.

9.2.2 Adversarial Attacks for RL

Given a learned policy $\kappa(\cdot; \theta)$, an adversarial attack is an input sequence $v = (v_0, v_1, \dots)$ with some restricted “attack size” $\epsilon > 0$ that is designed to reduce the expected cumulative reward of the policy as much as possible. While attacks can be designed to change the closed-loop behavior in many ways, the most common structure is an additive perturbation to the policy input (i.e., perturbations to the state measurements, Fig. 9.1) [4, 13]. The problem can be formulated as

$$\min_v R \quad \text{s.t.} \quad x_{t+1} = f(x_t, \kappa(x_t + v_t; \theta), w_t), \quad \|v_t\| \leq \epsilon, \quad \forall t, \quad (9.4)$$

where $\|\cdot\|$ can be any p -norm with $1 \leq p \leq \infty$. If the reward function r_t and dynamic model f are both differentiable and known to the attacker, (9.4) can be solved by gradient descent methods. If part of the dynamical system is unknown or not differentiable, solving (9.4) is difficult or impossible and it is common instead to solve the simplified problem:

$$\max_{v_t} \quad \|\kappa(x_t + v_t; \theta) - \kappa(x_t; \theta)\| \quad \text{s.t.} \quad \|v_t\| \leq \epsilon. \quad (9.5)$$

That is, find an admissible attack v_t which leads to large policy output perturbation at each time step, regardless of its effect on the final reward [4]. Problem (9.5) can be solved by attack methods like Projected Gradient Descent (PGD) [7].

9.2.3 Lipschitz-Bounded Deep Networks

The adversarial attack problem in (9.5) is exactly the calculation of the (local) Lipschitz constant of the policy network κ . We can therefore control the effect of adversarial attacks everywhere in state space by bounding the global Lipschitz constant $\text{Lip}(\kappa) \leq \gamma$. Policy networks with smaller γ are then “smoother” and are likely to be more robust to small attacks than those with a large γ .

In deep RL, the policy network κ is often parameterized by a multi-layer perceptron (MLP) or a convolutional neural network (CNN) of the form of

$$\kappa = g_L \circ \sigma \circ g_{L-1} \circ \dots \circ \sigma \circ g_1, \quad (9.6)$$

where σ is a fixed monotone and 1-Lipschitz scalar activation (e.g., ReLU, tanh, sigmoid, etc.) and g_k is a linear layer $g_k(x) = W_k x + b_k$ with W_k, b_k as the weights and biases, respectively. The Lipschitz constraint for (9.6) is then

$$\text{Lip}(\kappa) = \sup_{J_2, \dots, J_L} \|W_L J_L W_{L-1} \dots J_2 W_1\|_2 \leq \gamma, \quad (9.7)$$

where J_k is a diagonal matrix with $0 \leq J_{k,ii} \leq 1$. Since it is NP-hard to compute the exact Lipschitz constant [23], a practical approach is to find an upper bound $\bar{\gamma}$ for $\text{Lip}(\kappa)$ and then impose the constraint $\bar{\gamma} = \gamma$. There are many bound estimation methods resulting in different constructions of Lipschitz networks. A metric to measure the expressive power of Lipschitz networks is *tightness*, which is defined as $\underline{\gamma}/\gamma$ [24] with $\underline{\gamma}$ as the empirical lower Lipschitz bound:

$$\underline{\gamma} := \max_{x \in \mathbb{X}, \|v\|_2 \leq \epsilon} \|\kappa(x+v) - \kappa(x)\|_2 / \|v\|_2, \quad (9.8)$$

where $v \in \mathbb{R}^n$ and $\mathbb{X} \subset \mathbb{R}^n$ is some compact region. Note that $\text{Lip}(\kappa) \in [\underline{\gamma}, \gamma]$.

A simple Lipschitz bound estimation is the layer-wise spectral norm bound $\bar{\gamma}_s = \prod_{k=1}^L \|W_k\|_2$. This approach uses the fact that common activation functions σ are 1-Lipschitz. Thus, we can construct Lipschitz policy networks via scaling factors and 1-Lipschitz linear layers, where three representative examples are given as follows.

- **(SN)** The spectral normalization (SN) layer [9] is a linear layer with weight $W = (1/\rho)A$ where A is a learnable weight, and ρ is its maximal singular value (i.e., $\|W\|_2 = 1$). Lipschitz networks composed of SN layers often have quite loose Lipschitz bounds such that $\underline{\gamma}/\gamma$ is very small.
- **(AOL)** The almost orthogonal Lipschitz (AOL) layer [15] is a linear layer with weight $W = AD$ where A is a learnable matrix and D is a diagonal matrix with $D_{ii} = \sqrt{\sum_j |A^\top A|_{ij}}$. The experimental results in [15] show that the learned weight W tends to have singular values close to 1, which helps to improve the model tightness.
- **(Cayley)** To obtain an even tighter Lipschitz bound, an orthogonal layer was proposed in [22] by leveraging the Cayley transform. Specifically, given a free learnable weight $P \in \mathbb{R}^{n \times n}$, one first obtains a skew symmetric matrix $A = P - P^\top$ and then construct the weight matrix by $W = (I - A)(I + A)^{-1}$. It is easy to verify that $W^\top W = I$ and so all singular values of W are 1.

A tighter and more expressive bound estimation method was recently proposed in [3], which explores both the monotonicity and Lipschitz properties of the activation σ by leveraging the integral quadratic constraint (IQC) framework [8]. Direct (i.e., unconstrained) parameterizations based on IQC were proposed in [16] for deep equilibrium networks, in [1] for residual networks, and in [24] for deep MLPs and CNNs. In particular, [24] shows that the IQC-based approach can lead to a 1-Lipschitz nonlinear layer as follows:

- **(Sandwich)** The Sandwich layer is a 1-Lipschitz nonlinear layer of the form:

$$g(x) = \sqrt{2}A^\top \Psi \sigma(\sqrt{2}\Psi^{-1}Bx + b), \quad (9.9)$$

where $Q = \begin{bmatrix} A & B \end{bmatrix}$ is a semi-orthogonal matrix ($QQ^\top = I$) parameterized by the Cayley transformation, and Ψ is a positive diagonal matrix parameterized via an

exponential mapping. Based on the above 1-Lipschitz layer, we can construct the γ -Lipschitz policy network κ via

$$\kappa(x) = \sqrt{\gamma} g_L \circ g_{L-1} \circ \cdots \circ g_1(\sqrt{\gamma}x). \quad (9.10)$$

Note that (9.10) can be transformed back to the MLP form (9.6).

The Sandwich layer (9.9) is an “expressive” layer architecture in that it contains all 1-Lipschitz linear layers as a special case, and it allows the spectral norm bound of each layer (and their product) to be greater than one [24]. Experimental results in [24] on image datasets show that (9.10) can achieve better performance than 1-Lipschitz linear layers such as AOL and Cayley. It is natural to ask whether similar results hold true for deep RL.

9.3 Experimental Setup

We study two classic RL problems—pendulum swing-up and Atari Pong—to investigate the research questions outlined in Sect. 9.1. Our code and training details are available on GitHub.^{1,2}

Pendulum Swing-up aims to swing a physical pendulum to its upright equilibrium based on the quadratic reward $r_t = -(\alpha_t^2 + 0.1\dot{\alpha}_t^2 + 0.001u_t^2)$, where α_t is the pendulum angle (wrapped to $[-\pi, \pi]$) and u_t is the pendulum torque. The optimal policy is well known to have sharp decision boundaries which make it susceptible to chattering and instability under small measurement perturbations, delays, or uncertainty. We trained MLP policies (9.6) without any bounds on the network Lipschitz constant, and Lipschitz-bounded policies using the Sandwich parameterization (9.9). We investigated the robustness of each policy to two sources of perturbations: (a) sample delays and (b) adversarial attacks with constrained ℓ_2 norm. Adversarial attacks were computed by solving (9.4) over a sequence of four 50-sample windows with gradient descent.

Atari Pong is a video game in which two players each control a paddle that can move up and down and try to deflect a puck into their opponent’s goal (Fig. 9.6). The reward is the net game score, with a maximum score of 21 goals to nil. An automated “computer” player controls the left paddle and the RL policy controls the right paddle. It is a commonly studied benchmark in deep RL [10, 18]. The game can be written as an RL problem (9.3) where the states x_t are grayscale gameplay images and the control actions u_t are discrete paddle movements. We trained a classic, unconstrained CNN model (9.6) and four different Lipschitz-bounded policy architectures (SN, AOL, Sandwich, Cayley) with various Lipschitz bounds. We compared the robustness

¹ <https://github.com/nic-barbara/Lipschitz-RL-MJX>.

² <https://github.com/nic-barbara/Lipschitz-RL-Atari>.

of each policy network to: (a) uniform random noise, (b) PGD attacks (9.5) with constrained ℓ_2 norm, and (c) PGD attacks (9.5) with constrained ℓ_∞ norm.

9.4 Results and Discussion

We first study the advantages of Lipschitz-bounded policy networks in terms of robustness to perturbations and adversarial attacks, using pendulum swing-up as an illustrative example (Sect. 9.4.1). We extend this study to vision-based feedback control in Pong (Sect. 9.4.2) and compare the benefits of different Lipschitz-bounded policy architectures. In Figs. 9.4, 9.5, and Table 9.1, Lipschitz lower bounds $\underline{\gamma}$ were computed for each policy by performing gradient ascent on (9.8).

9.4.1 Illustrative Example—Pendulum Swing-Up

Let us first consider the effect of small Lipschitz bounds on unperturbed policy networks. Figure 9.2 compares the policy landscape and (empirically estimated) local Lipschitz constant in phase space for an unconstrained MLP policy and a Lipschitz-bounded policy composed of Sandwich layers with $\gamma = 4$. The unconstrained policy has sharp decision boundaries, either side of which flips the sign of the control torque (limited to ± 1 N.m). While these sharp changes are optimal in the unperturbed case, it is clear that any small uncertainty in the pendulum’s position or velocity will cause the network to apply a drastically different control action, potentially driving the system to instability. In contrast, imposing a bound on the Lipschitz constant with Sandwich layers visibly smooths the decision boundaries with negligible penalty to the final test reward (-153 for unconstrained and -157 for Lipschitz bounded).

Figure 9.3 illustrates the effect of this smoothing on each policy’s robustness to sample delays and ℓ_2 -optimal adversarial attacks. The simulated trajectories in Fig. 9.3a, b indicate that both policy networks perform similarly in nominal operation. However, when introducing a small sample delay (2 time samples or 0.1 s, Fig. 9.3c, d) or a small adversarial attack ($\epsilon = 0.11$, Fig. 9.3e, f), the unconstrained policy is unable to hold the pendulum stable and upright, whereas the Lipschitz-bounded policy is successful and only exhibits minor oscillations about the equilibrium under adversarial attacks.

We find that this improvement in robustness is highly correlated with the policy’s Lipschitz bound. Comparing unconstrained policies to Lipschitz-bounded policies with various γ in Fig. 9.4, there is a smooth transition from high-to-low robustness to sample delays and ℓ_2 -optimal adversarial attacks as the policy’s Lipschitz bound increases. Interestingly, it appears that there is a “best choice” for γ , and that restricting it to very small values ($\gamma = 3$) harms the closed-loop performance. This is to be expected, since the optimal policy for pendulum swing-up is known to be nonsmooth, hence with excessive regularization it is likely that the network’s parameter space

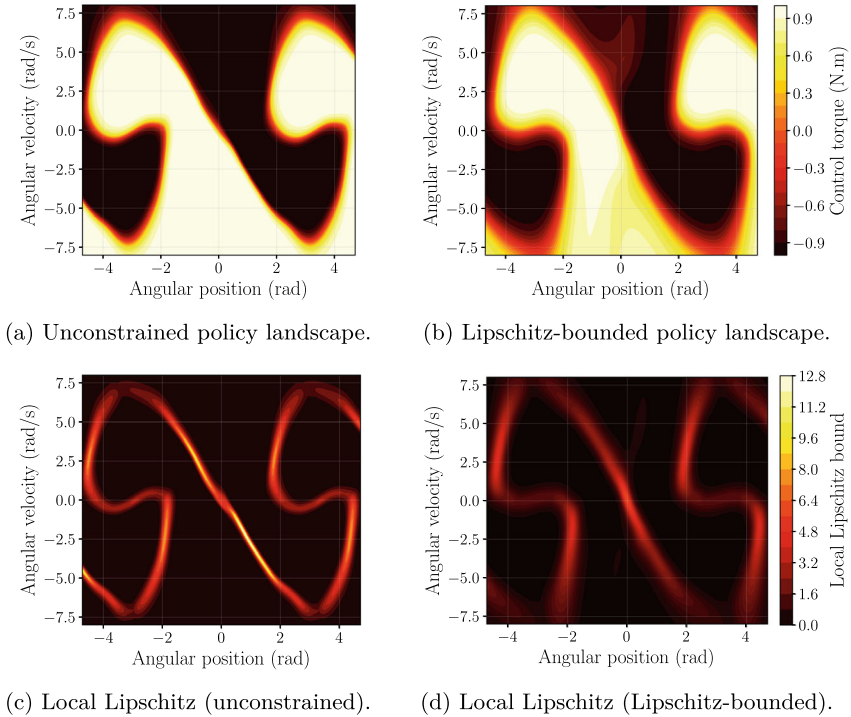


Fig. 9.2 Contours of control actions (a, b) and local Lipschitz bounds (c, d) for an unconstrained (MLP) and a Lipschitz-bounded (Sandwich, $\gamma = 4$) policy show how Lipschitz bounds control a policy’s smoothness

does not contain any high-performing policies. Aside from the policies with $\gamma = 3$, all models were successfully trained to approximate the same final reward (reward curves omitted for brevity). It is therefore clear from Figs. 9.2, 9.3, and 9.4 that, at least in the context of pendulum swing-up, Lipschitz-bounded policy networks significantly improve robustness to disturbances and adversarial attacks over standard, unconstrained networks.

9.4.2 Comparing Architectures—Atari Pong

In the game of Pong, we expect small amounts of image noise to make very little difference to the state of the game and the optimal action. We would therefore hope that smooth, Lipschitz-bounded policies can improve robustness to perturbations like random noise and adversarial attacks. We see this immediately in Fig. 9.5, where we compare the robustness of unconstrained CNN policies and Lipschitz-

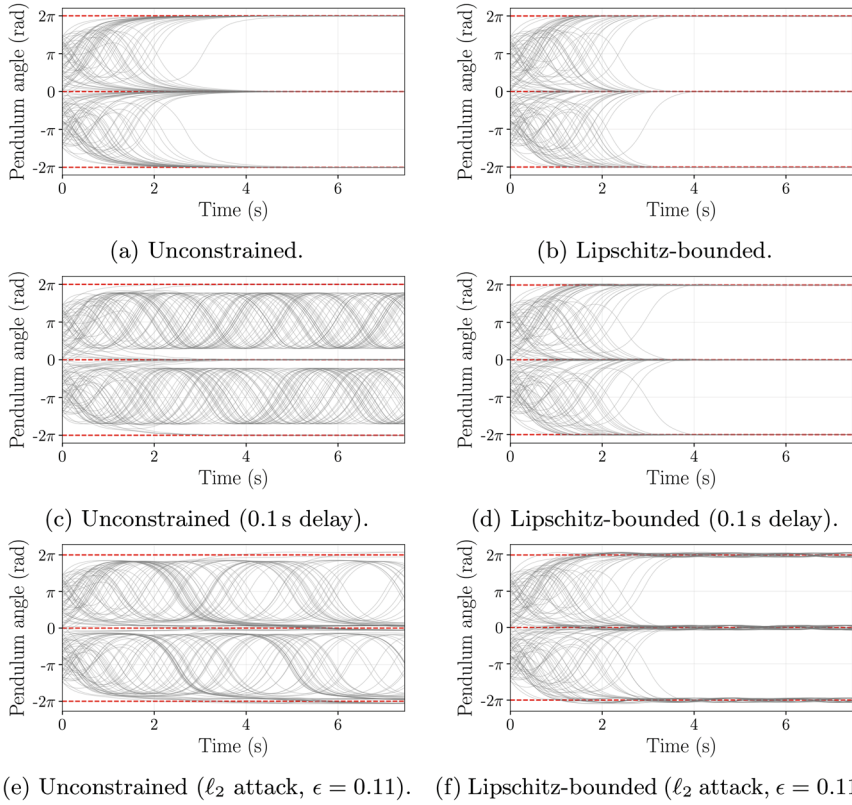


Fig. 9.3 Pendulum trajectories generated by unconstrained (MLP) and Lipschitz-bounded (Sandwich, $\gamma = 4$) policies in nominal operation (a, b), with sample delays (c, d), and with ℓ_2 adversarial attacks (e, f). Red lines indicate the target

bounded Sandwich policies to uniform random noise, ℓ_2 PGD attacks, and ℓ_∞ PGD attacks. The same qualitative results observed for the pendulum in Fig. 9.4 can be seen in Fig. 9.5: there is a smooth transition from high-to-low robustness as γ decreases; robustness is improved not just for ℓ_2 -constrained attacks (which we expect for policies with a small ℓ_2 Lipschitz bound), but also for random noise and ℓ_∞ -constrained attacks, and if γ is too small (e.g., $\gamma = 5$ here), the policy's nominal performance and robustness to perturbations is degraded. In this case, it is possible that the $\gamma = 5$ models have not finished training and could perform better if trained over more epochs (Fig. 9.7d).

It is interesting to look deeper into the effect of adversarial attacks on these models. Figure 9.6 shows just how much of an improvement Lipschitz-bounded policy networks provide in Pong over a standard, unconstrained CNN. The CNN loses the game when subject to very small amounts of random noise and almost imperceptible adversarial attacks. In contrast, the Lipschitz-bounded policy is only

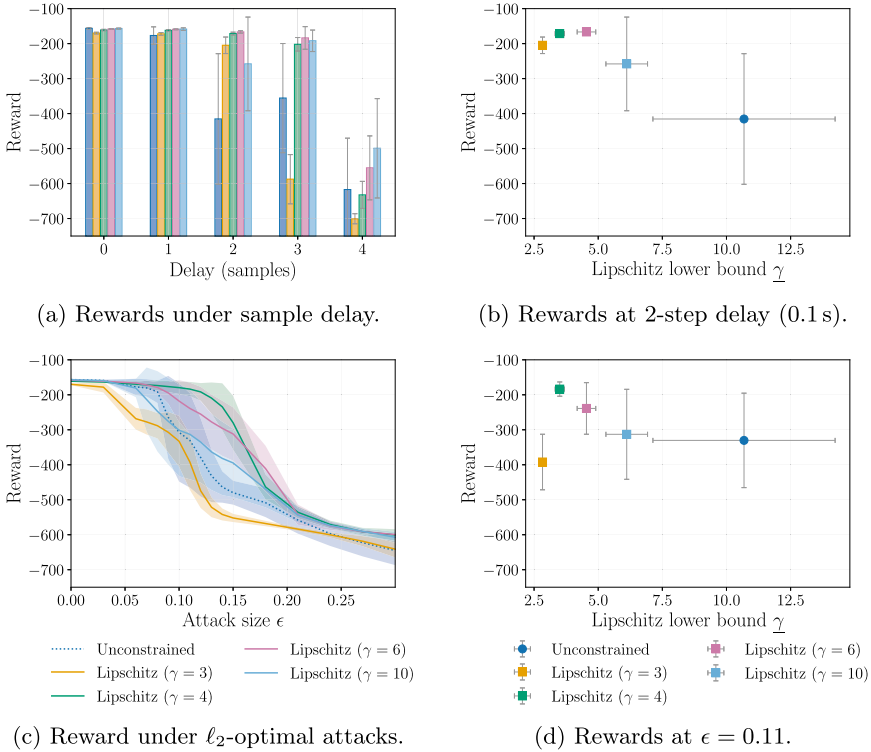
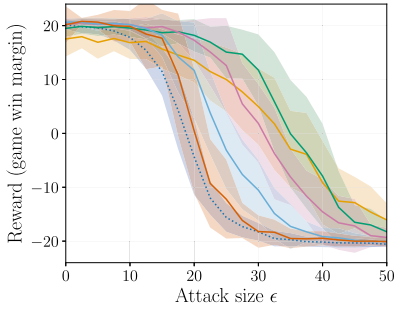


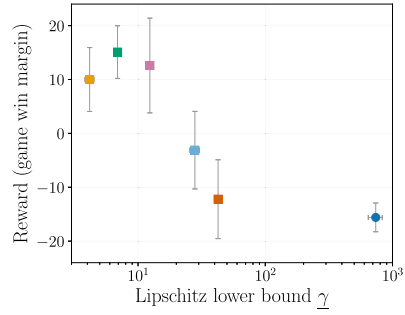
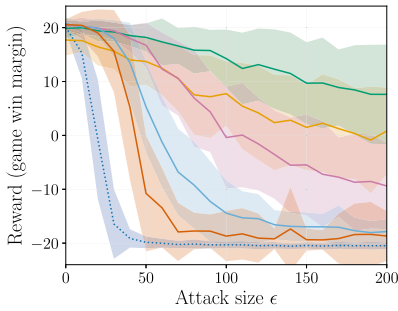
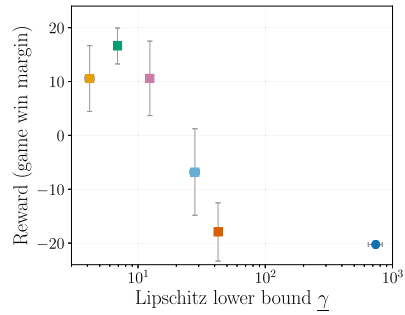
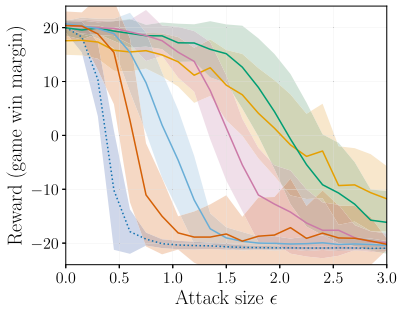
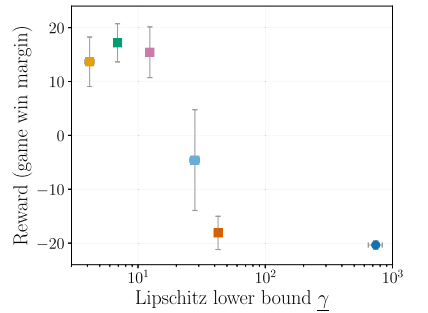
Fig. 9.4 Robust performance of unconstrained (MLP) and Lipschitz-bounded (Sandwich) policies on pendulum swing-up under sample delays and ℓ_2 -optimal adversarial attacks. Panels **b**, **d** show cross sections of **a**, **c** as a function of each model’s empirically estimated Lipschitz lower bound. Bands and error bars show one standard deviation over 10 random model initializations

beaten with a level of random noise that would even make the game difficult for a human. Moreover, highly structured ℓ_2 -constrained attacks are required to beat the Lipschitz-bounded policy. Looking closely at Fig. 9.6, successful ℓ_2 PGD attacks try to trick the policy into thinking the opponent’s paddle is shifted from where it actually is while also trying to hide the exact location of the puck. The ℓ_2 attacks also seem to focus on the white boundary walls of the game. It is less clear why the policies should be sensitive to these features, but we hypothesize that the straight lines of the walls may appear similar to a paddle in feature space after passing through convolutional layers. There is no clear structure to the ℓ_∞ PGD attacks and they remain rather small, since the Lipschitz-bounded policies have a constrained ℓ_2 Lipschitz bound, and the ℓ_2 norm is only a loose upper bound of the ℓ_∞ norm in high-dimensional spaces. The additional robustness to ℓ_∞ PGD attacks over CNN policies is a nice bonus.

So far, we have only investigated Lipschitz-bounded policy networks constructed from Sandwich layers to illustrate that smoother policies can improve robustness



(a) Uniform random noise.

(b) Cross-section at $\epsilon = 25$.(c) ℓ_2 PGD attacks.(d) Cross-section at $\epsilon = 70$.(e) ℓ_∞ PGD attacks.(f) Cross-section at $\epsilon = 1.05$.

..... CNN
 Sandwich ($\gamma = 5$)
 Sandwich ($\gamma = 10$)
 Sandwich ($\gamma = 20$)
 Sandwich ($\gamma = 60$)
 Sandwich ($\gamma = 100$)

..... CNN
 Sandwich ($\gamma = 5$)
 Sandwich ($\gamma = 10$)
 Sandwich ($\gamma = 20$)
 Sandwich ($\gamma = 60$)
 Sandwich ($\gamma = 100$)

Fig. 9.5 Robust performance of unconstrained (CNN) and Lipschitz-bounded (Sandwich) policies for Atari Pong. Bands and error bars show one standard deviation over four random model initializations

in deep RL. It turns out that in addition to choosing its upper bound γ , the layer architecture we use to bound the Lipschitz constant $\text{Lip}(\kappa)$ of a policy κ is extremely

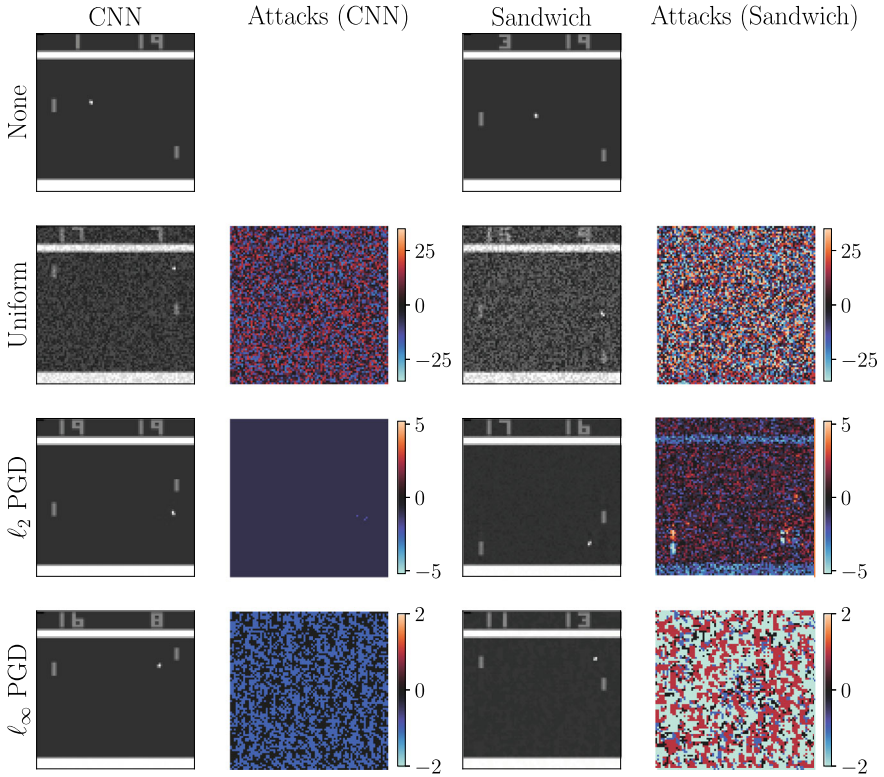


Fig. 9.6 Adversarial examples showing the smallest attacks required to make an unconstrained (CNN) and a Lipschitz-bounded (Sandwich, $\gamma = 10$) policy lose the game. In each image, the “computer” controls the paddle on the left, while the policy controls the paddle on the right. All attacked frames show scenarios where the policy is about to concede a goal (puck moving to the right)

important. Figure 9.7 compares the training curves of all four Lipschitz-bounded policy architectures from Sect. 9.2.3 with unconstrained CNN policies, while Table 9.1 summarizes their nominal and robust performance. It is immediately clear from Fig. 9.7a, b that the two-layer architectures known to have conservative bounds, SN and AOL, perform poorly when γ is small—e.g., when $\gamma \leq 20$, neither architecture ever produces a winning policy. For larger γ , the SN and AOL policies learn to win the game, but their training dynamics are extremely slow in comparison to the CNN (this is a known problem for AOL, see [15, Sect. 7]). Moreover, Table 9.1 shows that the estimated lower bound $\underline{\gamma}$ on $\text{Lip}(\kappa)$ for these policies is very small when γ is small. This suggests that the conservative parameterization of SN and AOL layers restricts their parameter spaces to a small set of very smooth models which does not include high-performing policies. In contrast, Fig. 9.7c, d shows that the two-layer architectures with much tighter bounds on $\text{Lip}(\kappa)$, Cayley and Sandwich,

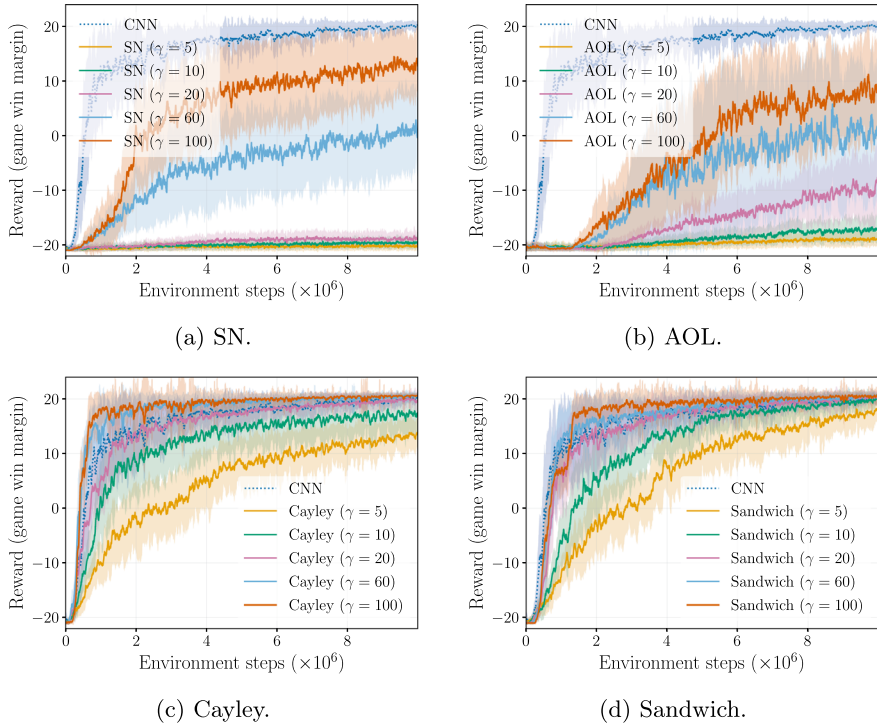


Fig. 9.7 Test reward during training on Atari Pong for all layer architectures. Bands show one standard deviation over four random model initializations

perform quite well even for small choices of γ . The choice of γ still seems to have a strong impact on the training dynamics for these layers—as γ increases, so too does the speed at which the policies converge on a winning strategy. This raises interesting questions about the coupling between a policy’s Lipschitz constant and the exploration of its parameter space, which we leave for future work.

Looking closer at Table 9.1 reveals that simply having a tight Lipschitz bound (i.e., close lower and upper bounds $\underline{\gamma}$ and $\bar{\gamma}$) is not the only factor contributing to a policy’s performance and robustness. Instead, there appears to be an advantage to using “expressive” policy networks, particularly when high robustness (small γ) is required. Table 9.1 indicates that both the Sandwich and Cayley policies exhibit strong performance and robustness for $\gamma \geq 20$. Either layer architecture is therefore a suitable choice for reasonable improvement over existing methods in robust RL like SN. For smaller $\gamma < 20$, however, Sandwich policies are far superior, and with $\gamma = 10$ they are the most robust of any policy across all three input perturbations while still achieving a strong unperturbed reward of 19.6. This is despite the fact that Cayley policies often have a tighter Lipschitz bound than the Sandwich policies (take $\gamma = 10$ as an example). We suggest that this is due to the less conservative parameterization of the Sandwich layers. Each of the SN, AOL, and Cayley policies

Table 9.1 Averaged results (4 random model initializations and 20 random game and attack seeds) on performance and robustness of policy networks trained on Atari Pong. γ is the certified Lipschitz upper bound for a network and $\underline{\gamma}$ is its empirically estimated lower bound. The unperturbed reward is the final mean test reward achieved during training. Results for each attack strategy are the smallest average attack size ϵ required to beat the policy (i.e., reward < 0). Attack results are not provided for policies that did not learn a positive reward. **Bold** values indicate the overall best performing models in each column. Values with an * indicate the best performing models for each γ

Policy	Lipschitz		Reward	Smallest winning attack size ϵ ($\uparrow$)		
	γ	$\underline{\gamma}$	Unperturbed	Uniform	ℓ_2 PGD	ℓ_∞ PGD
CNN	–	738	20.1	18.7	19.5	0.38
SN	5	1.14	−20.2	–	–	–
	10	2.78	−19.4	–	–	–
	20	6.05	−18.7	–	–	–
	60	24.7	1.21	15.0	9.80	0.21
	100	42.4	14.8	21.5	21.3	0.49
AOL	5	1.40	−18.9	–	–	–
	10	2.95	−16.9	–	–	–
	20	7.62	−9.28	–	–	–
	60	15.3	1.31	12.4	19.2	0.52
	100	17.4	8.85	23.6*	42.3	1.20*
Cayley	5	4.56	13.6	23.7	106	1.52
	10	8.99	17.8	28.7	154	1.59
	20	16.6	20.0	23.1	129*	1.36
	60	34.8	20.6*	20.9	67.4*	0.98*
	100	47.0	20.4*	17.1	54.0*	0.76
Sandwich	5	4.16	17.5*	33.4*	183*	2.01*
	10	6.90	19.5*	35.0*	> 200*	2.08*
	20	12.4	20.2*	30.8*	99.2	1.53*
	60	28.0	20.6*	23.9*	58.1	0.94
	100	42.5	20.2	20.1	43.3	0.63

are composed of linear layers with a spectral norm of approximately 1 (exactly 1 for Cayley). In contrast, Sandwich layers have no direct restriction on their spectral norm [24, Fig. 4], and instead constrain $\text{Lip}(\kappa)$ via a composition of nonlinear layers which are a complete parameterization of *all* networks satisfying the tightest known bounds on the Lipschitz constant of DNNs [3], of which 1-Lipschitz linear layers are a special case. This expressivity allows the Sandwich models to converge on policy networks that are both performant and robust even when their parameter space is restricted by a small value of γ , allowing finer control over the performance-robustness trade-off in deep RL.

9.5 Conclusions

This paper has studied the robustness benefits of Lipschitz-bounded policy networks in deep RL. We have found that policy networks with small Lipschitz bounds are significantly more robust to perturbations such as disturbances, random noise, and targeted adversarial attacks. Moreover, we have observed that choosing a policy with non-conservative Lipschitz bounds and an expressive layer architecture gives the user finer control over the performance-robustness trade-off than existing methods based on spectral normalization. This raises interesting questions for future study, such as whether Lipschitz-bounded policy networks can augment or alleviate the need for adversarial training, and whether the observed benefits can be transferred to real-world robotic systems.

Acknowledgements This work was supported in part by Google LLC, the Australian Research Council (DP230101014), and the Centre for Advanced Defence Research in Robotics and Autonomous Systems (CADR-RAS). The authors would also like to thank Jack Naylor for his assistance with computational resources.

References

1. Araujo, A., Havens, A.J., Delattre, B., Allauzen, A., Hu, B.: A unified algebraic perspective on Lipschitz neural networks. In: International Conference on Learning Representations (2023)
2. Bjorck, J., Gomes, C.P., Weinberger, K.Q.: Towards deeper deep reinforcement learning. *Adv. Neural Inf. Process. Syst.* (2021)
3. Fazlyab, M., Robey, A., Hassani, H., Morari, M., Pappas, G.J.: Efficient and accurate estimation of Lipschitz constants for deep neural networks. *Adv. Neural Inf. Process. Syst. (NeurIPS)* (2019)
4. Huang, S., Papernot, N., Goodfellow, I., Duan, Y., Abbeel, P.: Adversarial attacks on neural network policies. In: International Conference on Learning Representations (2017)
5. Kalashnikov, D., Irpan, A., Pastor, P., Ibarz, J., Herzog, A., Jang, E., Quillen, D., Holly, E., Kalakrishnan, M., Vanhoucke, V., Levine, S.: Scalable deep reinforcement learning for vision-based robotic manipulation. In: *Proceedings of The 2nd Conference on Robot Learning*, pp. 651–673 (2018)
6. Kumar, A., Levine, A., Feizi, S.: Policy smoothing for provably robust reinforcement learning. In: International Conference on Learning Representations (2022)
7. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. In: International Conference on Learning Representations (2018)
8. Megretski, A., Rantzer, A.: System analysis via integral quadratic constraints. *IEEE Trans. Autom. Control* **42**(6), 819–830 (1997)
9. Miyato, T., Kataoka, T., Koyama, M., Yoshida, Y.: Spectral normalization for generative adversarial networks. In: International Conference on Learning Representations (2018)
10. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015)
11. Nie, B., Ji, J., Fu, Y., Gao, Y.: Improve robustness of reinforcement learning against observation perturbations via ℓ_∞ lipschitz policy networks. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 14457–14465 (2024)

12. Oikarinen, T., Zhang, W., Meche, M., Megretski, A., Daniel, L., Weng, T.W.: Robust deep reinforcement learning through adversarial loss. *Adv. Neural Inf. Process. Syst.* **34**, 26156–26167 (2021)
13. Pattanaik, A., Tang, Z., Liu, S., Bommannan, G., Chowdhary, G.: Robust deep reinforcement learning with adversarial attacks. In: *International Joint Conference on Autonomous Agents and Multiagent Systems*, vol. 3, pp. 2040–2042 (2017)
14. Pauli, P., Koch, A., Berberich, J., Kohler, P., Allgower, F.: Training robust neural networks using Lipschitz bounds. *IEEE Control Syst. Lett.* **6**, 121–126 (2022)
15. Prach, B., Lampert, C.H.: Almost-orthogonal layers for efficient general-purpose Lipschitz networks. In: *European Conference on Computer Vision*, pp. 350–365. Springer (2022)
16. Revay, M., Wang, R., Manchester, I.R.: Lipschitz bounded equilibrium networks. [arXiv:2010.01732](https://arxiv.org/abs/2010.01732) (2020)
17. Rudin, N., Hoeller, D., Reist, P., Hutter, M.: Learning to walk in minutes using massively parallel deep reinforcement learning. *Proc. Mach. Learn. Res.* **164**, 91–100 (9 2021)
18. Russo, A., Proutiere, A.: Towards optimal attacks on reinforcement learning policies. In: *American Control Conference*, pp. 4561–4567 (2021)
19. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347) (2017)
20. Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., Fergus, R.: Intriguing properties of neural networks. In: *International Conference on Learning Representations* (2013)
21. Takase, R., Yoshikawa, N., Mariyama, T., Tsuchiya, T.: Stability-certified reinforcement learning control via spectral normalization. *Mach. Learn. Appl.* **10**, 100409 (2022)
22. Trockman, A., Kolter, J.Z.: Orthogonalizing convolutional layers with the Cayley transform. In: *International Conference on Learning Representations* (2021)
23. Virmaux, A., Scaman, K.: Lipschitz regularity of deep neural networks: analysis and efficient estimation. *Adv. Neural Inf. Process. Syst.* **31** (2018)
24. Wang, R., Manchester, I.: Direct parameterization of Lipschitz-bounded deep networks. In: *International Conference on Machine Learning* (2023)
25. Wu, F., Li, L., Huang, Z., Vorobeychik, Y., Zhao, D., Li, B.: Crop: certifying robust policies for reinforcement learning through functional smoothing. In: *International Conference on Learning Representations* (2022)

Chapter 10

Solving Partial Differential Equations with Equivariant Extreme Learning Machines



Hans Harder, Jean Rabault, Ricardo Vinuesa, Mikael Mortensen,
and Sebastian Peitz

Abstract We utilize extreme learning machines for the prediction of partial differential equations (PDEs). Our method splits the computational domain into multiple windows that are predicted individually using a single model. Despite requiring only few data points (in some cases, our method can learn from a single full-state snapshot), it still achieves high accuracy and can predict the flow of PDEs over long time horizons. Moreover, we show how additional symmetries can be exploited to increase sample efficiency and to enforce equivariance.

Keywords Extreme learning machines · Partial differential equations · Data-driven prediction · High-dimensional systems

10.1 Introduction

The efficient modeling, prediction, and control of complex systems governed by partial differential equations (PDEs) via data-driven and machine-learning-based techniques has received massive attention in recent years. PDEs are an extremely important class of equations that govern the physics of many applications such as heat transfer, continuum mechanics, fluid dynamics, or quantum mechanics.

H. Harder (✉) · S. Peitz
Department of Computer Science, Paderborn University, Paderborn, Germany
e-mail: hans.harder@uni-paderborn.de

J. Rabault
Independent Researcher, Oslo, Norway

R. Vinuesa
FLOW, Engineering Mechanics, KTH, Stockholm, Sweden

M. Mortensen
Department of Mathematics, University of Oslo, Oslo, Norway

In the past decades, we have seen a plethora of various surrogate-modeling techniques for PDEs. They are useful for a few reasons: (1) in the absence of exact equations or in case of noise, they can learn equations purely from data; (2) surrogate models can often predict larger time steps than conventional solvers, improving computational efficiency; (3) they can automatically identify underlying low-dimensional structures and thereby mitigate the curse of dimensionality [10]; and (4) when implemented in modern deep learning frameworks, they are differentiable by definition (this is useful for control-based applications). Popular examples are the proper orthogonal decomposition [16, 29, 35]—where we project the known equations onto a low-dimensional subspace spanned by basis functions learned from data—as well as many non-intrusive methods such as the dynamic mode decomposition [34], i.e., the Koopman operator framework [22, 24, 28, 33]. Quite unsurprisingly, deep learning plays a central role as well, see [25, 41, 43, 45]. More recently, the notion of physics-informed machine learning [6, 14] has become increasingly popular. There—instead of treating the system as a black box—one encodes system knowledge into the training process, for instance, in the form of the governing equations [8, 19, 21, 32]. This significantly reduces the amount of required training data, while at the same time improving robustness. In addition, one may enforce symmetries—also referred to as geometric deep learning [1] or group convolutional neural networks [15] in the machine learning community—to increase efficiency, see, for instance, [3, 7] for translational and rotational symmetries in image classification. Using group convolutional neural networks to leverage either invariance or equivariance properties of the problem considered (depending on the underlying problem and the task to accomplish) can result in significant improvement in model performance and robustness, as well as improved data efficiency during the training phase [17]. In reinforcement-learning-based control of PDEs, the exploitation of translational symmetries gives rise to efficient multi-agent concepts [9, 30, 38, 44]. For complex control tasks where the underlying problem presents strong symmetries which the controller should be aware of, but where these symmetries should be broken to reach a better state, one may obtain significant gains by combining concepts of multi-agent control, group convolutional neural network, and positional encoding [12]. This results in controllers that are symmetries-aware, but that are also able to choose to actively break existing symmetries if necessary.

When it comes to symmetry exploitation in data-driven modeling of PDEs, mostly translational equivariance has been exploited until now [26, 27, 31, 39]. These works are based on reservoir computing, i.e., recurrent architectures, and it is well known that these can be challenging to train and require a substantial initialization phase to adjust the latent variables before they can make useful predictions. The goal of this paper is (1) to increase the sample efficiency while avoiding recurrence and (2) to include symmetries beyond translation. To this end, we are going to make use of extreme learning machines (ELMs) [4, 5, 11, 40] to learn the flow map of chaotic PDEs. ELMs are two-layer networks that can be trained using simple linear regression. When exploiting translational equivariance similar to [27], ELMs can be trained from few time steps and make accurate predictions over long time horizons (Sect. 10.3). In some instances, a *single* time step suffices to learn a precise

approximation of the PDE. Including additional symmetries (i.e., rotations) further increases the efficiency (Sect. 10.3.3). All simulations have been implemented using the spectral PDE solver *Shenfun* [23]. Our Python code for the experiments is freely available under <https://github.com/graps1/ELM-PDEs>.

10.2 Preliminaries

Consider a continuous-time dynamical system on some state space $\mathcal{F}$,

$$\Phi : [0, \infty) \times \mathcal{F} \rightarrow \mathcal{F}, (t, v) \mapsto \Phi_t(v).$$

In our setting, $\mathcal{F}$ is a function space (e.g., Sobolev space) containing scalar fields of the form $v : \Omega \rightarrow \mathbb{R}$, where $\Omega \subseteq \mathbb{R}^n$ is a set of n -dimensional spatial coordinates. The dynamical system is induced by a PDE of the form

$$\partial_t v(t, x) = f(v(t, x), Dv(t, x), D^2v(t, x), \dots, D^n v(t, x)), \quad (10.1)$$

where $v(t) := v(t, \cdot) \in \mathcal{F}$ and $D^k v(t, x)$ is a vector containing all spatial derivatives of order k of $v(t)$ evaluated at x . Throughout this text, we have $\Omega = [0, L]^n$, $L > 0$, and assume periodic boundary conditions, that is, $v(t, 0) = v(t, L)$ for $x \in \{0, L\}^n$. The associated flow satisfies

$$\Phi_{\Delta t}(v(t)) = v(t + \Delta t),$$

where $\Delta t \geq 0$ is often fixed and hence omitted. Note that we use v to denote both time-dependent states ($v(t) = v(t, \cdot) \in \mathcal{F}$) as well as individual states ($v \in \mathcal{F}$). The meaning will be clear from the context.

We study the Cahn–Hilliard (CH) equation in two spatial dimensions and three variants of the Kuramoto–Sivashinsky (KS) equation in one or two spatial dimensions; all equations are considered for periodic boundary conditions. The Cahn–Hilliard equation is given by

$$\partial_t v = -\Delta v - \gamma \Delta^2 v + \Delta^2(v^3), \quad (10.2)$$

where Δ and Δ^2 are the Laplace and biharmonic operators, and γ is a parameter. The first variant of the KS equation is defined in two spatial dimensions by

$$\partial_t v = -\Delta v - \Delta^2 v - \frac{1}{2} |\nabla v|^2, \quad (10.3)$$

where ∇ is the gradient. Another variant is given for one spatial dimension by

$$\partial_t v = -\partial_{xx}^2 v - \partial_{xxx}^4 v - (\partial_x v)v, \quad (10.4)$$

which is obtained by taking (10.3) for the case of one spatial dimension, differentiating with respect to x and then substituting $\partial_x v$ by v . This form is popular in fluid dynamics, and, due to its chaotic behavior, often used as a test bed for learning algorithms [27, 31, 39]. Finally, we consider a variant of (10.4) introduced in [27] that has an explicit dependence on the spatial coordinate:

$$\partial_t v = -\partial_{xx}^2 v - \partial_{xxx}^3 v - (\partial_x v)v + \mu \cos(2\pi x/\lambda), \quad (10.5)$$

where μ and λ are also parameters.

There are multiple ways to handle the spatial dimensions of a PDE, one common approach being the discretization of space into a grid where every node stores an estimate of the state at that coordinate. For a one-dimensional system like (10.4), the state is approximated by a vector, in the case of two dimensions by a matrix, etc. For example, in case of one spatial dimension, one can discretize $\Omega = [0, L]$ into a grid of m evenly distributed nodes by introducing

$$\bar{v} \in \mathbb{R}^m, \quad \bar{v}_i = v(i \Delta x), \quad \text{for } i = 0, \dots, m-1 \text{ and } \Delta x = L/m.$$

One then models the flow of $v(t) \in \mathcal{F}$ in the discretized space, $\bar{v}(t) \mapsto \bar{v}(t + \Delta t)$.

To approximate spatial derivatives, one can then use finite-difference coefficients (or stencils for more than one dimension) [18, 37] that linearly aggregate the values of neighboring nodes. In our setting (more formally, for hyperbolic PDEs), this means that the value of the temporal derivative at a node's position depends only on a local neighborhood around it and its spatial coordinate. Again for $\Omega = [0, L]$, this means that for sufficiently small Δt , one can approximate

$$\bar{v}_i(t + \Delta t) \quad \text{from} \quad \bar{v}_{i-\ell}(t), \dots, \bar{v}_{i+\ell}(t),$$

that is, from a few “neighboring” values of its previous state, where ℓ is usually small. (In the case of spatial inhomogeneities, one also has a dependence on i .) When it comes to *learning* these PDEs, this is commonly exploited using CNN-based architectures [20, 32, 41]. Our method is similar, but we explicitly train one small model that is applied everywhere.

Extreme learning machines

We use extreme learning machines (ELMs) [4, 11] to predict the flow. Simply put, ELMs are two-layered neural networks that are trained in a certain way: The weights of the first layer are chosen randomly (in our case, always uniformly from $-\sqrt{k}, \dots, \sqrt{k}$, where $k = 1/l_{\text{in}}$ and l_{in} is the number of inputs), and only the second bias-free layer is trained. In contrast to the usual gradient-based methods, this is done by directly solving a least-square problem. Given a pair of random variables (z, z^+) modeling the input–output relationship with values in $\mathbb{R}^{l_{\text{in}}}$ and $\mathbb{R}^{l_{\text{out}}}$, respectively, ELMs define a mapping $E : \mathbb{R}^{l_{\text{in}}} \rightarrow \mathbb{R}^{l_{\text{out}}}$ by solving

$$\min_{\Theta \in \mathbb{R}^{l_{\text{out}} \times l_{\text{hid}}}} \mathbb{E}[\|E(\mathbf{z}) - \mathbf{z}^+\|_2^2], \quad E(\mathbf{z}) = \Theta \phi(\mathbf{z}), \quad (10.6)$$

where l_{hid} is the hidden dimension and $\phi : \mathbb{R}^{l_{\text{in}}} \rightarrow \mathbb{R}^{l_{\text{hid}}}$ composes the first layer with the activation function. Therefore, only the linear output layer, parameterized by Θ , is learned. Setting the derivative of the loss with respect to Θ to zero, we may express the optimal solution as

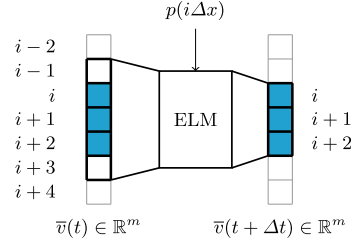
$$\Theta^* = \mathbb{E}[\mathbf{z}^+ \phi(\mathbf{z})^T] \mathbb{E}[\phi(\mathbf{z}) \phi(\mathbf{z})^T]^{-1} = C D^{-1}. \quad (10.7)$$

The $l_{\text{out}} \times l_{\text{hid}}$ and $l_{\text{hid}} \times l_{\text{hid}}$ -matrices C and D are estimated—not computed exactly—by averaging finitely many known input–output pairs. Note that reservoir computers (e.g., [26]) are trained very similarly; they are the recurrent pendant to the ELM’s feedforward architecture. The mapping ϕ can be seen as a random l_{hid} -dimensional dictionary of observables. That is, it takes the data and maps it into a potentially higher dimension, making it easier to learn a linear mapping from embedded input to output. This training procedure is shared by, or at least similar to, many other approaches such as sparse identification of nonlinear dynamics (SINDy) [2], radial basis function networks or extended dynamic mode decomposition [42]. Due to the random initialization, an advantage of ELMs is that the hyperparameter tuning is omitted up to choosing an activation function and an initialization strategy for the first layer. Its advantage over gradient-based methods is that the optimal solution is highly precise and recovered almost instantaneously.

10.3 Extreme Learning Machines for PDEs

When it comes to predicting PDEs, a simple and frequently used test bed is the one-dimensional KS equation, which comes in a spatially inhomogeneous (10.5) and homogeneous variant (10.4). With regard to this setting, most approaches are based on recurrent neural networks or more specifically reservoir computing [27, 31, 39]. Falling into the latter category, the method in [27] trains one (in case of spatial homogeneity) or multiple (in case of spatial inhomogeneity) reservoirs that handle different parts of the space. We employ a similar approach but always train a single ELM that is applied everywhere. Essentially, our implementation shifts a window over the coordinate space and predicts the PDE’s state at the succeeding and smaller window, with an optional “positional encoding” that tells the ELM its current location. Besides demonstrating our approach on Eqs. (10.4) and (10.5), we show that complex systems such as (10.3) and (10.2) in two spatial dimensions are also learnable; moreover, for the case of the two-dimensional KS equation, we show how one can exploit multiple inherent symmetries.

Fig. 10.1 Sliding-window approach with an extent of $\ell = 1$ and a stride of $s = 3$



10.3.1 Model Architecture and Training

We describe our method for the one-dimensional case only, since its extension to two or more dimensions is straight forward. In this case, the ELM is a function of the form $E : \mathbb{R}^{2\ell+s} \rightarrow \mathbb{R}^s$, where ℓ is the *extent* and s the *stride*. Intuitively, s defines how many values are predicted simultaneously, while ℓ defines the amount of “extra information” that is taken into account left and right; see Fig. 10.1 for an illustration. In two dimensions, the window is a matrix, and the value at a certain location is approximated by a function accepting a two-dimensional window as its input. Taking the (periodic) boundary conditions into account, one can then view the ELM as an approximation to the flow $\bar{v}(t) \mapsto \bar{v}(t + \Delta t)$ by “gluing” copies of the same model together. The training is done following (10.6) and (10.7), where $\mathbf{z}$ and $\mathbf{z}^+$ are randomly sampled input–output windows: Taking states from a single trajectory by sampling a random time t and a random index i , we define $\mathbf{z}$ and $\mathbf{z}^+$ by

$$\begin{aligned} \mathbf{z} &= (\bar{v}_{i-\ell}(t), \dots, \bar{v}_{i+s-1+\ell}(t)) && \text{(with values in } \mathbb{R}^{2\ell+s}), \\ \mathbf{z}^+ &= (\bar{v}_i(t + \Delta t), \dots, \bar{v}_{i+s-1}(t + \Delta t)) && \text{(with values in } \mathbb{R}^s). \end{aligned}$$

For an explicit dependence on the spatial coordinate as in (10.5), one can include information about the model’s current location in the mapping, which we do by applying a positional encoding and concatenating it with the model’s input. In that case, we found it best to work with a transformer-inspired binary-like encoding of the position:

$$p(x) = (\cos(\pi x/L), \cos(2\pi x/L), \cos(4\pi x/L), \dots, \cos(2^k \pi x/L)),$$

with the idea that the model could “branch” on the components of $p(x)$ whenever it needs to know its current location. The positional encoding is then concatenated with the input window (in this case, $E : \mathbb{R}^{2\ell+s+k+1} \rightarrow \mathbb{R}^s$).

10.3.2 Experiments

In this section, we conduct multiple experiments on different PDEs. Every experiment illuminates a different aspect of the proposed method: (1) for the homogeneous one-dimensional KS equation, we train multiple models and demonstrate their robustness for different initializations; (2) for its inhomogeneous variant, we show that positional encodings allow the model to recover typical patterns to a higher degree than in their absence; (3) we show that a *single* input–output pair suffices for training an ELM on the two-dimensional CH and KS equations; and (4) for the two-dimensional KS equation, we show that multiple system-inherent symmetries can be exploited, giving rise to *equivariant* ELMs.

For the general setup, we always use the softplus nonlinearity (which worked best for all tested nonlinearities), apply min-max normalization, and train the ELM by adding zero-mean Gaussian noise on the input (this is essential for the model’s stability). The remaining parameters are shown in Table 10.1—these values were chosen rather ad hoc, but there is a general trade-off between speed and precision: If speed is a priority, one should use a small hidden dimension, a small extent, a large stride (so that more values are predicted in parallel). For precision, one should do the opposite.

Training is done as described in the previous section; we iteratively pick a random snapshot and a window of input-dimensional size, compute its embedding and the target window, and improve the estimates of C and D from (10.7) using running averages. To quantify prediction quality, we use the squared error between simulated ($\bar{v}$) and predicted ($\hat{v}$) states that is then normalized by the performance of a “baseline”

Table 10.1 Experimental setup of both simulation and ELM for different PDEs

Parameter	Hom. KS (10.4)	Inhom. KS (10.5)	2d KS (10.3)	2d CH (10.2)
Domain size L	200	200	60π	100
Grid size m	512	512	256	512
Initial state $\bar{v}(0)$	On attractor	On attractor	On attractor	Uniformly $-1 \dots 1$
Time discretization Δt	0.05	0.05	0.05	0.05
Additional parameters	–	$\lambda = 50, \mu = 0.05$	–	$\gamma = 0.5$
Extent ℓ	7	7	2	4
Stride s	4	4	4	4
Hidden units l_{hid}	150	150	600	500
Noise std. σ	10^{-4}	10^{-4}	10^{-4}	10^{-3}
Training samples	20	200	1	1
Positional encoding k	0	3	0	0

model that predicts $\bar{v}(t)$'s average value (the computed quantity is then proportional to $\bar{v}(t)$'s variance when indices are sampled randomly). We obtain the relative squared error (RSE) as a function of time:

$$\text{RSE}(t) = 100\% \cdot \frac{\|\bar{v}(t) - \hat{v}(t)\|_2^2}{\|\bar{v}(t) - \frac{1}{m} \sum_{i=0}^{m-1} \bar{v}_i(t)\|_2^2}.$$

Model statistics (Eq. (10.4)). For the homogeneous KS equation, we show the simulated trajectory together with an ELM-based prediction and their deviation from each other on the left side of Fig. 10.2. To showcase the robustness of this approach, we have trained multiple models for different initial weights and plotted their deviation from the target trajectory on the right side of Fig. 10.2. In all cases, the ELM accurately predicts the first few time steps. The error increases with time, which is expected for chaotic systems, and converges to an average of 200%. The obtained results are similar to that of reservoir-based methods, compare, for example, the right side of Fig. 10.2 with Fig. 5 in [26].

Using positional encodings (Eq. (10.5)). For the inhomogeneous KS equation, Fig. 10.3 shows that positional encodings significantly improve the prediction quality

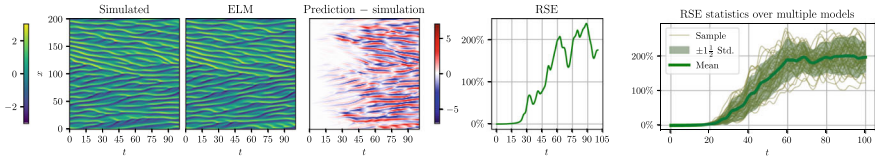


Fig. 10.2 Exemplary simulation of the homogeneous KS system compared to an ELM's prediction (left) and statistics over multiple (~ 50) models (right)

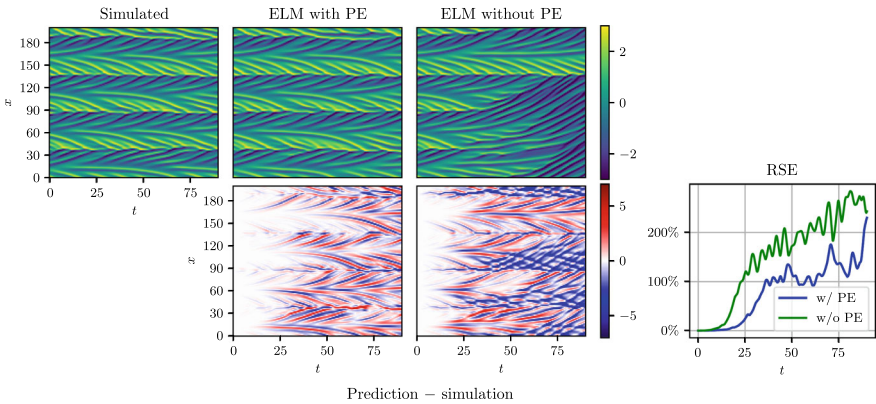


Fig. 10.3 Simulation of the inhomogeneous KS system compared to the ELM's prediction with and without positional encoding (PE)

and allow the ELM to recover typical patterns. In contrast, if there are no positional encodings, then the “peaks” eventually collapse. This is also supported by Fig. 10.4, in which we estimate the first three raw moments of the predicted trajectories as functions of x by averaging over time. (Intuitively, these moments are related to the mean, variance, and skewness of the PDE state at some location x .) Comparing the resulting graphs, the statistics generated by the ELM with positional encodings closely resemble those of the simulated trajectory.

Learning from a single sample (Eqs. (10.2) and (10.3)). The Cahn–Hilliard equation describes phase separation processes in binary mixtures, where two components are separating over time due to differences in concentration. As the equation evolves, the initially small and numerous domains merge or disappear, forming areas where one component is more concentrated than the other, see the top row in Fig. 10.5. It is possible to learn this system using an ELM, and, surprisingly, a single input–output pair suffices to train it. That is, even though the ELM was trained on a single frame

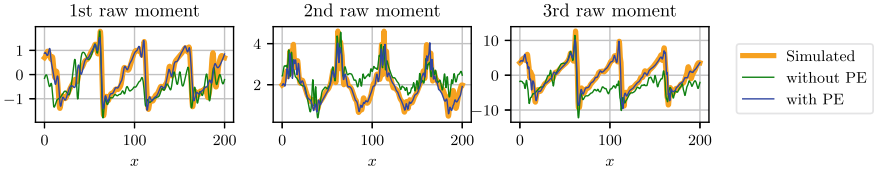


Fig. 10.4 Statistics from the inhomogeneous KS system. The k -th raw moment of a trajectory $\bar{v}$ (simulated or predicted) is $\mathbb{E}[\bar{v}(t)^k]$, where t is sampled uniformly from the time interval. Similar flow statistics were used by [36]

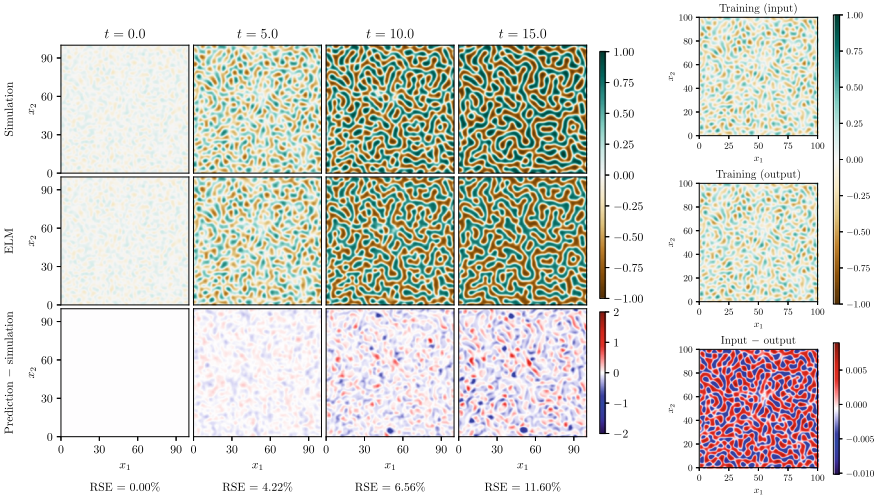


Fig. 10.5 Simulation of the two-dimensional Cahn–Hilliard equation compared to the ELM’s prediction (left), together with the training data (right)

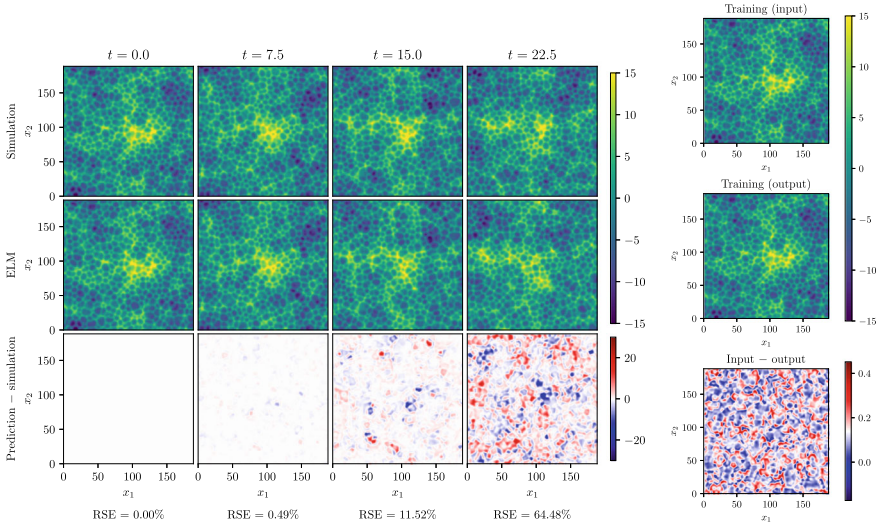


Fig. 10.6 Simulation of the two-dimensional Kuramoto–Sivashinsky equation compared to the ELM’s prediction (left), together with the training data (right)

(right column in Fig. 10.5), it is still able to accurately predict the PDE’s evolution for qualitatively different states (center row of Fig. 10.5). For the KS equation, note that variant (10.3) is different from (10.4) and (10.5). In fact, in contrast to the latter two equations, it is unstable in the sense that the state’s mean diverges over time (e.g., [13]). To obtain consistent behavior, one can solve the equation by zeroing out the state’s mean, apply one or multiple integration steps, and add the previously subtracted mean back in again. For the ELM-based surrogate, we make this part of the pre- and post-processing procedure, so that it always receives zero-mean states as its input. The ELM’s prediction together with the simulated trajectory and their deviation is shown in Fig. 10.6. Again, a single input–output pair suffices for training.

10.3.3 Exploiting More Symmetries and Enforcing Equivariance

The KS system (10.3) for two (and even arbitrary) spatial dimensions admits many symmetries that can be exploited. In particular, it is equivariant with respect to the *Euclidean group* $E(2)$ that contains functions of the form $R : x \mapsto Ax + b$, where $x, b \in \mathbb{R}^2$ and $A \in \mathbb{R}^{2 \times 2}$ is an orthogonal matrix, together with function composition as the group operation. Composition of R with v , i.e., $(v, R) \mapsto v \circ R$, defines a right group action on $\mathcal{F}$. To see that *equivariance* of (10.3) holds, i.e., that

$$\Phi(v \circ R) = \Phi(v) \circ R, \quad (10.8)$$

one can first argue that Δ , Δ^2 and $|\nabla(\cdot)|$ are equivariant (appendix, Lemma 10.2) so that, consequently, equivariance of the temporal derivative is established. This argument extends to the PDE's solution (appendix, Lemma 10.3).

Since we are working with small windows on finite grids, we restrict ourselves to the symmetries of the square—a subgroup of $E(2)$ —defined in our case by appropriate reflections or rotations of windows across their centers. Overall, there are eight group elements (identity; rotations of 90, 180, and 270 degrees; and finally reflections along the horizontal, vertical, and two diagonal axes) that act on the windows (see Fig. 10.7a). We exploit this in two ways: first, we add new artificial samples by letting some or all group elements act on known input–output pairs (see Fig. 10.7a). This decreases the overall number of real samples needed. Second, we use multiple symmetries of one sample to generate a single output and therefore improve the prediction quality. That is, we predict an output window by averaging over some or all group elements as shown in Fig. 10.7b. If we include all eight, this enforces full equivariance. Figure 10.8 shows that both approaches work well. Adding artificial training samples significantly reduces the overall number of “real” samples needed, and using multiple symmetries for prediction improves the model’s accuracy. Notice that both steps can be executed separately: It is possible to train a model on multiple symmetries but to make predictions using less, and vice versa.

Conclusion. An ELM-based approach appears to be highly promising for the task of modeling PDEs. This is due to several reasons. First, since the trained model is very small, it has only low data requirements. Second, it is not recurrent like previous approaches, a fact that significantly simplifies the setup including training process and prediction. Third, it is possible to exploit additional system-inherent symmetries

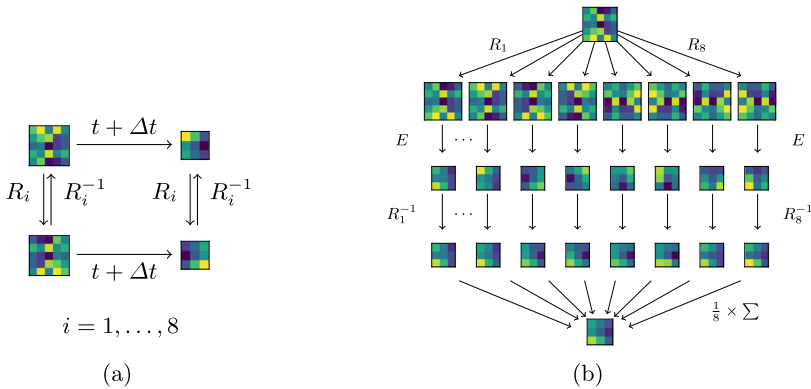


Fig. 10.7 Schemes describing how the system’s equivariance is transferred and can be used on a window level. (Stride $s = 3$ and extend $\ell = 1$). **(a)** Transforming an input window, evolving it forward and transforming it back yields the same result. Both top and bottom row contain valid input–output pairs. **(b)** Equivariance can be enforced by transforming the input using each group element, applying the ELM, inverting each transformation and averaging the outputs

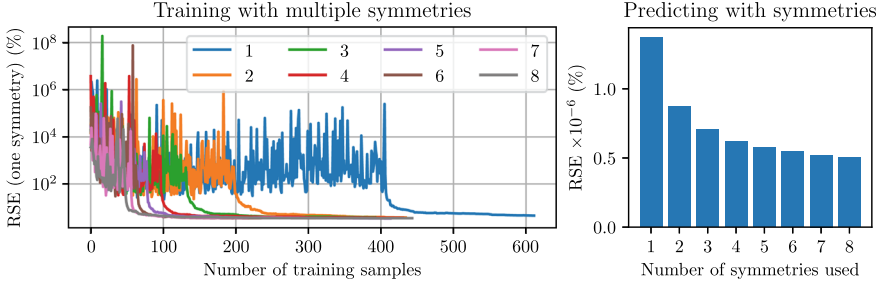


Fig. 10.8 Symmetries can be used to increase data efficiency (left) and to improve the prediction quality (right)

to decrease the number of necessary training samples and to improve the prediction quality.

Acknowledgements HH and SP acknowledge financial support by the project “SAIL: SustAInable Life-cycle of Intelligent Socio-Technical Systems” (Grant ID NW21-059D), which is funded by the program “Netzwerke 2021” of the Ministry of Culture and Science of the State of North Rhine Westphalia, Germany.

A Appendix

Let $E(n)$ be the Euclidean group of dimension n , let $R \in E(2)$ and denote $R(x) = Ax + d$, where $A \in \mathbb{R}^{n \times n}$ is orthogonal and $d \in \mathbb{R}^n$ arbitrary. For $v : \mathbb{R}^n \rightarrow \mathbb{R}$ and $\mathbf{i} \in J^m$, denote $\partial_{\mathbf{i}}^m v := \partial_{i_1, \dots, i_m}^m v$ and $|\nabla v|(x) := |\nabla v(x)|$.

Lemma 10.1 *If $v : \mathbb{R}^n \rightarrow \mathbb{R}$ is m -times differentiable and $\mathbf{i} \in J^m$, then*

$$\partial_{\mathbf{i}}^m (v \circ R)(x) = \sum_{\mathbf{k} \in J^m} (\partial_{\mathbf{k}}^m v \circ R)(x) \prod_{s=1}^m A_{k_s, i_s}. \quad (10.9)$$

Proof This is done by induction on m . For $m = 1$, notice that

$$\begin{aligned} \partial_i (v \circ R)(x) &= \lim_{\delta \rightarrow +0} \frac{1}{\delta} (v(R(x + \delta \mathbf{1}_i)) - v(R(x))) \\ &= \lim_{\delta \rightarrow +0} \frac{1}{\delta} (v((Ax + d) + \delta A \mathbf{1}_i) - v(Ax + d)) \\ &= \nabla_{A \mathbf{1}_i} v(Ax + d) = (A \mathbf{1}_i)^T \nabla v(Ax + d) = \sum_{j=1}^n A_{ji} (\partial_j v \circ R)(x), \end{aligned}$$

where $\mathbf{1}_i \in \mathbb{R}^n$ is the indicator vector for index i and $\nabla_{(\cdot)}$ the directional derivative. For $m \geq 1$, we have inductively for $\mathbf{i} \in J^{m+1}$, $\mathbf{i}' = (i_1, \dots, i_m)$, $i = i_{m+1}$:

$$\begin{aligned}
\partial_i^{m+1}(v \circ R)(x) &= \partial_i \partial_i^m (v \circ R)(x) \\
&= \partial_i \sum_{k \in J^m} (\partial_k^m v \circ R)(x) \prod_{s=1}^m A_{k_s, i'_s} \\
&= \sum_{k \in J^m} \left(\sum_{k=1}^n A_{k,i} (\partial_{k,k}^{m+1} v \circ R)(x) \right) \prod_{s=1}^m A_{k_s, i'_s} \\
&= \sum_{k \in J^{m+1}} (\partial_k^{m+1} v \circ R)(x) \prod_{s=1}^{m+1} A_{k_s, i_s}.
\end{aligned}$$

Lemma 10.2 *If $v : \mathbb{R}^n \rightarrow \mathbb{R}$ is four times differentiable and $AA^T = I$, then*

$$|\nabla(v \circ R)| = |\nabla v \circ R|, \quad \Delta(v \circ R) = \Delta v \circ R, \quad \Delta^2(v \circ R) = \Delta^2 v \circ R.$$

Proof Notice that $AA^T = I$ implies $\sum_{i=1}^n A_{k,i} A_{j,i} = 1$ if $k = j$ and 0 otherwise. Denote this fact by (*). Then:

$$\begin{aligned}
|\nabla(v \circ R)(x)|^2 &= \sum_{i=1}^n (\partial_i(v \circ R)(x))^2 \\
&= \sum_{i=1}^n \left(\sum_{j=1}^n A_{ji} (\partial_j v \circ R)(x) \right)^2 \tag{10.9} \\
&= \sum_{i=1}^n \sum_{j_1, j_2=1}^n A_{j_1, i} A_{j_2, i} (\partial_{j_1} v \circ R)(x) (\partial_{j_2} v \circ R)(x) \\
&= \sum_{j_1, j_2=1}^n (\partial_{j_1} v \circ R)(x) (\partial_{j_2} v \circ R)(x) \sum_{i=1}^n A_{j_1, i} A_{j_2, i} \\
&= \sum_{j=1}^n (\partial_j v \circ R)(x)^2 = |\nabla v \circ R(x)|^2. \tag{*}
\end{aligned}$$

$$\begin{aligned}
\Delta(v \circ R)(x) &= \sum_{i=1}^n \partial_{i,i}^2 (v \circ R)(x) \\
&= \sum_{i=1}^n \sum_{j_1, j_2=1}^n A_{j_1, i} A_{j_2, i} (\partial_{j_1, j_2}^2 v \circ R)(x) \tag{10.9} \\
&= \sum_{j_1, j_2=1}^n (\partial_{j_1, j_2}^2 v \circ R)(x) \sum_{i=1}^n A_{j_1, i} A_{j_2, i} \\
&= \sum_{j=1}^n (\partial_{j,j}^2 v \circ R)(x) = (\Delta v \circ R)(x). \tag{*}
\end{aligned}$$

$$\begin{aligned}
\Delta^2(v \circ R)(x) &= \sum_{i=1}^n \sum_{j=1}^n \partial_{i,i,j,j}^4 (v \circ R)(x) \\
&= \sum_{i=1}^n \sum_{j=1}^n \sum_{k \in J^4} A_{k_1, i} A_{k_2, i} A_{k_3, j} A_{k_4, j} (\partial_k^4 v \circ R)(x) \tag{10.9} \\
&= \sum_{k \in J^4} (\partial_k^4 v \circ R)(x) \sum_{i=1}^n A_{k_1, i} A_{k_2, i} \sum_{j=1}^n A_{k_3, j} A_{k_4, j} \\
&= \sum_{k_1=1}^n \sum_{k_3=1}^n (\partial_{k_1, k_1, k_3, k_3}^4 v \circ R)(x) = (\Delta^2 v \circ R)(x). \tag{*}
\end{aligned}$$

□

Lemma 10.3 *Define the explicit Euler method for $v \in \mathcal{F}$, $t \geq 0$, $\delta > 0$ by*

$$\Phi_t^\delta(v) := \begin{cases} v + tf(v) & t \leq \delta \\ \Phi_{t-\delta}^\delta(v) + \delta f(\Phi_{t-\delta}^\delta(v)) & t > \delta, \end{cases}$$

where $f : \mathcal{F} \rightarrow \mathcal{F}$ satisfies $f(v \circ R) = f(v) \circ R$ for all $v \in \mathcal{F}$, and assume that $\lim_{\delta \rightarrow +0} \Phi_t^\delta(v) = \Phi_t(v)$ pointwise for $v \in \mathcal{F}$ and $t \geq 0$. Then

$$\Phi_t(v \circ R) = \Phi_t(v) \circ R. \tag{10.10}$$

Proof We show $\Phi_t^\delta(v \circ R) = \Phi_t^\delta(v) \circ R$ by induction on $M = \lceil t/\delta \rceil$. For $M \leq 1$, we have $\delta \geq t$. Using invariance of f and linearity of the composition operator,

$$\Phi_t^\delta(v \circ R) = (v \circ R) + tf(v \circ R) = (v + tf(v)) \circ R = \Phi_t^\delta(v) \circ R.$$

For the induction hypothesis, fix an $M \geq 1$ such that $\Phi_t^\delta(v \circ R) = \Phi_t^\delta(v) \circ R$ for all $\delta > 0, t \geq 0$ with $\lceil t/\delta \rceil \leq M$. For the induction step, let $\lceil t/\delta \rceil = M + 1$. Then

$$2 \leq M + 1 = \lceil t/\delta \rceil < t/\delta + 1,$$

and therefore $\delta < t$. Thus,

$$\Phi_t^\delta(v \circ R) = \Phi_{t-\delta}^\delta(v \circ R) + \delta f(\Phi_{t-\delta}^\delta(v \circ R)).$$

Since

$$\lceil (t - \delta)/\delta \rceil = \lceil t/\delta - 1 \rceil = \lceil t/\delta \rceil - 1 = M,$$

we can use the induction hypothesis: $\Phi_{t-\delta}^\delta(v \circ R) = \Phi_{t-\delta}^\delta(v) \circ R$. Using invariance of f and linearity of the composition operator, we obtain

$$\Phi_t^\delta(v \circ R) = (\Phi_{t-\delta}^\delta(v) + \delta f(\Phi_{t-\delta}^\delta(v))) \circ R = \Phi_t^\delta(v) \circ R.$$

This concludes the induction. Taking δ to zero, we finally have

$$\begin{aligned} \Phi_t(v \circ R) &= \lim_{\delta \rightarrow +0} \Phi_t^\delta(v \circ R) \\ &= \lim_{\delta \rightarrow +0} (\Phi_t^\delta(v) \circ R) \\ &= (\lim_{\delta \rightarrow +0} \Phi_t^\delta(v)) \circ R = \Phi_t(v) \circ R. \end{aligned}$$

References

1. Bronstein, M.M., Bruna, J., Cohen, T.S., Velicković, P.: Geometric deep learning: grids, groups, graphs, geodesics, and gauges. [arXiv:2104.13478](https://arxiv.org/abs/2104.13478) (2021)
2. Brunton, S.L., Proctor, J.L., Kutz, J.N.: Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proc. Natl. Acad. Sci.* **113**(15), 3932–3937 (2016)
3. Cohen, T.S., Welling, M.: Group equivariant convolutional networks. In: *International Conference on International Conference on Machine Learning*, vol. 33, pp. 2990–2999 (2016)
4. Ding, S., Xinzhen, X., Nie, R.: Extreme learning machine and its applications. *Neural Comput. Appl.* **25**(3–4), 549–556 (2014)
5. Dwivedi, V., Srinivasan, B.: Physics Informed Extreme Learning Machine (PIELM)—A rapid method for the numerical solution of partial differential equations. *Neurocomputing* **391**, 96–118 (2020)
6. Eivazi, H., Tahani, M., Schlatter, P., Vinuesa, R.: Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations. *Phys. Fluids* **34**(7) (2022)
7. Esteves, C., Allen-Blanchette, C., Makadia, A., Daniilidis, K.: Learning SO(3) equivariant representations with spherical CNNs. In: *Proceedings of the European Conference on Computer Vision (ECCV)* (2018)
8. Geneva, N., Zabaras, N.: Modeling the dynamics of PDE systems with physics-constrained deep auto-regressive networks. *J. Comput. Phys.* **403**, 109056 (2020)

9. Guastoni, L., Rabault, J., Schlatter, P., Azizpour, H., Vinuesa, R.: Deep reinforcement learning for turbulent drag reduction in channel flows. *Eur. Phys. J. E* **46**(4) (2023)
10. Han, J., Jentzen, A., Weinan, E.: Solving high-dimensional partial differential equations using deep learning. *Proc. Natl. Acad. Sci.* **115**(34), 8505–8510 (2018)
11. Huang, G.-B., Zhu, Q.-Y., Siew, C.-K.: Extreme learning machine: theory and applications. *Neurocomputing* **70**(1–3), 489–501 (2006)
12. Jeon, J., Rabault, J., Vasanth, J., Alcántara-Ávila, F., Baral, S., Vinuesa, R.: Advanced deep-reinforcement-learning methods for flow control: group-invariant and positional-encoding networks improve learning speed and quality. *arXiv preprint [arXiv:2407.17822](https://arxiv.org/abs/2407.17822)* (2024)
13. Kalogirou, A., Keaveny, E.E., Papageorgiou, D.T.: An in-depth numerical study of the two-dimensional Kuramoto–Sivashinsky equation. *Proc. R. Soc. Math. Phys. Eng. Sci.* **471**(2179), 20140932 (2015)
14. Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Yang, L.: Physics-informed machine learning. *Nat. Rev. Phys.* **3**(6), 422–440 (2021)
15. Knigge, D.M., Romero, D.W., Bekkers, E.J.: Exploiting redundancy: separable group convolutional networks on lie groups. In: *International Conference on Machine Learning*, pp. 11359–11386. PMLR (2022)
16. Kunisch, K., Volkwein, S.: Control of the burgers equation by a reduced-order approach using proper orthogonal decomposition. *J. Optim. Theory Appl.* **102**(2), 345–371 (1999)
17. Lafarge, M.W., Bekkers, E.J., Pluim, J.P.W., Duits, R., Veta, M.: Roto-translation equivariant convolutional networks: application to histopathology image analysis. *Med. Image Anal.* **68**, 101849 (2021)
18. LeVeque, R.J.: *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial and Applied Mathematics (2007)
19. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A.: Fourier neural operator for parametric partial differential equations. [arXiv:2010.08895](https://arxiv.org/abs/2010.08895) (2020)
20. Long, Z., Lu, Y., Ma, X., Dong, B.: Pde-net: learning pdes from data. In: *International Conference on Machine Learning*, pp. 3208–3216. PMLR (2018)
21. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E.: Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **3**(3), 218–229 (2021)
22. Mauroy, A.: Koopman operator framework for spectral analysis and identification of infinite-dimensional systems. *Mathematics* **9**(2495) (2021)
23. Mortensen, M.: Shenfun: high performance spectral Galerkin computing platform. *J. Open Source Softw.* **3**(31), 1071 (2018)
24. Nakao, H., Mezić, I.: Spectral analysis of the Koopman operator for partial differential equations. *Chaos Interdisc. J. Nonlinear Sci.* **30**(11) (2020)
25. Pandey, S., Teutsch, P., Mäder, P., Schumacher, J.: Direct data-driven forecast of local turbulent heat flux in Rayleigh–Bénard convection. *Phys. Fluids* **34**(4), 045106 (2022)
26. Pandey, S., Schumacher, J.: Reservoir computing model of two-dimensional turbulent convection. *Phys. Rev. Fluids* **5**(11), 113506 (2020)
27. Pathak, J., Hunt, B., Girvan, M., Lu, Z., Ott, E.: Model-free prediction of large spatiotemporally chaotic systems from data: a reservoir computing approach. *Phys. Rev. Lett.* **120**(2), 024102 (2018)
28. Peitz, S., Harder, H., Nüske, F., Philipp, F., Schaller, M., Worthmann, K.: Partial observations, coarse graining and equivariance in Koopman operator theory for large-scale dynamical systems. [arXiv:2307.15325](https://arxiv.org/abs/2307.15325) (2023)
29. Peitz, S., Ober-Blöbaum, S., Dellnitz, M.: Multiobjective optimal control methods for the Navier-Stokes equations using reduced order modeling. *Acta Applicandae Mathematicae* **161**(1), 171–199 (2019)
30. Peitz, S., Stenner, J., Chidananda, V., Wallscheid, O., Brunton, S.L., Taira, K.: Distributed control of partial differential equations using convolutional reinforcement learning. *Physica D: Nonlinear Phenomena* **461**, 134096 (2024)

31. Rafayelyan, M., Dong, J., Tan, Y., Krzakala, F., Gigan, S.: Large-scale optical reservoir computing for spatiotemporal Chaotic systems prediction. *Phys. Rev. X* **10**(4), 041037 (2020)
32. Ren, P., Rao, C., Liu, Y., Wang, J.-X., Sun, H.: PhyCRNet: physics-informed convolutional-recurrent network for solving spatiotemporal PDEs. *Comput. Methods Appl. Mech. Eng.* **389**, 114399 (2022)
33. Rowley, C.W., Mezić, I., Bagheri, S., Schlatter, P., Henningson, D.S.: Spectral analysis of nonlinear flows. *J. Fluid Mech.* **641**, 115–127 (2009)
34. Schmid, P.J.: Dynamic mode decomposition of numerical and experimental data. *J. Fluid Mech.* **656**, 5–28 (2010)
35. Sirovich, L.: Turbulence and the dynamics of coherent structures Part I: Coherent structures. *Q. Appl. Math.* **XLV**(3), 561–571 (1987)
36. Srinivasan, P.A., Guastoni, L., Azizpour, H., Schlatter, P., Vinuesa, R.: Predictions of turbulent shear flows using deep neural networks. *Phys. Rev. Fluids* **4**(5), 054603 (2019)
37. Strikwerda, J.C.: *Finite Difference Schemes and Partial Differential Equations*, 2nd edn. Society for Industrial and Applied Mathematics (2004)
38. Vignon, C., Rabault, J., Vasanth, J., Alcántara-Ávila, F., Mortensen, M., Vinuesa, R.: Effective control of two-dimensional Rayleigh-Bénard convection: invariant multi-agent reinforcement learning is all you need. *Phys. Fluids* **35**(6), 065146 (2023)
39. Vlachas, P.R., Pathak, J., Hunt, B.R., Sapsis, T.P., Girvan, M., Ott, E., Koumoutsakos, P.: Backpropagation algorithms and Reservoir Computing in Recurrent Neural Networks for the forecasting of complex spatiotemporal dynamics. *Neural Netw.* **126**, 191–217 (2020)
40. Wang, J., Lu, S., Wang, S.-H., Zhang, Y.-D.: A review on extreme learning machine. *Multimedia Tools Appl.* **81**(29), 41611–41660 (2022)
41. Werner, S., Peitz, S.: Numerical evidence for sample efficiency of model-based over model-free reinforcement learning control of partial differential equations. In: *European Control Conference (ECC)*, pp. 2958–2964. IEEE (2024)
42. Williams, M.O., Kevrekidis, I.G., Rowley, C.W.: A data-driven approximation of the koopman operator: extending dynamic mode decomposition. *J. Nonlinear Sci.* **25**, 1307–1346 (2015)
43. Winovich, N., Ramani, K., Lin, G.: ConvPDE-UQ: convolutional neural networks with quantified uncertainty for heterogeneous elliptic partial differential equations on varied domains. *J. Comput. Phys.* **394**, 263–279 (2019)
44. Zeng, K., Graham, M.D.: Symmetry reduction for deep reinforcement learning active control of chaotic spatiotemporal dynamics. *Phys. Rev. E: Stat. Phys. Plasmas Fluids Relat. Interdisc. Top.* **104**(1), 04210 (2021)
45. Zeng, K., Linot, A.J., Graham, M.D.: Data-driven control of spatiotemporal chaos with reduced-order neural ODE-based models and reinforcement learning. *Proc. R. Soc. Math. Phys. Eng. Sci.* **478**(2267), 20220297 (2022)

Chapter 11

Adaptive Robust L2 Loss Function Using Fractional Calculus



Mert Can Kurucu[✉], Müjde Güzelkaya[✉], Ibrahim Eksin[✉],
and Tufan Kumbasar[✉]

Abstract Robust loss functions are crucial in deep neural network training for managing outliers and noisy data. This paper presents an adaptive, robust variant of the L2 loss function, leveraging the fractional calculus. The fractional derivative order α in the proposed Fractional L2 Loss (FL2L) function acts as a bridge, enabling a family of loss functions ranging from L2 to L1 losses. Specifically, as α increases the FL2L transitions from L2's strict penalty on large residuals (associated with faster convergence), to L1's penalizing less (robustness to outliers). Thus, α acts as an interpretable hyperparameter that adjusts the robustness level of the L2 loss function. Furthermore, we transform α to a dynamic parameter, allowing the FL2L to automatically adapt its loss landscape during gradient-based optimization, improving error minimization performance. Our experiments in linear regression, drone system identification, and battery cycle life prediction demonstrate the FL2L's significant improvements.

Keywords Deep learning · Fractional calculus · Loss function · Adaptive loss · Robust loss

M. C. Kurucu (✉) · M. Güzelkaya · I. Eksin · T. Kumbasar
Department of Control and Automation Engineering, Istanbul Technical University, Istanbul, Türkiye
e-mail: kurucum@itu.edu.tr

M. Güzelkaya
e-mail: guzelkaya@itu.edu.tr

I. Eksin
e-mail: eksin@itu.edu.tr

T. Kumbasar
e-mail: kumbasart@itu.edu.tr

11.1 Introduction

Deep learning has revolutionized many fields in control system theory, from system identification [10] and model predictive control [6] to robotics systems [13] and autonomous vehicle control [9]. Moreover, recent literature demonstrates that the use of fractional calculus has introduced significant flexibility to Deep Neural Network (DNN) architectures [16], activation functions [2, 18, 19], and gradient-based optimization methods [17], among many others [8]. This mathematical approach can also be leveraged to enhance the versatility of a key element in DNN training: the loss function.

It is important to use a loss function that has a proper level of robustness when training DNNs, ensuring that the impact of outlier data is minimized relative to inliers [1, 5, 11]. This is a common concept in various learning scenarios, where a robust loss function such as L1 loss may be favored over a less robust loss like L2 loss, due to its reduced sensitivity to large error values. The Huber loss function is known for taking the strengths of both L2 and L1 losses, adjusting its loss landscape depending on a specific threshold value [7]. Zhou et al. [20] have developed a new kind of asymmetric loss function aimed at increasing robustness in both classification and regression tasks. A variety of other robust loss functions have been formulated to address challenges across diverse domains, with a comprehensive overview available in [4]. The aforementioned loss functions work well in their respective domains by introducing additional hyperparameters to tune. Yet, the selection of an optimal robust loss function prior to training and the manual adjustment of its hyperparameters can be challenging and often depends on a trial-and-error approach, since prior knowledge about outliers in datasets is usually unavailable.

This study introduces an adaptive and robust variant of the traditional L2 loss function, the Fractional L2 Loss (FL2L) function, by leveraging the fractional derivative operator. We demonstrate that by changing the fractional derivative order α in the proposed FL2L, we can modify the shape of the L2 loss to create a spectrum of losses spanning from L2 to L1. In this manner, α is introduced as a new hyperparameter, providing a clear method to adjust the robustness and convergence properties of the FL2L. Although the robustness hyperparameter α adds a layer of flexibility, determining an optimal value for α prior to training poses a challenge, especially for big data where outlier information is usually unknown. To solve this issue, we have transformed α from a fixed hyperparameter to a dynamic parameter, allowing gradient-based optimization frameworks to dynamically adjust α . This modification allows the FL2L to automatically adjust its level of robustness to outliers and noise during the learning process. In our experiments, we first demonstrate the effectiveness of FL2L in managing outliers through a simple regression analysis using an illustrative toy dataset. Next, we apply FL2L to a system identification task involving a dataset from an actual drone, observing significant improvements in learning performance for x and y positional dynamics. Additionally, FL2L significantly enhances the accuracy of battery cycle life predictions made by a DNN architecture, particularly in scenarios where the cycle lives are long and uncertainties are large.

11.2 Adaptive Robust L2 Loss Function Using Fractional Calculus

Here, we detail the derivation and fundamental characteristics of FL2L and demonstrate our method for dynamically changing its level of robustness during the training.

11.2.1 Fractional L2 Loss Function

Fractional calculus generalizes the traditional derivative operator from integer orders to non-integer orders, with the expression $D^\alpha f(x) = \frac{d^\alpha f(x)}{dx^\alpha}$, where D^α is the fractional derivative operator and $\alpha \in \mathbb{R}$. For a power function $f : \mathbb{R} \mapsto \mathbb{R}$ the Riemann–Liouville fractional derivative operator D^α is defined as [3]

$$D^\alpha f(x) = \frac{\Gamma(k+1)}{\Gamma(k+1-\alpha)} x^{k-\alpha}. \quad (11.1)$$

Here, the gamma function extends the factorial function, originally defined for integers, to real and complex numbers. It has the following form:

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt. \quad (11.2)$$

Our approach involves leveraging D^α to change the robustness characteristics of the L2 loss function. We define the L2 loss function $f : \mathbb{R} \mapsto \mathbb{R}$ as follows:

$$f(x) = \frac{1}{2} (x/c)^2, \quad (11.3)$$

with $x \in \mathbb{R}$ representing the residuals and $c > 0$ acting as a scaling factor to adjust the quadratic bowl's size around $x = 0$. We apply the fractional derivative operator as outlined in (11.1) to $f(|x|)$ rather than $f(x)$, to maintain symmetry in the FL2L. The FL2L function $g_\alpha : \mathbb{R} \mapsto \mathbb{R}$ is formulated as

$$g_\alpha(x) := D^\alpha f(|x|) = \frac{|x/c|^{2-\alpha}}{c^2 \Gamma(3-\alpha)}. \quad (11.4)$$

For the specific case where $\alpha = 0$ and $c = 1$, $g_\alpha(x)$ simplifies to the L2 loss, represented by

$$g_0(x) = D^0 f(|x|) = \frac{1}{2} x^2, \quad (11.5)$$

and for $\alpha = 1$ and $c = 1$, it reduces to the L1 loss:

$$g_1(x) = D^1 f(|x|) = |x|. \quad (11.6)$$

From (11.4), we can see that the parameter $\alpha \in [0, 1]$ dictates the characteristic of $g_\alpha(x)$ (between the L2 and L1 losses) as shown in Fig. 11.1a.

To explore the impact of α on $g_\alpha(x)$ through the lens of gradient-based loss minimization, consider how a reduction in α from 1 to 0 transitions the shape of the FL2L. This change makes FL2L gradually move from the shape of L1 loss, which is more forgiving of large residuals (thereby offering better robustness against outliers), to the more strict penalty of L2 loss, known for its quicker convergence but sensitivity to outliers [15]. For a more in-depth examination, we will calculate the gradient of $g_\alpha(x)$ with respect to x ($\nabla g_\alpha(x)$):

$$\nabla g_\alpha(x) = \frac{(2 - \alpha) \text{sgn}(x)}{c^2 \Gamma(3 - \alpha)} |x/c|^{1-\alpha}. \quad (11.7)$$

In Fig. 11.1b, we can observe that for $|x| > c$, as α increases from 0 to 1, there's a noticeable decrease in the magnitude of $\nabla g_\alpha(x)$, thereby diminishing the impact of larger residuals on the process of gradient-based loss minimization. Conversely, a decrease in α from 1 to 0 leads to an increase in $\nabla g_\alpha(x)$, which accelerates the convergence speed at the expense of increased sensitivity to outliers. Therefore, $\alpha \in [0, 1]$ is a hyperparameter that balances robustness and the rate of convergence.

The FL2L has several key properties as follows:

- (i) It is symmetric because we applied D^α to $f(|x|)$ rather than to $f(x)$.
- (ii) For $x \neq 0$ and $\forall \alpha \in [0, 1]$, the FL2L is smooth, belonging to the class C^∞ .
- (iii) At the origin, the value of FL2L equals zero.

$$g_\alpha(0) = 0 \quad (11.8)$$

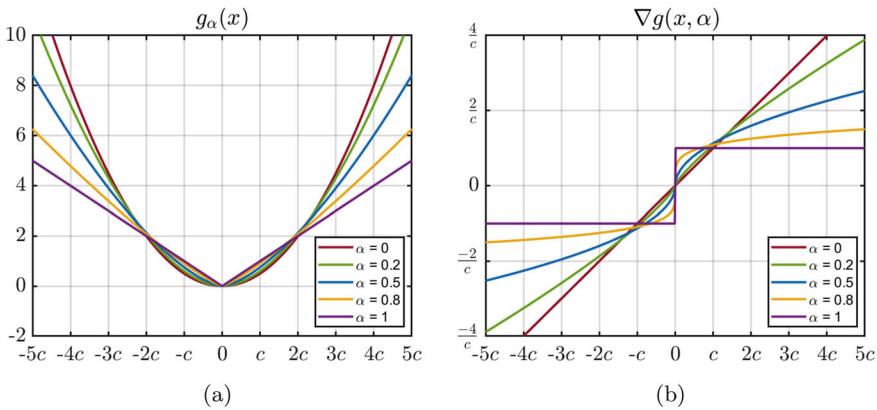


Fig. 11.1 **a** Fractional L2 loss function and **b** its gradient

- (iv) The FL2L demonstrates a monotonic increase with respect to $|x|$ for $\forall \alpha \in [0, 1]$.

$$\frac{\partial}{\partial |x|} g_\alpha(x) \geq 0. \quad (11.9)$$

- (v) The parameter α serves a dual purpose, shaping the FL2L's form and enabling a balance between robustness and speed of convergence.

These properties make FL2L compatible with gradient-based optimization techniques. However, the adaptability offered by the FL2L required the introduction of an extra hyperparameter, α . Finding the optimal value for α before training presents a challenge since information about outliers is often unknown.

11.2.2 Adaptive Robustness with Fractional L2 Loss

In this section, we convert the fractional derivative order, α , from a fixed hyperparameter to a dynamic parameter, $g_\alpha(x) \rightarrow g(x, \alpha)$. This change allows for the optimization of the loss function's robustness by dynamically adjusting α throughout training, within the empirical risk minimization framework.

In empirical risk minimization, when provided with m independently and identically distributed data $(z_1, y_1), (z_2, y_2), \dots, (z_m, y_m)$, where z belongs to set Z and y belongs to set Y , the objective is to derive a predictor $\mathcal{F} : Z \rightarrow Y$ characterized by learnable parameters θ . By considering α as an additional parameter to be learned, we aim to fine-tune the shape of the L2 loss function during the minimization of FL2L $g(\mathcal{F}(z_i; \theta) - y_i, \alpha)$. The problem definition of finding optimal parameters θ^* and α^* is formulated as follows:

$$(\theta^*, \alpha^*) = \underset{(\theta, \alpha)}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m g(\mathcal{F}(z_i; \theta) - y_i, \alpha). \quad (11.10)$$

Graphical representations of the FL2L (Fig. 11.1a) indicate that for residuals roughly $|x| < 2c$, the $g(x, \alpha)$ value monotonically increase with α ($\frac{\partial}{\partial \alpha} g(x, \alpha) > 0$), while for residuals $|x| > 2c$, $g(x, \alpha)$ exhibits a monotonic decrease ($\frac{\partial}{\partial \alpha} g(x, \alpha) < 0$). This phenomenon introduces an important trade-off in learning α during training: enhancing robustness against larger residuals ($|x| > 2c$) results in penalization of small residuals ($|x| < 2c$).

This equilibrium in the adjustment process of α maintains a strategic balance, preventing α from increasing to infinity. This would cause the loss function to inaccurately label each data point as an outlier, minimizing the loss but learning nothing [1]. In [1], a similar balance is achieved by using the negative log likelihood of a probability distribution corresponding to the shifted loss function. The FL2L can achieve a similar trade-off naturally, without needing any extra steps.

To establish update mechanisms for the fractional derivative order α for gradient-based optimization as outlined in (11.10), it's necessary to calculate the partial derivative of FL2L with respect to α :

$$\begin{aligned} \frac{\partial}{\partial \alpha} g(x, \alpha) = & - \frac{|x/c|^{2-\alpha} \log(|x/c|)}{\Gamma(3-\alpha)} \\ & + \frac{|x/c|^{2-\alpha} \Gamma'(3-\alpha)}{\Gamma(3-\alpha)^2}. \end{aligned} \quad (11.11)$$

Rearranging the terms gives

$$\frac{\partial}{\partial \alpha} g(x, \alpha) = |x/c|^{2-\alpha} \left(-\frac{\log(|x/c|)}{\Gamma(3-\alpha)} + \frac{\Gamma'(3-\alpha)}{\Gamma(3-\alpha)^2} \right) \quad (11.12)$$

which simplifies to

$$\frac{\partial}{\partial \alpha} g(x, \alpha) = \left(\frac{\Gamma'(3-\alpha)}{\Gamma(3-\alpha)} - \log(|x/c|) \right) g(x, \alpha). \quad (11.13)$$

Here, Γ' denotes the derivative of Γ . The digamma function ψ , defined as

$$\psi(z+1) = \frac{\Gamma'(z+1)}{\Gamma(z+1)} = -\gamma + \sum_{n=1}^{\infty} \left(\frac{z}{n(n+z)} \right), \quad (11.14)$$

and thus the derivative of $g(x, \alpha)$ with respect to α can be expressed as

$$\frac{\partial}{\partial \alpha} g(x, \alpha) = (\psi(3-\alpha) - \log(|x/c|)) g(x, \alpha). \quad (11.15)$$

Equation (11.15) can be implemented using backpropagation within automatic differentiation processes.

11.3 Experiments

In this section, a series of comparative experiments is conducted to demonstrate the effectiveness of the FL2L. For every experiment, we repeat the process five times with different seeds to maintain reliability between comparisons. All models are trained using the Adam optimizer which is gradient based.

11.3.1 A Linear Regression Example

A simple regression experiment is carried out to showcase the effectiveness of FL2L in handling outliers on a toy dataset designed for illustrative purposes. To generate the dataset, we define a linear relationship, $y = 0.7x + 0.15$, where x is a vector in $\mathbb{R}^d$ with $x_i \sim \text{Uniform}(0, 1)$ for $i = 1, \dots, d$ and $d = 50$. Also, we add synthetic noise to the dataset following a normal distribution with a standard deviation of 0.025. Furthermore, we intentionally alter about 10% of the data into extreme outliers through the transformation $0.05 + 0.4(1 - \text{sign}(y_r - 0.5))$ where y_r represents randomly chosen outputs. We then solve a basic linear regression problem on this dataset using the Adam optimization algorithm with a learning rate of 0.01.

Figure 11.2 illustrates how the fitting line derived from using a standard L2 loss function is notably affected by outliers (highlighted with blue circles), causing an obvious shift in the fitting line in the direction of these outliers. On the other hand, the regression model using FL2L demonstrates increased robustness against the outliers. This robustness is attributed to FL2L's ability to modify its loss landscape by adjusting the α values throughout the training phase.

Figure 11.3 demonstrates that the FL2L gradually shifts to L1 loss as the α parameter increases from 0 to 1 throughout the training process. Initially, when α is near 0, the optimization process emphasizes convergence speed, resulting in a loss shape similar to L2 loss. As the training progresses, α swiftly approaches 1, changing the focus from the speed of convergence to the robustness against outliers, thereby transitioning to L1 loss. The dynamic adjustment of α values during training allows the FL2L to effectively reduce the impact of outliers on the model's fitted lines.

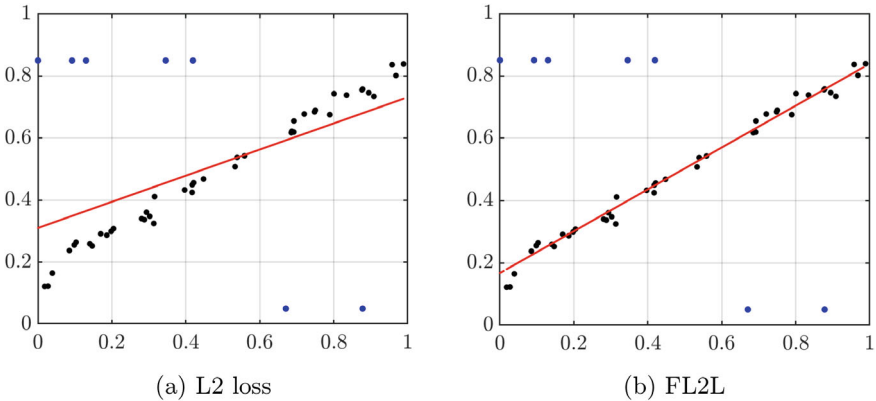
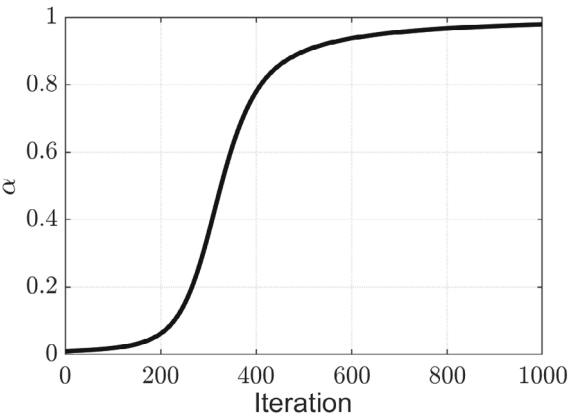


Fig. 11.2 Comparison of fitting performance between FL2L and standard L2 loss functions. Blue circles: Synthetic outliers added for robustness testing

Fig. 11.3 Variation of α parameter during training for linear regression



11.3.2 System Identification

We use a dataset containing 17,751 samples collected from a real Parrot Mambo drone [14], utilizing the roll angle (ϕ) and pitch angle (θ) as inputs, with the drone’s x-y-plane coordinates as the outputs. In this data, the yaw angle reference is at 0° , and the altitude reference is 1 m above ground level, thus the drone model can be treated as a system with two inputs and two outputs ($q = p = 2$).

70% of the dataset is chosen for training and the remaining 30% for testing. Additionally, synthetic white noise, amounting to 5% of the output’s maximum value, is introduced into the dataset to assess the enhancements in robustness provided by the FL2L over traditional base L2 loss. A long short-term memory network, consisting of two layers with 300 cells each, a dropout layer, and a fully connected layer, is utilized for analysis. As the information vector (regressor), input and output signals along with their lagged versions up to two samples are used [14].

Table 11.1 illustrates that employing FL2L for training substantially enhances test trajectory prediction accuracy compared to the L2 loss function. Specifically, for x-axis dynamics, using FL2L boosted the performance over the standard L2 loss by

Table 11.1 Comparison of FL2L and standard L2 loss functions on battery cycle life prediction and drone system identification tasks

Experiments		Metrics	Loss functions	
			L2	FL2L
Battery cycle Life prediction		RMSE	137.67 ± 29.34	129.09 ± 31.85
		MAE	92.88 ± 22.34	90.90 ± 18.38
Drone system identification*	x	RMSE	20.79 ± 30.08	6.58 ± 1.20
		MAE	32.78 ± 21.85	22.87 ± 2.14
	y	RMSE	19.48 ± 31.55	5.91 ± 2.88
		MAE	31.32 ± 25.75	21.06 ± 5.42

*The metrics are scaled by 100

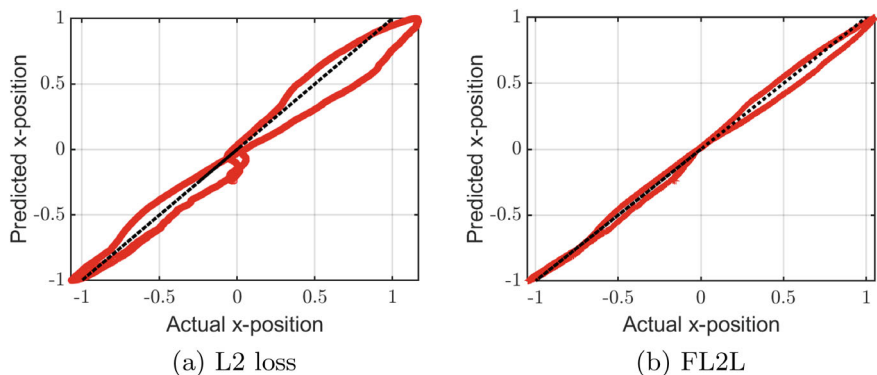


Fig. 11.4 Predicted versus actual x-position values illustrated in scatter plots

68.35% for RMSE and 30.23% for MAE. In the case of y-axis dynamics, the FL2L enhanced the base log-cosh loss results by 69.66% for RMSE and 32.76% for MAE.

Figures 11.4 and 11.5 display scatter plots that compare the actual versus predicted positions along the x- and y-axes, respectively. These figures reveal that LSTM models trained with FL2L rather than L2 loss functions align more closely with the diagonal line, indicating a more accurate representation of the drone dynamics.

Figure 11.6a, b shows the learning behavior of the α parameter. In the early training phase, up to 2000 iterations, α rises from 0 to 0.1, with little increase in robustness. After 3000 iterations, α starts to climb more steeply, marking an increase in the model's robustness. This upward trend in α , which directly boosts robustness, continues through the second half of training, as seen in both figures.

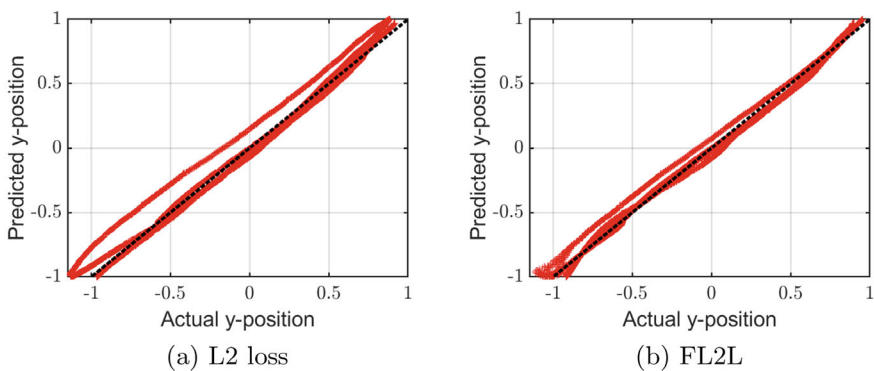


Fig. 11.5 Predicted versus actual y-position values illustrated in scatter plots

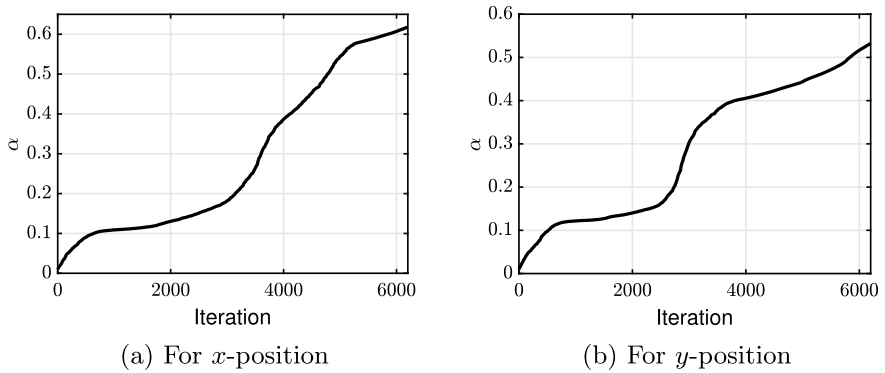


Fig. 11.6 Variation of the α parameter during training in drone system identification experiment

11.3.3 Battery Cycle Life Prediction

Data for predicting battery cycle life comes from 40 lithium-ion cells, each undergoing multiple charge and discharge cycles. We use data from 30 cells for training, 5 cells for validation, and another 5 for testing. The objective is to precisely predict the remaining cycle life before the battery’s capacity falls to 80% of its initial value, with cycle life spans ranging from 150 to 2300 cycles. The analysis employs the model and methodology described by [12].

Table 11.1 shows that employing FL2L improves the accuracy of battery cycle life predictions by reducing the RMSE and MAE metrics. Figure 11.7 showcases scatter plots comparing the actual versus predicted battery cycle lives. Ideally, these plots should reveal all points closely aligned along the diagonal, indicating minimal prediction error, with a tight confidence interval. Predicting the remaining life is easier as a battery nears its end of life. Nonetheless, in the early stages of a battery’s cycle life, when the actual cycle life is longer, predictions tend to be more uncertain.

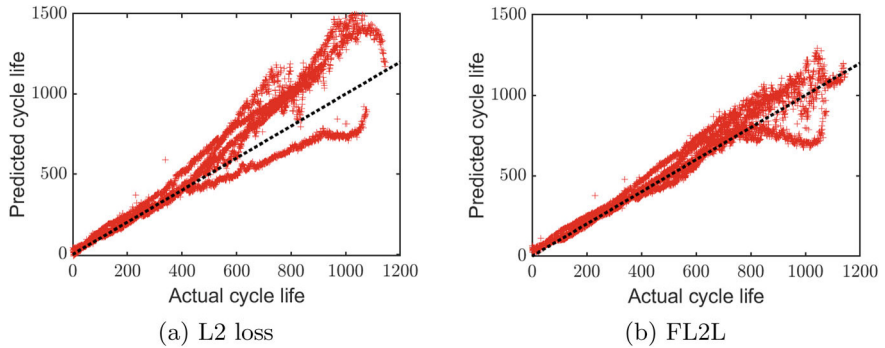
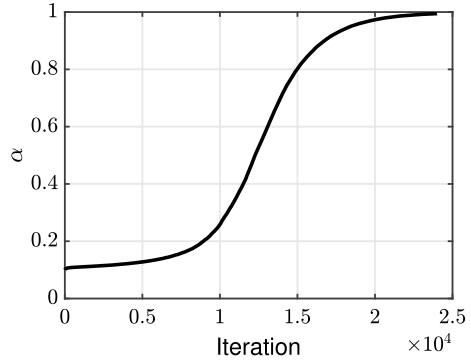


Fig. 11.7 Predicted versus actual battery cycle life values illustrated in scatter plots

Fig. 11.8 Variation of the α parameter during training in battery cycle life prediction experiment



As illustrated in Fig. 11.7, the model trained with FL2L outperforms that trained with base L2 loss function, more accurately mirroring the diagonal trend. This is particularly evident in predictions of cycle lives exceeding 400 cycles, where the L2 loss-trained model exhibits more scatter, while the FL2L-trained model outputs predictions that are more aligned with the diagonal.

Figure 11.8 shows the FL2L's shift from L2 loss to L1 loss during training. Initially, α is near 0, which means the model starts with L2 loss around the first 1000 iterations. As training progresses, α gradually increases and moves toward 1, indicating a smooth transition to L1 loss.

11.4 Conclusion and Future Work

In this paper, we present an adaptive and robust variant of the L2 loss function called FL2L, utilizing the fractional derivative operator. This FL2L spans a family of loss functions between the traditional L2 loss and the L1 loss, by varying the fractional derivative order α . The adjustability of α serves as a tunable hyperparameter, enabling precise control over the robustness of the FL2L to outliers. Additionally, we transform α to a dynamic parameter within the gradient-based optimization process, allowing for the automatic tuning of the loss landscape to enhance both robustness and performance in error minimization. Our experiments, drone system identification, and predicting the cycle life of batteries demonstrate the significant enhancements in performance achieved by DNNs using the FL2L instead of traditional L2 loss.

Moving forward, our goal is to establish a versatile framework that leverages the fractional derivative operator, enabling its application to any given loss function to enhance its adaptability and robustness. Then, we plan to evaluate this family of loss functions within well-known DNN tasks, including computer vision, to demonstrate their effectiveness and potential benefits.

References

1. Barron, J.T.: A general and adaptive robust loss function. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4331–4339 (2019)
2. Beke, A., Kumbasar, T.: Learning with type-2 fuzzy activation functions to improve the performance of deep neural networks. *Eng. Appl. Artif. Intell.* **85**, 372–384 (2019)
3. Das, S.: *Functional Fractional Calculus*, vol. 1. Springer (2011)
4. Fu, S., Wang, X., Tang, J., Lan, S., Tian, Y.: Generalized robust loss functions for machine learning. *Neural Netw.* **171**, 200–214 (2024)
5. Ghosh, A., Kumar, H., Sastry, P.S.: Robust loss functions under label noise for deep neural networks. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31 (2017)
6. Hewing, L., Wabersich, K.P., Menner, M., Zeilinger, M.N.: Learning-based model predictive control: toward safe learning in control. *Annu. Rev. Control Robot. Auton. Syst.* **3**, 269–296 (2020)
7. Huber, P.J.: Robust estimation of a location parameter. In: *Breakthroughs in Statistics: Methodology and Distribution*, pp. 492–518. Springer (1992)
8. Joshi, M., Bhosale, S., Vyawahare, V.A.: A survey of fractional calculus applications in artificial neural networks. *Artif. Intell. Rev.* 1–54 (2023)
9. Kuutti, S., Bowden, R., Jin, Y., Barber, P., Fallah, S.: A survey of deep learning applications to autonomous vehicle control. *IEEE Trans. Intell. Transp. Syst.* **22**(2), 712–733 (2020)
10. Ljung, L., Andersson, C., Tiels, K., Schön, T.B.: Deep learning and system identification. *IFAC-PapersOnLine* **53**(2), 1175–1181 (2020)
11. Manwani, N., Sastry, P.: Noise tolerance under risk minimization. *IEEE Trans. Cybern.* **43**(3), 1146–1151 (2013)
12. Severson, K.A., Attia, P.M., Jin, N., Perkins, N., Jiang, B., Yang, Z., Chen, M.H., Aykol, M., Herring, P.K., Fraggedakis, D., et al.: Data-driven prediction of battery cycle life before capacity degradation. *Nat. Energy* **4**(5), 383–391 (2019)
13. Sünderhauf, N., Brock, O., Scheirer, W., Hadsell, R., Fox, D., Leitner, J., Upcroft, B., Abbeel, P., Burgard, W., Milford, M., et al.: The limits and potentials of deep learning for robotics. *Int. J. Robot. Res.* **37**(4–5), 405–420 (2018)
14. Tuna, T., Beke, A., Kumbasar, T.: Deep learning frameworks to learn prediction and simulation focused control system models. *Appl. Intell.* **52**(1), 662–679 (2022)
15. Wang, Q., Ma, Y., Zhao, K., Tian, Y.: A comprehensive survey of loss functions in machine learning. *Ann. Data Sci.* 1–26 (2020)
16. Wei, Y., Kang, Y., Yin, W., Wang, Y.: Generalization of the gradient method with fractional order gradient direction. *J. Frank. Inst.* **357**(4), 2514–2532 (2020)
17. Yao, X., Wang, Z.: Fractional order echo state network for time series prediction. *Neural Process. Lett.* **52**, 603–614 (2020)
18. Zamora, J., Rhodes, A.D., Nachman, L.: Fractional adaptive linear units. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36 (2022)
19. Zamora Esquivel, J., Cruz Vargas, A., Camacho Perez, R., Lopez Meyer, P., Cordourier, H., Tickoo, O.: Adaptive activation functions using fractional calculus. In: Proceedings of the IEEE International Conference on Computer Vision Workshops (2019)
20. Zhou, X., Liu, X., Zhai, D., Jiang, J., Ji, X.: Asymmetric loss functions for noise-tolerant learning: theory and applications (2023)

Chapter 12

Sparse Reconstruction of Forces, Torques, and Velocity Signals for a Swimmer in a Wake



Ivan Gushkov, Simon Hoff, Amer Orucevic, Kristin Y. Pettersen,
and Jan Tommy Gravdahl

Abstract In this paper, we investigate the reconstruction of hydrodynamic forces and torques, as well as the fluid velocity profile along the body of a swimmer in a Kármán Vortex Street (KVS) through data-driven methods. We collect hydrodynamic data from a high-fidelity Computational Fluid Dynamic (CFD) simulation of the swimmer, and construct a latent data-driven model through Principal Component Analysis (PCA). We generate optimal sensor positions through a QR decomposition of the latent space, allowing us to identify non-trivial sensor positions that hold the most information about the PCA latent variables. We use the measurements from the sparse set of optimal sensors, in conjunction with the PCA model to reconstruct the latent variables, and subsequently the features in the data which are not measured, resulting in a computationally efficient method. We demonstrate the method in two applications: recovering forces and torques, and estimating the speed profile of the fluid along the boundary of the swimmer.

Keywords Compressed sensing · Snake robots · Data-driven hydrodynamic modeling

12.1 Introduction

Articulated swimmers, such as Underwater Snake Robots (USRs) and fish robots, represent an exciting new development in biomimetics. These fascinating mechanisms draw inspiration from nature to present promising solutions to current issues we face, such as ecological surveying, inspection and maintenance of underwater industrial installations, and energy harvesting. Swimmers operate in complex hydrodynamic environments and are often tasked with solving challenging control problems with access to limited data. Hydrodynamic modeling for vehicle control is especially problematic. On the one hand, there exists a wealth of analytical theory,

I. Gushkov (✉) · S. Hoff · A. Orucevic · K. Y. Pettersen · J. Tommy Gravdahl
Norwegian University of Science and Technology (NTNU), Trondheim, Norway
e-mail: ivanig@stud.ntnu.no

which is prohibitively complex to use in analytical controller design, or optimal control schemes like MPC. On the other hand, we live in a time of data abundance and powerful computers, which can offer numerical solutions to otherwise intractable fluid dynamics problems. These views need not be mutually exclusive. Indeed, simulations based on first-principles equations can aid in data generation and save time and costs when compared to physical experiments. Data generated through simulations can be used to make efficient models, which are computationally tractable for online control. Conversely, data-driven methods can be used to efficiently enhance solutions based on first principles [1, 2]. For the field of biomimetics, this presents an enticing opportunity, as we can simulate the sensing capabilities of fish and other marine fauna in order to analyze, understand, and eventually harness the potential of the vast oceans.

Marine creatures display an amazing capability of sensing and interacting with their hydrodynamic environment. In [3], it is discussed how blind cave-dwelling fish use flow sensing organs for achieving advanced behaviors such as station keeping and maintaining their position within the cave, detection of vibrating sources in the flow, and even 3D reconstruction of their environment. In [4], it is studied how rainbow trout swim behind a cylinder in uniform flow, which is a canonical flow scenario known as a Kármán Vortex Street (KVS), see Fig. 12.1. They showed that the trout alters its body kinematics in a gait they called the Kármán gait, where the fish synchronize their tail beat frequency with the shed frequency of the KVS. In a follow-up paper, [5], they showed that the rainbow trout slaloms between the vortices, exploiting low-pressure regions and reducing muscle activity. Replicating this sort of performance in our engineering systems would be of great benefit.

The way fish and other marine life exploit their hydrodynamic environment is enabled by sophisticated mechanosensory organs that allow the fish to extract information from hydrodynamic measurements. Fish use their so-called lateral line systems to obtain sparse instantaneous measurements of the local flow [6]. The lateral lines are organs consisting of arrays of velocity or pressure sensors, optimally placed by evolution over time. Human-made velocity and pressure sensors have shown to be as sensitive and robust as their nature counterparts [7, 8]. Studies such as [9, 10] investigate the use of artificial lateral lines (ALLs) in a KVS flow regime.



Fig. 12.1 An articulated swimmer in a KVS. The KVS is a canonical flow regime characterized by vortex shedding behind a bluff body, in our case, a cylinder. The vortices are symmetrically offset from a centerline and are shed alternately above and below the centerline

They show that the parameters of the flow, such as the shed frequency, vortex strength, and vortex spacing, can be estimated from snapshots of the velocity distribution along the body. In [11, 12], artificial lateral line (ALL)s with pressure sensors are used to extract features of the flow regime, and in [13] the pressure lateral line is used to enhance the swimming efficiency of a fish robot by synchronizing its tail beats with the frequency of the wake. In [14], an articulated swimmer in a Computational Fluid Dynamics (CFD) simulation study is stabilized to track the centerline of a KVS by way of two antennae-line sensors on either side of the swimmer's head, resulting in reduced energy usage in comparison to swimming against the free-stream current.

Another branch of research is data-driven analysis of fluid flows. While the ALL community relies on prior knowledge about the flow regime, and uses local measurements to recover features and parameters of the flow, the big data approach to fluid analysis takes a more global approach. Using massive quantities of data from the fluid domain, one can analyze and discover low-dimensional and generalizable latent structures in the flow. The Principal Component Analysis (PCA), often called Proper Orthogonal Decomposition (POD) in this field, has been a workhorse algorithm for finding latent empirical bases for fluid flows, with both theoretical and practical success [15, 16]. In [17], a sparse set of optimal sensors are placed on a cylinder in uniform flow, and subsequently used to reconstruct the solution of the fluid around the cylinder. In [18], a deep learning model is used to predict the trajectory of the fluid flow from sparse measurements.

In this paper, we seek to make progress in unifying the approaches of the ALL and the big data fluid communities. In [17], the case of sensors mounted on a stationary cylinder is considered, which is decidedly less complex than a dynamic articulated swimmer. We leverage theory from compressed sensing and latent variable modeling to propose a framework for reconstruction of signals which are hard or impossible to measure in a real setting. The proposed methodology is simple to implement and test, and places focus on achieving signal reconstruction with few sensors. The advantage of using a sparse sensor setup is twofold, from a practical perspective; the cost of sensors can be considerable, and having to process fewer measurements can result in lighter computational loads. We consider two cases for signals to reconstruct, namely, the hydrodynamic forces and torques, as well as the velocity profile along the body of the swimmer. In the first case, we leverage vorticity measurements, while, in the second case, we reconstruct the entire velocity profile along the body of the swimmer from only a handful of measurements.

This paper is organized as follows: in Sect. 12.2, we present the problem mathematically, and develop a solution based on the PCA. In Sect. 12.3, we describe the simulation setup, and practical aspects of generating the dataset. In Sect. 12.4, we build upon the theory presented in Sect. 12.2 to create models for two applications of sparse signal reconstruction for the swimmer, which are subsequently tested in Sect. 12.5. Finally, we summarize our findings and discuss potential avenues for future work in Sect. 12.6.

12.2 Model Structure and Mathematical Background

In this section, we will give an overview of the articulated swimmer, and the problem of sparse reconstruction of signals in the context of articulated swimmers. We present a solution based on a latent variable PCA model, and propose a framework for optimally placing a sparse set of sensors based on a QR decomposition of the latent space.

12.2.1 The Swimmer and Its Variables

In this work, we consider a planar swimmer, completely submerged and unactuated. The swimmer consists of N_l elliptical links connected with joints. The angle that link i forms with the global x -axis is denoted θ_i . The joint angles, or the relative link angles, are defined here as

$$\phi_i = \theta_{i+1} - \theta_i, \quad i = 1, \dots, N_l - 1, \quad (12.1)$$

which we can collect in the joint angle vector $\boldsymbol{\phi} \in \mathbb{R}^{N_l-1}$. We define the swimmer's configuration vector as

$$\mathbf{q} = [x_1, y_1, \theta_1, \boldsymbol{\phi}]^T \in \mathbb{R}^{N_l+2}, \quad (12.2)$$

where (x_1, y_1) is the position of the first link in the world frame. For each link, the fluid exerts hydrodynamic forces and torques (F_x, F_y, T) . We collect these in a generalized force vector for all the links

$$\boldsymbol{\tau}_{\text{hyd}} = [\mathbf{F}_x, \mathbf{F}_y, \mathbf{T}]^T \in \mathbb{R}^{3N_l}. \quad (12.3)$$

The hydrodynamic forces and torques in (12.3) are, in general, difficult to measure and recover. We aim to reconstruct these signals by placing a set of sensors along the body of the robot, which can measure quantities in the fluid, to be specified in Sect. 12.4. To that end, we first assemble a dataset of n temporal snapshots of the data from N_s sensors on the boundary of the swimmer, and potentially other easily recoverable quantities, and the generalized hydrodynamic forces. For the purpose of training the model, the difficult to measure hydrodynamic forces and torques can be recovered from a CFD simulator with a fluid–structure interaction module.

$$\mathbf{X}_{\partial S} = \begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} = \begin{bmatrix} \mathbf{x}_{s,1} & \boldsymbol{\tau}_{\text{hyd},1} \\ \vdots & \vdots \\ \mathbf{x}_{s,n} & \boldsymbol{\tau}_{\text{hyd},n} \end{bmatrix} \in \mathbb{R}^{n \times (N_s + 3N_l)}. \quad (12.4)$$

12.2.2 Model Structure

We will use the PCA to discover latent relations between our original data variables. In PCA, the data matrix is decomposed as follows:

$$\mathbf{X}_{n \times m} = \mathbf{U}_{n \times n} \mathbf{\Sigma}_{n \times n} \mathbf{V}_{n \times m}^T. \quad (12.5)$$

An r -truncation of these matrices gives an l_2 approximation of the original data:

$$\mathbf{X}_{\partial C} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T, \quad \mathbf{V}_r \in \mathbb{R}^{m \times r}. \quad (12.6)$$

The columns of the $\mathbf{V}_r^T$ matrix serve as basis vectors in our r -dimensional latent variable space. Consequently, we can express a vector of the original features as

$$\mathbf{x} = \mathbf{\Psi} \mathbf{a}, \quad \mathbf{\Psi} = \mathbf{V}_r, \quad (12.7)$$

for some $\mathbf{a} \in \mathbb{R}^r$, where the choice of r is a matter of model order selection.

Suppose now that we measure some subset of our original features vector $\mathbf{x}$:

$$\mathbf{y} = \mathbf{C} \mathbf{x}. \quad (12.8)$$

Relating this to the dataset structure in (12.4), we consider the measurement vector to be a subset of the original sensor measurements along the body of the swimmer:

$$\mathbf{C} = [\mathbf{C}_s \quad \mathbf{0}] \in \mathbb{R}^{p \times m}. \quad (12.9)$$

We seek to recover an estimate of $\mathbf{x}$ from $\mathbf{y}$. The theory of compressed sensing tells us that the vector $\mathbf{x}$ is recoverable from a sparse measurement $\mathbf{y}$ if $\mathbf{x}$ can be expressed as a sparse signal in a set of transform coordinates [16]. Moreover, recent results in the field indicate that latent embeddings, such as the one recovered from PCA, can be used in sparse frameworks [19, 20]. We can thus cast the problem as recovering an r -dimensional vector $\mathbf{a}$ of coefficients in the latent space:

$$\mathbf{y} = \mathbf{C} \mathbf{x} = \mathbf{C} \mathbf{\Psi} \mathbf{a}, \quad (12.10)$$

and then using the relationship in (12.7) to reconstruct an estimate of the feature vector $\mathbf{x}$. The particular way of solving (12.10) for $\mathbf{a}$ depends on the problem setup. In this paper, we assume that the number of sparse sensors p is at least as large as the number of latent coefficients, that is, $N_s \gg p \geq r$. As we will see later on, this is hardly a restrictive assumption, as r can end up being small, allowing for $\mathbf{y}$ to be a quite sparse subset of sensors. Under the assumption of $p \geq r$ we get an over-determined system of equations, and we can readily solve it using the pseudo-inverse

$$\hat{\mathbf{x}} = \mathbf{\Psi} \hat{\mathbf{a}} = \mathbf{\Psi} (\mathbf{C} \mathbf{\Psi})^\dagger \mathbf{y}. \quad (12.11)$$

12.2.3 Optimal Choice of Sensors

Computation of an optimal set of sensors is a key topic in compressed sensing. We wish to design a C such that the measurement vector is as informative in the latent space as possible. A greedy approximation of the optimal C matrix is often found through QR decomposition with pivoting on the V_r matrix [16]. The method, however, fails for oversampling, i.e., when $p > r$, and will only generate r meaningful sensors. An extension of QR pivoting for generating an optimal C matrix can be found by performing pivoting on $V_r V_r^T \in \mathbb{R}^{m \times m}$, which will produce the optimal C for $p > r$, albeit in a more computationally expensive procedure [21]:

$$(V_r V_r^T)C^T = QR. \quad (12.12)$$

12.3 Simulation Setup and Dataset

In this section, we describe the simulation setup that was used to generate the training data. A high-level overview of the CFD model and its parameters is presented. We also give a detailed overview of how the sensors are placed on the link and used to sample quantities in the fluid.

12.3.1 Coupled CFD-Swimmer Simulator

The system was simulated with a high-fidelity CFD simulator based on methods presented in [14]. The algorithm relies on vortex-particle mesh (VPM) techniques coupled with a multi-body system solver (MBS). It consists of a CFD module that handles the fluid dynamics, fluid–structure interaction, and placement of rigid bodies in the fluid environment. The CFD module interacts with a MBS that handles the dynamics of the swimmer.

Studying the swimmer’s behavior under complex conditions requires us to analyze how the complex fluid–structure interactions relate to the swimmer’s kinematics. To simplify that task, a prescribed motion model was employed. The prescribed motion model allows the user to constrain the motion along certain degrees of freedom, while allowing the rest of the swimmer’s configuration to evolve freely. The prescribed motion model is used in [22] in order to precisely control the x -position of a swimmer behind a cylinder to discover an optimal position for harvesting energy. In this work, we use the prescribed motion model in order to hold the swimmer stable in a constant x -position while we gather a dataset. The prescribed motion model is presented in full detail in Chap. 2 of [23].

12.3.2 Simulation Setup and Sensors

For gathering the training dataset, the simulation domain's physical size was set to $3 \text{ m} \times 2.5 \text{ m}$, with the CFD's discretization grid set to 460×384 . The simulation was performed at Reynolds number $\text{Re} = U D_{\text{cyl}}/\nu = 200$, with the free-stream velocity U set to 0.1 m s^{-1} , cylinder diameter $D_{\text{cyl}} = 0.1 \text{ m}$, and the kinematic viscosity given by ν .

The cylinder was centered along the y -domain and positioned one-fifth of the way into the x -domain. The swimmer was initialized 2.5 cylinder diameters downstream from the center of the cylinder. The cylinder was spun in the first 4 s of the simulation, which accelerates the formation of the vortex wake. This setup resulted in the swimmer locking in with the wake, achieving a periodic oscillatory motion, as seen in Fig. 12.2. A dataset was generated for around five and a half periods after the system had reached the periodic steady-state part of the solution. The raw dataset included the configuration of the swimmer and its derivatives ($\mathbf{q}$, $\dot{\mathbf{q}}$, $\ddot{\mathbf{q}}$), the hydrodynamic forces and torques from the fluid on the individual links, and the velocity and vorticity fields of the fluid over time. The sampling of these fluid fields by sensors on the boundary of the swimmer was done in post-processing, by placing each sensor in the world frame via an SE(2) transformation and sampling the relevant field at that location. We used a cubic interpolation scheme to sample between the grid points of the CFD simulator.

Sampling of vector-valued quantities like the velocity is more subtle than sampling the pressure or vorticity fields. A physically meaningful velocity sensor will sample the velocity in the body frame of the rigid body it is mounted to. Simply sampling the velocity in the world frame does not correspond to how this data will be obtained in a real experimental setting. Transformation of the data to world frame is possible, but this presents issues of its own, like the uncertainty of the pose estimates corrupting the sensor data transformation. Thus, for velocity sensor measurements, we choose to decompose the velocity signals in the link frame corresponding to the sensor sampling the velocity field. Explicitly, the term *sensor measurement of the velocity field* means a sample of the velocity vector at the position of the sensor, decomposed

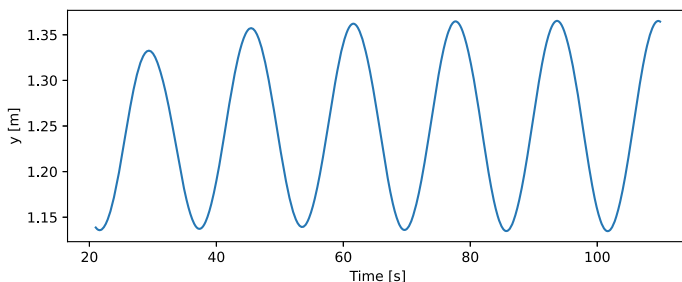


Fig. 12.2 The y -position of the first link naturally stabilizes to a periodic solution

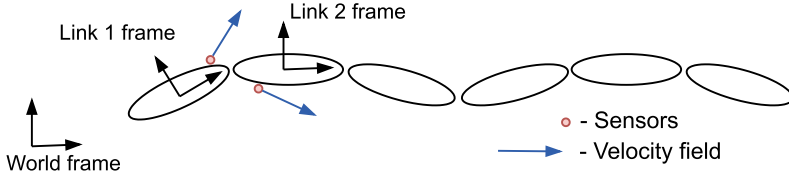


Fig. 12.3 Sensor's setup and link frames of the swimmer

in the body frame of the link the sensor is mounted to. Figure 12.3 illustrates the sensors and their frames.

12.3.3 Sensor Placement Along the Body

Computing the placement of a sensor in the body frame of a link is not entirely trivial. Ideally we want $2M$ sensors to uniformly subdivide the arc of an ellipse, such that we do not over-represent one part of the ellipse in the final dataset, i.e., $N_s = N_l \cdot 2M$. Suppose we parameterize the link's ellipse curve through a parameter $\varphi \in [0, 2\pi)$ as

$$(x(\varphi), y(\varphi)) = (l \sin(\varphi), w \cos(\varphi)), \quad (12.13)$$

where l and w are the ellipses' semi-major and semi-minor axes, respectively. The arc length of an ellipse as a function of a parameter value φ_k is given through the incomplete elliptic integral of the second kind [24]:

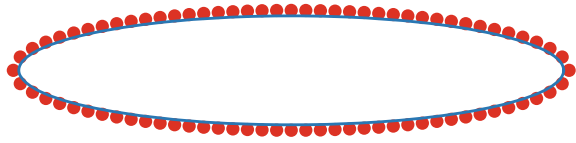
$$E(\varphi_k, m) = \int_0^{\varphi_k} \sqrt{1 - m \sin^2(t)} dt. \quad (12.14)$$

Here m is the parameter of the elliptic integral, given as $m = k^2$, where $k = \sqrt{1 - \frac{w^2}{l^2}}$ is the eccentricity of the ellipse, and (12.14) can be evaluated numerically using libraries like SciPy. In order to compute the parameter t_i for a sensor, we solve the following optimization problem:

$$\varphi_i = \underset{\varphi}{\operatorname{argmin}} |E(\varphi, m) - E(\varphi_{i-1}, m) - c_s|, \quad (12.15)$$

where c_s is the length of the arc segment between sensors, i.e., $c_s = \frac{E(2\pi, m)}{2M}$. Note that on the very boundary of the swimmer, we can get numerical issues for the fluid solution. The sensors were therefore made to stick out by using $l^* = l + 0.05w$, $w^* = w + 0.05w$ in (12.13) when generating the positions of the sensors. Figure 12.4 shows the positions of the sensors relative to the body frame of a link ellipse.

Fig. 12.4 The link and 80 sensors along its boundary



12.4 PCA and Model Training

In this section, we discuss the data analysis and training of the model. In particular, we outline a scheme for selecting the PCA model rank, and present two applications of the theory in Sect. 12.2, namely, reconstruction of hydrodynamic forces and torques, and the fluid velocity profile along the boundary of the swimmer.

12.4.1 Training and Model Order Selection

We have to decide on a truncation order for the PCA in the scheme presented in Sect. 12.2. This can be done by performing cross-validation on the training dataset. From the total dataset described in Sect. 12.3, which covers five and a half periods of the steady-state solution, we set aside four periods for training and the last period and a half as withheld data. We evaluate the validation error as the Frobenius norm of the validation fold residual matrix, when projected onto PCA models built from the training fold for all truncation ranks, see Eq. (12.16).

$$e_v(r) = \|E_v\| = \|X_v - \hat{X}_v\| = \|X_v - X_v V_r V_r^T\|. \quad (12.16)$$

12.4.2 Forces and Torques Model

The main goal here is to obtain good estimates for the hydrodynamic forces and torques acting on the snake robot. Such estimates would be greatly beneficial for active control or for planning energy optimal trajectories. The variables used in this model are vorticity measurements and the generalized hydrodynamic forces τ_{hyd} . The goal is to reconstruct an estimate of the forces and torques on each link, from limited measurements of the fluid on the force/torque level. We reiterate that in practice we would measure pressure. However, the structure of the model would be the same, and the pressure measurements would contain data with similar numerical variability. The data matrix structure for this model looks as follows:

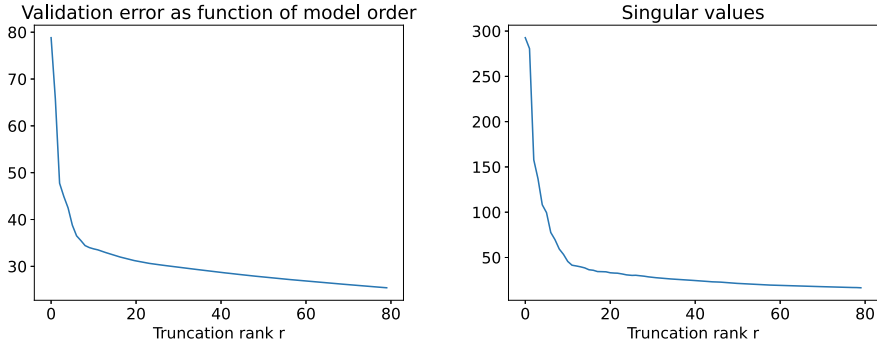


Fig. 12.5 Validation error and singular values for the force and torque model

$$\mathbf{X}_{\partial S, \text{vort}} = \begin{bmatrix} \mathbf{x}_{\text{vort},1} & \boldsymbol{\tau}_{\text{hyd},1} \\ \vdots & \vdots \\ \mathbf{x}_{\text{vort},n} & \boldsymbol{\tau}_{\text{hyd},n} \end{bmatrix} \in \mathbb{R}^{N_t \times (N_s + 3N_t)}. \quad (12.17)$$

Validation error and singular value plots for the training matrix are found in Fig. 12.5, where using the elbow method we can pick the truncation rank of $r = 11$ for this model. We generate optimal sensor positions through the procedure described in Sect. 12.2.3. The swimmer with the 30 most optimal sensors is visualized in Fig. 12.6. It is interesting to note that there is a high concentration of sensors on both sides of the first link. This is not surprising, as the first link is the one located in the most turbulent area of the wake, and the one which interacts with the vortices first. Some of the subsequent links have sensor placements on the rear of the link, but not always on both sides. Note that the second link has almost no sensors, probably owing to the fact that the first link has many sensors, and they contain enough information to recover the signals of the second link.

12.4.3 Boundary Velocity Model

The goal of this model is to recover the velocity profile of the fluid along the body of the swimmer. These signals can be used to estimate the parameters of the flow regime, or averaged to produce an estimate of the ocean currents the vehicle experiences. For the sake of simplicity, we limit the variables used in the model to only velocity measurements. The data matrix then looks as follows:



Fig. 12.6 The optimal sensors chosen for the reconstruction of hydrodynamic forces

$$\mathbf{X}_{\partial S, \text{vel}} = \begin{bmatrix} \mathbf{x}_{\text{vel}_x, 1} & \mathbf{x}_{\text{vel}_y, 1} \\ \vdots & \vdots \\ \mathbf{x}_{\text{vel}_x, n} & \mathbf{x}_{\text{vel}_y, n} \end{bmatrix} \in \mathbb{R}^{N_t \times (N_t \cdot 2N_s)}, \quad (12.18)$$

where at each sensor position we sample both the x - and y -components of the velocity vector in the local link frame. We note that the row vector $(\mathbf{x}_{\text{vel}_x}, \mathbf{x}_{\text{vel}_y})$ is a slight abuse of notation, as the data actually contains signals decomposed in different frames. Model order selection goes along the same lines as for the model in Sect. 12.4.2. The plots are omitted for brevity, as they are similar to the ones in Fig. 12.5, and we pick the truncation order $r = 13$ for this model. As with the force and torque model, we have plotted the first 30 optimal sensors generated through QR decomposition. Note, however, that there are now double the amount of sensors as we sample both the x - and y -components of the velocity. We did not, however, generate 30 sensors for each component of the velocity vector, mainly because it is interesting to see what the scheme picks if it is limited to only 30 in total. The scheme picked roughly equal amounts of x - and y -sensors. They are, however, not uniformly distributed, as there are more x -sensors near the front of the robot, and more y -sensors toward the rear, see Fig. 12.7.

12.5 Testing on Unseen Data

In this section, we test the models' performance in two cases; the withheld data from Sect. 12.4, and a new simulation scenario, where the swimmer was held at 3.5 cylinder diameters downstream from the cylinder and the free-stream velocity was set to 0.15 m s^{-1} . The second test case is completely new and represents a good benchmark for the generalizability of the model.

12.5.1 Reconstructing Forces and Torques

The model from Sect. 12.4.2 reconstructs the hydrodynamic forces and torques of each link. By averaging over the links, we can recover the hydrodynamic forcing on the vehicle as a whole. Figure 12.8 shows the averaged forces and torques reconstructed by our model for the first test case, which is an unseen section of the training scenario. We see that the model manages to find the trends in the signals. Note also



Fig. 12.7 The optimal sensors for velocity reconstruction. The red dots are x -velocity sensors, while the green ones are y -velocity sensors

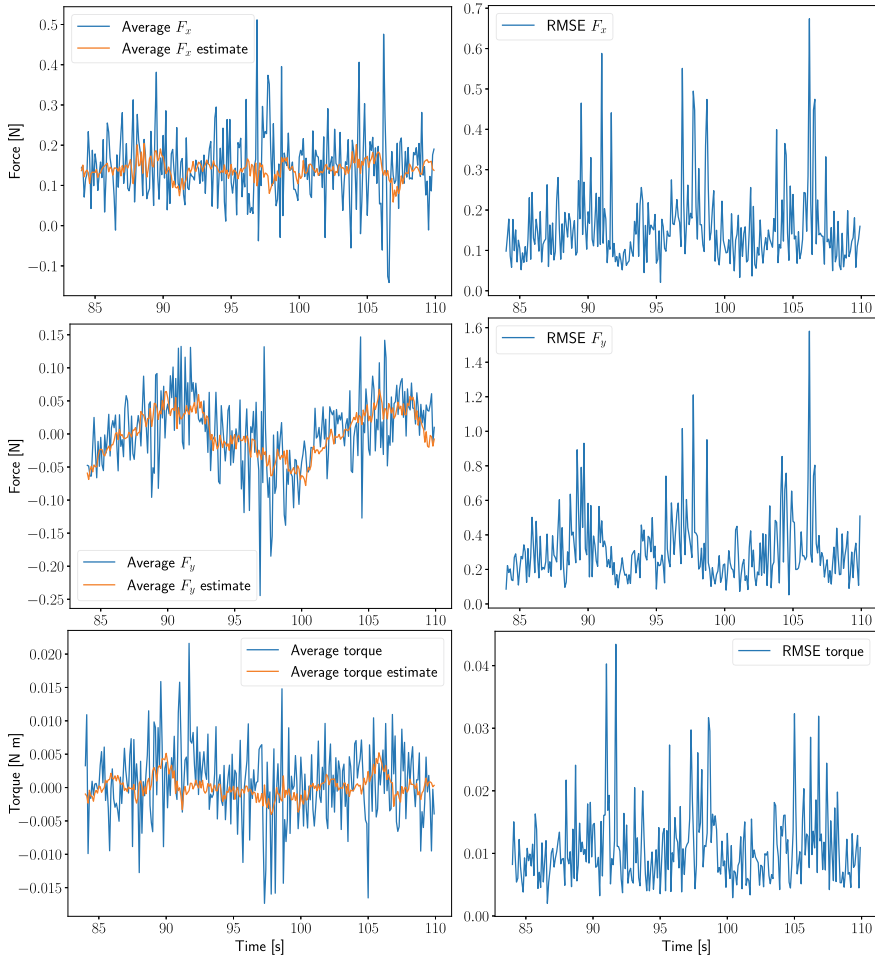


Fig. 12.8 Average hydrodynamic force and torque estimates, and their RMSE for test case one

that the model does not capture the large spikes which coincide with the last link coming to an extremum in its y -trajectory. This point is usually associated with high forces and torques, as the links change direction and induce a forceful interaction with the fluid. Results for the second test case, where the vehicle is simulated further downstream and with higher free-stream velocity, are shown in Fig. 12.9. For the y -forces and torques, the model still manages to keep up with the trend, albeit with a higher RMSE. The model, however, struggles with the x -force in the second test case, where for the first 10s the swimmer has not been forced by a vortex, as the shedding is yet to begin. This is a completely new phenomenon for the model, and, unsurprisingly, it struggles to capture the trend there.

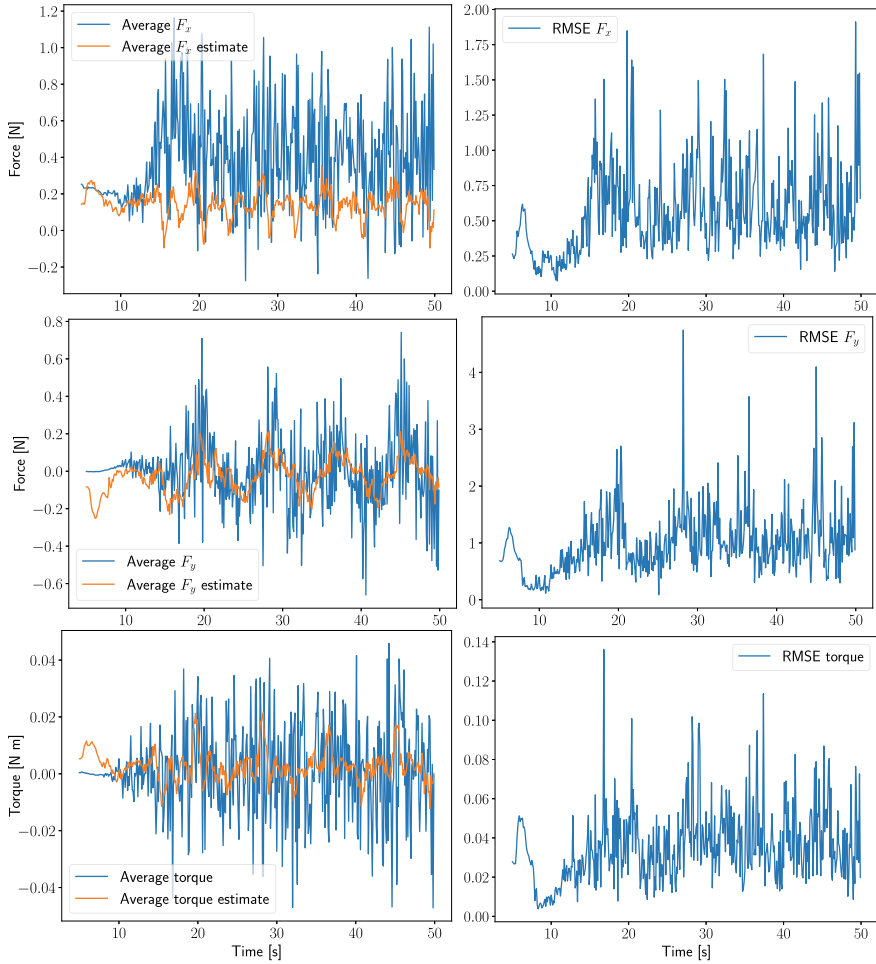


Fig. 12.9 Average hydrodynamic force and torque estimates, and their RMSE for test case two

12.5.2 Reconstructing the Boundary Velocity

Reconstruction of the velocity profile was done with the model proposed in Sect. 12.4.3. Figure 12.10 shows the results for both test cases, where it can be seen that the model performs well even in test case two, a new scenario. The plots report the averaged velocities in the world frame. The reconstructed velocity signals were transformed from their respective link frames to the world frame and then averaged, recall the discussion in Sect. 12.3.2. As with the forces and torques, it is interesting to see how different results the two simulation scenarios produce. Different free-stream velocities and different positions in the wake result in completely different fluid velocity profiles along the body of the swimmer. The difference is,

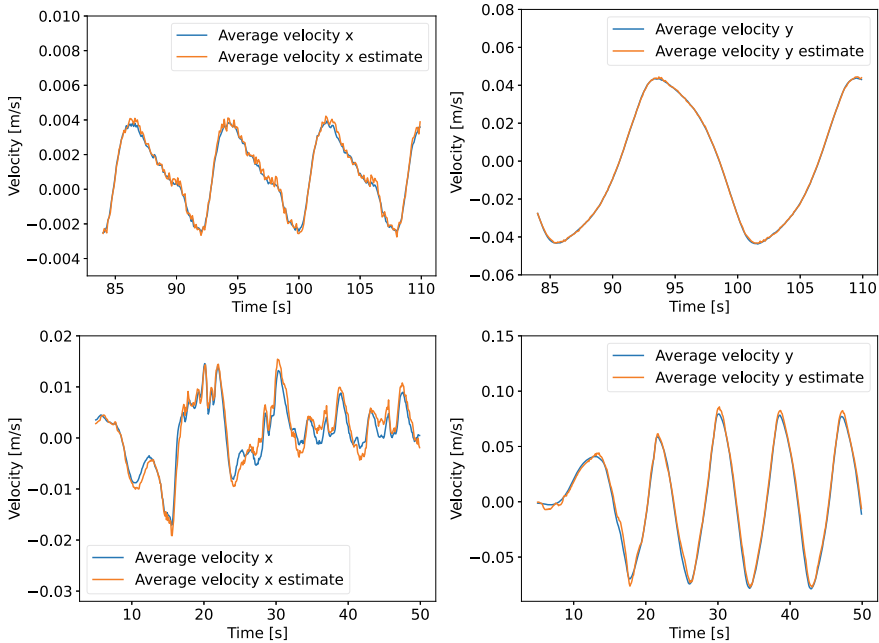


Fig. 12.10 Averaged velocity profile and its reconstruction for test case one (above) and test case two (below)

however, most pronounced in the x -velocity component, while the y -velocity looks qualitatively similar in the two test cases.

12.6 Conclusion and Future Work

In this paper, we have investigated the possibility of using fluid measurements to reconstruct other signals, such as the hydrodynamic forces and torques and fluid velocity profiles, on an articulated swimmer in the wake of a bluff body. We used a PCA model to empirically discover a set of latent coordinates, which we are able to reconstruct through sparse sensor measurements of quantities within the fluid, and subsequently map back to a reconstruction of the original features. We tested the models in two cases, first on a segment of the original simulation scenario that was not used for training data, and then on a new scenario, where the robot was placed in a different part of the wake. The hydrodynamic force and torque reconstruction model was able to discover the trend within these signals. The model was not able to capture the peaks associated with the swimmer completing a half-period within the wake, but rather filters these away. This low-pass filtering property is often desirable for use in a control and planning application. The model's performance for the x -forces

degraded for the second case, while the estimated y -forces still managed to capture the trend. This may indicate that the forces in the x -direction were more affected by changes in the simulation setup than the ones in the y -direction. The velocity profile reconstruction showed promising results, even when tested on data from the second case. While the model performed well on the second test case, the evolution of the averaged x -component of the velocity looked quite different from the first case, while the y -component looked fairly similar to the first test case. This is further indication that the wake is more uniform in its y -behavior as one travels downstream.

In future work, we plan to put these models to use in control and planning frameworks in order to achieve efficient locomotion and harvest energy from the wake. To that end, the degradation of performance in different positions of the wake will have to be solved. A perfectly maintained x -position is not a realistic scenario, even if that variable is actively stabilized, and so we need a model that operates well in all parts of the wake, while a controller stabilizes the swimmer to a specific position. If the models are to be employed in planning, we would also have to extend their capabilities to include some form of forecasting. Ideally, one would also like to make use of past sensor measurements, including some memory element in the model. Finally, it would be interesting to investigate not only optimal sensor positions, but also the optimal number of sensors. In this paper, we used 30 sensors. This number was chosen to avoid tuning a hyperparameter while trying to investigate the viability of the method. An in-depth investigation of the optimal number of sensors will be an interesting topic of future study.

Acknowledgements This result is part of a project that has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme, through the ERC Advanced Grant 101017697-CRÈME.

References

1. Ahmed, S.E., Pawar, S., San, O., Rasheed, A., Iliescu, T., Noack, B.R.: On closures for reduced order models—A spectrum of first-principle to machine-learned avenues. *Phys. Fluids* **33**, 091301 (2021)
2. Maulik, R., San, O., Rasheed, A., Vedula, P.: Subgrid modelling for two-dimensional turbulence using neural networks. *J. Fluid Mech.* **858**, 122–144 (2019)
3. Montgomery, J.C., Coombs, S., Baker, C.F.: The Mechanosensory Lateral Line System of the Hypogean form of *Astyanax Fasciatus*. *Environ. Biol. Fishes* **62**(1), 87–96 (2001)
4. Liao, J.C., Beal, D.N., Lauder, G.V., Triantafyllou, M.S.: The Kármán gait: novel body kinematics of rainbow trout swimming in a vortex street. *J. Exp. Biol.* **206**, 1059–1073 (2003)
5. Liao, J.C., Beal, D.N., Lauder, G.V., Triantafyllou, M.S.: Fish exploiting vortices decrease muscle activity. *Science* **302**, 1566–1569 (2003)
6. Bleckmann, H., Zelick, R.: Lateral line system of fish. *Integr. Zool.* **4**(1), 13–25 (2009)
7. Peleshanko, S., Julian, M., Ornatska, M., McConney, M., LeMieux, M., Chen, N., Tucker, C., Yang, Y., Liu, C., Humphrey, J., Tsukruk, V.: Hydrogel-encapsulated microfabricated haircells mimicking fish cupula neuromast. *Adv. Mater.* **19**, 2903–2909 (2007)

8. McConney, M., Chen, N., Lu, D., Hu, H., Coombs, S., Liu, C., Tsukruk, V.: Biologically inspired design of hydrogel-capped hair sensors for enhanced underwater flow detection. *Soft Matter* **5**, 01 (2009)
9. Ren, Z., Mohseni, K.: A model of the lateral line of fish for vortex sensing. *Bioinspiration Biomimetics* **7**(3), 036016 (2012)
10. Yang, Y., Chen, J., Engel, J., Pandya, S., Chen, N., Tucker, C., Coombs, S., Jones, D.L., Liu, C.: Distant touch hydrodynamic imaging with an artificial lateral line. In: *Proceedings of the National Academy of Sciences*, vol. 103, no. 50, pp. 18891–18895 (2006)
11. Akanyeti, O., Venturelli, R., Visentin, F., Chambers, L., McGill, W.M., Fiorini, P.: What information do Kármán streets offer to flow sensing? *Bioinspiration Biomimetics* **6**(3), 036001 (2011)
12. Venturelli, R., Akanyeti, O., Visentin, F., Jezov, J., Chambers, L., Toming, G., Brown, J., Kruusmaa, M., McGill, W., Fiorini, P.: Hydrodynamic pressure sensing with an artificial lateral line in steady and unsteady flows. *Bioinspiration Biomimetics* **7**, 036004 (2012)
13. Jezov, J., Akanyeti, O., Chambers, L., Kruusmaa, M.: Sensing oscillations in unsteady flow for better robotic swimming efficiency. In: *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, pp. 91–96 (2012)
14. Bernier, C., Gazzola, M., Ronsse, R., Chatelain, P.: Simulations of propelling and energy harvesting articulated bodies via vortex particle-mesh methods. *J. Comput. Phys.* **392**, 34–55 (2019)
15. Holmes, P., Lumley, J.L., Berkooz, G.: *Proper Orthogonal Decomposition*, pp. 86–128. Cambridge Monographs on Mechanics, Cambridge University Press (1996)
16. Brunton, S.L., Kutz, J.N.: *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, 2nd edn. Cambridge University Press (2022)
17. Bright, I., Lin, G., Kutz, J.N.: Compressive sensing based machine learning strategy for characterizing the flow around a cylinder with limited pressure measurements. *Phys. Fluids* **25**, 127102 (2013)
18. Zhang, X., Ji, T., Xie, F., Zheng, H., Zheng, Y.: Unsteady flow prediction from sparse measurements by compressed sensing reduced order modeling. *Comput. Methods Appl. Mech. Eng.* **393**, 114800 (2022)
19. Qi, H., Hughes, S.M.: Invariance of principal components under low-dimensional random projection of the data. In: *19th IEEE International Conference on Image Processing*, pp. 937–940 (2012)
20. Fowler, J.E.: Compressive-projection principal component analysis. *IEEE Trans. Image Process.* **18**(10), 2230–2242 (2009)
21. Manohar, K., Brunton, B.W., Kutz, J.N., Brunton, S.L.: Data-driven sparse sensor placement for reconstruction: demonstrating the benefits of exploiting known patterns. *IEEE Control Syst. Mag.* **38**(3), 63–86 (2018)
22. Orucevic, A., Lysø, M.E.B., Schmidt-Didlauskies, H., Pettersen, K.Y., Gravidahl, J.T.: Optimal positioning of snake robots in vortex wakes using extremum-seeking control. In: *Proceedings of the 22nd IFAC World Congress (Yokohama, Japan)*, Jul 2023
23. Orucevic, A.: *Energy Harvesting and Practical Stabilization of Underwater Snake Robots*. Phd thesis, NTNU, Trondheim (2023)
24. Abramowitz, M., Stegun, I.A.: *Handbook of Mathematical Functions: With Formulas, Graphs, and Mathematical Tables*. Dover Books on Mathematics (1965)

Chapter 13

Control Theoretic Approach to Fine-Tuning and Transfer Learning



Erkan Bayram, Shenyu Liu, Mohamed-Ali Belabbas, and Tamer Başar

Abstract Given a training set in the form of a paired $(\mathcal{X}, \mathcal{Y})$, we say that the control system $\dot{x} = f(x, u)$ has learned the paired set via the control u^* if the system steers each point of $\mathcal{X}$ to its corresponding target in $\mathcal{Y}$. If the training set is expanded, most existing methods for finding a new control u^* require starting from scratch, resulting in a quadratic increase in complexity with the number of points. To overcome this limitation, we introduce the concept of *tuning without forgetting*. We develop an *iterative algorithm* to tune the control u^* when the training set expands, whereby points already in the paired set are still matched, and new training samples are learned. At each update of our method, the control u^* is projected onto the kernel of the end-point mapping generated by the controlled dynamics at the learned samples. It ensures keeping the end points for the previously learned samples constant while iteratively learning additional samples.

Keywords Control of ensemble of points · Geometric control · Fine-tuning · Control for learning

E. Bayram, M.-A. Belabbas and T. Başar—Research of UIUC authors was supported in part by the ARO Grant W911NF-24-1-0085, NSF-CCF 2106358, ARO W911NF-24-1-0105 and AFOSR FA9550-20-1-0333.

E. Bayram (✉) · M.-A. Belabbas · T. Başar
Coordinated Science Laboratory, University of Illinois at Urbana-Champaign, Urbana, IL, USA
e-mail: ebayram2@illinois.edu

M.-A. Belabbas
e-mail: belabbas@illinois.edu

T. Başar
e-mail: basar1@illinois.edu

S. Liu
The School of Automation, Beijing Institute of Technology, Beijing, China
e-mail: shenyuliu@bit.edu.cn

13.1 Introduction

In recent years, neural networks and other data-driven models have grown significantly in size and capability, often requiring extensive computation for training. To address the challenge of retraining models for additional tasks or new data, methods like *transfer learning* and *fine-tuning* have become popular. These methods allow models to be adapted for new tasks or updated with new examples without complete retraining.

In this paper, we show that control theory provides a natural framework, as well as efficient algorithms, to perform these tasks. Mathematically, fine-tuning and transfer learning are often treated similarly, as these methods are blind to the nature of the examples they are fed [1]. This also applies to the algorithms we propose here, but we focus on fine-tuning for the sake of exposition.

The models we consider are controlled differential equations of the type

$$\dot{x} = f(x, u) \quad (13.1)$$

(see Sect. 13.2 for their use in learning and the proper definitions). For a given paired training set $(\mathcal{X}, \mathcal{Y})$ with finite cardinality q , the learning task is to find a control u^* so that the flow $\varphi_t(u^*, \cdot)$ generated by (13.1) satisfies

$$R(\varphi_T(u^*, x^i)) = y^i, \text{ for all } (x^i, y^i) \in (\mathcal{X}, \mathcal{Y})$$

for some finite $T \geq 0$ and a given fixed readout map R . This task is also known as the control of an ensemble of points [2]. Thus, we interchangeably refer to the training sample set $\mathcal{X}$ as the input ensemble and the training label set $\mathcal{Y}$ as the output ensemble in line with the usage in the control theory literature.

In this work, our goal is to adapt a control function u^* , trained on the original set, to an expanded training set. This aligns with the main goal of fine-tuning, where parameters u^* are adapted by further training on an expanded dataset.

One of the main issues addressed by fine-tuning is to avoid a loss of performance on previous tasks while specializing the model to a new dataset. This problem is also known as *catastrophic forgetting* in the continual learning (incremental learning) literature.

Common approaches involve learning a sequence of tasks one by one and aiming to match the performance as if they were observed simultaneously [3]. However, in continual learning, access to previously observed data points is typically not available. When such access exists, it is usually discussed in the literature on robust fine-tuning. Similar approaches to our work can be found in optimization-based continual learning methods that incorporate memory [4].

In the control theory literature, finding a control u that steers each point in $\mathcal{X}$ with the cardinality q to the associated labels in $\mathcal{Y}$ is done by concatenating all points in $\mathcal{X}$ in a state of higher dimension, and replicate the controlled dynamics q times [2]. Then, this approach requires solving a controlled ordinary differential

equation (CODE) for the q -tuple system. We call this method the q -folded method, details of which will be discussed later in the paper. This approach has two practical shortcomings:

- **Tuning Ability:** If the training set is augmented with additional samples, it is necessary to apply the q -folded method on the new set from scratch.
- **Scalability:** The q -folded method has a complexity of $\mathcal{O}(n^2 q^2 N)$ per iteration with N time-step discretization. However, our method has a complexity of $\mathcal{O}(n^2 q N)$ per iteration with N time-step discretization. Therefore, as the size of the training set, q , grows, the complexity of the q -folded method increases quadratically, leading to scalability issues while the complexity of our method increases linearly.

We introduce here a novel fine-tuning method. It is based on the hereby introduced concept of *tuning without forgetting* $\mathcal{X}$. The method consists of an iterative algorithm in which each update on the control function u , denoted by δu , is selected as a projection of the gradient of the given cost functional onto the kernel of end-point mapping generated by the controlled dynamics. It ensures keeping the end points for the previously steered points constant up to the first order while simultaneously steering the new points of the training set to their associated labels.

Our work thus contributes to the tuning ability and scalability of the methods using controlled dynamical systems for supervised learning and addresses the challenges stemming from training set updates. We will compare our proposed method to an existing fine-tuning method in a computational example.

Furthermore, as our method iteratively learns from samples $(\mathcal{X}, \mathcal{Y})$, we do not need to have a control system of dimension scaling with q . This is particularly important since, as we mentioned earlier, this scaling causes computational issues, but it can additionally run into memory capacity issues as well. These features make our approach a preferred alternative to the existing q -folded method in the control of an ensemble of points.

Related Works: A popular idea is to restrict the distance between the updated parameters and the original parameters [1] to avoid a loss of performance on the previous task while specializing the model to a new dataset. The works [5, 6] propose to use Euclidean norm and the matrix absolute row sum matrix norm (MARS) regularization to restrict the distance of tuned parameters from the reference parameters. The work [7] proposes a trainable projected gradient method (TPGM) for fine-tuning to automatically learn the distance constraints for each layer.

Other optimization-based approaches in the continual learning literature include [4], which proposes the so-called “gradient episodic memory (GEM),” that performs an update in weight in the direction of gradient of loss function if it has negative inner product with the gradient of loss for the previous samples. Furthermore, the works [8] propose Orthogonal Gradient Descent (OGD) that stores the gradients for the initially observed points and rectifies the current gradient to be orthogonal to them. The study [9] introduces an algorithm called Adam-NSCL, which sequentially optimizes network parameters in the null space of previous tasks. This null space is approximated by applying singular value decomposition to the covariance matrix of all input features of previous tasks for each linear layer.

The dynamical system approach, which we adopt here, has been used in the past in the learning literature. For example, the studies [10–12] consider a continuous flow and introduce neural ordinary differential equations (NODE). The works presented in [13, 14] focus on the universal approximation capabilities of deep residual neural networks from a geometric control viewpoint. Our work falls within the scope of these two pieces of literature.

13.2 Preliminaries

In this section, we introduce the problem and provide the preliminaries for the iterative algorithm. Consider the paired sets $(\mathcal{X}, \mathcal{Y}) = \{(x^i, y^i)\}_{i=1}^q$, where the $x^i \in \mathbb{R}^n$'s are called initial points so that they are in a connected Riemannian manifold $\mathcal{M}$ and the associated $y^i \in \mathbb{R}^{n_o}$'s are target points (labels). We assume that the elements of the input ensemble (training sample set) $\mathcal{X}$ are pairwise distinct, i.e., $x^i \neq x^j$ for $i \neq j$ and let $\mathcal{I}$ be an index set labeling the entries of the ensembles. We take the system:

$$\dot{x}(t) = f(u(t)x(t)), u \in L_\infty([0, T], \mathbb{R}^{\bar{n} \times \bar{n}}), \quad (13.2)$$

where $x(t) \in \mathbb{R}^{\bar{n}}$ is the state vector at time t and $f(\cdot)$ is a smooth vector field on $\mathbb{R}^{\bar{n}}$. Let $E : x \in \mathbb{R}^n \mapsto \bar{x} \in \mathbb{R}^{\bar{n}}$ be an injective function with $\bar{n} \geq n$, called the up-lift function. Let $R : \mathbb{R}^{\bar{n}} \rightarrow \mathbb{R}^{n_o}$ be a given function, called the *readout map* so that the Jacobian of R is of full row rank. Note that both functions $E(\cdot)$ and $R(\cdot)$ are independent of the control u .

We note that we have adopted a model that is continuous in time for theoretical analysis. However, controlled dynamical systems can be viewed in the context of neural networks with residual connections. When using a model with residual connections, such as a neural network with infinitely many layers, it can be represented by a controlled dynamical system in the form of (13.2) [10, 15]. For example, if we select $f(\cdot) = \tanh(\cdot)$, the function is applied element-wise to the product $u(t)x(t)$, resulting in $\dot{x}(t) = \tanh(u(t)x(t))$. If our numerical approach uses a discretization of 10 time steps, it is equivalent to a 10-layer deep residual network with a $\tanh(\cdot)$ activation function. However, the depth of the network affects the discretization error. We next define the memorization property for a dynamical system.

Definition 13.1 (*Memorization Property*) Assume that a paired set $(\mathcal{X}, \mathcal{Y})$, a fixed readout map R , and an up-lift function E are given. The control u is said to have *memorized the ensemble* $(\mathcal{X}, \mathcal{Y})$ for the model $\dot{x}(t) = f(x(t), u(t))$ if the following holds for a finite $T \geq 0$:

$$R(\varphi_T(u, E(x^i))) = y^i, \forall x^i \in \mathcal{X},$$

where the flow $\varphi_t(u, \cdot)$ is generated by the system at the control function u .

We suppress the subscript T in the notation at $t = T$ for simplicity. We call $R(\varphi(u, E(\cdot)))$ the *end-point mapping*. That is, we have the following end-point mapping for a given u at x^i :

$$x^i \in \mathbb{R}^n \xrightarrow{E} \bar{x}^i \in \mathbb{R}^{\bar{n}} \xrightarrow{\varphi_T(u, \cdot)} \bar{y}^i \in \mathbb{R}^{\bar{n}} \xrightarrow{R} y^i \in \mathbb{R}^{n_o}.$$

In other words, the dynamical system has memorized the ensemble if the end-point mapping maps each $x^i \in \mathbb{R}^n$ to the corresponding $y^i \in \mathbb{R}^{n_o}$. The learning problem turns into a “multi-motion planning” problem (in which we look for a control function u that steers all $\bar{x}^i$ in $\mathcal{X}$ to the corresponding $\bar{y}^i$). One can also see that, from the deep learning perspective, the memorization property is equivalent to the universal interpolation property of neural networks [14].

For convenience, we assume that E is the identity function, meaning $\bar{n} = n$, but our result holds for any injective up-lift function. Thus, we interchangeably use x^i and $E(x^i)$ for any $x^i \in \mathcal{X}$. If $n = n_o$ and R is the identity map, we call the problem the control of an ensemble of points with *fixed end points*. Otherwise, we call the problem the control of an ensemble of points with *partially constrained end points*. In this work, we consider the latter. For the sake of simplicity, we let the map R be the orthogonal projection of a point in $\mathbb{R}^n$ onto the subspace $\mathbb{R}^{n_o}$; precisely, $R : x \in \mathbb{R}^n \mapsto Cx \in \mathbb{R}^{n_o}$ where $C = [0_{n_o \times n - n_o} \ I_{n_o \times n_o}] \in \mathbb{R}^{n_o \times n}$. We define per-sample cost functional $\mathcal{J}^i(u)$ for a given point x^i as follows:

$$\mathcal{J}^i(u) = \frac{1}{2} \|C\varphi(u, x^i) - y^i\|^2.$$

Then, the cost functional for the control of an ensemble of points with partially constrained end points including regularization is defined as follows:

$$\mathcal{J}(u, \mathcal{X}) := \sum_{i=1}^q \|C\varphi(u, x^i) - y^i\|^2 + \lambda \int_0^T \|u(\tau)\|^2 d\tau, \quad (13.3)$$

where λ is some regularization coefficient. If we had a single initial point, the minimization of the functional (13.3) subject to the dynamic (13.2) would be the Bolza optimal control problem with a fixed time [16]. However, we have an ensemble of points $\mathcal{X}$, which might be a finite set of points or a continuum.

q-Folded Method: The authors in [15] discuss the following method, known as the *q-folded method*, to find the control function u . This turns the problem into a Bolza problem with the following steps: Copy all the points in $\mathcal{X}$ into an q -tuple, denoted by X_0 .

Similarly, stack all the output points into a vector of dimension $n_o q$, denoted by Y , and copy n -dimensional dynamics in (13.2) q -times, creating an nq -dimensional vectors $F(u(t), X(t))$. Then, they find the flow $\vec{\varphi}_t(u, \cdot) : \mathbb{R}^{nq} \times \mathbb{R} \rightarrow \mathbb{R}^{nq}$, by solving

the minimization problem associated with the following functional for a finite time T :

$$\mathcal{J}(u, X_0) := \|\Lambda(C)\vec{\varphi}_T(u, X_0) - Y\|^2 + \int_0^T \|u(\tau)\|^2 d\tau,$$

where $\Lambda(C) := \text{diag}(C, \dots, C) \in \mathbb{R}^{n_0 q \times n q}$ subject to $\dot{X}(t) = F(u(t), X(t))$, $X(0) = X_0$. The stacked structure of $\dot{X}(t)$ and the definition of $\Lambda(C)$ ensure that the flow $\vec{\varphi}_t(u, \cdot)$ is constructed by concatenating q instances of the flow $\varphi_t(u, \cdot)$.

One can easily see that q -folded method requires solving a boundary value problem on qn dimension. Solving a boundary value problem by using the shooting method requires solving backward-forward initial value problem for CODE [17] at each iteration. An explicit solver of an initial value problem for CODE on $\mathbb{R}^n$ has the complexity of $\mathcal{O}(n^2 N)$ per iteration with N time-step discretization. Consequently, the q -folded method has complexity of $\mathcal{O}(n^2 q^2 N)$ per iteration. Therefore, as the size of the training set, q , grows, the complexity of the q -folded method increases quadratically.

13.3 Main Results

Controllability on Partially Constrained Ensembles: We first state a simple condition for the existence of a control function u that memorizes a given ensemble $(\mathcal{X}, \mathcal{Y})$. To be more precise, we introduce the following subspaces of our control space:

$$U(x^i, y^i) := \{u \in L_\infty([0, T], \mathbb{R}^{n \times n}) \mid R(\varphi(u, x^i)) = y^i\}.$$

Then, we need to show that there exists a control function $u \in \bigcap_{i=1}^q U(x^i, y^i)$ to prove that the model has memorization property.

Let g_1 and g_2 be differentiable vector fields in $\mathcal{M} \subseteq \mathbb{R}^n$. We call the Lie bracket of g_1 and g_2 [18], denoted by $[g_1, g_2](x)$, the vector field:

$$[g_1, g_2](x) := \frac{\partial g_2(x)}{\partial x} g_1(x) - \frac{\partial g_1(x)}{\partial x} g_2(x), \text{ for } x \in \mathcal{M}.$$

Definition 13.2 (*Control distributions*) Let $\dot{x} = f(x, u)$, $u \in L_\infty([0, T], \mathbb{R}^{n \times n})$, we associate it with the sets of vector fields defined recursively

$$\mathcal{F}^k = \mathcal{F}^{k-1} \cup \{[g_i, g_j](x) \mid g_i, g_j \in \mathcal{F}^{k-1}\}$$

for $k \geq 1$ with the set of control vectors $\mathcal{F}^0 = \{f(x, u) \mid u \in L_\infty([0, T], \mathbb{R}^{n \times n})\}$. The corresponding distributions at point $x \in \mathbb{R}^n$ are then

$$\mathcal{D}_x^k(\mathcal{F}) = \text{span} \{g(x) \mid g \in \mathcal{F}^k\}.$$

One can see that the distributions are nested $\mathcal{D}_x^k(\mathcal{F}) \subseteq \mathcal{D}_x^{k+1}(\mathcal{F})$. The control distribution $\mathcal{D}_x^k(\mathcal{F})$ is said to be involutive if $\mathcal{D}_x^k(\mathcal{F}) = \mathcal{D}_x^{k+1}(\mathcal{F})$ [18]. We call $\mathcal{F}$ *bracket-generating* if $\mathcal{D}_x^\infty(\mathcal{F})$ spans $T_x E(\mathcal{M})$ for all $x \in E(\mathcal{M})$.

We denote q times Cartesian product of $E(\mathcal{M})$ by $E(\mathcal{M})^q := E(\mathcal{M}) \times \cdots \times E(\mathcal{M})$. A finite ensemble of points is a q -tuple $X = [E(x^1)^\top, \dots, E(x^q)^\top]^\top \in E(\mathcal{M})^q \subseteq \mathbb{R}^{nq}$. We define the set $\Delta^q := \{[E(x^1)^\top, \dots, E(x^q)^\top]^\top \in E(\mathcal{M})^q \mid E(x^i) = E(x^j) \text{ for } i \neq j\}$. Let $E(\mathcal{M})^{(q)} := E(\mathcal{M})^q \setminus \Delta^q$ be the complement of Δ^q on $E(\mathcal{M})^q$. Please note that if $\dim E(\mathcal{M}) > 1$, then $E(\mathcal{M})^{(q)}$ is an open connected subset and submanifold of $E(\mathcal{M})^q$ [15].

The set of control vectors of the q -folded system, denoted by $\tilde{\mathcal{F}}^0$, is then

$$\tilde{\mathcal{F}}^0 = \{[f(x^1, u)^\top, f(x^2, u)^\top, \dots, f(x^q, u)^\top]^\top \in \mathbb{R}^{nq} \mid u \in L_\infty([0, T], \mathbb{R}^{n \times n})\}.$$

We note that the control function u still belongs to $L_\infty([0, T], \mathbb{R}^{n \times n})$, and not $L_\infty([0, T], \mathbb{R}^{nq \times nq})$. Therefore, having $\mathcal{D}_X^\infty(\tilde{\mathcal{F}}) = T_X E(\mathcal{M})^{(q)} (= \otimes_{i=1}^q T_{x^i} E(\mathcal{M}))$ for all $X \in E(\mathcal{M})^{(q)}$ is stronger requirement than having $\mathcal{D}_x^\infty(\mathcal{F}) = T_x E(\mathcal{M})$ for all $x \in E(\mathcal{M})$.

Lemma 13.1 *Assume that the ensemble $\mathcal{X}$ consists of finite pairwise distinct points and $n > n_o$. For the readout map $R(x) = Cx$, if the set of control vector fields of q -folded system is bracket-generating in $E(\mathcal{M})^{(q)} (= E(\mathcal{M})^q \setminus \Delta^q)$, then there exists a control function u and a finite time $T \geq 0$ such that the system (13.2) memorizes the ensemble $(\mathcal{X}, \mathcal{Y})$ by the control function u .*

Proof First consider that $i \neq j \Leftrightarrow y^i \neq y^j$. In this case, the result trivially follows from [2, Prop 6.1]. Now assume that it is not the case; then, since the cardinality of $\mathcal{Y}$ is finite, we can always construct a set $\mathcal{Y}' \subset \mathbb{R}^n$, with elements $\tilde{y}^i$ having the property that $i \neq j \Leftrightarrow \tilde{y}^i \neq \tilde{y}^j$ and $R(\tilde{y}^i) = y^i$ for all i . We can then appeal to [2, Prop 6.1] to conclude. $\square$

Tuning without forgetting: We describe here the core of our approach to developing numerical methods. Let $\mathcal{X}^j = \{x^i \in \mathcal{X} \mid i = 1, 2, \dots, j\}$ be a subset of the ensemble $\mathcal{X}$ (called *batch* or *sub-ensemble*). Let $\mathcal{Y}^j$ be the corresponding batch of labels. We denote by a superscript the value of the control at a given iteration, i.e., u^k is the control function at the k th iteration of the algorithm. Assume that u^k has memorized the ensemble $(\mathcal{X}^j, \mathcal{Y}^j)$ for the model (13.2). Expand the ensemble by adding the point x^{j+1} with its corresponding label y^{j+1} . Clearly, it does not necessarily hold that $C\varphi(u^k, x^{j+1}) = y^{j+1}$. We propose an iterative method to find a control u^* such that $C\varphi(u^*, x^i) = y^i$ for all $x^i \in \mathcal{X}^{j+1} (= \mathcal{X}^j \cup \{x^{j+1}\})$.

Definition 13.3 Consider an ensemble $(\mathcal{X}, \mathcal{Y})$. Assume that the control u^k has memorized the sub-ensemble $(\mathcal{X}^j, \mathcal{Y}^j)$ for (13.2) for some $j < |\mathcal{I}|$. If the update δu^k satisfies the following:

1. $\mathcal{J}^{j+1}(u^k + \delta u^k) \leq \mathcal{J}^{j+1}(u^k)$,
2. $R(\varphi(u^k + \delta u^k, x^i)) = y^i + o(\delta u^k), \forall x^i \in \mathcal{X}^j$,

then the control function $u^{k+1}(:= u^k + \delta u^k)$ has been *tuned* for $\mathcal{X}^{j+1}$ *without forgetting* $\mathcal{X}^j$.

Paraphrasing, the definition says that we need to select an update δu^k satisfying the following two conditions: (i) it decreases the per-sample cost functional for the additional point x^{j+1} , equivalently, the control system with the updated control u^{k+1} steers the point x^{j+1} to a point whose projection onto the output subspace $\mathbb{R}^{n_o}$ gets closer to the label y^{j+1} , and (ii) the points in $\mathcal{X}^j$ are mapped to points whose projection onto the output subspace is within $o(\delta u^k)$ of their corresponding labels. To be more precise, in *tuning without forgetting*, we aim to minimize the per-sample cost for the new point x^{j+1} , denoted by $\mathcal{J}^{j+1}(u)$, with $u \in \bigcap_{i=1}^j U(x^i, y^i)$.

A projected gradient descent method: Consider the flow $\varphi_t(u, \cdot)$. It yields the trajectory $t \mapsto \varphi_t(u, x^i)$ of (13.2) with control u and initialized at x^i at $t = 0$. The first-order variation of the trajectory $\varphi_t(u, x^i)$ in δu is defined as $\delta \varphi_t(u, x^i) := \varphi_t(u + \delta u, x^i) - \varphi_t(u, x^i)$. Under Condition 2 in Definition 13.3, it should hold that $C\delta \varphi_T(u, x^i) = 0$ for all $x^i \in \mathcal{X}^j$ up to first order in δu .

It is well known that $\delta \varphi_t(u, x^i)$ obeys the linear time-varying equation, which is simply the linearization of the control system (13.2) about the trajectory $\varphi_t(u, x^i)$. Thus, we define the following property:

Definition 13.4 (*Linearized Controllability Property*) We say the system $\dot{x}(t) = f(x(t), u(t))$ has the Linearized Controllability Property (LCP) at x^i for all $u \in L_\infty([0, T], \mathbb{R}^{n \times n})$ if the linear time-varying system

$$\dot{z}(t) = \left(\frac{\partial f(x, u)}{\partial x} \Big|_{(x=\varphi_t(u, x^i), u)} \right) z(t) + \left(\frac{\partial f(x, u)}{\partial u} \Big|_{(x=\varphi_t(u, x^i), u)} \right) v(t) \quad (13.4)$$

$v(t) \in L_\infty([0, T], \mathbb{R}^{n \times n})$ is controllable.

Denote by $\Phi_{(u, x^i)}(t, \tau)$ the state transition matrix of (13.4).

Lemma 13.2 Suppose that a given control function u has memorized the pair of points (x^i, y^i) for the model (13.2). Then

$$\delta \varphi_t(u, x^i) = \int_0^t \Phi_{(u, x^i)}(t, \tau) \frac{\partial f(x, u)}{\partial u} \Big|_{(x=\varphi_\tau(u, x^i), u)} \delta u(\tau) d\tau$$

up to first order in $\delta u(t)$.

Proof For a small variation of the control function $u(\tau)$, denoted by $\delta u(\tau)$, we have the following:

$$\dot{x}(t) + \delta \dot{x}(t) = f(x(t) + \delta x(t), u(t) + \delta u(t)). \quad (13.5)$$

Taking the *first-order* Taylor expansion of (13.5) around the trajectory $x(t) = \varphi_t(u, x^i)$ and subtracting $\varphi_t(u, x^i)$, we get

$$\frac{d \delta \varphi_t(u, x^i)}{dt} = \frac{\partial f(x, u)}{\partial x} \delta \varphi_t(u, x^i) + \frac{\partial f(x, u)}{\partial u} \delta u \quad (13.6)$$

up to first order in δu . To emphasize the linearity of the system above in the control update δu , we can introduce the notation $z(t) = \delta \varphi_t(u, x^i)$. Then, from the introduced notation, one can easily see that (13.6) matches with (13.4). Using the variation of constants formula [16], we have the following:

$$z(t) = \Phi_{(u, x^i)}(t, 0)z(0) + \int_0^t \Phi_{(u, x^i)}(t, \tau) \frac{\partial f(x, u)}{\partial u} \Big|_{(x=\varphi_\tau(u, x^i), u)} \delta u(\tau) d\tau.$$

We have $z(0) = 0$ since we have $\varphi_0(u + \delta u, x^i) = x^i$ and $\varphi_0(u, x^i) = x^i$. This completes the proof. $\square$

Based on Lemma 13.2, we define an affine operator from the space of bounded functions over the time interval $[0, T]$, $\delta u(t) \in L_\infty([0, T], \mathbb{R}^{n \times n})$, to $\mathbb{R}^n$, mapping a control variation to the resulting variation in the end point of the trajectory see through the readout map R . This operator is defined as

$$\mathcal{L}_{(u, x^i)}(\delta u) := R \left(\int_0^T \Phi_{(u, x^i)}(T, \tau) \frac{\partial f(x(\tau), u(\tau))}{\partial u} \delta u(\tau) d\tau \right)$$

for $x(\tau) = \varphi_\tau(u, x^i)$. Next, we let

$$\mathcal{K}(u, x^i) := \text{span}\{\delta u \in L_\infty([0, T], \mathbb{R}^{n \times n}) \mid \mathcal{L}_{(u, x^i)}(\delta u) = 0\}$$

be the kernel of the operator $\mathcal{L}_{(u, x^i)}(\cdot)$. Then, we define the intersection of the kernel $\mathcal{K}(u, x^i)$ for all $x^i \in \mathcal{X}^j$ as follows:

$$\mathcal{K}(u, \mathcal{X}^j) := \text{span}\{\delta u \in L_\infty([0, T], \mathbb{R}^{n \times n}) \mid \delta u \in \bigcap_{x^i \in \mathcal{X}^j} \mathcal{K}(u, x^i)\}.$$

We define the gradient of the given per-sample cost functional for the sample x^i at control u , as the first-order variation of $\mathcal{J}^i(u)$ in δu , precisely, $\nabla_u \mathcal{J}^i(u) := \mathcal{J}^i(u + \delta u) - \mathcal{J}^i(u)$. Then, one can see that we have the following:

$$\nabla_{u(t)} \mathcal{J}^i(u) := \delta \varphi_t^\top(u, x^i) C^\top (C \varphi(u, x^i) - y^i).$$

Note that it is a function of time t via the first-order variation $\delta \varphi_t(u, x^i)$. We define the projection of $\nabla_{u(t)} \mathcal{J}^{j+1}(u)$ on a given subspace of functions $\mathcal{K}(u, \mathcal{X}^j)$ as the solution of the following optimization problem:

$$\text{proj}_{\mathcal{K}(u, \mathcal{X}^j)} \nabla_{u(t)} \mathcal{J}^{j+1}(u) := \arg \min_{d(t) \in \mathcal{K}(u, \mathcal{X}^j)} \int_0^T \|d(\tau) - \nabla_{u(\tau)} \mathcal{J}^{j+1}(u)\|^2 d\tau.$$

Now, we can state the main result:

Theorem 13.1 *Consider model (13.2) and suppose that the control vector fields of q -folded system (13.2) are bracket-generating in $E(\mathcal{M})^{(q)}$, and the control function u^k has memorized the ensemble $(\mathcal{X}^j, \mathcal{Y}^j)$. Assume the space of controls that memorize $(\mathcal{X}, \mathcal{Y})$ is connected Banach submanifold of $L_\infty([0, T], \mathbb{R}^{n \times n})$. If δu^k is selected as $\text{proj}_{\mathcal{K}(u, \mathcal{X}^j)} \nabla_{u(t)} \mathcal{J}^{j+1}(u)$, then the control function $u^{k+1} (:= u^k + \delta u^k)$ for $\mathcal{X}^{j+1}$ has been tuned without forgetting $\mathcal{X}^j$ up to the first order.*

We note that Theorem 13.1 assumes that the space of controls that memorize $(\mathcal{X}, \mathcal{Y})$ is a connected Banach submanifold of $L_\infty([0, T], \mathbb{R}^{n \times n})$. This assumption guarantees that gradient descent to minimize $\mathcal{J}^{j+1}(u)$ subject to $u \in \bigcap_{i=1}^j U(x^i, y^i)$ is a well-founded approach. However, by using LCP, we have the following theorem to relax this assumption.

Theorem 13.2 *Assume the system $\dot{x}(t) = f(x(t), u(t))$ on a manifold $E(\mathcal{M})$ has the LCP for each x^i , and that the set of control vector fields of q -folded system is bracket-generating in $E(\mathcal{M})^{(q)}$. Assume that $E(\mathcal{M})$ is connected and that the fundamental group $\pi_1(E(\mathcal{M})) = 0$. Then, the space of controls that memorize $(\mathcal{X}, \mathcal{Y})$ is connected Banach submanifold of $L_\infty([0, T], \mathbb{R}^{n \times n})$ of finite codimension.*

Due to our focus on algorithms and because of space limitations, we do not include the proof of Theorem 13.2 here. We will provide it an extended version. Now, we can present the proof of Theorem 13.1.

Proof (Theorem 13.1) From Lemma 13.1, we know that there exists a control u^{k+1} such that

$$C\varphi(u^{k+1}, \mathcal{X}^{j+1}) = \mathcal{Y}^{j+1}.$$

From definition of $\mathcal{L}_{(u^k, x^i)}(\cdot)$ and Lemma 13.2, we have

$$C\varphi(u^k + \delta u^k, x^i) = C\varphi(u^k, x^i) + \mathcal{L}_{(u^k, x^i)}(\delta u^k), \forall x^i \in \mathcal{X}^j$$

up to first order in δu^k . Then, we project $\nabla_u \mathcal{J}^{j+1}(u^k)$ onto $\mathcal{K}(u^k, \mathcal{X}^j)$, that is, we select δu^k as $\delta u^k = \text{proj}_{\mathcal{K}(u^k, \mathcal{X}^j)} \nabla_u \mathcal{J}^{j+1}(u^k)$. The selection satisfies Condition 1 in Definition 13.3. By the construction of $\mathcal{K}(u^k, \mathcal{X}^j)$ and from the memorization assumption, we have that $\mathcal{L}_{(u^k, x^i)}(\delta u^k) = 0, \forall x^i \in \mathcal{X}^j$ and $C\varphi(u^k, x^i) = y^i, \forall x^i \in \mathcal{X}^j$, respectively. Then, we have

$$C(\varphi(u^k + \delta u^k, x^i)) = y^i + o(\delta u^k), \forall x^i \in \mathcal{X}^j.$$

It matches the Condition 2 in Definition 13.3, and this completes the proof. $\square$

We emphasize that we *only* consider the intersection of the kernel $\mathcal{K}(u^k, x^i)$ for all the points $x^i \in \mathcal{X}^j$ because we need to keep the first-order variation of the end-point mapping $C(\varphi(u^k, x^i))$ zero for all points $x^i \in \mathcal{X}^j$ while we allow variation on the end-point mapping for the rest of the points.

13.4 Numerical Method for Tuning Without Forgetting

Building on the results of the previous section, we describe in this section a numerical algorithm to *tune* a control function *without forgetting* $\mathcal{X}^j$ up to the first order. The algorithm comprises three main phases, on which we elaborate below. Assume that a control u^0 has memorized $(\mathcal{X}^j, \mathcal{Y}^j)$ for a given control-affine system. Consider an expansion of the ensemble, $\{x^i\}_{i=j+1}^q$. For the iteration $k = 1$, we let $u^1 = u^0$. We assume that $T = 1$ without loss of generality. For the sake of exposition, we also rely on an explicit Euler method to solve the ODEs involved; using different solvers would entail a trivial modification. We let N be the number of discretization points.

For the sake of notation simplicity, we assume that the control system is affine in the control: $\dot{x}(t) = \sum_{d=1}^p u_d(t) f_d(x(t))$, $u_d(t) \in \mathbb{R}$, $u \in L_\infty([0, T], \mathbb{R}^p)$ for some smooth functions $f_d(\cdot) \in \text{Vect}(E(\mathcal{M}))$, $d = 1, \dots, p$. However, one can easily apply the algorithm by vectorizing the control function in the matrix form (13.2).

We let $u^k \in \mathbb{R}^{pN}$ be the discretization of a time-varying control function and denote the corresponding discretized trajectory by $\varphi_{[1:N]}(u^k, x^i) \in \mathbb{R}^{n \times N}$. We let $\delta u^k \in \mathbb{R}^{pN}$ be the discretization of a time-varying control variation.

Algorithm 13.1 Approximation of $\mathcal{L}_{(u, x^i)}(\cdot)$

```

1: Input:  $u, x^i$ 
2:  $z \leftarrow \varphi_{[1:N]}(u, x^i)$ ,  $\Phi_{N+1} \leftarrow I_{n \times n}$ 
3: for  $\ell = N$  to 1 do
4:    $F[\ell] \leftarrow f(z[\ell], u[\ell])$ 
5:    $\Phi_\ell \leftarrow \Phi_{\ell+1}(I + \frac{\partial F[\ell]}{\partial x})$ 
6: end for
7: return  $C[\Phi_1 F[1], \Phi_2 F[2], \dots, \Phi_N F[N]]$ 

```

Approximation of $\mathcal{L}_{(u, x^i)}(\cdot)$: We first provide a method to compute a numerical approximation of $\mathcal{L}_{(u^1, x^i)}(\cdot)$ for all $x^i \in \mathcal{X}^j$. This is Algorithm 13.1, which computes the numerical approximation of $d_u \varphi(u^k, x^i)$. The algorithm iteratively computes the state transition matrix $\Phi_{(u^k, x^i)}(T, \tau)$ of the system in (13.4) for a given initial point x^i and a control function u^k . Then, $\Phi_{(u^k, x^i)}(T, \tau)$ is multiplied with the matrix of vector fields in (13.2) and the product is stored in a matrix, which is the numerical approximation of $d_u \varphi(u^k, x^i)$. Let $L_i \in \mathbb{R}^{n_o \times pN}$ be the output of Algorithm 13.1 for a given u^k and an initial point x^i . We have $\mathcal{L}_{(u^k, x^i)}(\delta u^k) \approx L_i \delta u^k$. Then, the right kernel of the matrix L_i is the numerical approximation of $\mathcal{K}(u^k, x^i)$.

Phase I: We now implement the statement of Theorem 13.1. We state the Phase I in Algorithm 13.2. We first compute L_i for all $x^i \in \mathcal{X}^j$ by using Algorithm 13.1 to construct $\mathcal{K}(u^k, \mathcal{X}^j)$ for a given j . Then, we column-wise concatenate them and place the concatenated matrix into the first block of $L \in \mathbb{R}^{n_o q \times pN}$ and fill the remaining entries with zeros. One can easily see that the right kernel of the matrix L , denoted by $\mathcal{N}(L)$, is a numerical approximation of $\mathcal{K}(u^1, \mathcal{X}^j)$ for a given j . Following

Theorem 13.1, we project $\nabla_u \mathcal{J}^{j+1}(u^k) \in \mathbb{R}^{pN}$ onto $\mathcal{N}(L)$, which is a linear operation. We have the following for the update control u^{k+1} :

$$u^{k+1} = u^k - \alpha^k \text{proj}_{\mathcal{N}(L)} \nabla_u \mathcal{J}^{j+1}(u^k), \quad (13.7)$$

where $\alpha^k \in \mathbb{R}^+$ is a step size. We repeat the iteration in (13.7), and we compute the matrix L_i for $i = 1, \dots, j$ for the control u^k to replace corresponding blocks in the matrix L until the per-sample loss converges. Once it converges, we pick the next point x^{j+2} and compute the matrix L_i for $i = 1, \dots, j + 1$ to replace corresponding blocks in the matrix L . We then continue picking new points.

Algorithm 13.2 Phase I: Kernel Projected Gradient Descent

```

1:  $L_i \leftarrow \text{Algorithm 13.1}(u^0, x^i), \forall i \in \mathcal{I}^j$ 
2:  $u \leftarrow u^0$ 
3:  $L \leftarrow [L_1; L_2; \dots; L_j; 0; \dots; 0]$ 
4: for  $i = j + 1$  to  $q$  do
5:   repeat
6:      $u \leftarrow u - \alpha \text{proj}_{\mathcal{N}(L)} \nabla_u \mathcal{J}^i(u)$ 
7:      $L_\ell \leftarrow \text{Algorithm 13.1}(u, x^\ell)$ , for  $\ell = 1, \dots, i - 1$ 
8:     Update the corresponding block  $L_\ell$  in  $L, \forall \ell \in \mathcal{I}^{i-1}$ 
9:   until convergence
10: end for
11: return  $L, u^k$ 
```

Remark 13.1 If we consider $\mathcal{X}^0$ as an empty set and $u^0 = \vec{0}$, it results in L to be a zero matrix. Thus, one can use the proposed algorithm to design a control function u that sends given initial states to desired final ones. This is an alternative to the q -folded method.

In the next two phases, we focus on minimizing the cost-functional $\mathcal{J}(u, \mathcal{X})$ in (13.3), which comprises two sub-cost functionals: the sum of per-sample costs and the L^2 norm of the control u . First, we minimize the L^2 norm of the control u , and then we minimize the sum of per-sample costs. We call the consecutive execution of Phases II and III *refinement rounds*. One can employ multiple refinement rounds to reduce the cost.

Phase II In this phase, we minimize the L^2 norm of the control u . We project the gradient of the L^2 norm of the control function onto the subspace of functions $\mathcal{K}(u^k, \mathcal{X})$ at each iteration. We state Phase II in Algorithm 13.3.

Phase III In this phase, we aim to refine the control u to steer all the points closer to their associated end points. Let $\mathcal{P}$ be the number of iterations per sample in the algorithm. For a given point x^i , we first call Algorithm 13.1 to compute L_ℓ for all $\ell \in \mathcal{I} \setminus \{i\}$. Notably, we do not update the kernel for x^i . Then, we update the matrix

L accordingly and compute the projected gradient. Then, we pick the next point x^{i+1} . We repeat these steps until the last point in the training set. Then, we again pick x^1 and repeat the steps above $\mathcal{P}$ times.

Algorithm 13.3 Phase II: Regularization

```

1:  $L, u \leftarrow \text{Algorithm 13.2}()$ 
2: for  $i = 1$  to  $q$  do
3:   repeat
4:      $L_\ell \leftarrow \text{Algorithm 13.1}(u, x^\ell), \forall x^\ell \in \mathcal{X}$ 
5:     Update the corresponding block  $L_\ell$  in  $L, \forall \ell \in \mathcal{I}$ 
6:      $u \leftarrow u - \alpha \text{proj}_{\mathcal{N}(L)}(u)$ 
7:   until convergence
8: end for
9: return  $L, u$ 

```

Algorithm 13.4 Phase III: Refinement

```

1:  $L, u \leftarrow \text{Algorithm 13.3}()$ 
2: for  $m = 1$  to  $\mathcal{P}$  do
3:   for  $i = 1$  to  $q$  do
4:      $L_\ell \leftarrow \text{Algorithm 13.1}(u, x^\ell), \forall x^\ell \in \mathcal{X} \setminus \{x^i\}$ 
5:     Update the corresponding block  $L_\ell$  in  $L, \forall \ell \in \mathcal{I} \setminus \{i\}$ 
6:      $u \leftarrow u - \alpha \text{proj}_{\mathcal{N}(L)} \nabla_u \mathcal{J}^i(u)$ 
7:   end for
8: end for
9: return  $u$ 

```

In the *tuning without forgetting* method, we solve a boundary value problem on $\mathbb{R}^n$ for each sample point. Thus our method has a complexity of $\mathcal{O}(qn^2N)$ per iteration in total. The q -folded method has complexity of $\mathcal{O}(q^2n^2N)$ per iteration.

13.5 A Computational Example

In this section, we provide computational examples. We consider the model

$$\dot{x}(t) = W_2(t) \tanh(W_1(t)x(t) + b_1(t)) + b_2(t), \quad (13.8)$$

where $W_1(t), W_2(t) \in L_\infty([0, T], \mathbb{R}^{\bar{n} \times \bar{n}})$ and $b_1(t), b_2(t) \in L_\infty([0, T], \mathbb{R}^{\bar{n}})$. We discretize trajectories with $N = 10$. For a given $x^i \in \mathcal{X} \in \mathbb{R}^2$, we consider the following end-point mapping:

$$x^i \in \mathbb{R}^2 \xrightarrow{E} [x^i{}^\top, 0_{1 \times 6}]^\top \in \mathbb{R}^8 \xrightarrow{\varphi_T(u, \cdot)} \bar{y}^i \in \mathbb{R}^8 \xrightarrow{C=[0, \dots, 0, 1]} y^i = \begin{cases} -1 & , \|x^i\|_2^2 \leq 1 \\ 1 & , \|x^i\|_2^2 > 1 \end{cases}.$$

In words, we consider a unit radius ball in $\mathbb{R}^2$ centered at the origin. If a point is inside the ball, we aim to steer it to the hyperplane $x_8 = 1$, otherwise, to the hyperplane $x_8 = -1$ (a similar example is provided in [11]). One can see that the set of control vector fields for the (overparametrized) q -folded control system (13.8) is bracket-generating in $E(\mathcal{M})^{(q)}$. We define the average error for a control u on the ensemble $\mathcal{X}$ as

$$\mathcal{E}(u, \mathcal{X}) := \frac{1}{|\mathcal{X}|} \sum_{x^i \in \mathcal{X}} \|C\varphi(u, x^i) - y^i\|.$$

We have a set $\mathcal{X}$ of cardinality 64 whose elements are indexed by $\mathcal{I}$. First, we have that an initial training set (sub-ensemble) $\mathcal{X}^j$ is given for some j . We apply the q -folded method in Sect. 13.2 to learn these samples and denote the control function that q -method gives by u^0 . Then, we expand the training set $\mathcal{X}^j$ to $\mathcal{X}^i$ by adding samples in the order provided by $\mathcal{I}$. For convenience, we denote the difference of the subensembles $\mathcal{X}_j^i := \{x^\ell \in \mathcal{X} : j < \ell \leq i\} (= \mathcal{X}^i \setminus \mathcal{X}^j)$.

To adapt the control function u^0 to new samples while remembering previous ones, we use the *tuning without forgetting* $\mathcal{X}^j$ method with multiple refinement rounds to obtain the control function u^* . We also compare our results with the *Penalty Method*, which encourages the tuned control function $\tilde{u}$ to stay close to u^0 using a penalty term in the cost functional [1]:

$$\mathcal{J}(\tilde{u}, \mathcal{X}) := \sum_{i=1}^M \|C\varphi(\tilde{u}, x^i) - y^i\|^2 + \lambda \int_0^T \|\tilde{u}(\tau) - u^0(\tau)\|^2 d\tau.$$

Figure 13.1 depicts the average error on different sets as a function of number of rounds for both algorithms. First, we observe that, in Fig. 13.1, the average error on the new points $\mathcal{E}(u^0, \mathcal{X}_j^i)$ is remarkably higher compared to $\mathcal{E}(u^0, \mathcal{X}^j)$ for any given training set at round 0 (where $u^* = u^0$ and $\tilde{u} = u^0$). As expected, it shows that the control u^0 has *not* learned the additional samples $\mathcal{X}_j^i$ for the given model.

For the learning plasticity [3], we measure the difference in the performance of model on a task between its joint training performance $\mathcal{E}(u^0, \mathcal{X}^i)$ and fine-tuning performance $\mathcal{E}(\cdot, \mathcal{X}_j^i)$. We observe that both $\mathcal{E}(u^*, \mathcal{X}_j^i)$ and $\mathcal{E}(\tilde{u}, \mathcal{X}_j^i)$ for $(i, j) = (64, 52), (32, 8), (32, 25)$ are close to $\mathcal{E}(u^0, \mathcal{X}^i)$. It shows that the control u^* and $\tilde{u}$ has learned the additional samples satisfactorily. For the memory stability [3], we compare the error on the previously learned points $\mathcal{E}(\cdot, \mathcal{X}^j)$ as the tuning continues. We observe that $\mathcal{E}(u^*, \mathcal{X}^j)$ for $j = 8, 16, 25$ is close to $\mathcal{E}(u^0, \mathcal{X}^j)$. It shows that the control u^* keeps the performance on the previously learned points $\mathcal{X}^j$ nearly constants. Comparing our method to the penalty method, we observe that $\mathcal{E}(\tilde{u}, \mathcal{X}^j)$ is higher than $\mathcal{E}(u^0, \mathcal{X}^j)$ for any given j . It shows that the control function $\tilde{u}$ has learned the additional points but has forgotten the previously learned points. Thus, the penalty method does not directly address the issue of catastrophic forgetting.

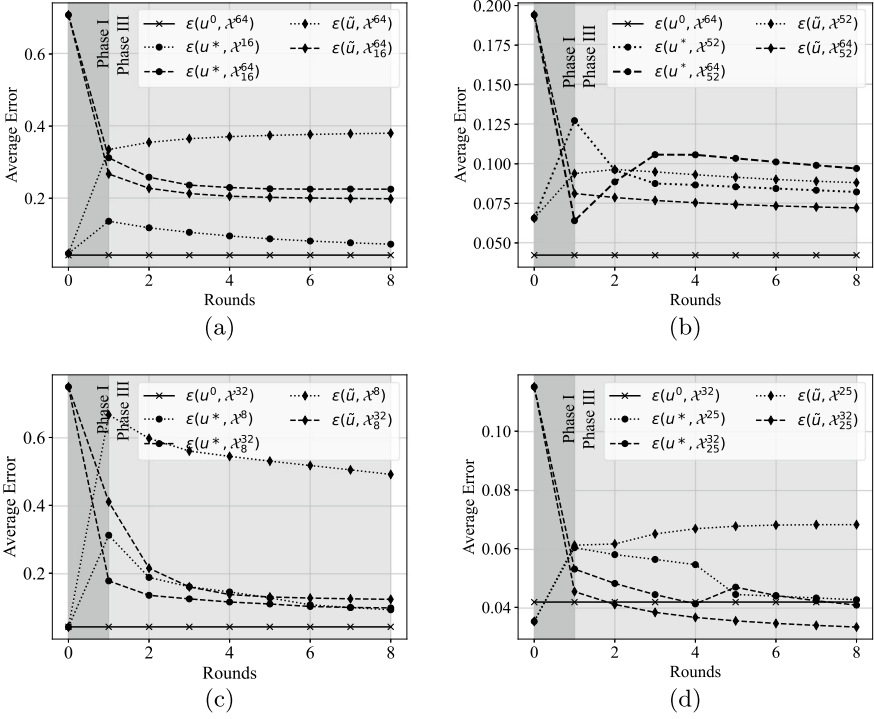


Fig. 13.1 **a** and **b** average error as a function of number of rounds for $|\mathcal{X}| = 64$ for $j = 16$ and $j = 52$, respectively. **c** and **d** average error as a function of number of rounds for $|\mathcal{X}| = 32$ and $j = 8$ and $j = 25$, respectively. The dark-gray region is Phase I region and the light-gray region is Phase III region (each round is followed by Phase II). Average error on the given set for the control functions u^* , $\tilde{u}$, and u^0 are marked by $\bullet$, $\diamond$, and $\times$, respectively

13.6 Summary and Outlook

In this work, we have considered a controlled dynamical system to learn a task, focusing on the impact of training set expansion. We have addressed the issue of model performance loss on the original task while adapting the model to a new task-specific dataset, known as catastrophic forgetting. We have introduced a novel fine-tuning method, *tuning without forgetting* up to first order. Our work contributes to the scalability of control methods, offering a novel approach to adaptively handle training set expansions. In our numerical results, we have observed that the proposed algorithm effectively handles changes in ensemble cardinalities, preserving previously learned points and adopting new points.

The present work can be extended in several directions. Our method has the disadvantage of storing $\mathcal{X}^j$ to compute the set $\mathcal{K}(u, \mathcal{X}^j)$. To address this, one can

keep track of the variation in the kernels to approximate the new kernel. Also, we will provide the proof for Theorem 13.2 which relaxes some of the assumptions for Theorem 13.1 in an extended version.

References

1. Gouk, H., Hospedales, T.M., Pontil, M.: Distance-based regularisation of deep networks for fine-tuning. In: ICLR, 2021
2. Agrachev, A., Sarychev, A.: Control in the spaces of ensembles of points. *SIAM J. Control Optim.* **58**(3), 1579–1596 (2020)
3. Wang, L., Zhang, X., Su, H., Zhu, J.: A comprehensive survey of continual learning: theory, method and application. *IEEE Trans. Pattern Anal. Mach. Intell.* (2024)
4. Lopez-Paz, D., Ranzato, M.: Gradient episodic memory for continual learning. *Adv. Neural Inf. Process. Syst.* **30** (2017)
5. Xuhong, L., Grandvalet, Y., Davoine, F.: Explicit inductive bias for transfer learning with convolutional networks. In: International Conference on Machine Learning, pp. 2825–2834. PMLR (2018)
6. Li, X., Xiong, H., Wang, H., Rao, Y., Liu, L., Huan, J.: Delta: deep learning transfer using feature map with attention for convolutional networks. In: International Conference on Learning Representations (2018)
7. Tian, J., He, Z., Dai, X., Ma, C.-Y., Liu, Y.-C., Kira, Z.: Trainable projected gradient method for robust fine-tuning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 7836–7845 (2023)
8. Farajtabar, M., Azizan, N., Mott, A., Li, A.: Orthogonal gradient descent for continual learning. In: International Conference on Artificial Intelligence and Statistics, pp. 3762–3773. PMLR (2020)
9. Wang, S., Li, X., Sun, J., Xu, Z.: Training networks in null space of feature covariance for continual learning. In: Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition, pp. 184–193 (2021)
10. Chen, R.T., Rubanova, Y., Bettencourt, J., Duvenaud, D.K.: Neural ordinary differential equations. *Adv. Neural Inf. Process. Syst.* **31** (2018)
11. Dupont, E., Doucet, A., Teh, Y.W.: Augmented neural ODEs. *Adv. Neural Inf. Process. Syst.* **32** (2019)
12. Lu, Y., Zhong, A., Li, Q., Dong, B.: Beyond finite layer neural networks: bridging deep architectures and numerical differential equations. In: International Conference on Machine Learning, pp. 3276–3285. PMLR (2018)
13. Cuchiero, C., Larsson, M., Teichmann, J.: Deep neural networks, generic universal interpolation, and controlled odes. *SIAM J. Math. Data Sci.* **2**(3), 901–919 (2020)
14. Tabuada, P., Gharesifard, B.: Universal approximation power of deep residual neural networks through the lens of control. *IEEE Trans. Autom. Control* **68**, 2715–2728 (2023)
15. Agrachev, A., Sarychev, A.: Control on the manifolds of mappings with a view to the deep learning. *J. Dyn. Control Syst.* **28**(4), 989–1008 (2022)
16. Liberzon, D.: *Calculus of Variations and Optimal Control Theory: A Concise Introduction*. Princeton University Press (2011)
17. Keller, H.B.: *Numerical Solution of Two Point Boundary Value Problems*. SIAM (1976)
18. Brockett, R.: The early days of geometric nonlinear control. *Automatica* **50**(9), 2203–2224 (2014)

Part III

Model Predictive Control

Chapter 14

Accelerating Multi-objective Model Predictive Control Using High-Order Sensitivity Information



Markus Herrmann-Wicklmayr and Kathrin Flaßkamp

Abstract Multi-objective model predictive control (MOMPC) is a challenging task in two regards: firstly, the Pareto front approximation has to be provided under real-time constraints and, secondly, an automated decision-making has to be implemented. Thus, we propose a combination of the Pascoletti–Serafini scalarization method with high-order sensitivity computations to approximate the Pareto front at the current MPC step using polynomials. The decision-making is designed to guarantee stability of the MPC loop. In our numerical evaluation, this method outperforms a standard approach agnostic to sensitivity information both in quality of the result and computation time.

Keywords Multi-objective optimization · Stabilizing model predictive control · Scalarization method · High-order sensitivities · Pareto front approximation

14.1 Introduction

Model predictive control (MPC) has been widely used in industry for more than 20 years [1]. This is mainly due to the possibility of rigorously considering system and input constraints as well as optimality criteria in the feedback control. However, the real-time capability of the approach, especially for highly dynamic processes, remains probably the biggest challenge up until today. Several strategies to tackle this challenge for nonlinear MPC have been developed in the area of schemes (e.g., [2, 3]), solvers (e.g., [4, 5]), and complete software frameworks (e.g., [6, 7]).

The challenge of real-time capability becomes even more imminent in the field of multi-objective model predictive control (MOMPC) where typically not one, but multiple solutions have to be computed. Within each MPC step, the multi-objective

M. Herrmann-Wicklmayr (✉) · K. Flaßkamp
Systems Modeling and Simulation, Saarland University, Saarbrücken, Germany
e-mail: markus.herrmannwicklmayr@uni-saarland.de

K. Flaßkamp
e-mail: kathrin.flasskamp@uni-saarland.de

optimal solution consists of the set of optimal compromises, the so-called Pareto front (PF). In the recent survey paper [8, Sect. 4] from 2018, several MOMPC schemes are discussed which can be divided into ones with an offline phase (not in the focus of this paper), e.g., explicit MPC, and without. The schemes without an offline phase either *compute single points using scalarization techniques* of various kinds, or they *approximate the entire Pareto set by means of evolutionary algorithms*.

This paper proposes a novel numerical approach to MOMPC problems that lies in between these two subcategories. We present a new scalarization method, called spherical ray intersection (SRI), based on Pascoletti–Serafini [9] scalarization for nonlinear optimization problems, which we tailor for MOMPC. Our method computes only a few samples of the PF, but also high-order sensitivity information (SI) at these samples. The information obtained in this way can be used to approximate the Pareto set, i.e., the decision variables which are the preimages to the PF, by polynomials, cf. [10]. This is done with (a) “standard” polynomials that do not enforce externally specified derivatives at sample points and (b) Hermite polynomials. The interpolation is currently restricted to lines (which means a restriction to two objectives) since it is hard to decide how to interpolate (hyper)surfaces *efficiently*, i.e., with as few points as possible.

Within the MOMPC scheme, the polynomial-based approximation of the PF is then used in an automated decision-making (DM) process. The goal of the chosen exemplary DM is to determine the knee point of the PF as accurate as possible. We compare the method using the two different polynomial approaches.

Outline: In Sect. 14.2, a discrete-time zero-terminal constraint MOMPC optimization problem (OP) is introduced. Section 14.3 presents the spherical ray intersection (SRI) method and automated DM which are used within the SRI MOMPC scheme. Section 14.4 describes how the computational cost can be reduced by using approximations of the PF. The presented method is then evaluated with a numerical example in Sect. 14.5 and the paper is concluded in Sect. 14.6.

Contribution: This paper demonstrates the interaction between the novel SRI scalarization method and efficient high-order SI computation, achieving an improved Pareto front approximation for stabilizing MOMPC. This approximation facilitates an automated DM process, accelerating the MPC feedback determination and enhancing the real-time capability of the multi-objective strategy.

Notation: The element-wise multiplication is denoted with $\odot$. The vertical concatenation of two scalars, vectors or matrices, X and Y with consistent column dimensions is denoted with $[X \mid Y] := (X^\top, Y^\top)^\top$.

14.2 Multi-objective Model Predictive Control

In this section, we introduce the basic type of OP that is solved repeatedly for MOMPC. The discrete-time zero-terminal constraint MOMPC OP reads

$$\begin{aligned}
\min_{\bar{x}} \quad & \bar{J}_{\text{MO}}(\bar{x}, c_{\mathcal{J}}) = [J_1 \mid \dots \mid J_{n_J}] \\
\text{s.t.} \quad & z_{j+1} = f_{\text{dyn}}(z_j, v_j) \quad \forall j = 0, 1, \dots, N-1, \\
& z_j \in \mathbb{Z}, v_j \in \mathbb{V} \quad \forall j = 0, 1, \dots, N-1, \\
& z_0 = z^0, z_N = 0
\end{aligned} \tag{14.1a}$$

with the optimization variable and the individual cost functions

$$\bar{x} = [z_0 \mid \dots \mid z_N \mid v_0 \mid \dots \mid v_{N-1}], \tag{14.1b}$$

$$J_i = c_i \sum_{j=0}^{N-1} \ell_i(z_j, v_j), \quad i \in \{1, \dots, n_J\}. \tag{14.1c}$$

The coefficient $c_i > 0$ should be understood as a scaling factor and not as a weighting factor.¹ The collection of scaling factors is denoted as $c_{\mathcal{J}} := [c_1 \mid \dots \mid c_{n_J}]$. By assumption, the sets $\mathbb{Z}, \mathbb{V}$ can be described by linear (e.g., box constraints) or nonlinear inequalities.

The OP (14.1a) can compactly be written as

$$\min_{\bar{x}} \quad \bar{J}_{\text{MO}}(\bar{x}, c_{\mathcal{J}}) \quad \text{s.t.} \quad \bar{x}^{\text{lb}} \leq \bar{x} \leq \bar{x}^{\text{ub}}, \quad \bar{g}^{\text{lb}} \leq \bar{g}(\bar{x}) \leq \bar{g}^{\text{ub}}, \tag{14.2}$$

where the absence of a upper/lower bound is indicated with $\pm\infty$. This formulation with bilateral box constraints for the optimization variable $\bar{x}$ and the nonlinear constraints $\bar{g}$ is adapted from the problem formulation of CasADi [11]. This software will be used in Sect. 14.5 to formulate the problem and interface suitable solvers.

14.3 Spherical Ray Intersection MOMPC Iteration Scheme

In this section, the spherical ray intersection (SRI), a method to scalarize multi-objective OPs, is introduced and an automated DM process is described. This serves as a basis for the MOMPC iteration scheme.

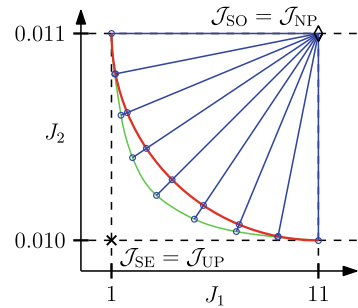
The most relevant quantities in this section and their descriptions are summarized in Table 14.1.²

¹ The weighted-sum scalarization method is not applied. However, these factors allow us to scale the image space in order to get desired properties, e.g., having a specific distance between points.

² According to [12, Chap. 2.2]: Finding the *real* nadir point is a “very difficult problem” for which “no efficient method” for a general problem is known. Hence, “heuristics are often used”, e.g., the estimation via “pay-off tables” (we called it matrix).

Table 14.1 Naming of quantities related to costs and parameters

	Description/Definition
$\mathcal{J}_{\text{MO}}$	Multi-objective cost or cost vector; $\mathcal{J}_{\text{MO}}(x, c_{\mathcal{J}}) := \tilde{\mathcal{J}}_{\text{MO}}(\bar{x}, c_{\mathcal{J}})$ where x is an augmented version of $\bar{x}$, see $(\mathcal{P}(p, \phi))$
Φ	Pay-off matrix; $\Phi := (\mathcal{J}_{\text{MO}}(x_1^*), \dots, \mathcal{J}_{\text{MO}}(x_{n_f}^*))$, where x_i^* is the optimal solution of the single-objective OP with cost function J_i
$\mathcal{J}_{\text{SO}}$	Shooting origin
$\mathcal{J}_{\text{SE}}$	Shooting edge; can be understood as a counterpart to $\mathcal{J}_{\text{SO}}$
$\mathcal{J}_{\text{NP}}$	(Practical ²) nadir point; row-wise maximum of Φ
$\mathcal{J}_{\text{UP}}$	Utopia point; row-wise minimum of Φ
p	Parameter of $(\mathcal{P}(p, \phi))$ (with respect to which the sensitivity information is computed)
p_{DM}	Parameter that is the result of the automated DM process
$p^*(t_k)$	Parameter at k -th iteration with corresponding optimal solution $x(p^*(t_k))$, from which the system inputs to be applied are extracted
$p^\star(t_k)$	Parameter at k -th iteration that is the best possible choice w.r.t. to the DM criterion

Fig. 14.1 Sampling of a stretched circle segment (red) and an exemplary PF (green) via spherical rays (blue)

14.3.1 SRI—A Tailored Scalarization for MOMPC

This subsection is devoted to describe how we obtain a scalarization of an MOMPC OP. It is based on the method of Pascoletti–Serafini [9] and closely related to the normal boundary intersection method [13].

The scalarization method builds upon the inequality constraint $0 \leq \mathcal{J}_{\text{SO}} + dl - \mathcal{J}_{\text{MO}}$ that consists of the shooting origin $\mathcal{J}_{\text{SO}}$, the shooting direction vector d , and its scaling factor l . Visually speaking, the method of Pascoletti–Serafini shoots rays³ from $\mathcal{J}_{\text{SO}}$ with the direction d toward the PF, see Fig. 14.1.

Now, what makes our scalarization tailored to MOMPC is the choice of $\mathcal{J}_{\text{SO}}$ and the shooting direction vector $d(\tau, \mathcal{J}_{\text{SO}}, \mathcal{J}_{\text{SE}})$. We denote $\mathcal{J}_{\text{SE}}$ as the *shooting edge*

³ Note that this visual interpretation neglects the fact that there are cones at the end of the rays in which the points of the Pareto front may lie. This fact becomes relevant when the rays point toward dominated Pareto points.

which can be understood as a counterpart to $\mathcal{J}_{\text{SO}}$.⁴ Appropriate choices for them guarantee that all possible Pareto optimal solutions lie in the so-called descent cone, where all cost functionals do not increase, can be obtained.

In our method, the shooting direction vector d can be imagined as a vector pointing to the surface of a stretched n_J -dimensional hypersphere and can be derived as follows: With $\tilde{n} = n_J - 1$, $\alpha \in \mathbb{R}^{\tilde{n}}$ the parameterization of a hypersphere can be chosen as [14]

$$x_{\text{HS}}(\alpha) = - \begin{pmatrix} \sin(\alpha_1) \\ \text{-----} \\ \cos(\alpha_1) \sin(\alpha_2) \\ \cos(\alpha_1) \cos(\alpha_2) \sin(\alpha_3) \\ \vdots \\ \cos(\alpha_1) \cdots \cos(\alpha_{\tilde{n}-2}) \cos(\alpha_{\tilde{n}-1}) \sin(\alpha_{\tilde{n}}) \\ \text{-----} \\ \cos(\alpha_1) \cdots \cos(\alpha_{\tilde{n}-2}) \cos(\alpha_{\tilde{n}-1}) \cos(\alpha_{\tilde{n}}) \end{pmatrix}.$$

This allows us to parameterize the hypersphere segment $x_{\text{seg}}(\tau) = x_{\text{HS}}(\tau \cdot \pi/2)$ with $x_{\text{seg}}(\tau) \leq 0 \ \forall \ \tau \in [0, 1]^{n_J-1}$. The parameterizations are depicted for $n_J = 2$ and $n_J = 3$ in Fig. 14.2.

Now, our method chooses

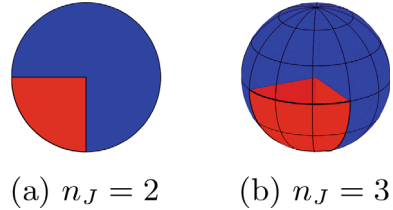
$$d(\tau, \mathcal{J}_{\text{SO}}, \mathcal{J}_{\text{SE}}) = (\mathcal{J}_{\text{SO}} - \mathcal{J}_{\text{SE}}) \odot x_{\text{seg}}(\tau). \quad (14.3)$$

With $e_{i,n}$ as a column vector of length n that is equal to one in the i -th component and zero in all other components and

$$\tau_{\text{axis}}(i) = \begin{cases} e_{i,n_J-1}, & i < n_J \\ 0_{n_J-1 \times 1}, & i = n_J \end{cases}, \quad x_{\text{seg}}(\tau_{\text{axis}}(i)) = -e_{i,n_J}, \quad \mathcal{J}_{\text{SO}} > \mathcal{J}_{\text{SE}}$$

we can conclude that with an appropriate choice of τ the shooting direction vector d can point in the negative direction of each axis.

Fig. 14.2 Marked blue and red/red: points parameterized by $x_{\text{HS}}(\alpha)/x_{\text{seg}}(\tau)$



⁴ With $\mathcal{J}_{\text{SO}} > \mathcal{J}_{\text{SE}}$ it can be imagined that these two points span a hypercuboid.

The stretching/scaling of x_{seg} in (14.3) has advantages that can most easily be explained for the case $n_J = 2$. Here, an equidistant sampling of τ yields an equidistant sampling of a stretched circle segment. This approach leads to a near-equidistant sampling of the PF if it does not deviate too much from a circle segment, see Fig. 14.1. Similar advantages apply for higher dimensional PFs.

We coin the presented scalarization method spherical ray intersection (SRI), because rays are shot from the center of a stretched n -sphere toward the Pareto front. Although it was designed for MOMPC, the SRI scalarization method can also be applied to general multi-objective OPs.

Applying this method with $l \geq 0$ and $p - \tau = 0$ to (14.2) yields the parameterized single-objective OP

$$\begin{aligned} \min_x f(x) &= -l \\ \text{s.t. } \underbrace{\begin{pmatrix} 0 \\ \bar{x}^{\text{lb}} \\ 0_{n_p \times 1} \end{pmatrix}}_{=: x^{\text{lb}}} &\leq \underbrace{\begin{pmatrix} l \\ \bar{x} \\ \tau \end{pmatrix}}_{=: x} &\leq \underbrace{\begin{pmatrix} \infty \\ \bar{x}^{\text{ub}} \\ 1_{n_p \times 1} \end{pmatrix}}_{=: x^{\text{ub}}}, &\underbrace{\begin{pmatrix} 0_{n_J \times 1} \bar{g}^{\text{lb}} \\ 0_{n_p \times 1} \end{pmatrix}}_{=: g^{\text{lb}}} &\leq \underbrace{\begin{pmatrix} \mathcal{J}_{\text{SO}} + dl - \mathcal{J}_{\text{MO}} \\ \bar{g} \\ p - \tau \end{pmatrix}}_{=: g(x, p, \phi)} &\leq \underbrace{\begin{pmatrix} \infty \\ \bar{g}^{\text{ub}} \\ 0_{n_p \times 1} \end{pmatrix}}_{=: g^{\text{ub}}} \end{aligned} \quad (\mathcal{P}(p, \phi))$$

with $\mathcal{J}_{\text{MO}}(x, c_{\mathcal{J}}) := \tilde{\mathcal{J}}_{\text{MO}}(\bar{x}, c_{\mathcal{J}})$. The direction $d = d(\tau, \mathcal{J}_{\text{SO}}, \mathcal{J}_{\text{SE}})$ can be set by the *optimization variable* τ that is constrained by $p - \tau = 0$, i.e., an equality to the *parameter* p is enforced. The chosen formulation renders the function g linear in p . This fact can be exploited in Sect. 14.4.2 because p is the parameter with respect to which the sensitivity information is computed. Furthermore, $(\mathcal{P}(p, \phi))$ consists of the auxiliary parameter $\phi := [\mathcal{J}_{\text{SO}} \mid \mathcal{J}_{\text{SE}} \mid c_{\mathcal{J}} \mid z^0]$.

If the nadir and utopia points are known, choose (i) $\mathcal{J}_{\text{SO}} = \mathcal{J}_{\text{NP}}$ in order to avoid shooting toward dominated points on the PF; (ii) $\mathcal{J}_{\text{SE}} = \mathcal{J}_{\text{UP}}$ for near-equidistant sampling. Furthermore, compute the scaling vector $c_{\text{NP}, \text{UP}}$ as the component-wise reciprocal value of $\mathcal{J}_{\text{NP}} - \mathcal{J}_{\text{UP}}$.

If these information are not available (yet), choose $\mathcal{J}_{\text{SO}}$ and $\mathcal{J}_{\text{SE}}$ suitably. For example, it suffices to set $\mathcal{J}_{\text{SE}} = \tilde{\mathcal{J}}_{\text{SE}} := 0$.

Both quantities $\mathcal{J}_{\text{SO}}$ and d can be adjusted in each MOMPC iteration via p and ϕ . Now, by solving $(\mathcal{P}(p, \phi))$ with various p one can obtain samples of the Pareto front. We emphasize that our method uses a single scalarization technique instead of mixing several methods. This unifies the MOMPC implementation and yields consistent OPs that are used in the computation of the SI, see Sect. 14.4.3.

14.3.2 Algorithm

Remark 14.1 For the auxiliary parameter ϕ the separation between *edgelinner points* is reflected by the augmented notation through the indices “ed”/“in”.

Proof (outline) W.l.o.g. we assume that $N_{v,\text{appl}} = 1$ and do not update the scaling, i.e., $c_{\mathcal{J}}(t_{k+1}) = c_{\mathcal{J}}(t_k)$.

Recursive feasibility and asymptotic stability can, as in the single-objective case [16], be shown by the use of a candidate input sequence $\check{v}(t_{k+1}) = [v_1^*(t_k) \mid \dots \mid v_{N-1}^*(t_k) \mid 0]$ that guarantees the existence of a solution $\mathcal{J}'(t_{k+1})$ that satisfies

$$\mathcal{J}'_i(t_{k+1}) \leq \mathcal{J}_i^*(t_k) - c_i \ell_i(z_0^*(t_k), v_0^*(t_k)) \leq \mathcal{J}_i^*(t_k), \quad i \in 1, \dots, n_J$$

component-wise for all $(z_0^*(t_k), v_0^*(t_k)) \neq 0$. Then, the parameterization of the SRI method and the choice $p^*(t_k) \in [0, 1]^{n_J-1}$ guarantees to yield a Pareto optimal solution $x^*(t_{k+1})$ with $\mathcal{J}_{\text{MO}}(x^*(t_{k+1}), c_{\mathcal{J}}(t_{k+1})) = \mathcal{J}^*(t_{k+1}) \leq \mathcal{J}^*(t_k)$.

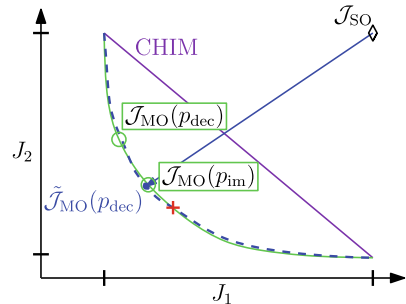
The proof can easily be adapted to the quasi-infinite horizon case that uses a terminal region $\mathbb{Z}^f$ instead of a terminal point.

14.3.3 Automated Decision-Making

For automated decision-making a criterion has to be defined from which the desired choice of a Pareto optimal solution can be derived. We use the so-called *knee point*, for which several characterizations exist: Our implementation is based on the convex hull of individual minima (CHIM) that was introduced with the normal boundary intersection method [13]. The authors in [17] suggest that the knee point can be characterized as the point on the PF that is furthest to the CHIM. An exemplary *furthest to CHIM point* is shown in Fig. 14.4. This means that the goal is to find the point on the PF that maximizes the minimal distance of a point on the PF to the hyperplane defined by the individual minima.

The perpendicular distance of a point $\rho \in \mathbb{R}^{n_J}$ to a hyperplane defined by $w^\top \chi + b = 0$, $\chi \in \mathbb{R}^{n_J}$, is $d_{\text{chm}}(\rho) = |w^\top \rho + b|/\|w\|$ [18, p. 450]. Note that the constants $w \in \mathbb{R}^{n_J}$, $b \in \mathbb{R}$ depend on pay-off matrix Φ . Let the normal vector w point toward the PF. Then, $w^\top \rho + b > 0$. This means that, alternatively, one can use $f_{\text{DM}}(\rho) =$

Fig. 14.4 Idea behind ($\text{OP}_{\text{DM}}^{\text{im}}$). The CHIM line is shown in violet and the furthest to CHIM point is marked by the red cross



$-\tilde{d}_{\text{chim}}(\rho) = -(w^\top \rho + b)$ as the quantity that should be minimized in order to find the furthest to CHIM point. The scaling of the image space $c_{\mathcal{J}}$ evidently does not affect the parameter p that maximizes $\tilde{d}_{\text{chim}}$.

Remark 14.2 Note that we have dropped the time dependence t_k for some quantities in the following paragraphs.

Discrete case: Assume that we computed samples $x(p)$ of the PF. Now, we can evaluate the objective function $f_{\text{DM}}(\mathcal{J}_{\text{MO}}(x(p), c_{\mathcal{J}}))$ for all samples and choose the one that minimizes f_{DM} .

Continuous case: Assume that we have a function $\tilde{x}(p)$ that is an approximation of the optimal solution for each feasible parameter p . This renders $\tilde{\mathcal{J}}_{\text{MO}}(p) := \mathcal{J}_{\text{MO}}(\tilde{x}(p), c_{\mathcal{J}})$ a continuous function of the parameter p , too. A way to obtain $\tilde{x}(p)$ is by interpolation in the decision space based on the available information at the sampling points, cf. Sect. 14.4.1.

Then, the problem

$$\min_{\tau_{\text{dec}} \in [0, 1]} f_{\text{DM}}(\tilde{\mathcal{J}}_{\text{MO}}(\tau_{\text{dec}})) \quad (\text{OP}_{\text{DM}}^{\text{dec}})$$

intends to yield the knee point. A guarantee cannot be given since $\tilde{\mathcal{J}}_{\text{MO}}(p)$ is only an approximation. A slightly different variant of $(\text{OP}_{\text{DM}}^{\text{dec}})$ is

$$\begin{aligned} \min_{\tau_{\text{im}}, \tau_{\text{dec}}, l} f_{\text{DM}}(\tilde{\mathcal{J}}_{\text{MO}}(\tau_{\text{dec}})) \\ \text{s.t. } \tau_{\text{im}} \in [0, 1], \tau_{\text{dec}} \in [0, 1], l \in \mathbb{R}_+^+ \quad (\text{OP}_{\text{DM}}^{\text{im}}) \\ \mathcal{J}_{\text{SO}} + d(\tau_{\text{im}}, \mathcal{J}_{\text{SO}}, \mathcal{J}_{\text{SE}})l - \tilde{\mathcal{J}}_{\text{MO}}(\tau_{\text{dec}}) = 0. \end{aligned}$$

The used indices and superscripts “dec”/“im” are abbreviations for decision/image and refer to the concept of the decision/image space.

The problem $(\text{OP}_{\text{DM}}^{\text{im}})$ can be beneficial when for some points the point-to-point correspondence is bad, i.e., for the same parameter τ_{dec} , the distance between the actual and the approximated PF point $\|\mathcal{J}_{\text{MO}}(\tau_{\text{dec}}) - \tilde{\mathcal{J}}_{\text{MO}}(\tau_{\text{dec}})\|$ can be large, but both fronts are close to each other for differing p , i.e., for any fixed $\tau_{\text{dec}} \in [0, 1]$ there exists a $\tau_{\text{im}} \in [0, 1]$ such that $\|\mathcal{J}_{\text{MO}}(\tau_{\text{im}}) - \tilde{\mathcal{J}}_{\text{MO}}(\tau_{\text{dec}})\| < r$ with r being a small constant. The idea of this approach is illustrated in Fig. 14.4.

With $p_{\text{DM}} = \tau_{\text{dec}}^*$ for $(\text{OP}_{\text{DM}}^{\text{dec}})$ and $p_{\text{DM}} = \tau_{\text{im}}^*$ for $(\text{OP}_{\text{DM}}^{\text{im}})$ we solve $\mathcal{P}(p_{\text{DM}}, \phi_{\text{in}})$ with $\tilde{x}(p_{\text{DM}})$ as an initial guess. This guarantees to yield a feasible optimal solution.

Finally, as in the discrete case, we evaluate f_{DM} for the edge points, inner points, and additionally the point corresponding to p_{DM} and choose the one that minimizes f_{DM} .

The results of both cases, cf. the sequence of computation in Fig. 14.5, are the optimal parameter p^* , solution $x^* = x(p^*)$, and costs $\mathcal{J}^* = \mathcal{J}_{\text{MO}}(x(p^*), c_{\mathcal{J}})$.

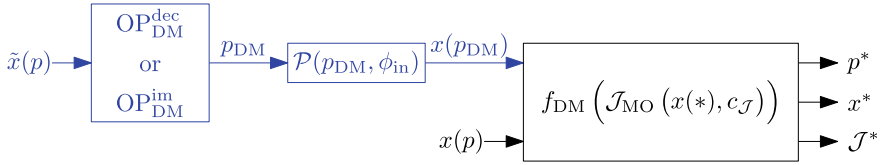


Fig. 14.5 Decision-making flow chart. The black part corresponds to the discrete case. For the continuous case, the blue part is added. Note that $\tilde{x}(p)$ is a function of p but $x(p)$, $x(p_{\text{DM}})$ are optimal solutions for finitely many parameters p , p_{DM}

14.4 High-Order Sensitivity Information

Computing many points on the PF is typically not an option in the context of MOMPC due to real-time constraints. To overcome this, interpolation using high-order SI can be applied.

14.4.1 Reducing the Computational Cost

As a first step, the PF has to be sampled as good as possible w.r.t. the DM objective, i.e., trying to compute samples close to the unknown knee point. We point out that the automated DM is not restricted to our choice that is detecting the knee point.

Remark 14.3 For the remainder of the paper we restrict ourselves to the bi-objective case, i.e., $n_J = 2$.

To reduce the computational cost, we approximate the one-dimensional Pareto front by interpolation in the decision space. This means that we solve the parameterized OP for various parameters and use the gained information to perform an interpolation. The result is an approximation of the optimal solution of the parameterized OP depending on the parameter, denoted as $\tilde{x}(p)$. This yields the PF approximation $\tilde{\mathcal{J}}_{\text{MO}}(p) := \mathcal{J}_{\text{MO}}(\tilde{x}(p))$.

In standard (piece-wise polynomial) interpolation methods like linear, PCHIP, modified Akima, and spline interpolation, only the values at specific evaluation points are considered [19, Chap. 8].

In Hermite interpolation, derivative or sensitivity information (at the evaluation points) up to the N_{der} -th order is also used in determining the polynomial [20, p. 285]. This allows us to determine a Hermite polynomial of two neighboring samples that has the degree $2(1 + N_{\text{der}}) - 1$. An explicit formula to compute the coefficients of the Hermite polynomial is obtained offline and only needs to be evaluated.

We investigate the effect of the use of the resulting polynomials on the performance in the MOMPC scheme in Sect. 14.5.

14.4.2 High-Order Sensitivity Information of a Parameterized Optimization Problem

To compute the Hermite polynomials, we need to obtain sensitivity information at the optimal solutions of parameterized OPs. Since the function $x(p)$ is not explicitly given but implicitly defined by the parameterized OP, the SI $\partial x(p)/\partial p|_{p=p_{\text{eval}}}$ is not directly available. However, using some Karush–Kuhn–Tucker conditions (stationarity and complementary slackness condition for x and g) of the parameterized OP and the implicit function theorem this information can be obtained. In the work [21], it is discussed in detail how to apply this to problem formulations like $(\mathcal{P}(p, \phi))$.

Remark 14.4 Note that the Lagrangian defined in [21, Eq. (7)] is not entirely correct since the summands $\lambda_x^\top x$ and $\lambda_p^\top p$ are missing. Defining the correct Lagrangian (cf. the follow-up work [11]) as

$$\bar{\mathcal{L}}(x, p, \lambda_g, \lambda_x, \lambda_p) := f(x, p) + \lambda_g^\top g(x, p, \phi) + \lambda_x^\top x + \lambda_p^\top p$$

and computing the stationarity and sensitivity condition $\nabla_x \bar{\mathcal{L}} = 0, \nabla_p \bar{\mathcal{L}} = 0$ to obtain [11, Eq. (8a) and (8b)] resolves this issue.

In the subsequent discussion, we mostly stick to the notation of [21] and denote a part of the Lagrangian $\bar{\mathcal{L}}$ as $\mathcal{L}(x, p, \lambda_g) = f(x, p) + \lambda_g^\top g(x, p)$. With $z := [x \mid \lambda_g] =: Q(p)$ we can compute the SI with

$$\frac{\partial Q}{\partial p} = - \left(\frac{\partial F(z, p)}{\partial z} \right)^{-1} \frac{\partial F(z, p)}{\partial p},$$

where $F(z, p)$ encodes the stationarity and complementary slackness condition for x and g as a linear system of equations. By construction, we know that in our case the objective f is not a function of p and that g is linear in p , cf. $(\mathcal{P}(p, \phi))$. We can conclude that

$$\frac{\partial}{\partial p} \nabla_x \mathcal{L} = 0, \quad \frac{\partial}{\partial p} g \stackrel{\text{see def.}}{=} \begin{pmatrix} 0_{n_g - n_p \times n_p} \\ \mathbb{I}_{n_p} \end{pmatrix} \text{ and thus } \frac{\partial F}{\partial p} \stackrel{\text{cf. [21]}}{=} \overbrace{\begin{pmatrix} 0_{n_x + n_g - n_p \times n_p} \\ \mathbb{I}_{n_p} \end{pmatrix}}^{=: M_{0,1}}.$$

Furthermore, $\partial g/\partial p$ and $\nabla_x^2 \mathcal{L}$ are independent of p . As a result, $\partial F/\partial z$ is not explicitly dependent on p , so $\partial Q/\partial p$ is not either. However, z implicitly depends on p .

Instead of symbolically inverting $\partial F/\partial z$ and multiplying it with $\partial F/\partial p$ we can leverage CasADi's algorithmic differentiation capabilities and compute the SI by a linear system solve operation

$$\frac{\partial Q}{\partial p} = \text{solve} \left(\frac{\partial F(z)}{\partial z}, -M_{0,1} \right) =: Q_p(z). \quad (14.6)$$

The resulting quantity Q_p can be further differentiated to obtain higher order SI $\partial^r Q_p / \partial p^r$ with $r = 0, 1, \dots, N_{\text{der}} - 1$. These expressions to compute the high-order SI can be prepared offline and are evaluated online.

Remark 14.5 When differentiating a quantity $q_*(z, p)$ w.r.t. p_i and the dependence $z = Q(p)$ is not explicitly encoded, the proper differentiation has to be applied “by hand”, i.e.:

$$\frac{dq_*(z, p)}{dp_i} = \frac{dq_*(Q(p), p)}{dp_i} \frac{\partial q_*}{\partial z} \cdot Q_{p_i} + \frac{\partial q_*}{\partial p_i},$$

where Q_{p_i} is the i -th column of Q_p .

14.4.3 Sensitivity Computation with Consistent Problem Formulations

We call a problem formulation consistent if all solutions of $(\mathcal{P}(p, \phi))$ are computed using the same auxiliary parameter ϕ . This is obviously not given for the edge points, with $p = \tau_{\text{axis}}(i)$, $i \in \{1, \dots, n_J\}$,⁷ that are computed with $\phi_{\text{ed}} \neq \phi_{\text{in}}$ (cf. Sect. 14.3.2).

However, consistent problem formulations are desirable to achieve a consistent computation of sensitivities using the stationary points (x, λ_g) that depend on the choice of ϕ . In order to keep the total number of optimizations as low as possible, we want to obtain the stationary points (x, λ_g) of $\mathcal{P}(p, \phi_{\text{in}})$, without performing any further optimizations.

By design we know that $\mathcal{P}(p, \phi_{\text{ed}})$ and $\mathcal{P}(p, \phi_{\text{in}})$ yield the same points on the PF, i.e., they have the same values for $\bar{x}$ (and also $\bar{\lambda}_g$). They can only differ in l and the Lagrange multipliers associated to the constraint $0 \leq \mathcal{J}_{\text{SO}} + dl - \mathcal{J}_{\text{MO}}$. The latter happens when $\mathcal{J}_{\text{SO}}$ in ϕ_{ed} was not the nadir point. Then SRI shoots in the direction of dominated points of the PF, i.e., not all constraints $0 \leq \mathcal{J}_{\text{SO}} + dl - \mathcal{J}_{\text{MO}}$ are active.

By construction we know that the $l = 1$ for $\mathcal{P}(p, \phi_{\text{in}})$. When choosing $\mathcal{J}_{\text{SO}} = \mathcal{J}_{\text{NP}}$ all inequalities of $0 \leq \mathcal{J}_{\text{SO}} + dl - \mathcal{J}_{\text{MO}}$ are active. Previously inactive constraints will have altered Lagrange multipliers. The Lagrange multiplier associated with an inequality constraint represents the rate of change of the objective function with respect to a small change in the constraint boundary. This hypothetical change would happen next to dominated points of the PF where the PF is steep. Hence, only a small change in the objective $f = -l$ can be expected. For this reason, we choose to set the corresponding Lagrange multipliers for the lower bound to 10^{-6} and for the upper bound to 0.

⁷ This choice for p is assumed for the rest of the subsection.

14.5 Numerical Example

As an example system, we use a single track car model $\dot{z} = f_{\text{dyn}}^{\text{CT}}(z, v)$ where

$$f_{\text{dyn}}^{\text{CT}}(z, v) = (z_3 \cos(z_4), z_3 \sin(z_4), v_1, z_3/l \tan(z_5), v_2)^\top, \\ z = (x_{\text{pos}}, y_{\text{pos}}, v_{\text{spd}}, \psi, \delta)^\top, \quad v = (a_{\text{acc}}, \omega)^\top.$$

The state consists of the position of the car $(x_{\text{pos}}, y_{\text{pos}})$, its speed v_{spd} , orientation ψ , and steering angle δ . The length of the car is l . The system inputs are the acceleration a_{acc} and the steering angle velocity ω .

For the OP (14.1a), we further choose (i) box constraints $-0.05 \leq v_{\text{spd}} \leq 3$, $|\delta| \leq 20^\circ$, $|a_{\text{acc}}| \leq 10$, $|\omega| \leq 90^\circ$; (ii) the running costs $\ell_1(z, v) = z^\top W z$, $W = \text{diag}(1, 1, 10^{-3}, 10^{-3}, 10^{-3})$, $\ell_2(z, v) = v^\top R v$, $R = \text{diag}(1, 0.1)$; (iii) the prediction horizon $T = 5s$ and $N = 50$ time steps. The initial condition of the single track car model is $z_{\text{meas}}(0) = (-5, -6, 1, 110^\circ, 0)^\top$.

The discrete-time dynamics f_{dyn} are obtained by numerical integration of $f_{\text{dyn}}^{\text{CT}}$ via the *classical RK4* method with a piece-wise constant input and the step size $h = T/N$.

The resulting MOMPC OP is scalarized as described in Sect. 14.3.1. We apply $N_{v, \text{appl}} = 3$ inputs to the system to cause a larger change in the PF due to a larger change in the initial condition $z_{\text{meas}}(t_k)$.

In the MOMPC loop with $N_{\text{iter}} = 16$ iterations, we proceed as described in the shown iteration scheme. Our comparison is mainly divided into the two parts. In the k -th iteration, we do one of the following interpolation types:

- (a) Standard interpolation: We compute the two edge points and $N_{\text{in}} \in \{1, 3\}$ inner points⁸

$$p = \text{saturation}(p^*(t_{k-1})), 0.3, 0.7), \\ p \in \{-0.1, 0, 0.1\} + \text{saturation}(p^*(t_{k-1})), 0.3, 0.7).$$

The samples are interpolated using PCHIP.

- (b) Hermite interpolation: We compute the two edge points and $N_{\text{in}} = 1$ inner point

$$p = \text{saturation}(p^*(t_{k-1})), 0.3, 0.7).$$

The samples at the edges of the PF are post-processed as described in Sect. 14.4.3. Then, the high-order SI with $N_{\text{der}} = 1, 2$ at the PF samples with $\phi = \phi_{\text{in}}$ are computed. We determine the Hermite polynomial for the neighboring parameter pairs $(0, p)$ and $(p, 1)$ by evaluating the coefficient formula.

⁸ The saturation function $\bar{y} = \text{saturation}(y, a, b)$ prevents $\bar{y}$ from taking on a value smaller than a and larger than b .

Table 14.2 Numerical results (t_{ip} , t_{total} in ms)

	(OP _{DM} ^{dec})				(OP _{DM} ^{im})			
	(a) $N_{\text{in}}=1$	(b) $N_{\text{der}}=1$	(a) $N_{\text{in}}=3$	(b) $N_{\text{der}}=2$	(a) $N_{\text{in}}=1$	(b) $N_{\text{der}}=1$	(a) $N_{\text{in}}=3$	(b) $N_{\text{der}}=2$
KPI _{mean}	25.6	6.5	6.79	3.72	17.8	3.49	5.11	2.21
KPI _{max}	115	60.1	20	40.4	111	32.2	17.1	13.9
t_{ip}	1.71	25.9	2.64	67.2	1.66	25.1	2.78	67.6
t_{total}	332	350	444	400	331	359	454	400

Both approaches yield an approximation of the optimal primal variables $\tilde{x}(p)$. We use both DM approaches described in Sect. 14.3.3 to determine p_{DM} and compute the sample $\mathcal{P}(p_{\text{DM}}, \phi_{\text{in}})$.

To evaluate the performance of the DM approaches (OP_{DM}^{dec}) and (OP_{DM}^{im}), we introduce the following error and key performance indicators (KPIs): Assume that we obtain a $p^*(t_k)$ from one of the two approaches. Now, let $p^\star(t_k)$ be the parameter that exactly corresponds to knee point, i.e., $p^\star(t_k)$ minimizes $f_{\text{DM}}(\mathcal{J}_{\text{MO}}(x(p^\star(t_k))))$. Finding this parameter would require to know the whole PF exactly. Since this is not possible in our numerical tests the *ground truth* PF is obtained by a very fine sampling of the PF followed by an interpolation of these samples. Based on this *ground truth* the parameter $p^\star(t_k)$ is determined. The DM error made in the k -th iteration of the MOMPC loop is then defined as

$$e_k := \left\| \mathcal{J}_{\text{MO}}(x(p^*(t_k)), c_{\text{norm}}(t_k)) - \mathcal{J}_{\text{MO}}(x(p^\star(t_k)), c_{\text{norm}}(t_k)) \right\|, \quad (14.7)$$

where $c_{\text{norm}}(t_k) := c_{\text{NP,UP}} \odot c_{\mathcal{J}}(t_k)$. This means that the error is computed in a normed image space ($\mathcal{J}_{\text{NP}} - \mathcal{J}_{\text{UP}} = 1_{n_I \times 1}$). By this, we compute, in a sense, the relative error rather than the absolute error. The KPIs then are

$$\text{KPI}(e) := (\text{KPI}_{\text{mean}}, \text{KPI}_{\text{max}}) = 10^3 \cdot (\text{mean}(e), \text{max}(e)), \quad (14.8)$$

with $e = (e_1, \dots, e_{N_{\text{iter}}})$. Note that a small KPI indicates a good performance.

Based on our DM objective f_{DM} we choose $\mathcal{J}^*(t_k)$, $p^*(t_k)$ and $x^*(t_k)$ among the computed samples. The DM error is determined using (14.7) where $p^\star(t_k)$ is obtained using a PCHIP interpolation of $N_{\text{exact}} = 140$ PF samples and (OP_{DM}^{dec}). Each MOMPC iteration scheme is run 20 times to compensate for fluctuations in the calculation time. The implementation is done in MATLAB[®] and the code is executed on a *12th Gen Intel(R) Core(TM) i7-1260P* processor with 2.10 GHz. In MATLAB[®], the software tool CasADi interfaces the SQP solver WORHP [4]. The OPs are warmstarted.

The numerical results can be seen in Table 14.2. It can be seen that both approaches benefit from using the image space-oriented problem (OP_{DM}^{im}) as all KPI values (cf. (14.8)) are smaller. For this reason, we restrict our discussion to this case. Both cases {a}, $N_{\text{in}} = 1$ and {b}, $N_{\text{der}} = 1$ use one inner point and take roughly the same amount of total time. The increase time t_{total} for the SI-based approach

{b), $N_{\text{der}} = 1$ } mainly comes from the evaluation of the sensitivities needed for the interpolation (increase in t_{ip}). However, {b), $N_{\text{der}} = 1$ } has a significantly better performance both in the mean ($\sim 5\times$) and max ($\sim 3\times$) KPI. By increasing the number of inner points for a) ($\rightarrow$ {a), $N_{\text{in}} = 3$ }) our proposed method using first-order SI ($\rightarrow$ {b), $N_{\text{der}} = 1$ }) still outperforms the other method while now being faster. The case {b), $N_{\text{der}} = 2$ } with second-order SI further improves the KPI values but has the drawback of an increased computation time. Again, this results from the evaluation of the sensitivities.

14.6 Conclusion and Future Work

We presented a multi-objective model predictive control scheme that uses a tailored scalarization method called spherical ray intersection (SRI). Using this method we could obtain Pareto optimal solutions that lead to a convergent control scheme. We showed how to obtain high-order sensitivity information (SI) at these samples, so that Hermite polynomials can be defined to approximate the Pareto front (PF). Based on the PF approximation an automated decision-making (DM) process can be executed to obtain a new sample. The presented method was compared to PCHIP interpolation that does not use any SI. The resulting key performance indicators in combination with the computation times show the effectiveness of the novel approach.

Although SRI is applicable for any number of objectives, the interpolation described in this paper currently is restricted to the bi-objective case. Determining a PF approximation that has a good trade-off between accuracy and computational effort, that results from the number of solutions that have to be computed and the evaluation of their SI, remains open for three or more objectives. Furthermore, other DM criteria within the proposed SRI MOMPC scheme have to be investigated. Lastly, an evaluation on larger sets of benchmark problems has to be performed.

References

1. Qin, S.J., Badgwell, T.A.: A survey of industrial model predictive control technology. *Control Eng. Pract.* **11**(7), 733–764 (2003). [https://doi.org/10.1016/S0967-0661\(02\)00186-7](https://doi.org/10.1016/S0967-0661(02)00186-7)
2. Diehl, M., Bock, H.G., Schlöder, J.P.: A real-time iteration scheme for nonlinear optimization in optimal feedback control. *SIAM J. Control Optim.* **43**(5), 1714–1736 (2005). <https://doi.org/10.1137/S0363012902400713>
3. Yang, X., Biegler, L.T.: Advanced-multi-step nonlinear model predictive control. *J. Process Control* **23**(8), 1116–1128 (2013). <https://doi.org/10.1016/j.jprocont.2013.06.011>
4. Büskens, C., Wassel, D.: The ESA NLP Solver WORHP. In: Fasano, G., Pintér, J.D. (eds.) *Modeling and Optimization in Space Engineering*, pp. 85–110. Springer Optimization and Its Applications. Springer, New York, NY (2013). https://doi.org/10.1007/978-1-4614-4469-5_4
5. Zanelli, A., Domahidi, A., Jerez, J., Morari, M.: FORCES NLP: an efficient implementation of interior-point methods for multistage nonlinear nonconvex programs. *Int. J. Control* **93**(1), 13–29 (2020). <https://doi.org/10.1080/00207179.2017.1316017>

6. Verschueren, R., Frison, G., Kouzoupis, D., Frey, J., Duijkeren, N.v., Zanelli, A., Novoselnik, B., Albin, T., Quirynen, R., Diehl, M.: acados-a modular open-source framework for fast embedded optimal control. *Math. Program. Comput.* (2021). <https://doi.org/10.1007/s12532-021-00208-8>
7. Englert, T., Völz, A., Mesmer, F., Rhein, S., Graichen, K.: A software framework for embedded nonlinear model predictive control using a gradient-based augmented Lagrangian approach (GRAMPC). *Optim. Eng.* **20**(3), 769–809 (2019). <https://doi.org/10.1007/s11081-018-9417-2>
8. Peitz, S., Dellnitz, M.: A survey of recent trends in multiobjective optimal control-surrogate models, feedback control and objective reduction. *Math. Comput. Appl.* **23**(2), 30 (2018). <https://doi.org/10.3390/mca23020030>
9. Pascoletti, A., Serafini, P.: Scalarizing vector optimization problems. *J. Optim. Theory Appl.* **42**(4), 499–524 (1984). <https://doi.org/10.1007/BF00934564>
10. Berger, A.: Use of parametric sensitivities for multi-objective optimisation. Ph.D. thesis, Universität Bremen (Jun 2022), <https://doi.org/10.26092/elib/2098>
11. Andersson, J.A.E., Gillis, J., Horn, G., Rawlings, J.B., Diehl, M.: CasADi: a software framework for nonlinear optimization and optimal control. *Math. Program. Comput.* **11**(1), 1–36 (2019). <https://doi.org/10.1007/s12532-018-0139-4>
12. Ehrgott, M.: *Multicriteria Optimization*. Springer, Berlin, 2nd edn. (2005). <https://doi.org/10.1007/3-540-27659-9>
13. Das, I., Dennis, J.E.: Normal-boundary intersection: a new method for generating the pareto surface in nonlinear multicriteria optimization problems. *SIAM J. Optim.* **8**(3), 631–657 (1998). <https://doi.org/10.1137/S1052623496307510>
14. Blumenson, L.E.: A derivation of n-dimensional spherical coordinates. *Am. Math. Mon.* **67**(1), 63 (1960). <https://doi.org/10.2307/2308932>
15. Stieler, M.: Performance Estimates for Scalar and Multiobjective Model Predictive Control Schemes. Ph.D. thesis (Mar 2018). https://epub.uni-bayreuth.de/id/eprint/3783/1/thesis_mstieler.pdf
16. Mayne, D., Michalska, H.: Receding horizon control of nonlinear systems. *IEEE Trans. Autom. Control* **35**(7), 814–824 (1990). <https://doi.org/10.1109/9.57020>
17. Yu, G., Jin, Y., Olhofer, M.: Benchmark problems and performance indicators for search of knee points in multiobjective optimization. *IEEE Trans. Cybern.* **50**(8), 3531–3544 (2020). <https://doi.org/10.1109/TCYB.2019.2894664>
18. Cheney, W.: *Linear Algebra: Theory And Applications*. Jones and Bartlett Publishers (2010)
19. Quarteroni, A., Sacco, R., Saleri, F.: *Numerical Mathematics*. No. 37 in *Texts in Applied Mathematics*. Springer, New York, Berlin, Heidelberg (2000)
20. Dahmen, W., Reusken, A.: *Numerik für Ingenieure und Naturwissenschaftler*. Springer Lehrbuch, Springer, Berlin, Heidelberg (2008). <https://doi.org/10.1007/978-3-540-76493-9>
21. Andersson, J.A., Rawlings, J.B.: Sensitivity analysis for nonlinear programming in CasADi. In: *IFAC-PapersOnLine*, vol. 51, pp. 331–336 (2018). <https://doi.org/10.1016/j.ifacol.2018.11.055>

Chapter 15

On Discount Functions for Economic Model Predictive Control Without Terminal Conditions



Lukas Schwenkel, Daniel Briem, Matthias A. Müller, and Frank Allgöwer

Abstract In this paper, we investigate discounted economic model predictive control (E-MPC) schemes without terminal conditions in scenarios where the optimal operating behavior is a periodic orbit. For such a setting, it is known that a linearly discounted stage cost guarantees asymptotic stability of any arbitrarily small neighborhood of the optimal orbit if the prediction horizon is sufficiently long. However, in some examples, very long prediction horizons are needed to achieve the desired performance. In this work, we extend these results by providing the same qualitative stability guarantees for a large class of discount functions. Numerical examples illustrate the influence of the discount function and show that with suitable discounting we can achieve significantly better performance than the linearly discounted E-MPC, even for short prediction horizons.

Keywords Economic model predictive control · Periodic optimal control · Practical asymptotic stability

L. Schwenkel thanks the International Max Planck Research School for Intelligent Systems (IMPRS-IS) for supporting him.

L. Schwenkel (✉) · D. Briem · F. Allgöwer
Institute for Systems Theory and Automatic Control, University of Stuttgart, Stuttgart, Germany
e-mail: schwenkel@ist.uni-stuttgart.de

D. Briem
e-mail: daniel.briem@outlook.de

F. Allgöwer
e-mail: allgower@ist.uni-stuttgart.de

M. A. Müller
Institute of Automatic Control, Leibniz University Hannover, Hannover, Germany
e-mail: mueller@irt.uni-hannover.de

15.1 Introduction

Economic model predictive control (E-MPC) has gained popularity due to its ability to directly optimize a certain performance criterion while satisfying constraints for nonlinear systems [4, 5, 9, 11]. In E-MPC, the cost to be minimized by solving an optimal control problem could represent, e.g., the production capacity or energy demand in process control [4]. The optimal operating behavior in such a setting is not pre-specified as in tracking MPC, but implicitly specified by the interplay of the system, the stage cost, and the constraints. Thus, the optimal operation can result in a steady state (e.g., [1, 2, 7, 8]), a periodic orbit (e.g., [10, 14, 16, 17]), or an even more general trajectory (e.g., [3, 12]). To give guarantees on stability and performance, terminal conditions are used in [1] for steady state and in [16] for periodic optimal operation. In [16], however, the stability is only proven for one specific phase on the orbit. Moreover, implementing terminal conditions requires significant design effort including knowledge of the optimal behavior, which is why terminal conditions are often avoided in practice. For E-MPC without terminal conditions in scenarios where the optimal operating behavior is a steady state, [7, 8] provide practical asymptotic stability under usual controllability and dissipativity assumptions. However, if the optimal operating behavior is a periodic orbit and the cost is not constant on the orbit [10], this MPC scheme could fail to approach optimal average performance even for an arbitrarily long prediction horizon as seen in examples in [17, Ex. 6] or [14, Ex. 4, Ex. 5]. To overcome this problem, the authors of [14] propose a p^* -step E-MPC scheme, where p^* is the period length of the optimal orbit. However, this setup requires system- and cost-specific knowledge of the orbit length and can only provide asymptotic convergence but no stability guarantees. As a remedy, [17] introduced a linearly discounted stage cost function and provided guarantees on asymptotic average performance and practical asymptotic stability. This means that any arbitrarily small neighborhood of the optimal orbit can be asymptotically stabilized if the prediction horizon length is long enough. However, for some examples, extremely long prediction horizons may be necessary to achieve the desired performance [17, Ex. 19].

In this work, we investigate for which discount functions we can get the same qualitative guarantees as in [17] and show in numerical examples that even for short prediction horizons the performance can be improved significantly. In particular, we prove stability for a large class of discount functions, thereby enabling the control engineer to use the discount function as a tuning variable. Further, it follows from this result that the asymptotic average performance is optimal up to an error that vanishes with growing prediction horizons. Our numerical examples show the performance of various discounts and we observe in these examples that the linear discount from [17] can be significantly outperformed if, for example, only the second half of the horizon is linearly discounted. We emphasize that this work aims to solve the undiscounted infinite-horizon optimal control problem. This differs from related works on exponentially discounted economic MPC (e.g., [6, 15] and references therein),

which focus on solving the exponentially discounted infinite-horizon optimal control problem.

Outline. Section 15.2 introduces the problem setup and the discounted E-MPC scheme. Section 15.3 contains the main results, while the technical parts of the proof are moved to Appendix A. The numerical examples are presented in Sect. 15.4 and our conclusions are drawn in Sect. 15.5.

Notation. In this work, $\mathbb{N}$, $\mathbb{R}$, and $\mathbb{R}_0^+$ denote the sets of naturals, reals, and non-negative reals. Integers in the interval $[a, b]$ are denoted by $\mathbb{I}_{[a,b]}$ for $a \leq b$. The modulo operator $[k]_p$ returns the remainder when dividing $k \in \mathbb{N}$ by the natural $p \geq 1$. The floor operator $\lfloor x \rfloor$ returns the largest integer that is smaller than or equal to $x \in \mathbb{R}$ and, respectively, $\lceil x \rceil$ as the ceiling operator denotes the smallest integer that is larger than or equal to x . The function class $\mathcal{K}$ contains functions $\alpha : \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+$, which are continuous and strictly increasing with $\alpha(0) = 0$. It is $\alpha \in \mathcal{K}_\infty$ if α is additionally unbounded. We denote $\delta \in \mathcal{L}$ if $\delta : \mathbb{N} \rightarrow \mathbb{R}_0^+$ is non-increasing and $\lim_{k \rightarrow \infty} \delta(k) = 0$. Further, $\gamma \in \mathcal{KL}$ denotes functions $\gamma : \mathbb{R}_0^+ \times \mathbb{N} \rightarrow \mathbb{R}_0^+$ with the property $\gamma(\cdot, t) \in \mathcal{K}$ and $\gamma(r, \cdot) \in \mathcal{L}$.

15.2 Problem Setup

We consider a nonlinear discrete-time system

$$x(k+1) = f(x(k), u(k)), \quad x(0) = x_0 \quad (15.1)$$

with state and input constraints $x(k) \in \mathbb{X} \subseteq \mathbb{R}^n$ and $u(k) \in \mathbb{U} \subseteq \mathbb{R}^m$. The solution of (15.1) at time step k resulting from an input sequence $u \in \mathbb{U}^K$ of length $K \in \mathbb{N}$ and initial condition $x_0 \in \mathbb{X}$ is denoted by $x_u(k, x_0)$. The input sequence $u \in \mathbb{U}^K$ is feasibly denoted by $u \in \mathbb{U}^K(x_0)$, if $x_u(k, x_0) \in \mathbb{X}$ for all $k \in \mathbb{I}_{[0, K]}$. We aim to operate the system optimally concerning a (not necessarily positive definite) stage cost $\ell : \mathbb{X} \times \mathbb{U} \rightarrow \mathbb{R}$, in particular, we want to minimize the asymptotic average performance

$$J_\infty^{\text{av}}(x, u) := \limsup_{T \rightarrow \infty} \frac{1}{T} J_T(x, u) \quad \text{with} \quad J_T(x, u) := \sum_{k=0}^{T-1} \ell(x_u(k, x), u(k)) \quad (15.2)$$

and to stabilize the optimal operating behavior. We assume that the optimal operating behavior is a periodic orbit, which is reachable and controllable.

Definition 15.1 (*Periodic orbit*) Let $\Pi_{\mathbb{X}} : \mathbb{I}_{[0, p-1]} \rightarrow \mathbb{X}$ and $\Pi_{\mathbb{U}} : \mathbb{I}_{[0, p-1]} \rightarrow \mathbb{U}$ for $p \in \mathbb{N}$. Then $\Pi = (\Pi_{\mathbb{X}}, \Pi_{\mathbb{U}})$ is called a p -periodic orbit of system (15.1), if

$$\Pi_{\mathbb{X}}([k+1]_p) = f(\Pi_{\mathbb{X}}(k), \Pi_{\mathbb{U}}(k)) = f(\Pi(k)) \quad (15.3)$$

for all $k \in \mathbb{I}_{[0, p-1]}$. A p -periodic orbit Π is called minimal, if $\Pi_{\mathbb{X}}$ is injective. The distance of a point $(x, u) \in \mathbb{X} \times \mathbb{U}$ to the image of the function Π is defined by $\|(x, u)\|_{\Pi} := \min_{k \in \mathbb{I}_{[0, p-1]}} \|(x, u) - \Pi(k)\|$.

Assumption 15.1 (*Optimal periodic orbit*) There exists a minimal p^* -periodic orbit Π^* that satisfies for any $p \in \mathbb{N}$ and any p -periodic orbit Π that

$$\ell^* := \frac{1}{p^*} \sum_{k=0}^{p^*-1} \ell(\Pi^*(k)) \leq \frac{1}{p} \sum_{k=0}^{p-1} \ell(\Pi(k)). \quad (15.4)$$

The following assumptions characterize the problem class for which we can provide performance and stability guarantees. These assumptions are standard in the context of periodic E-MPC (cf. [10, 14, 16, 17]).

Assumption 15.2 (*Strict dissipativity*) There exist $\underline{\alpha}_{\tilde{\ell}} \in \mathcal{K}_{\infty}$ and a continuous storage function $\lambda : \mathbb{X} \rightarrow \mathbb{R}$ such that for all $x \in \mathbb{X}$ and $u \in \mathbb{U}^1(x)$ we have

$$\tilde{\ell}(x, u) := \ell(x, u) - \ell^* + \lambda(x) - \lambda(f(x, u)) \geq \underline{\alpha}_{\tilde{\ell}}(\|(x, u)\|_{\Pi^*}). \quad (15.5)$$

We call $\ell(x, u) - \ell^*$ the supply rate and $\tilde{\ell}$ the rotated stage cost.

Assumption 15.3 (*Continuity and compactness*) The functions f and ℓ are continuous, and the constraint set $\mathbb{X} \times \mathbb{U}$ is compact. Hence, $\bar{\lambda} := \sup_{x \in \mathbb{X}} |\lambda(x)|$ and $\bar{\ell} := \sup_{(x, u) \in \mathbb{X} \times \mathbb{U}} |\ell(x, u)|$ are finite.

Assumption 15.4 (*Local controllability at $\Pi_{\mathbb{X}}^*$*) There exists $\kappa > 0$, $M_1 \in \mathbb{N}$, $\alpha_c \in \mathcal{K}_{\infty}$ such that for all $z \in \Pi_{\mathbb{X}}^*$ and $x, y \in \mathbb{X}$ with $\|x - z\| \leq \kappa$ and $\|y - z\| \leq \kappa$ a control input sequence $u \in \mathbb{U}^{M_1}(x)$ exists with $x_u(M_1, x) = y$ and

$$\|(x_u(k, x), u(k))\|_{\Pi^*} \leq \alpha_c(\max\{\|x\|_{\Pi_{\mathbb{X}}^*}, \|y\|_{\Pi_{\mathbb{X}}^*}\}), \quad \forall k \in \mathbb{I}_{[0, M_1-1]}. \quad (15.6)$$

Assumption 15.5 (*Finite-time reachability of Π^**) For $\kappa > 0$ from Assumption 15.4, $\exists M_2 \in \mathbb{I}_{\geq 1}$ such that $\forall x \in \mathbb{X}$ there exist $M \in \mathbb{I}_{[0, M_2]}$ and an input sequence $u \in \mathbb{U}^M(x)$ such that $\|x_u(M, x)\|_{\Pi_{\mathbb{X}}^*} \leq \kappa$.

Remark 15.1 We assume finite-time reachability of Π^* for all $x \in \mathbb{X}$ for simplicity of our proofs, similar to [14]. As detailed in [17, Sect. 4] for the linearly discounted E-MPC scheme, this assumption can be relaxed to hold only on a subset of $\mathbb{X}$. A similar relaxation is also possible in the setting with general discounts, which is, however, beyond the scope of the present paper.

15.2.1 Discounted Economic MPC

In this section, we describe the discounted economic MPC scheme without terminal conditions. Given the current measurement $x = x(t)$, we solve at every time step t the optimal control problem

$$V_N^\beta(x) = \min_{u \in \mathbb{U}^N(x)} J_N^\beta(x, u) = \min_{u \in \mathbb{U}^N(x)} \sum_{k=0}^{N-1} \beta\left(\frac{k}{N}\right) \ell(x_u(k, x), u(k)). \quad (15.7)$$

Assumption 15.3 guarantees that for all $x \in \mathbb{X}$ and all $N \in \mathbb{N}$ an (possibly non-unique) optimal control sequence u_x^* exists, i.e., $V_N^\beta(x) = J_N^\beta(x, u_x^*)$. For convenience of notation, we will also use $x_x^*(k) := x_{u_x^*}(k, x)$. Although this lucid notation hides it, both u_x^* and x_x^* depend on the prediction horizon length N and the chosen discount function β . The MPC feedback we apply is $\mu_N^\beta(x) = u_x^*(0)$. The discount function $\beta : [0, 1] \rightarrow [0, 1]$ defines a uniform shape of the discount for all horizon lengths N . This is crucial as the following example shows.

Example 15.1 (*A non-uniform discount*) Let us consider the example $x(t+1) = u(t)$ with only three feasible states $\mathbb{X} = \{-1, 0, 1\}$ and four feasible transitions $\{(-1, -1), (-1, 0), (0, 1), (1, 0)\} \subseteq \mathbb{X} \times \mathbb{U}$. Thus, the only possible decision to make is at $x = -1$ whether to go to $x = 0$ and alternate at the 2-periodic orbit between $x = 0$ and $x = 1$ or whether to stay at $x = -1$. The cost values are $\ell(-1, -1) = 1$, $\ell(-1, 0) = 1$, $\ell(0, 1) = 0$, $\ell(1, 0) = 1.5$ and hence the system is optimally operated at the periodic orbit with average cost $\ell^* = 0.75$, which is better than staying at $x = -1$ for cost 1. However, when considering optimal trajectories of odd length N , it is better to wait one step at $x = -1$ for cost 1 and then approach the orbit rather than approaching it directly and ending with the step of cost 1.5 on the orbit. It was discovered in [14, Example 4] that this phenomenon causes an undiscounted E-MPC scheme with odd prediction horizon length to wait at $x = -1$ forever. In [17, Example 6], it was shown that linear discounts $\beta^{\text{lin}}(\frac{k}{N}) = 1 - \frac{k}{N}$ solve this problem and that a linearly discounted E-MPC goes directly to the optimal orbit for all prediction horizons $N \geq 2$. Let us consider a slight modification of this linear discount $\beta_N(k-1) = \beta_N(k) = \beta^{\text{lin}}(\frac{k}{N})$ for all $k \in 2\mathbb{N}$. We introduced the notation $\beta_N(k)$ as this discount is not uniform, i.e., there exists no function $\beta : [0, 1] \rightarrow [0, 1]$ such that $\beta_N(k) = \beta(\frac{k}{N})$ for all $N \in \mathbb{N}$ and $k \in \mathbb{I}_{[0, N-1]}$. Due to the special shape of the discount, when comparing the strategy of waiting one step with the one of directly going to the orbit, we need to compare only the cost for $k \in \{0, 1, 2\}$ as for $k \geq 3$ both strategies are on the optimal orbit and contribute the same cost as $\beta_N(k-1) = \beta_N(k)$ for all $k \in 2\mathbb{N}$ and the horizon length is effectively always odd as for even N we have $\beta_N(N-1) = 0$. Waiting one step has a cost of $1 + \frac{N-2}{N}$ in these first 3 steps and is hence the optimal strategy for all $N \in \mathbb{N}$ as going directly to the orbit costs $1 + 1.5 \frac{N-2}{N}$. As MPC applies only the first step of the optimal strategy it ends up waiting forever. When using a uniform discount, this problem cannot occur.

The following assumption confines the choice of the discount function β . This is required to guarantee practical asymptotic stability of the discounted E-MPC.

Assumption 15.6 (*Discount function*) The discount function $\beta : [0, 1] \rightarrow [0, 1]$ is non-increasing, piece-wise continuously differentiable, Lipschitz continuous with Lipschitz constant L_β and satisfies $\beta(0) = 1$, $\beta(1) = 0$, and $\liminf_{\xi \rightarrow 1} \beta'(\xi) < 0$.

The technical assumption $\liminf_{\xi \rightarrow 1} \beta'(\xi) < 0$ is made to simplify our proofs and does not pose a relevant constraint in practice.

Example 15.2 (*Discount functions*) Discount functions that satisfy Assumption 15.6 are, for example, the linear discount $\beta^{\text{lin}}(\xi) = 1 - \xi$ from [17], the half-undiscounted-half-linear discount $\beta^{\text{half-lin}}(\xi) = \min\{1, 2 - 2\xi\}$, the polynomial discount $\beta^{\text{poly}}(\xi) = 1 - \xi^q$ with degree $q \in \mathbb{N}$, and many more. The undiscounted discount $\beta^{\text{un}}(\xi) = 1$ for all $\xi \in [0, 1]$ does not satisfy $\beta(1) = 0$.

15.3 Main Result

The main result of this work is that a discounted E-MPC scheme without terminal conditions asymptotically stabilizes a set that contracts to the optimal orbit Π^* with increasing prediction horizons if the discount satisfies Assumption 15.6.

Theorem 15.1 (Practical asymptotic stability) *Let Assumptions 15.1–15.6 hold. Then, there exists $\varepsilon \in \mathcal{L}$ such that Π^* is practically asymptotically stable w.r.t. $\varepsilon(N)$ under the discounted E-MPC feedback μ_N^β for all $N \in \mathbb{N}$, i.e., there exists $\gamma \in \mathcal{KL}$ such that for all $x \in \mathbb{X}$ and $k \in \mathbb{N}$ we have*

$$\left\| \left(x_{\mu_N^\beta}(k, x), \mu_N^\beta(x_{\mu_N^\beta}(k, x)) \right) \right\|_{\Pi^*} \leq \max \{ \gamma(\|x\|_{\Pi_x^*}, k), \varepsilon(N) \}.$$

Proof Throughout the proof, we assume without loss of generality that ℓ is non-negative (cf. [17, Remark 20]). Analogous to J_N^β , V_N^β , u_x^* , and x_x^* , we define $\tilde{J}_N^\beta$, $\tilde{V}_N^\beta$, $\tilde{u}_x^*$, and $\tilde{x}_x^*$ for the case by replacing ℓ by the rotated stage cost $\tilde{\ell}$ from (15.5). We prove the statement by showing that the rotated value function $\tilde{V}_N^\beta$ is a practical Lyapunov function in the sense of [17, Definition 13]. Showing positive definiteness of $\tilde{\ell}$ and $\tilde{V}_N^\beta$ is not different from the linearly discounted case [17, Lemmas 27, 28] and is therefore not shown again here. The challenging part is to show that there exists $\delta \in \mathcal{L}$ such that the Lyapunov decrease condition

$$\tilde{V}_N^\beta \left(x_{\mu_N^\beta}(1, x) \right) - \tilde{V}_N^\beta(x) \leq -\tilde{\ell}(x, \mu_N^\beta(x)) + \delta(N) \quad (15.8)$$

holds for all $x \in \mathbb{X}$. To establish this inequality, we need the following two lemmas, which are proven in Appendix A.

Lemma 15.1 (Almost optimal candidate solution) *Let Assumptions 15.1–15.6 hold. Then there exists $\delta_2 \in \mathcal{L}$ such that for all $x \in \mathbb{X}$ and $N \in \mathbb{N}$ we have*

$$V_N^\beta(x_x^*(1)) - V_N^\beta(x) \leq -\ell(x, u_x^*(0)) + \ell^* + \delta_2(N). \quad (15.9)$$

Lemma 15.2 (Difference of value functions) *Let Assumptions 15.1–15.6 hold. Then, there exists $\delta_3 \in \mathcal{L}$ such that for all $x, y \in \mathbb{X}$, $N \in \mathbb{N}$ the following inequality holds*

$$\tilde{V}_N^\beta(y) - \tilde{V}_N^\beta(x) \leq \lambda(y) - \lambda(x) + V_N^\beta(y) - V_N^\beta(x) + \delta_3(N). \quad (15.10)$$

These two lemmas yield

$$\begin{aligned} \tilde{V}_N^\beta(x_x^*(1)) - \tilde{V}_N^\beta(x) &\stackrel{(15.10)}{\leq} \lambda(x_x^*(1)) - \lambda(x) + V_N^\beta(x_x^*(1)) - V_N^\beta(x) + \delta_3(N) \\ &\stackrel{(15.9)}{\leq} \lambda(x_x^*(1)) - \lambda(x) - \ell(x, u_x^*(0)) + \ell^* + \delta_2(N) + \delta_3(N) \\ &= -\tilde{\ell}(x, u_x^*(0)) + \delta(N) \end{aligned}$$

with $\delta := \delta_2 + \delta_3 \in \mathcal{L}$. Since $u_x^*(0) = \mu_N^\beta(x)$ and $x_x^*(1) = x_{\mu_N^\beta}(1, x)$, this establishes (15.8) and hence [17, Theorem 14] guarantees practical asymptotic stability with respect to $\varepsilon(N)$ for some $\varepsilon \in \mathcal{L}$. $\square$

Due to Theorem 15.1 we directly obtain that the asymptotic average performance is optimal up to an error vanishing with growing prediction horizons.

Corollary 15.1 (Asymptotic average performance) *Let Assumptions 15.1–15.6 hold. Then there exists $\varepsilon_2 \in \mathcal{L}$ such that the MPC feedback μ_N^β achieves an asymptotic average performance of $J_\infty^{\text{av}}(x, \mu_N^\beta) \leq \ell^* + \varepsilon_2(N)$.*

Proof For given N and x , let $T_1 \in \mathbb{N}$ be large enough such that $\gamma(\|x\|_{\Pi^*}, T_1) \leq \varepsilon(N)$. For all $T \geq T_1$ we split $J_T(x, \mu_N^\beta)$ in the interval $\mathbb{I}_{[0, T_1-1]}$ and the interval $\mathbb{I}_{[T_1, T-1]}$ in which $(x_x^*(k), u_x^*(k))$ is inside the $\varepsilon(N)$ neighborhood of Π^* due to Theorem 15.1. We use $\bar{\ell}$ to bound the first interval as well as $\varepsilon_2 \in \mathcal{L}$ and the bound from [17, Lemma 24] to bound the second interval, i.e.:

$$\frac{1}{T} J_T(x, \mu_N^\beta) \leq \frac{T_1}{T} \bar{\ell} + \frac{T - T_1 + p^* - 1}{T} \ell^* + \varepsilon_2(N).$$

Now, apply $\limsup_{T \rightarrow \infty}$ to both sides to obtain the desired result. $\square$

Remark 15.2 In [17], the same asymptotic average performance guarantee was given without assuming minimality of Π^* and continuity of λ . This required an individual proof, as it could not be stated as corollary of the practical asymptotic stability result, which assumed minimality of Π^* and continuity of λ . To keep this paper concise, we did not present such a proof but we conjecture that the performance bound could similarly be established without these assumptions.

15.4 Numerical Example

In this section, we present two numerical examples, compare different discount functions, and compare different E-MPC schemes for periodic operation.

Example 15.3 (Periodic orbit) Consider the scalar system $x(t+1) = -x(t) + u(t)$ equipped with the stage cost $\ell(x, u) = x^3$ and the constraint sets $\mathbb{X} = [-1, 1]$

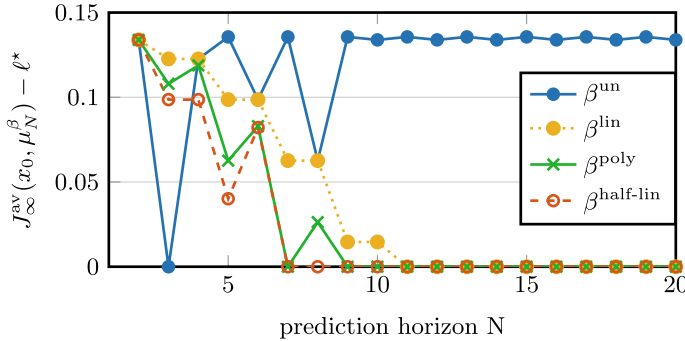


Fig. 15.1 Performance comparison for different discounts in Example 15.3

and $\mathbb{U} = [-0.1, 0.1]$. The system is optimally operated at the 2-periodic orbit $\Pi^*(0) = (-1, -0.1)$, $\Pi^*(1) = (0.9, -0.1)$ with the average cost $\ell^* = \frac{1}{2}(0.9^3 - 1) \approx -0.136$. In Fig. 15.1, we compare the asymptotic average performance of the proposed discounted E-MPC scheme, starting from the initial condition $x_0 = 0.05$ for the different discount functions $\beta \in \{\beta^{\text{lin}}, \beta^{\text{half-lin}}, \beta^{\text{un}}, \beta^{\text{poly}}\}$ from Example 15.2, where we chose degree $q = 2$ for the polynomial discount. We observe that the undiscounted scheme does not converge to the optimal average performance for all prediction horizon lengths $N \neq 3$. For all odd prediction horizons $N \geq 9$, the scheme gets stuck in the steady state $(0.05, 0.1)$ and for all even prediction horizons $N \geq 10$ the scheme gets stuck in the periodic orbit $\Pi_s(0) = (0.05, -0.1)$, $\Pi_s(1) = (-0.15, -0.1)$. This failure is not surprising, as β^{un} does not satisfy Assumption 15.6, and hence our theoretical guarantees do not hold for β^{un} . Among the other discounts, which satisfy Assumption 15.6, we can see that $\beta^{\text{half-lin}}$ performs best and achieves optimal performance for all $N \geq 7$, with β^{poly} we need $N \geq 9$, and with β^{lin} we need $N \geq 11$.

For comparison, we take a look at the p^* -step E-MPC scheme from [14], which converges to the optimal orbit for all prediction horizon lengths $N \geq 3$ in this example. Hence, in this aspect, it is better than the three discounted E-MPC schemes, however, it needs to know the optimal period length p^* in advance, provides only convergence but no stability guarantees, and as discussed in [17, Example 18] may lead to suboptimal transient performance.

Moreover, we compare to an E-MPC with terminal equality constraints (a special case of [16]), i.e., at time t we solve (15.7) with $\beta = \beta^{\text{un}}$ and the additional constraint that $x_u(N, x) = \Pi_{\mathbb{X}}^*([\phi_{\text{end}} + t]_2)$ with phase $\phi_{\text{end}} \in \{0, 1\}$. We denote the resulting time-dependent MPC feedback by $\mu_N^{\text{TC}}(x, t)$. If we start at $t = 0$ at $x_0 = \Pi_{\mathbb{X}}^*(\phi_0)$ with phase ϕ_0 , then either $[\phi_0 + N]_2 = \phi_{\text{end}}$ or $[\phi_0 + N]_2 \neq \phi_{\text{end}}$. In the first case, the OCP is feasible for all horizon lengths $N \geq 1$ and the closed loop follows the optimal orbit Π^* . In the second case, however, we need $N \geq 20$ for feasibility in order to leave Π^* and approach it with the synchronized phase again, which additionally leads to a rather high transient cost. For example, starting at $\phi_0 = 0$, i.e., $x_0 = -1$, choosing

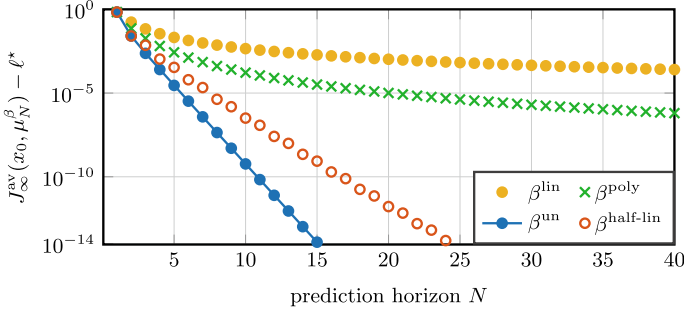


Fig. 15.2 Performance comparison for different discounts in Example 15.4

$N = 20$ and $\phi_{\text{end}} = 1$, leads to $J_{20}(x_0, \mu_{20}^{\text{TC}}) = -1$ and $J_{21}(x, \mu_{20}^{\text{TC}}) = -0.271$ in closed loop, whereas $J_{20}(x_0, \mu_7^{\beta^{\text{half-lin}}}) = -2.71$ and $J_{21}(x, \mu_7^{\beta^{\text{half-lin}}}) = -3.71$.

Example 15.4 (Economic growth) Consider the scalar system $x(t+1) = u(t)$, which is equipped with the stage cost $\ell(x, u) = -\log(5x^{0.34} - u)$, constrained to the sets $\mathbb{X} = [0, 10]$, $\mathbb{U} = [0.1, 10]$, and initialized at $x(0) = 0.1$. It is known that the optimal cost $\ell^* \approx -1.47$ is achieved at the steady state $\Pi^* \approx (2.23, 2.23)$, i.e., we have an optimal orbit of length $p^* = 1$ (compare [8, Example 5.1]). Therefore, although β^{un} does not satisfy Assumption 15.6, we still have guaranteed practical stability from [8]. In Fig. 15.2, we compare the asymptotic average performance for the discount functions β^{un} , β^{lin} , $\beta^{\text{half-lin}}$, and β^{poly} with degree $q = 2$. As known from [8] and [17, Example 19], we can see that the performance converges exponentially in N for β^{un} and with the approximate rate $\sim \frac{1}{N^2}$ for β^{lin} . Further, we can see that $\beta^{\text{half-lin}}$ is able to restore the exponential convergence rate, and thus significantly outperforms β^{lin} and β^{poly} . For example, for an error of less than 10^{-10} we need $N \geq 11$ with β^{un} , $N \geq 17$ with $\beta^{\text{half-lin}}$ and $N \geq 6 \cdot 10^4$ for β^{lin} (compare [17]).

15.5 Conclusion and Outlook

We have presented in this paper a discounted E-MPC scheme without terminal conditions for nonlinear systems which behave optimally on a periodic orbit. We provided requirements on discount functions such that practical asymptotic stability and asymptotic average performance can be guaranteed. As seen in Example 15.4, the quantitative performance level can vary strongly for different discounts and the performance of the linear discount from [17] can be outperformed significantly. Based on the findings in our numerical examples we conjecture that $\beta^{\text{half-lin}}$ is generally a good choice when dealing with periodic orbits, i.e., linearly discounting only the second half of the prediction horizon. When dealing with steady states (optimal period length $p^* = 1$), we found in our examples no advantage of using discounts instead

of the standard undiscounted E-MPC scheme. However, a more extensive numerical study is required to draw conclusions about which discount is best in which situation. Additionally, it is interesting to theoretically investigate under which assumption on the discount function an exponential convergence rate is guaranteed, as is the case for optimal steady-state operation without discounting [8].

A Proof of Lemmas 15.1 and 15.2

In order to prove Lemmas 15.1 and 15.2, we first show in Lemma 15.3 that the weak turnpike property (cf. [17, Definition 7]) holds and we use it in Lemma 15.4 to compute the average of a function over turnpike trajectories.

Lemma 15.3 (Weak turnpike property) *Let Assumptions 15.1–15.6 hold. We define the number of points of the optimal trajectory u_x^* in an ε -neighborhood of the optimal orbit Π^* as $Q_\varepsilon^\beta(N, x) := \#\{k \in \mathbb{I}_{[0, N-1]} \mid \| (x_x^*(k), u_x^*(k)) \|_{\Pi^*} \leq \varepsilon\}$. Then, there exists $\alpha \in \mathcal{K}_\infty$ such that for all $x \in \mathbb{X}$, $N \in \mathbb{N}$, and $\varepsilon > 0$ we have*

$$Q_\varepsilon^\beta(N, x) \geq N - \frac{\sqrt{N}}{\alpha(\varepsilon)}. \quad (15.11)$$

Proof First, let us establish an upper bound on the rotated cost functional $\tilde{J}_N^\beta(x, u_x^*) = \sum_{k=0}^{N-1} \beta\left(\frac{k}{N}\right) \tilde{\ell}(x_x^*(k), u_x^*(k))$ starting with

$$\tilde{J}_N^\beta(x, u_x^*) \stackrel{(15.5)}{=} V_N^\beta(x) - \sum_{k=0}^{N-1} \beta\left(\frac{k}{N}\right) \ell^* + \lambda(x) - \sum_{k=1}^N \left(\beta\left(\frac{k-1}{N}\right) - \beta\left(\frac{k}{N}\right) \right) \lambda(x_x^*(k)).$$

Due to Assumptions 15.2, 15.3, and 15.6 we have

$$- \sum_{k=1}^N \left(\beta\left(\frac{k-1}{N}\right) - \beta\left(\frac{k}{N}\right) \right) \lambda(x_x^*(k)) \leq \sum_{k=1}^N \left(\beta\left(\frac{k-1}{N}\right) - \beta\left(\frac{k}{N}\right) \right) \bar{\lambda} = \underbrace{(\beta(0) - \beta(1))}_{=1} \bar{\lambda}.$$

Due to Assumptions 15.4 and 15.5 we can reach the optimal orbit $\Pi_{\mathbb{X}}^*$ from x in at most $M_3 = M_1 + M_2$ steps. Therefore, Lemma 22 from [17] yields

$$V_N^\beta(x) - \sum_{k=0}^{N-1} \beta\left(\frac{k}{N}\right) \ell^* + \lambda(x) + \bar{\lambda} \leq M_3(\bar{\ell} - \ell^*) + 2\bar{\lambda} + \ell^* p^* =: C.$$

Although this lemma was only stated for linear discounts, it is straightforward to see that the proof is valid for general β satisfying Assumption 15.6 as only the fact that β is non-increasing and $\beta(1) = 0$, $\beta(0) = 1$ is used. Furthermore, due to Assumption 15.6 we know that $\liminf_{\xi \rightarrow 1} \beta'(\xi) < 0$ and β is non-increasing, hence there must exist a constant $c > 0$ such that β is lower bounded by the scaled linear discount $\beta(\xi) \geq c\beta^{\text{lin}}(\xi)$ for all $\xi \in [0, 1]$. Hence, as $\bar{\ell}$ is non-negative, we obtain

$\tilde{J}_N^{\beta^{\text{lin}}}(x, u_x^*) \leq c^{-1} \tilde{J}_N^\beta(x, u_x^*) \leq c^{-1} C$. Therefore, we can apply Lemma 23 from [17], which guarantees that the weak turnpike property holds with $\alpha(\varepsilon) = \sqrt{\frac{c}{2C} \underline{\alpha}_{\tilde{\ell}}(\varepsilon)}$, $\alpha \in \mathcal{K}_\infty$. $\square$

Lemma 15.4 (Average over turnpike trajectories) *Let Assumptions 15.1–15.6 hold and let $g : \mathbb{X} \times \mathbb{U} \rightarrow \mathbb{R}$ be a continuous function and let $g^* := \frac{1}{p^*} \sum_{k=0}^{p^*-1} g(\Pi^*(k))$, then there exists $\delta_1 \in \mathcal{L}$ such that for all $x \in \mathbb{X}$*

$$\left| g^* - \sum_{k=1}^{N-1} \left(\beta \left(\frac{k-1}{N} \right) - \beta \left(\frac{k}{N} \right) \right) g(x_x^*(k), u_x^*(k)) \right| \leq \delta_1(N). \quad (15.12)$$

Proof Due to Lemma 15.3 the weak turnpike property (15.11) holds, which yields with $\varepsilon = \sigma(N) := \alpha^{-1}(\sqrt[4]{N})$, $\sigma \in \mathcal{L}$ that for all $N \in \mathbb{N}$ and $x \in \mathbb{X}$ we have

$$Q_{\sigma(N)}^\beta(N, x) \geq N - \sqrt[4]{N^3}. \quad (15.13)$$

Hence, there are at most $\sqrt[4]{N^3}$ points of (x_x^*, u_x^*) outside the $\sigma(N)$ -neighborhood of Π^* . Denote the left-hand side of (15.12) by Σ_1 and let us split the sum in Σ_1 over $k \in \mathbb{I}_{[1, N-1]}$ into pieces $k \in \mathcal{I}_j = \mathbb{I}_{[jp^*+1, (j+1)p^*]}$ of length p^* for $j \in \mathbb{I}_{[0, \lfloor (N-1)/p^* \rfloor - 1]}$ and let $\mathcal{I}_{\lfloor (N-1)/p^* \rfloor} = \mathbb{I}_{[\lfloor (N-1)/p^* \rfloor p^*, N-1]}$ contain the remainder. Let $\mathcal{J}$ be the set of indices j for which the piece $\mathcal{I}_j$ is not fully inside the $\sigma(N)$ -neighborhood of Π^* and of the remainder piece, hence $\#\mathcal{J} \leq \sqrt[4]{N^3} + 1$. Now, using Lipschitz continuity $|\beta(\frac{k-1}{N}) - \beta(\frac{k}{N})| \leq \frac{L_\beta}{N}$, we bound all pieces in $\mathcal{J}$ by

$$\left| \sum_{j \in \mathcal{J}} \sum_{k \in \mathcal{I}_j} \left(\beta \left(\frac{k-1}{N} \right) - \beta \left(\frac{k}{N} \right) \right) g(x_x^*(k), u_x^*(k)) \right| \leq (\sqrt[4]{N^3} + 1) p^* \bar{g} \frac{L_\beta}{N} =: \delta_4(N),$$

where continuity of g and compactness of $\mathbb{X} \times \mathbb{U}$ imply $\bar{g} = \max_{(x, u) \in \mathbb{X} \times \mathbb{U}} |g(x, u)|$ exists and is finite. Note that $\delta_4 \in \mathcal{L}$. Further, with the same reasons there exists $\bar{\alpha}_g \in \mathcal{K}_\infty$ that satisfies $|g(x, u) - g(\Pi^*(k))| \leq \bar{\alpha}_g(\|(x, u) - \Pi^*(k)\|)$ for all $k \in \mathbb{I}_{[0, p^*-1]}$ and all $(x, u) \in \mathbb{X} \times \mathbb{U}$. Next, define the complement of $\mathcal{J}$ by $\tilde{\mathcal{J}} := \mathbb{I}_{[0, \lfloor (N-1)/p^* \rfloor]} \setminus \mathcal{J}$, and compute

$$\begin{aligned} \Sigma_1 &\leq \left| g^* - \sum_{j \in \tilde{\mathcal{J}}} \sum_{k \in \mathcal{I}_j} \left(\beta \left(\frac{k-1}{N} \right) - \beta \left(\frac{k}{N} \right) \right) g(x_x^*(k), u_x^*(k)) \right| + \delta_4(N) \\ &\leq \left| g^* - \sum_{j \in \tilde{\mathcal{J}}} \sum_{k \in \mathcal{I}_j} \left(\beta \left(\frac{k-1}{N} \right) - \beta \left(\frac{k}{N} \right) \right) g(\Pi^*(i_k)) \right| + \underbrace{L_\beta \bar{\alpha}_g(\sigma(N))}_{=: \delta_5(N)} + \delta_4(N), \end{aligned}$$

where $i_k \in \mathbb{I}_{[0, p^*-1]}$ is such that $\|\Pi^*(i_k) - (x_x^*(k), u_x^*(k))\| \leq \sigma(N)$ and $\delta_5 \in \mathcal{L}$. First, we consider $N \geq N_0$ and choose $N_0 \in \mathbb{N}$ large enough such that $\sigma(N_0) \leq \bar{\varepsilon}$ with $\bar{\varepsilon}$ from [14, Lemma 15]. Then this lemma guarantees that the points in $\mathcal{I}_j$ follow the orbit, since they are inside the $\sigma(N)$ -neighborhood of Π^* , i.e., $i_{k+1} = [i_k + 1]_{p^*}$ for all $k \in \mathcal{I}_j$. Hence, the p^* elements of $\{i_k | k \in \mathcal{I}_j\}$ are exactly $[0, p^* - 1]$ and

for each $p \in [0, p^* - 1]$ and $j \in \bar{\mathcal{J}}$ there exists a $k_{p,j} \in \mathcal{I}_j$ such that $i_{k_{p,j}} = p$. Therefore, we can continue as follows:

$$\begin{aligned} \Sigma_1 &\leq \left| \sum_{p=0}^{p^*-1} g(\Pi^*(p)) \left(\frac{1}{p^*} - \sum_{j \in \bar{\mathcal{J}}} \left(\beta \left(\frac{k_{p,j} - 1}{N} \right) - \beta \left(\frac{k_{p,j}}{N} \right) \right) \right) \right| + \delta_5(N) \\ &\leq \sum_{p=0}^{p^*-1} |g(\Pi^*(p))| \left| \frac{1}{p^*} - \sum_{j \in \bar{\mathcal{J}}} \left(\beta \left(\frac{k_{p,j} - 1}{N} \right) - \beta \left(\frac{k_{p,j}}{N} \right) \right) \right| + \delta_5(N). \end{aligned} \quad (15.14)$$

Since β is piece-wise continuously differentiable, we can use the mean value theorem (on all differentiable pieces separately and taking the infimum) to obtain

$$\sum_{j \in \bar{\mathcal{J}}} \left(\beta \left(\frac{k_{p,j} - 1}{N} \right) - \beta \left(\frac{k_{p,j}}{N} \right) \right) \leq \sum_{j \in \bar{\mathcal{J}}} -\frac{1}{N} \inf_{\xi \in \left[\frac{k_{p,j}-1}{N}, \frac{k_{p,j}}{N} \right]} \beta'(\xi),$$

where in the inf we ignore the finitely many values ξ in the interval at which β' does not exist. Further, since $\left[\frac{k_{p,j}-1}{N}, \frac{k_{p,j}}{N} \right] \subseteq [\xi_j, \xi_{j+1}]$ with $\xi_j := \frac{j p^*}{N}$ for $j \in \mathbb{I}_{[0, \lfloor (N-1)/p^* \rfloor]}$ and $\xi_{\lfloor (N-1)/p^* \rfloor + 1} := 1$, we can continue

$$\sum_{j \in \bar{\mathcal{J}}} -\frac{1}{N} \inf_{\xi \in \left[\frac{k_{p,j}-1}{N}, \frac{k_{p,j}}{N} \right]} \beta'(\xi) \leq - \sum_{j=0}^{\lfloor (N-1)/p^* \rfloor} \frac{\xi_{j+1} - \xi_j}{p^*} \inf_{\xi \in [\xi_j, \xi_{j+1}]} \beta'(\xi),$$

where we used that $\beta'(\xi) \leq 0$ as β is non-increasing. The partition of the interval $[0, 1]$ defined by ξ_j satisfies $\xi_{j+1} - \xi_j \leq \frac{p^*}{N} \rightarrow 0$ as $N \rightarrow \infty$ and thus the sum converges to the Riemann integral of β' as $N \rightarrow \infty$, i.e., there exists $\delta_6 \in \mathcal{L}$ such that

$$- \sum_{j=0}^{\lfloor (N-1)/p^* \rfloor} \frac{\xi_{j+1} - \xi_j}{p^*} \inf_{\xi \in [\xi_j, \xi_{j+1}]} \beta'(\xi) \leq -\frac{1}{p^*} \int_0^1 \beta'(\xi) d\xi + \delta_6(N) = \frac{1}{p^*} + \delta_6(N),$$

where we used $-\int_0^1 \beta'(\xi) d\xi = \beta(0) - \beta(1) = 1$. With similar arguments we can construct a lower bound and obtain

$$\frac{1}{p^*} - \delta_6(N) \leq \sum_{j \in \bar{\mathcal{J}}} \left(\beta \left(\frac{k_{p,j} - 1}{N} \right) - \beta \left(\frac{k_{p,j}}{N} \right) \right) \leq \frac{1}{p^*} + \delta_6(N).$$

Using this in (15.14) and defining $\delta_1(N) := \delta_5(N) + \sum_{p=0}^{p^*-1} |g(\Pi^*(p))| (\delta_6(N))$, which satisfies $\delta_1 \in \mathcal{L}$, we obtain the desired result for $N \geq N_0$. For $N < N_0$ we

know that $\Sigma_1 \leq |g^*| + \bar{g}$ and thus by redefining $\delta_1(N) := \max\{\delta_1(N_0), |g^*| + \bar{g}\}$ for $N < N_0$ the result follows for all $N \in \mathbb{N}$ as still $\delta_1 \in \mathcal{L}$. $\square$

Proof (Proof of Lemma 15.1) To begin with, we construct a candidate solution for $V_N^\beta(x_x^*(1))$ based on u_x^* . Therefore, we use the weak turnpike property and $\sigma(N) = \alpha^{-1}(\sqrt[4]{N})$ to obtain (15.13). Let $P \in \mathbb{I}_{[0, N-1]}$ be the largest time point where (x_x^*, u_x^*) is in the $\sigma(N)$ -neighborhood of Π^* , then (15.13) implies $P \geq N - \sqrt[4]{N^3} - 1$. First, we consider only $N \geq N_0$ and choose $N_0 \in \mathbb{N}$ large enough such that $P \geq 1$ and $\sigma(N) \leq \kappa$ for all $N \geq N_0$ with κ from Assumption 15.4. Hence, at P the point $x_x^*(P)$ is in the local controllability neighborhood of $\Pi_{\mathbb{X}}^*$ such that we know that there exists $u_1 \in \mathbb{U}^{M_1}(x_x^*(P))$ with $x_{u_1}(M_1, x_x^*(P)) = x_x^*(P)$. Thus, for $V_N^\beta(x_x^*(1))$ we can construct a candidate solution $\bar{u} \in \mathbb{U}^N(x_x^*(1))$ and the resulting state trajectory $x_{\bar{u}}$ based on the solution for $V_N^\beta(x)$ as follows:

$$\bar{u}(k) = \begin{cases} u_x^*(k+1) & \text{for } k \in \mathbb{I}_{[0, P-2]} \\ u_1(k-P+1) & \text{for } k \in \mathbb{I}_{[P-1, P+M_1-2]} \\ u_x^*(k-M_1+1) & \text{for } k \in \mathbb{I}_{[P+M_1-1, N-1]}. \end{cases} \quad (15.15)$$

$$x_{\bar{u}}(k, x_x^*(1)) = \begin{cases} x_x^*(k+1) & \text{for } k \in \mathbb{I}_{[0, P-2]} \\ x_{u_1}(k-P+1, x_x^*(P)) & \text{for } k \in \mathbb{I}_{[P-1, P+M_1-2]} \\ x_x^*(k-M_1+1) & \text{for } k \in \mathbb{I}_{[P+M_1-1, N-1]}. \end{cases} \quad (15.16)$$

With this candidate solution we can upper bound $V_N^\beta(x_x^*(1)) \leq J_N^\beta(x_x^*(1), \bar{u}) = \Sigma_2 + \Sigma_3 + \Sigma_4$ where $\Sigma_2, \Sigma_3, \Sigma_4$ sum up the three pieces in (15.15) and (15.16)

$$\begin{aligned} \Sigma_2 &:= \sum_{k=0}^{P-2} \beta\left(\frac{k}{N}\right) \ell(x_x^*(k+1), u_x^*(k+1)) \\ \Sigma_3 &:= \sum_{k=P-1}^{P+M_1-2} \beta\left(\frac{k}{N}\right) \ell(x_{u_1}(k-P+1, x_x^*(P)), u_1(k-P+1)) \\ \Sigma_4 &:= \sum_{k=P+M_1-1}^{N-1} \beta\left(\frac{k+M_1}{N}\right) \ell(x_x^*(k+1), u_x^*(k+1)). \end{aligned}$$

We start to bound Σ_3 as follows:

$$\Sigma_3 \leq M_1 \bar{\ell} \beta\left(\frac{P-1}{N}\right) \leq M_1 \bar{\ell} \beta\left(\frac{N - \sqrt[4]{N^3} - 2}{N}\right) = M_1 \bar{\ell} \beta\left(1 - \frac{1}{\sqrt[4]{N}} - \frac{2}{N}\right) =: \delta_7(N),$$

where $\delta_7 \in \mathcal{L}$ due to $\beta(1) = 0$ and β being continuous. Further, as β is non-increasing and β, ℓ are non-negative, we can bound $\Sigma_2 + \Sigma_4$ as follows:

$$\begin{aligned}
\Sigma_2 + \Sigma_4 &\leq \sum_{k=1}^{N-1} \beta \left(\frac{k-1}{N} \right) \ell(x_x^*(k), u_x^*(k)) \\
&= V_N^\beta(x) - \ell(x, u_x^*(0)) + \sum_{k=1}^{N-1} \left(\beta \left(\frac{k-1}{N} \right) - \beta \left(\frac{k}{N} \right) \right) \ell(x_x^*(k), u_x^*(k)) \\
&\stackrel{\text{Lemma 15.4}}{\leq} V_N^\beta(x) - \ell(x, u_x^*(0)) + \ell^* + \delta_1(N)
\end{aligned}$$

for some $\delta_1 \in \mathcal{L}$. Define $\delta_2(N) := \delta_1(N) + \delta_7(N)$ then the desired inequality (15.9) follows for $N \geq N_0$. For $N < N_0$, we know that (15.9) holds with $\delta_2(N) := \max\{(N_0 + 1)\bar{\ell}, \delta_2(N_0)\}$ and since this guarantees $\delta_2 \in \mathcal{L}$, the desired result follows for all $N \in \mathbb{N}$. $\square$

Proof (Proof of Lemma 15.2) This lemma is the generalization of [17, Lemma 29] from linear discounts to discounts that satisfy Assumption 15.6. Also the proof follows the same line of arguments. First, note that if we replace the stage cost ℓ with the rotated stage cost $\tilde{\ell}$ from (15.5), then Assumptions 15.2–15.6 still hold and thus Lemma 15.3 applies and solutions of $\tilde{V}_N^\beta$ satisfy the weak turnpike property as well. The weak turnpike property (15.11) for $V_N^\beta(x)$, $V_N^\beta(y)$, $\tilde{V}_N^\beta(x)$, and $\tilde{V}_N^\beta(y)$ guarantees that there are at least $N - \frac{4\sqrt{N}}{\alpha(\varepsilon)}$ time points $k \in \mathbb{I}_{[0, N-1]}$ where these four trajectories are ε close to Π^* . Choosing $\varepsilon = \sigma_2(N) := \alpha^{-1}(4\sqrt[4]{N})$ yields that there are at least $N - \sqrt[4]{N^3}$ such points. We denote the largest one by $P \geq N - \sqrt[4]{N^3} - 1$ and choose N_0 large enough such that $P \geq 1$ and $\sigma_2(N) \leq \kappa$ for $N \geq N_0$. First, consider only $N \geq N_0$, then we use Assumption 15.4 to construct a candidate solution $\bar{u} \in \mathbb{U}^N(y)$ for $\tilde{V}_N^\beta(y)$ which consists of the first P steps of the optimal solution of $V_N^\beta(y)$ and goes afterward in M_1 steps to Π^* using the local controllability and then remains there. Using this candidate solution, we can upper bound

$$\tilde{V}_N^\beta(y) \leq \tilde{J}_N^\beta(y, \bar{u}) \leq \sum_{k=0}^{P-1} \beta \left(\frac{k}{N} \right) \tilde{\ell}(x_y^*(k), u_y^*(k)) + \delta_8(N), \quad (15.17)$$

where we used $\tilde{\ell}(\Pi^*(k)) = 0$ (see [17, Lemma 27]) and (15.6) from Assumption 15.4 to compute $\delta_8(N) := M_1 \bar{\alpha}_{\tilde{\ell}}(\alpha_c(\sigma_2(N)))$, which is $\delta_8 \in \mathcal{L}$. Furthermore, non-negativity of $\tilde{\ell}$ implies

$$\tilde{V}_N^\beta(x) \geq \sum_{k=0}^{P-1} \beta \left(\frac{k}{N} \right) \tilde{\ell}(\tilde{x}_x^*(k), \tilde{u}_x^*(k)). \quad (15.18)$$

Therefore, using the definition of the rotated cost (15.5), we have

$$\begin{aligned}
\tilde{V}_N^\beta(y) - \tilde{V}_N^\beta(x) &\leq \delta_8(N) + \underbrace{\sum_{k=0}^{P-1} \beta \left(\frac{k}{N} \right) \left(\ell(x_y^*(k), u_y^*(k)) - \ell(\tilde{x}_x^*(k), \tilde{u}_x^*(k)) \right)}_{=: \Sigma_5}
\end{aligned}$$

$$\begin{aligned}
& + \lambda(y) - \lambda(x) + \underbrace{\sum_{k=1}^{P-1} \left(\beta \left(\frac{k-1}{N} \right) - \beta \left(\frac{k}{N} \right) \right) \left(\lambda(\tilde{x}_x^*(k)) - \lambda(x_y^*(k)) \right)}_{=: \Sigma_6} \\
& + \beta \left(\frac{P-1}{N} \right) \left(\lambda(\tilde{x}_x^*(P)) - \lambda(x_y^*(P)) \right). \tag{15.19}
\end{aligned}$$

Let us investigate these terms further starting with

$$\beta \left(\frac{P-1}{N} \right) \left(\lambda(\tilde{x}_x^*(P)) - \lambda(x_y^*(P)) \right) \leq 2\bar{\lambda}\beta \left(\frac{P-1}{N} \right) \leq 2\bar{\lambda}\beta \left(1 - \frac{1}{\sqrt[4]{N}} - \frac{2}{N} \right)$$

and define $\delta_9(N) = 2\bar{\lambda}\beta \left(1 - \frac{1}{\sqrt[4]{N}} - \frac{2}{N} \right)$ and note that $\delta_9 \in \mathcal{L}$ as $\beta(1) = 0$ and β is continuous. Now let us turn to Σ_6 and use Lemma 15.4 to obtain that there exists $\delta_{10} \in \mathcal{L}$ with $\Sigma_6 \leq \delta_{10}(N)$, where the assumptions are satisfied since λ is continuous and the weak turnpike property holds not only for x_y^*, u_y^* but also for $\tilde{x}_x^*, \tilde{u}_x^*$. The fact that the sum in Σ_6 goes only until $P-1$ and not until $N-1$ just implies that we get another $\frac{L_\beta}{N}(\sqrt[4]{N^3} + 1)2\bar{\lambda} \in \mathcal{L}$ on top of δ_{10} , as $N - P \leq \sqrt[4]{N^3} + 1$. Next, we consider the part Σ_5 and construct a candidate solution $\bar{u} \in \mathbb{U}^N(x)$ for $V_N^\beta(x)$ which consists of the first P steps of the optimal solution $\tilde{u}_x^*$ of $\tilde{V}_N^\beta(x)$ and afterward uses the local controllability to go to $x_{u_y^*}(P, y)$ in $K \in \mathbb{I}_{[M_1, M_1 + p^* - 1]}$ steps¹ and then applies $u_y^*(k)$ starting at $k = P$ to remain on $V_N^\beta(y)$. Hence,

$$\begin{aligned}
\Sigma_5 &= \sum_{k=0}^{P-1} \beta \left(\frac{k}{N} \right) \ell(x_y^*(k), u_y^*(k)) - J_N^\beta(x, \bar{u}) + \sum_{k=P}^{P+K-1} \beta \left(\frac{k}{N} \right) \ell(x_{\bar{u}}(k, x), \bar{u}(k)) \\
&+ \sum_{k=P}^{N-K-1} \beta \left(\frac{k+K}{N} \right) \ell(x_y^*(k), u_y^*(k)) \\
&\leq V_N^\beta(y) - V_N^\beta(x) + \beta \left(\frac{P}{N} \right) K \bar{\ell},
\end{aligned}$$

where the last inequality holds as ℓ and β are non-negative and β is non-increasing. As $P \geq N - \sqrt[4]{N^3} - 1$, we know that $\beta \left(\frac{P}{N} \right) K \bar{\ell} \leq \beta \left(\frac{N - \sqrt[4]{N^3} - 1}{N} \right) K \bar{\ell} =: \delta_{12}(N)$ where $\delta_{12} \in \mathcal{L}$ as β is continuous and $\beta(1) = 0$. Finally, define $\delta_3(N) := \delta_9(N) + \delta_{10}(N) + \delta_{12}(N)$ for $N \geq N_0$ and plug the bounds on Σ_5 and Σ_6 into (15.19) to obtain the desired result for $N \geq N_0$. As (15.10) holds with $\delta_3(N) := \max\{\delta_3(N_0), N_0(2\bar{\ell} + 4\bar{\lambda}) + 2\bar{\lambda}\}$ for $N < N_0$, which guarantees $\delta_3 \in \mathcal{L}$, the desired result follows for all $N \in \mathbb{N}$. $\square$

¹ Since $\tilde{x}_x^*(P)$ and $x_y^*(P)$ are both in the $\sigma_2(N)$ neighborhood of $\Pi_{\bar{x}}^*$, but not necessarily in the $\sigma(N)$ -neighborhood of the same $\Pi_{\bar{x}}^*(j)$, we need at most $p^* - 1$ additional steps to have the same phase.

References

1. Amrit, R., Rawlings, J.B., Angeli, D.: Economic optimization using model predictive control with a terminal cost. *Ann. Rev. Control* **35**, 178–186 (2011). <https://doi.org/10.1016/j.arcontrol.2011.10.011>
2. Angeli, D., Amrit, R., Rawlings, J.B.: On average performance and stability of economic model predictive control. *IEEE Trans. Automat. Control* **57**, 1615–1626 (2011). <https://doi.org/10.1109/TAC.2011.2179349>
3. Dong, Z., Angeli, D.: Analysis of economic model predictive control with terminal penalty functions on generalized optimal regimes of operation. *Int. J. Robust Nonlinear Control* **28**(16), 4790–4815 (2018). <https://doi.org/10.1002/rnc.4283>
4. Ellis, M., Durand, H., Christofides, P.D.: A tutorial review of economic model predictive control methods. *J. Process Control* **24**, 1156–1178 (2014). <https://doi.org/10.1016/j.jprocont.2014.03.010>
5. Faulwasser, T., Grüne, L., Müller, M.A.: Economic nonlinear model predictive control. *Found. Trends Syst. Control* **5**, 1–98 (2018). <https://doi.org/10.1561/26000000014>
6. Gaitsgory, V., Grüne, L., Höger, M., Kellett, C.M., Weller, S.R.: Stabilization of strictly dissipative discrete time systems with discounted optimal control. *Automatica* **93**, 311–320 (2018). <https://doi.org/10.1016/j.automatica.2018.03.076>
7. Grüne, L.: Economic receding horizon control without terminal constraints. *Automatica* **49**, 725–734 (2013). <https://doi.org/10.1016/j.automatica.2012.12.003>
8. Grüne, L., Stieler, M.: Asymptotic stability and transient optimality of economic MPC without terminal conditions. *J. Process Control* **24**, 1187–1196 (2014). <https://doi.org/10.1016/j.jprocont.2014.05.003>
9. Grüne, L., Pannek, J.: *Nonlinear Model Predictive Control*. Springer (2017). <https://doi.org/10.1007/978-3-319-46024-6>
10. Köhler, J., Müller, M.A., Allgöwer, F.: On periodic dissipativity notions in economic model predictive control. *IEEE Control Syst. Lett.* **2**, 501–506 (2018). <https://doi.org/10.1109/LCSYS.2018.2842426>
11. Köhler, J., Müller, M.A., Allgöwer, F.: Analysis and design of model predictive control frameworks for dynamic operation - An overview. *Ann. Rev. Control* **57** (2024). <https://doi.org/10.1016/j.arcontrol.2023.100929>
12. Martin, T., Köhler, P.N., Allgöwer, F.: Dissipativity and economic model predictive control for optimal set operation. In: *Proceedings of the American Control Conference (ACC)*, pp. 1020–1026 (2019). <https://doi.org/10.23919/ACC.2019.8814305>
13. Müller, M.A., Grüne, L., Allgöwer, F.: On the role of dissipativity in economic model predictive control. In: *Proceedings of the 5th IFAC Conference on Nonlinear Model Predictive Control (NMPC)*, pp. 110–116 (2015). <https://doi.org/10.1016/j.ifacol.2015.11.269>
14. Müller, M.A., Grüne, L.: Economic model predictive control without terminal constraints for optimal periodic behavior. *Automatica* **70**, 128–139 (2016). <https://doi.org/10.1016/j.automatica.2016.03.024>
15. Zanon, M., Gros, S.: A new dissipativity condition for asymptotic stability of discounted economic MPC. *Automatica* **141**, 110287 (2022). <https://doi.org/10.1016/j.automatica.2022.110287>
16. Zanon, M., Grüne, L., Diehl, M.: Periodic optimal control, dissipativity and MPC. *IEEE Trans. Automat. Control* **62**, 2943–2949 (2016). <https://doi.org/10.1109/TAC.2016.2601881>
17. Schwenkel, L., Hadorn, A., Müller, M.A., Allgöwer, F.: Linearly discounted economic MPC without terminal conditions for periodic optimal operation. *Automatica* **159** (2024). <https://doi.org/10.1016/j.automatica.2023.111393>

Chapter 16

Multi-Parametric Programming with Constraint Relaxation for the Optimal Operation of Micro-grids Integrating Renewables



Federico Bianchi, Alessandro Falsone, and Maria Prandini

Abstract Motivated by the integration of distributed renewable energy sources in the electrical grid, we address the optimal operation of a micro-grid that has to adhere to a daily power exchange profile agreed in advance with the grid, by compensating local deviations that may occur along the day using dispatchable generators, loads, and storage units. We propose a solution that is based on explicit Model Predictive Control (eMPC) with constraint relaxation. More specifically, we formulate the problem as a multi-parametric quadratic program (mp-QP) whose solution is the optimal power to be dispatched expressed as a function of a parameter vector representing the actual power consumption/generation profile, which can then be assessed during the day via reliable predictions. Preferred operating regions for the involved dispatchable units are encoded in the mp-QP by introducing soft constraints, which are relaxed using an exact penalty approach when determining the eMPC solution so that they are violated but only when needed to recover feasibility. Performance of the proposed approach is shown on a self-consumption numerical example.

Keywords Micro-grids · Renewables integration · Multi-parametric programming · Explicit MPC · Exact penalty

This paper is supported by the PRIN PNRR project P2022NB77E “A data-driven cooperative framework for the management of distributed energy and water resources” (CUP: D53D23016100001), funded by the NextGeneration EU program (Mission 4, Component 2, Investment 1.1).

This work has been financed by the Research Fund for the Italian Electrical System under the Three-Year Research Plan 2022-2024 (DM MITE n. 337, 15.09.2022), in compliance with the Decree of April 16th, 2018.

F. Bianchi (✉)

Dipartimento di Tecnologie di Generazione e Materiali, Ricerca sul Sistema Energetico – RSE S.P.A., Milano, Italy
e-mail: federico.bianchi@rse-web.it

A. Falsone · M. Prandini

Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy

16.1 Introduction

The massive diffusion of renewable energy sources (RESs) is significantly reshaping the electrical system which was initially conceived as a centralized network model with unidirectional power flows from large-scale production systems to consumption units. Indeed, the decentralized nature of RESs makes traditional centralized electrical plants less adept at ensuring the secure operation of the electrical system in this evolving model, prompting a policy shift advocating for the adoption of distributed energy resources (DERs). This shift in policy reflects an acknowledgment of the potential of DERs to enhance grid resilience, promote energy independence, and facilitate the integration of RESs into the system. This necessitates defining favorable configurations or architectures of DERs, such as micro-grids (MGs), for effective management and control. Essentially, an MG is a single organized power subsystem comprising multiple distributed generation systems, both renewable (such as photovoltaic, wind power devices) and/or conventional generation (such as internal combustion engines, micro-turbines, and diesel generators), loads, and storages [1, 2]. From a main grid point of view, an MG acts as a single controlled load within the power system, and, also, having the generation situated in close proximity to the load, significantly reduces transmission losses. However, uncertainties related to RESs availability, and load fluctuations can pose challenges to the secure and economical operation of MGs. This asks for suitable daily optimal MG operation scheme, as well as coordination and interaction mechanisms with the main grid [3]. Typically, the MG and the main grid coordinate in advance to establish a daily exchange profile. As a result, each day, the MG strives to maintain adherence to this scheduled profile, in order to avoid incurring imbalance charges from the main grid operator. Within an MG, dispatchable generators and loads, and energy storages can be employed as flexible resources to pursue the fulfillment of scheduled programs, thus coping with the uncertainties and fluctuations of the source and load.

To solve this exchange profile enforcement problem, several control schemes based on Model Predictive Control (MPC) applied at MG level have been proposed. In [4], a hierarchical control scheme is implemented to manage the exchange of flexibility within a hybrid AC/DC grid consisting of multiple MGs. Specifically, an MPC strategy is initially applied to each MG to locally address power management. Then, a coordination mechanism among MGs facilitates flexibility exchange in situations where local power sources cannot adequately compensate for variability. In [5], a two-layer control scheme based on MPC operating at two different timescales is proposed. The high-level optimizer operates at a slow timescale and is responsible for calculating the nominal operating conditions for each component over a long time horizon. The low-level controller operates at a higher frequency and fine-tunes the MG operation to minimize the discrepancy between the planned energy exchange and the actual one. Similarly, in [6], a multi-time scale (day-ahead and intra-day) economic scheduling method combining robust optimization and MPC has been proposed.

In this paper, we firstly frame the problem of adhering to the exchange profile agreed with the grid as an optimization program with quadratic cost and linear constraints where the power to be dispatched is determined by minimizing the operating costs subject to both hard and soft constraints on the dispatchable units. While hard constraints must be enforced to have a solution that is applicable in practice, soft constraints represent preferred operating conditions and can be relaxed if needed to enforce the pre-defined exchange profile with the grid. Based on the evidence that the quadratic program problem depends on several quantities that need to be updated at each time slot t , such as the DERs production, loads consumption, and the state of flexible units, we opt for computing offline the optimal map which provides the power to dispatch as a function of these quantities that are treated as a parameter vector according to the explicit MPC (eMPC) approach. This allows to determine the power to be dispatched by simply evaluating the optimal map for the value taken by the uncertain quantities and without solving online the optimization problem. Differently from standard eMPC (see, e.g., [7]), however, we adopt the approach in [8] where violation of the soft constraints is allowed but subject to a penalization of its 1-norm in the cost through an additive term that is weighted by some coefficient.

Paper [8] fully characterizes the connection between the original constrained problem and its relaxed version as the penalty coefficient varies, proving effective in providing an optimal solution for the original constrained optimization problem within its feasible region, while returning a solution with minimal 1-norm violation of the soft constraints otherwise. Since hard constraints that cannot be relaxed are still enforced in [8], the obtained eMPC solution with relaxed soft constraints is particularly suited for the considered MG operation problem where operative non-relaxable constraints and performance/comfort relaxable requirements co-exist. Performance of the proposed approach is shown on a numerical example where an MG is operating in island mode, i.e., with zero net power exchange with the main grid. The eMPC policy with constraint relaxation derived according to [8] is applied during the reference 1-day time window according to the receding-horizon strategy, using a control horizon of 1-hour over which predictions of DERs production and loads consumption are quite reliable.

The paper is organized as follows. In Sect. 16.2, the MG optimal operation problem is presented together with the proposed solution based on multi-parametric programming with constraint relaxation. Section 16.3 provides a model for dispatchable units. Section 16.4 describes the considered case study and the obtained results. Finally, concluding remarks are drawn in Sect. 16.5.

Notation We denote with $\mathbb{R}$ the set of real numbers, and with $\mathbb{R}_+$ the set of non-negative real numbers. The transpose of a matrix $M \in \mathbb{R}^{n_r \times n_c}$ is denoted with $M^\top$. Inequalities between two vectors u and v , $u \leq v$, have to be intended as component-wise. For a symmetric matrix $M = M^\top$, $M > 0$ ($M \geq 0$) denotes that M is positive (semi-)definite. For a vector $v \in \mathbb{R}^n$, $\|v\|_1$ denotes its 1-norm and $[v]_+ = \max\{v, 0_n\}$, where $0_n \in \mathbb{R}^n$ is the zero vector, has to be intended as the component-wise maximum between the two vectors.

16.2 Optimal MG Operation Problem and Proposed Solution

We consider the operation of an MG over a 1-day horizon divided in time slots of a certain duration Δ . The MG is connected with the grid and is equipped with both dispatchable (e.g., a battery, a thermostatically controlled load, or a generator like a diesel internal combustion engine/microturbine) and non-dispatchable (e.g., a wind turbine, a solar photovoltaic panel installation, a non-flexible load) units. The two sets of units are, respectively, denoted as $\mathcal{D}$ and $\mathcal{N}$. Accordingly, the (average) power $p_{net}(t)$ exchanged by the MG with the grid in time slot t is given by

$$p_{net}(t) = \sum_{i \in \mathcal{D}} p_i(t) + \sum_{i \in \mathcal{N}} p_i(t), \quad (16.1)$$

where $p_i(t)$ denotes the power output of the i -th unit (see Fig. 16.1).

We suppose that the MG and the main grid agreed a daily power exchange profile $\bar{p}(t)$, $t = 1, \dots, N$, with N denoting the number of time slots in a day. The goal of the MG is to enforce the scheduled profile, i.e., $p_{net}(t) = \bar{p}(t)$, $t = 1, \dots, N$, by compensating the possible deviations of the power exchanged by the non-dispatchable units in $\mathcal{N}$ with respect to the profile that was predicted earlier. This entails optimally dispatching power among the flexible units $\mathcal{D}$ within the limits dictated by their operative constraints.

To this purpose, one could adopt the Model Predictive Control (MPC) approach and compute at each time slot t the power of the dispatchable units over some prediction horizon $H_t = \{t, t+1, \dots, t+T-1\}$ of length $T > 0$ as the solution $p_i^*(t+k)$, $i \in \mathcal{D}$, $k \in \mathcal{T} = \{0, 1, \dots, T-1\}$, to the following optimization problem:

$$\min_{\{p_i(t+k)\}_{i \in \mathcal{D}, k \in \mathcal{T}}} \mathcal{J}^{\mathcal{D}} = \sum_{i \in \mathcal{D}} \sum_{k \in \mathcal{T}} \mathcal{J}_i^{\mathcal{D}}(p_i(t+k)) \quad (16.2a)$$

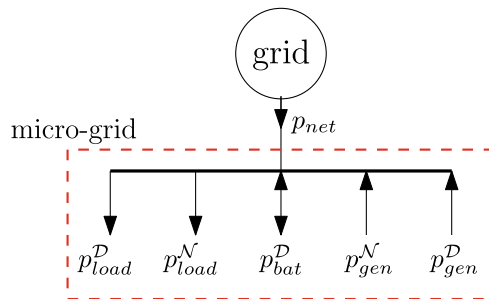


Fig. 16.1 Schematic view of the exchange flows between a micro-grid and the grid

$$\text{s.t.: } \sum_{i \in \mathcal{D}} p_i(t+k) + \sum_{i \in \mathcal{N}} p_i(t+k) = \bar{p}(t+k), \quad k \in \mathcal{T} \quad (16.2b)$$

$$\sum_{k \in \mathcal{T}} a_{j,k} p_i(t+k) \leq d_{j,i}(t), \quad j = 1, \dots, n_{c,i}, i \in \mathcal{D}, \quad (16.2c)$$

where $\mathcal{J}^{\mathcal{D}}$ in (16.2a) is the sum of operating costs of the dispatchable units $\mathcal{J}_i^{\mathcal{D}}$ which are typically quadratic or linear functions of the power, (16.2b) is the tracking constraint, and (16.2c) collects all $n_{c,i}$ operative constraints (including those that can be relaxed) per dispatchable unit $i \in \mathcal{D}$ that are assumed to be linear. The bounds $d_{j,i}(t)$, $j = 1, \dots, n_{c,i}$, in (16.2c) may depend on the time slot t as it is indeed the case for the capacity constraint of a battery that depends on its initial status of charge or for the ramp-rate constraint of a generator that depends on the power production in the previous time slot. According to the receding-horizon strategy adopted in MPC, one then dispatches only the power $p_i(t) = p_i^*(t)$, $i \in \mathcal{D}$, solves again the problem at $t+1$, and so on. This allows to select at each t the best $p_i^*(t)$, $i \in \mathcal{D}$, according to the most updated information on the power exchanged by the non-dispatchable units $\sum_{i \in \mathcal{N}} p_i(k)$ over H_t . However, this basic MPC approach has two main drawbacks: i) it requires to repeatedly solve problem (16.2) online and ii) it does not allow to distinguish between hard constraints representing, e.g., actuation or safety limits, and soft constraints representing, e.g., desired operating regions.

In this paper, we propose an alternative approach which overcomes these two drawbacks by collecting in a parameter vector ϑ all the quantities appearing in (16.2) that depends on t except for the decision variables (i.e., $\sum_{i \in \mathcal{N}} p_i(t+k)$ and $\bar{p}(t+k)$, $k \in \mathcal{T}$, and $d_{j,i}(t)$, $j = 1, \dots, n_{c,i}$, $i \in \mathcal{D}$), and solving offline a variant of the resulting multi-parametric Quadratic Program (mp-QP) where the 1-norm violation of the soft constraints is penalized in the cost. Based on the results in [8], we can derive an optimal map that provides the power values $p_i^*(t+k)$, $i \in \mathcal{D}$, $k \in \mathcal{T}$, to dispatch as a function of ϑ and has the following properties: for those values of ϑ such that the associated original problem (16.2) is feasible, then, the map returns the optimal solution to (16.2), otherwise it provides a solution that minimizes the 1-norm of the relaxed constraints violation. At each time slot t , one can then determine the power dispatch $p_i^*(t)$, $i \in \mathcal{D}$, by simply evaluating the map at the value of the parameter vector ϑ corresponding to $\sum_{i \in \mathcal{N}} p_i(t+k)$ and $\bar{p}(t+k)$, $k \in \mathcal{T}$, and $d_{j,i}(t)$, $j = 1, \dots, n_{c,i}$, $i \in \mathcal{D}$, where the contribution of non-dispatchable units over H_t can be assessed based on reliable predictors.

We next reformulate problem (16.2) according to the notations in the reference work [8] and recall its main results that are instrumental to the MG optimal operation problem solution.

Under the assumption that the operating cost $\mathcal{J}^{\mathcal{D}}$ is quadratic or linear and due to the linearity of all constraints, problem (16.2) results in an mp-QP in the form:

$$\begin{aligned} \min_z \quad & \frac{1}{2} z^\top Q z + c^\top z \\ \text{s.t.:} \quad & G_h z \leq b_h + S_h \vartheta \end{aligned} \quad (16.3)$$

$$G_s z \leq b_s + S_s \vartheta,$$

where $z \in \mathbb{R}^{n_z}$ is the decision vector collecting $p_i(t+k)$, $i \in \mathcal{D}$, $k \in \mathcal{T}$; $\vartheta \in \mathbb{R}^{n_\vartheta}$ is the parameter vector collecting $\sum_{i \in \mathcal{N}} p_i(t+k)$, $k \in \mathcal{T}$, $\bar{p}(t+k)$, $k \in \mathcal{T}$, and $d_{j,i}(t)$, $j = 1, \dots, n_{c,i}$, $i \in \mathcal{D}$; $0 \leq Q = Q^\top \in \mathbb{R}^{n_z, n_z}$ and $c \in \mathbb{R}^{n_z}$ define the cost function; $G_h \in \mathbb{R}^{n_h, n_z}$, $b_h \in \mathbb{R}^{n_h}$, and $S_h \in \mathbb{R}^{n_h, n_\vartheta}$ define the n_h hard constraints (including the tracking constraint (16.2b) that is converted into double-sided inequalities); and $G_s \in \mathbb{R}^{n_s, n_z}$, $b_s \in \mathbb{R}^{n_s}$, and $S_s \in \mathbb{R}^{n_s, n_\vartheta}$ define the n_s soft constraints.

Assumption 16.1 (Well-posedness [8]) The set $\Theta_f \subseteq \mathbb{R}^{n_\vartheta}$ of ϑ for which (16.3) is feasible is full dimensional. Moreover, the constraint set $\mathcal{Z}_\vartheta = \{z \in \mathbb{R}^{n_z} : G_h z \leq b_h + S_h \vartheta\}$ is bounded for some $\vartheta \in \mathbb{R}^{n_\vartheta}$.

Under the quite standard well-posedness Assumption 16.1, problem (16.3) admits as a solution an optimal map $z^* : \Theta_f \rightarrow \mathcal{Z}$ taking values in $\mathcal{Z} = \bigcup_{\vartheta \in \Theta_f} \mathcal{Z}_\vartheta$ which is a Piece-Wise Affine (PWA) function given by

$$z^*(\vartheta) = \Gamma_z^j \vartheta + \gamma_z^j, \quad \vartheta \in \Theta_j, \quad j = 1, \dots, r, \quad (16.4)$$

where $\{\Theta_j\}_{j=1}^r$ is a polyhedral partition of Θ_f , which is also polyhedral [9].

There might exist values of $\vartheta \notin \Theta_f$ such that problem (16.3) without the soft constraints would be feasible. In the considered MG optimal operation problem, infeasibility may, for instance, arise due to violations of comfort/performance requirements rather than hard actuation/safety constraints. By admitting violations of soft constraints, one can enlarge the set of parameter values for which the problem admits a solution from Θ_f to $\Theta = \{\vartheta \in \mathbb{R}^{n_\vartheta} : \mathcal{Z}_\vartheta \neq \emptyset\} \supseteq \Theta_f$ while still enforcing the hard constraints. To this purpose, one can consider the following relaxed counterpart of problem (16.3):

$$\begin{aligned} \min_z \quad & \frac{1}{2} z^\top Q z + c^\top z + \mu \|G_s z - b_s - S_s \vartheta\|_+ \\ \text{s.t.} \quad & G_h z \leq b_h + S_h \vartheta, \end{aligned} \quad (16.5)$$

where $\mu > 0$ is a penalty coefficient that can be added to ϑ and interpreted as a further parameter of the resulting multi-parametric optimization problem.

In [8], the relation between problem (16.3) and its relaxed counterpart in (16.5) is fully characterized by means of the following two theorems.

Theorem 16.1 ([8]) Under Assumption 16.1, any PWA optimal map $z : \Theta \times \mathbb{R}_+ \rightarrow \mathcal{Z}$ of the multi-parametric problem (16.5) satisfies

$$\lim_{\mu \rightarrow \infty} z(\vartheta, \mu) = \bar{z}(\vartheta), \quad (16.6)$$

where $\bar{z} : \Theta \rightarrow \mathcal{Z}$ is a PWA function defined over a polyhedral partition $\{\Theta_j\}_{j=1}^q$ of Θ , which is continuous whenever $z(\vartheta, \mu)$ is continuous.

In particular, for each $j = 1, \dots, q$, Θ_j is related to the j -th region $\mathcal{R}_j = \{(\vartheta, \mu) \in \Theta \times \mathbb{R}_+ : F_j \vartheta + f_j \mu \leq g_j\}$ of $z(\vartheta, \mu)$ receding along the μ -axis (i.e., with $f_j \leq 0$) and is given by $\Theta_j = \{\vartheta \in \Theta : F_j^0 \vartheta \leq g_j^0\}$, with F_j^0 and g_j^0 containing the rows of F_j and g_j corresponding to zero elements of f_j . Also, for each $j = 1, \dots, q$,

$$\bar{z}(\vartheta) = \Gamma_z^j \vartheta + \gamma_z^j, \quad \vartheta \in \Theta_j, \quad (16.7)$$

is derived from the expression of $z(\vartheta, \mu)$ over $\mathcal{R}_j$ which is independent of μ and given by $z(\vartheta, \mu) = \Gamma_z^j \vartheta + \gamma_z^j$, $(\vartheta, \mu) \in \mathcal{R}_j$.

Theorem 16.2 ([8]) Under Assumption 16.1, for all $\vartheta \in \Theta$, any limit map $\bar{z}(\vartheta)$ obtained via (16.6) satisfies

$$\begin{aligned} \bar{z}(\vartheta) \in \arg \min_z \quad & \frac{1}{2} z^\top Q z + c^\top z \\ \text{s.t.} \quad & G_h z \leq b_h + S_h \vartheta \\ & \|[G_s z - b_s - S_s \vartheta]_+\|_1 \leq \bar{v}(\vartheta), \end{aligned} \quad (16.8)$$

with

$$\begin{aligned} \bar{v}(\vartheta) = \min_z \quad & \|[G_s z - b_s - S_s \vartheta]_+\|_1 \\ \text{s.t.} \quad & G_h z \leq b_h + S_h \vartheta. \end{aligned} \quad (16.9)$$

According to Theorem 16.1, any PWA optimal map $z(\vartheta, \mu)$ of (16.5) approaches a PWA limit map $\bar{z}(\vartheta)$ as μ grows to infinity, whereas Theorem 16.2 indicates that, as $\mu \rightarrow \infty$, solving (16.5) for a given ϑ becomes identical to first identifying all $z \in \mathcal{Z}_\vartheta$ that minimize the 1-norm of all soft constraint violations and then selecting, among these, those z that minimize the original cost. This implies that $\bar{z}(\vartheta)$ aligns with the optimal map $z^*(\vartheta)$ in (16.4) for all values of the parameter $\vartheta \in \Theta_f$ for which (16.3) remains feasible. In [8, Algorithm 1], a method is provided to calculate the limit map $\bar{z}(\vartheta)$ directly, without requiring the evaluation of any limit, by leveraging the constructive proof of Theorem 16.1.

Note that the dimensionality of the parameter space increases with the time horizon length T . This could, in principle, create a scalability issue when solving the resulting mp-QP. However, in the considered context, T is typically small since the controlled units are operated on time slots of a quarter of an hour and accurate predictions of DERs production and loads consumption are typically available over a few hours only.

We now present a possible modeling of flexible resources within the MG that fits the presented problem formulation.

16.3 Resource Modeling

As discussed in [10], the set of admissible power profiles offered by many different types of dispatchable units can be modeled via a polyhedron set $\mathcal{P}_i$, possibly accounting for both operating constraints and comfort/performance conditions, expressed in terms of power and energy limits. We next focus on those dispatchable units that are more relevant to our MG application.

16.3.1 Dispatchable Generator

The power output $p_i(t + k)$, $k \in \mathcal{T}$, is constrained by

$$\ell_h^{(p,i)} \leq \ell_s^{(p,i)} \leq p_i(t + k) \leq u_s^{(p,i)} \leq u_h^{(p,i)}, \quad k \in \mathcal{T}, \quad (16.10)$$

to remain between a minimum and a maximum value. Values $\ell_h^{(p,i)}$ and $u_h^{(p,i)}$ represent safe operating limits (hard constraints), whereas $\ell_s^{(p,i)}$ and $u_s^{(p,i)}$ represent desired performance conditions (soft constraints), e.g., generators are rarely operated at their maximum/minimum safety limits, but conservative operative ranges are defined so as to maintain a margin for further increasing or decreasing the power in response to specific requests from the grid.

We can also consider ramp-rate constraints

$$\ell_h^{(r,i)} \leq \ell_s^{(r,i)} \leq p_i(t + k) - p_i(t + k - 1) \leq u_s^{(r,i)} \leq u_h^{(r,i)}, \quad k \in \mathcal{T}, \quad (16.11)$$

if the rate at which the generator power can vary is limited. Note that for $k = 0$, $p(t - 1)$ appears in (16.11) should be included in ϑ as a parameter.

Operating Costs.

For a small-scale fuel generator within an MG, the predominant operating cost typically stems from fuel expenses, which can be defined as

$$\begin{aligned} \mathcal{J}_i^{\mathcal{D}}(p_i(t + k)) &= c_g^{(i)} \mathcal{F}_g^{(i)}(p_i(t + k)) \\ &= c_g^{(i)} [\alpha_g^{(i)} (p_i(t + k))^2 + \beta_g^{(i)} p_i(t + k) + \delta_g^{(i)}], \end{aligned} \quad (16.12)$$

where $c_g^{(i)}$ is the fuel price [€/p.u.] and $\mathcal{F}_g^{(i)}(p_i(t + k))$ denotes the fuel consumption [p.u.] which is modeled as a quadratic cost, with $\alpha_g^{(i)}$, $\beta_g^{(i)}$, and $\delta_g^{(i)}$ being comprehensive cost coefficients of the generator output [13]. Cost $c_g^{(i)}$ accounts for the cost for fuel and other costs, such as transportation and on-site storage facility costs. The operation cost of fuel cells and micro-turbines can also be captured by (16.12), albeit with different cost coefficients.

16.3.2 Battery

In case of a battery, besides having power and (possibly) rate constraints like (16.10) and (16.11), respectively, we have to account also for constraints on the minimum/maximum amount of energy $e_i(t + k + 1)$ that can be stored at the beginning of time slot $t + k + 1$, where $e_i(t + k + 1)$ is described by the following (recursive) equation:

$$e_i(t + k + 1) = \zeta_i e_i(t + k) + \Delta p_i(t + k), \quad (16.13)$$

with $\zeta_i \in (0, 1]$ being the self-discharge coefficient¹ and Δ the discrete-time slot duration. Generally, batteries are never operated at their physical limits due to degradation phenomena, which are more emphasized at very high and low state of charge, and also to prevent over/under-voltage issues and battery stability problems (for very low charge levels) [11]. This motivates the presence of the following hard and soft limits:

$$\ell_h^{(e,i)} \leq \ell_s^{(e,i)} \leq e_i(t + 1 + k) \leq u_s^{(e,i)} \leq u_h^{(e,i)}, \quad k \in \mathcal{T}. \quad (16.14)$$

By unrolling the dynamics of e_i in (16.13), one obtains

$$e_i(t + 1 + k) = \zeta_i^{k+1} e_i(t) + \Delta \sum_{s=t}^{t+k} \zeta_i^{t+k-s} p_i(s), \quad k \in \mathcal{T},$$

with $e_i(t)$ being the energy content at the beginning of time slot t . Accordingly, constraint (16.14) can be equivalently expressed in terms of power as

$$\tilde{\ell}_*^{(e,i)}(t, k) \leq \Delta \sum_{s=0}^k \zeta_i^{k-s} p_i(t + s) \leq \tilde{u}_*^{(e,i)}(t, k), \quad k \in \mathcal{T}, \quad (16.15)$$

with $\tilde{\ell}_*^{(e,i)}(t) = \ell_*^{(e,i)} - \zeta_i^{k+1} e_i(t)$ and $\tilde{u}_*^{(e,i)}(t) = u_*^{(e,i)} - \zeta_i^{k+1} e_i(t)$, where $*$ denotes hard (h) or soft (s) limits. Notice that (16.15) depends on the state at the beginning of the time slot t , to be included in ϑ .

Operating Costs. The cost of operating a battery can be calculated as proposed in [6]:

$$\mathcal{J}_i^{\mathcal{D}}(p_i(t + k)) = c_b^{(i)}(p_i(t + k))^2, \quad (16.16)$$

where the cost coefficient $c_b^{(i)}$ accounts for the kWh price cost, heat power losses during charging and discharging, which in turn depends on specific battery design parameters, and maintenance costs [13]. The latter can be thought of as the cost incurred for having 1 kWh of storage capacity available, which depends on the

¹ Most lithium-ion batteries have a self-discharge rate $(1 - \zeta_i)$ of between $0.5 \div 3\%$ per month.

Table 16.1 Parameters of the resources in the micro-grid. Within brackets the desired soft operative limits

	Resource	$P_{min} \div P_{max}$ [kW]	E_n [kWh]	$E_{min} \div E_{max}$ [kWh]
$\mathcal{N}$	Resistive load	$0 \div 10$	–	–
	Electronic load	$0 \div 10$	–	–
	PV	$0 \div 10$	–	–
$\mathcal{D}$	Generator	$0 \div 10(7)$	–	–
	Battery	$-20 \div 20$	20	$[0.02(0.2) \div 0.98(0.8)]E_n$

replacement cost and the total lifetime cycling capacity of the battery.²

Notice that, thermostatically controlled loads such as residential air conditioners, heat pumps, space heaters, refrigerators, and water heaters can be viewed as general energy storage devices hence modeled in the form of (16.13), [12]. In this case, power and energy limits are related to temperature limits and the operating cost can be expressed as a linear function of the absorbed power. This cost represents, for example, the expense incurred to maintain home temperature within the desired comfort range.

16.4 Numerical Example

We consider an MG that consists of five distributed units, with a total generation capacity of 20 kW, 20 kW of load and 20 kWh of storage. Our goal is letting the MG work in island mode (net-zero exchange with the main grid) over a 1-day horizon divided in 96 time slots of duration $\Delta = 15$ minutes. To this purpose, we set $\bar{p}(t) = 0, \forall t$, and consider an optimization problem in the form of (16.2) with a finite horizon of 1-hour length (i.e., $T = 4$), parametric in $\{\sum_{i \in \mathcal{N}} p_i(t + k)\}_{k \in \mathcal{T}}$ and the initial battery energy content, with power and energy constraints as defined in (16.10) and (16.14) (see Table 16.1, where values between brackets denote (tighter) soft operational limits imposed on both the generator power and the battery energy content over time).

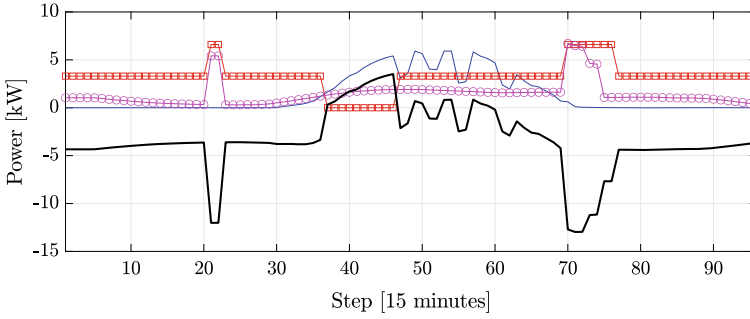
Values for $\sum_{i \in \mathcal{N}} p_i(t)$ over the 1-day horizon are taken from [14] and reported in Fig. 16.2. Table 16.2 presents coefficients for determining operational costs for both the battery, [6], and the generator, [13].

To investigate the (positive) impact of the violation of performance limits imposed on power and energy limits (soft constraints) on the fulfillment of the scheduled power exchange enforcement problem, we run Algorithm 1 in [8] (Step 1 was performed using the MPT3 toolbox) to solve offline the relaxed mp-QP problem (16.5) and

² The total lifetime cycling capacity depends on the rated depth of discharge, capacity, and lifetime.

Table 16.2 Operational cost coefficients for dispatchable resources (Eqs. (16.12) and (16.16))

	Resource	Cost coefficients
$\mathcal{D}$	Generator	$c_g = 1.5, \alpha_g = 3.1e - 4, \beta_g = 0.052, \delta_g = 0.8$
	Battery	$c_b = 0.02$

**Fig. 16.2** Daily profiles of loads and PV generation. Non-dispatchable net power (black solid line); PV generation (blue solid line); resistive load (red squared markers); electronic load (magenta circle markers). Data shown are taken from [14]

obtain the limit map $\bar{z}(\vartheta)$ (16.6) providing the optimal PWA power dispatch $\{p_i^*(t + k)\}_{i \in \mathcal{D}, k \in \mathcal{T}}$ as a function of the parameter vector containing $\{\sum_{i \in \mathcal{N}} p_i(t + k)\}_{k \in \mathcal{T}}$ and the battery energy content at the beginning of the time slot t . Such an optimal map is then applied in a receding-horizon fashion. That is, at any given time slot t , we evaluate the map at the current parameter value, apply only $\{p_i^*(t)\}_{i \in \mathcal{D}}$, move to the next time slot $t + 1$, and repeat the process.

We report in Fig. 16.3 the resulting optimal power dispatch when the initial battery energy content is set to 10%, hence below the lower operative (soft) limit of 20%. Firstly, observe that we nonetheless succeeded in obtaining a net-zero power exchange with the main grid (green dotted line). Secondly, the performed run confirms the nature of the adopted softening approach in achieving the minimization of the 1-norm of the soft constraint violation as a primary objective and then the minimization of the cost function of (16.3), as a secondary objective. In fact, in the early stages, the generator is operated at its upper operative (soft) limit so as to charge the battery in order to minimize the energy battery soft limit violation, and meanwhile satisfying the non-dispatchable load demand. Another interesting aspect is the optimal dispatch policy just before the first demand peak at step 21. Indeed, according to control horizon length ($T = 4$), from step 18 to 20, the generator is operated to charge the battery, which will subsequently provide the required power to meet the peak demand without causing the generator to exceed its upper power limit (soft constraint). The second peak demand, starting at step 70, is managed similarly, but in this case the generator is asked to violate its upper power limit so as to satisfy the demand.

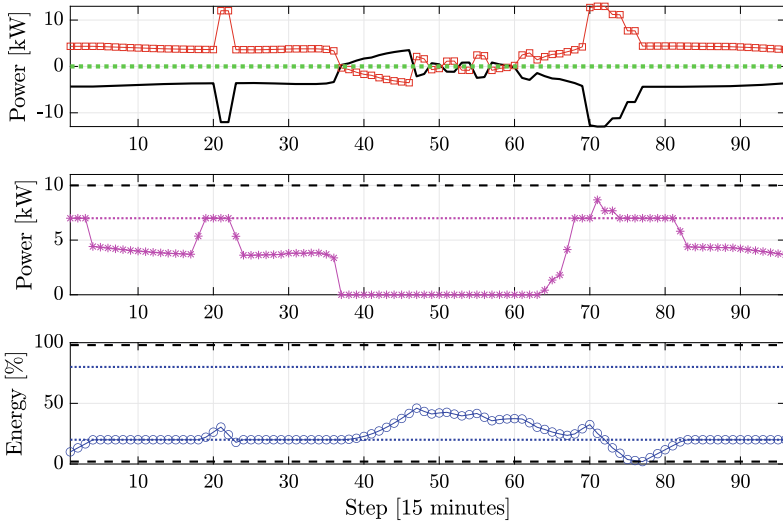


Fig. 16.3 Results for the case when the initial battery energy content is 10%. Top: non-dispatchable net power (black solid line); dispatchable net power (red square markers); resulting exchange program (green dotted line). Middle: generator dispatched power (magenta star markers). Bottom: battery dispatched energy (blue circle markers). Dotted and dashed lines denote soft and hard limits, respectively

Figure 16.4 reports the optimal dispatch of a second run carried out considering an initial battery energy content equal to 50%, thus within the imposed operative (soft) limit. Due to the battery operational cost which is consistently lower than that of the generator for power values below 10 kW, the battery is primarily designated to meet the initial net load demand (approximately equal to 5 kW) thus discharging rapidly. Gradually, the generator's contribution increases, enabling the battery to smoothly reduce its injection and reach its lower (soft) energy limit just before step 18. Following this, similar to the previously illustrated case, the generator starts charging the battery to address the peak demand at step 21. Subsequently, the optimal dispatch corresponds to that obtained for an initial battery energy content of 10%.

16.5 Conclusion

This paper focuses on optimally controlling the operation of a micro-grid to ensure that effective power exchanges with the main grid adhere to the scheduled ones, thereby responding to fluctuations in DERs production and loads consumption. Specifically, we frame this task as a multi-parametric quadratic programming problem and we leverage recent advancements in soft-constrained optimization for this class of problem to handle infeasibility issues resulting from violations of performance and operational constraints on resources. The proposed approach is

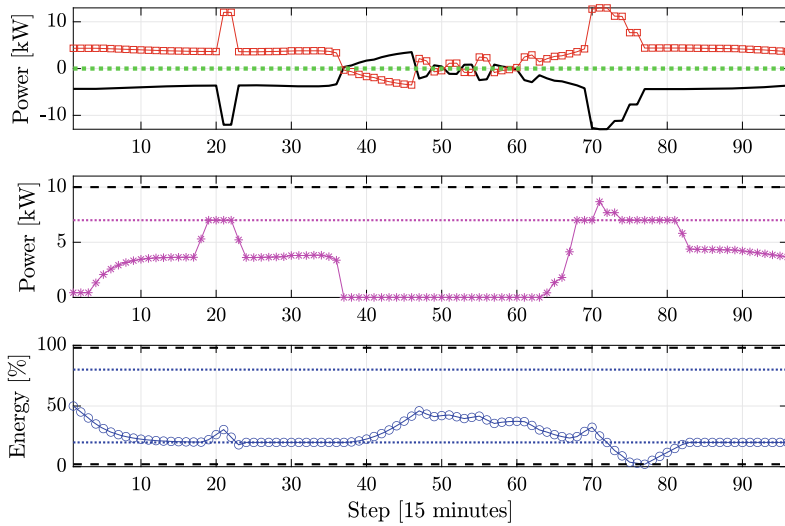


Fig. 16.4 Results for the case when the initial battery energy content is 50%. Top: non-dispatchable net power (black solid line); dispatchable net power (red square markers); resulting exchange program (green dotted line). Middle: generator dispatched power (magenta star markers). Bottom: battery dispatched energy (blue circle markers). Dotted and dashed lines denote soft and hard limits, respectively

applied to MGs operating in island mode, with results demonstrating effective alignment with zero net power exchange programs. Also, results give evidence that the adopted relaxation approach indeed minimizes constraint violation in the region of infeasibility.

References

1. Mariam, L., Basu, M., Conlon, M.F.: Microgrid: Architecture, policy and future trends. *Renew. Sustain. Energy Rev.* **64**, 477–489 (2016)
2. Sandroni, C., Verga, M., Lazzari, R., Fantini, M., Sacchi, M., Prandoni, V.: RSE's microgrid: A facility for research, development and testing of future distributed generation and microgrid technologies. In: 2016 AEIT International Annual Conference (AEIT), pp. 1–6. IEEE (2016)
3. Edmunds, C., Galloway, S., Elders, I., Bukhsh, W., Telford, R.: Design of a DSO-TSO balancing market coordination scheme for decentralised energy. *IET Gen. Trans. Distrib.* **14**(5), 707–718 (2020)
4. Bonassi, F., La Bella, A., Lazzari, R., Sandroni, C., Scattolini, R.: Supervised control of hybrid AC-DC grids for power balance restoration. *Electr. Power Syst. Res.* **196**, 107107 (2021)
5. Cominesi, S.R., Farina, M., Giulioni, L., Picasso, B., Scattolini, R.: A two-layer stochastic model predictive control scheme for microgrids. *IEEE Trans. Control Syst. Technol.* **26**(1), 1–13 (2017)
6. He, Y., Li, Z., Zhang, J., Shi, G., Cao, W.: Day-ahead and intraday multi-time scale microgrid scheduling based on light robustness and MPC. *Int. J. Electr. Power Ener. Syst.* **144**, 108546 (2022)

(2023)

7. Bemporad, A., Borrelli, F., Morari, M.: Model predictive control based on linear programming-the explicit solution. *IEEE Trans. Autom. Control* **47**(12), 1974–85 (2002)
8. Falsone, A., Bianchi, F., Prandini, M.: Dealing with infeasibility in multi-parametric programming for application to explicit model predictive control. *Automatica* **157**, 111279 (2023)
9. Jones, C.N., Morari, M.: Multiparametric linear complementarity problems. In: *Proceedings of the 45th IEEE Conference on Decision and Control*, pp. 5687–5692. IEEE (2006)
10. Barot, S., Taylor, J.A.: A concise, approximate representation of a collection of loads described by polytopes. *Int. J. Electr. Power Ener. Syst.* **84**, 55–63 (2017)
11. Zhu, J., et al.: Investigation of capacity fade for 18650-type lithium-ion batteries cycled in different state of charge (SoC) ranges. *J. Power Sour.* **489**, 229422 (2021)
12. Zhao, L., Zhang, W., Hao, H., Kalsi, K.: A geometric approach to aggregate flexibility modeling of thermostatically controlled loads. *IEEE Trans. Power Syst.* **32**(6), 4721–4731 (2017)
13. Nguyen, T.A., Crow, M.L.: Stochastic optimization of renewable-based microgrid operation incorporating battery operating cost. *IEEE Trans. Power Syst.* **31**(3), 2289–2296 (2015)
14. Lazzari, R., La Bella, A.: Optimal distributed control of hybrid AC/DC grids. In: *2023 International Conference on Clean Electrical Power (ICCEP)*, pp. 571–578. IEEE (2023)
15. Herceg, M., Kvasnica, M., Jones, C. N., Morari, M.: Multi-parametric toolbox 3.0. In *2013 European control conference (ECC)*, pp. 502–510. IEEE (2013)

Chapter 17

Multi-Objective Learning Model Predictive Control



Siddharth H. Nair, Charlott Vallon, and Francesco Borrelli

Abstract Multi-objective learning model predictive control is a novel data-driven control scheme which improves a linear system's closed-loop performance with respect to several convex control objectives over iterations of a repeated task. At each task iteration, collected system data is used to construct terminal components of a model predictive controller. The formulation presented in this paper ensures that closed-loop control performance improves between successive iterations with respect to each objective. We provide proofs of recursive feasibility and performance improvement, and show that the converged policy is Pareto optimal. Simulation results demonstrate the applicability of the proposed approach.

Keywords Data-driven control · Model predictive control · Multi-objective optimization

17.1 Introduction

Iterative learning control (ILC) considers the problem of solving the same task multiple times (consider robotic manipulators performing repetitive motions [1], drone racing [2], or batch processes [3]). Each task execution is referred to as an iteration. In *data-driven* ILC, system data collected during an iteration is used to improve the control performance at subsequent iterations. This approach can be particularly helpful if designing an optimal controller a priori is difficult, for example due to modeling challenges or task complexity.

The Learning Model Predictive Control (LMPC) method in [4, 5] proposes an MPC controller formulation that iteratively improves the system's closed-loop performance on the task with respect to a single control objective while guaranteeing constraint satisfaction during the learning process. Here we extend the LMPC frame-

Nair and Vallon—Contributed equally.

S. H. Nair · C. Vallon (✉) · F. Borrelli
University of California, Berkeley, USA
e-mail: charlottvallon@berkeley.edu

work to the case where performance is measured with respect to multiple distinct control objectives. Most real-world control applications require balancing multiple objectives, such as reducing actuation costs and completing a necessary task, or considering both individual and shared objectives in multi-agent systems [6]. Designing control formulations that are both implementable and result in the desirable closed-loop behavior can be difficult, especially if there are competing objectives, e.g., minimize both time to complete a task and energy required to complete a task [7–9]. In a number of examples, carefully tuned MPC cost function formulations have been proposed that balance multiple application-specific objectives, including for power converters [10], a distillation column [11], vehicle adaptive cruise control [12], and efficient micro-grid management [13]. These works do not include proofs of stability or feasibility; rather, the control parameters are formulated as a function of application-specific values and are validated in simulations or experiments.

In multi-objective MPC, input trajectories are evaluated with respect to multiple objective functions. In general, a single trajectory which minimizes all costs at the same time does not exist. Instead, there will be a Pareto frontier of solutions which do not dominate each other [14]. A common approach for application-agnostic multi-objective MPC frameworks is to execute the following steps at each time step: *i*) calculate the Pareto frontier based on the current system and environment state, *ii*) select a specific point on the Pareto frontier, and *iii*) apply the corresponding input. Because calculating the Pareto frontier of a multi-objective optimization problem can be difficult and computationally intensive, a number of methods have been proposed to increase efficiency [15–17], including only calculating a relevant subset of the Pareto frontier [18].

In contrast, we propose using an iterative, data-driven approach for reducing multiple objectives over iterations of a repeated task. Here, the MPC controller does not have full knowledge of control invariant sets typically used for guaranteeing stability; instead, this set is constructed iteratively from a single initial feasible trajectory. Critically, no individual closed-loop control cost is allowed to increase between subsequent iterations. Specifically, we

1. propose an application-agnostic Multi-Objective LMPC (MO-LMPC) formulation for iterative closed-loop performance improvement with respect to multiple objectives;
2. provide proofs of stability and iterative performance improvement with respect to each individual objective;
3. prove that if the MO-LMPC converges to a steady-state trajectory, the converged policy is Pareto optimal; and
4. demonstrate the effectiveness of the formulation in a simulation example.

The proofs in Sect. 17.3.1 build on the idea introduced in [4, 5] of using the convex hull of trajectories as an MPC terminal set and the realized cost-to-go of trajectories as a terminal cost. Here we adapt these ideas to the case of additional multi-objective performance-improvement constraints in the MPC formulation.

Running Example: Consider a fleet of autonomous electric delivery vehicles. Each day, a high-level scheduler assigns a number of deliveries to each vehicle, which

must occur within specific time windows as expected by clients. Task assignment is done on the basis of time and energy cost estimates for routes between delivery locations. The vehicles begin the day fully charged at a centralized depot, and must return there after completing their deliveries without running out of charge.

We will refer to this running example throughout the remainder of this paper, to help explain concepts and motivate specific formulations. In this paper, we focus on the design of the low-level autonomous vehicle controllers. We refer to [19] for an example formulation of the high-level scheduler.

Notation: Unless otherwise indicated, vectors are column vectors constructed using parentheses (). $[A, B]$ denotes the set of integers $\{A, A + 1, \dots, B - 1, B\}$.

17.2 Problem Formulation

We consider the linear discrete-time system

$$x_{t+1} = Ax_t + Bu_t, \quad (17.1)$$

where $x_t \in \mathbb{R}^{n_x}$ and $u_t \in \mathbb{R}^{n_u}$ are the system state and input, respectively, at time step t . The states and inputs are subject to polytopic constraints given by

$$\mathcal{X} = \{x \in \mathbb{R}^{n_x} \mid G_x x \leq g_x\}, \quad \mathcal{U} = \{u \in \mathbb{R}^{n_u} \mid G_u u \leq g_u\}. \quad (17.2)$$

We seek feedback policies $\pi = \{\pi_t(\cdot)\}_{t \geq 0}$ to be applied to (17.1) that optimize the performance of the closed-loop system with respect to M different control objectives V_i , where

$$V_i(x_0, \pi) = \sum_{t=0}^{\infty} h_i(x_t, \pi_t(x_t)), \quad i \in [1, M]. \quad (17.3)$$

The i th control objective $V_i(x_0, \pi)$ is characterized by the stage cost $h_i(\cdot, \cdot)$ incurred over the state and input trajectories that results from applying policy π to the system (17.1) beginning at state x_0 at time $t = 0$. We consider stage costs $h_i(\cdot, \cdot)$ which are continuous, jointly convex, and satisfy

$$\begin{aligned} h_i(x_t, u_t) &\geq 0 \quad \forall x_t \in \mathbb{R}^{n_x} \setminus x_F, \quad u_t \in \mathbb{R}^{n_u} \setminus u_F, \\ \exists i \in [1, M] : h_i(x_t, u_t) &> 0 \quad \forall x_t \in \mathbb{R}^{n_x} \setminus x_F, \quad u_t \in \mathbb{R}^{n_u} \setminus u_F, \\ h_i(x_F, u_F) &= 0 \quad \forall i \in [1, M] \end{aligned} \quad (17.4)$$

for some goal state $x_F \in \mathcal{X}$ which is an equilibrium state under input $u_F \in \mathcal{U}$:

$$x_F = Ax_F + Bu_F. \quad (17.5)$$

For any given initial state $x_0 = x$, the feedback policy synthesis problem can be posed as the infinite-horizon, multi-objective optimal control problem:

$$\begin{aligned}
 V^*(x) = \min_{\pi} \quad & (V_1(x_0, \pi), \dots, V_M(x_0, \pi)) \\
 \text{s.t.} \quad & x_{t+1} = Ax_t + B\pi_t(x_t) \\
 & x_t \in \mathcal{X}, \quad \pi_t(x_t) \in \mathcal{U} \quad \forall t \geq 0 \\
 & x_0 = x \\
 & = (V_1(x, \pi^*), \dots, V_M(x, \pi^*)),
 \end{aligned} \tag{17.6}$$

where the vector $V^*(x) \in \mathbb{R}^M$ contains the M objective values (17.3) associated with the optimal policy $\pi^* = \{\pi_t^*(\cdot)\}_{t \geq 0}$. We define a solution to (17.6) in Definition 17.1.

Running Example: Each route between two delivery locations will be associated with a controller, to be applied to a vehicle with state x . In order to maximize the number of deliveries that can be completed by the fleet, each controller aims to minimize $M = 2$ different objectives along the particular route: V_1 measures the time required to complete a particular delivery, and V_2 measures the battery charge required to complete a particular delivery. (Here we consider that a linear approximation of the vehicle model is used for path planning.)

There are two main challenges to solving the optimal control problem (17.6). First, (17.6) is infinite dimensional, due to the unbounded time horizon and the optimization over functions $\pi = \pi_0(\cdot), \pi_1(\cdot), \dots$ as decision variables. The aim of minimizing multiple objectives, which may be competing, introduces additional complexity in defining a solution to the problem.

Toward addressing the infinite dimensionality of (17.6), we adapt techniques from Learning MPC (LMPC) to iteratively approximate the optimal feedback policy using collected trajectory data of (17.1). Here we restate only the most relevant concepts; we refer to [20] for additional details and to [5] for extensive analysis and modeling considerations for the single-objective linear LMPC formulation.

To address the multi-objective nature of (17.6), we use *Pareto optimality*, which describes solutions for which no individual objective function $V_i(x_0, \cdot)$ can be decreased without causing an increase in another objective function. Pareto optimal solutions can be obtained by a scalarization procedure which converts (17.6) into a single weighted-sum objective optimization problem.

Definition 17.1 A solution π is said to be Pareto optimal if $\nexists (\pi', i \in [1, M])$ such that $V_i(x_0, \pi') < V_i(x_0, \pi)$ and $V_j(x_0, \pi') \leq V_j(x_0, \pi) \quad \forall j \in [1, M] \setminus i$.

17.3 Multi-Objective LMPC

Our goal is to minimize multiple control costs over repeated task iterations. Specifically, we will formulate the Multi-Objective LMPC (MO-LMPC) to ensure that no single control objective V_i (17.3) increases between any two task iterations $j \geq 0$ and $j + 1$.

Running Example: Each time a delivery vehicle traverses a particular route between two locations is considered an iteration j . The vehicles begin and end each of these iterations at the same set of geographical location x_S and x_F , respectively. The high-level scheduler updates its estimated time and energy costs for each route based on recent vehicle performance. To ensure feasibility of the assigned schedule (i.e., no vehicle runs out of charge while completing a delivery or misses the assigned delivery window), we require that the low-level controllers never consume more time or energy on a route than during the previous iteration.

Let x_t^j , u_t^j denote the state and input of the system, respectively, at time t of iteration j , and with $\pi^j = \{\pi_t^j(\cdot)\}_{t \geq 0}$ the sequence of policies implemented during iteration j . We assume that at each iteration j the system starts from the same initial state, $x_0^j = x_S$, $\forall j \geq 0$. At iteration j , we define the convex set $\mathcal{CS}^j$ as

$$\mathcal{CS}^j = \text{conv} \left(\bigcup_{r=0}^{j-1} \bigcup_{t \geq 0} \{x_t^r\} \right), \quad (17.7)$$

the convex hull of all state trajectories from previous task iterations. By definition, each state $x \in \mathcal{CS}^j$ is associated with a set $\Lambda^j(x)$ of multipliers λ , where

$$\Lambda^j(x) = \left\{ \lambda = \bigcup_{r=0}^{j-1} \bigcup_{t \geq 0} \lambda_t^r \left| \begin{array}{l} \sum_{r=0}^{j-1} \sum_{t \geq 0} \lambda_t^r x_t^r = x, \\ \sum_{r=0}^{j-1} \sum_{t \geq 0} \lambda_t^r = 1, \lambda_t^r \geq 0 \end{array} \right. \right\}. \quad (17.8)$$

Each $\lambda \in \Lambda^j(x)$ induces a specific set of M control objective estimates $\hat{V}_i^j$, one for each objective V_i , $i \in [1, M]$:

$$\hat{V}_i^j(x, \lambda) = \begin{cases} \sum_{r=0}^{j-1} \sum_{t \geq 0} \lambda_t^r V_i(x_t^r, \pi^r) & \lambda \in \Lambda^j(x) \\ \infty & \text{else.} \end{cases} \quad (17.9)$$

By definition of $\mathcal{CS}^j$, each state $x \in \mathcal{CS}^j$ is a convex combination of states traversed during a previous iteration; for each state $x \in \mathcal{CS}^j$, $\hat{V}_i^j(x, \lambda)$ is a corresponding barycentric interpolation of the i th control objectives previously incurred from those states x_t^r . Thus the function $\hat{V}_i^j(x, \lambda)$ is a data-driven, piece-wise linear under-approximation of the optimal i th objective cost, with the interpolation weights specified by λ . As more data is collected during additional iterations, the approximation error decreases. An example construction of $\hat{V}_i^j(x, \lambda)$ is shown in Fig. 17.1.

For a particular set of predicted state and input trajectories

$$\mathbf{x}_t^j = [x_{t|t}^j, x_{t+1|t}^j, \dots, x_{t+N|t}^j], \quad \mathbf{u}_t^j = [u_{t|t}^j, u_{t+1|t}^j, \dots, u_{t+N-1|t}^j],$$

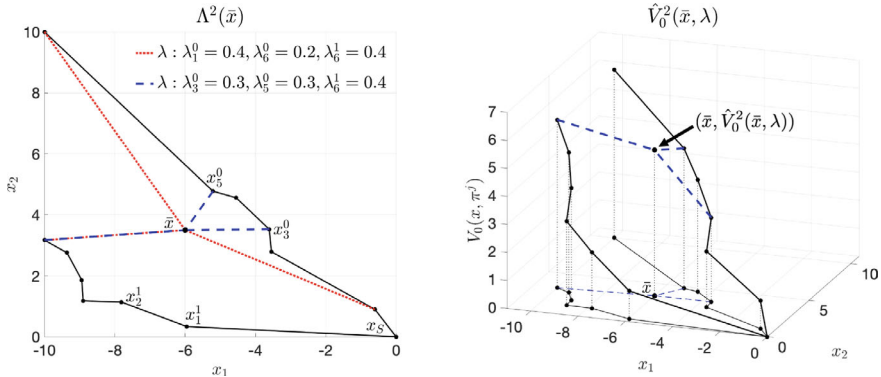


Fig. 17.1 The left plot depicts two possible values $\lambda \in \Lambda^2(\bar{x})$ for a particular $\bar{x}$. Each λ corresponds to a particular convex combination of states from previous trajectories (here, segments of $\mathbf{x}^0$ and $\mathbf{x}^1$ are shown). The right plot depicts how $\hat{V}_1^2(\bar{x}, \lambda)$ is interpolated for a particular choice of $\lambda \in \Lambda^2(\bar{x})$

and $\lambda_t^j \in \Lambda^j(x_{t+N|t}^j)$, we define for each $i \in [1, M]$ a running cost estimate:

$$J_i^j(\mathbf{x}_t^j, \mathbf{u}_t^j, \lambda_t^j) = \hat{V}_i^{j-1}(x_{t+N|t}^j, \lambda_t^j) + \sum_{k=t}^{t+N-1} h_i(x_{k|t}^j, u_{k|t}^j). \quad (17.10)$$

Each $J_i^j(\mathbf{x}_t^j, \mathbf{u}_t^j, \lambda_t^j)$ estimates the i th control objective accumulated from the current state x_t^j to the end of the task. This estimate consists of the i th stage cost accumulated over the predicted N -step trajectories $\mathbf{x}_t^j, \mathbf{u}_t^j$ plus the control objective estimate (17.9) of the last predicted state in the N -step trajectory, evaluated using λ_t^j .

We now formulate the MO-LMPC problem for a particular choice $\alpha \in \mathbb{R}^M$ where $0 < \alpha_i < 1$, $\sum_{i=1}^M \alpha_i = 1$ (we refer to Sect. 17.4 for a discussion on the effect of choosing different α). At each time t of iteration $j \geq 1$, we solve

$$J_{t,N}^{\text{MO-LMPC},j}(x_t^j) = \min_{\mathbf{x}_t, \mathbf{u}_t, \lambda_t} \sum_{i=1}^M \alpha_i J_i^j(\mathbf{x}_t, \mathbf{u}_t, \lambda_t) \quad (17.11a)$$

$$\text{s.t. } x_{k+1|t} = Ax_{k|t} + Bu_{k|t} \quad (17.11b)$$

$$x_{k+1|t} \in \mathcal{X}, u_{k|t} \in \mathcal{U} \quad \forall k \in [t, t+N-1] \quad (17.11c)$$

$$x_{t+N|t} \in \mathcal{CS}^j \quad (17.11d)$$

$$\lambda_t \in \Lambda^j(x_{t+N|t}) \quad (17.11e)$$

$$x_{t|t} = x_t^j \quad (17.11f)$$

$$J_i^j(\mathbf{x}_t, \mathbf{u}_t, \lambda_t) \leq \begin{cases} V_i(x_0, \pi^{j-1}) & t = 0 \\ J_i^j(\mathbf{x}_{t-1}^{j,*}, \mathbf{u}_{t-1}^{j,*}, \lambda_{t-1}^{j,*}) - h_i(x_{t-1}^j, u_{t-1}^j) & \text{else} \end{cases} \quad \forall i \in [1, M], \quad (17.11g)$$

where $\mathbf{x}_{t-1}^{j,\star}, \mathbf{u}_{t-1}^{j,\star}, \boldsymbol{\lambda}_{t-1}^{j,\star}$ are the optimizers of $J_{t-1,N}^{\text{MO-LMPC},j}(x_{t-1}^j)$ (17.11) at the previous time step $t - 1$. The optimal control problem (17.11) searches for a feasible state and input trajectory of length N beginning from the current state x_t^j and ending in $\mathcal{CS}^j$, the convex hull of states from which the system has successfully completed the task in a previous iteration. The controller then applies the first optimal input

$$u_t^j = \pi^{\text{MO-LMPC}}(x_t^j) = u_{t|t}^*, \quad (17.12)$$

before re-solving (17.11) at time $t + 1$, resulting in a receding-horizon control scheme.

As in single-objective LMPC [5], MO-LMPC (17.11) searches for a feasible state trajectory that ends in the convex hull of all previous state trajectories. MO-LMPC (17.11) differs from single-objective LMPC in two main ways. First, the objective function in (17.11) is now a sum of multiple control objectives. We have used the scalarization procedure to formulate the multi-objective optimization into a scalar optimization problem, using our choice of α to indicate potential prioritization between different objectives. Second, we have incorporated a time-varying constraint (17.11g). Next we show this constraint (17.11g) ensures that no objectives increase over successive iterations.

Note (17.10) is a convex function in $\mathbf{x}_t, \mathbf{u}_t, \boldsymbol{\lambda}_t$, and the quantities $h_i(x_{t-1}^j, u_{t-1}^j)$ and $J_i^j(\mathbf{x}_{t-1}^{j,\star}, \mathbf{u}_{t-1}^{j,\star}, \boldsymbol{\lambda}_{t-1}^{j,\star})$ are known scalars. Thus the MO-LMPC formulation (17.11) maintains the convexity of the single-objective LMPC formulation.

Assumption 17.1 At iteration $j = 1$, we assume the set $\mathcal{CS}^{j-1} = \mathcal{CS}^0$ contains a feasible state trajectory (with respect to (17.6)) converging to x_F (17.5).

Assumption 17.1 is not restrictive in practice: any initial suboptimal trajectory satisfying the constraints of (17.6) and converging to x_F can initialize $\mathcal{CS}^0$.

Running Example: In our example, the set $\mathcal{CS}^0$ will contain trajectories between two delivery locations that result in conservative time and energy usages, for example, from a controller driving strictly below the speed limit. Such a conservative initialization may mean the high-level scheduler cannot confidently plan as many delivery routes as desired. As more data is collected and the low-level controller performance improves with each iteration, the time and energy cost estimates along each route are lowered, allowing the high-level scheduler to assign increasingly productive delivery routes.

17.3.1 Properties

Theorem 17.1 (Feasibility and Stability) Consider system (17.1) controlled by the MO-LMPC controller (17.11)–(17.12). Let $\mathcal{CS}^j$ be the sampled safe set at iteration j as defined in (17.7). Let Assumption 17.1 hold. Then, the MO-LMPC (17.11)–(17.12) is feasible for all time steps $t \geq 0$ at every iteration $j \geq 1$. Moreover, the

equilibrium point x_F (17.5) is asymptotically stable for the closed-loop system (17.1), (17.11)–(17.12) at every iteration $j \geq 1$.

Proof By Assumption 17.1, the set $\mathcal{CS}^0$ is non-empty. Since $\mathcal{CS}^{j-1} \subseteq \mathcal{CS}^j$, it follows that $\mathcal{CS}^j$ is also a non-empty set. Furthermore, we have $x_0^j = x_S$, $\forall j \geq 0$.

At time $t = 0$ of iteration $j \geq 1$, consider the N -step state trajectory, corresponding input sequence and convex multiplier vector given by

$$\mathbf{x}'_0 = [x_0^{j-1}, x_1^{j-1}, \dots, x_N^{j-1}], \mathbf{u}'_0 = [u_0^{j-1}, u_1^{j-1}, \dots, u_{N-1}^{j-1}] \quad (17.13a)$$

$$\lambda'_0 = \left\{ \bigcup_{r=0}^{j-1} \bigcup_{t \geq 0} \lambda_t^r \mid \lambda_N^{j-1} = 1, \lambda_t^r = 0 \text{ else} \right\}. \quad (17.13b)$$

Note (17.13) correspond to the state and input trajectories taken by the system in the previous iteration $j - 1$, and therefore satisfy constraints (17.11b), (17.11c), and (17.11e). At time $t = 0$, it remains to verify if (17.13) satisfies constraint (17.11g). From (17.10), (17.9) we have for all $i \in [1, M]$:

$$\begin{aligned} J_i^j(\mathbf{x}'_0, \mathbf{u}'_0, \lambda'_0) &= \sum_{t=0}^{N-1} h_i(x_t^{j-1}, u_t^{j-1}) + \hat{V}_i^{j-1}(x_N^{j-1}, \lambda'_0) \\ &= \sum_{t=0}^{N-1} h_i(x_t^{j-1}, u_t^{j-1}) + \sum_{r=0}^{j-1} \sum_{t \geq 0} \lambda_t^r V_i(x_t^r, \pi^r) \\ &= \sum_{t=0}^{N-1} h_i(x_t^{j-1}, u_t^{j-1}) + V_i(x_N^{j-1}, \pi^{j-1}) \\ &= \sum_{t=0}^{\infty} h_i(x_t^{j-1}, u_t^{j-1}) = V_i(x_S, \pi^{j-1}). \end{aligned}$$

Thus, (17.13) satisfies (17.11g) with equality, and (17.13) is a feasible solution to (17.11) at time $t = 0$ of iteration $j \geq 1$.

Assume the MO-LMPC (17.11)–(17.11g) is feasible at time t of iteration j , with

$$\mathbf{x}_t^{j,\star} = [x_{t|t}^{j,\star}, x_{t+1|t}^{j,\star}, \dots, x_{t+N|t}^{j,\star}], \mathbf{u}_t^{j,\star} = [u_{t|t}^{j,\star}, u_{t+1|t}^{j,\star}, \dots, u_{t+N-1|t}^{j,\star}] \quad (17.14a)$$

$$\lambda_t^{j,\star} = \left\{ \bigcup_{r=0}^{j-1} \bigcup_{k \geq 0} \lambda_{k|t,j}^{r,\star} \right\}. \quad (17.14b)$$

Because of (17.12), (17.11f), and no model mismatch between (17.1) and (17.11c), we have $u_t^j = u_{t|t}^{j,\star}$, $x_t^j = x_{t|t}^{j,\star}$, $x_{t+1}^j = x_{t+1|t}^{j,\star}$. It further follows from (17.11d) that $x_{t+N|t}^{j,\star} = \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,\star} x_k^r$. Let us define

$$\bar{u} = \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,*} u_k^r, \quad \bar{x} = \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,*} x_{k+1}^r = \sum_{r=0}^{j-1} \sum_{k \geq 1} \lambda_{k-1|t,j}^{r,*} x_k^r,$$

where $\bar{u} \in \mathcal{U}$, and $\bar{x} \in \mathcal{C}S^{j-1}$. Consider the candidate trajectories at time $t+1$,

$$\begin{aligned} \mathbf{x}'_{t+1} &= [x_{t+1|t}^{j,*}, x_{t+2|t}^{j,*}, \dots, x_{t+N|t}^{j,*}, \bar{x}], \quad \mathbf{u}'_{t+1} = [u_{t+1|t}^{j,*}, u_{t+2|t}^{j,*}, \dots, u_{t+N-1|t}^{j,*}, \bar{u}] \\ \lambda'_{t+1} &= \{\lambda \mid \lambda_k^r = \lambda_{k-1|t,j}^{r,*}, \lambda_0^r = 0\} \end{aligned} \quad (17.15)$$

which satisfy constraints (17.11b), (17.11c), and (17.11e). At time $t+1$, it remains to verify if (17.15) satisfies constraint (17.11g). From (17.10), (17.9), we have for all $i \in [1, M]$:

$$J_i^j(\mathbf{x}'_{t+1}, \mathbf{u}'_{t+1}, \lambda'_{t+1}) = \sum_{k=t+1}^{t+N} h_i(x_{k|t}^{j,*}, u_{k|t}^{j,*}) + \hat{V}_i^{j-1}(\bar{x}, \lambda'_{t+1}) \quad (17.16a)$$

$$\begin{aligned} &= J_i^j(\mathbf{x}_t^j, \mathbf{u}_t^j, \lambda_t^j) - h_i(x_{t|t}^{j,*}, u_{t|t}^{j,*}) + \hat{V}_i^{j-1}(\bar{x}, \lambda'_{t+1}) - \hat{V}_i^{j-1}(x_{t+N|t}^{j,*}, \lambda_t^j) + \\ &\quad + h_i(x_{t+N|t}^{j,*}, \bar{u}) \end{aligned} \quad (17.16b)$$

$$\begin{aligned} &= J_i^j(\mathbf{x}_t^j, \mathbf{u}_t^j, \lambda_t^j) - h_i(x_t^j, u_t^j) + \hat{V}_i^{j-1}(\bar{x}, \lambda'_{t+1}) - \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,*} V_i(x_k^r, \boldsymbol{\pi}^r) + \\ &\quad + h_i(x_{t+N|t}^{j,*}, \bar{u}) \end{aligned} \quad (17.16c)$$

$$\begin{aligned} &= J_i^j(\mathbf{x}_t^j, \mathbf{u}_t^j, \lambda_t^j) - h_i(x_t^j, u_t^j) + \hat{V}_i^{j-1}(\bar{x}, \lambda'_{t+1}) - \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,*} V_i(x_{k+1}^r, \boldsymbol{\pi}^r) + \\ &\quad + h_i(x_{t+N|t}^{j,*}, \bar{u}) - \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,*} h_i(x_k^r, u_k^r). \end{aligned} \quad (17.16d)$$

From Jensen's inequality, it follows that

$$h_i(x_{t+N|t}^{j,*}, \bar{u}) \leq \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,*} h_i(x_k^r, u_k^r). \quad (17.17)$$

Furthermore, by definition of λ'_{t+1} in (17.15), we have

$$\hat{V}_i^{j-1}(\bar{x}, \lambda'_{t+1}) = \sum_{r=0}^{j-1} \sum_{k \geq 1} \lambda_{k|t,j}^{r,*} V_i(x_k^r, \boldsymbol{\pi}^r) = \sum_{r=0}^{j-1} \sum_{k \geq 0} \lambda_{k|t,j}^{r,*} V_i(x_{k+1}^r, \boldsymbol{\pi}^r). \quad (17.18)$$

From (17.16), (17.17), (17.18), it follows that

$$J_i^j(\mathbf{x}'_{t+1}, \mathbf{u}'_{t+1}, \boldsymbol{\lambda}'_{t+1}) \leq J_i^j(\mathbf{x}_t^j, \mathbf{u}_t^j, \boldsymbol{\lambda}_t^j) - h_i(x_t^j, u_t^j) \quad \forall i \in [1, M],$$

and thus (17.15) satisfies (17.11g) and is a feasible solution to (17.11) at time $t + 1$ of iteration j . We have shown that at iteration $j \geq 1$, the MO-LMPC is feasible at time $t = 0$, and that feasibility of the MO-LMPC at time t implies feasibility at time $t + 1$. We conclude by induction that the MO-LMPC (17.11)–(17.12) is feasible at all time $t \geq 0$ of all iterations $j \geq 1$.

Next we show asymptotic stability of x_F (17.5) under the MO-LMPC policy (17.11)–(17.12) by showing that the cost function $J_{t,N}^{\text{MO-LMPC},j}(\cdot)$ (17.11) is a time-varying Lyapunov function for the closed-loop system (17.1), (17.12) for the equilibrium state x_F (17.5). Continuity of $J_{t,N}^{\text{MO-LMPC},j}(\cdot)$ can be shown from [21], and it follows from (17.4) that

$$J_{t,N}^{\text{MO-LMPC},j}(x) > 0 \quad \forall x \in \mathbb{R}^{n_x} \setminus \{x_F\}, \quad J_{t,N}^{\text{MO-LMPC},j}(x_F) = 0. \quad (17.19)$$

Moreover, recursive feasibility of (17.11) and compactness of the constraints imply that $J_{t,N}^{\text{MO-LMPC},j}(x)$ can be bounded above by some positive definite function $W(x)$ independent of time t . It remains to show that $J_{t,N}^{\text{MO-LMPC},j}(\cdot)$ decreases along the closed-loop trajectory, where

$$J_{t,N}^{\text{MO-LMPC},j}(x_t^j) = \sum_{i=1}^M \alpha_i J_i^j(\mathbf{x}_t^{j,*}, \mathbf{u}_t^{j,*}, \boldsymbol{\lambda}_t^{j,*}). \quad (17.20)$$

Note constraint (17.11g) ensures that for all $i \in [1, M]$

$$J_i^j(\mathbf{x}_t^{j,*}, \mathbf{u}_t^{j,*}, \boldsymbol{\lambda}_t^{j,*}) \leq J_i^j(\mathbf{x}_{t-1}^{j,*}, \mathbf{u}_{t-1}^{j,*}, \boldsymbol{\lambda}_{t-1}^{j,*}) - h_i(x_{t-1}^j, u_{t-1}^j),$$

and since all $\alpha_i \geq 0$, it follows that

$$\sum_{i=1}^M \alpha_i J_i^j(\mathbf{x}_t^{j,*}, \mathbf{u}_t^{j,*}, \boldsymbol{\lambda}_t^{j,*}) \leq \sum_{i=1}^M \alpha_i J_i^j(\mathbf{x}_{t-1}^{j,*}, \mathbf{u}_{t-1}^{j,*}, \boldsymbol{\lambda}_{t-1}^{j,*}) - \sum_{i=1}^M \alpha_i h_i(x_{t-1}^j, u_{t-1}^j),$$

and therefore

$$J_{t,N}^{\text{MO-LMPC},j}(x_t^j) - J_{t-1,N}^{\text{MO-LMPC},j}(x_{t-1}^j) \leq - \sum_{i=1}^M \alpha_i h_i(x_{t-1}^j, u_{t-1}^j) < 0. \quad (17.21)$$

Thus, by (17.21), positive definiteness of each stage cost $h_i(\cdot, \cdot)$ (17.4), and continuity of $J_{t,N}^{\text{MO-LMPC},j}(\cdot)$ (17.10), we conclude that the state x_F (17.5) is asymptotically stable for the system (17.1) in closed loop with the MO-LMPC (17.11)–(17.12) ■.

Theorem 17.2 (Performance Improvement) *Consider system (17.1) in closed loop with the MO-LMPC controller (17.11)–(17.12). Let $\mathcal{CS}^j$ be the sampled safe set at the iteration j as defined in (17.7). Let Assumption (17.1) hold. Then, no individual control objective V_i increases over successive iterations j and $j - 1$, i.e., $V_i(x_S, \pi^j) \leq V_i(x_S, \pi^{j-1})$, $\forall i \in [1, M]$.*

Proof At time $t = 0$ of iteration j , the constraint (17.11g) enforces $\forall i \in [1, M]$

$$J_i^j(\mathbf{x}_0^{j,*}, \mathbf{u}_0^{j,*}, \lambda_0^{j,*}) \leq V_i(x_0^j, \pi^{j-1}) = \sum_{t=0}^{\infty} h_i(x_t^{j-1}, u_t^{j-1}). \quad (17.22)$$

Similarly, the constraints at time $t \geq 1$ enforce

$$\begin{aligned} J_i^j(\mathbf{x}_0^{j,*}, \mathbf{u}_0^{j,*}, \lambda_0^{j,*}) &\geq J_i^j(\mathbf{x}_1^{j,*}, \mathbf{u}_1^{j,*}, \lambda_1^{j,*}) + h_i(x_0^j, u_0^j) \\ &\geq J_i^j(\mathbf{x}_2^{j,*}, \mathbf{u}_2^{j,*}, \lambda_2^{j,*}) + \sum_{t=0}^1 h_i(x_t^j, u_t^j) \\ &\geq \lim_{k \rightarrow \infty} [J_i^j(\mathbf{x}_k^{j,*}, \mathbf{u}_k^{j,*}, \lambda_k^{j,*}) + \sum_{t=0}^{k-1} h_i(x_t^j, u_t^j)]. \end{aligned} \quad (17.23)$$

From Theorem 17.1, we have $\lim_{t \rightarrow \infty} x_t^j = x_F$, and therefore by continuity of each $h_i(\cdot, \cdot)$, $\lim_{t \rightarrow \infty} J_i^j(\mathbf{x}_t^{j,*}, \mathbf{u}_t^{j,*}, \lambda_t^{j,*}) = 0$, resulting in

$$J_i^j(\mathbf{x}_0^j, \mathbf{u}_0^j, \lambda_0^j) \geq \sum_{t=0}^{\infty} h_i(x_t^j, u_t^j). \quad (17.24)$$

From (17.22) and (17.24), it follows that

$$\sum_{t=0}^{\infty} h_i(x_t^{j-1}, u_t^{j-1}) \geq J_i^j(\mathbf{x}_0^j, \mathbf{u}_0^j, \lambda_0^j) \geq \sum_{t=0}^{\infty} h_i(x_t^j, u_t^j), \quad (17.25)$$

from which we conclude that each objective V_i exhibits non-increase between successive iterations. ■

Assumption 17.2 There exists at least one $i' \in [1, M]$ such that $h_{i'}(\cdot, \cdot)$ is strongly convex and $\alpha_{i'} > 0$.

Theorem 17.3 (Pareto Optimality) *Consider system (17.1) in closed loop with the MO-LMPC controller (17.11)–(17.12). Let $\mathcal{CS}^j$ be the sampled safe set at the iteration j as defined in (17.7). Let Assumptions (17.1) and (17.2) hold. Assume that after a finite number of iterations c , the closed-loop trajectory converges to fixed-point state and input trajectories such that $(\mathbf{x}^j, \mathbf{u}^j) = (\mathbf{x}^\infty, \mathbf{u}^\infty)$, $\forall j \geq c$. Then the policy π^j inducing the closed-loop trajectory $(\mathbf{x}^\infty, \mathbf{u}^\infty)$ is a Pareto optimal solution to the infinite-horizon multi-objective optimal control problem (17.6).*

Proof The proof of Theorem 17.3 may be found in [20].

The proof of Theorem 17.3 proceeds by first showing that $x_t^\infty, \dots, x_{t+N}^\infty, u_t^\infty, \dots, u_{t+N-1}^\infty$ is the optimizer of (17.11) for any control horizon $N, \forall t \geq 0$ and $\forall j \geq c + 1$. Then the closed-loop trajectory $(\mathbf{x}^\infty, \mathbf{u}^\infty)$ is shown to be equivalent to the solution of (17.11) at $t = 0$ as $N \rightarrow \infty$. Finally, Assumption 17.2 is used to show that (17.11) (as $N \rightarrow \infty$) is equivalent to some scalarization of (17.6), with the optimizer $(\mathbf{x}^\infty, \mathbf{u}^\infty)$ being Pareto optimal because of the convexity of (17.11) [22].

17.4 Numerical Example

We apply the MO-LMPC framework for the constrained optimal control of an LTI system. Consider an agent described by the following discrete-time point mass model for motion on the 2D plane,

$$\underbrace{\begin{bmatrix} X_{t+1} \\ Y_{t+1} \\ \dot{X}_{t+1} \\ \dot{Y}_{t+1} \end{bmatrix}}_{x_{t+1}} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \underbrace{\begin{bmatrix} X_t \\ Y_t \\ \dot{X}_t \\ \dot{Y}_t \end{bmatrix}}_{x_t} + \begin{bmatrix} \frac{1}{2}\Delta t^2 & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \underbrace{\begin{bmatrix} \ddot{X}_t \\ \ddot{Y}_t \end{bmatrix}}_{u_t}, \quad (17.26)$$

where at time step t the state x_t comprises the agent's position (X_t, Y_t) and velocity $(\dot{X}_t, \dot{Y}_t)$. The input u_t is the acceleration in each direction $(\ddot{X}_t, \ddot{Y}_t)$, and the discretization time step is $\Delta t = 0.25$ s. The agent is subject to constraints

$$\mathcal{X} = \{x \mid x_{\text{lb}} \leq x \leq x_{\text{ub}}\}, \quad \mathcal{U} = \{u \mid u_{\text{lb}} \leq u \leq u_{\text{ub}}\}, \quad (17.27)$$

where $x_{\text{ub}} = -x_{\text{lb}} = (4 \text{ m}, 4 \text{ m}, 4 \text{ ms}^{-1}, 4 \text{ ms}^{-1})$ and $u_{\text{ub}} = -u_{\text{lb}} = (1 \text{ ms}^{-2}, 1 \text{ ms}^{-2})$.

The iterative control task is to steer the agent from the initial state $x_S = (4 \text{ m}, 3 \text{ m}, 0 \text{ ms}^{-1}, 0 \text{ ms}^{-1})$ to standstill at the origin $x_F = (0, 0, 0, 0)$. Control performance is evaluated with respect to two distinct objectives:

$$V_1(x_S, \pi) = \sum_{t \geq 0} x_t^\top Q x_t + \pi(x_t)^\top R \pi(x_t) \quad [\text{Time to Target}], \quad (17.28)$$

$$V_2(x_S, \pi) = \sum_{t \geq 0} \eta \Delta t \sqrt{\dot{X}_t^2 + \dot{Y}_t^2} \quad [\text{Energy Consumption}], \quad (17.29)$$

where $Q = \text{diag}((0.5, 0.5, 0.02, 0.02))$, $R = \text{diag}((0.6, 0.6))$, and $\eta = 12.0 \text{ m}^{-1}$. The first objective (17.28) is a proxy to approximately penalize the time taken to reach the task target x_F using a quadratic cost for strong convexity required by Assumption 17.2. (Note an additional objective $V_3(x_S, \pi) = \sum_{t \geq 0} \Delta t$ can be

introduced to penalize time exactly, but is omitted in this study for ease of visualization.) The second objective (17.29) penalizes the agent's energy consumption during the task. The agent is assumed to have a battery that is initially fully charged, with 100% state of charge (SOC) at $t = 0$. While energy consumption is generally a function of both speed and acceleration [23], here we assume a simpler model where the battery drains proportionally with the agent's speed:

$$SOC_{t+1} = SOC_t - \eta \Delta t \sqrt{\dot{X}_t^2 + \dot{Y}_t^2}. \quad (17.30)$$

Note SOC_t is not an explicit state in our model (17.26).

The initialization trajectory for $\mathcal{CS}^0$ (17.7) is created using an MPC policy which models the system states as in (17.26) but with the additional SOC state (17.30) bounded via the nonlinear constraints $0 \leq SOC_k \leq 100$, $\forall k = t, \dots, t + N$, resulting in a nonlinear MPC formulation. The objective is given by (17.28), with horizon $N = 20$, and the problem is solved using IPOPT [24]. After initializing $\mathcal{CS}^0$, we solve the multi-objective control task using the MO-LMPC formulation in (17.11), with the objective functions (17.28) and (17.29), $\alpha = (0.4, 0.6)$, and MPC horizon $N = 5$. The convex optimization problem is solved using ECOS [25].¹

The closed-loop state trajectories and objectives are depicted for 26 iterations in Figs. 17.2 and 17.3. As shown in Theorem 17.1, all closed-loop trajectories converge to x_F . Figure 17.2 compares the initialization trajectory with the final trajectory ($j = 26$): during iteration $j = 26$ the agent state converges to the origin in fewer

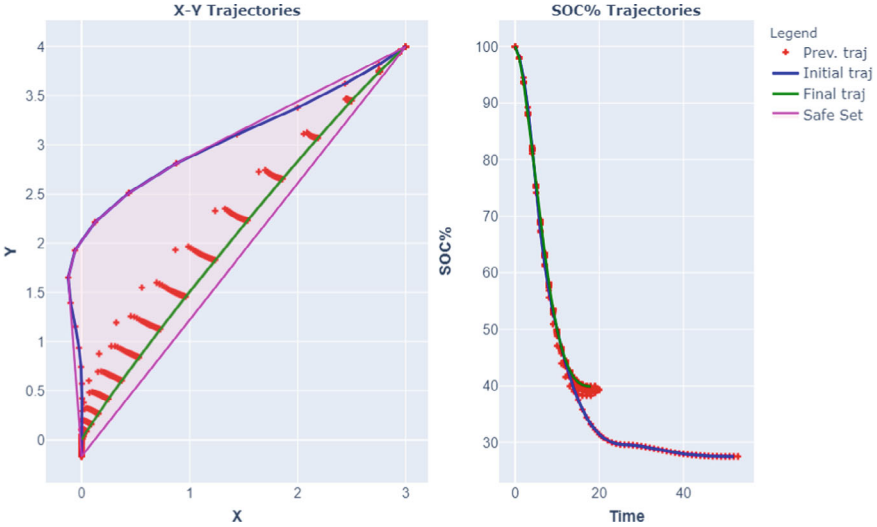


Fig. 17.2 $X - Y$ and SOC% trajectories of the system. Notice that the agent is stabilized sooner while consuming less SOC% in iteration $j = 26$, compared to iteration $j = 0$

¹ Our code is available at <https://github.com/shn66/Multi-Objective-LMPC>.

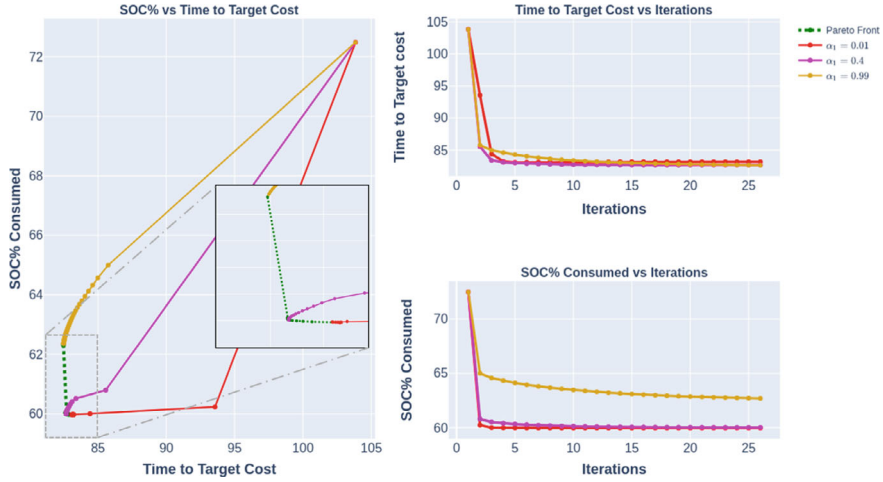


Fig. 17.3 Trajectory costs (17.28) and SOC% consumed (17.29) across iterations, depicting iterative improvement in both objectives, and the effect of α on converged solutions

time steps (minimizing (17.28)) while consuming less energy (minimizing (17.29)). Figure 17.3 shows that both control costs improve with each task iteration, as in Theorem 17.2.

The choice of α suggests a prioritization between control objectives. As seen in Fig. 17.3, increasing α_1 from 0.01 to 0.99 results in trajectories that increasingly prioritize minimizing (17.28). We note that this strict performance improvement cannot be guaranteed via choice of time-invariant α alone; the constraint (17.11g) is required. As shown in Theorem 17.3, by varying α the MO-LMPC converges to different Pareto optimal solutions. This is supported by results in Fig. 17.3, where the Pareto frontier was obtained by varying α_1 in the range $0.01 \leq \alpha_1 \leq 0.99$ and running MO-LMPC to convergence; all converged solutions are non-dominating. Note that due to our requirement of iterative performance improvement in each control objective, parts of the Pareto frontier may not be reachable using MO-LMPC. While MO-LMPC always converges to a point on the Pareto frontier, if certain Pareto optimal policies induce a larger cost in any control objective than the initialization trajectory, those policies are not feasible for the MO-LMPC formulation. Future work will consider the design of an outer loop algorithm for relaxing the performance improvement condition to incorporate designer preferences in the converged Pareto optimal solution.

17.5 Conclusion

This paper introduced the Multi-Objective Learning Model Predictive Controller (MO-LMPC), a control approach for iterative performance improvement with respect to multiple control costs. We proved that the MO-LMPC controller is recursively feasible, asymptotically stable, iteratively reduces multiple control costs, and converges to a Pareto optimal solution to an infinite-horizon multi-objective optimal control problem. The method was demonstrated via a numerical example for capacity-constrained control of a linear system.

References

1. Meng, T., He, W.: Iterative learning control of a robotic arm experiment platform with input constraint. *IEEE Trans. Ind. Electr.* **65**(1), 664–672 (2017)
2. Lv, S., Gao, Y., Che, J., Quan, Q.: Autonomous drone racing: Time-optimal spatial iterative learning control within a virtual tube. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 3197–3203 (2023)
3. Geng, Y., Ruan, X., Yang, X., Zhou, Q.: Data-based iterative learning control for multiphase batch processes. *Asian J. Control* **25**(2), 1392–1406 (2023)
4. Rosolia, U., Borrelli, F.: Learning model predictive control for iterative tasks: a data-driven control framework. *IEEE Trans. Autom. Control* **63**(7), 1883–1896 (2018)
5. Rosolia, U., Borrelli, F.: Learning model predictive control for iterative tasks: A computationally efficient approach for linear system. 20th IFAC World Congress (2017)
6. Camisa, A., Köhler, P.N., Müller, M.A., Notarstefano, G., Allgöwer, F.: A distributed optimization algorithm for Nash bargaining in multi-agent systems. *IFAC-PapersOnLine* **53**(2), 2684–2689 (2020)
7. Exadaktylos, V., Taylor, C.J.: Multi-objective performance optimisation for model predictive control by goal attainment. *Int. J. Control* **83**(7), 1374–1386 (2010)
8. Zhu, M., Bemporad, A., Piga, D.: Preference-based MPC calibration. In: 2021 European Control Conference (ECC), pp. 638–645 (2021)
9. Eichfelder, G., Grüne, L., Krügel, L., Schießl, J.: Relaxed dissipativity assumptions and a simplified algorithm for multiobjective MPC. *Comput. Optim. Appl.* **86**(3), 1081–1116 (2023)
10. Hu, J., Zhu, J., Lei, G., Platt, G., Dorrell, D.G.: Multi-objective model-predictive control for high-power converters. *IEEE Trans. Energy Conv.* **28**(3), 652–663 (2013)
11. Wojsznis, W., Mehta, A., Wojsznis, P., Thiele, D., Blevins, T.: Multi-objective optimization for model predictive control. *ISA Trans.* **46**(3), 351–361 (2007)
12. Zhao, R., Wong, P.-K., Xie, Z., Zhao, J.: Real-time weighted multi-objective model predictive controller for adaptive cruise control systems. *Int. J. Autom. Technol.* **18**(4), 279–292 (2017)
13. Vásquez, L.O.P., Redondo, J.L., Álvarez Hervás, J.D., Ramírez, V.M., Torres, J.L.: Balancing CO₂ emissions and economic cost in a microgrid through an energy management system using MPC and multi-objective optimization. *Appl. Energy* **347**, 120998 (2023)
14. Boyd, S., Vandenberghe, L.: *Convex optimization*. Cambridge University Press (2004)
15. Bemporad, A., Muñoz de la Peña, D.: Multiobjective model predictive control based on convex piecewise affine costs. In: 2009 European Control Conference (ECC), pp. 2402–2407 (2009)
16. Gharib, A., Stenger, D., Ritschel, R., Voßwinkel, R.: Multi-objective optimization of a path-following MPC for vehicle guidance: A bayesian optimization approach. In: 2021 European Control Conference (ECC), pp. 2197–2204 (2021)
17. Peitz, S., Schäfer, K., Ober-Blöbaum, S., Eckstein, J., Köhler, U., Dellnitz, M.: A multiobjective MPC approach for autonomously driven electric vehicles. In: *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 8674–8679 (2017). 20th IFAC World Congress

18. Grüne, L., Stieler, M.: Multiobjective model predictive control for stabilizing cost criteria. *Discr. Continuous Dyn. Syst.- B* **24**(8), 3905–3928 (2019)
19. Vallon, C., Pinto, A., Stellato, B., Borrelli, F.: Learning hierarchical control for multi-agent capacity-constrained systems. *arXiv e-prints*, pp. arXiv-2403 (2024)
20. Nair, S.H., Vallon, C., Borrelli, F.: Multi-objective learning model predictive control. *arXiv preprint* [arXiv:2405.11698](https://arxiv.org/abs/2405.11698) (2024)
21. Mayne, D., Rawlings, J., Rao, C., Sokaert, P.: Constrained model predictive control: Stability and optimality. *Automatica* **36**, 789–814 (2000)
22. Miettinen, K.: Nonlinear multiobjective optimization, vol. 12. Springer Science & Business Media (1999)
23. Joa, E., Choi, E.Y., Borrelli, F.: Eco-driving under localization uncertainty for connected vehicles on Urban roads: Data-driven approach and Experiment verification. *arXiv preprint* [arXiv:2402.01059](https://arxiv.org/abs/2402.01059) (2024)
24. Biegler, L.T., Zavala, V.M.: Large-scale nonlinear programming using IPOPT: An integrating framework for enterprise-wide dynamic optimization. *Comput. Chem. Eng.* **33**(3), 575–582 (2009)
25. Domahidi, A., Chu, E., Boyd, S.: ECOS: An SOCP solver for embedded systems. In: 2013 European Control Conference (ECC), pp. 3071–3076. IEEE (2013)

Chapter 18

Terminal Set of Nonlinear Model Predictive Control with Koopman Operators



Yajing Zhang, Yangyang Feng, Shuyou Yu, and Hong Chen

Abstract A large terminal set of model predictive control results in a large region of attraction of the closed-loop systems, which can help to reduce the computational burden of the involved optimization problem. In this paper, a novel scheme is proposed to obtain a terminal set and a terminal penalty of nonlinear model predictive control. Firstly, the nonlinear system is approximated through the Koopman operator theory, whereby a linear system with unknown but bounded disturbances is generated. Then, a polytopic terminal set is obtained accordingly, where the nonlinear system is described by a linear model with disturbances. The effectiveness of the proposed scheme is demonstrated using a benchmark problem.

Keywords Terminal set · Nonlinear model predictive control · Koopman operator theory

18.1 Introduction

Model predictive control (MPC), referred to as receding-horizon control, is a widely used optimization-based control scheme. Solving an optimization problem by measuring the system state at each moment, a finite horizon control sequence is obtained accordingly. Only the first element of the obtained control sequence is applied to the system. At the next moment, the complete process is repeated with the updated system state.

Y. Zhang · Y. Feng · S. Yu · H. Chen

Department of Control Science and Engineering, Jilin University, Changchun, China

S. Yu (✉)

The Key Laboratory of Industrial Internet of Things and Networked Control, Chongqing University of Posts and Telecommunications, Chongqing, China

e-mail: shuyou@jlu.edu.cn

H. Chen

College of Electronics and Information Engineering, Tongji University, Shanghai, China

MPC with guaranteed nominal stability [1, 2] is one of the most important model predictive control schemes, where a terminal constraint set and a terminal penalty are imposed in order to guarantee asymptotic stability. Furthermore, specific constructions of the terminal set and the terminal penalty are provided with the assumption of Lipschitz continuity. A large terminal set results in a large region of attraction for the closed-loop system, which can also reduce the online computational burden. Generally, however, finding a non-conservative terminal penalty and a terminal set is not an easy task. An approach to obtain a terminal set is developed based on a linear-quadratic regulator designed in the neighborhood of the origin, where the higher order nonlinear effect of the system is assumed to be bounded [3, 4]. Thus, it leads to a method of calculating terminal set for a large dimensional system, and terminal penalty of nonlinear systems [5], where extra degrees are added to reduce the conservativeness of the offline optimization problem.

Machine learning techniques are adopted to construct a tailored quadratic and convex terminal cost which approximates the cost-to-go function of constrained linear model predictive control frameworks [6, 7]. An algorithm is proposed, which learns the terminal penalty and adjusts the MPC parameters based on a stability metric [8]. The terminal penalty is formulated as a Lyapunov function neural network with the aim of recovering or enlarging the attraction region of the initial demonstrator using a short prediction horizon. Learning-based method is adopted to construct the terminal penalty by linking it to an infinite-horizon optimal control problem, where the Lyapunov function is the optimal cost [9].

Recently, Koopman operator theory is increasingly prevalent in engineered system design and data analysis. In this paper, motivated by linear model identification for control [10, 11], a scheme for solving the terminal set of nonlinear model predictive control is proposed. First, approximate the nonlinear system using a linear model with unknown but bounded disturbances, which act additively on the state and control inputs. Then, results are presented which permit the calculation of a maximal robust positively invariant set of discrete-time linear time-invariant system with disturbances. The maximal robust positively invariant set, also known as the 0-reachable set, is designated as the terminal set.

The remainder of the paper has the following structure. Section 18.2 defines the problem setup, and introduces the preliminaries. Section 18.3 presents the main results, including Koopman operator-based linear systems with disturbances, terminal sets of nonlinear model predictive control with a Koopman operator. To demonstrate the effectiveness of the proposed scheme, a simulation example is provided in Sect. 18.4. Section 18.5 concludes the paper.

18.2 Problem Setup and Preliminaries

Consider a discrete-time nonlinear system

$$z_{k+1} = f(z_k, v_k) \quad (18.1)$$

where $z_k \in \mathbb{R}^n$ is the state and $v_k \in \mathbb{R}^m$ is the manipulation input at the moment k . The state and control inputs are subject to the following constraints:

$$z_k \in \mathcal{Z}, \quad k \geq 0, \quad (18.2a)$$

$$v_k \in \mathcal{V}, \quad k \geq 0, \quad (18.2b)$$

where $\mathcal{Z} \subseteq \mathbb{R}^n$ and $\mathcal{V} \subseteq \mathbb{R}^m$ are the admissible sets of z_k and v_k , respectively.

Suppose that the system state z_k is measured instantaneously, and there is neither model perturbation nor external disturbance at all.

Furthermore, the following assumptions are made:

Assumption 18.1 The point $(0, 0) \in \mathbb{R}^n \times \mathbb{R}^m$ is an equilibrium point of system (18.1), i.e., $f(0, 0) = 0$, and $f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ is continuously differentiable.

Assumption 18.2 The sets of $\mathcal{Z}$ and $\mathcal{V}$ are compact, and $(0, 0) \in \mathcal{Z} \times \mathcal{V}$.

At the moment k , a finite-horizon open-loop optimization problem is defined as follows:

Problem 18.1

$$\underset{V_k}{\text{minimize}} \quad \mathcal{C}(z_k, V_k)$$

s.t.

$$z_{k+i+1|k} = f(z_{k+i|k}, v_{k+i|k}), \quad z_{k|k} = z_k, \quad i \in \mathbb{N}_{[0, N_p-1]}, \quad (18.3a)$$

$$z_{k+i|k} \in \mathcal{Z}, \quad i \in \mathbb{N}_{[1, N_p-1]}, \quad (18.3b)$$

$$v_{k+i|k} \in \mathcal{V}, \quad i \in \mathbb{N}_{[0, N_p-1]}, \quad (18.3c)$$

$$z_{k+N_p|k} \in \mathbb{Z}_f, \quad (18.3d)$$

where the cost functional

$$\mathcal{C}(z_k, V_k) = \sum_{i=0}^{N_p-1} \|z_{k+i|k}\|_{\bar{E}}^2 + \|v_{k+i|k}\|_{\bar{F}}^2 + \|z_{k+N_p|k}\|_{\bar{G}}^2 \quad (18.4)$$

and $V_k := \{v_{k|k}, v_{k+1|k}, \dots, v_{k+N_p-1|k}\}$ denotes the N_p -step control sequence, $\bar{E} \in \mathbb{R}^{n \times n}$ and $\bar{F} \in \mathbb{R}^{m \times m}$ are positive definite weighting matrices. The index $\mathbb{N}$ denotes

the set of positive integers and $\mathbb{N}_{[1, N_p-1]}$ as the integers $1, 2, \dots, N_p$. The index $k+i|k$ denotes the values at the moment $k+i$ predicted at time k , $i \in \mathbb{N}_{[1, N_p-1]}$. The term $\mathbb{Z}_f$ is the terminal constraints set, $\|z\|_{\bar{G}}^2$ is the terminal penalty. Both $\mathbb{Z}_f$ and $\|z\|_{\bar{G}}^2$ will be determined in Sect. 18.3.

At the moment k , a optimal control sequence

$$V_k^* := \{v_{k|k}^*, v_{k+1|k}^*, \dots, v_{k+N_p-1|k}^*\} \quad (18.5)$$

is determined by solving Problem 18.1. However, only the first control element $v_{k|k}^*$ is applied to the system. At the next moment $k+1$, the whole process is repeated with the new measurement of the system states.

Note that at the moment k , the control sequence V_k is feasible for Problem 18.1, and suppose that

- (i) constraints (18.3b)–(18.3d) are satisfied;
- (ii) the cost function (18.4) is finite, i.e., $\mathcal{C}(z_k, V_k) < \infty$.

At the equilibrium point $(0, 0)$, the Jacobian linearization of system (18.1) is

$$z_{k+1} = Az_k + Bv_k, \quad (18.6)$$

where $A := \frac{\partial f}{\partial z}|_{(0,0)}$ and $B := \frac{\partial f}{\partial v}|_{(0,0)}$.

Assumption 18.3 System (18.6) is stabilizable.

Assumption 18.3 implies that a linear feedback control law $v = K_J z$ can be determined such that $A_k := A + BK_J$ is asymptotically stable. Without loss of generality, assume further that A_k has at least one non-zero eigenvalue.

The following lemma guarantees the existence of the terminal control law $K_J z$, the terminal set $\mathbb{Z}_f$, and the terminal penalty $\|z\|_{\bar{G}}^2$ [12, 13]. Note that the terminal control law $K_J z$ is only used to determine $\mathbb{Z}_f$ and $\|z\|_{\bar{G}}^2$, and never be applied to system (18.1).

Lemma 18.1 Suppose that Assumptions 18.1, 18.2 and Assumption 18.3 hold, and denote $\sigma_{\max}(A_k)$ as the maximum eigenvalues of matrix A_k . Then,

(1) There exists a unique positive definite matrix $\bar{G}$ such that

$$\kappa^2 A_k^T \bar{G} A_k - \bar{G} = -(\bar{E} + K_J^T \bar{F} K_J) \quad (18.7)$$

where $\kappa \in \left(1, \frac{1}{\sigma_{\max}(A_k)}\right)$.

(2) There exists a level set $\mathbb{Z}_f \subseteq \mathbb{Z}$, and

$$\mathbb{Z}_f := \{z \in \mathbb{R}^n \mid z^T \bar{G} z \leq \beta, \beta > 0\},$$

such that

- (1) $K_J z \in \mathcal{V}$ for any $z \in \mathbb{Z}_f$.
- (2) $\mathbb{Z}_f$ is a positive invariant set for the original nonlinear system (18.1), i.e., with the feedback control law $v = K_J z$, $z_{k+1} \in \mathbb{Z}_f$ for any $z_k \in \mathbb{Z}_f$.
- (3) for all $z_0 \in \mathbb{Z}_f$, and for the system (18.1), with the linear feedback control law $v = K_J z$

$$\sum_{i=0}^{\infty} \|z_{i|0}\|_{\bar{E}}^2 + \|v_{i|0}\|_{\bar{F}}^2 \leq z^T \bar{G} z, \quad (18.8)$$

where $z_{0|0} = z_0$ and $v_{i|0} = K_J z_{i|0}$.

18.3 Terminal Set of Nonlinear Systems Based on Koopman Operators

The Koopman operator is a way to handle nonlinear systems through a globally linear representation [14]. In principle, the Koopman operator transforms a finite-dimensional nonlinear system into an infinite-dimensional linear system. Alternatively, a finite approximation through a linear system with disturbances provides an effective tool on analysis and synthesis of nonlinear control systems.

For nonlinear systems (18.1), the Koopman operator ϖ is defined as follows:

$$\varpi \xi(z_k) = \xi(z_{k+1}) = \xi(f(z_k, v_k)), \quad (18.9)$$

where $\xi(\cdot)$ is an observation function.

The dynamic mode decomposition (DMD) method provides a finite-dimensional approximation to the Koopman operator ϖ [15]. The state and input data in the DMD algorithm need to be collected from the nonlinear system. Then, the state matrix identification is

$$\begin{aligned} Z_D &= [z_1 \ z_2 \ \cdots \ z_M] \\ Z_D^+ &= [z_2 \ z_3 \ \cdots \ z_{M+1}] \\ V_D &= [v_1 \ v_2 \ \cdots \ v_M], \end{aligned} \quad (18.10)$$

where $Z_D \in \mathbb{R}^{n \times M}$, $Z_D^+ \in \mathbb{R}^{n \times M}$, $V_D \in \mathbb{R}^{m \times M}$, and M is an integer.

The linear approximation of the nonlinear dynamics can be expressed as

$$Z_D^+ = A_D Z_D + B_D V_D = [A_D \ B_D] \begin{bmatrix} Z_D \\ V_D \end{bmatrix} = G \Phi, \quad (18.11)$$

where $G = [A_D \ B_D]$ and $\Phi = [Z_D \ V_D]^T$.

Note that the state matrix G can be obtained by solving the following least-square optimization problem:

$$\underset{\Phi}{\text{minimize}} \left\| Z_D^+ - G\Phi \right\|_F, \quad (18.12)$$

where $\|\cdot\|_F$ denotes the Frobenius norm.

The problem (18.12) can be solved using the pseudo-inverse matrix of Φ , i.e.:

$$G = Z_D^+ \Phi^\dagger, \quad (18.13)$$

where $\dagger$ denotes the Moore-pseudo-inverse.

The singular value decomposition of Φ is as follows:

$$\begin{aligned} \Phi &= [Z_D \ V_D]^T \\ &= U \Sigma V^T, \end{aligned}$$

where $U \in \mathbb{R}^{(n+m) \times (n+m)}$, $\Sigma \in \mathbb{R}^{(n+m) \times (n+m)}$, and $V \in \mathbb{R}^{M \times (n+m)}$.

Then, matrix G can be obtained by the following approximation:

$$\begin{aligned} G &\approx Z_D V \Sigma^{-1} U^T \\ &= Z_D V \Sigma^{-1} [U_1 \ U_2]^T, \end{aligned}$$

where $U_1 \in \mathbb{R}^{(n+m) \times n}$, $U_2 \in \mathbb{R}^{(n+m) \times m}$.

That is,

$$\begin{aligned} A_D &\approx \tilde{A}_D = Z_D V \Sigma^{-1} U_1^T \\ B_D &\approx \tilde{B}_D = Z_D V \Sigma^{-1} U_2^T. \end{aligned}$$

Rewrite $Z_D^+ = A_D Z_D + B_D V_D$ as

$$[z_2 \ z_3 \ \cdots \ z_{M+1}] = A_D [z_1 \ z_2 \ \cdots \ z_M] + B_D [v_1 \ v_2 \ \cdots \ v_M].$$

Then, the linear model of the nonlinear system can be obtained as

$$z_{k+1} = \tilde{A}_D z_k + \tilde{B}_D v_k + w_k, \quad (18.14)$$

where w_k is the deviation between true value and its approximate value of system (18.1). Note that the presence of modeling error w_k is unavoidable due to the finite-dimensional approximation using the Koopman operator theory.

Assume that w_k is amplitude bounded, namely:

$$w_k \in \mathcal{W} = \{w_k \in \mathbb{R}^n : \|w_k\|_\infty \leq w^{\max}\},$$

where $\mathcal{W}$ is a compact set and $0 \in \mathcal{W}$.

18.3.1 Terminal Set of Nonlinear Systems

For the Koopman linear model (18.14), assume that there exists a feedback control law $v = K_D z$ such that $A_D^k := \tilde{A}_D + \tilde{B}_D K_D$ is asymptotically stable. Define $Q_D^* = \tilde{E} + K_D^T \tilde{F} K_D$, then the terminal gain K_D and the terminal matrix P_D satisfy [16]

$$(A_D^k)^T P_D (A_D^k) - P_D \leq -Q_D^* \quad (18.15)$$

with $P_D \in \mathbb{R}^{n \times n}$ and P_D is positive definite.

Based on the Koopman linear model with the bounded disturbance w_k , a maximal robust positive invariant set can be chosen as the terminal set. The maximal robust positive invariant set is defined as follows.

Definition 18.1 (Robust positive invariant set [17]): The set $\tilde{\mathbb{Z}}_f$ is a robust positive invariant set of the uncertain system (18.14), if $(\tilde{A}_D + \tilde{B}_D K_D)z_k + w_k \in \tilde{\mathbb{Z}}_f$ for any $z_k \in \tilde{\mathbb{Z}}_f$ and $w_k \in \mathcal{W}$.

Definition 18.2 (Maximal robust positive invariant set [17]): The robust positive invariant set $\tilde{\mathbb{Z}}_f$ is the maximal robust positive invariant set of the uncertain system (18.14), if $\tilde{\mathbb{Z}}_f$ is included in all robust positive invariant set of the uncertain system (18.14).

The maximal and polytopic positive invariant set can be described as

$$\tilde{\mathbb{Z}}_f := \{z \in \mathbb{R}^n | A_t z \leq b_t, A_t \in \mathbb{R}^{n \times n}, b_t \in \mathbb{R}^n\}.$$

Note that in order to obtain the maximal robust positive invariant set, the one-step backward reachable operator is defined as follows[17]:

$$\text{Pre}(\Omega) = \{z_k \in \mathcal{Z} | \exists K_D z_k \in \mathcal{V} : (A_D + B_D K_D) z_k + w_k \in \Omega, \forall w_k \in \mathcal{W}\}, \quad (18.16)$$

where Ω is a robust positive invariant set of system (18.14), $\text{Pre}(\Omega)$ is referred to as the one-step backward reachable set of the set Ω . Note that $\text{Pre}(\Omega)$ can be computed by Multi-Parametric Toolbox 3 [18].

By using (18.16), a maximal robust positive invariant set is computed by the iteration of Algorithm 18.1. Further, the terminal set of the nonlinear system (18.1) can be obtained by choosing [17]

$$\tilde{\mathbb{Z}}_f := \Omega_\infty, \quad (18.17)$$

where Ω_∞ is the maximum robust position invariant set of system (18.1), and the terminal set $\tilde{\mathbb{Z}}_f$ satisfies [17]

- (i) $\tilde{\mathbb{Z}}_f \subseteq \mathcal{Z}$, $\tilde{\mathbb{Z}}_f$ is closed and the equilibrium point $(0, 0) \in \tilde{\mathbb{Z}}_f$;
- (ii) $v_k = K_D z_k \subseteq \mathcal{V}$, for all $z_k \in \tilde{\mathbb{Z}}_f$;

Algorithm 18.1 Computation of the terminal set $\tilde{\mathbb{Z}}_f$

Input: $\mathcal{Z}, \mathcal{V}, \mathcal{W}, K_D$

Output: The terminal set $\tilde{\mathbb{Z}}_f$

- 1: Initialization. $\Omega_0 = \mathcal{Z}$.
 - 2: Iteration procedure. At any moment $k \geq 0$, $\Omega_{k+1} = \Omega_k \cap \text{Pre}(\Omega_k)$
 - 3: End condition. If $\Omega_{k+1} = \Omega_k$, then set $\Omega_\infty = \Omega_{k+1}$, $\tilde{\mathbb{Z}}_f := \Omega_\infty$. Else, set $k = k + 1$, and go to Step 2.
-

$$(iii) \ A_D^k z_k \in \tilde{\mathbb{Z}}_f.$$

Remark 18.1 Both the invariance of terminal sets and the cost from the end of the prediction horizon to infinity are guaranteed by the terminal control law $K_D z$, where the linear control gain K_D is calculated by (18.15) offline.

18.4 Numerical Example

In this section, a nonlinear system is used to verify the effectiveness of the proposed scheme, in which dynamics is

$$\begin{cases} \dot{z}_{k+1}^1 = z_k^1 + 0.1z_k^2 + 0.1v_k(\mu + (1-\mu)z_k^1) \\ \dot{z}_{k+1}^2 = z_k^2 + 0.1z_k^1 + 0.1v_k(\mu - 4(1-\mu)z_k^2) \end{cases} \quad (18.18)$$

where $\mu \in (0, 1)$. Terms of $z = [z^1 \ z^2]^T$ and v represent the state and input, respectively. Note that the nonlinear system (18.18) is unstable and its linearized system is stabilizable. Moreover, the input and state of the system (18.18) are constrained as

$$\begin{aligned} & -2 \leq v \leq 2 \\ & \begin{bmatrix} -1 \\ -1 \end{bmatrix} \leq \begin{bmatrix} z_k^1 \\ z_k^2 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \end{bmatrix}. \end{aligned}$$

Scenario 18.1 The parameter $\mu = 0.5$.

The collected datasets consist of 20 episodes, each containing data of 400 time steps. The initial state is randomly chosen within the range of $[-1, 1]$, and the control input is within the range of $[-2, 2]$.

The matrices $\tilde{A}_D$, $\tilde{B}_D$, and the bounded disturbance w_k by the Koopman operator theory are calculated as

$$\tilde{A}_D = \begin{bmatrix} 1.0150 & 0.0965 \\ 0.0693 & 1.0379 \end{bmatrix}, \quad \tilde{B}_D = \begin{bmatrix} 0.0500 \\ 0.1035 \end{bmatrix}, \quad \|w_k\|_\infty \leq 0.1.$$

The weighting matrices $\bar{E}$ and $\bar{F}$ in the optimization problem of MPC are given as

$$\bar{E} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}, \quad \bar{F} = 1.$$

Further, the terminal gain K_D and the terminal matrix P_D are calculated offline by (18.15), and

$$K_D = [1.3006 \ 1.6392], \quad P_D = \begin{bmatrix} 12.1326 & 8.6059 \\ 8.6059 & 13.8252 \end{bmatrix}.$$

As a comparison, the matrices A and B by the Jacobian linearization method at the equilibrium point $(0, 0)$ are calculated as

$$A = \begin{bmatrix} 1.005 & 0.1002 \\ 0.1002 & 1.0005 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0526 \\ 0.0526 \end{bmatrix}.$$

Similarly, the terminal gain K_J and the terminal matrix $\bar{G}$ are calculated using Lemma 18.1, that is,

$$K_J = [2.118 \ 2.118], \quad \bar{G} = \begin{bmatrix} 16.5926 & 11.359 \\ 11.359 & 16.5926 \end{bmatrix}, \quad \beta = 0.7.$$

Under the same input, a simulation experiment is implemented to compare the evolution of the nonlinear system (18.18), the Koopman linear system, and the Jacobian linearization system. The simulation results are shown in Fig. 18.1. It can be seen that the Koopman linear model and the Jacobian linearization model can accurately approximate the nonlinear system (18.18).

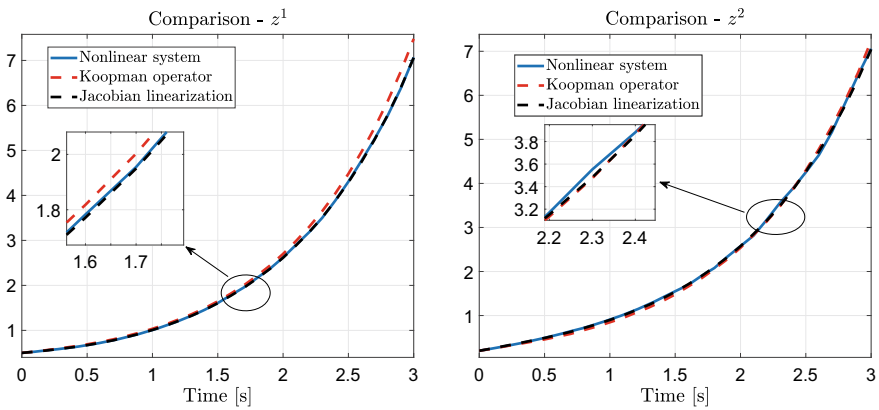
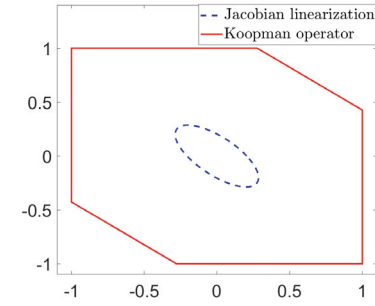


Fig. 18.1 Scenario 18.1 Validation of the Koopman linear model and the Jacobian linearization model ($\mu = 0.5$)



(a) Comparison of terminal sets

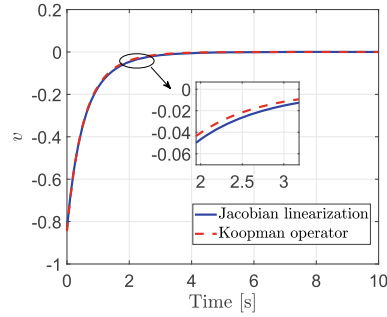
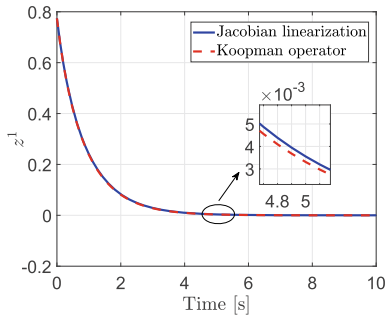
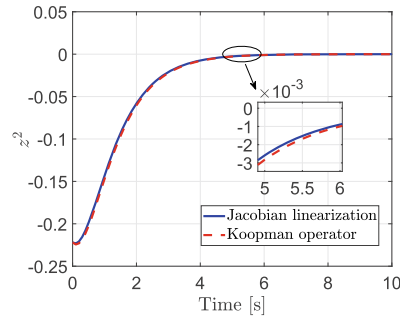
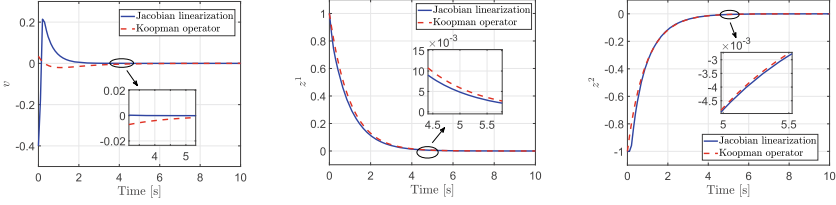
(b) Comparison of dynamic response of the system: v (c) Comparison of dynamic response of the system: z^1 (d) Comparison of dynamic response of the system: z^2

Fig. 18.2 Simulation results of Scenario 18.1 for the initial state $z_0 = [0.774 \ -0.222]^T$ and the prediction horizon $N_p = 15$

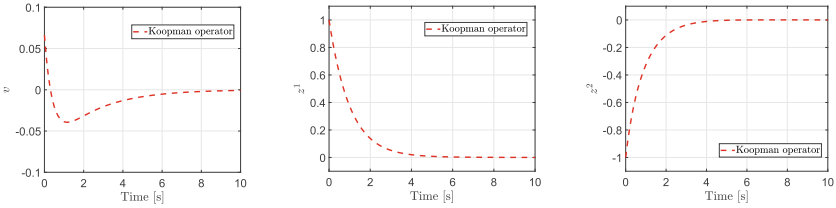
The terminal set $\tilde{\mathbb{Z}}_f$ computed based on the Koopman linear model with the bounded disturbance w_k and the terminal set $\mathbb{Z}_f$ computed based on the Jacobian linearization model are shown in Fig. 18.2a. It can be found that the terminal set $\tilde{\mathbb{Z}}_f$ computed based on the Koopman linear model is larger. Figure 18.2b–d shows the evolution of control input, and the dynamic response of the system at the initial state $z_0 = [0.774 \ -0.222]^T$, while the prediction horizon $N_p = 15$. The dynamic response of the system with the terminal ingredients obtained based on the Koopman linear model is marked as the red solid line, and the dynamic response of the system with the terminal ingredients obtained based on the Jacobian linearization model is marked as the blue dashed line. It can be observed that the dynamic response trajectory of the system can asymptotically converge to the equilibrium point within the control input limitations.

Figure 18.3 shows the dynamic response of the system at the initial state $z_0 = [1 \ -1]^T$, while the prediction horizon $N_p = 10$. Figure 18.3a shows that the control



(a) Comparison of dynamic response of the system: v (b) Comparison of dynamic response of the system: z^1 (c) Comparison of dynamic response of the system: z^2

Fig. 18.3 Simulation results of Scenario 18.1 for the initial state $z_0 = [1 \ -1]^T$ and the prediction horizon $N_p = 10$



(a) Comparison of dynamic response of the system: v (b) Comparison of dynamic response of the system: z^1 (c) Comparison of dynamic response of the system: z^2

Fig. 18.4 Simulation results of Scenario 18.1 for the initial state $z_0 = [1 \ -1]^T$ and the prediction horizon $N_p = 5$

inputs at the initial moment solved by the Problem 18.1 with the terminal ingredients obtained based on the Koopman linear model is smoother than using the terminal ingredients obtained based on the Jacobian linearization model. However, while the prediction horizon $N_p = 5$, Problem 18.1 with the terminal ingredients obtained based on the Jacobian linearization model has no feasible solution at the initial moment. Problem 18.1 with the terminal ingredients obtained based on the Koopman linear model has feasible solution at the initial time instant. Figure 18.4 shows the simulation results of the proposed control scheme for the initial state $z_0 = [1 \ -1]^T$ while $N_p = 5$.

Scenario 18.2 The parameter $\mu = 0.8$.

The collected datasets consist of 25 episodes, each containing data of 610 time steps. The initial state is randomly chosen within the range of $[-1, 1]$, and the control input is allowed within the range of $[-2, 2]$.

The matrices $\tilde{A}_D$, $\tilde{B}_D$ and the bounded disturbance w_k by the Koopman operator theory are calculated as

$$\tilde{A}_D = \begin{bmatrix} 1.0061 & 0.0993 \\ 0.0968 & 1.0130 \end{bmatrix}, \quad \tilde{B}_D = \begin{bmatrix} 0.0840 \\ 0.0875 \end{bmatrix}, \quad \|w_k\|_\infty \leq 0.1.$$

The weighting matrices $\bar{E}$, $\bar{F}$ and the prediction horizon N_p in the optimization problem of MPC are given as

$$\bar{E} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \bar{F} = 1, \quad N_p = 15.$$

Further, the terminal gain K_D and the terminal matrix P_D are calculated by (18.15) offline, that is,

$$K_D = [1.3116 \ 1.3718], \quad P_D = \begin{bmatrix} 10.0617 & 7.4326 \\ 7.4326 & 10.7298 \end{bmatrix}.$$

As a comparison, the matrices A and B by the Jacobian linearization method at the equilibrium point $(0, 0)$ are calculated as

$$A = \begin{bmatrix} 1.005 & 0.1002 \\ 0.1002 & 1.0005 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0841 \\ 0.0841 \end{bmatrix}.$$

Similarly, the terminal gain K_J and the terminal matrix $\bar{G}$ are calculated using Lemma 18.1 as

$$K_J = [1.2901 \ 1.3062], \quad \bar{G} = \begin{bmatrix} 9.6999 & 6.9486 \\ 6.9486 & 9.8201 \end{bmatrix}, \quad \beta = 5.2035.$$

While $\mu = 0.8$, the simulation experimental result is shown in Figs. 18.5 and 18.6. Figure 18.5 shows that the Koopman linear model and the Jacobian linearization model can accurately approximate the nonlinear system (18.18) under Scenario 18.2. Figure 18.4a shows that the proposed method can obtain a larger terminal set under

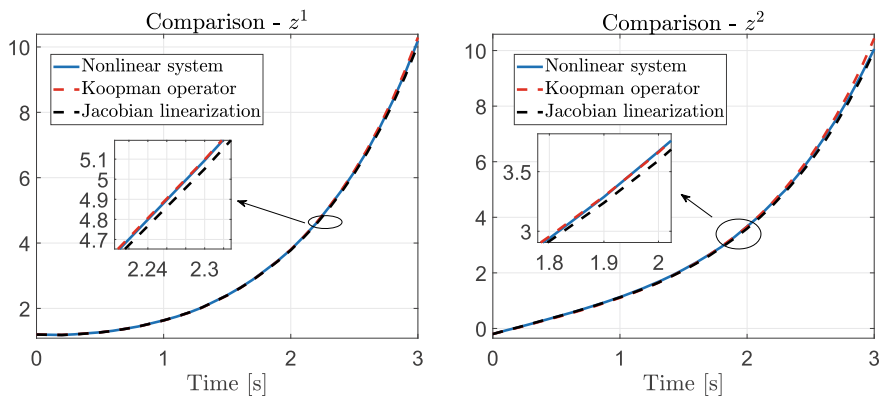
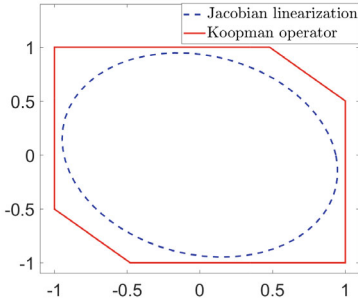


Fig. 18.5 Scenario 18.2 Validation of the Koopman linear model and the Jacobian linearization model ($\mu = 0.8$)



(a) Comparison of the terminal set

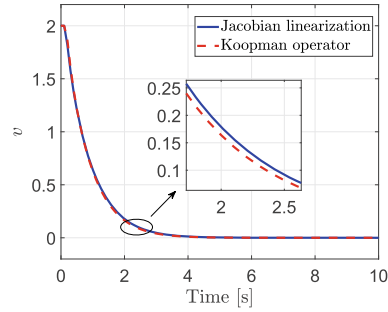
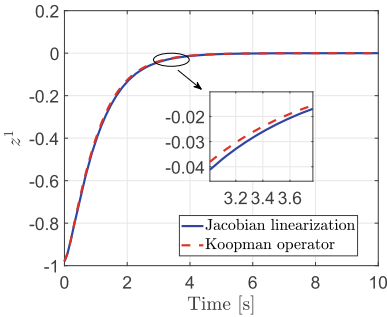
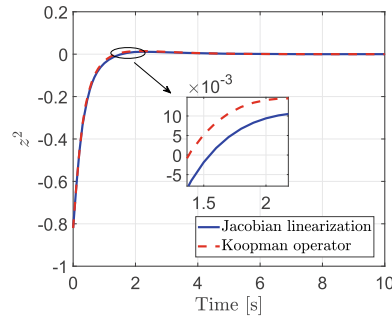
(b) Comparison of dynamic response of the system: v (c) Comparison of dynamic response of the system: z^1 (d) Comparison of dynamic response of the system: z^2

Fig. 18.6 Simulation results of Scenario 18.2 for the initial state $z_0 = [-0.98 \ -0.82]^T$ and the prediction horizon $N_p = 15$

Scenario 18.2. Figure 18.4b–d shows that the dynamic response trajectory of the system can asymptotically converge to the equilibrium point under the control input limitations, where the initial state of the system is set as $z_0 = [-0.98 \ -0.82]^T$.

18.5 Conclusion

In this paper, a scheme to solve the terminal set of model predictive control of nonlinear systems with constraints was proposed. The considered nonlinear system was approximated by a Koopman operator dynamics model, which was a linear system with bounded disturbances. Then, an algorithm to offline determine the terminal cost, terminal set, and terminal control law was developed, where the terminal set was a

maximal robust invariant set of the linear dynamical model with the terminal control law, and the terminal penalty was the related cost function. A simulation example under different scenarios demonstrated the effectiveness of the proposed scheme.

Funding This work was supported by the National Natural Science Foundation of China (No. U62473167) and the Science Foundation of Jilin Province (No.20240402079GH), China.

References

1. Chen, H., Allgöwer, F.: A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. *Automatica* **34**(10), 1205–1217 (1998)
2. Mayne, D.Q., Rawlings, J.B., Rao, C.V., Scokaert, P.O.M.: Constrained model predictive control: stability and optimality. *Automatica* **36**(6), 789–814 (2000)
3. Rajhans, C., Griffith, D., Patwardhan, S.C., Biegler, L.T., Pillai, H.K.: Two approaches for terminal region characterization in discrete time quasi-infinite horizon NMPC. In: 6th IFAC Conference on Nonlinear Model Predictive Control, IFAC, Madison, WI, USA (2018)
4. Rajhans, C., Griffith, D.W., Patwardhan, S.C., Biegler, T., Pillai, H.K.: Terminal region characterization and stability analysis of discrete time quasi-infinite horizon nonlinear model predictive control. *J. Proc. Contr.* **483**, 30–52 (2019)
5. Yu, S., Feng, Y., Kim, J.S., Chen, H.: Computation of terminal set for nonlinear model predictive control. *ACTA Automatica Sinica* **48**(1), 144–151 (2022)
6. Abdulfattokhov, S., Zonon, M., Bemproad, A.: Learning convex terminal costs for complexity reduction in MPC. In: Proc, pp. 2163–2168. Amer. Contr. Conf, Austin, Texas (2021)
7. Abdulfattokhov, S., Zonon, M., Bemproad, A.: Learning Lyapunov terminal costs from data for complexity reduction in nonlinear model predictive control. *Int. J. Robust Nonlinear Control* (2024). <https://doi.org/10.1002/rnc.7411>
8. Mittal, M., Gallieri, M., Quaglino, A., Salehian, S.S.M., Koutnik, J.: Neural Lyapunov model predictive control: Learning safe global controllers from sub-optimal examples. In: [arXiv:2002.10451v2](https://arxiv.org/abs/2002.10451v2) (2021)
9. Moreno-Mora, F., Beckenbach, L., Streif, S.: Predictive control with learning-based terminal costs using approximate value iteration. *IFAC PaperOnLine* **56–2**, 3874–3879 (2023)
10. Milan, K., Igor, M.: Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica* **93**, 149–160 (2018)
11. Zhang, X., Pan, W., Scattolini, R., Yu, S., Xu, X.: Robust tube-based model predictive control with koopman operators. *Automatica* **137**(110), 114 (2022)
12. Yu, S., Hou, C., Qu, T., Chen, H.: A revisit to mpc of discrete-time nonlinear systems. In: 2015 IEEE International Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER), pp 7–12 (2015). <https://doi.org/10.1109/CYBER.2015.7287901>
13. Yu, S., Qu, T., Xu, F., Chen, H., Hu, Y.: Stability of finite horizon model predictive control with incremental input constraints. *Automatica* **79**, 265–272 (2017)
14. Korda, M., Mezić, I.: Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica* **93**, 149–160 (2018)
15. Proctor, J.L., Brunton, S.L., Kutz, J.N.: Dynamic mode decomposition with control (2014). <https://arxiv.org/abs/1409.6358>
16. Feng, Y., Shuyou, Yu.Y., Li, Shi S., Yu, J., Chen, H.: Distributed mpc of vehicle platoons considering longitudinal and lateral coupling. *IEEE Trans. Intell. Transp. Syst.* **25**(3), 2293–2310 (2024)

17. Borrelli, F., Bemporad, A., Morari, M.: Predictive control for linear and hybrid Systems. Cambridge University Press, Cambridge, U.K. (2017)
18. Herceg, M., Kvasnica, M., Jones, C., Morari, M.: Multi-parametric toolbox 3.0. In: Proc. Eur. Control Conf., Zurich, Switzerland (2013). <https://www.mpt3.org/>

Part IV

Optimization

Chapter 19

Optimal Dynamic Pricing in Energy Markets: A Stackelberg Game Approach



Panagiotis D. Grontas, Marta Fochesato, Carlo Cenedese,
Giuseppe Belgioioso, Florian Dörfler, and John Lygeros

Abstract This paper proposes a novel hierarchical energy market design modeling the interactions between a distribution system operator (DSO) and a community of smart prosumers equipped with renewable energy generation and energy storage devices. The DSO sets dynamic energy prices to optimize its revenue, whereas prosumers strategize their energy procurement and the operational schedules of their storage devices to minimize costs. We formalize the problem of designing optimal dynamic energy prices as a large-scale Stackelberg game and deploy BIG Hype, a novel first-order algorithm, to obtain a locally optimal solution. Finally, we show via numerical simulations that the increased flexibility of our market design yields substantial economic advantages to the DSO, and that the proposed solution approach is effective even in scenarios involving a large prosumers base.

Keywords Demand response · Bilevel optimization · Game theory

P. D. Grontas (✉) · M. Fochesato · C. Cenedese · G. Belgioioso · F. Dörfler · J. Lygeros
Automatic Control Laboratory, Department of Electrical Engineering and Information Technology,
ETH Zürich, Zürich, Switzerland
e-mail: pgrontas@ethz.ch

M. Fochesato
e-mail: mfochesato@ethz.ch

C. Cenedese
e-mail: ccenedese@ethz.ch

G. Belgioioso
e-mail: gbelgioioso@ethz.ch

F. Dörfler
e-mail: dorfler@ethz.ch

J. Lygeros
e-mail: jlygeros@ethz.ch

19.1 Introduction

Dynamic pricing refers to an essential demand-side management strategy that charges variable prices over time to induce desirable shifts in the energy consumption patterns. In recent years, this technique has been integrated within the energy markets of numerous European countries, including Denmark, Sweden, Finland, Norway, Spain, Austria, UK, and the Netherlands [1, 2].

Dynamic pricing mutually benefits grid operators and consumers by aligning retail electricity rates with the variable costs of energy generation needed to meet demand [3]. Under a flat pricing model, grid operators are forced to maintain excessive, often non-renewable, power reserves to accommodate peak usage times. This approach not only increases wholesale prices but also leads to inefficient use of the energy system [4]. Conversely, dynamic pricing adjusts retail rates based on wholesale costs and overall demand, effectively passing some price volatility to the consumers. This encourages consumers to shift their consumption to times when energy is cheaper, thereby lowering their energy costs and promoting a more balanced and efficient energy consumption pattern.

One benefit of dynamic pricing is peak-shaving of the overall demand profile. The resulting decrease in variability of the demand allows the grid operator to reduce inefficiency, and size correctly the grid capacity. Moreover, it fosters the use of renewable energy sources that, because of their unpredictability, cannot be employed efficiently in energy networks with high-demand peaks [5].

The flexible consumption profile of energy assets such as electric vehicles and heat pumps makes them highly responsive to dynamic pricing models [6, 7]. Instead, other energy assets, including some domestic appliances, are not interruptible. Therefore, the full potential of dynamic pricing can be achieved when energy storage devices are in place. These systems enable customers to meet their inflexible demand while shifting and shedding the flexible one, aligning energy usage with periods of lower prices.

In this paper, we focus on a local electricity market comprising a Distribution System Operator (DSO), responsible for setting a dynamic retail electricity price, and a population of energy prosumers. For simplicity, we assume that the latter are smart buildings equipped with on-site renewable energy generation and storage devices. The strategic interactions between DSO and smart buildings establish a hierarchical relationship that we model as a Stackelberg game [8], with the DSO as the leader and the prosumers as the followers.

The common approach for computing equilibria of Stackelberg games is to formulate them as Mathematical Programs with Equilibrium Constraints (MPECs). These can be tackled using various relaxation techniques [9], or be transformed into mixed-integer programs through the Big-M reformulation [10], enabling their solution via off-the-shelf solvers. However, both approaches suffer from scalability issues, making them impractical for realistic applications with a large number of decision variables. This motivated the development of ad hoc solution approaches, specifically designed for particular subclasses of Stackelberg games. For the sub-

class of aggregative quadratic Stackelberg games, the authors in [11] developed a semi-decentralized algorithm that combines a relaxation technique with iterative convexification. More recently, [12] proposed a first-order-type algorithm inspired by implicit differentiation techniques from machine learning [13]. Both algorithms offer enhanced scalability over MPEC methods, at the price of yielding only locally optimal solutions. Crucially, their applicability is limited to scenarios where the leader's decision enters linearly in the lower level problem, reducing the scope of potential leader interventions.

The main contributions of this paper are as follows:

- (i) We propose a dynamic pricing model that grants a larger intervention flexibility to the DSO via a more general parametrization of the price map. This model assumes little about the DSO's objectives, thereby allowing for a wide array of supervisory objectives, ranging from revenue maximization to promoting fairness. Further, it empowers the DSO with direct control over grid capacity distribution among prosumers, enabling a more reliable and targeted management of energy resources.
- (ii) We formalize the problem of optimally designing the parameters of the pricing map and the grid capacity distribution as a Stackelberg game. To address this, we deploy the BIG Hype algorithm [14]—a novel scalable first-order-type method. This approach yields a scalable market mechanism that leverages the problem's hierarchical and distributed nature. Moreover, it can be readily extended to incorporate further enhancements, including incentives for flexibility or rebound counteractions.
- (iii) We illustrate the merits of our market design via numerical simulations with a community of 50 prosumers with realistic energy assets and consumption profiles. Our results indicate that the enhanced design flexibility translates to considerable economic gains for a profit-oriented DSO. Moreover, the proposed market mechanism offers, as a by-product, valuable insights into the prosumers' behavior in response to variable energy prices.

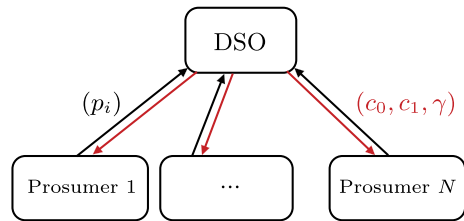
The remainder of this paper is organized as follows. In Sect. 19.2, we formulate the considered hierarchical energy market model, while in Sect. 19.3 we describe a first-order algorithm to solve it. We present numerical results in Sect. 19.4 and conclude the paper in Sect. 19.5.

19.2 Energy Market Model

We consider a distribution network where a DSO determines the price of the electricity sold to a community of N prosumers, indexed by $i \in \mathcal{N} := \{1, \dots, N\}$. To satisfy their energy demand, each prosumer can procure electricity from the DSO as well as from their local assets, assumed to be an electric storage facility and photovoltaic (PV) panels. We consider a planning horizon with energy trading taking place over T time intervals of equal length $\Delta\tau > 0$, indexed by $\tau \in \mathcal{T} := \{0, \dots, T\}$. In the

Table 19.1 Main variables of the energy market model

DSO	
$c_1, c_0 \in \mathbb{R}_{\geq 0}^T$	Price map coeff.
$\gamma \in \mathbb{R}_{\geq 0}^N$	Grid capacity allocation
Prosumers	
$d_i \in \mathbb{R}_{\geq 0}^T$	Power demand
$p_i \in \mathbb{R}_{\geq 0}^T$	Power purchased
$e_i \in \mathbb{R}_{\geq 0}^T$	Battery state of charge
$p_i^c \in \mathbb{R}_{\geq 0}^T$	Battery charging power
$p_i^{dc} \in \mathbb{R}_{\geq 0}^T$	Battery discharging power

Fig. 19.1 Graphical representation of energy market model

following, we assume that DSO and prosumers aim at minimizing their individual operational costs by strategizing on the dynamic price of electricity and the profile of energy procurement, respectively. The emerging market structure is inherently hierarchical, with the DSO (upper level) decision depending on the (lower level) decisions of the prosumers.

The key variables of the proposed market structure are described in the next sections and summarized in Table 19.1, together with a graphical representation in Fig. 19.1.

19.2.1 Lower Level: Prosumers' Decision Process

Each prosumer $i \in \mathcal{N}$ aims at meeting their daily electricity demand profile d_i at the minimum monetary cost possible by optimally determining their electricity procurement profile p_i , and the operational profile of the energy storage facility, namely, the battery charging (discharging) schedule p_i^c (p_i^{dc}). The energy cost depends on the energy price profile q , which is dynamically set by the DSO as

$$q \left(\sum_{j \in \mathcal{N}} p_j \right) := C_1 \left(\sum_{j \in \mathcal{N}} p_j \right) + c_0, \quad (19.1)$$

where $\sum_{j \in \mathcal{N}} p_j$ is the aggregate energy procurement profile, and $c_0 \in \mathbb{R}_{\geq 0}^T$, $C_1 = \text{diag}(c_1)$, with $c_1 \in \mathbb{R}_{\geq 0}^T$, are parameters of the pricing mapping (19.1) set by the DSO. Overall, the daily cost incurred by prosumer i is given by

$$J_i := q \left(\sum_{j \in \mathcal{N}} p_j \right)^\top p_i + \delta_i \left(\|p_i^c\|^2 + \|p_i^{\text{dc}}\|^2 \right),$$

where the first term is the procurement cost for purchasing p_i units of energy and the second term is a cost associated with battery degradation due to charging/discharging, with δ_i being a positive constant used to weigh the two terms.

The energy procurement and charging/discharging profiles must satisfy the following operational constraints, for all $\tau \in \mathcal{T}$:

$$p_{i,\tau} + p_{i,\tau}^{\text{dc}} + s_{i,\tau} = d_{i,\tau} + p_{i,\tau}^c, \quad (19.2a)$$

$$e_{i,\tau} = e_{i,\tau-1} + \Delta\tau(\eta^c p_{i,\tau}^c - \eta^{\text{dc}} p_{i,\tau}^{\text{dc}}), \quad (19.2b)$$

$$0 \leq p_{i,\tau}^c \leq p_i^{\max}, \quad (19.2c)$$

$$0 \leq p_{i,\tau}^{\text{dc}} \leq p_i^{\max}, \quad (19.2d)$$

$$0 \leq e_{i,\tau} \leq e_i^{\max}, \quad (19.2e)$$

$$0 \leq p_{i,\tau}. \quad (19.2f)$$

Equation (19.2a) represents the local power balance, with $s_{i,\tau}$ being the renewable energy production from the PV panels at time τ ; (19.2b) represents the evolution of the state of charge $e_{i,\tau}$ of the energy storage device, with $\eta^c, \eta^{\text{dc}} \in (0, 1)$ being the respective charging and discharging efficiency; and (19.2c)–(19.2e) model physical limits on the charging/discharging and state of charge of the energy storage device. Constraint (19.2f) ensures that prosumers can only buy energy from the grid and we impose it for the sake of model simplicity. To ensure feasibility, we assume that the local energy production from the PV panels falls short of the local demand and as in [15], i.e., $d_{i,\tau} - s_{i,\tau} \geq 0$. The exploration of sell-back schemes accommodating $p_{i,\tau} < 0$ is left for future investigation.

To maintain the distribution grid's operational reliability, the energy procurement of each prosumer is constrained by

$$p_i \leq \gamma_i g, \quad (19.3)$$

where g represents the total capacity of the grid and γ_i specifies the fraction of this capacity allocated to prosumer i .

To streamline notation, we collect all the decision variables of prosumer $i \in \mathcal{N}$ in the vector¹ $y_i := (p_i, p_i^c, p_i^{\text{dc}})$, the collective decisions of all prosumers in the stacked vector $\mathbf{y} = (y_1, \dots, y_N)$, the aggregate decision of all prosumer as $\sigma(\mathbf{y}) =$

¹ The variables $e_{i,\tau}$ can be eliminated by inverting the state-of-charge dynamics in (19.2b), hence it is not included in the decision vector y_i .

$\sum_{i \in \mathcal{N}} y_i$, and the relevant parameters controlled by the DSO in the vector $x := (c_0, c_1, (\gamma_i)_{i \in \mathcal{N}})$.

With this notation in place, the scheduling problem faced by each prosumer $i \in \mathcal{N}$ can be formally cast as

$$(\forall i \in \mathcal{N}) : \begin{cases} \text{minimize} & J_i(x, y_i, \sigma(y)) \\ \text{subject to} & y_i \in \mathcal{Y}_i(x), \end{cases} \quad (19.4)$$

where the local feasible set $\mathcal{Y}_i$ is defined as

$$\mathcal{Y}_i(x) := \{y_i \in \mathbb{R}_{\geq 0}^{3T} \mid (2) - (3) \text{ hold}\}, \quad (19.5)$$

and the dependence of both the cost function J_i and operational constraints $\mathcal{Y}_i$ on the pricing and grid allocation parameters in x is made explicit. Moreover, since J_i depends on the decisions of the other prosumers, through the aggregate quantity $\sigma(y)$, the inter-dependent problems (19.4) describing the collective decision of the prosumers constitute a parametric aggregative game [16].

Remark 19.1 The local feasible set $\mathcal{Y}_i(x)$ in (19.4) is polyhedral and thus can be cast in the general form:

$$\mathcal{Y}_i(x) = \left\{ y_i \in \mathbb{R}^{3T} \mid \begin{cases} A_i y_i \leq b_i + G_i x, \\ C_i y_i = f_i + H_i x \end{cases} \right\}, \quad (19.6)$$

where $A_i, G_i, C_i, H_i, b_i, d_i$ are defined in Appendix 19.5.1.

19.2.2 Upper Level: DSO Decision Process

The goal of the DSO is to maximize the revenue from selling electricity to the community of prosumers by strategically designing the electricity price. In practice, the DSO can design the energy price by modulating the parameters c_0 and c_1 of the price mapping $q(\cdot)$ in (19.1). Given the monopolistic nature of the proposed market structure, we assume that the pricing parameters c_0 and c_1 adhere to a pre-established bilateral agreement between DSO and the prosumers. To accurately represent these agreements within our model, we introduce the following constraints, for all $\tau \in \mathcal{T}$, $j = 0, 1$:

$$\underline{c_j} \leq c_{j,\tau} \leq \overline{c_j}, \quad (19.7a)$$

$$\frac{1}{T} \sum_{\tau \in \mathcal{T}} c_{j,\tau} \leq \frac{\underline{c_j} + \overline{c_j}}{2}, \quad (19.7b)$$

where $0 \leq \underline{c}_j \leq \overline{c}_j$. Constraint (19.7a) limits each $c_{j,\tau}$ within a specified range, while (19.7b) prevents the DSO from exploiting its monopoly position by consistently charging the maximum energy price.

Remark 19.2 The constraints (19.7a) and (19.7b) are just one instance of a reasonable bilateral contract between the DSO and the prosumers. In fact, our framework is compatible with any contract prescribed by affine constraints, e.g., bounds on the rate of change of the price to prevent abrupt fluctuations.

Further, to maintain the operational reliability and integrity of the grid the DSO must set the capacity fractions γ_i in (19.3) such that the total electricity procurement does not surpass the physical capacity of the grid, namely, $\sum_{i \in \mathcal{N}} p_i \leq g$. This condition is prescribed by the following constraints:

$$\gamma_i \in [\underline{\gamma}, 1], \quad \forall i \in \mathcal{N} \quad (19.8a)$$

$$\sum_{i \in \mathcal{N}} \gamma_i = 1. \quad (19.8b)$$

In (19.8a), $\underline{\gamma} \in (0, 1)$ is a constant ensuring that each prosumer is given access to sufficient fraction of the grid capacity to satisfy its energy requirements. The fraction γ_i is fixed throughout the time horizon; a time-varying formulation is also possible, but would provide the DSO with excessive control over the prosumers' energy procurement profile.

Remark 19.3 Since the DSO designs the γ_i 's solely based on revenue maximization, the resulting grid capacity allocation might be unfair toward prosumers, i.e., this allocation is agnostic to prosumers' energy needs. Such concerns can readily be mitigated by including a term to the DSO's cost that promotes fairness, e.g., allocation should be approximately proportional to energy demands. We leave such extensions for future work.

The DSO objective is minimizing the negated revenue

$$\varphi(\sigma(y)) := -q \left(\sum_{i \in \mathcal{N}} p_i \right)^\top \sum_{i \in \mathcal{N}} p_i. \quad (19.9)$$

Moreover, the feasible set of the DSO decision vector is

$$\mathcal{X} := \{x \in \mathbb{R}^{2T+N} \mid (19.7) - (19.8) \text{ hold}\}. \quad (19.10)$$

19.2.3 Stackelberg Game Formulation

The proposed energy market model operates as follows: the DSO communicates the pricing mapping to the prosumers, who in turn update their procurement and energy storage profiles as the solution of the game in (19.4). We adopt the Nash equilibrium

as the solution concept of (19.4). This is a set of feasible strategy profiles where no prosumer can reduce its cost by unilaterally deviating from it.

Definition 19.1 Given x , a strategy profile $\mathbf{y}^* \in \mathcal{Y}(x) := \prod_{i \in \mathcal{N}} \mathcal{Y}_i(x)$ is a Nash equilibrium (NE) of (19.4) if:

$$J_i(x, y_i^*, \sigma(\mathbf{y}^*)) \leq J_i(x, y_i, \sigma((y_i, \mathbf{y}_{-i}^*))),$$

for all strategies $y_i \in \mathcal{Y}_i(x)$, and for all $i \in \mathcal{N}$. $\square$

It follows by [17, Prop. 1.4.2] that a strategy profile $\mathbf{y}^*$ is an NE of (19.4) if and only if it is a solution to the following parametric Variational Inequality (VI):

$$F(x, \sigma(\mathbf{y}^*))^\top (\mathbf{y} - \mathbf{y}^*) \geq 0, \quad \forall \mathbf{y} \in \mathcal{Y}(x), \quad (19.11)$$

where $F(x, \sigma(\mathbf{y})) := (\nabla_{y_i} J_i(x, y_i, \sigma((y_i, \mathbf{y}_{-i})))_{i \in \mathcal{N}}$ is the so-called pseudo-gradient.

The next statement leverages this connection to prove the existence and uniqueness of an NE for the parametric aggregative game in (19.4).

Lemma 19.1 *For any fixed DSO decision vector $x \in \mathcal{X}$, the parametric game (19.4) admits a unique NE.* $\square$

The proof is based on the fact that the objective functions J_i are strongly convex with respect to y_i and follows closely [18, App. A], hence it is omitted here in the interest of space.

In view of Lemma 19.1, we know that the set of solutions to (19.11), denoted as $\text{SOL}(F(x, \cdot), \mathcal{Y}(x))$, is a singleton. Therefore, we can express the DSO's pricing design problem as the following single-leader multi-follower Stackelberg game:

$$\begin{array}{ll} \underset{x}{\text{minimize}} & \varphi(\sigma(\mathbf{y})) \end{array} \quad (19.12a)$$

$$\text{subject to} \quad x \in \mathcal{X} \quad (19.12b)$$

$$\mathbf{y} = \text{SOL}(F(x, \cdot), \mathcal{Y}(x)). \quad (19.12c)$$

The dependence of φ on $\mathbf{y}$ highlights that the DSO anticipates the rational response of the prosumers to x .

19.3 Methodology

One could theoretically pursue the global optimum of (19.12) using mixed-integer programming techniques combined with off-the-shelf solvers [15]. However, the computational burden associated with this approach increases dramatically with the problem size, making it impractical for handling large problems effectively [14]. Motivated by these scalability issues, we focus instead on first-order-type algorithms

[12, 14, 19] that obtain locally optimal solutions within reasonable computational times. In this paper, we adopt BIG Hype [14], a recently proposed algorithm with attractive computational performance, scalability properties, and convergence guarantees. The core idea of BIG Hype is to exploit uniqueness of the lower level equilibrium to write (19.12) as

$$\underset{x \in \mathcal{X}}{\text{minimize}} \quad \varphi(\sigma(\mathbf{y}^*(x))) =: \hat{\varphi}(x), \quad (19.13)$$

where the equilibrium constraint (19.12b) is eliminated using the implicit, single-valued, parameter-to-NE mapping $\mathbf{y}^*(\cdot) : x \mapsto \text{SOL}(F(x, \cdot), \mathcal{Y}(x))$. Then, a local solution to (19.13) can be found using a first-order method of the form:

$$x^{k+1} = \mathbb{P}_{\mathcal{X}} [x^k - \alpha^k \text{"}\nabla\text{"}\hat{\varphi}(x^k)], \quad (19.14)$$

where the operator “ ∇ ” is used instead of the standard gradient since the objective $\hat{\varphi}(\cdot)$ is non-differentiable and non-convex. Whenever $\hat{\varphi}(\cdot)$ is differentiable at x , an expression for the operator “ ∇ ” $\hat{\varphi}$, typically known as *hypergradient*, can be obtained by applying the chain rule, yielding

$$\text{"}\nabla\text{"}\hat{\varphi}(x) = \mathbf{J}\mathbf{y}^*(x)^\top (\mathbf{1}_N \otimes I) \nabla \varphi(\sigma(\mathbf{y}^*(x))). \quad (19.15)$$

A similar expression for “ ∇ ” $\hat{\varphi}$ can be derived even if $\hat{\varphi}(\cdot)$ is not differentiable at x , where the standard Jacobian $\mathbf{J}\mathbf{y}^*(x)$ is replaced with an element of a *conservative Jacobian* [20]. We refer the interested reader to [14] for formal analysis of this case in the context of hypergradient descent.

Algorithm 19.1 Distributed hypergradient descent

Parameters: Step sizes $\{\alpha^k\}_{k \in \mathbb{N}}$, tolerances $\{\sigma^k\}_{k \in \mathbb{N}}$.

Initialization: $k \leftarrow 0$, $x^k \in \mathcal{X}$, $\mathbf{y}^k \in \mathbb{R}^n$, $s^k \in \mathbb{R}^{m \times n}$.

Iterate until convergence:

(DSO)

Energy price and grid allocation update:

$$x^{k+1} = \mathbb{P}_{\mathcal{X}} [x^k - \alpha^k \sigma(s^k)^\top \nabla \varphi(\sigma(\mathbf{y}^k))]$$

(Prosumers)

Operational profile and sensitivity updates:

$$(\mathbf{y}^{k+1}, s^{k+1}) = \text{Algorithm 19.2}(x^{k+1}, \mathbf{y}^k, s^k, \sigma^k)$$

$k \leftarrow k + 1$

To evaluate the hypergradient (19.15), the DSO must compute the NE $\mathbf{y}^*(x)$ induced by x , as well as its Jacobian $\mathbf{J}\mathbf{y}^*(x)$ describing the NE sensitivity to changes in x . Remarkably, both quantities can be estimated simultaneously and in a distributed manner by leveraging the aggregative structures within (19.12). Following [14], the NE $\mathbf{y}^*(x)$ can be estimated using local updates of the form:

$$(\forall i \in \mathcal{N}) : \quad \tilde{\mathbf{y}}_i^{\ell+1} = h_i(x, \tilde{\mathbf{y}}^\ell), \quad (19.16)$$

where $h_i(x, \cdot)$ is a projected gradient mapping defined as

$$h_i(x, \mathbf{y}) = \mathbb{P}_{\mathcal{Y}_i(x)}[y_i - \beta F_i(x, \mathbf{y})], \quad (19.17)$$

and $F_i(x, \mathbf{y}) = \nabla_{y_i} J_i(x, y_i, \sigma(\mathbf{y}))$ is the i -th component of the pseudo-gradient (19.11). For any step size $\beta > 0$ small enough, the sequence generated by (19.17), for all $i \in \mathcal{N}$, converges linearly to $\mathbf{y}^*(x)$.

To estimate $\mathbf{J}\mathbf{y}^*(x)$, the prosumers can locally run updates of the form:

$$(\forall i \in \mathcal{N}) : \quad \tilde{s}_i^{\ell+1} = S_{2,i} \tilde{\mathbf{s}}^\ell + S_{1,i}, \quad (19.18)$$

where $S_{2,i} = \mathbf{J}_2 h_i(x, \tilde{\mathbf{y}}^\ell)$ and $S_{1,i} = \mathbf{J}_1 h_i(x, \tilde{\mathbf{y}}^\ell)$ are the partial Jacobians of (19.17) with respect to $\mathbf{y}$ and x , respectively. In practice, computing these matrices requires differentiating through the projection operator $\mathbb{P}_{\mathcal{Y}_i(\cdot)}[\cdot]$ in (19.17). From a computational perspective, this operation amounts to solving systems of linear equations of the form (19.19), as explained in [13]. A numerical procedure for obtaining $S_{2,i}$ and $S_{1,i}$ is summarized in Algorithm 19.3 for completeness. By using the implicit function theorem, we can show that the sequence generated by (19.18) converges to some matrices $(s_i^*)_{i \in \mathcal{N}}$, where s_i^* is the i -th row block of $\mathbf{J}\mathbf{y}^*(x)$.

The BIG Hype algorithm applied to our market model is summarized in Algorithm 19.1, the distributed subroutine for evaluating $\mathbf{y}^*(\cdot)$ and $\mathbf{J}\mathbf{y}^*(\cdot)$ in Algorithm 19.2, and the numerical procedure to obtain the local PG Jacobians in Algorithm 19.3. The following statement establishes convergence of Algorithm 19.1 to a critical point of (19.13) under the following design choices:

Algorithm 19.2 Nash equilibrium and sensitivity learning

Parameters: step size β , termination constant η .

Input: $x, \mathbf{y}, s, \sigma$.

Initialization: $\ell \leftarrow 0, \tilde{s}^\ell = s, \tilde{\mathbf{y}}^\ell = \mathbf{y}$,

Iterate until termination

For all prosumers $i \in \mathcal{N}$:

- Projected-gradient (PG) update:

$$\tilde{y}_i^{\ell+1} = h_i(x, \tilde{y}^\ell)$$
- Jacobians update:

$$(S_{1,i}, S_{2,i}) = \mathbf{Algorithm\ 19.3}(x, \tilde{y}^{\ell+1})$$
- Sensitivity update:

$$\tilde{s}_i^{\ell+1} = S_{2,i}\tilde{s}^\ell + S_{1,i}$$

termination =

$$\max \{(\eta)^\ell, \sum_{j=0}^{\ell} (\eta)^{\ell-j} \|\tilde{y}^{j+1} - \tilde{y}^j\|\} \leq \sigma$$

$\ell \leftarrow \ell + 1$

Output: $\bar{y} = \tilde{y}^\ell, \bar{s} = \tilde{s}^\ell$.

$$\begin{bmatrix} I & A_i^\top & C_i^\top \\ \text{diag}(\lambda^*)A_i & \text{diag}(b_i + G_i x - A_i z^*) & 0 \\ C_i & 0 & 0 \end{bmatrix} \begin{bmatrix} dz \\ d\lambda \\ dv \end{bmatrix} = \begin{bmatrix} dy \\ \text{diag}(\lambda^*)G_i dx \\ H_i dx \end{bmatrix},$$

with A_i, C_i, G_i, H_i, b_i as in Appendix A. (19.19)

- C1) The step size sequence $\{\alpha^k\}_{k \in \mathbb{N}}$ in Algorithm 19.1 is non-negative, non-summable, and square summable.
- C2) The tolerance sequence $\{\sigma^k\}_{k \in \mathbb{N}}$ in Algorithm 19.1 is non-negative, and satisfies $\sum_{k=0}^{\infty} \alpha^k \sigma^k < \infty$ along with the technical preconditions of [14, Th. 1].
- C3) The step size β in Algorithm 19.2 is sufficiently small.
- C4) The constant $\eta \in [0, 1)$ in the termination condition of Algorithm 19.2 is an upper bound on the contraction rate of (19.16). In practice, this can be either analytically computed or numerically estimated.

Algorithm 19.3 PG Jacobian computation

Input: (x, y) .

2. $(z^*, \lambda^*, v^*) \leftarrow$ primal-dual solution to $h_i(x, y)$
3. $(G_1, \sim, \sim) \leftarrow$ solve (19.19) with $dx = I$ and $dy = 0$
4. $(G_2, \sim, \sim) \leftarrow$ solve (19.19) with $dx = 0$ and $dy = I$
5. $S_1 = G_1 - \gamma G_2 \mathbf{J}_1 F_i(x, y)$
6. $S_2 = G_2(I - \gamma \mathbf{J}_2 F_i(x, y))$

Output: S_1, S_2 .

Proposition 19.1 *Under the design choices (C1)–(C4) and the technical condition in [14, Assumption 3], any limit point of the sequence $\{x^k\}_{k \in \mathbb{N}}$ generated by Algorithm 19.1 is a critical point of the problem (19.13). $\square$*

Proof The proof follows [18, Proposition 1] and amounts to verifying that Standing Assumptions 1–4 in [14] hold for the market model presented in Sect. 19.2.

Note that any local minimum of (19.13) is a critical point of (19.13) by [20, Proposition 1].² Hence, the limit points of Algorithm 19.1 only satisfy a necessary condition for local optimality and could potentially correspond to spurious points, such as saddle-points. Yet, we observe in practice, and illustrate via numerical simulations in the next section, that this is enough to ensure satisfactory results.

19.4 Numerical Simulations

In this section, we demonstrate the proposed market model via numerical studies with a community of 50 prosumers from 9 classes with distinct features, such as energy demand and energy storage size. For model parameters such as the demands d_i and grid capacity g , we use the values in [15]. The length of each time interval is $\Delta\tau = 1$ hour and we consider a planning horizon of $T = 24$ hours. The code used for the simulations along with a general implementation of BIG Hype is available online at https://github.com/p-grontas/BIG_hype_algorithm. All simulations run on a MacBook Air with 16 GB of RAM. With this setup, Gurobi [21], a state-of-the-art commercial mixed-integer solver, requires more than 6 hours to solve a scenario involving merely 2 prosumers. By contrast, the BIG Hype algorithm consistently achieves convergence in under 1 minute for substantially larger scenarios with 50 prosumers.

In our first numerical study, we compare the DSO's revenue under three different pricing schemes q in (19.1):

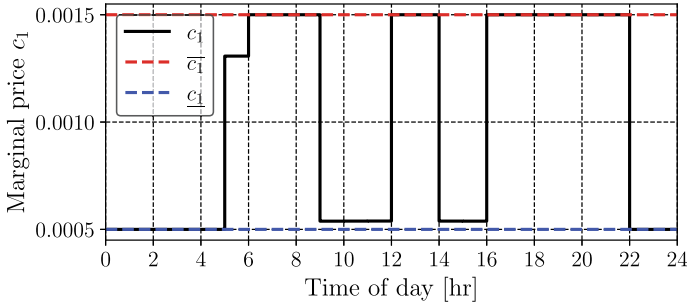
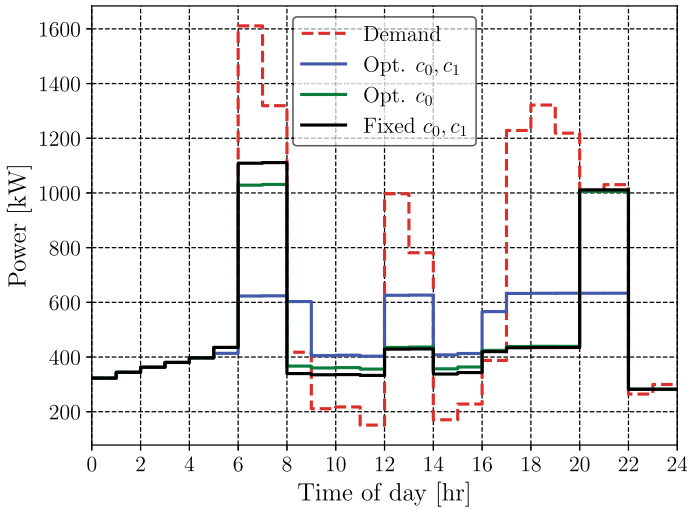
- (i) Both c_0 and c_1 are fixed a priori according to a day–night tariff, where the minimum (resp. maximum) price is applied during the night (resp. day). Specifically, we apply the day tariff between 08:00 and 20:00.
- (ii) The parameter c_0 is optimized via Algorithm 19.1, whereas the parameter c_1 is fixed as described in (i).
- (iii) Both parameters c_0 and c_1 are optimized via Algorithm 19.1.

We set the price bounds as $c_0 = 0.05$, $\overline{c_0} = 0.1$, $c_1 = 0.0005$, $\overline{c_1} = 0.0015$. The market outcome, summarized in Table 19.2, suggests that dynamic pricing, (ii), and especially dynamic marginal pricing, (iii), can significantly increase the DSO's revenue compared to using (i). This is a consequence of the tailored price profile, shown in Fig. 19.2 for c_1 , closely matching the demand, as illustrated Fig. 19.3.

² A local minimum of (19.13) is guaranteed to exist since $\mathbf{y}^*(\cdot)$ is continuous (see [14]), thus $\hat{\varphi}(\cdot)$ is also continuous, and $\mathcal{X}$ is compact.

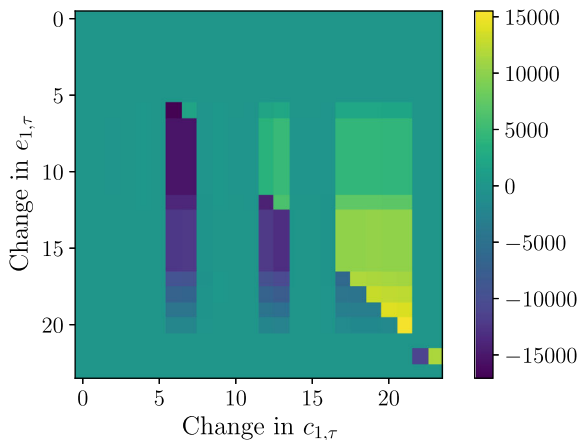
Table 19.2 DSO's revenue for different pricing schemes

	Fixed c_0, c_1	Optimized c_0	Optimized c_0, c_1
Revenue [\$]	6267	6404	8098


Fig. 19.2 Profile of the marginal price c_1 over $\mathcal{T}$ and its upper and lower bounds, $\bar{c}_1$ and $\underline{c}_1$, respectively

Fig. 19.3 Total purchased power for pricing schemes (i)–(iii) (solid lines) and the net demand $\sum_{i \in \mathcal{N}} (d_i - s_i)$ profile (red dashed line)

Beyond financial benefits for the DSO, our dynamic pricing mechanism induces a considerable peak-shaving effect. In Fig. 19.3, we present the total net demand $\sum_{i \in \mathcal{N}} (d_i - s_i)$ of the community for each hour of the day, along with the total purchased energy $\sum_{i \in \mathcal{N}} p_i$ for each of the three pricing schemes. We observe that optimizing both c_0 and c_1 promotes a valley-filling effect and reduces peak energy demand by roughly 40% compared to schemes (i) and (ii). Interestingly, under (iii),

Fig. 19.4 Sensitivity of the state of charge e_1 with respect to the price parameter c_1 , at each time interval $\tau \in \mathcal{T}$



prosumers purchase more energy during periods of low demand compared to (ii). Moreover, Fig. 19.3 shows that having a dynamic c_0 produces an effect that is qualitatively similar to (i), whereas allowing the DSO to design also c_1 generates a substantial change in how prosumers purchase energy, especially during high-demand periods. This emphasizes the benefit of using the devised market scheme.

Remarkably, the BIG Hype algorithm also allows us to study the sensitivity of the community's behavior to energy prices, thus enhancing the interpretability of the market outcome. In Fig. 19.4, we illustrate $\mathbf{J}_{c_1} e_1$ which is the sub-block of the sensitivity $\mathbf{J} \mathbf{y}^*(x)$ in (19.15). It intuitively represents the change of state of charge e_1 of prosumer 1's energy storage device induced by a unit change of the marginal price c_1 . The (i, j) -th pixel in Fig. 19.4 represents $[\mathbf{J}_{c_1} e_1]_{i,j} = \frac{\partial e_{1,i}}{\partial c_{1,j}}$, namely, the derivative of $e_{1,i}$ with respect to $c_{1,j}$. For example, $\frac{\partial e_{1,17}}{\partial c_{1,17}}$ has a large negative value since an increase in price at time 17 will motivate the prosumer to deplete their battery at that time instead of purchasing power. Instead, $\frac{\partial e_{1,i}}{\partial c_{1,17}}$ is positive for $i < 17$, which means that the prosumer opts to keep its battery charged in anticipation of the increased electricity price at time 17. This behavior is observed only during high-demand periods since the effect of c_1 on the price during those periods is the highest.

In Fig. 19.5, we illustrate $\mathbf{J}_\gamma p_1$, namely, the change in energy procurement of prosumer 1 induced by a change of grid capacity allocation γ_i . We notice that the effect of γ_i , for $i \in [2, 50]$, is minimal, namely, prosumer 1 is virtually not affected by the grid capacity percentage allocated to other agents. On the other hand, an increase in local capacity allocation γ_1 motivates prosumer 1 to shift its energy procurement profile to periods of low demand.

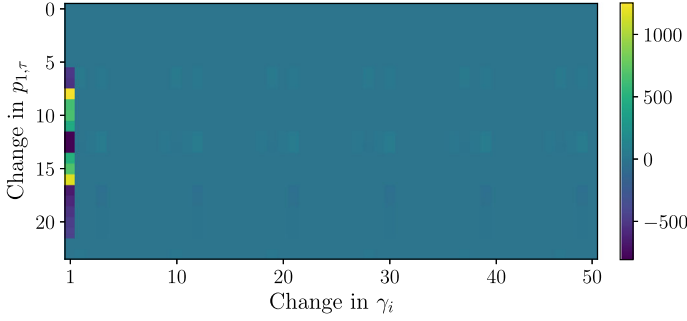


Fig. 19.5 Sensitivity of p_1 with respect to the grid capacity percentage γ_i allocated to each agent $i \in \mathcal{N}$

19.5 Conclusion

This paper presents a scalable solution to the optimization of dynamic pricing schemes in hierarchical energy markets, through the implementation of the BIG Hype algorithm. Our approach demonstrates significant improvements in DSO revenue and operational efficiency, paving the way for advanced pricing strategies and deeper insights into prosumers' behavior. Applying our methodology in practice would require utilizing real-world data to estimate the lower level prosumer parameters. This could be achieved in an end-to-end fashion using gradient-based techniques similar to BIG Hype, as discussed in [19]. Future research will expand on such practical issues, and explore the design of incentive schemes for flexibility provision and fair strategies for grid capacity allocation.

19.5.1 Polyhedral Description of $\mathcal{Y}_i(x)$

We denote by $\mathbf{0}_{n_r \times n_c} \in \mathbb{R}^{n_r \times n_c}$ (resp. $\mathbf{1}_{n_r \times n_c}$) the matrix of all zeros (resp. ones); we omit the subscript when clear from the context. We let $Z := [I_T - I_T]^\top \in \mathbb{R}^{2T \times T}$ and define

$$L := \Delta \tau \cdot \begin{bmatrix} 1 & 0 & \dots & 0 \\ 1 & 1 & \dots & 0 \\ 1 & 1 & \dots & 1 \end{bmatrix} \in \mathbb{R}^{T \times T}.$$

Then, for the inequality constraints we define

$$A_i := \begin{bmatrix} Z & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & Z & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & Z \\ \mathbf{0} & \eta^c L & -\eta^{dc} L \\ \mathbf{0} & -\eta^c L & \eta^{dc} L \end{bmatrix}, \quad b_i := \begin{bmatrix} \mathbf{0}_{2T} \\ p_i^{\max} \mathbf{1}_T \\ \mathbf{0}_T \\ p_i^{\max} \mathbf{1}_T \\ \mathbf{0}_T \\ (e_i^{\max} - e_{i,0}) \mathbf{1}_T \\ e_{i,0} \mathbf{1}_T \end{bmatrix},$$

$$G_i := \begin{bmatrix} \mathbf{0}_{T \times (2T+i-1)} & g & \mathbf{0}_{T \times (N-i)} \\ & \mathbf{0}_{7T \times (2T+N)} & \end{bmatrix}.$$

Similarly, for the equality constraints we let $C_i := [I_T - I_T \ I_T]$, $f_i := s_i - d_i$, $H_i := \mathbf{0}_{T \times (2T+N)}$.

Disclosure of Interest The authors have no competing interests to declare that are relevant to the content of this article.

Acknowledgements This work was supported by NCCR Automation, a National Centre of Competence in Research, funded by the Swiss National Science Foundation (grant number 180545).
Disclosure of Interest The authors have no competing interests to declare that are relevant to the content of this article.

References

1. ACER.: Report on distribution tariff methodologies in Europe. [Online]. Available: <https://europa.eu/!PndntN> (2021)
2. Forouli, A., Bakirtzis, E.A., Papazoglou, G., Ourelidis, K., Gkountis, V., Candido, L., Ferrer, E.D., Biskas, P.: Assessment of demand side flexibility in european electricity markets: A country level review. *Energies* **14**(8), 2324 (2021)
3. Muratori, M., Rizzoni, G.: Residential demand response: Dynamic energy management and time-varying electricity pricing. *IEEE Trans. Power Syst.* **31**(2), 1108–1117 (2015)
4. Muratori, M., Schuelke-Leech, B.-A., Rizzoni, G.: Role of residential demand response in modern electricity markets. *Renew. Sustain. Energy Rev.* **33**, 546–553 (2014)
5. U.S. Department of Energy.: Benefits of demand response in electricity markets and recommendations for achieving them—a report to the united states congress pursuant to section 1252 of the energy policy act of 2005 (2005)
6. Parise, F., Colombino, M., Grammatico, S., Lygeros, J.: Mean field constrained charging policy for large populations of plug-in electric vehicles. In: 53rd IEEE Conference on Decision and Control, pp. 5101–5106 (2014)
7. Cenedese, C., Fabiani, F., Cucuzzella, M., Scherpen, J.M.A., Cao, M., Grammatico, S.: Charging plug-in electric vehicles as a mixed-integer aggregative game. In: 2019 IEEE 58th Conference on Decision and Control (CDC), pp. 4904–4909 (2019)
8. Von Stackelberg, H.: Market structure and equilibrium. Springer Science & Business Media (2010)
9. Hoheisel, T., Kanzow, C., Schwartz, A.: Theoretical and numerical comparison of relaxation methods for mathematical programs with complementarity constraints. *Math. Program.* **137**(1), 257–288 (2013)

10. Fortuny-Amat, J., McCarl, B.: A representation and economic interpretation of a two-level programming problem. *J. Oper. Res. Soc.* **32**, 783–792 (1981)
11. Fabiani, F., Tajeddini, M.A., Kebriaei, H., Grammatico, S.: Local Stackelberg equilibrium seeking in generalized aggregative games. *IEEE Trans. Autom. Control* **67**(2), 965–970 (2022)
12. Maljkovic, M., Nilsson, G., Geroliminis, N.: On finding the leader's strategy in quadratic aggregative Stackelberg pricing games. In: *European Control Conference (ECC)*. **2023**, 1–6. IEEE (2023)
13. Amos, B., Kolter, J.Z.: Optnet: Differentiable optimization as a layer in neural networks. *arXiv preprint [arXiv:1703.00443](https://arxiv.org/abs/1703.00443)* (2017)
14. Grontas, P.D., Belgioioso, G., Cenedese, C., Fochesato, M., Lygeros, J., Dörfler, F.: BIG Hype: Best intervention in games via distributed hypergradient descent. *IEEE Trans. Autom. Control*. 1–16 (2024)
15. Fochesato, M., Cenedese, C., Lygeros, J.: A Stackelberg game for incentive-based demand response in energy markets. In: *2022 IEEE 61st Conference on Decision and Control (CDC)*, pp. 2487–2492 IEEE (2022)
16. Belgioioso, G., Yi, P., Grammatico, S., Pavel, L.: Distributed generalized Nash equilibrium seeking: An operator-theoretic perspective. *IEEE Control Syst. Mag.* **42**(4), 87–102 (2022)
17. Facchinei, F., Pang, J.-S.: *Finite-dimensional variational inequalities and complementarity problems*. Springer (2003)
18. Grontas, P.D., Cenedese, C., Fochesato, M., Belgioioso, G., Lygeros, J., Dörfler, F.: Designing optimal personalized incentive for traffic routing using BIG Hype. In: *2023 62nd IEEE Conference on Decision and Control (CDC)*, pp. 3142–3147 (2023)
19. Li, J., Yu, J., Nie, Y., Wang, Z.: End-to-end learning and intervention in games. *Adv. Neural Inf. Process. Syst.* **33**(16), 653–665 (2020)
20. Bolte, J., Pauwels, E.: Conservative set valued fields, automatic differentiation, stochastic gradient method and deep learning. *Math. Programm.* (2020). [Online]. Available: <https://hal.archives-ouvertes.fr/hal-02521848>
21. Gurobi Optimization, LLC.: *Gurobi optimizer reference manual* (2023). [Online]. Available: <https://www.gurobi.com>

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.



Chapter 20

Distributed Newton Optimization with ADMM-Based Consensus



Guido Carnevale, Nicola Bastianello, Giuseppe Notarstefano,
and Ruggero Carli

Abstract In this paper, we propose two novel second-order distributed algorithms for consensus optimization both employing ADMM as a consensus protocol. The idea is to benefit from the robustness properties of ADMM, on the one hand, and the enhanced performance of Newton-based algorithms, on the other. The designed algorithms use GIANT (a recent second-order optimization method) and Jacobi-like descent directions. We employ tools from system theory, especially singular perturbations, to prove the linear convergence of our distributed schemes with strongly convex costs. We conclude with numerical simulations that confirm our theoretical results and compare the proposed algorithm with state-of-the-art alternatives.

Keywords Distributed optimization · Newton method · ADMM · Singular perturbations

20.1 Introduction

In recent years, multi-agent systems have been deployed in an increasing number of engineering domains, e.g., robotics, power grids, traffic networks, and IoT [17]. This is made possible by interconnecting devices equipped with computational and communication capabilities, with the goal of cooperatively performing different tasks, e.g., learning, resource allocation, and exploration. These tasks can be formulated as consensus optimization problems, and the objective thus is to design resilient and efficient distributed algorithms to solve them. Different classes of algorithms have been proposed, which can be categorized in first-order, second-order, and dual

G. Carnevale · G. Notarstefano
Alma Mater Studiorum - Università di Bologna, Bologna, Italy

N. Bastianello (✉)
KTH Royal Institute of Technology, Stockholm, Sweden
e-mail: nicolba@kth.se

R. Carli
University of Padova, Padova, Italy

methods (ADMM); we reference [15, 17, 23] for recent comprehensive surveys. Second-order distributed algorithms have been proposed, for example, in [5, 12–14, 21], with different convergence guarantees. In particular, [13] proposes a fully distributed version of [20] employing a tracking mechanism and Newton-like updates.

First- and second-order methods both employ (dynamic) average consensus to enable cooperation (e.g., to perform gradient tracking). However, the resulting algorithms have been shown to be fragile to some of the challenges that arise in practice, such as asynchrony and packet losses [5, 19], and noisy communications [4]. It is therefore necessary to design more robust algorithms, and one approach is to replace the average consensus protocol with a more resilient one, see, e.g., [5, 8, 19]. In this paper, we follow the approach of [7, 8] in *choosing distributed ADMM to implement dynamic consensus*.

Dual methods, ADMM in particular, are a powerful class of algorithms with provable convergence guarantees in different scenarios, including asynchrony and packet losses. [2]. The drawback of these algorithms is that the agents' local updates require the solution of a minimization problem which, in general, does not have closed-form solution. However, dynamic consensus can be cast as an online quadratic problem, for which the ADMM updates simplify to linear updates [1], making it a suitably robust and efficient consensus protocol.

In this paper, thus, we combine ADMM as a consensus protocol with distributed methods mimicking the second-order GIANT algorithm proposed in [20] and the well-known Jacobi method, benefiting from the best of both: the enhanced performance of Newton-based methods and the robustness of ADMM. In the Jacobi version of the algorithm, the agents' Hessians are approximated via their diagonals only, significantly reducing the computational complexity while still improving performance with respect to first-order methods. Following a recent successful trend [9, 10], the analysis of the proposed distributed algorithms is conducted using system theory tools based on singular perturbations. In detail, we prove the algorithms' linear convergence in the case of strongly convex problems.

The paper is organized as follows. Section 20.2 describes the problem setup. Section 20.3 describes the proposed distributed algorithms and provides their convergence properties. Section 20.4 theoretically proves the stated convergence properties. Finally, in Sect. 20.5, we test the proposed schemes via numerical simulations.

Notation The symbol I_m denotes the identity matrix in $\mathbb{R}^{m \times m}$. The symbol $\otimes$ denotes the Kronecker product. The vectors of N ones and N zeroes are denoted by $\mathbf{1}_N$ and $\mathbf{0}_N$, respectively, while $\mathbf{1}_{N,n} := \mathbf{1}_N \otimes I_n$ $\mathbf{0}_{N,n} := \mathbf{0}_N \otimes I_n$. Given a finite set S , we use the symbol $|S|$ to denote its cardinality. Given N column vectors $v_1, \dots, v_N$, we denote with $\text{COL}(v_i)_{i \in \{1, \dots, N\}}$ their vertical concatenation. Given $A \in \mathbb{R}^{n \times n}$, we denote as $\text{diag}\{A\} \in \mathbb{R}^{n \times n}$ the matrix having the main diagonal coinciding with A and 0 elsewhere.

20.2 Problem Setup

We consider $N \in \mathbb{N}$ agents in a network whose goal is to cooperatively solve the optimization problem:

$$\min_{x \in \mathbb{R}^n} \sum_{i=1}^N f_i(x), \quad (20.1)$$

where each $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ is the local objective function of agent $i \in \{1, \dots, N\}$ and all depend on the common decision variable $x \in \mathbb{R}^n$. We use $f(x) := \sum_{i=1}^N f_i(x)$ to denote the global cost function of problem (20.1). We are interested in developing an optimization method to iteratively address (20.1) according to the distributed paradigm. Namely, we want to obtain an algorithm that can be executed by peer-to-peer networks of units each using only local data (e.g., the local function f_i) and neighboring communication. In particular, we assume that the agents communicate through an undirected graph $\mathcal{G} = (\{1, \dots, N\}, \mathcal{E})$, with the set of edges $\mathcal{E} \subset \{1, \dots, N\} \times \{1, \dots, N\}$ such that $(i, j) \in \mathcal{E}$ if i and j can exchange information. We use the symbol $\mathcal{N}_i := \{j \in \{1, \dots, N\} \mid (i, j) \in \mathcal{E}\}$ to indicate the neighbors of agent i , the symbol $i := |\mathcal{N}_i|$ to indicate the so-called degree of agent i , and define $\mathbf{d} := \sum_{i=1}^N i = 2|\mathcal{E}|$.

We enforce the following assumption about the class of graphs considered in this work.

Assumption 20.1 [Graph] The graph $\mathcal{G}$ is undirected and connected. □

Assumption 20.2 [Cost functions] For all $i \in \{1, \dots, N\}$, the objective function f_i is $\mathcal{C}^3$, c -strongly convex, and its gradient ∇f_i is L -Lipschitz continuous. □

We notice that Assumption 20.2 ensures that f is Nc -strongly convex and, thus, Problem (20.1) admits a unique solution $x^* \in \mathbb{R}^n$.

20.3 GIANT-ADMM and Jacobi-ADMM

To iteratively address Problem (20.1), given the iteration index $t \in \mathbb{N}$, we maintain a solution estimate $x_i^t \in \mathbb{R}^n$ in each agent i and update it according to

$$x_i^{t+1} = x_i^t + u_i^t, \quad (20.2)$$

where $u_i^t \in \mathbb{R}^n$ is the corresponding update direction. In particular, we need to design each u_i^t to (i) force consensus among all the agents' estimates and (ii) converge to the solution of Problem (20.1). We pursue the first goal by including the consensus

action $\frac{1}{N} \sum_{j=1}^N x_j^t - x_i^t$ into u_i^t . As for the second purpose, with an eye to second-order methods, we leverage on the term $\frac{\delta}{N} \mathcal{H}_i(x_i^t)^{-1} \sum_{j=1}^N \nabla f_j(x_j^t)$, in which $\delta > 0$ is the stepsize and we allow for two possible choices of $\mathcal{H}_i : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$, namely:

$$\begin{cases} \mathcal{H}_i(x_i) = \nabla^2 f_i(x_i)^{-1} & \text{GIANT-like update} \\ \mathcal{H}_i(x_i) = \text{diag}\{\nabla^2 f_i(x_i)\}^{-1} & \text{Jacobi-like update.} \end{cases} \quad (20.3)$$

Therefore, the whole update direction u_i^t would read as

$$u_i^t = \left(\frac{1}{N} \sum_{j=1}^N x_j^t - x_i^t \right) - \frac{\delta}{N} \mathcal{H}_i(x_i^t)^{-1} \sum_{j=1}^N \nabla f_j(x_j^t). \quad (20.4)$$

However, in a distributed setting, the global terms $\frac{1}{N} \sum_{j=1}^N x_j^t$ and $\frac{1}{N} \sum_{j=1}^N \nabla f_j(x_j^t)$ are not available for agent i . Independently on the choice about $\mathcal{H}_i$, we thus modify (20.4) by introducing in each agent two auxiliary variables $y_i^t, s_i^t \in \mathbb{R}^n$ aimed at reconstructing $\frac{1}{N} \sum_{j=1}^N x_j^t$ and $\frac{1}{N} \sum_{j=1}^N \nabla f_j(x_j^t)$, respectively, thus obtaining

$$u_i^t = (y_i^t - x_i^t) - \delta \mathcal{H}_i(x_i^t)^{-1} s_i^t. \quad (20.5)$$

It is worth noting that when $y_i^t = \frac{1}{N} \sum_{j=1}^N x_j^t$ and $s_i^t = \frac{1}{N} \sum_{j=1}^N \nabla f_j(x_j^t)$ the desired update (20.4) and the implementable one (20.5) coincide. For this reason, we follow the approach in [1, 7, 8] by pursuing the consensus goals of y_i^t and s_i^t by considering, for each iteration $t \geq 0$, the online optimization problem:

$$\min_{\substack{(y_1, \dots, y_N) \in \mathbb{R}^{Nn} \\ (s_1, \dots, s_N) \in \mathbb{R}^{Nn}}} \sum_{i=1}^N \ell_i^t(y_i, s_i) \quad \text{s.t.:} \quad \begin{bmatrix} y_i \\ s_i \end{bmatrix} = \begin{bmatrix} y_j \\ s_j \end{bmatrix} \quad \forall (i, j) \in \mathcal{E}, \quad (20.6)$$

where, for all $i \in \{1, \dots, N\}$, the cost function $\ell_i^t : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ is defined as

$$\ell_i^t(y_i, s_i) = \frac{1}{2} \|y_i - x_i^t\|^2 + \frac{1}{2} \|s_i - \nabla f_i(x_i^t)\|^2.$$

Indeed, if the graph $\mathcal{G}$ is connected, then [1] guarantees that Problem (20.6) admits a unique solution $(y_\star^t, s_\star^t) \in \mathbb{R}^{2Nn}$ defined as

$$\begin{bmatrix} y_\star^t \\ s_\star^t \end{bmatrix} = \begin{bmatrix} \mathbf{1}_{N,n} \frac{1}{N} \sum_{j=1}^N x_j^t \\ \mathbf{1}_{N,n} \frac{1}{N} \sum_{j=1}^N \nabla f_j(x_j^t) \end{bmatrix}.$$

Thus, by focusing on Problem (20.6), we choose the distributed ADMM proposed in [2] to update y_i^t and s_i^t , namely, in each agent i , we introduce an additional variable $z_{ij}^t \in \mathbb{R}^{2n}$ for each neighbor $j \in \mathcal{N}_i$ and implement

$$\begin{bmatrix} y_i^t \\ s_i^t \end{bmatrix} = \arg \min_{\substack{y_i \in \mathbb{R}^n \\ s_i \in \mathbb{R}^n}} \left\{ \ell_i^t(y_i, s_i) - [y_i^\top \ s_i^\top] \sum_{j \in \mathcal{N}_i} z_{ij}^t + \frac{\rho_i}{2} (\|y_i\|^2 + \|s_i\|^2) \right\} \quad (20.7a)$$

$$z_{ij}^{t+1} = (1 - \alpha)z_{ij}^t + \alpha \left(-z_{ji}^t + 2\rho \begin{bmatrix} y_j^t \\ s_j^t \end{bmatrix} \right), \quad (20.7b)$$

where $\rho > 0$ and $\alpha \in (0, 1)$. By observing that each function ℓ_i^t is quadratic, we note that Problem (20.7a) admits a closed-form solution given by

$$\begin{bmatrix} y_i^t \\ s_i^t \end{bmatrix} = \frac{1}{1 + \rho_i} \left(\begin{bmatrix} x_i^t \\ \nabla f_i(x_i^t) \end{bmatrix} + \sum_{j \in \mathcal{N}_i} z_{ij}^t \right). \quad (20.8)$$

Further, in order to pursue a timescale separation approach, we add a tuning gain $\gamma > 0$ to (20.2) to arbitrarily control the rate of variation of x_i^t and, thus, allow for a proper interconnection with (20.8), namely:

$$x_i^{t+1} = x_i^t + \gamma u_i^t. \quad (20.9)$$

By combining (20.5), (20.7), (20.8), and (20.9) we obtain two different distributed algorithms based on the choice made for $\mathcal{H}_i$ (see (20.3)). In particular, by choosing $\mathcal{H}_i = \nabla^2 f_i(x_i^t)$, we get Algorithm 20.1 and we name it GIANT-ADMM.

Algorithm 20.1 GIANT-ADMM (Agent i)

Initialization: $x_i^0 \in \mathbb{R}^n$, $z_{ij}^0 \in \mathbb{R}^{2n}$ for each $j \in \mathcal{N}_i$.

for $t = 0, 1, \dots$ **do**

$$\begin{bmatrix} y_i^t \\ s_i^t \end{bmatrix} = \frac{1}{1 + \rho_i} \left(\begin{bmatrix} x_i^t \\ \nabla f_i(x_i^t) \end{bmatrix} + \sum_{j \in \mathcal{N}_i} z_{ij}^t \right)$$

$$x_i^{t+1} = x_i^t + \gamma(y_i^t - x_i^t) - \gamma \delta \nabla^2 f_i(x_i^t)^{-1} s_i^t$$

for $j \in \mathcal{N}_i$ **do**

$$z_{ij}^{t+1} = (1 - \alpha)z_{ij}^t + \alpha \left(-z_{ji}^t + 2\rho \begin{bmatrix} y_j^t \\ s_j^t \end{bmatrix} \right)$$

end for

end for

On the contrary, in Algorithm 20.2, we report the distributed algorithm obtained with $\mathcal{H}_i = \text{diag}\{\nabla^2 f_i(x_i^t)\}$ and we name it Jacobi-ADMM.

Algorithm 20.2 Jacobi-ADMM (Agent i)

Initialization: $x_i^0 \in \mathbb{R}^n$, $z_{ij}^0 \in \mathbb{R}^{2n}$ for each $j \in \mathcal{N}_i$.
for $t = 0, 1, \dots$ **do**

$$\begin{bmatrix} y_i^t \\ s_i^t \end{bmatrix} = \frac{1}{1+\rho_i} \left(\begin{bmatrix} x_i^t \\ \nabla f_i(x_i^t) \end{bmatrix} + \sum_{j \in \mathcal{N}_i} z_{ij}^t \right)$$

$$x_i^{t+1} = x_i^t + \gamma(y_i^t - x_i^t) - \gamma \delta \text{diag}\{\nabla^2 f_i(x_i^t)\}^{-1} s_i^t$$
for $j \in \mathcal{N}_i$ **do**

$$z_{ij}^{t+1} = (1 - \alpha) z_{ij}^t + \alpha \left(-z_{ji}^t + 2\rho \begin{bmatrix} y_j^t \\ s_j^t \end{bmatrix} \right)$$
end for
end for

Both Algorithms 20.1 and 20.2 can be implemented by satisfying the desired distributed paradigm since when running them each agent i only uses local quantities and performs neighboring communication. More in detail, in order to execute Algorithms 20.1 and 20.2, each agent i must maintain the local solution estimate $x_i^t \in \mathbb{R}^n$ and the auxiliary variable $z_i^t := \text{COL}(z_{ij}^t)_{j \in \mathcal{N}_i} \in \mathbb{R}^{2ni}$ stacking all the variables z_{ij}^t . Hence, each agent i needs to store vectors with overall size $(1 + 2i)n$. As for the communication burden, we note that each agent i needs to transmit the vector $-z_{ij}^t + 2\rho [(y_i^t)^\top (s_i^t)^\top]^\top \in \mathbb{R}^{2n}$ to each neighbor $j \in \mathcal{N}_i$. We also remark that it is possible to exactly recover the ADMM-Tracking Gradient distributed algorithm developed in [8] by choosing $\mathcal{H}_i(x_i) = I_n$ for all $i \in \{1, \dots, N\}$. Next, we formally claim the convergence features of Algorithms 20.1 and 20.2. Indeed, as will become clearer in the analysis, the similarities between the considered schemes and our system-theoretic approach allow for a unified examination.

Theorem 20.1 *Consider Algorithm 20.1 and 20.2 and let Assumptions 20.1 and 20.2 hold. Then, for all $(x_i^0, z_i^0) \in \mathbb{R}^n \times \mathbb{R}^{2ni}$ for all $i \in \{1, \dots, N\}$, there exist $\bar{\gamma}, \bar{\delta}, c_1, c_2 > 0$ such that, for any $\rho > 0$, $\alpha \in (0, 1)$, $\gamma \in (0, \bar{\gamma})$, $\delta \in (0, \bar{\delta})$, it holds*

$$\|x_i^t - x^*\| \leq c_1 \exp(-c_2 t). \quad \square$$

Theorem 20.1 is proved in Sect. 20.4.4. In detail, with tools from system theory, we will show semi-global exponential stability of $(\mathbf{1}_{N,n} x^*, z_\perp^*)$ for a suitable and restricted reformulation of the aggregate forms of GIANT-ADMM and Jacobi-ADMM, where the quantity $z_\perp^*$ will be specified later.

20.4 Convergence Analysis of GIANT-ADMM and Jacobi-ADMM

In this section, we analyze Algorithms 20.1 and 20.2 by resembling the system-theoretic approach performed in [8]. First, in Sect. 20.4.1, we provide the aggregate form of GIANT-ADMM and Jacobi-ADMM and show that it is possible to interpret

it as a *singularly perturbed* system. By taking advantage of this form, in Sects. 20.4.2 and 20.4.3, we then separately analyze the stability properties of two auxiliary systems associated to the original one. These results are finally exploited in Sect. 20.4.4 to prove Theorem 20.1 by leveraging on singular perturbation arguments.

20.4.1 GIANT-ADMM and Jacobi-ADMM as Singularly Perturbed Systems

We start by building a dynamical system describing the aggregate form of GIANT-ADMM and Jacobi-ADMM (according to the choice made for $\mathcal{H}_i(x_i^t)$). To this end, we introduce the permutation matrix $P \in \mathbb{R}^{2nd \times 2nd}$ that swaps the ij -th element with the ji -th one, the matrices $A_x \in \mathbb{R}^{2nd \times Nn}$, $A_\nabla \in \mathbb{R}^{nd \times Nn}$, $A \in \mathbb{R}^{2nd \times Nn}$, $M \in \mathbb{R}^{Nn \times Nn}$, $\mathcal{M} \in \mathbb{R}^{2Nn \times 2Nn}$, and $\mathcal{H} : \mathbb{R}^{Nn} \times \mathbb{R}^{Nn \times Nn}$ defined as

$$\begin{aligned} A_x &:= \begin{bmatrix} \mathbf{1}_{1,n} & & & \\ \mathbf{0}_{1,n} & \ddots & & \\ & \ddots & \ddots & \\ & & \mathbf{1}_{N,n} & \\ & & & \mathbf{0}_{N,n} \end{bmatrix}, \quad A_\nabla := \begin{bmatrix} \mathbf{0}_{1,n} & & & \\ \mathbf{1}_{1,n} & \ddots & & \\ & \ddots & \ddots & \\ & & \mathbf{0}_{N,n} & \\ & & & \mathbf{1}_{N,n} \end{bmatrix}, \\ A &:= \begin{bmatrix} \mathbf{1}_{1,2n} & & & \\ & \ddots & & \\ & & \ddots & \\ & & & \mathbf{1}_{N,2n} \end{bmatrix}, \quad \mathcal{H}(x) := \begin{bmatrix} \mathcal{H}_1(x_1) & & \\ & \ddots & \\ & & \mathcal{H}_N(x_N) \end{bmatrix} \\ M &:= \begin{bmatrix} \frac{1}{1+\rho_1} I_n & & & \\ & \ddots & & \\ & & \ddots & \\ & & & \frac{1}{1+\rho_N} I_n \end{bmatrix}, \quad \mathcal{M} := \begin{bmatrix} \frac{1}{1+\rho_1} I_{2n} & & & \\ & \ddots & & \\ & & \ddots & \\ & & & \frac{1}{1+\rho_N} I_{2n} \end{bmatrix} \end{aligned}$$

Then, by defining the stacking vectors $x^t := \text{COL}(x_1^t, \dots, x_N^t) \in \mathbb{R}^{Nn}$ and $z^t := \text{COL}(z_1^t, \dots, z_N^t) \in \mathbb{R}^{2nd}$, the aggregate form of Algorithms 20.1 and 20.2 reads as

$$x^{t+1} = x^t + \gamma \left(M \left(x^t + A_x^\top z^t \right) - x^t \right) - \gamma \delta \mathcal{H}(x^t)^{-1} M \left(G(x^t) + A_\nabla^\top z^t \right) \quad (20.10a)$$

$$z^{t+1} = z^t - \alpha (I + P - 2\rho P A \mathcal{M} A^\top) z^t + 2\alpha \rho P A \mathcal{M} v(x^t), \quad (20.10b)$$

where, given $x := \text{COL}(x_1, \dots, x_N) \in \mathbb{R}^{Nn}$ with $x_i \in \mathbb{R}^n$ for all $i \in \{1, \dots, N\}$, the operators $G : \mathbb{R}^{Nn} \rightarrow \mathbb{R}^{Nn}$ and $v : \mathbb{R}^{Nn} \rightarrow \mathbb{R}^{2Nn}$ are defined as

$$G(x) := \left[\nabla f_1(x_1)^\top \dots \nabla f_N(x_N)^\top \right]^\top \quad (20.11)$$

$$v(x) := \left[x_1^\top \nabla f_1(x_1)^\top \dots x_N^\top \nabla f_N(x_N)^\top \right]^\top. \quad (20.12)$$

We know that, by construction, the matrix $(I + P - 2\rho P A \mathcal{M} A^\top)$ has some eigenvalues in 0 [2]. Moreover, these eigenvalues are semi-simple [2]. By relying on this property, we now follow the same approach as in [8] to obtain a conve-

nient restricted system description. In particular, we employ a decomposition that expresses z^t according to a basis of the subspace corresponding to the null space of $(I + P - 2\rho P A M A^\top)$. Specifically, let $b \in \mathbb{N}$ denote the dimension of the subspace $\mathcal{S}$ generated by the eigenvectors of $(I + P - 2\rho P A M A^\top)$ associated with the eigenvalue 0, $B \in \mathbb{R}^{2nd \times b}$ be the matrix having columns corresponding to an orthonormal basis of $\mathcal{S}$, and $B_\perp \in \mathbb{R}^{2nd \times n_d}$ the matrix such that $B_\perp^\top B_\perp = I$ and $B^\top B_\perp = 0$, with $n_d := 2nd - b$. The matrices B and $B_\perp$ have some useful properties formalized as follows.

Lemma 20.1 ([8, Lemma III.1]) *The matrices B and $B_\perp$ satisfy*

$$A_x^\top B = 0, \quad A_\nabla^\top B = 0 \quad (20.13a)$$

$$B^\top P A = 0 \quad (20.13b)$$

$$B^\top (I + P - 2\rho P A M A^\top) = 0. \quad (20.13c)$$

Following the same approach in [8], we now introduce the change of variables:

$$\bar{z} := B^\top z, \quad z_\perp := B_\perp^\top z. \quad (20.14)$$

Lemma 20.1 and the novel coordinates (20.14) allow for reformulating system (20.10) as

$$x^{t+1} = x^t + \gamma \left(M \left(x^t + A_x^\top B_\perp z_\perp^t \right) - x^t \right) - \gamma \delta \mathcal{H}(x^t)^{-1} M \left(G(x^t) + A_\nabla^\top B_\perp z_\perp^t \right) \quad (20.15a)$$

$$\bar{z}^{t+1} = \bar{z}^t \quad (20.15b)$$

$$z_\perp^{t+1} = z_\perp^t - \alpha B_\perp^\top (I + P - 2\rho P A M A^\top) B_\perp z_\perp^t + 2\alpha \rho B_\perp^\top P A M v(x^t). \quad (20.15c)$$

Notice that this is the same structure derived in [8], with the difference that now $\mathcal{H}(x^t)$ contains second-order information (see (20.6)). In particular, the state $\bar{z}^t$ does not influence the other states of (20.15). Hence, we study system (20.15) without considering the variable $\bar{z}^t$ and using the novel coordinate $\tilde{x}^t := x^t - \mathbf{1}_{N,n} x^*$. Therefore, we focus on

$$\tilde{x}^{t+1} = \tilde{x}^t + \gamma \left(M \left(x^t + A_x^\top B_\perp z_\perp^t \right) - x^t \right) - \gamma \delta \mathcal{H}(x^t)^{-1} M \left(G(x^t) + A_\nabla^\top B_\perp z_\perp^t \right) \quad (20.16a)$$

$$z_\perp^{t+1} = z_\perp^t - \alpha F z_\perp^t + 2\alpha \rho B_\perp^\top P A M v(x^t), \quad (20.16b)$$

in which we also use

$$F := B_\perp^\top (I + P - 2\rho P A M A^\top) B_\perp. \quad (20.17)$$

Notably, we remark that $(I - \alpha F)$ is Schur for all $\alpha \in (0, 1)$ [8, Lemma III.2].

System (20.16) matches the form of the so-called singularly perturbed systems since it exhibits the following two key properties. First, the variations of the subsystem (20.16a) can be arbitrarily reduced via the tuning parameter γ and, accordingly, plays the role of the so-called slow system. Second, the subsystem (20.16b) plays the role of the so-called fast system since its equilibria are parametrized in the slow state $\tilde{x}^t$ through the function $z_{\perp}^{\text{eq}} : \mathbb{R}^{Nn} \rightarrow \mathbb{R}^{nd}$ defined as

$$z_{\perp}^{\text{eq}}(\tilde{x}^t) := 2\rho F^{-1} B_{\perp}^{\top} P A \mathcal{M} v(\tilde{x}^t + \mathbf{1}_{N,n} x^{\star}). \quad (20.18)$$

We conclude the section by reporting the following useful properties about z^{eq} . Given any $\tilde{x} \in \mathbb{R}^{Nn}$ and the corresponding $x = \tilde{x} + \mathbf{1}_{N,n} x^{\star}$, it holds

$$M A_x^{\top} B_{\perp} z_{\perp}^{\text{eq}}(\tilde{x}) = \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N} x - M x \quad (20.19a)$$

$$M A_{\nabla}^{\top} B_{\perp} z_{\perp}^{\text{eq}}(\tilde{x}) = \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N} G(x) - M G(x). \quad (20.19b)$$

20.4.2 Boundary Layer System

In this section, we study the so-called boundary layer system associated to (20.16), i.e., the auxiliary system obtained by plugging an arbitrarily fixed state $\tilde{x}^t \equiv \tilde{x} \in \mathbb{R}^{Nn}$ into the fast dynamics (20.16b) and using the error coordinate $\tilde{z}_{\perp}^t := z_{\perp}^t - z_{\perp}^{\text{eq}}(\tilde{x})$. By using the definition of $z_{\perp}^{\text{eq}}$ (cf. (20.18)), such a boundary layer system reads as

$$\tilde{z}_{\perp}^{t+1} = \tilde{z}_{\perp}^t - \alpha F \tilde{z}_{\perp}^t. \quad (20.20)$$

Not surprisingly, since the consensus-oriented part of GIANT-ADMM and Jacobi-ADMM is the same as the one characterizing the distributed method provided in [8], the boundary layer system (20.20) associated to (20.16) is exactly the same considered in [8]. Then, we already know the stability properties of (20.20).

Lemma 20.2 ([8, Lemma III.2]) *Consider (20.20). Then, for all $\alpha \in (0, 1)$, there exist $U : \mathbb{R}^{nd} \rightarrow \mathbb{R}$ and $b_1, b_2, b_3, b_4 > 0$ such that*

$$b_1 \|\tilde{z}_{\perp}\|^2 \leq U(\tilde{z}_{\perp}) \leq b_2 \|\tilde{z}_{\perp}\|^2 \quad (20.21a)$$

$$U(\tilde{z}_{\perp}^t - \alpha F \tilde{z}_{\perp}^t) - U(\tilde{z}_{\perp}) \leq -b_3 \|\tilde{z}_{\perp}\|^2 \quad (20.21b)$$

$$|U(\tilde{z}_{\perp}) - U(\tilde{z}_{\perp}')| \leq b_4 \|\tilde{z}_{\perp} - \tilde{z}_{\perp}'\| \|\tilde{z}_{\perp}\| + b_4 \|\tilde{z}_{\perp} - \tilde{z}_{\perp}'\| \|\tilde{z}_{\perp}'\|, \quad (20.21c)$$

for all $\tilde{z}_{\perp}, \tilde{z}_{\perp}' \in \mathbb{R}^{nd}$. □

20.4.3 Reduced System

Here, we focus on the auxiliary system associated to (20.16) named reduced system. The latter is obtained from (20.16a) by plugging $z_{\perp}^t = z_{\perp}^{\text{eq}}(\tilde{x}^t)$ for all $t \in \mathbb{N}$. Then, by also using (20.19), the reduced system of (20.16) reads as

$$\tilde{x}^{t+1} = \tilde{x}^t - \gamma \left(I - \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N} \right) \tilde{x}^t - \gamma \delta \mathcal{H}(x^t)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N} G(\tilde{x}^t + \mathbf{1}_{N,n} x^{\star}). \quad (20.22)$$

The upcoming lemma establishes a Lyapunov function for system (20.22).

Lemma 20.3 *Consider (20.22). Then, there exist $W : \mathbb{R}^{Nn} \rightarrow \mathbb{R}$, $\bar{\gamma}_1, \bar{\delta}, c_1, c_2, c_3, c_4 > 0$ such that, for all $\gamma \in (0, \bar{\gamma}_1)$ and $\delta \in (0, \bar{\delta})$, it holds*

$$c_1 \|\tilde{x}\|^2 \leq W(\tilde{x}) \leq c_2 \|\tilde{x}\|^2 \quad (20.23a)$$

$$W(\tilde{x}^{t+1}) - W(\tilde{x}^t) \leq -c_3 \gamma \|\tilde{x}\|^2 \quad (20.23b)$$

$$|W(\tilde{x}) - W(\tilde{x}')| \leq c_4 \|\tilde{x} - \tilde{x}'\| \|\tilde{x}\| + c_4 \|\tilde{x} - \tilde{x}'\| \|\tilde{x}'\|, \quad (20.23c)$$

for all $\tilde{x}, \tilde{x}', \tilde{x}^t \in \mathbb{R}^{Nn}$, and $\tilde{x}^{t+1}$ computed according to (20.22) starting from x^t .

Proof Following the same derivation as in [8], but applying it to the novel Algorithms 20.1 and 20.2, we take advantage of suitable novel coordinates. To this end, let $R \in \mathbb{R}^{Nn \times (N-1)n}$ be a matrix such that $R^{\top} \mathbf{1}_{N,n} = 0$ and $R^{\top} R = I_{(N-1)n}$. Then, let $\bar{x} \in \mathbb{R}^n$ and $x_{\perp} \in \mathbb{R}^{(N-1)n}$ be defined as

$$\bar{x} := \frac{\mathbf{1}_{N,n}^{\top}}{N} \tilde{x}, \quad x_{\perp} := R^{\top} \tilde{x}. \quad (20.24)$$

By employing the change of variables (20.24), $\mathbf{1}_{N,n} \in \ker(I_{Nn} - \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N})$, and $R^{\top} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N} = 0$, system (20.22) turns out to be equivalent to

$$\bar{x}^{t+1} = \bar{x}^t - \gamma \delta \frac{\mathbf{1}_{N,n}^{\top}}{N} \mathcal{H}(x^t)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N} G(x^t) \quad (20.25a)$$

$$x_{\perp}^{t+1} = (1 - \gamma) x_{\perp}^t - \gamma \delta R^{\top} \mathcal{H}(x^t)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^{\top}}{N} G(x^t), \quad (20.25b)$$

in which we used $x^t = \mathbf{1}_{N,n} \bar{x}^t + R x_{\perp}^t$ in the argument of $\mathcal{H}$ and G . Let $\bar{G} : \mathbb{R}^n \times \mathbb{R}^{(N-1)n} \rightarrow \mathbb{R}^n$ and $G_{\perp} : \mathbb{R}^n \times \mathbb{R}^{(N-1)n} \rightarrow \mathbb{R}^n$ be defined as

$$\begin{aligned}
\bar{G}(\bar{x}, x_\perp) &:= -\frac{\mathbf{1}_{N,n}^\top}{N} \mathcal{H}(x)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^\top}{N} G(\mathbf{1}_{N,n} \bar{x} + R x_\perp + \mathbf{1}_{N,n} x^*) \\
&\quad + \frac{\mathbf{1}_{N,n}^\top}{N} \mathcal{H}(x)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^\top}{N} G(\mathbf{1}_{N,n} \bar{x} + \mathbf{1}_{N,n} x^*) \\
G_\perp(\bar{x}, x) &:= -R^\top \mathcal{H}(x)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^\top}{N} G(\mathbf{1}_{N,n} \bar{x} + R x_\perp + \mathbf{1}_{N,n} x^*),
\end{aligned}$$

in which, we used $x = \mathbf{1}_{N,n} \bar{x} + R x_\perp$ in the argument of $\mathcal{H}$. Let us inspect the term $\frac{\mathbf{1}_{N,n}^\top}{N} \mathcal{H}(x)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^\top}{N} G(\mathbf{1}_{N,n} \bar{x} + \mathbf{1}_{N,n} x^*)$. In particular, it holds

$$\frac{\mathbf{1}_{N,n}^\top}{N} \mathcal{H}(x)^{-1} \frac{\mathbf{1}_{N,n} \mathbf{1}_{N,n}^\top}{N} G(\mathbf{1}_{N,n} \bar{x} + \mathbf{1}_{N,n} x^*) = \frac{1}{N^2} \left(\sum_{i=1}^N \mathcal{H}_i(x_i)^{-1} \right) \nabla f(\bar{x} + x^*),$$

in which we lighten the notation by denoting with $x_i \in \mathbb{R}^n$ the i -th block of the vector $x = \mathbf{1}_{N,n} \bar{x} + \mathbf{1}_{N,n} x^* + R x_\perp$. By adding $\pm \frac{\gamma \delta}{N^2} \left(\sum_{i=1}^N \mathcal{H}_i(x_i^t)^{-1} \right) \nabla f(\bar{x}^t + x^*)$ into (20.25a), we compactly rewrite system (20.25) as

$$\bar{x}^{t+1} = \bar{x}^t - \frac{\gamma \delta}{N^2} \left(\sum_{i=1}^N \mathcal{H}_i(x_i^t)^{-1} \right) \nabla f(\bar{x}^t + x^*) + \gamma \delta \bar{G}(\bar{x}^t, x_\perp^t) \quad (20.27a)$$

$$x_\perp^{t+1} = (1 - \gamma) x_\perp^t + \delta \gamma G_\perp(\bar{x}^t, x_\perp^t). \quad (20.27b)$$

Now, we prove that the origin is a globally exponentially stable equilibrium point of (20.27). To this end, we consider the Lyapunov function $\tilde{W} : \mathbb{R}^n \times \mathbb{R}^{(N-1)n} \rightarrow \mathbb{R}$ defined as

$$\tilde{W}(\bar{x}) := \underbrace{f(\bar{x} + x^*) - f(x^*)}_{:= \tilde{W}_{\bar{x}}(\bar{x})} + \underbrace{\frac{1}{2} \|x_\perp\|^2}_{:= \tilde{W}_{x_\perp}(x_\perp)}. \quad (20.28)$$

We recall that each ∇f_i is L -Lipschitz continuous (cf. Assumption 20.2) and, thus, by using the Descent Lemma [3, Prop. 6.1.2], for all $\bar{x} \in \mathbb{R}^n$, we get

$$f(\bar{x} + x^*) - f(x^*) \leq \nabla f(x^*)^\top \bar{x} + \frac{NL}{2} \|\bar{x}\|^2 \stackrel{(a)}{\leq} \frac{NL}{2} \|\bar{x} - x^*\|^2, \quad (20.29)$$

where in (a) we use the fact that $\nabla f(x^*) = 0$ since x^* is the solution of Problem (20.1). Moreover, since f is Nc -strongly convex (cf. Assumption 20.2), it holds

$$f(\bar{x} + x^*) - f(x^*) \geq \frac{Nc}{2} \|\bar{x}\|^2, \quad (20.30)$$

for all $\bar{x} \in \mathbb{R}^n$. Hence, by using the bounds (20.29) and (20.30) and the fact that $\tilde{W}_{x_\perp}$ is quadratic, it holds

$$\min\{Nc, 1\}/2 (\|\bar{x}\|^2 + \|x_\perp\|^2) \leq \tilde{W}(\bar{x}, x_\perp) \leq \max\{NL, 1\}/2 (\|\bar{x}\|^2 + \|x_\perp\|^2), \quad (20.31)$$

for all $\bar{x} \in \mathbb{R}^n$ and $x_\perp \in \mathbb{R}^{(N-1)n}$. Let us now evaluate $\Delta \tilde{W}_{\bar{x}}(\bar{x}^t) := \tilde{W}_{\bar{x}}(\bar{x}^{t+1}) - \tilde{W}_{\bar{x}}(\bar{x}^t)$ along (20.27a) using the Descent Lemma [3, Prop. 6.1.2] and thus obtaining

$$\begin{aligned} \Delta \tilde{W}_{\bar{x}}(\bar{x}^t) &= -\frac{\gamma\delta}{N^2} \nabla f(\bar{x}^t + x^*)^\top \left(\sum_{i=1}^N \mathcal{H}_i(\bar{x} + x^*)^{-1} \right) \nabla f(\bar{x}^t + x^*) \\ &\quad + \gamma\delta \nabla f(\bar{x}^t + x^*)^\top \tilde{G}(\bar{x}^t, x_\perp^t) \\ &\quad + \frac{\gamma^2\delta^2NL}{2} \left\| \frac{1}{N^2} \nabla f(\bar{x}^t + x^*) + \tilde{G}(\bar{x}^t, x_\perp^t) \right\|^2 \\ &\stackrel{(a)}{\leq} -\frac{\gamma\delta}{N^2} \nabla f(\bar{x}^t + x^*)^\top \left(\sum_{i=1}^N \mathcal{H}_i(\bar{x} + x^*)^{-1} \right) \nabla f(\bar{x}^t + x^*) \\ &\quad + \gamma\delta \left\| \nabla f(\bar{x}^t + x^*) \right\| \left\| \tilde{G}(\bar{x}^t, x_\perp^t) \right\| \\ &\quad + \gamma^2\delta^2NL \left\| \frac{1}{N^2} \nabla f(\bar{x}^t + x^*) \right\|^2 + \gamma^2\delta^2NL \left\| \tilde{G}(\bar{x}^t, x_\perp^t) \right\|^2, \quad (20.32) \end{aligned}$$

where in (a) we bound the second term using the Cauchy–Schwarz inequality and Young’s inequality for the last one. Being each cost f_i c -strongly convex with L -Lipschitz continuous gradients (cf. Assumption 20.2), both choices in (20.3) satisfy

$$\frac{1}{L_{\mathcal{H}}} I_n \leq \sum_{i=1}^N \mathcal{H}_i(\bar{x})^{-1} \leq \frac{1}{c_{\mathcal{H}}} I_n, \quad (20.33)$$

for all $\bar{x} \in \mathbb{R}^n$, with $L_{\mathcal{H}} = L/N$ and $c_{\mathcal{H}} = c/N$ if $\mathcal{H}_i(x_i) = \nabla^2 f_i(x_i)$ and $L_{\mathcal{H}} = NL$ and $c_{\mathcal{H}} = Nc$ if $\mathcal{H}_i(x_i) = \text{diag}\{\nabla^2 f_i(x_i)\}$. The bounds in (20.33) lead to

$$-\nabla f(\bar{x})^\top \left(\sum_{i=1}^N \mathcal{H}_i(\bar{x})^{-1} \right) \nabla f(\bar{x}) \leq -\frac{1}{L_{\mathcal{H}}} \|\nabla f(\bar{x})\|^2 \stackrel{(a)}{\leq} -\frac{N^2c^2}{L_{\mathcal{H}}} \|\bar{x}\|^2, \quad (20.34)$$

where (a) uses the fact that f is Nc -strongly convex (cf. Assumption 20.2). Moreover, we recall that $\nabla f(x^*) = 0$ and that the gradients ∇f_i are L -Lipschitz continuous (cf. Assumption 20.2). Hence, we have

$$\left\| \nabla f(\bar{x} + x^*) \right\| = \left\| \nabla f(\bar{x} + x^*) - \nabla f(x^*) \right\| \stackrel{(a)}{\leq} LN \|\bar{x}\|, \quad (20.35)$$

for all $\bar{x} \in \mathbb{R}^n$. Further, the bound (20.33), the Lipschitz continuity of each ∇f_i (cf. Assumption 20.2), $\|\mathbf{1}_{N,n}^\top / N\| = \sqrt{N}$, $\|\frac{1}{N} \mathbf{1}_{N,n} \mathbf{1}_{N,n}^\top\| = 1$, and $\|R\| = 1$ lead to

$$\|\bar{G}(\bar{x}^t, x_\perp^t)\| \leq \frac{L\sqrt{N}}{c_{\mathcal{H}}} \|x_\perp^t\|. \quad (20.36)$$

By combining (20.34), (20.35), and (20.36), we bound (20.32) as

$$\begin{aligned} \Delta \tilde{W}_{\bar{x}}(\bar{x}^t) &\leq -\gamma\delta \frac{N^2 c^2}{L_{\mathcal{H}}} \|\bar{x}^t\|^2 + \gamma^2 \delta^2 \frac{N\sqrt{N}L^2}{c_{\mathcal{H}}} \|\bar{x}^t\| \|x_\perp^t\| \\ &\quad + \gamma^2 \delta^2 \frac{L^3}{N} \|\bar{x}^t\|^2 + \gamma^2 \delta^2 \frac{N\sqrt{N}L^2}{c_{\mathcal{H}}} \|x_\perp^t\|^2. \end{aligned} \quad (20.37)$$

Let us now consider $\tilde{W}_{x_\perp}$. Along the trajectories (20.27b), we bound $\Delta \tilde{W}_{x_\perp}(x_\perp^t) := \tilde{W}_{x_\perp}(x_\perp^{t+1}) - \tilde{W}_{x_\perp}(x_\perp^t)$ as

$$\begin{aligned} \Delta \tilde{W}_{x_\perp}(x_\perp^t) &\leq -\gamma \left(1 - \frac{\gamma}{2}\right) \|x_\perp^t\|^2 + \gamma\delta(1 - \gamma) \|x_\perp^t\| \|G_\perp(\bar{x}^t, x_\perp^t)\| \\ &\quad + \gamma^2 \delta^2 \frac{1}{2} \|G_\perp(\bar{x}^t, x_\perp^t)\|^2. \end{aligned} \quad (20.38)$$

Since $\mathcal{H}_i(\bar{x})^{-1} \leq \frac{1}{c}$ in light of Assumption 20.2, each ∇f_i is L -Lipschitz continuous (cf. Assumption 20.2), $\|\mathbf{1}_{N,n} \mathbf{1}_{N,n}^\top / N\| = 1$, and $\|R\| = 1$, it holds

$$\|G_\perp(\bar{x}^t, x_\perp^t)\| \leq \frac{NL}{c_{\mathcal{H}}} \|x_\perp^t\|,$$

which allows us to further bound (20.38) as

$$\Delta \tilde{W}_{x_\perp}(x_\perp^t) \leq -\gamma \left(1 - \frac{\gamma}{2}\right) \|x_\perp^t\|^2 + \gamma\delta(1 - \gamma) \frac{NL}{c_{\mathcal{H}}} \|x_\perp^t\|^2 + \gamma^2 \delta^2 \frac{N^2 L^2}{2c_{\mathcal{H}}^2} \|x_\perp^t\|^2. \quad (20.39)$$

By defining $\Delta \tilde{W}(\tilde{x}^t) := \Delta \tilde{W}_{\bar{x}}(\bar{x}^t) + \Delta \tilde{W}_{x_\perp}(x_\perp^t)$ and using (20.37) and (20.39), we get

$$\Delta \tilde{W}(\tilde{x}^t) \leq -\gamma \begin{bmatrix} \|\bar{x}^t\| \\ \|x_\perp^t\| \end{bmatrix}^\top Q \begin{bmatrix} \|\bar{x}^t\| \\ \|x_\perp^t\| \end{bmatrix}, \quad (20.40)$$

where we introduced the matrix $Q(\gamma, \delta) = Q(\gamma, \delta)^\top \in \mathbb{R}^{2 \times 2}$ given by

$$Q(\gamma, \delta) := \begin{bmatrix} \delta \frac{N^2 c^2}{L_{\mathcal{H}}} - \gamma \delta^2 \frac{L^3}{N} & \gamma \delta^2 \frac{N\sqrt{N}L^2}{2c_{\mathcal{H}}} \\ \gamma \delta^2 \frac{N\sqrt{N}L^2}{2c_{\mathcal{H}}} & (1 - \gamma/2) - p(\gamma, \delta) \end{bmatrix},$$

where

$$p(\gamma, \delta) = \gamma \delta^2 \frac{N \sqrt{N} L^2}{c_{\mathcal{H}}} - \delta(1 - \gamma) \frac{NL}{c_{\mathcal{H}}} \|x_{\perp}^t\|^2 - \gamma \delta^2 \frac{N^2 L^2}{2c_{\mathcal{H}}^2} \|x_{\perp}^t\|^2.$$

Let us arbitrarily choose $q \in \min\{\frac{Nc^2}{L^2}, 1\}$. By Sylvester criterion, it holds

$$Q(\gamma, \delta) > \begin{bmatrix} \delta q & 0 \\ 0 & q \end{bmatrix} \iff \begin{cases} \delta \frac{N^2 c^2}{L_{\mathcal{H}}} - \gamma \delta^2 \frac{L^3}{N} > \delta q \\ \delta \left(\delta \frac{N^2 c^2}{L_{\mathcal{H}}} - \gamma \delta^2 \frac{L^3}{N} - q \right) ((1 - \gamma/2) - p(\gamma, \delta) - q) > \gamma^2 \delta^4 \frac{N^3 L^4}{4c_{\mathcal{H}}^2}. \end{cases} \quad (20.41)$$

Hence, let $\bar{\gamma}_1 := \min \left\{ 2(1 - q), \left(\frac{N^2 c^2}{L_{\mathcal{H}}} - q \right) \frac{N}{L^3} \right\}$ and fix $\gamma \in (0, \bar{\gamma}_1)$. Thus, it holds

$$\begin{cases} \delta \frac{N^2 c^2}{L_{\mathcal{H}}} - \gamma \delta^2 \frac{L^3}{N} > \delta q \\ (1 - \gamma/2) > q, \end{cases}$$

for all $\delta \in (0, 1)$ and, thus, the second condition in (20.41) reduces to

$$q^2 > \gamma^2 \delta^3 \frac{N^3 L^4}{4c_{\mathcal{H}}^2} + qp(\gamma, \delta).$$

Moreover, we note that $\lim_{\delta \rightarrow 0} (\gamma^2 \delta^3 N^3 L^4 / 4c_{\mathcal{H}}^2 + qp(\gamma, \delta)) = 0$, for all $\gamma \in \mathbb{R}$ and, thus, since γ has been fixed, there exists $\bar{\delta}_1 > 0$ such that $\gamma^2 \delta^3 \frac{N^3 L^4}{4c_{\mathcal{H}}^2} + qp(\gamma, \delta) < q^2$ for all $\delta \in (0, \bar{\delta}_1)$. Therefore, by defining $\bar{\delta} := \min\{1, \bar{\delta}_1\}$ and choosing $\gamma \in (0, \bar{\gamma}_1)$ and $\delta \in (0, \bar{\delta})$, we can bound (20.40) as

$$\Delta \tilde{W}(\bar{x}^t, x_{\perp}^t) \leq -\gamma q (\delta \|\bar{x}^t\|^2 + \|x_{\perp}^t\|^2).$$

By combining the last inequality and the bounds (20.31), we ensure that the origin is a globally exponentially stable equilibrium of (20.22) [11, Theorem 13.2]. With this result at hand, the existence of the Lyapunov function W satisfying (20.23) follows by invoking the converse Lyapunov theorem (see, e.g., [6, Theorem 5.8]).

20.4.4 Proof of Theorem 20.1

Since the origin is a globally exponentially stable equilibrium of the boundary layer system (20.20) (cf. Lemma 20.2) and the origin is a globally exponentially stable equilibrium of the reduced system (20.22) (cf. Lemma 20.3), we now prove that

the origin is an exponentially stable equilibrium of the original interconnected system (20.16) via singular perturbations by slightly extending [8, Theorem II.3].

More in detail, the mentioned result requires the two stability properties provided by Lemmas 20.2 and 20.3, the Lipschitz continuity of the equilibrium function z^{eq} (cf. (20.18)), and the Lipschitz continuity of the dynamics characterizing (20.16). The Lipschitz continuity of the gradients of the objective functions (cf. Assumption 20.2) allows for claiming the Lipschitz continuity of z^{eq} but, as customary for second-order schemes, we cannot globally guarantee the desired property for the dynamics characterizing (20.16) due to the presence of $\mathcal{H}_i(x_i^t)$. However, for each algorithm initial condition, by defining $V(\tilde{x}, \tilde{z}_\perp) := U(\tilde{z}_\perp) + W(\tilde{x})$ (with U and W provided by Lemmas 20.2 and 20.3, respectively) and the level set $\Omega_0 := \{(\tilde{x}, \tilde{z}_\perp) \mid V(\tilde{x}, \tilde{z}_\perp) \leq V(\tilde{x}^0, \tilde{z}_\perp^0)\}$, we can use the continuity of each $\mathcal{H}_i$ (cf. Assumption 20.2) and the compactness of Ω_0 (see (20.21a) and (20.23a)), to ensure the desired Lipschitz continuity within Ω_0 . Once this property is achieved, we can easily adapt the arguments of [8, Theorem II.3] to guarantee the existence of $\bar{\gamma} \in (0, \bar{\gamma}_1)$ such that (i) Ω_0 is forward invariant for system (20.16) for all $\gamma \in (0, \bar{\gamma})$ and (ii) the origin is a semi-global exponentially stable equilibrium of system (20.16). The proof follows by using the exponential stability definition and reverting to the coordinates (x, z) .

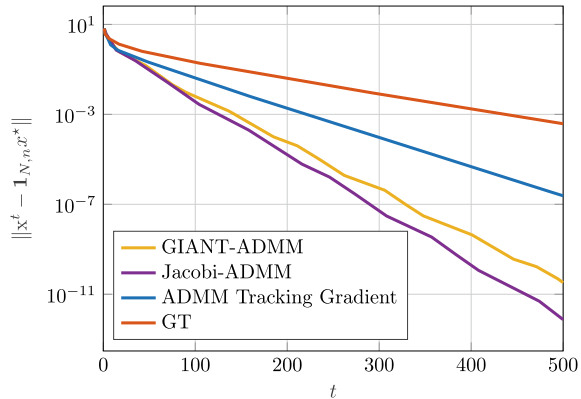
20.5 Numerical Simulations

This section reports numerical simulations comparing the proposed GIANT-ADMM, Jacobi-ADMM, with ADMM-Tracking Gradient [7, 8], and the well-known Gradient Tracking algorithm [16, 18, 22]. We focus on a classification task over a network of $N = 20$ agents. Each agent i locally stores $m_i \in \mathbb{N}$ data points $p_{i,1}, \dots, p_{i,m_i} \in \mathbb{R}^{n-1}$ and each of them is labeled with $l_{i,k} \in \{-1, 1\}$ for all $k \in \{1, \dots, m_i\}$. The goal is to train a linear classification model by solving the problem:

$$\min_{w \in \mathbb{R}^{n-1}, b \in \mathbb{R}} \sum_{i=1}^N \sum_{k=1}^{m_i} \log \left(1 + e^{-l_{i,k}(w^\top p_{i,k} + b)} \right) + \frac{C}{2} \left\| \begin{bmatrix} w \\ b \end{bmatrix} \right\|^2,$$

where $C > 0$ is the regularization weight, which ensures the strong convexity of the cost. We choose $n = 2$ and employ randomly generated data. The agents communicate according to a ring graph. We set $\alpha = 0.9$, $\rho = 0.9$, and $\gamma = 0.3$, and $\delta = 1$ for GIANT-ADMM, $\alpha = 0.9$, $\rho = 0.9$, and $\gamma = 0.3$, and $\delta = 1$ for Jacobi-ADMM, $\alpha = 0.9$, $\rho = 0.9$, $\gamma = 0.5$, and $\delta = 1$ for ADMM-Tracking Gradient, and $\gamma = 0.01$ for Gradient Tracking. Figure 20.1 compares the algorithms' performance by providing the evolution of $\|x^t - \mathbf{1}_{N,n} x^*\|$ over the iterations t . In particular, Fig. 20.1 confirms the linear convergence claimed by Theorem 20.1 and shows that both GIANT-ADMM and Jacobi-ADMM outperform the other considered distributed schemes thus proving the advantages of using second-order updates.

Fig. 20.1 Logistic regression scenario: comparison between GIANT-ADMM, Jacobi-ADMM, ADMM-Tracking Gradient, and Gradient Tracking (GT)



20.6 Conclusions

In this paper, we proposed GIANT-ADMM and Jacobi-ADMM, namely, two novel distributed methods for consensus optimization. The two algorithms, respectively, combine GIANT-like and Jacobi-like methods with ADMM-consensus to reconstruct the unavailable global quantities. The designed algorithms have been analyzed using tools of system theory based on singular perturbations. This approach guaranteed the linear convergence of the algorithms in the case of strongly convex costs. Numerical simulations confirmed the algorithms' effectiveness.

References

1. Bastianello, N., Carli, R.: Admm for dynamic average consensus over imperfect networks. *IFAC-PapersOnLine* **55**(13), 228–233 (2022)
2. Bastianello, N., Carli, R., Schenato, L., Todescato, M.: Asynchronous distributed optimization over lossy networks via relaxed admm: Stability and linear convergence. *IEEE Trans. Autom. Control* **66**(6), 2620–2635 (2020)
3. Bertsekas, D.P., Scientific, A.: *Convex optimization algorithms*. Athena Scientific Belmont (2015)
4. Bin, M., Notarnicola, I., Parisini, T.: Stability, linear convergence, and robustness of the wangelia algorithm for distributed consensus optimization. In: *2022 IEEE 61st Conference on Decision and Control (CDC)*, pp. 1610–1615. IEEE (2022)
5. Bof, N., Carli, R., Notarstefano, G., Schenato, L., Varagnolo, D.: Multiagent Newton-Raphson Optimization Over Lossy Networks. *IEEE Trans. Autom. Control* **64**(7), 2983–2990 (2019)
6. Bof, N., Carli, R., Schenato, L.: Lyapunov theory for discrete time systems. *arXiv preprint [arXiv:1809.05289](https://arxiv.org/abs/1809.05289)* (2018)
7. Carnevale, G., Bastianello, N., Carli, R., Notarstefano, G.: Distributed consensus optimization via admm-tracking gradient. In: *2023 62nd IEEE Conference on Decision and Control (CDC)*, pp. 290–295. IEEE (2023)
8. Carnevale, G., Bastianello, N., Notarstefano, G., Carli, R.: Admm-tracking gradient for distributed optimization over asynchronous and unreliable networks. *arXiv preprint [arXiv:2309.14142](https://arxiv.org/abs/2309.14142)* (2023)

9. Carnevale, G., Mimmo, N., Notarstefano, G.: A unifying system theory framework for distributed optimization and games. arXiv preprint [arXiv:2401.12623](https://arxiv.org/abs/2401.12623) (2024)
10. Carnevale, G., Notarstefano, G.: Nonconvex distributed optimization via lasalle and singular perturbations. *IEEE Control Syst. Lett.* **7**, 301–306 (2022)
11. Chellaboina, V., Haddad, W.M.: Nonlinear dynamical systems and control: A Lyapunov-based approach. Princeton University Press (2008)
12. Mansoori, F., Wei, E.: A Fast Distributed Asynchronous Newton-Based Optimization Algorithm. *IEEE Trans. Autom. Control* **65**(7), 2769–2784 (2019). <https://doi.org/10.1109/TAC.2019.2933607>
13. Maritan, A., Sharma, G., Schenato, L., Dey, S.: Network-GIANT: Fully distributed Newton-type optimization via harmonic Hessian consensus (2023). 10.48550/arXiv.2305.07898. [ArXiv:2305.07898](https://arxiv.org/abs/2305.07898) [cs, math]
14. Mokhtari, A., Shi, W., Ling, Q., Ribeiro, A.: A Decentralized Second-Order Method with Exact Linear Convergence Rate for Consensus Optimization. *IEEE Trans. Signal Inf. Process. Netw.* **2**(4), 507–522 (2016). <https://doi.org/10.1109/TSIPN.2016.2613678>
15. Nedić, A., Liu, J.: Distributed optimization for control. *Ann. Rev. Control Robot. Autonom. Syst.* **1**, 77–103 (2018)
16. Nedić, A., Olshevsky, A., Shi, W.: Achieving geometric convergence for distributed optimization over time-varying graphs. *SIAM J. Optim.* **27**(4), 2597–2633 (2017)
17. Notarstefano, G., Notarnicola, I., Camisa, A.: Distributed optimization for smart cyber-physical networks. *Found. Trends® Syst. Control* **7**(3), 253–383 (2019)
18. Shi, W., Ling, Q., Wu, G., Yin, W.: EXTRA: An exact first-order algorithm for decentralized consensus optimization. *SIAM J. Optim.* **25**(2), 944–966 (2015)
19. Tian, Y., Sun, Y., Scutari, G.: Achieving linear convergence in distributed asynchronous multiagent optimization. *IEEE Trans. Autom. Control* **65**(12), 5264–5279 (2020)
20. Wang, S., Roosta, F., Xu, P., Mahoney, M.W.: Giant: Globally improved approximate newton method for distributed optimization. In: *Advances in Neural Information Processing Systems*, vol. 31, pp. 2332–2342 (2018)
21. Wei, H., Qu, Z., Wu, X., Wang, H., Lu, J.: Decentralized approximate Newton methods for convex optimization on networked systems. *IEEE Trans. Control Network Syst.* **8**(3), 1489–1500 (2021). <https://doi.org/10.1109/TCNS.2021.3070663>
22. Xin, R., Khan, U.A.: A linear algorithm for optimization over directed graphs with geometric convergence. *IEEE Control Syst. Lett.* **2**(3), 315–320 (2018)
23. Yang, T., Yi, X., Wu, J., Yuan, Y., Wu, D., Meng, Z., Hong, Y., Wang, H., Lin, Z., Johansson, K.H.: A survey of distributed optimization. *Ann. Rev. Control* **47**, 278–305 (2019)

Chapter 21

Inexactness in Bilevel Nonlinear Optimization: A Gradient-Free Newton's Method Approach



Torbjørn Cunis and Ilya Kolmanovsky

Abstract Recurring challenges in bilevel optimization involve the need to solve the lower level optimization exactly and the fact that first- and second-order methods require differentiability of its value function. In this paper, we propose a “gradient-free” framework to analyze inexactness in bilevel nonlinear optimization in the sense that it does not depend on gradients of the lower level optimization problem. To that extent, we derive a joint system of Karush–Kuhn–Tucker necessary conditions. We then solve the bilevel optimization problem by a multi-step Newton-type method for generalized equations. In this setting, we derive input-to-state stability properties for the bilevel optimization algorithms under moderate regularity conditions and provide sufficient conditions for convergence even if the lower level problem is solved inexactly.

Keywords Bilevel optimization · Generalized equations · Subdifferentiability · Newton's method · Input-to-state stability

21.1 Introduction

Bilevel optimization refers to a class of optimization problems which are informed by the solution to a secondary (“lower level”) optimization problem parameterized by the decision variables of the “upper level” optimization. Bilevel optimization problems appear frequently in machine learning [12], control co-design in multidisciplinary design optimization [5], calibration of weights (imitation learning) or models in model predictive control [1, 11], and decomposed optimization approaches [13]. It is for that reason that several works have been dedicated to the mathematics of bilevel optimization problems [7, 20], and references herein.

T. Cunis (✉)
University of Stuttgart, Stuttgart, Germany
e-mail: tcunis@ifr.uni-stuttgart.de

T. Cunis · I. Kolmanovsky
University of Michigan, Ann Arbor, USA

A recurring issue in bilevel optimization is that the optimal value of the lower level problem and its solution are usually nondifferentiable, possibly discontinuous, and—in case of the optimal solution—often multi-valued functions of the upper level decision variable. In his thesis, Grothey [13] reviewed sufficient conditions for differentiability in nonlinear optimization problems and developed non-convex bundle methods for decomposed optimization which exploit a system of first-order approximations from gradient estimates. Mordukhovich [18] derived necessary conditions for bilevel optimization using the so-called basic subdifferential of the nondifferentiable lower level value function. Though unrelated to bilevel optimization, Toan [22] later provided a characterization for that subdifferential based on the lower level primal-dual solutions. However, subgradient descent methods for the upper level optimization problem exhibit poor convergence in general [2], Lemma 1.1. To avoid the issue of differentiability, Franceschi et al. [12] approximated the lower level problem by a finite number of iterations of a possibly gradient-based optimization algorithm, thus solving a smooth but inexact single-level optimization problem in lieu of the bilevel problem. Similarly, Ji et al. [14] exploited a finite number of lower level iterations but used warm-starting between updates of the upper level decision variables to ensure convergence to a stationary solution. Under the assumptions made in these references, the (exact) gradient of the lower level optimal value function exists.

Assuming existence of the lower level gradient, in [3], we analyzed inexact bilevel optimization via proximal gradient descent with warm-starting of the lower level solver. By viewing it as an interconnection of dynamic systems, we derived conditions on asymptotic convergence to a bilevel optimal solution within the control-theoretic framework of input-to-state stability [3]. More recently, we proved that Newton-type methods for generalized equations—which, if applied to the KKT conditions of a nonlinear optimization problem, yield the well-known sequential quadratic programming algorithm—are input-to-state stable to generic perturbations such as due to inexact computations [4]. However, the extension to generic bilevel optimization problems is nontrivial, as the notion of inexactness of the gradient estimate in the case of subdifferentiability is obscure.

In this paper, we combine the subdifferential characterization of [22] with non-smooth necessary conditions from [17] to derive a joint system of Karush–Kuhn–Tucker (KKT)-conditions for the bilevel optimization problem. We build upon results from variational analysis [8, 10] for regularity of the lower level KKT conditions. A Newton-type update of the upper level solution can then be understood as solving the joint KKT conditions using a multi-step Newton-type method [4], assuming that the lower level solver returns a (possibly inexact) primal-dual solution to the lower level KKT conditions. Notably, the bilevel KKT conditions and the upper level update are ‘gradient-free’ in the sense that neither of them involve the (sub)gradient of the lower level optimization problem.

Our main contributions are (1) to establish a subgradient estimate of the lower level optimal value function under constraint qualifications and regularity conditions that are insufficient for differentiability of the lower level problem; (2) to prove that the bilevel solution satisfies a joint KKT system under lower level regularity and a bilevel constraint qualification; and (3) to develop an ISS framework for analysis

of robust convergence of the bilevel optimization approach under inexactness, in particular, with inexact lower level evaluations. To that extent, Sect. 21.2 introduces terminology, definitions, and the bilevel optimization problem; Sect. 21.3 establishes the lower level subgradient estimate; and Sect. 21.4 demonstrates robust convergence of the proposed Newton-type method.

Notation. All vector spaces are possibly infinite-dimensional, complete norm spaces (Banach spaces) with norm $\|\cdot\|$. The dual space of X is the vector space X^* of linear forms $x^* : X \rightarrow \mathbb{R}$ with duality pairing $\langle x^*, \cdot \rangle$ and the adjoint of a linear operator $A : X \rightarrow Y$ is the operator $A^* : Y^* \rightarrow X^*$ satisfying that $\langle y^*, Ax \rangle = \langle A^*y^*, x \rangle$ for all $x \in X$ and $y^* \in Y^*$. The gradient of a function $f : X \rightarrow Y$ at $\bar{x} \in X$, if existent, is a linear operator $\nabla f(\bar{x}) : X \rightarrow Y$. We adopt the convention that for any aggregated space $X \times Y$, the norm $\|(x, y)\| := \|x\| + \|y\|$ is used. The domain and epigraph of a function $\varphi : X \rightarrow \overline{\mathbb{R}}$, where $\overline{\mathbb{R}} := \mathbb{R} \cup \{\pm\infty\}$ denotes the set of extended real numbers, are $\text{dom } \varphi = \{x \in X \mid \varphi(x) \in \mathbb{R}\}$ and $\text{epi } \varphi = \{(x, r) \in X \times \mathbb{R} \mid r \geq \varphi(x)\}$.

For many applications of bilevel optimization, it suffices to consider finite-dimensional vector spaces; in that case, one can associate $\|\cdot\|$ with the Euclidean norm, define duals by the inner (dot) product, obtain the adjoint by matrix transpose, and the gradient, if existent, is defined by the Jacobian matrix. In order to balance off generality of our results and readability, we will limit the discussion of technical assumptions only relevant to infinite-dimensional vector spaces. In particular, we refer the reader to [16] for definitions of Asplund spaces and sequential normal (epi-)compactness (SNC, SNEC).

21.2 Preliminaries

The (*basic or limiting*) *normal cone* to $\Omega \subset X$ at $\bar{x} \in \Omega$ is the set $N(\bar{x}; \Omega) \subset X^*$ of dual variables x^* for which there exists $\{(x_k, x_k^*, \epsilon_k)\}_{k \in \mathbb{N}}$ such that $\epsilon_k \searrow 0$, $x_k \rightarrow \bar{x}$, and x_k^* converges *weakly* to x^* , satisfying that $x_k \in \Omega$ and

$$\limsup_{x \rightarrow x_k} \frac{\langle x_k^*, x - x_k \rangle}{\|x - x_k\|} \leq \epsilon_k$$

for all $k \in \mathbb{N}$, and $N(x; \Omega) := \emptyset$ if $x \notin \Omega$. For a not necessarily continuous function $\varphi : X \rightarrow \overline{\mathbb{R}}$, the (*basic or limiting*) *subdifferential* at $\bar{x} \in \text{dom } \varphi$ is the set $\partial\varphi(\bar{x}) = \{x^* \in X^* \mid (x^*, -1) \in N((\bar{x}, \varphi(\bar{x})); \text{epi } \varphi)\}$. The elements of $\partial\varphi(\bar{x})$ are called *subgradients*. Similarly, the *singular subdifferential* of φ at $\bar{x}$, denoted $\partial^\infty\varphi(\bar{x})$, is the set of duals $x^* \in X^*$ such that $(x^*, 0) \in N((\bar{x}, \varphi(\bar{x})); \text{epi } \varphi)$. We have that $\partial^\infty\varphi(\bar{x}) = \{0\}$ if φ is Lipschitz continuous around $\bar{x}$ and that $\partial\varphi(\bar{x})$ is a singleton $\{\xi^*\}$ if φ is strictly differentiable at $\bar{x}$ (in which case $\xi^* = \nabla\varphi(\bar{x})$); both statements become necessary in finite-dimensional spaces for lower semicontinuous and locally Lipschitz continuous functions, respectively, [16, pp. 86–87].

21.2.1 Problem Setting

We consider the upper level optimization problem

$$\min_{p \in P} \varphi(p) \quad \text{s.t. } \mu(p) \leq 0 \text{ and } p \in \Omega, \quad (21.1)$$

where $\varphi : P \rightarrow \mathbb{R}$ is continuously differentiable, $\Omega \subset P$ is non-empty, and $\mu : P \rightarrow \mathbb{R}$ is given by the parameterized, lower level optimization problem:

$$\mu : p \mapsto \inf_{x \in X} f(x, p) \quad \text{s.t. } g(x, p) = 0 \text{ and } x \in \Theta, \quad (21.2)$$

where $f : X \times P \rightarrow \mathbb{R}$ and $g : X \times P \rightarrow \mathbb{R}^m$ are continuously differentiable and $\Theta \subset X$ is non-empty.

Under a suitable constraint qualification, the optimal solution of (21.1) satisfies the Karush–Kuhn–Tucker (KKT)-type conditions

$$\nabla \varphi(p) + q \partial \mu(p) + N(p; \Omega) \ni 0, \quad \mu(p) \in N(q; \mathbb{R}_{\geq 0}) \quad (21.3)$$

for some $q \in \mathbb{R}_{\geq 0}$. Furthermore, under similar constraint qualifications (e.g., the Mangasarian–Fromovitz constraint qualification for nonlinear programs), the optimal solution of (21.2) satisfies the KKT conditions

$$\nabla_x f(x, p) + \nabla_x g(x, p)^* y + N(x; \Theta) \ni 0, \quad g(x, p) = 0 \quad (21.4)$$

for some $y \in \mathbb{R}^m$. We denote $w = (p, q) \in W := P \times \mathbb{R}$ and $z = (x, y) \in Z := X \times \mathbb{R}^m$. For the remainder of the paper, we assume that (21.1) and (21.2) have (possibly non-unique) optimal solutions $\bar{p}$ and $\bar{x}$, respectively, and that (21.3) and (21.4) are satisfied by $\bar{p}$ and $\bar{x}$, respectively, for some $q \in \mathbb{R}$ and $y \in \mathbb{R}^m$.

We then propose a bilevel optimization scheme in which the lower level problem is solved by the primal-dual algorithm

$$z_{k+1} \in \Phi(p_k, e_k),$$

where $e_k \in \mathcal{E}$ is a degree of inexactness, possibly due to a finite number of iterations, and the upper level makes use of the gradient estimate

$$\widehat{\nabla \mu}(p; z) := \nabla_p f(x, p) + \nabla_p g(x, p)^* y \quad (21.5)$$

of μ at p for $z \in Z$. In the next section, we show that under strong regularity conditions for (21.4), $\widehat{\nabla \mu}(p_k; z_{k+1})$ becomes equal to the strict Hadamard derivative¹ of μ at p_k provided that z_{k+1} is the (unique and exact) primal-dual solution of the lower level problem. Subsequently, we derive sufficient conditions for robust convergence

¹ The strict Fréchet derivative if P has finite dimensions.

of our bilevel algorithm under weaker regularity conditions, where μ might not be differentiable, and inexact solutions of the lower level problem.

21.2.2 Set-Valued Mappings

A set-valued mapping $F : X \rightrightarrows Y$ yields a (possibly empty) set $F(x) \subset Y$ at $x \in X$. We say that F is single valued (at x) if $F(x) = \{y\}$ for some $y \in Y$. Moreover, we denote by $\text{dom } F$, $\text{graph } F$, and F^{-1} the domain, graph, and inverse of F , respectively. We now review the standard notions of set-valued continuity; to that extent, take $(\bar{x}, \bar{y}) \in \text{graph } F$. Then F has the *Aubin property*² at $\bar{x}$ for $\bar{y}$ if and only if $\text{graph } F$ is locally closed³ at $(\bar{x}, \bar{y})$ and there exist neighborhoods U of $\bar{x}$ and V of $\bar{y}$ and a constant $\kappa \geq 0$ satisfying

$$d(y; F(x_1)) \leq \kappa \|x_1 - x_2\| \quad (21.6)$$

for all $y \in F(x_2) \cap V$ and $x_1, x_2 \in U$, where $d(\cdot; C)$ denotes the distance function of a set C .

If F is single valued at $\bar{x}$, then (21.6) corresponds to Lipschitz continuity of a function. In particular, F has a *Lipschitz continuous single-valued localization* at $\bar{x}$ for $\bar{y}$ if and only if there exist neighborhoods U and V and a Lipschitz continuous function $\vartheta : X \rightarrow Y$ satisfying $F(x) \cap V = \{\vartheta(x)\}$ for all $x \in U$. On the other hand, F has the *isolated calmness property* at $\bar{x}$ for $\bar{y}$ if and only if there exist neighborhoods U and V and a constant $\kappa \geq 0$ such that $F(\bar{x}) \cap V = \{\bar{y}\}$ and (21.6) holds for $x_1 = \bar{x}$, $x_2 \in U$, and $y \in F(x_2) \cap V$. Using set-valued continuity, we can characterize the following regularity properties.⁴

Definition 21.1 (See also [8]) Let $F : X \rightrightarrows Y$ be a set-valued mapping with $(\bar{x}, \bar{y}) \in \text{graph } F$ and a constant $\kappa \geq 0$;

- F is *metrically regular* at $\bar{x}$ for $\bar{y}$ with constant κ if and only if F^{-1} has the Aubin property at $\bar{y}$ for $\bar{x}$ with constant κ ;
- F is *strongly regular* at $\bar{x}$ for $\bar{y}$ with constant κ if and only if F^{-1} has a Lipschitz continuous single-valued localization at $\bar{y}$ for $\bar{x}$ with constant κ ;
- F is *strongly subregular* at $\bar{x}$ for $\bar{y}$ with constant κ if and only if F^{-1} has the isolated calmness property at $\bar{y}$ for $\bar{x}$ with constant κ .

Important results in variational analysis are implicit function theorems for generalized equations under regularity properties. We summarize the main statements below, considering the solution mapping

² The Aubin property is also called Lipschitz-like continuity.

³ That is, $\text{graph } F \cap W$ is closed for some neighborhood W of $(\bar{x}, \bar{y})$.

⁴ See [8] for definitions and equivalences to set-valued continuity.

$$S : p \mapsto \{x \in X \mid \psi(x, p) + F(x) \ni 0\}$$

with $\psi : P \times X \rightarrow Y$ and $F : X \rightrightarrows Y$.

Theorem 21.1 Take $(\bar{p}, \bar{x}) \in \text{graph } S$ and assume that there exists a function $h : X \rightarrow Y$ satisfying $h(\bar{x}) = \psi(\bar{x}, \bar{p})$ and

- $h + F$ is strongly regular (metrically regular, strongly subregular) at $\bar{x}$ for 0 with constant $\kappa \geq 0$;
- ψ is uniformly Lipschitz continuous in p at $(\bar{x}, \bar{p})$ with constant $\gamma \geq 0$;
- $(\psi - h)$ is uniformly Lipschitz continuous in x at $(\bar{x}, \bar{p})$ with constant 0.

Then the mapping S has a Lipschitz continuous single-valued selection (has the Aubin property, has the isolated calmness property) at $\bar{p}$ for $\bar{x}$ with constant $\gamma\kappa$.

Proof This is Theorem 8.5 (under strong regularity), Theorem 5.5 (under metric regularity), and Theorem 12.4 (under strong subregularity) with $\mu = 0$ of [8]. $\square$

A corollary of Theorem 21.1 is that strong regularity, metric regularity, and strong subregularity of a set-valued mapping are preserved under (sufficiently small) Lipschitz continuous⁵ perturbation functions as well as linearization [8].

21.2.3 Basic Definitions and Assumptions

In the following, we consider the *feasible*, the *optimal*, and the *KKT solution mappings* of the parameterized optimization problem (21.2), namely:

$$\begin{aligned} K : p &\mapsto \{x \in X \mid g(x, p) = 0, x \in \Theta\} \\ S_{\text{opt}} : p &\mapsto \{x \in X \mid f(x, p) = \mu(p), x \in K(p)\} \\ S_{\text{kkt}} : p &\mapsto \{z \in Z \mid \nabla_z \ell(z, p) + N(z; \Theta \times \mathbb{R}^m) \ni 0\} \end{aligned}$$

with Lagrangian $\ell(x, y, p) \equiv f(x, p) + \langle y, g(x, p) \rangle$. We will make the following Mangasarian–Fromovitz constraint qualification (MFCQ)-type assumptions.

Assumption 21.1 (Lower level MFCQ) X and P are Asplund, Θ is locally closed, normally regular,⁶ and SNC at $\bar{x}$,

$$\{\nabla g(\bar{x}, \bar{p})^* y \mid y \in \mathbb{R}^m\} \cap (\{0\} \times [-N(\bar{x}; \Theta)]) = \{0\}, \quad (21.7)$$

and the vectors $\nabla g_1(\bar{x}, \bar{p}), \dots, \nabla g_m(\bar{x}, \bar{p})$ are linearly independent.

⁵ Strong subregularity is already preserved under calm perturbations.

⁶ A set is normally regular at a point ζ if its (basic, limiting) normal cone and its prenormal cone at this point coincide [16, Definitions 1.1 and 1.4]; in particular, any locally convex set is normally regular [16, Proposition 1.5].

Assumption 21.2 (Upper level MFCQ) P is Asplund, μ is Lipschitz at $\bar{p}$,

$$\{q, p^* \mid q \in N(\mu(\bar{p}); \mathbb{R}_{\leq 0}), p^* \in \partial\mu(\bar{p})\} \cap [-N(\bar{p}; \Omega)] = \{0\}, \quad (21.8)$$

and $\mu(\bar{p}) = 0$ only if $0 \notin \partial\mu(\bar{p})$.

As we are going to show, we can replace these two constraint qualifications by a single assumption provided that the inner problem is sufficiently regular.

Assumption 21.3 (Bilevel MFCQ) X and P are Asplund, Θ is locally closed, normally regular, and SNC at $\bar{x}$, graph K is locally closed at $(\bar{p}, \bar{x})$, and μ is SNEC⁷ at $\bar{p}$ as well as

$$\forall(p^*, y), \quad (0, p^*) \in \nabla g(\bar{x}, \bar{p})^* y + [\{0\} \times N(\bar{x}; \Theta)] \Rightarrow [p^* = 0 \wedge y = 0], \quad (21.9a)$$

$$\{q \nabla_p \ell(\bar{x}, y, \bar{p}) \mid (\bar{x}, y) \in S_{\text{kkt}}(\bar{p}), q \in N(f(\bar{x}, \bar{p}); \mathbb{R}_{\geq 0})\} \cap [-N(\bar{p}; \Omega)] = \{0\}, \quad (21.9b)$$

and $f(\bar{x}, \bar{p}) = 0$ only if $0 \neq \nabla_p \ell(\bar{x}, y, \bar{p})$ for all $(\bar{x}, y) \in S_{\text{kkt}}(\bar{p})$.

21.3 Differentiability in Parameterized Optimization

We provide sufficient conditions for the optimal value function μ of (21.2) to either be differentiable at $\bar{p}$ or, in lieu of differentiability, to be Lipschitz continuous around $\bar{p}$ with a characterization of the subdifferential at $\bar{p}$. In particular, we show that under sufficiency of the KKT conditions, strong regularity of the set-valued left-hand side in (21.4) implies differentiability, and that, in the case of a finite-dimensional nonlinear program, the assumption of strong regularity of the KKT system is not stronger than previous results in the literature. Moreover, either if the KKT system is metrically regular, or if S_{kkt} is inner semicontinuous⁸ and the KKT system is strongly subregular, we obtain a characterization of $\partial\mu(\bar{p})$ based on the lower level dual variables.

Theorem 21.2 ([22], Theorem 3.1) *Suppose that Assumption 21.1 is satisfied and that S_{opt} is inner semicontinuous at $\bar{p}$ for $\bar{x}$, then*

$$\partial\mu(\bar{p}) \subset \{\nabla_p \ell(\bar{x}, y, \bar{p}) \mid \nabla_x \ell(\bar{x}, y, \bar{p}) + N(\bar{x}; \Theta) \ni 0\}. \quad (21.10)$$

If, in addition, there exists a function $\varsigma : P \rightarrow X$ which is calm $\bar{p}$ and satisfies $\varsigma(\bar{p}) = \bar{x}$, and $\varsigma(p) \in S_{\text{kkt}}(p)$ for all p around $\bar{p}$, then (21.10), Eq. (1.48) holds with equality. $\triangleleft$

⁷ The SNEC property holds automatically if $\dim P < \infty$ [16, p. 121].

⁸ A set-valued mapping $S : P \rightrightarrows X$ is said to be *inner semicontinuous* at $\bar{p}$ for $\bar{x}$ if for every neighborhood V of $\bar{x}$ there exists a neighborhood U of $\bar{p}$ such that $F(p) \cap V \neq \emptyset$ for all $p \in U$; cf. [16, Definition 1.63].

Note that, for any $y \in \mathbb{R}^m$, we have $\nabla_x \ell(\bar{x}, y, \bar{p}) + N(\bar{x}; \Theta) \ni 0$ if and only if $(\bar{x}, y) \in S_{\text{kkt}}(\bar{p})$. In other words, the right-hand side of (21.10) is enumerated by the dual variables of (21.2). For μ to be (strictly) differentiable at $\bar{p}$, we need that μ is Lipschitz continuous around $\bar{p}$ and the lower level dual variables to be unique.⁹

Proposition 21.1 *Suppose that Assumption 21.1 is satisfied; if S_{opt} is inner semi-continuous at $\bar{p}$ for $\bar{x}$, μ is SNEC at $\bar{p}$,*

$$\forall (p^*, y), \quad (0, p^*) \in \nabla g(\bar{x}, \bar{p})^* y + [\{0\} \times N(\bar{x}; \Theta)] \Rightarrow p^* = 0, \quad (21.11)$$

and graph K is locally closed at $(\bar{p}, \bar{x})$, then μ is Lipschitz continuous around $\bar{p}$.

Proof Since f is continuously differentiable (a fortiori Lipschitz around $\bar{p}$), if graph K is locally closed at $(\bar{p}, \bar{x})$ and S_{opt} is inner semicontinuous at $\bar{p}$ for $\bar{x}$, we have by virtue of [16, Corollary 3.40], together with [16, Theorem 4.31](ii),

$$\partial^\infty \mu(\bar{p}) \subset \{p^* \in P^* \mid \exists y \in \mathbb{R}^m, (0, p^*) \in \nabla g(\bar{x}, \bar{p})^* y + [\{0\} \times N(\bar{x}; \Theta)]\} \quad (21.12)$$

as Θ is locally closed and normally regular at $\bar{x}$ by Assumption 21.1. By nonemptiness of $\partial^\infty \mu$ on $\text{dom } \mu$, if (21.11) holds, we then have that $\partial^\infty \mu(\bar{p}) = \{0\}$ and hence μ is Lipschitz continuous around $\bar{p}$ by virtue of [16] Theorem 3.52, since μ is SNEC and lower semicontinuous under the assumptions made [16, p. 300]. $\square$

The constraint qualification (21.11) together with (21.7) and linear independence of $\nabla g(\bar{x}, \bar{p})$ is jointly equivalent to Eq. (21.9a) in Assumption 21.3 [16, Corollary 4.41]. Having established Lipschitz continuity of the optimal value function, we obtain our first differentiability result.

Corollary 21.1 *Suppose that Assumption 21.1 and the sufficient conditions in Proposition 21.1 are satisfied. If there exists a unique $y' \in \mathbb{R}^m$ satisfying $(\bar{x}, y') \in S_{\text{kkt}}(\bar{p})$, then μ is strictly Hadamard¹⁰ differentiable at $\bar{p}$.*

Proof Since, by Proposition 21.1, μ is Lipschitz continuous around $\bar{p}$, we have that $\partial \mu(\bar{p}) \neq \emptyset$ by virtue of [16, Corollary 2.25]. Theorem 21.2 then implies that $\partial \mu(\bar{p}) = \{\nabla_p \ell(\bar{x}, y', \bar{p})\}$ by uniqueness of y' and strict Hadamard differentiability at $\bar{p}$ follows from [16, Theorem 3.54], again by Lipschitz continuity of μ . $\square$

It is important to note that even in finite dimensions, strict differentiability at $\bar{p}$ is not sufficient to conclude continuous differentiability around $\bar{p}$, but μ must be strictly differentiable on a neighborhood of that point [19, Theorem 25.23].

⁹ While the subdifferential is necessarily a singleton (bounded set) for a strictly differentiable (Lipschitz continuous) function, the reverse is not true; see [16] for a non-Lipschitzian, nondifferentiable function with a unique subgradient.

¹⁰ *Strict Hadamard differentiability* refers to the existence of a derivative derived from directional differentiation in Asplund spaces, which is necessary (in finite dimensions, also sufficient) for strict (Fréchet) differentiability [16, pp. 312–313].

Based on the considerations above, inner semicontinuity of the optimal solution mapping is crucial for Lipschitz continuity (and, hence, existence of at least one subderivative), but to fully characterize the set $\partial\mu(\bar{p})$ we would like to have a stronger property of the so-called calm selection ς as described in Theorem 21.2. As it turns out, the set-valued continuity notions defined in the previous section imply a calm selection.

Proposition 21.2 *Suppose that either of the following statements is true:*

- (a) S_{opt} has a Lipschitz continuous single-valued localization,
- (b) S_{opt} has the Aubin property, or
- (c) S_{opt} has the isolated calmness property and is inner semicontinuous

at $\bar{p}$ for $\bar{x}$. Then there exists a function $\varsigma : P \rightarrow X$ which is calm at $\bar{p}$ and satisfies $\varsigma(\bar{p}) = \bar{x}$ and $\varsigma(p) \in S_{\text{opt}}(p)$ for all p in a neighborhood of $\bar{p}$.

Proof Take $\varsigma(\bar{p}) = \bar{x}$ and recall that $\bar{x} \in S_{\text{opt}}(\bar{p})$. Suppose that case (c) holds true; by the isolated calmness property, there exist neighborhoods U of $\bar{p}$ and V of $\bar{x}$ together with a constant $\kappa \geq 0$ such that

$$\|\varsigma(p) - \varsigma(\bar{p})\| \leq \kappa \|p - \bar{p}\| \quad (21.13)$$

for any choice $\varsigma(p) \in S_{\text{opt}}(p) \cap V$, where the intersection is non-empty by inner semicontinuity (making U smaller if necessary), for all $p \in U$.

Suppose now that (b) holds true and let U and V be the corresponding neighborhoods of $\bar{p}$ and $\bar{x}$, respectively, then $S(p) \cap V \neq \emptyset$ for all $p \in U$ by virtue of [8, Proposition 4.1] and (21.6) implies (21.13) for some $\varsigma(p) \in S_{\text{opt}}(p)$ since $\bar{x} \in S_{\text{opt}}(\bar{p}) \cap V$. As (a) implies (b), our proof is complete. $\square$

Applying the implicit function theorem for generalized equations in the previous section (Theorem 21.1) to the parameterized KKT system (21.4), we obtain the following sufficient conditions for subdifferentiability of μ at $\bar{p}$.

Corollary 21.2 *Assume that the KKT conditions in (21.4) at $(z, p) \in Z \times P$ with $z = (x, y)$ are sufficient for optimality of x in the sense of (21.2) given p for all $(p, z) \in \text{graph } S_{\text{kkt}}$ sufficiently close to $(\bar{p}, \bar{z})$. Suppose that Assumption 21.1 is satisfied and that the set-valued mapping $\nabla_z \ell(\cdot, \bar{p}) + N(\cdot, \Theta \times \mathbb{R}^m)$ is either (a) metrically regular, or (b) strongly subregular and S_{kkt} is inner semicontinuous at $\bar{z}$ for 0. Then the characterization (21.10) of $\partial\mu(\bar{p})$ holds with equality.*

Moreover, if $\nabla_z \ell(\cdot, \bar{p}) + N(\cdot, \Theta \times \mathbb{R}^m)$ is strongly regular at $\bar{z}$ for 0, the optimal value μ is Lipschitz continuous around $\bar{p}$, and P has a finite dimension, then μ is continuously differentiable around $\bar{p}$.

Proof By virtue of Theorem 21.1 with $h : z \mapsto \nabla_z \ell(z, \bar{p})$ and $F = N(\cdot, \Theta \times \mathbb{R}^m)$, where uniform Lipschitz continuity is a consequence of continuous differentiability of f and g , the solution mapping S_{kkt} of (21.4) has the Aubin property (if $h + F$ is metrically regular) or has the isolated calmness property (if $h + F$ is strongly

subregular) at $\bar{p}$ for $\bar{z}$. Sufficiency of the KKT conditions implies that S_{sol} has the corresponding property (including inner semicontinuity) at $\bar{p}$ for $\bar{x}$. Theorem 21.2 together with Proposition 21.2 then gives us the desired result for $\mu(\bar{p})$.

If, in addition, $h + F$ is strongly regular at $\bar{p}$, then S_{kkt} and S_{opt} have Lipschitz continuous single-valued localizations; in particular, $\partial\mu(p)$ is single valued for p around $\bar{p}$. Lipschitz continuity of μ together with P being finite dimensional then yields continuous differentiability around $\bar{p}$ by virtue of [16, Theorem 3.54], and [19, Theorem 25.23]. $\square$

In the case of a classical nonlinear programming problem in finite dimensions (that is, f and g are twice differentiable and Θ is polyhedral), strong regularity of the KKT system in (21.4) at $\bar{p}$ implies that $\bar{x}$ is a *strongly stable solution* in the sense of [15]. Strong stability of an optimal solution, on the other hand, is necessary and sufficient for a strong second-order sufficient condition under the classical Mangasarian–Fromovitz (MFCQ) or linear independence (LICQ) constraint qualifications [15, Theorem 6.5 with Lemma 6.3 and 7.2], and the sufficiency condition holds around $\bar{p}$ too. Moreover, strong stability implies differentiability of μ around $\bar{p}$ together with S_{kkt} having a single-valued Lipschitz continuous localization [13, Theorem 3.9], that is, the KKT system is strongly regular at $\bar{p}$.

21.4 Bilevel Newton Method

We now turn our attention to the bilevel optimization problem of (21.1) subject to (21.2). To that extent, we derive a joint KKT system of necessary conditions for a bilevel optimal solution under Assumption 21.3. We then apply our recently proposed multi-step Newton-type method for multivariate generalized equations [4] to solve the joint KKT conditions. Here, the inner loop optimization is replaced by solving the (now partial) KKT system given by (21.4). Moreover, the joint KKT system does not depend on the subdifferential of the lower level optimal value function anymore. We conclude by formulating sufficient conditions for convergence of the bilevel optimization under inexact computations of the inner loop, which are derived within the input-to-state stability framework for dynamic systems. Our robust convergence result relies on the inexactness in the generalized equation (21.4) instead of the accuracy of the gradient estimate (21.5).

21.4.1 Bilevel Necessary Conditions

To begin with, note that the upper level KKT system (21.3) necessarily holds at $\bar{w} = (\bar{p}, \bar{q})$ under Assumption 21.2 by virtue of [16, Corollary 5.10 together with

Theorem 5.8(ii)], where $\partial(\bar{q} \mu)(\bar{p}) = \bar{q} \partial\mu(\bar{p})$ since $\bar{q} \geq 0$. If, in addition, Assumption 21.1 and the regularity and sufficiency conditions of Corollary 21.2 hold, we have that $\widehat{\nabla\mu}(\bar{p}; \bar{z}) \in \partial\mu(\bar{p})$. We then obtain the following joint KKT system.

Lemma 21.1 *Suppose that Assumption 21.3 as well as the regularity and sufficiency conditions of Corollary 21.2 are satisfied, then Assumptions 21.1 and 21.2 are satisfied and the system of generalized equations*

$$\begin{pmatrix} \nabla\varphi(p) + q \nabla_p \ell(x, y, p) \\ -f(x, p) \end{pmatrix} + N((p, q)); \Omega \times \mathbb{R}_{\geq 0}) \ni 0 \quad (21.14a)$$

$$\begin{pmatrix} \nabla_x f(x, p) + \nabla_x g(x, p)^* y \\ g(x, p) \end{pmatrix} + N((x, y)); \Theta \times \mathbb{R}^m) \ni 0 \quad (21.14b)$$

holds necessarily at $(p, x) = (\bar{p}, \bar{x})$ for some $(q, y) = (\bar{q}, \bar{y}) \in \mathbb{R}^{m+1}$.

Proof By virtue of [16, Corollary 4.41], the constraint qualification (21.9a) in Assumption 21.3 implies (21.7) and linear independence of $\nabla g(\bar{x}, \bar{p})$ in Assumption 21.1 as well as the constraint qualification (21.11) in Proposition 21.1; hence, Assumption 21.1 is satisfied and, since S_{opt} is inner semicontinuous due to the regularity and sufficiency assumptions, μ is Lipschitz continuous around $\bar{p}$. Moreover, we have that $p^* \in \partial\mu(\bar{p})$ if and only if $p^* = \nabla_x \ell(\bar{x}, y, \bar{p})$ with $(\bar{x}, y) \in S_{\text{kkt}}(\bar{p})$ for any $p^* \in P^*$; that is, (21.9b) in Assumption 21.3 and (21.8) in Assumption 21.2 are equivalent. As $\mu(\bar{p}) = f(\bar{x}, \bar{p})$, Assumption 21.2 thus is satisfied, too.

To conclude the proof,¹¹ note that Eq. (21.14a) is obtained from the upper level KKT condition (21.3) using the same characterization, where $(\bar{x}, y) \in S_{\text{kkt}}(\bar{p})$ if and only if the lower level KKT condition (21.4) is satisfied, since $g(\bar{x}, \bar{p}) = 0$ by optimality of $\bar{x}$ at $\bar{p}$, and (21.4) is equal to (21.14b). $\square$

To summarize Lemma 21.1, the joint KKT system (21.14) corresponds to the upper and lower level necessary conditions (21.3) and (21.4), where the subdifferential $\partial\mu(\bar{p})$ is replaced by the gradient estimate $\widehat{\nabla\mu}(\bar{p}; \bar{z}) = \nabla_p \ell(\bar{x}, \bar{y}, \bar{p})$ for some $\bar{z} \in S_{\text{kkt}}(\bar{p})$. However, while under the assumptions made here, any dual variable $y \in \mathbb{R}^m$ satisfies (21.14b) at $\bar{x}$ if and only if $\nabla_p \ell(\bar{x}, y, \bar{p}) \in \partial\mu(\bar{p})$, not all thus derived gradient estimates necessarily satisfy (21.14a) at $\bar{p}$. For the ease of notation in the subsequent derivation, note that the first term in (21.14b) is equal to $\nabla_{(x,y)} \ell(x, y, p)$.

21.4.2 A Multi-step Newton-Type Method

Having derived joint necessary conditions for the bilevel optimization problem (21.1)–(21.2), we propose a Newton-type iterative method that solves for $(w, z) \in$

¹¹ Recall that $N(w; \Omega \times \mathbb{R}_{\geq 0}) = N(p; \Omega) \times N(q; \mathbb{R}_{\geq 0})$ and $N(z; \Theta \times \mathbb{R}^m) = N(x; \Theta) \times \{0\}$ by virtue of [16, Proposition 1.2], as $N(y; \mathbb{R}^m) = \{0\}$ for all $y \in \mathbb{R}^m$.

$W \times Z$ satisfying (21.14). Specifically, we assume that at each iteration $k \in \mathbb{N}$, the lower level algorithm $z_{k+1} \in \Phi(p_k, e_k)$ *inexactly* solves the lower level necessary conditions (21.14b) for a given upper level solution $w_k = (p_k, q_k) \in W$, that is,

$$\nabla_z \ell(z_{k+1}, p_k) + N(z_{k+1}; \Theta \times \mathbb{R}^m) \ni e_k \quad (21.15)$$

with inexactness $e_k \in \mathcal{E} = X^* \times \mathbb{R}^m$. We then perform a Newton-type update for the upper level solution, given by the generalized equation

$$\widehat{\nabla L}(w_k; z_{k+1}) + H_{ww}(w_k; z_{k+1})(w_{k+1} - w_k) + N(w_{k+1}; \Omega \times \mathbb{R}_{\geq 0}) \ni 0, \quad (21.16)$$

where $H_{ww}(w; z) : W \rightarrow W^* = P^* \times \mathbb{R}$ is an approximation of the Hessian around the current upper level solution w_k , parameterized in the lower level solution z_{k+1} , and $\widehat{\nabla L}(w; z) := (\nabla \varphi(p) + q \widehat{\nabla \mu}(p; z), -f(x, p))$. We then combine (21.15) and (21.16) into the perturbed multivariate Newton-type step

$$\left(\widehat{\nabla L} \right)_k + \begin{bmatrix} H_{ww} & \star \\ H_{zw} & H_{zz} \end{bmatrix}_k \begin{pmatrix} w_{k+1} - w_k \\ z_{k+1} - z_{k+1} \end{pmatrix} + \begin{pmatrix} N(w_{k+1}; \Omega \times \mathbb{R}_{\geq 0}) \\ N(z_{k+1}; \Theta \times \mathbb{R}^m) \end{pmatrix} \ni \begin{pmatrix} 0 \\ d_k + e_k \end{pmatrix}, \quad (21.17)$$

where $H_{zw}(\cdot) : W \rightarrow Z^* = X^* \times \mathbb{R}^m$ and $H_{zz}(\cdot) : Z \rightarrow Z^*$ are operators analogous to $H_{ww}(\cdot)$ in (21.16); $d_k \in Z^*$ is an approximation error introduced by the block H_{zw} ; and the subscript “ k ” denotes evaluation at the solution (w_k, z_{k+1}) . Note that the upper right element of the combined operator H is omitted as it can be chosen arbitrarily. We rewrite (21.17) into

$$\Lambda_H(\vartheta^+, \vartheta) + \mathcal{N}(\vartheta^+) \ni v_k$$

with $\vartheta^+ = (w_{k+1}, z_{k+1})$, $\vartheta = (w_k, z_{k+1})$, and $v_k = (0, d_k + e_k)$. Observe that $\vartheta^+ = \vartheta = \bar{\vartheta} := (\bar{w}, \bar{z})$ is a solution of (21.17) for $v_k = 0$ since $(\bar{w}, \bar{z})$ satisfies (21.9). By rewriting the bilevel optimization scheme into a single Newton-type update, the error of the lower level gradient estimate $\widehat{\nabla \mu}$ now appears as perturbation on the right-hand side of (21.17). We adapt the following result from [4, Theorem 3].

Proposition 21.3 *Suppose that*

- (a) $\nabla_z \ell(\cdot, \bar{p}) + N(\cdot; \Theta \times \mathbb{R}^m)$ is strongly subregular with constant $\tilde{\kappa} \geq 0$ at $\bar{z}$ for 0;
- (b) $\Lambda_H(\cdot, (\bar{w}, \bar{z})) + \mathcal{N}$ is strongly subregular with constant $\kappa \geq 0$ at $(\bar{w}, \bar{z})$ for 0;
- (c) Λ_H is uniformly Lipschitz continuous in ϑ with constant $\gamma \geq 0$ at $(\bar{\vartheta}, \bar{\vartheta})$;
- (d) Λ_H is uniformly Lipschitz continuous in ϑ^+ at $(\bar{\vartheta}, \bar{\vartheta})$;

and $\kappa\gamma < 1$, then $\bar{\vartheta} = (\bar{w}, \bar{z})$ is locally input-to-state stable to v in the sense that there exists neighborhoods $O \subset W \times Z$ of $(\bar{w}, \bar{z})$ and $V \subset W^* \times Z^*$ of 0 such that any sequence $\{(w_k, z_k)\}_{k=0}^\infty \subset O$ generated by (21.15) and (21.16), such that (21.17) holds for some $\{v_k\}_{k=0}^\infty \subset V$, satisfies

$$\|(w_N, z_N) - (\bar{w}, \bar{z})\| \leq \alpha_N \|(w_0, z_0) - (\bar{w}, \bar{z})\| + \gamma_\infty \sup_{k \in \{0, \dots, N\}} \|v_k\| \quad (21.18)$$

for all $N \in \mathbb{N}$ with $\alpha_N, \gamma_\infty \geq 0$ satisfying $\lim_{N \rightarrow \infty} \alpha_N = 0$.

Proof As $\nabla_z \ell$ is Lipschitz continuous by the assumptions made in Sect. 21.2, the proof follows along the lines of [4, Proof Theorem 3] using a multivariate implicit function theorem for (21.17) under strong subregularity [4, Theorem 4]. $\square$

As a consequence of the multivariate implicit function theorem, any sequence $\{z_k\}_{k=0}^\infty$ generated by (21.15) for $\{(p_k, e_k)\} \subset P \times \mathcal{E}$ also satisfies

$$\|z_{k+1} - \bar{z}\| \leq \tilde{\gamma} \|p_k - \bar{p}\| + \tilde{\kappa} \|e_k\|$$

for all $k \in \mathbb{N}$ with $\tilde{\gamma} \geq 0$ if (z_k, p_k, e_k) remains close to $(\bar{z}, \bar{p}, 0)$. Note, however, that strong subregularity does not guarantee *existence* of suitable sequences [9].

21.4.3 Convergence Under Inexactness

An immediate consequence of Proposition 21.3 is that we get convergence of the bilevel optimization scheme (21.15)–(21.16) under *forcing sequences*, a concept well known from Newton methods for systems of conventional equations [6].

Corollary 21.3 *If, under the assumptions of Proposition 21.3, the perturbation $\{v_k\} \subset V$ satisfies $\lim_{k \rightarrow \infty} \|v_k\| = 0$, then any sequence $\{(w_k, z_k)\}_{k=0}^\infty \subset O$ satisfying (21.17) converges to $(\bar{w}, \bar{z})$. $\triangleleft$*

Convergence under forcing sequences is a useful property if we can control the degree of inexactness to which the lower level KKT conditions are satisfied. For example, we could adjust the number of iterations in the lower level solver to ensure that the norm of e_k in (21.15) is below a decreasing threshold. This is particularly advantageous as the degree of inexactness can be evaluated without the (a priori unknown) solutions $\bar{z}$ or $\bar{w}$, unlike the solution error $z_k - \bar{z}$ or the error of the gradient estimate $\widehat{\nabla} \mu$.

Remark 21.1 Unlike $e_k \in \mathcal{E}$, the approximation error d_k enters into (21.17) as result of the conflicting block H_{zw} . How to choose H in order to satisfy the assumptions in Proposition 21.3 and ensure that $\|d_k\| \rightarrow 0$ is left as an open question for now. A possible approach is to augment the left-hand side of (21.15) by a term

$$-H_{zw}(w_k; \hat{z}) \circ H_{ww}(w_k; \hat{z})^{-1} \circ \widehat{\nabla L}(w_k; z_{k+1})$$

provided that the operator H_{ww} is invertible around $(\bar{w}, \bar{z})$; if $\hat{z} = z_{k+1}$ and p_k satisfies (21.16), then (w_k, z_{k+1}) satisfies (21.17) with $d_k = 0$. On the other hand, given the robustness of Newton-type methods and assuming that the operators in H are continuous in the parameters, we conjecture that an approximation with $\hat{z} = z_k$ would still lead to acceptable results.

Our final result shows that convergence under inexactness can be ensured with a finite number of iterations for the lower level solver. Here, we assume that the approximation satisfies $d_k \equiv 0$ (see Remark 21.1). Suppose now that the lower level solution at the k -th upper level iteration is given by the dynamic system

$$z_{k+1} = \Psi_T(z_k, p_k), \quad e_k = \Gamma(z_k, p_k) \quad (21.19)$$

where $\Psi_T : Z \times P \rightarrow Z$ implements $T \in \mathbb{N}$ iterations of a suitable algorithm to solve (21.4), e.g., Newton's method for generalized equations; $\Gamma : Z \times P \rightarrow \mathcal{E}$ simply returns a value e_k satisfying (21.15) for (z_{k+1}, p_k) ; and p_k is again the primal component of the upper level solution w_k updated in (21.16). Let γ_∞ be again the gain of the nominal scheme in Proposition 21.3.

Assumption 21.4 The dynamic system (21.19) is locally input-to-output stable, that is, there exists a neighborhood $U := U_z \times U_p \subset Z \times P$ of $(\bar{z}, \bar{p})$ such that

$$\|\Gamma(z_N, p_N)\| \leq \beta_{N,T} \|z_0 - \bar{z}\| + \gamma_T \sup_{k \in \{0, \dots, N\}} \|p_k - \bar{p}\| \quad (21.20)$$

for all $N \in \mathbb{N}$, $z_0 \in U_z$, and $\{(z_k, p_k)\}_{k=0}^N \subset Z \times U_p$ satisfying (21.19), and $\beta_{N,T}, \gamma_T \geq 0$ satisfy $\lim_{N \rightarrow \infty} \beta_{N,T} = 0$ for all $T \in \mathbb{N}$ and $\limsup_{T \rightarrow \infty} \gamma_T < \gamma_\infty^{-1}$.

Assumption 21.4 states that the lower level solution algorithm (21.19) returns an increasingly accurate solution (measured by $\|e_k\|$) for (21.15) as the number of iterations T increases. Using a classical small-gain argument, we prove that (21.19) together with (21.16) converges to the bilevel optimal solution if T is sufficiently large (but finite).

Theorem 21.3 Suppose that Assumption 21.4 as well as the assumptions of Proposition 21.3 hold and that $d_k \equiv 0$; there exist a neighborhood $\tilde{O} \subset W \times Z$ and a number $T_0 \in \mathbb{N}$ such that any sequence $\{(w_k, z_k)\}_{k=0}^\infty \subset \tilde{O}$ generated by (21.19) and (21.16) with $T \geq T_0$ converges to $(\bar{w}, \bar{z})$.

Proof Choose $\tilde{O}$ such that any $(w, z) \in \tilde{O}$ satisfies $(w, z) \in O$ and $\Gamma(z, p) \in V$ (as defined in Proposition 21.3) as well as $(z, p) \in U$ (as defined in Assumption 21.4). Let $\{(w_k, z_k)\}_{k=0}^\infty \subset \tilde{O}$ be a sequence generated by (21.19) and (21.16) with some $T \in \mathbb{N}$, and observe that (w_k, z_k) and $e_k = \Gamma(z_k, p_k)$ satisfies (21.17) for $v_k = e_k$. Hence, Eqs. (21.18) and (21.20) imply, by virtue of Proposition 21.3, Assumption 21.4, and [21, Theorem 3.1 and comments], that

$$\limsup_{N \rightarrow \infty} \|(w_N, z_N) - (\bar{w}, \bar{z})\| \leq \gamma_\infty \limsup_{N \rightarrow \infty} \|e_N\| \leq \gamma_\infty \gamma_T \limsup_{N \rightarrow \infty} \|p_N - \bar{p}\|.$$

Since $\lim_{T \rightarrow \infty} \gamma_T < \gamma_\infty^{-1}$ by Assumption 21.4, there exists T_0 such that $\gamma_\infty \gamma_T < 1$ for all $T \geq T_0$, and noting that $\|p\| \leq \|(w, z)\|$ for any $w = (p, q) \in W$ and $z \in Z$, we have that $\sup_{N \rightarrow \infty} \|(w_N, z_N) - (\bar{w}, \bar{z})\| = 0$, desired result. $\square$

Assumption 21.4 is satisfied, for example, by the Newton method for the lower level KKT conditions under the assumption of strong subregularity.

Example 21.1 Suppose that (21.19) is given by a single Newton update [8, Eq. (15.3)]

$$\nabla_z \ell(z_k, p_k) + \nabla_{zz}^2 \ell(z_k, p_k)(z_{k+1} - z_k) + N(z_{k+1}; \Theta \times \mathbb{R}^m) \ni 0 \quad (21.21)$$

assuming that $\nabla_z \ell$ is continuously differentiable with respect to z around $(\bar{z}, \bar{p})$. If $\nabla_z \ell(\cdot, \bar{p}) + N(\cdot; \Theta \times \mathbb{R}^m)$ is strongly subregular at $\bar{z}$ for 0, then

$$\|z_{k+1} - \bar{z}\| \leq \alpha \|z_k - \bar{z}\| + \tilde{\gamma} \|p_k - \bar{p}\| \quad (21.22)$$

with $\alpha \in [0, 1)$ and $\tilde{\gamma} \geq 0$ by virtue of [4, Theorem 2], provided that $\{(z_k, p_k)\}_{k=0}^\infty$ exists and remains close to $(\bar{z}, \bar{p})$. Moreover, we have that

$$\begin{aligned} \delta_k &:= \nabla_z \ell(z_{k+1}, p_k) - \nabla_{zz}^2 \ell(z_k, p_k)(z_{k+1} - z_k) - \nabla_z \ell(z_k, p_k) \\ &\in \nabla_z \ell(z_{k+1}, p_k) + N(z_{k+1}; \Theta \times \mathbb{R}^m) \end{aligned}$$

and $\|\delta_k\| \leq \varrho \|z_{k+1} - z_k\|^2$ for some $\varrho \geq 0$ by Lipschitz continuity of $\nabla_z \ell$ (cf. [8, Proof of Theorem 15.1]). Equation (21.22) yields

$$\|z_N - \bar{z}\| \leq \alpha^N \|z_0 - \bar{z}\| + \frac{\tilde{\gamma}}{1 - \alpha} \sup_{k \in \mathbb{N}} \|p_k - \bar{p}\|$$

for all $N \in \mathbb{N}$ and, hence, with $\|z_{N+1} - z_N\| \leq \|z_{N+1} - \bar{z}\| + \|z_N - \bar{z}\|$ by the triangle inequality:

$$\begin{aligned} \|\delta_N\| &\leq 2\varrho(\alpha^{N+1} + \alpha^N)^2 \|z_0 - \bar{z}\|^2 + 8\varrho \left(\frac{\tilde{\gamma}}{1 - \alpha}\right)^2 \sup_k \|p_k - \bar{p}\|^2 \\ &:= \beta_N \|z_0 - \bar{z}\| + \hat{\gamma} \sup_k \|p_k - \bar{p}\| \end{aligned}$$

with $\beta_N \rightarrow 0$ as $N \rightarrow \infty$ and $\hat{\gamma} < \gamma_\infty^{-1}$ if p_k is sufficiently close to $\bar{p}$ for all $k \in \mathbb{N}$. Hence, (21.21) with $\Gamma(z_k, p_k) = \delta_k$ satisfies Assumption 21.4 where $\gamma_T \equiv \hat{\gamma}$.

Acknowledgements This work was partially completed during the first author's stay at Northeastern University. Support by Eduardo Sontag is gratefully acknowledged.

References

1. Amos, B., Rodriguez, I.D.J., Sacks, J., Boots, B., Kolter, J.Z.: Differentiable MPC for End-to-end Planning and Control. In: Annual Conference on Neural Information Processing Systems. Montréal (2018). <https://arxiv.org/abs/1810.13400>
2. Bagirov, A.M., Gaudioso, M., Karmitsa, N., Mäkelä, M.M., Taheri, S. (eds.): Numerical non-smooth optimization: State of the art algorithms. Springer, Cham (2020). <https://doi.org/10.1007/978-3-030-34910-3>
3. Cunis, T., Kolmanovsky, I.: Input-to-State Stability of a Bilevel Proximal Gradient Descent Algorithm. IFAC-PapersOnLine **56**(2), 7474–7479 (2023). <https://doi.org/10.1016/j.ifacol.2023.10.633>
4. Cunis, T., Kolmanovsky, I.: Input-to-State Stability of Newton Methods for Generalized Equations in Nonlinear Optimization. In: 2024 IEEE Conference on Decision and Control, pp. 5950–5956. Milano (2024). <https://doi.org/10.1109/CDC56724.2024.10885904>
5. Cunis, T., Kolmanovsky, I., Cesnik, C.E.: Integrating nonlinear controllability into a multi-disciplinary design process. Journal of Guidance, Control, and Dynamics **46**(6), 1026–1037 (2023). <https://doi.org/10.2514/1.G007067>
6. Dembo, R.S., Eisenstat, S.C., Steihaug, T.: Inexact Newton Methods. SIAM Journal on Numerical Analysis **19**(2), 400–408 (1982)
7. Dempe, S., Zemkoho, A. (eds.): Bilevel Optimization: Advances and Next Challenges. Springer, Cham (2020). <https://doi.org/10.1007/978-3-030-52119-6>
8. Dontchev, A.L.: Lectures on Variational Analysis. No. 205 in Applied Mathematical Sciences. Springer, Cham (2021). <https://doi.org/10.1007/978-3-030-79911-3>
9. Dontchev, A.L., Rockafellar, R.T.: Convergence of inexact newton methods for generalized equations. Mathematical Programming **139**(1–2), 115–137 (2013). <https://doi.org/10.1007/s10107-013-0664-x>
10. Dontchev, A.L., Rockafellar, R.T.: Implicit Functions and Solution Mappings, 2 edn. Springer (2014)
11. Drgoňa, J., Kiš, K., Tuor, A., Vrabie, D., Klaučo, M.: Differentiable predictive control: Deep learning alternative to explicit model predictive control for unknown nonlinear systems. Journal of Process Control **116**, 80–92 (2022). <https://doi.org/10.1016/j.jprocont.2022.06.001>
12. Franceschi, L., Frascioni, P., Salzo, S., Grazi, R., Pontil, M.: Bilevel Programming for Hyperparameter Optimization and Meta-Learning. In: Proceedings of the 35th International Conference on Machine Learning, pp. 1568–1577. Stockholm (2018)
13. Grothey, A.: Decomposition Methods for Nonlinear Nonconvex Optimization Problems. Ph.D. thesis, The University of Edinburgh, Edinburgh (2001)
14. Ji, K., Yang, J., Liang, Y.: Bilevel Optimization: Convergence Analysis and Enhanced Design. In: Proceedings of the 38th International Conference on Machine Learning, pp. 4882–4892. Virtual (2021)
15. Kojima, M.: Strongly Stable Stationary Solutions in Nonlinear Programs. In: S.M. Robinson (ed.) Analysis and Computation of Fixed Points, pp. 93–138. Academic Press, New York, NY (1980). <https://doi.org/10.1016/b978-0-12-590240-3.50009-4>
16. Mordukhovich, B.S.: Variational Analysis and Generalized Differentiation I: Basic Theory. No. 330 in Grundlehren der mathematischen Wissenschaften. Springer, Berlin, Heidelberg (2006). <https://doi.org/10.1007/3-540-31247-1>
17. Mordukhovich, B.S.: Variational Analysis and Generalized Differentiation II: Applications. No. 331 in Grundlehren der mathematischen Wissenschaften. Springer, Berlin, Heidelberg (2006). <https://doi.org/10.1007/3-540-31246-3>
18. Mordukhovich, B.S.: Bilevel Optimization and Variational Analysis. In: S. Dempe, A. Zemkoho (eds.) Bilevel Optimization: Advances and Next Challenges, pp. 197–226. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-52119-6_7
19. Schechter, E.: Handbook of Analysis and Its Foundations. Academic Press, San Diego, CA (1997). <https://doi.org/10.1016/B978-0-12-622760-4.X5000-6>

20. Shikhman, V.: Bilevel Optimization. In: Topological Aspects of Nonsmooth Optimization, pp. 141–165. Springer, New York, NY (2012). https://doi.org/10.1007/978-1-4614-1897-9_5
21. Sontag, E.D.: Input to State Stability: Basic Concepts and Results. In: P. Nistri, G. Stefani (eds.) Nonlinear and Optimal Control Theory, pp. 163–220. Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-77653-6_3
22. Toan, N.T.: Differential stability of discrete optimal control problems with mixed constraints. Positivity (2021). <https://doi.org/10.1007/s11117-021-00812-x>

Correction to: Investigation of the Influence of Training Data and Methods on the Control Performance of MPC Utilizing Gaussian Processes



Florian Diepers, Elmar Ahle, and Dirk Söffker

Correction to:
Chapter 6 in: J. Berberich et al. (eds.), *Systems Theory in Data and Optimization*, Lecture Notes in Control and Information Sciences - Proceedings,
https://doi.org/10.1007/978-3-031-83191-1_6

The original version of the book was inadvertently published with incorrect y-label and x-label of Fig. 6.4 and 6.5 in Chapter 6 which have now been corrected. The book and the chapter have been updated with the changes.

The updated version of this chapter can be found at
https://doi.org/10.1007/978-3-031-83191-1_6

Author Index

A

Ahle, Elmar, [87](#)
Allgöwer, Frank, [231](#)

B

Barbara, Nicholas H., [137](#)
Başar, Tamer, [197](#)
Bastianello, Nicola, [313](#)
Baumann, Dominik, [3](#)
Bayram, Erkan, [197](#)
Belabbas, Mohamed-Ali, [197](#)
Belgioioso, Giuseppe, [295](#)
Berberich, Julian, [69](#)
Bianchi, Federico, [247](#)
Borrelli, Francesco, [261](#)
Briem, Daniel, [231](#)

C

Carli, Ruggero, [313](#)
Carnevale, Guido, [313](#)
Cenedese, Carlo, [295](#)
Chen, Hong, [277](#)
Cunis, Torbjørn, [331](#)

D

Diepers, Florian, [87](#)
Dörfler, Florian, [295](#)

E

Eksin, Ibrahim, [169](#)

F

Falsone, Alessandro, [247](#)
Faulwasser, Timm, [105](#)
Feng, Yangyang, [277](#)
Flaßkamp, Kathrin, [215](#)
Fochesato, Marta, [295](#)

G

Gravdahl, Jan Tommy, [181](#)
Grontas, Panagiotis D., [295](#)
Gushkov, Ivan, [181](#)
Güzelkaya, Müjde, [169](#)

H

Harder, Hans, [153](#)
Herrmann-Wicklmayr, Markus, [215](#)
Hoff, Simon, [181](#)

K

Kim, Hyuntae, [37](#)
Köhler, Johannes, [69](#)
Kolmanovsky, Ilya, [331](#)
Kumbasar, Tufan, [169](#)
Kurucu, Mert Can, [169](#)

L

Lazar, Mircea, [121](#)

Liu, Shenyu, [197](#)

Lygeros, John, [295](#)

M

Manchester, Ian R., [137](#)

Marasini, Ganesh, [53](#)

McDermott, Thomas E., [53](#)

McDonald, Kenneth, [53](#)

Molodchyk, Oleksii, [105](#)

Mortensen, Mikael, [153](#)

Müller, Matthias A., [231](#)

N

Nair, Siddharth H., [261](#)

Nestor, Michael, [19](#)

Notarstefano, Giuseppe, [313](#)

O

Orucevic, Amer, [181](#)

P

Peitz, Sebastian, [153](#)

Pettersen, Kristin Y., [181](#)

Prandini, Maria, [247](#)

Q

Qu, Zhihua, [53](#)

R

Rabault, Jean, [153](#)

S

Schön, Thomas B., [3](#)

Schwenkel, Lukas, [231](#)

Söffker, Dirk, [87](#)

Strelnikova, Tatiana, [69](#)

T

Teng, Fei, [19](#)

Tokmak, Abdullah, [3](#)

V

Vallon, Charlott, [261](#)

Vinuesa, Ricardo, [153](#)

W

Wang, Ruigang, [137](#)

Y

Yu, Shuyou, [277](#)

Z

Zhang, Yajing, [277](#)